DRAFT

THE ULTIMATE DEVELOPER GUIDE

C++ MASTERY

Kapsamlı Rehber, Eğitim ve Referans Kitabı

Modern C++, Endüstri Pratikleri, Çift Dilli Anlatım ve Detaylı Örnekler

- Memory & Pointers
- OOP & Design
- Modern C++
- STL & Templates
- Concurrency
- Under the Hood

Bilingual EN/TR · Industry Standards · Core Guidelines · Best Practices

Murat Mecit KAHRAMANLI

Mart 2026

C++ Mastery

Örnekler ile Açıklanmış Kapsamlı C++ Eğitim Kılavuzu

11 Modül · 72 Kaynak Dosya · 20.281 Satır · C++17/C++20

Modül	Konu	Standart
1	Temeller ve Hafıza	C++17
2	Nesne Yönelimli Programlama	C++17
3	Güvenlik ve Performans	C++17
4	Şablonlar ve STL	C++17
5	Sertifikasyon Hazırlığı	C++17
6	Endüstri Standartları	C++17
7	OS Derinlikleri	C++17
8	Üst Düzey Kurallar	C++17
9	İleri Mekanikler	C++20
10	Tasarım Kalıpları	C++20
11	Gömülü Sistemler	C++20

Prepared by **Murat Mecit KAHRAMANLI**

Mart 2026

Telif Hakkı ve Yasal Bilgiler

Telif Hakkı

© 2026 Murat Mecit KAHRAMANLI. Tüm hakları saklıdır.

Lisans

Bu kitaptaki metin içeriği, açıklamalar ve düzen **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** lisansı altında lisanslanmıştır. Kaynak kod örnekleri ise **Apache License 2.0** altında sunulmaktadır.

Bu Ne Anlama Gelir?

Kitap içeriği (CC BY-NC-SA 4.0): Atıf vererek paylaşabilir, uyarlayabilirsiniz; ancak ticari amaçla kullanamazsınız ve türev çalışmalarınızı aynı lisansla paylaşmalısınız.

Kaynak kod örnekleri (Apache 2.0): Kopyalayabilir, değiştirebilir ve kendi projelerinizde (ticari dahil) serbestçe kullanabilirsiniz; ancak orijinal telif hakkı bildirimini korumalı, yapılan değişiklikleri belirtmeli ve lisansın bir kopyasını eklemelisiniz.

Sorumluluk Reddi

Bu kitap eğitim amaçlı hazırlanmıştır. Yazar, içerikteki bilgilerin doğruluğu veya eksiksizliği konusunda garanti vermez. Kod örnekleri öğretim amacıyla basitleştirilmiş olup, üretim ortamında doğrudan kullanılması önerilmez. Yazar, bu kitabın kullanımından doğabilecek herhangi bir zarardan sorumlu tutulamaz.

Atıf ve Kaynaklar

C++ standart referansları için cppreference.com, ISO/IEC 14882 ve C++ Core Guidelines kaynak olarak kullanılmıştır. Tüm ticari markalar ilgili sahiplerine aittir.

İlk Baskı: Mart 2026

Sürüm: 1.0 (Draft)

GitHub: github.com/nimbustan/cpp_mastery

Web: nimbustan.github.io/cpp_mastery

İletişim: nimbustan@github

İçindekiler

■ **Modül 1: Basics — Temeller ve Hafıza** (C++17) 6

→ 01_variables_and_memory.cpp 6
→ 02_pointers_and_references.cpp 8
→ 03_control_flow_and_functions.cpp 11
→ 04_operators_and_expressions.cpp 13
→ 05_control_flow_deepdive.cpp 16
→ 06_strings_and_arrays.cpp 20
→ 07_enums_type_conversions.cpp 23
→ 08_file_io_and_streams.cpp 27
→ 09_scope_namespace_recursion.cpp 30

■ **Modül 2: OOP — Nesne Yönelimli Programlama** (C++17) 34

→ 01_classes_objects.cpp 34
→ 02_inheritance_polymorphism.cpp 37
→ 03_rule_of_three_five.cpp 43
→ 04_multiple_inheritance.cpp 49
→ 05_encapsulation_abstraction.cpp 52
→ 06_static_and_friends.cpp 55
→ 07_nested_and_local_classes.cpp 61
→ 08_rtti_typeid.cpp 67

■ **Modül 3: Modern C++ — Güvenlik ve Performans** (C++17) 73

→ 01_smart_pointers.cpp 73
→ 02_lambdas.cpp 75
→ 03_move_semantics_rvalue.cpp 85
→ 04_modern_types.cpp 90
→ 05_structured_bindings_optional.cpp 93

■ **Modül 4: Templates & STL — Şablonlar ve STL** (C++17) 99

→ 01_templates_stl.cpp 99
→ 02_functors_callbacks.cpp 105
→ 03_stl_containers_deepdive.cpp 107
→ 04_iterators_and_adaptors.cpp 111
→ 05_algorithms_masterclass.cpp 118
→ 06_associative_containers.cpp 126
→ 07_string_operations.cpp 133
→ 08_ranges_and_views.cpp 140

■ **Modül 5: Certification Prep — Sertifikasyon Hazırlığı** (C++17) 149

→ 01_undefined_behavior.cpp 149
→ 02_object_slicing_virtual.cpp 151
→ 03_type_casting.cpp 153
→ 04_exceptions_handling.cpp 157
→ 05_const_correctness.cpp 160
→ 06_interview_gotchas.cpp 166

■ **Modül 6: Industry Standards — Endüstri Standartları** (C++17) 172

→ 01_concurrency_threads.cpp 172
→ 02_cmake_basics.txt 177
→ 03_unit_testing_gtest.cpp 178
→ 04_valgrind_profiling.cpp 184

→ 05_async_futures.cpp 189
→ 06_debugging_techniques.cpp 194
Modül 7: Under the Hood — OS Derinlikleri (C++17) 202
→ 01_vtable_memory_layout.cpp 202
→ 02_sigsegv_and_raii.cpp 204
→ 03_fork_vs_thread_pool.cpp 206
→ 04_memory_model_and_cache.cpp 209
→ 05_compilation_pipeline.cpp 216
→ 06_abi_and_name_mangling.cpp 222
Modül 8: C++ Core Guidelines — Üst Düzey Kurallar (C++17) 228
→ 01_philosophy_and_interfaces.cpp 228
→ 02_resource_management_rule_of_zero.cpp 231
→ 03_error_handling.cpp 234
→ 04_concurrency_and_performance.cpp 237
→ 05_smart_pointer_guidelines.cpp 240
→ 06_class_design_guidelines.cpp 245
Modül 9: Advanced Mechanics — İleri Mekanikler (C++20) 252
→ 01_bit_manipulation_and_alignment.cpp 252
→ 02_operator_overloading.cpp 254
→ 03_advanced_pointers.cpp 259
→ 04_constant_evaluation.cpp 262
→ 05_type_traits_and_concepts.cpp 264
→ 06_sfinae_and_tag_dispatch.cpp 266
→ 07_crtp_pattern.cpp 272
Modül 10: Design Patterns — Tasarım Kalıpları (C++20) 279
→ 01_solid_principles.cpp 279
→ 02_creational_patterns.cpp 285
→ 03_behavioral_patterns.cpp 291
→ 04_structural_patterns.cpp 298
→ 05_testable_design.cpp 304
→ 06_pattern_decision_guide.cpp 310
Modül 11: Embedded & Real-Time — Gömülü Sistemler (C++20) 320
→ 01_volatile_hardware.cpp 320
→ 02_bitfields_protocols.cpp 324
→ 03_embedded_constraints.cpp 328
→ 04_rtos_concurrency.cpp 334
→ 05_advanced_parallelism.cpp 339
Ek: Derleme Bilgisi (Compilation) 345

Giriş: main.cpp

Konu: C++ Mastery — Automotive & Embedded C++ Curriculum / Otomotiv ve Gömülü Sistem Odaklı

11 modules, 71 source files covering C++17/C++20. Bilingual (EN/TR) annotations with cppreference.com depth notes throughout. Designed for engineers targeting the Automotive & Embedded Systems domain. Over 20,000 lines of annotated, production-grade C++ examples.

11 modül, 71 kaynak dosya, C++17/C++20 kapsamında. Otomotiv ve Gömülü Sistem alanını hedefleyen mühendisler için iki dilli (EN/TR) açıklamalar ve cppreference.com derinlik notları ile donatılmıştır. 20.000+ satırlık açıklamalı, üretim kalitesinde C++ örnekleri içerir.

Kaynak Kod / Source Code

```cpp
#include <iostream>

int main() {
  std::cout << "\n";
  std::cout << "  ╔═══════════════════════════════════════════╗\n";
  std::cout << "  ║            C + +   M A S T E R Y          ╒\n";
  std::cout << "  ║   Automotive & Embedded Curriculum (2026)  ╒\n";
  std::cout << "  ╚═══════════════════════════════════════════╝\n";
  std::cout << "\n";
  std::cout << "  Modules: 11 | Files: 71 | C++17 / C++20\n";
  std::cout << "  Domain : Automotive & Embedded Systems\n";
  std::cout << "  Lines  : 20,800+\n\n";

  std::cout << "  [MODULE MAP]\n";
  std::cout << "  ──────────────────────────────────────────\n";
  std::cout << "   1. Basics            — Temeller ve Hafiza\n";
  std::cout << "   2. OOP               — Nesne Yonelimli\n";
  std::cout << "   3. Modern C++        — Guvenlik & Performans\n";
  std::cout << "   4. Advanced STL      — Ileri Kutuphane\n";
  std::cout << "   5. Certification     — Sertifikasyon\n";
  std::cout << "   6. Industry          — Endustri Uretimi\n";
  std::cout << "   7. Under the Hood    — OS Derinlikleri\n";
  std::cout << "   8. Core Guidelines   — Ust Duzey Kurallar\n";
  std::cout << "   9. Advanced Mech.    — Ileri Mekanikler\n";
  std::cout << "  10. Design Patterns   — Tasarim Kaliplari\n";
  std::cout << "  11. Embedded & RTOS   — Gomulu Sistemler\n";
  std::cout << "  ──────────────────────────────────────────\n";
  std::cout << "\n";
  std::cout << "  README.md  -> Full Table of Contents\n";
  std::cout << "  Happy Coding! / Iyi Calismalar!\n\n";
  return 0;
}
```

Modül 1: Basics — Temeller ve Hafıza

Temeller ve Hafıza

9 kaynak dosya · Standart: C++17

C++17

► 01. Variables And Memory

module_01_basics/01_variables_and_memory.cpp

Konu: Variables, Data Types, and Memory Model / Değişkenler, Veri Tipleri ve Bellek Modeli

Teori / Theory

THEORY: Primitive Data Types & Memory

This file demonstrates primitive data types in C++, how much memory they consume using the sizeof() operator, and the concept of constants (const, constexpr).

TEORİ: İlkel Veri Tipleri ve Bellek

Bu dosya C++'daki ilkel veri tiplerini, bellek tüketimlerini (sizeof() operatörü ile) ve sabitlerin (const, constexpr) kavramlarını gösterir.

CPPREF Depth: Undefined Behavior (UB) & Strict Aliasing

cppreference.com explicitly warns: "Reading an uninitialized block of memory is Undefined Behavior (UB)." Also, casting a `float*` to an `int*` violates the "Strict Aliasing Rule" in C++, allowing the optimizer to silently break your firmware.

CPPREF Derinlik: Tanımsız Davranış (UB) ve Sıkı Örtüşme

cppreference uyarıyor: İlk değer (0 veya başka bir sayı) atanmamış bir değişkeni ekrana basmak "Tanımsız Davranış (UB)" yaratır, firmware rastgele çöker. Ayrıca bir `float` pointer'ını zorla `int` pointer'ına çevirmek C++'ın "Strict Aliasing (Sıkı Örtüşme)" kuralını çiğner, derleyici optimizasyon yaparken kodu bozar!

Kaynak Kod / Source Code

```cpp
// Fixed width integer types (Sabit genişlikli tam sayı tipleri için)
#include <cstdint>
#include <iostream>

int main() {
  std::cout << "--- C++ Primitive Data Types & Memory ---" << std::endl;
  std::cout << "--- C++ İlkel Veri Tipleri ve Bellek ---\n" << std::endl;

  // 1. Integer Types (Tam Sayı Tipleri)
  // ==========================================================================================
  // EN: Integer sizes may vary based on OS and compiler. For robust porting, we use the
  // <cstdint> library.
  //
  // TR: C++'da tam sayı boyutları işletim sistemine ve derleyiciye göre değişebilir. Güvenilir
  // ve taşınabilir kod için <cstdint> kütüphanesini kullanırız.

  int age = 30; // Usually 4 bytes (Genellikle 4 bayt)
  std::cout << "int takes: " << sizeof(int) << " bytes (bayt). Example: " << age << std::endl;

  // [CPPREF DEPTH: Uninitialized Variables / CPPREF DERİNLİK: Başlatılmamış Değişkenler]
  // EN: DANGER (UB)! No value assigned. Whatever garbage value is at that RAM address will be
  // used.
  // TR: TEHLİKE (UB)! Değer verilmedi. RAM'de o an o adreste ne çöp (garbage value) varsa onu
  // alır.
  int uninitializedAge;
  // EN: Suppress warning. Reading it is UB!
  // TR: Uyarıyı bastırır. Okumak tanımsız davranıştır (UB)!
  (void)uninitializedAge;
  // EN: Doing this causes Undefined Behavior.
  // TR: Bunu yapmak Tanımsız Davranış'a (UB) neden olur. std::cout << uninitializedAge;

  // Fixed width (Sabit genişlikli)
  int16_t smallNumber = 32767; // Guaranteed 2 bytes (Garantili 2 bayt, 16 bits)
  // Guaranteed 4 bytes (Garantili 4 bayt, 32 bits)
  int32_t standardNumber = 2147483647;

  std::cout << "int16_t takes: " << sizeof(int16_t) << " bytes. Value: " << smallNumber <<
      std::endl;
  std::cout << "int32_t takes: " << sizeof(int32_t) << " bytes. Value: " << standardNumber << "\n"
      << std::endl;

  // 2. Floating Point Types (Ondalıklı Sayı Tipleri)
  // ==========================================================================================
  // 'f' indicates float (F harfi float olduğunu belirtir)
  float piFloat = 3.14159f;
  double piDouble = 3.14159265359; // Higher precision (Daha yüksek hassasiyet)

  std::cout << "float takes: " << sizeof(float) << " bytes. Value: " << piFloat << std::endl;
  std::cout << "double takes: " << sizeof(double) << " bytes. Value: " << piDouble << "\n" <<
      std::endl;

  // 3. Characters and Booleans (Karakterler ve Mantıksal Tipler)
  // ==========================================================================================
  char grade = 'A'; // Single quotes used (Tek tırnak kullanılır)
  // Only true (1) or false (0) (Sadece true veya false alır)
  bool isCppFun = true;

  std::cout << "char takes: " << sizeof(char) << " byte. Value: " << grade << std::endl;
  std::cout << "bool takes: " << sizeof(bool) << " byte. Value: " << isCppFun << "\n" << std::endl;
```

```cpp

    // 4. Constants (Sabitler)
    // ==========================================================================================
    // EN: const (Unchangeable at Run-time) vs constexpr (Calculated at Compile-time)
    // TR: const (Çalışma zamanında değişmez) vs constexpr (Derleme zamanında hesaplanır)

    // Cannot be changed during run-time. (Çalışırken değiştirilemez.)
    const int MAX_USERS = 100;

    // EN: constexpr came with C++11. Evaluated at compile-time for run-time performance.
    // TR: constexpr C++11 ile geldi. Derleme aşamasında hesaplanarak çalışma zamanında performans
    // kazandırır.
    constexpr int SECONDS_IN_MIN = 60;
    constexpr int MINS_IN_HOUR = 60;
    // Compiler hardcodes this as 3600!
    constexpr int SECONDS_IN_HOUR = SECONDS_IN_MIN * MINS_IN_HOUR;
                                      // (Derleyici bunu 3600 olarak yazar!)

    std::cout << "Constants:" << std::endl;
    std::cout << "MAX_USERS: " << MAX_USERS << std::endl;
    std::cout << "SECONDS_IN_HOUR (constexpr): " << SECONDS_IN_HOUR << "\n" << std::endl;

    return 0;
}
```

► 02. Pointers And References

module_01_basics/02_pointers_and_references.cpp

Konu: Basics: Pointers, References, and Memory Addresses / Temeller: İşaretçiler, Referanslar

Teori / Theory

THEORY: Why Do We Need Pointers (*) and References (&)?

When you pass a variable to a function by default in C++, it makes a FULL COPY of that variable (Pass by Value). If you pass a 1GB Image file to a function, the computer freezes while duplicating 1GB in RAM! Pointers (*) and References (&) solve this by passing the "Memory Address". You just give the function the GPS coordinates; it goes there and modifies the original.

TEORİ: Neden İşaretçilere ve Referanslara İhtiyacımız Var?

C++'ta bir değişkene parametre olarak müdahale ettiğinizde varsayılan olarak o değişkenin "Birebir Kopyası" çıkarılır (Değerle Geçiş - Pass by Value). Eğer 1 GB'lık yüksek çözünürlüklü bir resmi bir "Filtre" fonksiyonuna yollarsanız, sistem o 1 GB'ı RAM'de tamamen kopyalar ve bilgisayar donar! Çözüm: İşaretçiler (*) ve Referanslar (&). Bu ikili, nesneyi kopyalamak yerine RAM'deki "GPS Koordinatını (Adresini)" fonksiyona yollar.

THEORY: Pointer (*) vs Reference (&)

1. POINTER (*): Inherited from C. Can be NULL/nullptr. Flexible, can be REASSIGNED to point to another object during runtime. Can be dangerous. 2. REFERENCE (&): Introduced in C++. CAN NEVER be NULL (Must be initialized when declared). Safer and cleaner syntax. Once initialized, it gets LOCKED and cannot point to anything else.

TEORİ: İşaretçi vs Referans

1. POINTER (*): C dilinden kalmadır. NULL/nullptr olabilir. Esnektir, çalışırken başka bir nesneyi gösterecek şekilde DEĞİŞTİRİLEBİLİR. Tehlikeli olabilir. 2. REFERENCE (&): C++ ile gelmiştir. ASLA NULL olamaz (Tanımlandığı an bir hedefi olmalıdır). Daha güvenli ve temiz bir sözdizimi vardır. Oluşturulduktan sonra BAŞKA bir şeyi gösteremez, tamamen kilitlenir.

CPPREF Depth: Dangling References & Pointer Decay

Returning a reference or pointer to a LOCAL variable from a function is a catastrophic Undefined Behavior (Dangling Pointer). Because local variables die on the Stack when the function ends. Also, arrays naturally "decay" into a pointer to their first element.

CPPREF Derinlik: Bağlantısız Referanslar ve İşaretçi Azalması

Bir fonksiyondan "Yerel (Local)" bir değişkenin Referansını veya Pointer'ını `return` ile geri döndürmek felakettir (Dangling Pointer - Boşlukta Sallanan). Çünkü fonksiyon bitince (}) yerel obje Stack'ten (Yığın) silinir. Siz silinmiş bir RAM arazisini göstermiş olursunuz. (SIGSEGV - Hack!)

Kaynak Kod / Source Code

```cpp
#include <iostream>

// 1. PASS BY VALUE (KOPYA YÖNTEMİ) - Bad practice for large data / Büyük veriler için kötü
// pratik
void swapByValue(int a, int b) {
  int temp = a;
  a = b;
  b = temp;
  // EN: The original values in main() WILL NOT CHANGE because 'a' and 'b' are just copies.
  // TR: Main fonksiyonundaki asli değerler DEĞİŞMEYECEKTİR çünkü a ve b sadece 'kopya'lardı.
}

// 2. PASS BY POINTER (İŞARETÇİ YÖNTEMİ) - C style (Tehlikeli ve kalabalık sözdizimi)
void swapByPointer(int *ptrA, int *ptrB) {
  if (ptrA == nullptr || ptrB == nullptr)
    return; // Safety check is mandatory! / Güvenlik kontrolü şarttır!

  // * operator (Dereference): Access the VALUE at the address! / Adresin İÇİNDEKİ değere ulaş!
  int temp = *ptrA;
  *ptrA = *ptrB;
  *ptrB = temp;
}

// 3. PASS BY REFERENCE (REFERANS YÖNTEMİ) - The heart of Modern C++! / Modern C++'ın kalbi!
void swapByReference(int &refA, int &refB) {
  // EN: No null check needed. Cleaner syntax.
  // TR: Null kontrolüne gerek yoktur. Daha temizdir.
  int temp = refA;
  refA = refB;
  refB = temp;
}

// [CPPREF DEPTH: UB (Undefined Behavior) Example / CPPREF DERİNLİK: UB (Tanımsız Davranış) Örneği]
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wreturn-local-addr"
int *generateDanglingPointer_DO_NOT_DO_THIS() {
  int localBomb = 100;
```

```
  // EN: WARNING! Returning address of stack memory!
  // TR: Asla yapma! Stack'teki gecici degiskenin adresi dondurulmez.
  return &localBomb; // NOLINT — intentional UB demo
}
#pragma GCC diagnostic pop

int main() {
  std::cout << "=== MODULE 1: POINTERS & REFERENCES ===\n" << std::endl;

  int num1 = 10;
  int num2 = 99;

  std::cout << "[RAM COORDINATES / ADRESLER]" << std::endl;
  std::cout << "Address of num1 (&): " << &num1 << " | Value (Değer): " << num1 << std::endl;

  // 1. Test: COPY (Çalışmayacak / Won't work!)
  swapByValue(num1, num2);
  std::cout << "\n[TEST] After swapByValue (Copy): num1 = " << num1 << ", num2 = " << num2 <<
      std::endl;

  // 2. Test: POINTER (Çalışacak ama Çirkin / Will work but ugly!) Passing addresses instead of
  // values / "Değerleri değil, adresleri gönderiyorum"
  swapByPointer(&num1, &num2);
  std::cout << "[TEST] After swapByPointer (Pointer): num1 = " << num1 << ", num2 = " << num2 <<
      std::endl;

  // 3. Test: REFERENCE (Geri Çeviriyoruz, Zarif C++ Yolu / Reverting using elegant C++ way)
  // Syntactically passed like values, but actually passed by
  swapByReference( num1, num2);
                   // reference! / Tıpkı değer yollar gibi gönderiyoruz!
  std::cout << "[TEST] After swapByReference (Reference): num1 = " << num1 << ", num2 = " << num2
      << std::endl;

  // [Interview Question / Mülakat Sorusu]: Const Reference
  // EN: "How do I pass a massive object without COPYING it, but also ensuring the function
  // CANNOT MODIFY it?"
  // TR: "Eğer fonksiyona devasa bir nesneyi yollarken KOPYALANMASINI İSTEMİYORSAM ama aynı
  // zamanda değiştirmesini de İSTEMİYORSAM ne yapmalıyım?" Cevap: `const std::string& hugeData`
  // ! No copy (RAM saved) & Read-Only (Değiştirilemez/Salt Okunur).

  return 0;
}
```

► 03. Control Flow And Functions

module_01_basics/03_control_flow_and_functions.cpp

Konu: Basics: Control Flow and Function Overloading / Temeller: Akış Kontrolü ve Fonksiyon

Teori / Theory

THEORY: Switch-case vs If-else

In C++, 'switch' is generally FASTER [O(1) complexity] than many 'if-else' blocks because it creates a "Jump Table" under the hood. However, 'switch' only works with integral types (int, char, enum), strings are strictly incompatible.

TEORİ: Switch-case ve If-else Karşılaştırması

C++'ta 'switch', arka planda bir "Sıçrama Tablosu" (Jump Table) oluşturduğu için çok sayıda (10+) 'if-else' bloğundan DAHA HIZLIDIR (O(1) karmaşıklık sunar). Ancak 'switch' sadece tam sayılarla (int, char, enum) çalışır, stringler uyumsuzdur.

THEORY: Why is Function Overloading Possible?

In C, you CANNOT write two functions with the same name because the compiler only checks the exact name. In C++, the compiler implicitly modifies the function names behind the scenes by appending their parameter types (This is called "Name Mangling"). Example: print(int) -> _Z5printi print(float) -> _Z5printf Thus, you can have thousands of functions with the same name, as long as their parameters are different!

TEORİ: Aşırı Yükleme Neden Mümkündür?

C dilinde aynı isimde iki fonksiyon YAZAMAZSINIZ. Çünkü derleyici sadece "isimlere" bakar. C++'ta ise derleyici fonksiyon isimlerine arka planda parametreleri de ekleyerek gizlice isimlerini değiştirir (Buna "Name Mangling" denir). Örnek: yazdir(int) -> _Z6yazdiri yazdir(float) -> _Z6yazdirf Bu sayede aynı isimli binlerce fonksiyonunuz olabilir, yeter ki parametreleri farklı olsun!

CPPREF Depth: Overload Resolution Rules

How does the compiler decide which overloaded function to call? 1. Exact Match (e.g., passing exactly an `int` to `print(int)`). 2. Promotion (e.g., passing `char` -> promotes to `int` safely). 3. Standard Conversion (e.g., passing `float` -> converts to `double`). If it's ambiguous, the compiler throws a Hard Error.

CPPREF Derinlik: Aşırı Yükleme Çözüm Kuralları

Derleyici hangi fonksiyonu çağıracağına cppreference rank (rütbe) sistemiyle karar verir:

gönderilirse güvenle `int`'e yükseltilir). 3. Standart Çevirim (örn., `float` gönderilirse `double`'a dönüştürülür). İki fonksiyon da eşit uyuyorsa derleyici "Belirsizlik (Ambiguity)" hatası fırlatır ve build kapanır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

// 1. Function Overloading Examples / Aşırı Yükleme Örnekleri
void print(int value) { std::cout << "[INT] Printed: " << value << std::endl; }

void print(double value) {
  std::cout << "[DOUBLE] Printed: " << value << std::endl;
}

void print(std::string value) {
  std::cout << "[STRING] Printed: " << value << std::endl;
}

int main() {
  std::cout << "=== MODULE 1: CONTROL FLOW & OVERLOADING ===\n" << std::endl;

  // A. FUNCTION OVERLOADING / FONKSİYON AŞIRI YÜKLEMESİ
  std::cout << "--- Function Overloading ---" << std::endl;
  print(42);       // int version called / int sürümü çağrılır
  print(3.14159);  // double version called / double sürümü çağrılır
  print("Hello!"); // string version called / string sürümü çağrılır

  // B. MODERN C++11 RANGE-BASED FOR LOOP / MODERN FOR DÖNGÜSÜ
  std::cout << "\n--- Modern For Loop (Range-based) ---" << std::endl;
  int scores[] = {90, 85, 75, 100};

  // EN: Classic: for(int i=0; i<4; i++) { ... } -> Ugly and prone to out-of-bounds errors.
  // Modern C++11: Automatically finds the end of the array or std::vector. 'const auto&' means:
  // I will only read (const) and I will reference the address to save RAM (&).
  //
  // TR: Klasik: for(int i=0; i<4; i++) { ... } -> Çirkin ve index aşımı riski var. Modern C++11:
  // Dizinin veya Vector'ün sonunu kendi bulur. 'const auto&' demek: Sadece okuyacağım (const) ve
  // kopyalayıp RAM'i yormadan adresten bakacağım (&).

  int rank = 1;
  for (const auto &score : scores) {
    std::cout << rank++ << ". Score: " << score << std::endl;
  }

  return 0;
}
```

► 04. Operators And Expressions

module_01_basics/04_operators_and_expressions.cpp

Konu: Basics: Operators, Expressions & Precedence / Temeller: Operatörler, İfadeler ve

Teori / Theory

THEORY: Operator Categories in C++

C++ provides a rich set of operators grouped as: 1. Arithmetic (+, -, *, /, %) 2. Comparison (==, !=, <, >, <=, >=) 3. Logical (&&, ||, !) 4. Bitwise (&, |, ^, ~, <<, >>) 5. Assignment (=, +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=) 6. Ternary (condition ? trueVal : falseVal) 7. Increment (++x pre, x++ post, --x, x--)

TEORİ: C++ Operatör Kategorileri

C++ zengin bir operatör setine sahiptir: 1. Aritmetik (+, -, *, /, %) 2. Karşılaştırma(==, !=, <, >, <=, >=) 3. Mantıksal (&&, ||, !) 4. Bitsel (&, |, ^, ~, <<, >>) 5. Atama (=, +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=) 6. Üçlü (Ternary)(koşul ? doğru : yanlış) 7. Artırma/Azaltma(++x ön, x++ son, --x, x--)

CPPREF Depth: Operator Precedence & Associativity

Operators follow a strict "Precedence Table" (highest -> lowest): 1. `()` (Parentheses) — Highest 2. `* / %` — Left-to-Right (L→R) 3. `+ -` — L→R 4. `<< >>` — L→R 5. `< <= > >=` — L→R 6. `== !=` — L→R 7. `&` (Bitwise AND) — L→R 8. `^` (XOR) — L→R 9. `|` (Bitwise OR) — L→R 10. `&&` — L→R 11. `||` — L→R 12. `?:` (Ternary) — R→L 13. `= += -= ...` — R→L (Assignment is Right-to-Left!)

CPPREF Derinlik: Operatör Önceliği ve İlişkilendirme Yönü

Operatörler katı bir "Öncelik Tablosu"na uyar (yüksek → düşük): 1. `()` (Parantez) — En yüksek 2. `* / %` — Soldan Sağa (L→R) 3. `+ -` — L→R 4. `<< >>` — L→R 5. `< <= > >=` — L→R 6. `== !=` — L→R

L→R 10. `&&` — L→R 11. `||` — L→R 12. `?:` (Üçlü Ternary) — R→L 13. `= += -= ...` — R→L (Atama Sağdan Soladır!)
→ Ref: https://en.cppreference.com/w/cpp/language/operator_precedence

Kaynak Kod / Source Code

```cpp
#include <iostream>

int main() {
  std::cout << "=== MODULE 1: OPERATORS & EXPRESSIONS ===\n" << std::endl;

  // ==========================================================================================
  // 1. ARITHMETIC OPERATORS / ARİTMETİK OPERATÖRLER
  // ==========================================================================================
  std::cout << "--- 1. Arithmetic Operators ---" << std::endl;

  int rpm = 3500;                       // Engine RPM
  int idleRpm = 800;                    // Idle RPM threshold
  int rpmAboveIdle = rpm - idleRpm;     // 2700
  int doubleRpm = rpm * 2;             // 7000
  int avgRpm = (rpm + idleRpm) / 2;    // 2150
  int remainder = rpm % 1000;           // 500 (modulo)

  std::cout << "RPM: " << rpm << std::endl;
  std::cout << "RPM above idle: " << rpmAboveIdle << std::endl;
```

```cpp
    std::cout << "Double RPM: " << doubleRpm << std::endl;
    std::cout << "Average with idle: " << avgRpm << std::endl;
    std::cout << "RPM % 1000 (modulo): " << remainder << "\n" << std::endl;

    // EN: Integer division truncates! 7 / 2 = 3 (not 3.5)
    // TR: Tam sayı bölmesi kesir kısmını atar! 7 / 2 = 3 (3.5 değil)
    std::cout << "Integer division: 7 / 2 = " << (7 / 2) << std::endl;
    std::cout << "Double  division: 7.0 / 2 = " << (7.0 / 2) << "\n" << std::endl;

    // ==========================================================================================
    // 2. COMPARISON OPERATORS / KARŞILAŞTIRMA OPERATÖRLERİ
    // ==========================================================================================
    std::cout << "--- 2. Comparison Operators ---" << std::endl;

    int coolantTemp = 95;          // °C
    int overheatThreshold = 110;   // °C limit

    // EN: These all return bool (true=1 / false=0).
    // TR: Hepsi bool döndürür (true=1 / false=0).
    std::cout << "coolantTemp == 95  : " << (coolantTemp == 95) << std::endl;
    std::cout << "coolantTemp != 100 : " << (coolantTemp != 100) << std::endl;
    std::cout << "coolantTemp <  110 : " << (coolantTemp < overheatThreshold)
              << std::endl;
    std::cout << "coolantTemp >= 90  : " << (coolantTemp >= 90) << "\n"
              << std::endl;

    // ==========================================================================================
    // 3. LOGICAL OPERATORS / MANTIKSAL OPERATÖRLER
    // ==========================================================================================
    std::cout << "--- 3. Logical Operators ---" << std::endl;

    bool engineOn = true;
    bool seatbeltOn = false;

    // EN: && = AND, || = OR, ! = NOT
    // TR: && = VE, || = VEYA, ! = DEĞİL
    std::cout << "Engine ON && Seatbelt ON : " << (engineOn && seatbeltOn)
              << std::endl;
    std::cout << "Engine ON || Seatbelt ON : " << (engineOn || seatbeltOn)
              << std::endl;
    std::cout << "!seatbeltOn (NOT)        : " << (!seatbeltOn) << std::endl;

    // EN: Short-circuit evaluation: if left side of && is false, right side is NEVER evaluated.
    // This saves CPU cycles in embedded systems.
    // TR: Kısa devre değerlendirmesi: && ifadesinin sol tarafı false ise sağ taraf ASLA
    // değerlendirilmez. Gömülü sistemlerde CPU döngüsü kazandırır.
    if (engineOn && coolantTemp > 80) {
      std::cout << "[ECU] Engine running and coolant warm — normal.\n"
                << std::endl;
    }

    // ==========================================================================================
    // 4. INCREMENT / DECREMENT (ARTIRMA / AZALTMA)
    // ==========================================================================================
    std::cout << "--- 4. Increment & Decrement ---" << std::endl;

    int fuelLevel = 50; // % fuel remaining

    // EN: Pre-increment (++x): Increments FIRST, then uses the new value.
    // TR: Ön artırma (++x): ÖNCE artırır, sonra kullanır.
    std::cout << "Pre-increment ++fuelLevel: " << ++fuelLevel << std::endl; // 51

    // EN: Post-increment (x++): Uses current value FIRST, then increments.
    // TR: Son artırma (x++): ÖNCE mevcut değeri kullanır, sonra artırır.
    std::cout << "Post-increment fuelLevel++: " << fuelLevel++ << std::endl; // 51
    std::cout << "After post-increment     : " << fuelLevel << std::endl;   // 52

    // Pre-decrement and Post-decrement
    std::cout << "Pre-decrement --fuelLevel: " << --fuelLevel << std::endl; // 51
    std::cout << "Post-decrement fuelLevel--:" << fuelLevel-- << std::endl; // 51
    std::cout << "After post-decrement     : " << fuelLevel << "\n"
              << std::endl; // 50
```

```cpp

    // ==========================================================================================
    // 5. COMPOUND ASSIGNMENT / BİLEŞİK ATAMA OPERATÖRLERİ
    // ==========================================================================================
    std::cout << "--- 5. Compound Assignment ---" << std::endl;

    int throttle = 30; // Throttle position %
    std::cout << "Initial throttle: " << throttle << " %" << std::endl;

    throttle += 20; // throttle = throttle + 20
    std::cout << "After += 20     : " << throttle << " %" << std::endl;

    throttle -= 10; // throttle = throttle - 10
    std::cout << "After -= 10     : " << throttle << " %" << std::endl;

    throttle *= 2; // throttle = throttle * 2
    std::cout << "After *= 2      : " << throttle << " %" << std::endl;

    throttle /= 4; // throttle = throttle / 4
    std::cout << "After /= 4      : " << throttle << " %" << std::endl;

    throttle %= 7; // throttle = throttle % 7
    std::cout << "After %= 7      : " << throttle << " %\n" << std::endl;

    // ==========================================================================================
    // 6. TERNARY OPERATOR / ÜÇLÜ OPERATÖR
    // ==========================================================================================
    std::cout << "--- 6. Ternary Operator ---" << std::endl;

    int batteryVoltage = 12;
    // EN: condition ? value_if_true : value_if_false
    // TR: koşul ? doğruysa_değer : yanlışsa_değer
    std::string batteryStatus =
        (batteryVoltage >= 12) ? "NORMAL" : "LOW VOLTAGE";
    std::cout << "Battery (" << batteryVoltage
              << "V): " << batteryStatus << std::endl;

    // EN: Nested ternary (use sparingly — hurts readability)
    // TR: İç içe ternary (dikkatli kullanın — okunabilirliği bozar)
    int speedKmh = 135;
    std::string zone = (speedKmh > 130) ? "DANGER"
                       : (speedKmh > 90) ? "HIGHWAY"
                                         : "CITY";
    std::cout << "Speed " << speedKmh << " km/h -> Zone: " << zone << "\n"
              << std::endl;

    // ==========================================================================================
    // 7. BITWISE OPERATORS (Preview) / BİTSEL OPERATÖRLER (Ön İzleme)
    // ==========================================================================================
    std::cout << "--- 7. Bitwise Operators (Preview) ---" << std::endl;

    // EN: Covered in depth in Module 9. Quick demo here:
    // TR: Detaylı anlatım Modül 9'da. Burada kısa önizleme:
    unsigned char statusReg = 0b00001010; // Binary literal (C++14)

    std::cout << "OR  (0b1010 | 0b0101): " << (0b1010 | 0b0101) << std::endl;
    std::cout << "AND (0b1010 & 0b1100): " << (0b1010 & 0b1100) << std::endl;
    std::cout << "XOR (0b1010 ^ 0b1100): " << (0b1010 ^ 0b1100) << std::endl;
    std::cout << "Left Shift (1 << 3)  : " << (1 << 3) << std::endl;
    std::cout << "Right Shift (16 >> 2): " << (16 >> 2) << std::endl;
    std::cout << "NOT (~statusReg)     : " << static_cast<int>(
                  static_cast<unsigned char>(~statusReg))
              << "\n" << std::endl;

    // ==========================================================================================
    // 8. OPERATOR PRECEDENCE DEMO / OPERATÖR ÖNCELİK GÖSTERİMİ
    // ==========================================================================================
    std::cout << "--- 8. Precedence Demo ---" << std::endl;
```

```cpp

    // EN: Without parentheses, * binds tighter than +.
    // TR: Parantez olmadan * operatörü + operatöründen önce çalışır.
    int resultA = 2 + 3 * 4;     // 14 (not 20!)
    int resultB = (2 + 3) * 4;   // 20 (forced with parentheses)

    std::cout << "2 + 3 * 4     = " << resultA << " (mul first!)" << std::endl;
    std::cout << "(2 + 3) * 4   = " << resultB << " (forced add first)"
              << std::endl;

    // EN: Assignment is RIGHT-TO-LEFT (R→L).
    // TR: Atama operatörü SAĞDAN SOLA (R→L) çalışır.
    int a = 0;
    int b = 0;
    int c = 0;
    a = b = c = 42; // c=42, b=c(42), a=b(42)
    std::cout << "a = b = c = 42 -> a=" << a << " b=" << b << " c=" << c
              << std::endl;

    return 0;
}
```

► 05. Control Flow Deepdive

module_01_basics/05_control_flow_deepdive.cpp

Konu: Basics: Control Flow Deep Dive / Temeller: Akış Kontrolü Derinlemesine

Teori / Theory

THEORY: Decision Structures

Every program needs decision-making. C++ provides: - if / else if / else: General-purpose branching. - switch / case: Optimized jump-table for integral types (int, char, enum). - Ternary (?:): Inline conditional (covered in 04_operators).
In automotive ECU firmware, decision structures control everything: engine management, climate, ABS activation, lighting logic.

TEORİ: Karar Yapıları

Her program karar vermeye ihtiyaç duyar. C++ şunları sunar: - if / else if / else: Genel amaçlı dallanma. - switch / case: Tam sayı tipleri için optimize edilmiş atlama tablosu. - Ternary (?:): Satır içi koşul (04_operators dosyasında anlatıldı).

THEORY: Loop Structures

Loops repeat a block of code: - for: When the iteration count is KNOWN. - while: When it depends on a CONDITION (unknown iterations). - do-while: Same as while, but executes AT LEAST ONCE. - Range-based for (C++11): Safe iteration over containers. - break: Exit the loop immediately. - continue: Skip this iteration, go to next.

TEORİ: Döngü Yapıları

Döngüler bir kod bloğunu tekrarlar: - for: İterasyon sayısı BİLİNDİĞİNDE. - while: Bir KOŞULA bağlı olduğunda (belirsiz iterasyon sayısı). - do-while: while ile aynı ama EN AZ BİR KEZ çalışır. - Range-based for (C++11): Konteynerler üzerinde güvenli iterasyon. - break: Döngüden anında çık. - continue: Bu iterasyonu atla, sonrakine geç.

CPPREF Depth: Structured Bindings & Init-Statements (C++17)

C++17 allows `if (auto x = compute(); x > threshold)` — the variable is scoped ONLY inside the if-block. This reduces variable leakage. Also, `switch` supports init-statements: `switch (auto code = getDTC(); code)`.

CPPREF Derinlik: Yapısal Bağlamalar ve Başlatma İfadeleri (C++17)

C++17 ile `if (auto x = hesapla(); x > eşik)` yazabilirsiniz. Değişken SADECE if bloğunun içinde geçerlidir. Bu değişken sızıntısını azaltır. `switch` de başlatma ifadesini destekler.
→ Ref: https://en.cppreference.com/w/cpp/language/if
→ Ref: https://en.cppreference.com/w/cpp/language/switch

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

int main() {
  std::cout << "=== MODULE 1: CONTROL FLOW DEEP DIVE ===\n" << std::endl;

  // ==========================================================================================
  // 1. IF / ELSE IF / ELSE
  // ==========================================================================================
  std::cout << "--- 1. if / else if / else ---" << std::endl;

  int coolantTempC = 105;

  // EN: Multi-branch decision for coolant temperature monitoring.
  // TR: Soğutma suyu sıcaklığı izleme için çok dallı karar yapısı.
  if (coolantTempC > 120) {
    std::cout << "[ALARM] CRITICAL OVERHEAT: " << coolantTempC << " °C!"
              << std::endl;
  } else if (coolantTempC > 100) {
    std::cout << "[WARNING] Coolant high: " << coolantTempC << " °C"
              << std::endl;
  } else if (coolantTempC > 80) {
    std::cout << "[NORMAL] Coolant normal: " << coolantTempC << " °C"
              << std::endl;
  } else {
    std::cout << "[INFO] Engine cold: " << coolantTempC << " °C" << std::endl;
  }

  // C++17 if with init-statement / C++17 başlatma ifadeli if
  // EN: Variable 'voltage' is scoped ONLY inside this if-block.
  // TR: 'voltage' değişkeni SADECE bu if bloğunun içinde geçerlidir.
  if (double voltage = 13.8; voltage > 12.0) {
    std::cout << "[ECU] Battery voltage OK: " << voltage << " V\n"
              << std::endl;
  }

  // ==========================================================================================
  // 2. SWITCH / CASE
  // ==========================================================================================
  std::cout << "--- 2. switch / case ---" << std::endl;

  // EN: DTC (Diagnostic Trouble Code) severity levels: 1=Info, 2=Warning, 3=Critical. switch
  // creates an O(1) jump table.
  // TR: DTC (Teşhis Arıza Kodu) önem seviyesi. switch O(1) atlama tablosu oluşturur.
  int dtcSeverity = 2;

  switch (dtcSeverity) {
  case 1:
    std::cout << "[DTC] Severity 1: Informational" << std::endl;
```

```
50        break; // EN: Without break, execution "falls through" to next case!
51               // TR: break olmadan bir sonraki case'e düşer!
52      case 2:
53        std::cout << "[DTC] Severity 2: Warning — schedule maintenance"
54                  << std::endl;
55        break;
56      case 3:
57        std::cout << "[DTC] Severity 3: CRITICAL — stop vehicle!" << std::endl;
58        break;
59      default:
60        std::cout << "[DTC] Unknown severity level" << std::endl;
61        break;
62      }
63
64      // EN: Intentional fall-through demo (grouping cases).
65      // TR: Kasıtlı fall-through gösterimi (case'leri gruplama).
66      char gearPosition = 'D';
67      std::cout << "Gear: " << gearPosition << " -> ";
68      switch (gearPosition) {
69      case 'D':
70      case 'd':
71        std::cout << "Drive mode" << std::endl;
72        break;
73      case 'R':
74      case 'r':
75        std::cout << "Reverse mode" << std::endl;
76        break;
77      case 'N':
78      case 'n':
79        std::cout << "Neutral" << std::endl;
80        break;
81      case 'P':
82      case 'p':
83        std::cout << "Park" << std::endl;
84        break;
85      default:
86        std::cout << "Invalid gear!" << std::endl;
87        break;
88      }
89      std::cout << std::endl;
90
91      // ==========================================================================================
92      // 3. CLASSIC FOR LOOP / KLASİK FOR DÖNGÜSÜ
93      // ==========================================================================================
94      std::cout << "--- 3. Classic for loop ---" << std::endl;
95
96      // EN: Simulate scanning 6 CAN bus message slots.
97      // TR: 6 CAN bus mesaj slotunu taramak.
98      for (int slot = 0; slot < 6; ++slot) {
99        std::cout << "Scanning CAN slot #" << slot << std::endl;
100     }
101     std::cout << std::endl;
102
103     // ==========================================================================================
104     // 4. WHILE LOOP / WHILE DÖNGÜSÜ
105     // ==========================================================================================
106     std::cout << "--- 4. while loop ---" << std::endl;
107
108     // EN: Simulate coolant warm-up until threshold.
109     // TR: Soğutma suyu ısınmasını eşiğe kadar simüle et.
110     int warmUpTemp = 20;
111     while (warmUpTemp < 80) {
112       warmUpTemp += 15; // Simulating heating
113       std::cout << "Warming up... " << warmUpTemp << " °C" << std::endl;
114     }
115     std::cout << "[ECU] Operating temperature reached!\n" << std::endl;
116
117     // ==========================================================================================
118     // 5. DO-WHILE LOOP / DO-WHILE DÖNGÜSÜ
119     // ==========================================================================================
120     std::cout << "--- 5. do-while loop ---" << std::endl;
```

```cpp

    // EN: Executes AT LEAST ONCE, even if condition is already false. Use case: Sensor self-test
    // that must run at least once on boot.
    // TR: Koşul baştan yanlış olsa bile EN AZ BİR KEZ çalışır. Kullanım: Açılışta en az bir kez
    // çalışması gereken sensör öz testi.
    int sensorRetry = 0;
    do {
      std::cout << "Sensor self-test attempt #" << (sensorRetry + 1)
                << std::endl;
      sensorRetry++;
    } while (sensorRetry < 3);
    std::cout << "[ECU] Self-test complete.\n" << std::endl;

    // ==========================================================================================
    // 6. BREAK & CONTINUE
    // ==========================================================================================
    std::cout << "--- 6. break & continue ---" << std::endl;

    // EN: Scan 10 ECU addresses. Skip address 5 (reserved), stop at 8 (limit).
    // TR: 10 ECU adresini tara. 5'i atla (ayrılmış), 8'de dur (sınır).
    for (int addr = 1; addr <= 10; ++addr) {
      if (addr == 5) {
        std::cout << "  Address " << addr << " -> RESERVED (skip)" << std::endl;
        continue; // Skip this iteration / Bu iterasyonu atla
      }
      if (addr == 8) {
        std::cout << "  Address " << addr << " -> LIMIT reached (break)"
                  << std::endl;
        break; // Exit the for loop entirely / Döngüden tamamen çık
      }
      std::cout << "  Address " << addr << " -> OK" << std::endl;
    }
    std::cout << std::endl;

    // ==========================================================================================
    // 7. NESTED LOOPS / İÇ İÇE DÖNGÜLER
    // ==========================================================================================
    std::cout << "--- 7. Nested Loops ---" << std::endl;

    // EN: 4 cylinders × 3 sensors each = 12 readings.
    // TR: 4 silindir × 3 sensör = 12 okuma.
    for (int cyl = 1; cyl <= 4; ++cyl) {
      for (int sensor = 1; sensor <= 3; ++sensor) {
        std::cout << "  Cyl" << cyl << "-Sensor" << sensor << " ";
      }
      std::cout << std::endl;
    }
    std::cout << std::endl;

    // ==========================================================================================
    // 8. RANGE-BASED FOR (C++11) — revisited
    // ==========================================================================================
    std::cout << "--- 8. Range-based for (C++11) ---" << std::endl;

    // EN: Iterating over DTC codes stored in an array.
    // TR: Dizide saklanan DTC kodlarını dolaşma.
    std::string dtcCodes[] = {"P0300", "P0171", "P0420", "P0442"};

    for (const auto &dtc : dtcCodes) {
      std::cout << "  Active DTC: " << dtc << std::endl;
    }
    std::cout << std::endl;

    // ==========================================================================================
    // 9. INFINITE LOOP PATTERN (common in embedded) / SONSUZ DÖNGÜ KALIP
    // ==========================================================================================
    std::cout << "--- 9. Infinite Loop Pattern ---" << std::endl;
```

```
  // EN: In real embedded firmware, `while(true)` or `for(;;)` is the main super-loop. Here we
  // simulate with a counter.
  // TR: Gerçek gömülü firmware'de `while(true)` veya `for(;;)` ana süper-döngüdür. Burada bir
  // sayaçla simüle ediyoruz.
  int tick = 0;
  while (true) {
    std::cout << "  ECU tick #" << tick << std::endl;
    ++tick;
    if (tick >= 5) {
      std::cout << "  [DEMO] Exiting super-loop after 5 ticks.\n" << std::endl;
      break;
    }
  }

  return 0;
}
```

► 06. Strings And Arrays

module_01_basics/06_strings_and_arrays.cpp

Konu: Basics: Strings and Arrays / Temeller: Dizeler (Stringler) ve Diziler

Teori / Theory

THEORY: C-Strings vs std::string

C++ inherited NULL-terminated character arrays from C (`char str[]`). These are dangerous: no bounds checking, manual memory management, easy buffer overflows. Modern C++ uses `std::string` from `<string>`: - Automatic memory management (grows/shrinks as needed). - Safe concatenation with `+` operator. - Rich API: `.length()`, `.substr()`, `.find()`, `.compare()`, etc.

TEORİ: C-Stringleri vs std::string

C++ dilinden NULL sonlandırmalı karakter dizilerini miras aldı (`char str[]`). Bunlar tehlikelidir: sınır kontrolü yok, manuel bellek yönetimi, kolay buffer overflow. Modern C++ `<string>` kütüphanesindeki `std::string` kullanır: - Otomatik bellek yönetimi (ihtiyaca göre büyür/küçülür). - `+` operatörü ile güvenli birleştirme. - Zengin API: `.length()`, `.substr()`, `.find()`, `.compare()`, vb.

THEORY: C-Style Arrays vs std::array

C-style arrays (`int arr[5]`) decay to pointers when passed to functions, losing size information. `std::array<int, 5>` (C++11) wraps a fixed-size array with bounds checking (`.at()`) and knows its own size (`.size()`). For dynamic sizes, use `std::vector`.

TEORİ: C Dizileri vs std::array

C-tarzı diziler (`int arr[5]`) fonksiyonlara geçildiğinde pointer'a dönüşür ve boyut bilgisini kaybeder. `std::array<int, 5>` (C++11) sabit boyutlu diziyi sınır kontrolü (`.at()`) ve kendi boyutunu bilme (`.size()`) ile sarar. Dinamik boyut için `std::vector` kullanın.

CPPREF Depth: Small String Optimization (SSO)

Most `std::string` implementations apply SSO: strings shorter than ~15-22 characters are stored INSIDE the object (on the stack), avoiding heap allocation entirely. Longer strings trigger a `new` allocation. This is crucial for embedded systems where heap usage is expensive.

CPPREF Derinlik: Küçük String Optimizasyonu (SSO)

Çoğu `std::string` implementasyonu SSO uygular: ~15-22 karakterden kısa string'ler objenin İÇİNDE (stack'te) saklanır, heap tahsisinden tamamen kaçınılır. Uzun string'ler `new` ile heap'te yer açar. Heap kullanımının pahalı olduğu gömülü sistemlerde bu kritiktir.
→ Ref: https://en.cppreference.com/w/cpp/string/basic_string
→ Ref: https://en.cppreference.com/w/cpp/container/array

Kaynak Kod / Source Code

```cpp
#include <array>
#include <iostream>
#include <string>

int main() {
  std::cout << "=== MODULE 1: STRINGS & ARRAYS ===\n" << std::endl;

  // ==============================================================================================
  // 1. C-STYLE STRING (Legacy — avoid in new code)
  // ==============================================================================================
  std::cout << "--- 1. C-Style String (Legacy) ---" << std::endl;

  // EN: NULL-terminated character array. Last char is '\0' automatically.
  // TR: NULL sonlandırmalı karakter dizisi. Son karakter otomatik '\0' eklenir.
  char vinCstr[] = "WBAPH5C55BA271234";
  std::cout << "VIN (C-String): " << vinCstr << std::endl;
  std::cout << "First char    : " << vinCstr[0] << std::endl;
  std::cout << "Array size    : " << sizeof(vinCstr)
            << " (17 chars + 1 null)\n" << std::endl;

  // ==============================================================================================
  // 2. std::string (Modern C++)
  // ==============================================================================================
  std::cout << "--- 2. std::string (Modern C++) ---" << std::endl;

  std::string ecuName = "BCM_BodyControl";
  std::string version = "v2.4.1";

  // Concatenation / Birleştirme
  std::string fullId = ecuName + " [" + version + "]";
  std::cout << "Full ID       : " << fullId << std::endl;
  std::cout << "Length        : " << fullId.length() << std::endl;
  std::cout << "Char at [4]   : " << fullId[4] << std::endl;
  std::cout << "Char at .at(4): " << fullId.at(4) << " (bounds-checked)"
            << std::endl;

  // Substring / Alt dize
  std::string prefix = fullId.substr(0, 3); // "BCM"
  std::cout << "Prefix (0,3)  : " << prefix << std::endl;

  // Find / Arama
  size_t pos = fullId.find("Body");
  if (pos != std::string::npos) {
    std::cout << "Found 'Body' at index: " << pos << std::endl;
  }

  // Compare / Karşılaştırma
  std::string other = "BCM_BodyControl [v2.4.1]";
  std::cout << "Equal to other? : " << (fullId == other ? "Yes" : "No")
            << std::endl;

  // Append / Ekleme
  fullId.append(" — Active");
  std::cout << "After append  : " << fullId << std::endl;

  // Empty check / Boşluk kontrolü
  std::string emptyStr;
  std::cout << "emptyStr.empty(): " << emptyStr.empty() << "\n" << std::endl;
```

```cpp

    // ==========================================================================================
    // 3. C-STYLE ARRAY
    // ==========================================================================================
    std::cout << "--- 3. C-Style Array ---" << std::endl;

    // EN: Fixed-size array of sensor voltage readings (mV).
    // TR: Sabit boyutlu sensör voltaj okumaları dizisi (mV).
    int sensorMv[5] = {3300, 2800, 3100, 2950, 3050};

    // Classic for iteration
    for (int i = 0; i < 5; ++i) {
      std::cout << "  Sensor[" << i << "] = " << sensorMv[i] << " mV"
                << std::endl;
    }
    std::cout << "Array sizeof: " << sizeof(sensorMv)
              << " bytes (" << sizeof(sensorMv) / sizeof(sensorMv[0])
              << " elements)\n" << std::endl;

    // ==========================================================================================
    // 4. std::array<T, N> (C++11) — Modern Fixed-Size Array
    // ==========================================================================================
    std::cout << "--- 4. std::array (Modern C++11) ---" << std::endl;

    // EN: Knows its own size, supports bounds-checked access with .at().
    // TR: Kendi boyutunu bilir, .at() ile sınır kontrolü sağlar.
    std::array<double, 4> egtTemps = {650.5, 720.3, 695.0, 710.8};

    std::cout << "EGT array size: " << egtTemps.size() << std::endl;
    for (size_t i = 0; i < egtTemps.size(); ++i) {
      std::cout << "  EGT_Cyl" << (i + 1) << ": " << egtTemps[i] << " °C"
                << std::endl;
    }

    // Safe access with .at() — throws std::out_of_range if bad index.
    // TR: .at() ile güvenli erişim — kötü indeks verilirse exception fırlatır.
    std::cout << "egtTemps.at(2): " << egtTemps.at(2) << " °C" << std::endl;
    std::cout << "Front: " << egtTemps.front()
              << " | Back: " << egtTemps.back() << "\n" << std::endl;

    // ==========================================================================================
    // 5. MULTIDIMENSIONAL ARRAY / ÇOK BOYUTLU DİZİ
    // ==========================================================================================
    std::cout << "--- 5. Multidimensional Array ---" << std::endl;

    // EN: 3 ECUs × 4 sensor channels each (voltage mV readings).
    // TR: 3 ECU × 4 sensör kanalı (mV okuma değerleri).
    int sensorGrid[3][4] = {
        {3300, 2900, 3100, 3050}, // ECU 0: BCM
        {3200, 3000, 2800, 3150}, // ECU 1: Powertrain
        {3100, 3250, 2950, 3000}  // ECU 2: HVAC
    };

    const char *ecuNames[] = {"BCM", "Powertrain", "HVAC"};
    for (int ecu = 0; ecu < 3; ++ecu) {
      std::cout << "  " << ecuNames[ecu] << ": ";
      for (int ch = 0; ch < 4; ++ch) {
        std::cout << sensorGrid[ecu][ch] << " mV";
        if (ch < 3) std::cout << ", ";
      }
      std::cout << std::endl;
    }
    std::cout << std::endl;

    // ==========================================================================================
    // 6. STRING ITERATION / STRING ÜZERİNDE İTERASYON
    // ==========================================================================================
    std::cout << "--- 6. String Iteration ---" << std::endl;

    std::string vin = "WBAPH5C55BA271234";
    std::cout << "VIN chars: ";
```

```
  for (const char &ch : vin) {
    std::cout << ch << ' ';
  }
  std::cout << "\n" << std::endl;

  // EN: Count digits in VIN.
  // TR: VIN içindeki rakamları say.
  int digitCount = 0;
  for (const char &ch : vin) {
    if (ch >= '0' && ch <= '9') {
      ++digitCount;
    }
  }
  std::cout << "Digits in VIN: " << digitCount << std::endl;

  return 0;
}
```

► 07. Enums Type Conversions

module_01_basics/07_enums_type_conversions.cpp

Konu: Basics: Enumerations, auto, typedef/using & Type Conversions / Temeller:

Teori / Theory

THEORY: enum vs enum class

Classic C-style `enum` leaks its members into the enclosing scope and implicitly converts to `int`. This causes name collisions and bugs. Modern `enum class` (C++11) provides: - Scoped names: `Color::Red` (not just `Red`). - No implicit conversion to int — you must use `static_cast`. - Underlying type can be specified: `enum class Gear : uint8_t`.

TEORİ: enum ve enum class Karşılaştırması

Klasik C-tarzı `enum` üyelerini kapsam dışına sızdırır ve `int`'e dolaylı olarak dönüşür. Bu isim çakışmasına ve hatalara neden olur. Modern `enum class` (C++11): - Kapsamlı isimler: `Color::Red` (sadece `Red` değil). - int'e dolaylı dönüşüm YOK — `static_cast` kullanmalısınız. - Alt tür belirlenebilir: `enum class Gear : uint8_t`.

THEORY: Type Conversions

C++ performs two kinds of conversions: 1. Implicit (automatic): The compiler silently widens/narrows types (e.g., int → double). Narrowing conversions can lose data! 2. Explicit (casts): - `static_cast<T>()`: Compile-time type conversion (SAFEST). - `(T)x` or `T(x)`: C-style cast (DANGEROUS, no checks). - `reinterpret_cast`, `const_cast`, `dynamic_cast` for advanced cases.

TEORİ: Tür Dönüşümleri

C++ iki tür dönüşüm yapar: 1. Dolaylı (otomatik): Derleyici sessizce tipleri genişletir/daraltır. Daraltma dönüşümleri veri kaybına yol açabilir! 2. Açık (cast): - `static_cast<T>()`: Derleme zamanı dönüşümü (EN GÜVENLİ). - `(T)x` veya `T(x)`: C-tarzı cast (TEHLİKELİ, kontrol yok). - `reinterpret_cast`, `const_cast`, `dynamic_cast` ileri seviye durumlar için.

CPPREF Depth: auto Type Deduction

`auto` (C++11) tells the compiler to deduce the type from the initializer. It reduces boilerplate but should be used carefully: - Good: `auto it = vec.begin();` (avoids the long iterator type). - Bad: `auto x = 42;` (is it int? unsigned? long? — clarity lost). - `decltype(expr)`: Deduces the type of an expression without evaluating it.

CPPREF Derinlik: auto Tip Çıkarımı

`auto` (C++11) derleyiciye başlatıcıdan tipi çıkarmasını söyler. Standart kodu azaltır ama dikkatli kullanılmalıdır: - İyi: `auto it = vec.begin();` (uzun iterator tipinden kaçınır). - Kötü: `auto x = 42;` (int mi? unsigned mı? long mu? — netlik kaybolur). - `decltype(expr)`: Bir ifadenin tipini, onu çalıştırmadan çıkarır.
→ Ref: https://en.cppreference.com/w/cpp/language/enum
→ Ref: https://en.cppreference.com/w/cpp/language/static_cast
→ Ref: https://en.cppreference.com/w/cpp/language/auto

Kaynak Kod / Source Code

```cpp
#include <cstdint> // uint8_t
#include <iostream>
#include <string>

// ==========================================================================================
// 1. CLASSIC enum (C-Style — avoid in new code)
// ==========================================================================================
// EN: Members leak into surrounding scope. RED, GREEN, BLUE are globally visible
// TR: Üyeler dış kapsama sızar. RED, GREEN, BLUE global olarak görünür
enum LegacyIndicatorColor { RED, GREEN, BLUE };

// ==========================================================================================
// 2. MODERN enum class (C++11 — strongly typed)
// ==========================================================================================
// EN: Scoped and type-safe. Must use GearPosition::Park syntax.
// TR: Kapsamlı ve tip güvenli. GearPosition::Park sözdizimi zorunlu.
enum class GearPosition : uint8_t {
  Park = 0,
  Reverse = 1,
  Neutral = 2,
  Drive = 3,
  Sport = 4
};

// EN: ECU operating states for a Body Control Module.
// TR: Gövde Kontrol Modülü (BCM) için ECU çalışma durumları.
enum class ECUState { Off, Booting, Running, Sleep, Error };

// ==========================================================================================
// 3. typedef & using (Type Aliases)
// ==========================================================================================
// EN: Old C-style / TR: Eski C tarzı
typedef unsigned long SensorId;

// EN: Modern C++11 alias (preferred) / TR: Modern C++11 takma adı (tercih edilen)
using MilliVolts = int;
using TemperatureC = double;

int main() {
  std::cout << "=== MODULE 1: ENUMS, AUTO & TYPE CONVERSIONS ===\n"
            << std::endl;

  // ==========================================================================================
  // 1. CLASSIC ENUM DEMO
  // ==========================================================================================
  std::cout << "--- 1. Classic enum (Legacy) ---" << std::endl;

  LegacyIndicatorColor indicator = GREEN;
```

```cpp
    // EN: Implicitly converts to int! No type safety.
    // TR: int'e dolaylı olarak dönüşür! Tip güvenliği yok.
    int colorCode = indicator;
    std::cout << "Indicator color code (int): " << colorCode << "\n"
              << std::endl;

    // ==========================================================================================
    // 2. MODERN ENUM CLASS DEMO
    // ==========================================================================================
    std::cout << "--- 2. enum class (Modern C++11) ---" << std::endl;

    GearPosition gear = GearPosition::Drive;
    // int gearNum = gear;  // ERROR! No implicit conversion allowed!
    int gearNum = static_cast<int>(gear); // Must be explicit
    std::cout << "Gear position: " << gearNum << " (Drive)" << std::endl;

    ECUState bcmState = ECUState::Running;
    // EN: Switch on enum class values — type safe.
    // TR: enum class değerleri üzerinde switch — tip güvenli.
    switch (bcmState) {
    case ECUState::Off:
      std::cout << "BCM: OFF" << std::endl;
      break;
    case ECUState::Booting:
      std::cout << "BCM: Booting..." << std::endl;
      break;
    case ECUState::Running:
      std::cout << "BCM: Running normally" << std::endl;
      break;
    case ECUState::Sleep:
      std::cout << "BCM: Sleep mode" << std::endl;
      break;
    case ECUState::Error:
      std::cout << "BCM: ERROR!" << std::endl;
      break;
    }
    std::cout << std::endl;

    // ==========================================================================================
    // 3. typedef / using DEMO
    // ==========================================================================================
    std::cout << "--- 3. typedef & using aliases ---" << std::endl;

    SensorId egtId = 1001UL;
    MilliVolts reading = 3300;
    TemperatureC ambientTemp = 22.5;

    std::cout << "Sensor ID  : " << egtId << std::endl;
    std::cout << "Reading    : " << reading << " mV" << std::endl;
    std::cout << "Ambient    : " << ambientTemp << " °C\n" << std::endl;

    // ==========================================================================================
    // 4. auto TYPE DEDUCTION (C++11)
    // ==========================================================================================
    std::cout << "--- 4. auto keyword (C++11) ---" << std::endl;

    auto voltage = 13.8;        // Deduced as double
    auto ecuCount = 5;           // Deduced as int
    auto vinStr = std::string("WBAPH5C55BA271234"); // Deduced as std::string

    std::cout << "voltage (auto = double): " << voltage << std::endl;
    std::cout << "ecuCount (auto = int)  : " << ecuCount << std::endl;
    std::cout << "vinStr (auto = string) : " << vinStr << "\n" << std::endl;

    // ==========================================================================================
    // 5. IMPLICIT CONVERSIONS / DOLAYLI DÖNÜŞÜMLER
    // ==========================================================================================
    std::cout << "--- 5. Implicit Conversions ---" << std::endl;
```

```cpp

    // EN: Widening (safe): int → double. No data loss.
    // TR: Genişletme (güvenli): int → double. Veri kaybı yok.
    int rpm = 3500;
    double rpmDouble = rpm; // Implicit widening
    std::cout << "int 3500 -> double: " << rpmDouble << std::endl;

    // EN: Narrowing (DANGEROUS!): double → int. Fractional part lost!
    // TR: Daraltma (TEHLİKELİ!): double → int. Kesir kısmı kaybolur!
    double precisePressure = 2.75;
    int roughPressure = static_cast<int>(precisePressure); // 2 (not 2.75!)
    std::cout << "double 2.75 -> int: " << roughPressure
              << " (fractional part LOST!)" << std::endl;

    // EN: bool -> int conversion (true=1, false=0).
    // TR: bool -> int dönüşümü (true=1, false=0).
    bool absActive = true;
    int absFlag = absActive;
    std::cout << "bool true -> int: " << absFlag << "\n" << std::endl;

    // ==========================================================================================
    // 6. EXPLICIT CONVERSIONS (static_cast) / AÇIK DÖNÜŞÜMLER
    // ==========================================================================================
    std::cout << "--- 6. static_cast (Explicit) ---" << std::endl;

    // EN: Use static_cast for clear, compile-time checked conversions.
    // TR: Açık, derleme zamanı kontrollü dönüşümler için static_cast kullanın.
    int totalDistance = 15000; // meters
    int numSegments = 7;

    // EN: Without cast: integer division → truncated result (2142).
    // TR: Cast olmadan: tam sayı bölmesi → kırpılmış sonuç.
    std::cout << "Integer div: " << totalDistance / numSegments << std::endl;

    // EN: With cast: one operand becomes double → full precision.
    // TR: Cast ile: bir operand double olur → tam hassasiyet.
    double avgSegment =
        static_cast<double>(totalDistance) / static_cast<double>(numSegments);
    std::cout << "static_cast div: " << avgSegment << " m/segment" << std::endl;

    // EN: Casting enum class to its underlying type.
    // TR: enum class'ı temel türüne dönüştürme.
    auto gearValue = static_cast<uint8_t>(GearPosition::Sport);
    std::cout << "GearPosition::Sport -> uint8_t: "
              << static_cast<int>(gearValue) << "\n" << std::endl;

    // ==========================================================================================
    // 7. C-STYLE CAST (Avoid!) / C-TARZI CAST (Kaçının!)
    // ==========================================================================================
    std::cout << "--- 7. C-Style Cast (Dangerous Legacy) ---" << std::endl;

    // EN: C-style casts bypass ALL safety checks. Prefer static_cast.
    // TR: C-tarzı cast'ler TÜM güvenlik kontrollerini atlar. static_cast tercih edin.
    double sensorVoltage = 4.85;
    int rawADC = (int)sensorVoltage; // C-style: works but unsafe
    std::cout << "C-style (int)4.85 = " << rawADC
              << " (Compiles, but avoid in modern C++!)" << std::endl;

    return 0;
}
```

► 08. File Io And Streams

module_01_basics/08_file_io_and_streams.cpp

Konu: Basics: File I/O and Stream Operations / Temeller: Dosya G/Ç ve Akış İşlemleri

Teori / Theory

THEORY: C++ I/O Streams

C++ I/O is built on a stream hierarchy: - `std::cin` → Standard Input (keyboard). - `std::cout` → Standard Output (console). - `std::cerr` → Standard Error (unbuffered, for errors). - `std::clog` → Standard Log (buffered). For File I/O, `<fstream>` provides: - `std::ofstream` → Write to file (Output File Stream). - `std::ifstream` → Read from file (Input File Stream). - `std::fstream` → Read AND Write.

TEORİ: C++ G/Ç Akışları

C++ G/Ç bir akış hiyerarşisi üzerine kurulmuştur: - `std::cin` → Standart Girdi (klavye). - `std::cout` → Standart Çıktı (konsol). - `std::cerr` → Standart Hata (tamponlanmamış, hatalar için). - `std::clog` → Standart Günlük (tamponlanmış).
Dosya G/Ç için `<fstream>` şunları sağlar: - `std::ofstream` → Dosyaya yaz (Çıktı Dosya Akışı). - `std::ifstream` → Dosyadan oku (Girdi Dosya Akışı). - `std::fstream` → Hem Oku hem de Yaz.

CPPREF Depth: RAII and File Handles

`std::ofstream` and `std::ifstream` use RAII: the file is automatically closed when the object goes out of scope (destructor runs). You do NOT need to call `.close()` explicitly if the object is about to be destroyed. Calling `.close()` is only needed if you want to reopen or reuse the stream.
The `is_open()` check is critical — opening a file can FAIL silently (bad path, permissions), and writing to a failed stream is Undefined Behavior.

CPPREF Derinlik: RAII ve Dosya Tanıtıcıları

`std::ofstream` ve `std::ifstream` RAII kullanır: nesne kapsam dışına çıktığında dosya otomatik kapatılır (yıkıcı çalışır). Nesne yok edilecekse `.close()` çağırmanıza GEREK YOKTUR. `.is_open()` kontrolü kritiktir — dosya açma sessizce başarısız olabilir.
→ Ref: https://en.cppreference.com/w/cpp/io/basic_ofstream
→ Ref: https://en.cppreference.com/w/cpp/io/basic_ifstream

Kaynak Kod / Source Code

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
  std::cout << "=== MODULE 1: FILE I/O & STREAMS ===\n" << std::endl;

  // ==========================================================================================
  // 1. CONSOLE OUTPUT FORMATTING / KONSOL ÇIKTI BİÇİMLENDİRME
  // ==========================================================================================
  std::cout << "--- 1. Console Output ---" << std::endl;
```

```cpp

    // EN: std::endl flushes the buffer (slow). '\n' does not (fast). In embedded: prefer '\n' for
    // performance.
    // TR: std::endl tamponu temizler (yavaş). '\n' temizlemez (hızlı).
    std::cout << "Line 1 with endl" << std::endl; // Flushes
    std::cout << "Line 2 with \\n\n";              // No flush (faster)

    // EN: std::cerr is unbuffered — used for critical error messages.
    // TR: std::cerr tamponlanmamıştır — kritik hata mesajları için kullanılır.
    std::cerr << "[ERROR] Simulated ECU fault (stderr)\n" << std::endl;

    // ==========================================================================================
    // 2. CONSOLE INPUT (std::cin) / KONSOL GİRDİSİ
    // ==========================================================================================
    std::cout << "--- 2. Console Input (std::cin) ---" << std::endl;

    // EN: We skip interactive cin in automated builds. Showing the pattern:
    // TR: Otomatik derlemelerde interaktif cin atlıyoruz. Kalıbı gösteriyoruz:
    //
    // int userRpm; std::cout << "Enter RPM: "; std::cin >> userRpm; std::cout << "You entered: "
    // << userRpm << std::endl;
    //
    // EN: For reading full lines (including spaces):
    // TR: Boşluk içeren satır okumak için: std::string fullLine; std::getline(std::cin, fullLine);
    // std::cout << "Full line: " << fullLine << std::endl;
    //
    std::cout << "[SKIPPED] Interactive cin disabled for automated build.\n"
              << std::endl;

    // ==========================================================================================
    // 3. WRITE TO FILE (std::ofstream) / DOSYAYA YAZMA
    // ==========================================================================================
    std::cout << "--- 3. Write to File (std::ofstream) ---" << std::endl;

    // EN: Create/overwrite a DTC log file. RAII: auto-closes on scope exit.
    // TR: Bir DTC log dosyası oluştur/üzerine yaz. RAII: kapsam çıkışında otomatik kapanır.
    const std::string filename = "/tmp/ecu_dtc_log.txt";
    {
      std::ofstream outFile(filename);

      if (!outFile.is_open()) {
        std::cerr << "[ERROR] Cannot open file for writing: " << filename
                  << std::endl;
        return 1;
      }

      // EN: Writing DTC records to file.
      // TR: DTC kayıtlarını dosyaya yazma.
      outFile << "=== ECU DTC LOG ===" << std::endl;
      outFile << "DTC: P0300 — Random/Multiple Cylinder Misfire Detected"
              << std::endl;
      outFile << "DTC: P0171 — System Too Lean (Bank 1)" << std::endl;
      outFile << "DTC: P0420 — Catalyst Efficiency Below Threshold" << std::endl;
      outFile << "DTC: P0442 — EVAP System Leak Detected (Small)" << std::endl;
      outFile << "Total DTCs: 4" << std::endl;

      std::cout << "Written 4 DTCs to: " << filename << std::endl;
      // EN: outFile is auto-closed here (RAII destructor).
      // TR: outFile burada otomatik kapatılır (RAII yıkıcı).
    }

    // ==========================================================================================
    // 4. READ FROM FILE (std::ifstream) / DOSYADAN OKUMA
    // ==========================================================================================
    std::cout << "\n--- 4. Read from File (std::ifstream) ---" << std::endl;

    {
      std::ifstream inFile(filename);
```

```cpp

    if (!inFile.is_open()) {
      std::cerr << "[ERROR] Cannot open file for reading: " << filename
                << std::endl;
      return 1;
    }

    // EN: Read line by line using std::getline.
    // TR: std::getline ile satır satır oku.
    std::string line;
    int lineNum = 1;
    while (std::getline(inFile, line)) {
      std::cout << "  [L" << lineNum << "] " << line << std::endl;
      ++lineNum;
    }

    std::cout << "Read " << (lineNum - 1) << " lines from: " << filename
              << std::endl;
  }

  // ==========================================================================================
  // 5. APPEND TO FILE / DOSYAYA EKLEME
  // ==========================================================================================
  std::cout << "\n--- 5. Append to File ---" << std::endl;

  {
    // EN: std::ios::app opens the file in Append mode (doesn't erase).
    // TR: std::ios::app dosyayı Ekleme modunda açar (silmez).
    std::ofstream appendFile(filename, std::ios::app);

    if (!appendFile.is_open()) {
      std::cerr << "[ERROR] Cannot open file for appending." << std::endl;
      return 1;
    }

    appendFile << "DTC: P0128 — Coolant Temperature Below Thermostat Range"
               << std::endl;
    appendFile << "Total DTCs: 5 (updated)" << std::endl;

    std::cout << "Appended 1 more DTC to: " << filename << std::endl;
  }

  // ==========================================================================================
  // 6. RE-READ TO VERIFY APPEND / EKLEME DOĞRULAMA İÇİN TEKRAR OKU
  // ==========================================================================================
  std::cout << "\n--- 6. Verify Append ---" << std::endl;

  {
    std::ifstream verifyFile(filename);
    if (verifyFile.is_open()) {
      std::string line;
      while (std::getline(verifyFile, line)) {
        std::cout << "  " << line << std::endl;
      }
    }
  }

  // ==========================================================================================
  // 7. FILE STATUS CHECKS / DOSYA DURUM KONTROLLERİ
  // ==========================================================================================
  std::cout << "\n--- 7. File Status Checks ---" << std::endl;

  std::ifstream badFile("/tmp/non_existent_file_xyz.txt");
  std::cout << "Bad file is_open(): " << badFile.is_open() << std::endl;
  std::cout << "Bad file good()   : " << badFile.good() << std::endl;
  std::cout << "Bad file fail()   : " << badFile.fail() << std::endl;

  std::ifstream goodFile(filename);
  std::cout << "Good file is_open(): " << goodFile.is_open() << std::endl;
  std::cout << "Good file good()   : " << goodFile.good() << std::endl;

  return 0;
}
```

► 09. Scope Namespace Recursion

module_01_basics/09_scope_namespace_recursion.cpp

Konu: Basics: Scope, Lifetime, Namespace & Recursion / Temeller: Kapsam, Ömür, İsim Alanı ve

Teori / Theory

THEORY: Scope & Lifetime

Variables live in "scopes" — regions delimited by `{ }`. A variable declared inside a block is INVISIBLE outside. When the closing `}` is reached, the variable is DESTROYED (stack unwinding). Key scopes: - Global scope: Declared outside all functions. Lives for the whole program. - Function/Local scope: Inside a function or block `{ }`. - `static` local: Initialized once, persists across calls (like a global but scoped to the function). Useful for caching sensor calibration data.

TEORİ: Kapsam ve Ömür

Değişkenler "kapsam" (scope) içinde yaşar — `{ }` ile sınırlanır. Bir blok içinde tanımlanan değişken dışarıda GÖRÜNMEZdir. Kapanış `}` e ulaşıldığında yok edilir. Anahtar kapsamlar: - Global kapsam: Tüm fonksiyonların dışında. Program boyunca yaşar. - Fonksiyon/Yerel kapsam: Fonksiyon veya blok `{ }` içinde. - `static` yerel: Bir kez başlatılır, çağrılar arasında kalıcıdır.

THEORY: Namespaces

Namespaces prevent name collisions in large codebases. Without them, two libraries defining `init()` would cause a linker error. Use `::` to access members. `using namespace std;` pollutes the global scope — avoid it.

TEORİ: İsim Alanları

İsim alanları büyük kod tabanlarında isim çakışmalarını önler. `using namespace std;` global kapsamı kirletir — kaçının.

CPPREF Depth: Recursion & Stack Overflow

Recursion means a function calls ITSELF. Every call pushes a new frame onto the call stack. Without a proper "base case", the stack overflows (SIGSEGV). Stack size is typically 1-8 MB. For deeply recursive algorithms, prefer iterative solutions or tail-call optimization hints.

CPPREF Derinlik: Özyineleme ve Yığın Taşması

Özyineleme bir fonksiyonun KENDİSİNİ çağırması demektir. Her çağrı çağrı yığınına yeni bir çerçeve ekler. Uygun bir "temel durum" (base case) olmadan yığın taşar (SIGSEGV). Yığın boyutu genellikle 1-8 MB'tır.
→ Ref: https://en.cppreference.com/w/cpp/language/scope
→ Ref: https://en.cppreference.com/w/cpp/language/namespace

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

// ==========================================================================================
// 1. GLOBAL SCOPE VARIABLE / GLOBAL DEĞİŞKEN
// ==========================================================================================
// EN: Accessible everywhere. Lives for the entire program duration.
// TR: Her yerden erişilebilir. Programın tüm süresi boyunca yaşar.
const std::string FIRMWARE_VERSION = "v3.2.1";
int globalDtcCount = 0;

// ==========================================================================================
// 2. NAMESPACES / İSİM ALANLARI
// ==========================================================================================
// EN: Simulating two different ECU subsystem libraries.
// TR: İki farklı ECU alt sistem kütüphanesini simüle ediyoruz.
namespace Powertrain {
void init() {
  std::cout << "  [Powertrain] Engine management initialized." << std::endl;
}
int getStatus() { return 1; } // 1 = OK
} // namespace Powertrain

namespace BodyControl {
void init() {
  std::cout << "  [BodyControl] Door/window/lighting initialized." << std::endl;
}
int getStatus() { return 1; } // 1 = OK
} // namespace BodyControl

// EN: Nested namespace (C++17 shorthand).
// TR: İç içe isim alanı (C++17 kısaltması).
namespace Vehicle::Diagnostics {
void scanAll() {
  std::cout << "  [Vehicle::Diagnostics] Full OBD-II scan complete."
            << std::endl;
}
} // namespace Vehicle::Diagnostics

// ==========================================================================================
// 3. STATIC LOCAL VARIABLE / STATİK YEREL DEĞİŞKEN
// ==========================================================================================
// EN: Simulates a sensor reading counter that persists across calls.
// TR: Çağrılar arasında kalıcı bir sensör okuma sayacını simüle eder.
void readSensor() {
  // EN: `static` variables are initialized ONCE, then persist.
  // TR: `static` değişkenler BİR KEZ başlatılır, sonra kalıcıdır.
  static int readCount = 0;
  ++readCount;
  std::cout << "  Sensor read #" << readCount << " (static counter)"
            << std::endl;
}

// ==========================================================================================
// 4. DEFAULT PARAMETERS / VARSAYILAN PARAMETRELER
// ==========================================================================================
// EN: Parameters with default values can be omitted at call site.
// TR: Varsayılan değerli parametreler çağrı yerinde atlanabilir.
void logDTC(const std::string &code, int severity = 1,
            const std::string &module = "UNKNOWN") {
  ++globalDtcCount;
  std::cout << "  DTC[" << globalDtcCount << "]: " << code
            << " | Severity: " << severity << " | Module: " << module
            << std::endl;
}
```

```cpp

// ==========================================================================================
// 5. RECURSION / ÖZYİNELEME
// ==========================================================================================
// EN: Factorial: Classic recursion demo. factorial(5) = 5×4×3×2×1 = 120.
// TR: Faktöriyel: Klasik özyineleme. factorial(5) = 5×4×3×2×1 = 120.
int factorial(int n) {
  // Base case — stops recursion / Temel durum — özyinelemeyi durdurur
  if (n <= 1) return 1;
  // Recursive case — calls itself / Özyinelemeli durum — kendini çağırır
  return n * factorial(n - 1);
}

// EN: Fibonacci: Another classic. fib(6) = 8 (0,1,1,2,3,5,8...). WARNING: Exponential time
// complexity O(2^n) — see Module 4 for memoization.
// TR: Fibonacci: Bir diğer klasik. UYARI: Üssel zaman karmaşıklığı O(2^n).
int fibonacci(int n) {
  if (n <= 0) return 0;
  if (n == 1) return 1;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

// EN: Count-down recursion — simulates ECU shutdown sequence.
// TR: Geri sayım özyinelemesi — ECU kapatma sırasını simüle eder.
void shutdownCountdown(int n) {
  if (n < 0) return; // Base case
  std::cout << "  Shutdown in: " << n << "..." << std::endl;
  shutdownCountdown(n - 1); // Recursive call
}

int main() {
  std::cout << "=== MODULE 1: SCOPE, NAMESPACE & RECURSION ===\n" << std::endl;

  // ==========================================================================================
  // 1. SCOPE DEMONSTRATION / KAPSAM GÖSTERİMİ
  // ==========================================================================================
  std::cout << "--- 1. Scope & Lifetime ---" << std::endl;
  std::cout << "Global firmware version: " << FIRMWARE_VERSION << std::endl;

  {
    // EN: This variable lives ONLY inside this block.
    // TR: Bu değişken SADECE bu bloğun içinde yaşar.
    int localRpm = 3500;
    std::cout << "Inside block: localRpm = " << localRpm << std::endl;
  }
  // localRpm is DEAD here! / localRpm artık YOK! std::cout << localRpm; // ERROR: 'localRpm'
  // undeclared

  // EN: Variable shadowing — inner scope hides outer variable.
  // TR: Değişken gölgeleme — iç kapsam dış değişkeni gizler.
  int outerTemp = 25;
  std::cout << "Outer temp: " << outerTemp << std::endl;
  {
    int innerTemp = 90; // Different name to avoid -Wshadow
    std::cout << "Inner temp (different scope): " << innerTemp << std::endl;
  }
  std::cout << "Outer temp unchanged: " << outerTemp << "\n" << std::endl;

  // ==========================================================================================
  // 2. NAMESPACE DEMO / İSİM ALANI GÖSTERİMİ
  // ==========================================================================================
  std::cout << "--- 2. Namespaces ---" << std::endl;

  // EN: Both have init() but no collision thanks to namespaces.
  // TR: İkisi de init() fonksiyonuna sahip ama isim alanları sayesinde çakışma yok.
  Powertrain::init();
  BodyControl::init();
  Vehicle::Diagnostics::scanAll();

  std::cout << "Powertrain status: " << Powertrain::getStatus() << std::endl;
  std::cout << "BodyControl status: " << BodyControl::getStatus() << "\n"
            << std::endl;
```

```

    // ==========================================================================================
    // 3. STATIC LOCAL VARIABLE / STATİK YEREL DEĞİŞKEN
    // ==========================================================================================
    std::cout << "--- 3. Static Local Variable ---" << std::endl;

    // EN: readCount inside readSensor() persists across calls.
    // TR: readSensor() içindeki readCount çağrılar arasında kalıcıdır.
    readSensor(); // #1
    readSensor(); // #2
    readSensor(); // #3
    std::cout << std::endl;

    // ==========================================================================================
    // 4. DEFAULT PARAMETERS / VARSAYILAN PARAMETRELER
    // ==========================================================================================
    std::cout << "--- 4. Default Parameters ---" << std::endl;

    logDTC("P0300");                            // Uses all defaults
    logDTC("P0171", 2);                         // Custom severity
    logDTC("P0420", 3, "Powertrain");           // All specified
    std::cout << "Total DTCs logged: " << globalDtcCount << "\n" << std::endl;

    // ==========================================================================================
    // 5. RECURSION DEMOS / ÖZYİNELEME GÖSTERİMLERİ
    // ==========================================================================================
    std::cout << "--- 5. Recursion ---" << std::endl;

    // Factorial
    std::cout << "factorial(5) = " << factorial(5) << std::endl;  // 120
    std::cout << "factorial(10) = " << factorial(10) << std::endl; // 3628800

    // Fibonacci
    std::cout << "fibonacci(6) = " << fibonacci(6) << std::endl;  // 8
    std::cout << "fibonacci(10) = " << fibonacci(10) << std::endl; // 55

    // ECU Shutdown Countdown
    std::cout << "\nECU shutdown sequence:" << std::endl;
    shutdownCountdown(3);

    return 0;
}
```

Modül 2: OOP — Nesne Yönelimli Programlama

Nesne Yönelimli Programlama

8 kaynak dosya · Standart: C++17

C++17

► 01. Classes Objects

module_02_oop/01_classes_objects.cpp

Konu: OOP Part 1: Architecture of Classes, Objects, and Access Specifiers / OOP Kısım 1:

Teori / Theory

THEORY: What are Classes and Objects? Why do we need them?

In C++, primitive types (int, float, char) are often not enough to represent complex, real-world entities. For example, a 'TemperatureSensor' in an automotive ECU isn't just an integer. It has a 'sensorName' (string), 'currentTempC' (int), and behaviors like 'reportOverheat()' (function).
A 'Class' is a blueprint or a template. It takes up absolutely NO memory by itself. It just tells the compiler: "If someone creates a TemperatureSensor, it will look like this."
An 'Object' (or Instance) is the actual creation of that blueprint in the RAM. When we say `TemperatureSensor s1;`, memory is allocated, and the Object comes to life.

TEORİ: Sınıf ve Nesne Nedir?

C++'da ilkel tipler (int, float, char) çoğu zaman karmaşık, gerçek dünya varlıklarını temsil etmek için yeterli değildir. Örneğin, bir otomotiv ECU'daki 'SıcaklıkSensörü' sadece bir tam sayı değildir. Sensörün bir 'Adı' (string), 'MevcutSıcaklık' (int) ve 'asırılsınmaRaporla()' gibi davranışları (fonksiyon) vardır.
'Sınıf' (Class) bir donanım mimari çizimi, bir şablondur. Kendi başına RAM'de HİÇBİR yer kaplamaz. Sadece derleyiciye şunu söyler: "Eğer biri bir SıcaklıkSensörü oluşturursa, yapısı böyle olacak."
'Nesne' (Object veya Instance) ise o şablonun RAM'deki canlı kanlı üretimidir. `TemperatureSensor s1;` dediğimiz anda bellekte yer ayrılır ve Nesne hayata gözlerini açar.

THEORY: Why use Access Specifiers?

Why do we make data 'private' in a class? Why can't anyone just say `p1.health = -500;`? Because we want to protect the system's safety and integrity. Forcing the outside world to communicate with the variables ONLY through our permitted 'public' functions is called Encapsulation.

TEORİ: Erişim Belirleyiciler Neden Var?

Bir sınıftaki verileri neden 'private' (özel) yaparız? Neden herkes doğrudan `p1.health = -500;` diyemesin? Çünkü sistemin güvenliğini ve bütünlüğünü korumak isteriz. Dış dünyanın sadece bizim izin verdiğimiz 'public' (genel) fonksiyonlar üzerinden içerikteki değişkenlerle iletişim kurmasına Encapsulation (Kapsülleme) denir.

CPPREF Depth: Memory Layout & POD (Plain Old Data) Types

According to cppreference, as soon as you add `private` access specifiers or `virtual` functions, the Class is NO LONGER a "Standard Layout" (POD) type. This means you cannot safely use `memcpy()` or pass it to a C-language API.

CPPREF Derinlik: Bellek Düzeni ve POD (Düz Eski Veri) Tipleri

cppreference der ki: Bir sınıfa `private` değişkenler veya `virtual` fonksiyonlar eklerseniz, o sınıf artık C-uyumlu Standart Bellek Düzenine (POD) sahip değildir! Yani bu nesneyi `memcpy()` ile kopyalamak veya ağ üzerinden byte dizisi olarak ham haliyle göndermek Tanımsız Davranış (UB) yaratır!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

/**
 * @class TemperatureSensor
 * @brief Represents an automotive temperature sensor (e.g. EGT, coolant). / Bir otomotiv
 * sıcaklık sensörünü temsil eder (örn. EGT, soğutma suyu).
 *
 * @note
 * EN: Design Philosophy: Data is kept private. It cannot be directly modified from the outside.
 * Anyone who wants to change it must use our 'public' methods.
 * TR: Tasarım Felsefesi: Veriler private (saklı) tutulmuştur. Dışarıdan doğrudan
 * değiştirilemezler. Değişiklik yapmak isteyen, 'public' metodlarımızı kullanmak zorundadır.
 */
class TemperatureSensor {
  // EN: 'private' section is INACCESSIBLE from outside the class.
  // TR: 'private' bölümüne sınıfın DIŞINDAN erişilemez. Sadece sınıfın içindeki fonksiyonlar
  // görebilir.
private:
  std::string sensorName; /**< Sensor ID. */
  int currentTempC; /**< Current temperature in °C. */

  // EN: 'public' section is accessible from anywhere. It's the interface to the outside world.
  // TR: 'public' bölümüne her yerden erişilebilir. Sınıfın dış dünyayla olan arayüzüdür.
public:
  /**
   * @brief Constructor
   * TR: Yapıcı Fonksiyon
   *
   * @details
   * EN: [THEORY: What is a Constructor?] It's the first function that runs automatically the
   * moment an Object is created in RAM. It has the EXACT same name as the Class and NO return
   * type. Its purpose is to put the object in a "ready and valid" state. Member Initialization
   * List (":"): Used for performance. Instead of assigning inside the body "{}", we write
   * directly to memory upon creation.
   *
   * TR: [TEORİ: Constructor Nedir?] Nesne RAM'de "yaratıldığı anda" ilk çalışacak olan
   * fonksiyondur. Sınıf ile tamamen AYNI isme sahiptir ve herhangi bir dönüş tipi yoktur. Amacı
   * nesneyi kullanıma "hazır ve geçerli" (valid) bir hale getirmektir. Üye Başlatma Listesi
   * (Member Initialization List - ":"): Performans için önemlidir. Süslü parantez içinde atamak
   * yerine doğrudan belleğe yazmayı sağlar.
   *
   * @param pName Sensor identifier / Sensörün başlangıç adı
   * @param pTemp Initial temperature °C / Başlangıç sıcaklığı (°C)
   */
  TemperatureSensor(std::string pName, int pTemp)
      : sensorName(pName), currentTempC(pTemp) {
    std::cout << "[SYSTEM] Constructor Called: " << sensorName
              << " initialized at " << currentTempC << " °C." << std::endl;
  }
```

```cpp

  /**
   * @brief Reports a temperature change safely.
   * TR: Sıcaklık değişimini güvenli bir şekilde raporlar.
   *
   * @details
   * EN: The power of Encapsulation! By preventing direct access to 'currentTempC', we can
   * enforce an overheat threshold (120 °C) right here.
   * TR: Kapsülleme (Encapsulation) gücü! Sıcaklığı direkt değiştirmek yerine bu fonksiyonu
   * kullandığımız için, aşırı ısınma eşiğini (120 °C) burada zorla uygulayabiliriz.
   *
   * @param delta Temperature change in °C / Sıcaklık değişimi (°C)
   */
  void reportTemperatureChange(int delta) {
    std::cout << "[SENSOR] " << sensorName << " temp changes by +"
              << delta << " °C." << std::endl;
    currentTempC += delta;

    // EN: Our safety net — overheat threshold!
    // TR: Güvenlik eşiği!
    if (currentTempC > 120) {
      std::cout << "[ALARM] " << sensorName
                << " OVERHEAT! Temp: " << currentTempC
                << " °C (Limit: 120 °C)" << std::endl;
    } else {
      std::cout << "[STATUS] " << sensorName
                << " current temp: " << currentTempC << " °C" << std::endl;
    }
  }
};

/**
 * @brief Main function demonstrating Class instantiation.
 * TR: Sınıf nesnesi yaratılmasını gösteren ana fonksiyon.
 */
int main() {
  std::cout << "=== OOP Part 1: Classes & Objects ===" << std::endl;

  // EN: 1. Object Instantiation
  // TR: 1. Nesne Yaratma
  // EN: Creating a sensor object from the blueprint. Constructor runs here!
  // TR: Şablondan bir sensör nesnesi yaratıyoruz. Yapıcı burada çalışır!
  TemperatureSensor egtSensor("EGT_Cyl1", 85);

  // EN: 2. Encapsulation
  // TR: 2. Kapsülleme ve Sınırlar
  // EN: ERROR! 'currentTempC' is private.
  // TR: HATA! 'currentTempC' private (ozel) veridir. egtSensor.currentTempC = 9999;

  // EN: Using the safe Public Method
  // TR: Güvenli Public Metodu kullanıyoruz.
  egtSensor.reportTemperatureChange(15);
  // Trying to push temp over the limit / Sıcaklığı limit üstüne çıkarmayı deniyoruz.
  egtSensor.reportTemperatureChange(25);

  return 0;
}
```

► 02. Inheritance Polymorphism

module_02_oop/02_inheritance_polymorphism.cpp

Konu: OOP Part 2: Architecture of Inheritance and Polymorphism / OOP Kısım 2: Kalıtım ve Çok

Teori / Theory

THEORY: What is Inheritance? Why use it?

Software developers hate repeating code (DRY Principle - Don't Repeat Yourself). If we have a 'Dog', a 'Cat', and a 'Bird', they all have an 'age', 'weight', and an 'eat()' function. Instead of writing these properties 3 times, we create a "Base Class" called 'Animal'. The Dog and Cat "inherit" everything from Animal, and only add their unique features (like bark for Dog).

TEORİ: Kalıtım Nedir ve Neden Kullanılır?

Yazılım geliştiriciler aynı kodu tekrar yazmaktan nefret eder (DRY Prensibi). Bir 'Köpek', 'Kedi' ve 'Kuş' sınıfımız varsa, hepsinin bir 'yaş', 'ağırlık' değişkeni ve 'yemekYe()' fonksiyonu olacaktır. Bunları 3 defa ayrı ayrı yazmak yerine 'Hayvan' (Animal) adında bir "Temel Sınıf" (Base Class) yaratırız. Köpek ve Kedi, Hayvan sınıfındaki her şeyi miras alır, üzerine sadece köpeğe özgü şeyleri (havlama gibi) ekleriz.

THEORY: Polymorphism and V-Table Architecture

"Give a command, let the object react according to its true nature." If we have an 'Animal' pointer, but it actually points to a 'Dog', which sound will play if we call `makeSound()`? This is where 'virtual' comes in. 'virtual' tells the compiler to do "Late Binding / Dynamic Dispatch". "Look at the object's REAL type at runtime (using the V-Table) and call the Dog's function."

TEORİ: Çok Biçimlilik ve V-Table

"Bir komut ver, arkadaki nesne kendi doğasına göre tepki versin." Eğer 'Hayvan' tipinde bir işaretçimiz (pointer) varsa, ama bu işaretçi gerçekte bir 'Köpek' nesnesini tutuyorsa, `sesÇıkar()` dediğimizde kimin sesi çıkacak? İşte 'virtual' (sanal) anahtar kelimesi devreye girer. 'virtual', derleyiciye "Program çalışırken nesnenin gerçek tipine bak (V-Table / Sanal Tablo üzerinden) ve Köpeğin fonksiyonunu çağır (Geç Bağlama)" der. Eğer Polymorphism olmasaydı, karşımızdaki nesne gerçekte bir Köpek olsa bile, işaretçi tipi Hayvan olduğu için sıradan bir Hayvan sesi çıkardı.

CPPREF Depth: Object Slicing & V-Table Performance Penalty

"Object Slicing" occurs when you pass a Derived object by VALUE to a Base. The compiler literally "slices off" the extra data! ALWAYS pass polymorphic objects by Pointer (*) or Reference (&). Also, `virtual` functions have a slight CPU cost because they require an extra pointer jump (vptr -> vtable) at runtime, which blocks the compiler from 'inlining' the function.

CPPREF Derinlik: Nesne Dilimlemesi ve V-Tablo Performans Cezası

Eğer bir Köpek nesnesini `void feed(Animal a)` şeklinde Değer Kopyası ile gönderirseniz "Nesne Dilimlenmesi (Object Slicing)" yaşanır, köpeklik özellikleri kesilip atılır! Polimorfizm SADECE pointer/referans `feed(Animal& a)` ile çalışır. Ayrıca `virtual` fonksiyonlar oyun anında ekstra bir pointer araması (vptr) yaptıkları için derleyicinin "inlining" yapmasını engeller (Ufak Hız Kaybı).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <vector>

/**
 * @class Animal
 * @brief Base class (Temel Sınıf). Represents generic animal traits. / Genel hayvan
 * özelliklerini barındırır.
 */
class Animal {
public:
  Animal() { std::cout << "[Animal] Created. (Hayvan Doğdu)" << std::endl; }

  /**
   * @brief [THEORY / TEORİ]: The Vital Importance of Virtual Destructor / Virtual Yıkıcının
   * Hayati Önemi
   *
   * EN: If you're building a base class to be inherited from, its DESTRUCTOR MUST BE VIRTUAL!
   * Why? If we say `Animal* pet = new Dog();` and later `delete pet;`, without 'virtual', only
   * the Animal's destructor is called. The Dog's memory becomes a LEAK!
   *
   * TR: Eğer miras alınacak bir temel sınıf yapıyorsanız YIKICI FONKSİYON MUTLAKA VIRTUAL
   * OLMALIDIR. Neden? Eğer biz "Animal* pet = new Dog();" dersek ve ardından "delete pet;"
   * yaparsak komut sınıfı silmeye 'Animal'dan başlar. Eğer virtual demezek, Dog sınıfının
   * yıkıcısı ASLA çağrılmaz ve köpeğe ayırdığımız hafıza sızıntıya (memory leak) uğrar. O yüzden
   * 'virtual' deriz.
   */
  virtual ~Animal() {
    std::cout << "[Animal] Destroyed. (Hayvan Yok Oldu)" << std::endl;
  }

  /**
   * @brief Polymorphic function / Çok biçimli fonksiyon.
   *
   * EN: 'virtual' allows derived classes to override this behavior.
   * TR: 'virtual', bu sınıftan üretilen alt sınıfların bu fonksiyonu ezmesine (override) izin
   * verir.
   */
  virtual void makeSound() const {
    std::cout << "(Silence... Sessizlik...)" << std::endl;
  }
};

/**
 * @class Dog
 * @brief Derived class (Alt / Türetilmiş Sınıf). Inherits from Animal.
 */
// EN: Inheritance syntax
// TR: Kalıtım sözdizimi
class Dog : public Animal {
public:
  Dog() {
    std::cout << "[Dog] Born. (Köpek Hayata Gözlerini Açtı)" << std::endl;
  }

  ~Dog() override {
    std::cout << "[Dog] Destroyed. (Köpek Yok Oldu)" << std::endl;
  }

  /**
   * @brief Overrides the base class functionality / Temel sınıftaki fonksiyonu Köpeklerin
   * doğasına göre ezer.
   *
   * EN: 'override' is optional but HIGHLY RECOMMENDED. It tells the compiler to verify our
   * override.
   * TR: 'override' yazmak derleyicinin bizi denetlemesini sağlar, hata yaparsak uyarır.
   */
  void makeSound() const override {
    std::cout << "Dog says: Woof! Woof! (Hav! Hav!)" << std::endl;
  }
};
```

```cpp

/**
 * @class Cat
 * @brief Another Derived class / Başka bir türetilmiş sınıf.
 */
class Cat : public Animal {
public:
  Cat() { std::cout << "[Cat] Born. (Kedi Doğdu)" << std::endl; }
  ~Cat() override {
    std::cout << "[Cat] Destroyed. (Kedi Yok Oldu)" << std::endl;
  }

  void makeSound() const override {
    std::cout << "Cat says: Meow! (Miyav!)" << std::endl;
  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// EN: IDiagnosticService — Pure Abstract Interface (Automotive Diagnostics)
// TR: IDiagnosticService — Saf Soyut Arayüz (Otomotiv Teşhis)
// ═══════════════════════════════════════════════════════════════════════════════

/**
 * @class IDiagnosticService
 * @brief EN: Pure abstract interface for automotive diagnostic protocols. Any class implementing
 * this MUST provide all three methods.
 * TR: Otomotiv teşhis protokolleri için saf soyut arayüz. Bu arayüzü uygulayan her sınıf üç
 * metodu da sağlamalıdır.
 */
class IDiagnosticService {
public:
    // EN: Pure virtual — every implementation must provide a service name
    // TR: Saf sanal — her uygulama bir servis adı sağlamalı
    virtual std::string getName() const = 0;

    // EN: Pure virtual — runs a diagnostic check and returns pass/fail
    // TR: Saf sanal — teşhis kontrolü çalıştırır ve geçti/kaldı döner
    virtual bool runDiagnostic() = 0;

    // EN: Virtual destructor defaulted — required for safe polymorphic deletion
    // TR: Varsayılan sanal yıkıcı — güvenli polimorfik silme için gerekli
    virtual ~IDiagnosticService() = default;
};

// ─── OBD2Service — Concrete Implementation ─────────────────────────────────
// EN: OBD-II (On-Board Diagnostics) standard used in all modern vehicles
// TR: OBD-II (Araç İçi Teşhis) tüm modern araçlarda kullanılan standart

class OBD2Service : public IDiagnosticService {
private:
    std::string vehicleVIN_;

public:
    explicit OBD2Service(const std::string& vin) : vehicleVIN_(vin) {
        std::cout << "[OBD2Service] Initialized for VIN: " << vehicleVIN_ << std::endl;
    }

    std::string getName() const override {
        return "OBD-II Diagnostic (VIN: " + vehicleVIN_ + ")";
    }

    bool runDiagnostic() override {
        std::cout << "[OBD2] Scanning DTCs (Diagnostic Trouble Codes) for "
                  << vehicleVIN_ << "..." << std::endl;
        std::cout << "[OBD2] Result: No fault codes detected. PASS." << std::endl;
        return true;
    }
};
```

```cpp

// ─── CANDiagService — Another Concrete Implementation ──────────────────────────
// EN: CAN Bus (Controller Area Network) diagnostics for vehicle ECU networks
// TR: CAN Bus (Denetleyici Alan Ağı) araç ECU ağları için teşhis

class CANDiagService : public IDiagnosticService {
private:
    int busSpeed_;

public:
    explicit CANDiagService(int busSpeedKbps) : busSpeed_(busSpeedKbps) {
        std::cout << "[CANDiagService] Initialized at " << busSpeed_
                  << " kbps" << std::endl;
    }

    std::string getName() const override {
        return "CAN Bus Diagnostic (" + std::to_string(busSpeed_) + " kbps)";
    }

    bool runDiagnostic() override {
        std::cout << "[CAN] Probing CAN bus at " << busSpeed_
                  << " kbps..." << std::endl;
        std::cout << "[CAN] Result: Bus load 42%, all nodes responsive. PASS." << std::endl;
        return true;
    }
};

// ─── FinalConfig — Demonstrates 'final' keyword on a class ─────────────────────
// EN: A class marked 'final' cannot be inherited from. Useful for sealed configuration objects
// that must not be extended.
// TR: 'final' ile işaretlenen bir sınıftan miras alınamaz. Genişletilmemesi gereken kapalı
// yapılandırma nesneleri için kullanışlıdır.

class FinalConfig final {
private:
    std::string configName_;
    int configValue_;

public:
    FinalConfig(const std::string& name, int value)
        : configName_(name), configValue_(value) {}

    void print() const {
        std::cout << "Config: " << configName_ << " = " << configValue_ << std::endl;
    }
};

// EN: The following would cause a compile error:
// TR: Aşağıdaki derleme hatası verir: class DerivedConfig : public FinalConfig {};  // ERROR:
// cannot derive from 'final' class

// ─── Shape / Circle / FixedCircle — override and final on methods ──────────────
// EN: Demonstrates 'override' for correctness and 'final' to prevent further overriding
// TR: Doğruluk için 'override' ve daha fazla ezmeyi önlemek için 'final' gösterir

class Shape {
public:
    virtual double area() const {
        return 0.0;
    }
    virtual ~Shape() = default;
};

class Circle : public Shape {
protected:
    double radius_;

public:
    explicit Circle(double r) : radius_(r) {}
```

```cpp

    // EN: 'override' ensures we are truly overriding a base virtual method. If we misspelled
    // 'area' as 'Area', the compiler catches it.
    // TR: 'override', gerçekten bir temel sanal metodu ezdiğimizi garanti eder. 'area' yerine
    // 'Area' yazarsak derleyici yakalar.
    double area() const override {
        return 3.14159265358979 * radius_ * radius_;
    }
};

class FixedCircle final : public Circle {
public:
    explicit FixedCircle(double r) : Circle(r) {}

    // EN: 'final' on this method means NO class inheriting from FixedCircle can override area()
    // again. (But FixedCircle itself is also 'final', so no class can inherit from it at all.)
    // TR: Bu metod üzerindeki 'final', FixedCircle'dan miras alan hiçbir sınıfın area()'yı
    // tekrar ezemeyeceği anlamına gelir.
    double area() const final {
        return 3.14159265358979 * radius_ * radius_;
    }
};

int main() {
  std::cout << "=== OOP Part 2: Polymorphism (Çok Biçimlilik) ===\n" << std::endl;

  // EN: Pointers are Base (Animal), but objects are Derived.
  // TR: İşaretçimiz Base (Animal), ama işaret etiğimiz şeyler gerçek nesneler.
  Animal *pet1 = new Dog();
  std::cout << "---" << std::endl;
  Animal *pet2 = new Cat();

  std::cout << "\n[POLYMORPHISM IN ACTION / ÇOK BİÇİMLİLİK DEVREDE]" << std::endl;

  // EN: Thanks to the V-Table, the REAL types will speak!
  // TR: V-Table sayesinde gerçek tiplerine göre ses çıkaracaklar!
  pet1->makeSound();
  pet2->makeSound();

  std::cout << "\n[CLEANUP / TEMİZLİK]" << std::endl;
  // EN: Virtual Destructor guarantees perfect teardown. Memory Leak = 0!
  // TR: Virtual Destructor sayesinde MÜKEMMEL bir yıkım sırası izlenecek. Memory Leak 0!
  delete pet1;
  std::cout << "---" << std::endl;
  delete pet2;

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: ADVANCED POLYMORPHISM — Automotive Diagnostic Service Demos
  // TR: GELİŞMİŞ POLİMORFİZM — Otomotiv Teşhis Servisi Gösterimleri
  // ═══════════════════════════════════════════════════════════════════════════════

  std::cout << "\n\n=== ADVANCED POLYMORPHISM: AUTOMOTIVE DIAGNOSTICS ===\n" << std::endl;

  // ─── 1. Abstract Interface — IDiagnosticService ──────────────────────────────
  // EN: We cannot instantiate IDiagnosticService directly (it has pure virtual methods). We
  // create concrete implementations through base pointers.
  // TR: IDiagnosticService doğrudan örneklenemez (saf sanal metotları var). Temel işaretçiler
  // aracılığıyla somut uygulamalar oluştururuz.
  std::cout << "--- Demo: Abstract Interface — IDiagnosticService ---" << std::endl;
  {
      IDiagnosticService* svc1 = new OBD2Service("WVWZZZ3CZWE123456");
      IDiagnosticService* svc2 = new CANDiagService(500);

      std::cout << "\nService 1: " << svc1->getName() << std::endl;
      svc1->runDiagnostic();

      std::cout << "\nService 2: " << svc2->getName() << std::endl;
      svc2->runDiagnostic();
```

```cpp

        // EN: Virtual destructor ensures proper cleanup through base pointer
        // TR: Sanal yıkıcı, temel işaretçi aracılığıyla doğru temizliği sağlar
        delete svc1;
        delete svc2;
    }
    std::cout << std::endl;

    // ─── 2. final Class — FinalConfig ──────────────────────────────────────────────────────
    // EN: FinalConfig is marked 'final' — attempting to inherit from it would produce a
    // compile-time error. This is useful for sealed ECU configuration objects that must remain
    // unextended.
    // TR: FinalConfig 'final' olarak işaretlenmiş — ondan miras almaya çalışmak derleme zamanı
    // hatası üretir. Genişletilmemesi gereken kapalı ECU yapılandırma nesneleri için
    // kullanışlıdır.
    std::cout << "--- Demo: final Class — FinalConfig ---" << std::endl;
    {
        FinalConfig ecuConfig("MaxRPM", 7500);
        ecuConfig.print();

        FinalConfig brakeConfig("ABSThreshold_mbar", 4200);
        brakeConfig.print();

        // EN: Uncommenting the following line would cause: error: cannot derive from 'final' base
        // 'FinalConfig'
        // TR: Aşağıdaki satırın yorumunu kaldırmak şu hataya neden olur: error: 'final' temel
        // 'FinalConfig'ten türetilemez class ExtendedConfig : public FinalConfig {};  // COMPILE
        // ERROR!
    }
    std::cout << std::endl;

    // ─── 3. override Keyword — Shape / Circle ──────────────────────────────────────────────
    // EN: 'override' is a C++11 keyword that tells the compiler: "I intend to override a virtual
    // function in the base class. If I'm wrong (e.g., wrong signature), give me a compile error."
    // This prevents silent bugs.
    // TR: 'override', derleyiciye "Temel sınıftaki bir sanal fonksiyonu ezmeyi amaçlıyorum.
    // Yanlışsam derleme hatası ver" der. Sessiz hataları önler.
    std::cout << "--- Demo: override Keyword — Shape/Circle ---" << std::endl;
    {
        Shape baseShape;
        Circle wheel(17.5);
        FixedCircle hubcap(7.0);

        // EN: Polymorphism through base pointer — correct area() called each time
        // TR: Temel işaretçi aracılığıyla polimorfizm — her seferinde doğru area() çağrılır
        Shape* shapes[] = { &baseShape, &wheel, &hubcap };
        const char* names[] = { "Base Shape", "Wheel (Circle r=17.5)", "Hubcap (FixedCircle r=7.0)" };

        for (int i = 0; i < 3; ++i) {
            std::cout << names[i] << " area = " << shapes[i]->area() << std::endl;
        }
    }
    std::cout << std::endl;

    // ─── 4. Polymorphic Container — vector<unique_ptr<IDiagnosticService>> ─────────────────
    // EN: The idiomatic C++ way to manage polymorphic objects: store them in a vector of
    // unique_ptr to the base interface. No manual delete needed. This is how modern automotive
    // middleware (AUTOSAR Adaptive) manages service instances.
    // TR: Polimorfik nesneleri yönetmenin idiomatik C++ yolu: temel arayüze unique_ptr vektöründe
    // saklamak. Manuel delete gerekmez. Modern otomotiv ara yazılımı (AUTOSAR Adaptive) servis
    // örneklerini böyle yönetir.
    std::cout << "--- Demo: Polymorphic Container — vector<unique_ptr> ---" << std::endl;
    {
        std::vector<std::unique_ptr<IDiagnosticService>> diagnostics;

        diagnostics.push_back(std::make_unique<OBD2Service>("TMBJB9NE1L0123456"));
        diagnostics.push_back(std::make_unique<CANDiagService>(250));
        diagnostics.push_back(std::make_unique<OBD2Service>("WBA3A5C55CF256789"));
        diagnostics.push_back(std::make_unique<CANDiagService>(1000));

        std::cout << "\nRunning all diagnostics in polymorphic container:" << std::endl;
        std::cout << "================================================" << std::endl;

        int passCount = 0;
        int totalCount = 0;
```

```
352         for (const auto& svc : diagnostics) {
353             std::cout << "\n>> " << svc->getName() << std::endl;
354             if (svc->runDiagnostic()) {
355                 ++passCount;
356             }
357             ++totalCount;
358         }
359 
360         std::cout << "\n==============================================" << std::endl;
361         std::cout << "Diagnostic Summary: " << passCount << "/" << totalCount
362                   << " passed." << std::endl;
363 
364         // EN: When 'diagnostics' vector goes out of scope, unique_ptr automatically deletes each
365         // service. Zero memory leaks.
366         // TR: 'diagnostics' vektörü kapsam dışına çıkınca unique_ptr her servisi otomatik siler.
367         // Sıfır bellek sızıntısı.
368     }
369 
370     return 0;
371 }
```

► 03. Rule Of Three Five

module_02_oop/03_rule_of_three_five.cpp

Konu: OOP Part 3: Memory Leaks and "Rule of 3 / 5" / OOP Kisim 3: Bellek Sizintilari ve

Teori / Theory

THEORY: Why does the Rule of 0 / 3 / 5 exist? (Shallow Copy Crisis)

When you create a basic Class, the C++ compiler automatically generates 3 things for you:

1. A default Destructor 2. A default Copy Constructor (Sensor s2 = s1;) 3. A default Copy

Assignment Operator (p2 = p1;)
This is fine for simple variables (ints, floats). BUT! If your class has a POINTER that allocates memory dynamically (using `new` or `malloc`), the default copy performs a "Shallow Copy". Both objects will point to the SAME memory! When they are destroyed, they will "Double Free" and crash!

Temel bir sınıf yazdığınızda, C++ derleyicisi otomatik olarak 3 şey üretir: 1. Varsayılan Yıkıcı (Destructor) 2. Varsayılan Kopya Yapıcı (Copy Constructor) (Sensor s2 = s1;) 3. Varsayılan Kopya Atama Operatörü (Copy Assignment) (p2 = p1;)
Basit değişkenler (int, float) için bu yeterlidir. AMA! Sınıfınızda dinamik bellek tahsis eden (`new` veya `malloc` kullanan) bir POINTER varsa, varsayılan kopya "Sığ Kopya (Shallow Copy)" yapar. Her iki nesne de AYNI belleği işaret eder! Yıkıldıklarında "Double Free" olur ve program çöker!

RULE OF 3

"If a class needs a user-defined Destructor (i.e. manual `delete`), then you MUST also define these two yourself:"

3'LU KURAL

"Eğer bir sınıfın kullanıcı tanımlı bir Yıkıcıya (yani manuel `delete`) ihtiyacı varsa, şu ikisini de KENDİNİZ tanımlamalısınız:"

Assignment Operator)

CPPREF Depth: The Rule of 5 and `noexcept` Move

1. Yıkıcı (Destructor) 2. Kopya Yapıcı (Copy Constructor) 3. Kopya Atama Operatörü (Copy

C++11 introduced Move Constructors and Move Assignment (Rule of 5). CppReference strongly mandates: YOU MUST mark your Move functions as `noexcept`! If `noexcept` is missing, `std::vector` reallocation will refuse to Move your objects and will fall back to painfully slow Deep Copies!

CPPREF Derinlik: 5'li Kural ve `noexcept` Taşıma

C++11 ile "Move (Tasima)" mimarisi geldi ve Kural 5'e cikti. CppReference der ki: Tasima (Move) metotlarinizi KESINLIKLE `noexcept` (Hatalara Kapali) olarak isaretlemelisiniz! Eger eksikse, `std::vector` kapasitesini artirirken korkuya kapilir ve Tasima yapmak yerine objelerinizi tek tek Kopyalar (Program %70 yavaslar)! @note C++11 ile beraber "Move Semantics" gelince buralara "Move" olan 2 ozellik daha eklendi ve kuralin adi "Rule of 5" oldu.

Kaynak Kod / Source Code

```cpp
#include <cstring>
#include <iostream>
#include <utility>

/**
 * @class StringBox
 * @brief EN: Custom primitive string class that manages dynamic memory.
 * TR: Dinamik bellek yoneten ilkel metin sinifi.
 */
class StringBox {
private:
  // EN: Dynamically allocated text
  // TR: Dinamik RAM
  char *data;
  size_t length;

public:
  // 1. Constructor
  StringBox(const char *sourceText) {
    length = std::strlen(sourceText);
    // EN: Dynamic heap allocation
    // TR: Heap uzerinde RAM
    data = new char[length +
                    1];
    std::strcpy(data, sourceText);
    std::cout << "[Constructor] Memory allocated for: " << data << " (Hafiza ayrildi)" << std::endl;
  }
```

```cpp

  // EN: --- RULE OF 3 BEGINS! DESTRUCTOR is user-defined ---
  // TR: --- 3'LÜ KURAL BAŞLIYOR! YIKICI (DESTRUCTOR) kullanıcı tanımlıdır ---
  // EN: [1/3] RULE 1: DESTRUCTOR
  // TR: [1/3] KURAL 1: YIKICI (DESTRUCTOR)
  ~StringBox() {
    std::cout << "[Destructor] Deleting: " << data << " (Siliniyor)" << std::endl;
    delete[] data;
  }

  // EN: --- IF WE WROTE DELETE ABOVE, WE MUST HAND-WRITE THESE ---
  // TR: --- YUKARIDA DELETE YAZDIYSAN, BUNLARI DA KENDİN YAZMAK ZORUNDASIN ---

  // EN: [2/3] RULE 2: COPY CONSTRUCTOR (Deep Copy)
  // TR: [2/3] KURAL 2: KOPYA OLUŞTURUCU (Derin Kopya)
  // EN: Example: StringBox B = A;
  // TR: Ornek: StringBox B = A;
  StringBox(const StringBox &other) {
    length = other.length;
    std::cout << "[Copy Constructor] Deep copying: " << other.data << " (Derin kopyalandi)" <<
        std::endl;

    // EN: NEVER do 'data = other.data' (Shallow Copy disaster)! Allocate NEW RAM!
    // TR: Asla 'data = other.data' yapmayin! Yeni RAM ac!
    data = new char[length + 1];
    std::strcpy(data, other.data);
  }

  // EN: [3/3] RULE 3: COPY ASSIGNMENT OPERATOR
  // TR: [3/3] KURAL 3: KOPYA ATAMA OPERATÖRÜ
  // EN: Example: B = A; (B already exists, must free old content first!)
  // TR: Ornek: B = A; (B zaten yasiyorken eski icerigi silinmeli!)
  StringBox &operator=(const StringBox &other) {
    // EN: Self-assignment guard
    // TR: Kendine atama korumasi
    if (this == &other)
      return *this;

    std::cout << "[Copy Assignment] Overwriting with: " << other.data << " (Uzerine yazildi)" <<
        std::endl;

    // EN: Free old memory first
    // TR: Eski bellek once silinmeli
    delete[] data;

    length = other.length;
    data = new char[length + 1];
    std::strcpy(data, other.data);

    return *this;
  }

  void print() const {
    std::cout << "Box Content (Kutu Icerigi): " << data << std::endl;
  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// EN: ECUFirmwareBuffer — Full Rule of 5 Demonstration (Automotive Domain)
// TR: ECUFirmwareBuffer — Tam 5'li Kural Gösterimi (Otomotiv Alanı)
// ═══════════════════════════════════════════════════════════════════════════════

/**
 * @class ECUFirmwareBuffer
 * @brief EN: Simulates a firmware data buffer for an automotive ECU (Electronic Control Unit).
 * Demonstrates Rule of 5 with move semantics.
 * TR: Otomotiv ECU (Elektronik Kontrol Ünitesi) için firmware veri tamponunu simüle eder. Move
 * semantics ile 5'li Kuralı gösterir.
 */
```

```cpp
class ECUFirmwareBuffer {
private:
    char* data_;
    size_t size_;
    std::string label_;

public:
    // EN: Parameterized Constructor — allocates firmware buffer on the heap
    // TR: Parametreli Yapıcı — firmware tamponunu heap üzerinde ayırır
    ECUFirmwareBuffer(size_t size, const std::string& label)
        : data_(new char[size]), size_(size), label_(label) {
        std::memset(data_, 0, size_);
        std::cout << "[Constructor] ECU buffer '" << label_
                  << "' allocated (" << size_ << " bytes)" << std::endl;
    }

    // ─── 1. Destructor ──────────────────────────────────────────────────────
    // EN: [1/5] Destructor — releases the heap-allocated firmware buffer
    // TR: [1/5] Yıkıcı — heap'te ayrılan firmware tamponunu serbest bırakır
    ~ECUFirmwareBuffer() {
        if (data_) {
            std::cout << "[Destructor] Deleting ECU buffer '" << label_
                      << "' (" << size_ << " bytes freed)" << std::endl;
            delete[] data_;
        } else {
            std::cout << "[Destructor] ECU buffer '" << label_
                      << "' was moved-from, nothing to free" << std::endl;
        }
    }

    // ─── 2. Copy Constructor (Deep Copy) ────────────────────────────────────
    // EN: [2/5] Copy Constructor — performs a deep copy of the firmware buffer
    // TR: [2/5] Kopya Yapıcı — firmware tamponunun derin kopyasını yapar
    ECUFirmwareBuffer(const ECUFirmwareBuffer& other)
        : data_(new char[other.size_]), size_(other.size_), label_(other.label_ + "_copy") {
        std::memcpy(data_, other.data_, size_);
        std::cout << "[Copy Constructor] Deep copied '" << other.label_
                  << "' -> '" << label_ << "' (" << size_ << " bytes)" << std::endl;
    }

    // ─── 3. Copy Assignment Operator (Copy-and-Swap Idiom) ──────────────────
    // EN: [3/5] Copy Assignment — uses copy-and-swap for exception safety
    // TR: [3/5] Kopya Atama — istisna güvenliği için copy-and-swap kullanır
    ECUFirmwareBuffer& operator=(const ECUFirmwareBuffer& other) {
        if (this == &other) return *this;
        std::cout << "[Copy Assignment] Overwriting '" << label_
                  << "' with '" << other.label_ << "'" << std::endl;

        // EN: Create a temporary deep copy, then swap resources
        // TR: Geçici derin kopya oluştur, ardından kaynakları takas et
        ECUFirmwareBuffer temp(other);
        std::swap(data_, temp.data_);
        std::swap(size_, temp.size_);
        std::swap(label_, temp.label_);
        return *this;
    }

    // ─── 4. Move Constructor (C++11) ────────────────────────────────────────
    // EN: [4/5] Move Constructor — steals resources from source, sets it to null
    // TR: [4/5] Taşıma Yapıcı — kaynağın kaynaklarını çalar, kaynağı null yapar
    // EN: noexcept is CRITICAL: without it, std::vector refuses to move and falls back to slow
    // deep copies during reallocation!
    // TR: noexcept KRİTİKTİR: olmadan std::vector taşıma yapmayı reddeder ve yeniden tahsiste
    // yavaş derin kopyalara döner!
    ECUFirmwareBuffer(ECUFirmwareBuffer&& other) noexcept
        : data_(other.data_), size_(other.size_), label_(std::move(other.label_)) {
        std::cout << "[Move Constructor] Moved resources from '" << label_
                  << "' (" << size_ << " bytes stolen)" << std::endl;
        other.data_ = nullptr;
        other.size_ = 0;
    }
```

```cpp

    // ─── 5. Move Assignment Operator (C++11) ──────────────────────────────────────────────────
    // EN: [5/5] Move Assignment — swaps/steals resources from source
    // TR: [5/5] Taşıma Atama — kaynağın kaynaklarını takas eder/çalar
    ECUFirmwareBuffer& operator=(ECUFirmwareBuffer&& other) noexcept {
        std::cout << "[Move Assignment] Moving '" << other.label_
                  << "' into '" << label_ << "'" << std::endl;
        if (this != &other) {
            // EN: Release our current resources first
            // TR: Önce mevcut kaynaklarımızı serbest bırak
            delete[] data_;

            // EN: Steal resources from the source
            // TR: Kaynaklarını kaynaktan çal
            data_ = other.data_;
            size_ = other.size_;
            label_ = std::move(other.label_);

            // EN: Nullify the source so its destructor is safe
            // TR: Kaynağı null yap ki yıkıcısı güvenli olsun
            other.data_ = nullptr;
            other.size_ = 0;
        }
        return *this;
    }

    // EN: Print current buffer info
    // TR: Mevcut tampon bilgisini yazdır
    void print() const {
        std::cout << "ECU Buffer Label: " << label_
                  << " | Size: " << size_
                  << " | Data ptr: " << static_cast<const void*>(data_) << std::endl;
    }
};

// EN: Factory function — may trigger RVO (Return Value Optimization) or move
// TR: Fabrika fonksiyonu — RVO (Dönüş Değeri Optimizasyonu) veya taşıma tetikler
ECUFirmwareBuffer createECUBuffer(size_t size, const std::string& label) {
    ECUFirmwareBuffer buf(size, label);
    return buf;
}

int main() {
  std::cout << "=== MODULE 2: RULE OF 3 / 5 AND MEMORY MANAGEMENT ===\n" << std::endl;

  StringBox originalFile("Top_Secret_Password");
  originalFile.print();

  // EN: Copy Constructor is invoked here! If we hadn't written Rule of 3 and the compiler's
  // default Shallow Copy ran, 'copyFile' would only steal the pointer (address) of 'Top_Secret'.
  // When main() exits and 'originalFile' is destroyed, that address becomes invalid. Then
  // 'copyFile' tries to delete the SAME address again (Double Free) and OS assassinates the
  // program with SIGSEGV!
  // TR: Copy Constructor burada cagrildi!
  StringBox copyFile = originalFile;

  std::cout << "----------------------" << std::endl;
  copyFile.print();
```

```cpp

    // EN: Both are safely destroyed as separate physical memory blocks.
    // TR: Ikisi de ayri fiziksel bellek bloklari olarak guvenle yikildi.

    // ═══════════════════════════════════════════════════════════════════════════════════
    // EN: RULE OF 5 — ECUFirmwareBuffer Demonstrations
    // TR: 5'Lİ KURAL — ECUFirmwareBuffer Gösterimleri
    // ═══════════════════════════════════════════════════════════════════════════════════

    std::cout << "\n\n=== RULE OF 5: ECU FIRMWARE BUFFER DEMOS ===\n" << std::endl;

    // ─── 1. Move Construction ──────────────────────────────────────────────────────
    // EN: The Move Constructor is invoked when initializing from a temporary.
    // TR: Geçici bir nesneden başlatılırken Taşıma Yapıcı çağrılır.
    std::cout << "--- Demo: Rule of 5 — Move Construction ---" << std::endl;
    {
        ECUFirmwareBuffer engineECU(1024, "EngineECU_v3.2");
        engineECU.print();

        // EN: Move constructor steals engineECU's guts — engineECU becomes empty
        // TR: Taşıma yapıcı engineECU'nun içini çalar — engineECU boş kalır
        ECUFirmwareBuffer transferredECU(std::move(engineECU));
        std::cout << "After move construction:" << std::endl;
        transferredECU.print();
        engineECU.print();
    }
    std::cout << std::endl;

    // ─── 2. Move Assignment ────────────────────────────────────────────────────────
    // EN: Move Assignment transfers ownership between two existing objects.
    // TR: Taşıma Atama, iki mevcut nesne arasında sahipliği aktarır.
    std::cout << "--- Demo: Rule of 5 — Move Assignment ---" << std::endl;
    {
        ECUFirmwareBuffer absModule(512, "ABS_Controller_v1.0");
        ECUFirmwareBuffer escModule(256, "ESC_Module_v2.1");
        absModule.print();
        escModule.print();

        // EN: Move assignment — escModule steals absModule's resources
        // TR: Taşıma atama — escModule, absModule'un kaynaklarını çalar
        escModule = std::move(absModule);
        std::cout << "After move assignment:" << std::endl;
        escModule.print();
        absModule.print();
    }
    std::cout << std::endl;

    // ─── 3. std::move Explicit Transfer ────────────────────────────────────────────
    // EN: std::move does NOT move anything — it merely casts an lvalue to an rvalue reference,
    // enabling the move constructor/assignment to be called.
    // TR: std::move hiçbir şeyi taşımaz — yalnızca bir lvalue'yu rvalue referansına dönüştürerek
    // taşıma yapıcının/atamanın çağrılmasını sağlar.
    std::cout << "--- Demo: Rule of 5 — std::move Explicit Transfer ---" << std::endl;
    {
        ECUFirmwareBuffer transmissionECU(2048, "Transmission_TCU_v4.0");
        transmissionECU.print();

        // EN: After std::move, transmissionECU is in a valid-but-unspecified state
        // TR: std::move sonrası transmissionECU geçerli-ama-belirtilmemiş durumda
        ECUFirmwareBuffer newOwner = std::move(transmissionECU);
        std::cout << "New owner after explicit std::move:" << std::endl;
        newOwner.print();
```

```

        // EN: transmissionECU is now hollow — data_ is nullptr
        // TR: transmissionECU artık boş — data_ nullptr
        std::cout << "Original after std::move (moved-from state):" << std::endl;
        transmissionECU.print();
    }
    std::cout << std::endl;

    // ─── 4. Factory Function (RVO / Move) ──────────────────────────────────────────────────────
    // EN: When returning a local object, the compiler may apply RVO (Return Value Optimization)
    // and construct directly in the caller's space. If RVO is not applied, the move constructor is
    // used instead of a deep copy.
    // TR: Yerel bir nesne döndürülürken, derleyici RVO (Dönüş Değeri Optimizasyonu) uygulayabilir
    // ve doğrudan çağıranın alanında oluşturur. RVO uygulanmazsa, derin kopya yerine taşıma yapıcı
    // kullanılır.
    std::cout << "--- Demo: Rule of 5 — Factory Function (RVO/Move) ---" << std::endl;
    {
        ECUFirmwareBuffer airbagECU = createECUBuffer(4096, "Airbag_SRS_v5.1");
        std::cout << "Factory-created buffer:" << std::endl;
        airbagECU.print();
    }

    return 0;
}
```

► 04. Multiple Inheritance

module_02_oop/04_multiple_inheritance.cpp

Konu: OOP Part 4: Multiple Inheritance & The Diamond Problem / Çoklu Kalıtım ve Elmas

Teori / Theory

THEORY: Why is Multiple Inheritance Banned in Many Languages?

C++ is one of the very few languages that allows a class to have TWO or more Base classes. `class FlyingCar : public Car, public Airplane`. This sounds cool but causes a deadly issue known as the "Diamond Problem". Java and C# banned this entirely, relying on Interfaces instead.

TEORİ: Çoklu Kalıtım Neden Yasaktır?

C++, bir sınıfın AYNI ANDA İKİ farklı temel sınıftan (Base) miras almasına olanak tanıyan nadir dillerdendir. Örneğin Uçan Arabamız (FlyingCar) var; hem KaradaGiden hem de HavadaUcan sınıflarından kalıtım alır. Ancak bu harika görünen özellik, "Elmas Problemi (Diamond Problem)" denen felaketi doğurur. Java veya C# bunu tamamen yasaklamış, yerine `Interface` (Arayüz) zorunluluğu getirmiştir.

THEORY: The Diamond Problem & Virtual Inheritance

Imagine: A Base class 'Device' exists (with an int 'power'). - 'Printer' inherits from Device. - 'Scanner' inherits from Device. - 'Copier' inherits from BOTH Printer and Scanner! Result: 'Copier' now contains TWO sets of 'power'! A waste of RAM and ambiguous. C++ solves this using `virtual public` inheritance: "Only make 1 copy of Device."

TEORİ: Elmas Problemi ve Virtual Kalıtım

Düşünün: 'Alet' (Device) adında bir Ana sınıf var (içinde 'int power' eklentisi olsun). - 'Yazici' (Printer), Alet'ten miras alır. - 'Tarayici' (Scanner), Alet'ten miras alır. - 'HepsiBirArada' (Copier), hem Yazici'dan hem Tarayici'dan miras alır!
Sonuç: HepsiBirArada nesnesinin içinde İKİ ADET 'power' değişkeni oluşur! Bir RAM israfı ve Belirsizlik! C++ bunu çözmek için `virtual public` kalıtımını icat etti. "Güç değişkeninden sadece 1 adet kopyala."

CPPREF Depth: The Diamond Problem and Virtual Base Classes — Memory Layout

With normal (non-virtual) multiple inheritance, each path to a shared base class creates a separate sub-object. If both Printer and Scanner inherit from Device without `virtual`, Copier contains TWO Device sub-objects — sizeof(Copier) includes both. Virtual inheritance changes the ABI layout fundamentally. Instead of embedding the base sub-object directly, each virtual-inheriting class stores a VBASE OFFSET (or vbase pointer) inside its vtable. This offset tells the runtime where the single shared virtual base resides relative to the current sub-object. The shared Device is placed at the END of the most-derived object's memory layout. Construction order with virtual bases: ALL virtual bases are constructed FIRST, in left-to-right, depth-first order — and only by the MOST DERIVED class's constructor (not intermediate classes). Then non-virtual bases are constructed left-to-right, then the class itself. sizeof() increases with virtual inheritance due to the hidden vptr or vbase-offset pointers added by the compiler. On a typical 64-bit ABI, each virtual inheritance level adds 8 bytes per sub-object for the vptr. This is why `dynamic_cast` across virtual bases is more expensive — it must traverse offsets stored in the vtable to locate the target.

CPPREF Derinlik: Elmas Problemi ve Sanal Temel Sınıflar — Bellek Düzeni

Normal (sanal olmayan) çoklu kalıtımda, paylaşılan bir temel sınıfa giden her yol ayrı bir alt nesne oluşturur. Printer ve Scanner, Device'tan `virtual` olmadan miras alırsa, Copier İKİ Device alt nesnesi içerir. Sanal kalıtım ABI düzenini temelden değiştirir. Temel alt nesneyi doğrudan gömmek yerine, sanal miras alan her sınıf vtable'ında bir VBASE OFFSET (sanal taban ofseti) saklar. Bu ofset, paylaşılan sanal tabanın geçerli alt nesneye göre nerede bulunduğunu çalışma zamanında belirtir. Paylaşılan Device, en türetilmiş nesnenin bellek düzeninin SONUNA yerleştirilir. Sanal tabanlarda yapılandırma sırası: TÜM sanal tabanlar İLK ÖNCE, soldan sağa ve derinlik öncelikli sırada — yalnızca EN TÜRETİLMİŞ sınıfın yapıcısı tarafından oluşturulur. Sonra sanal olmayan tabanlar soldan sağa yapılandırılır, ardından sınıfın kendisi. sizeof() sanal kalıtımla artar çünkü derleyici gizli vptr veya vbase-ofset işaretçileri ekler. Tipik bir 64-bit ABI'de her sanal kalıtım seviyesi alt nesne başına 8 bayt ekler.
→ Ref: https://en.cppreference.com/w/cpp/language/derived_class

Kaynak Kod / Source Code

```cpp
#include <iostream>

// EN: 1. TOP LEVEL ROOT CLASS
// TR: 1. KÖK SINIF
class Device {
public:
  int powerConsumption;
  Device() : powerConsumption(100) {
    std::cout << "[Device] Root created. / Kök üretildi." << std::endl;
  }
};

// EN: 2. SUBCLASS 1 (Virtual Inheritance!)
// TR: 2. ALT SINIF 1 (Virtual Kalıtım!)
// EN: If we do not say 'virtual' here, the final child class will copy 'powerConsumption' twice!
// TR: Eğer 'virtual' demezsek, en aşağıdaki sentez cihaz kendi içinde 'powerConsumption'dan İKİ
// KOPYA üretir.
class Printer : virtual public Device {
public:
  Printer() {
    std::cout << "[Printer] Print capability added. / Yazdırma yeteneği eklendi." << std::endl;
  }
  void print() { std::cout << "Printing... / Baskı yapılıyor..." << std::endl; }
};
```

```

// EN: 3. SUBCLASS 2 (Virtual Inheritance!)
// TR: 3. ALT SINIF 2 (Virtual Kalıtım!)
class Scanner : virtual public Device {
public:
  Scanner() {
    std::cout << "[Scanner] Scan capability added. / Tarama yeteneği eklendi." << std::endl;
  }
  void scan() { std::cout << "Scanning... / Belge taranıyor..." << std::endl; }
};

// EN: 4. THE NIGHTMARE: MULTIPLE INHERITANCE
// TR: 4. KABUS: ÇOKLU KALITIM
class AllInOneCopier : public Printer, public Scanner {
public:
  AllInOneCopier() {
    std::cout << "[AllInOneCopier] Synthesis ready! / Sentez cihaz hazır!" << std::endl;
  }

  void showPower() {
    // EN: If we hadn't used 'virtual' inheritance above, 'powerConsumption' would be AMBIGUOUS
    // and the code WOULD NOT COMPILE. (It wouldn't know which 'power' to access).
    //
    // TR: Eğer yukarıda 'virtual' kullanmasaydık, 'powerConsumption' AMBIGUOUS (Belirsiz)
    // olacaktı ve kod DERLENMEYECEKTİ (Hangisinin 'power'ına ulaşmak istediğimizi
    // bilemeyecekti).
    std::cout << "Power Consumption (Güç Tüketimi): " << powerConsumption <<
        " Watt (Diamond Problem Solved / Problem çözüldü!)" << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 2: MULTIPLE INHERITANCE (DIAMOND PROBLEM) ===\n" << std::endl;

  AllInOneCopier machine;

  std::cout << "\nMachine Tests / Makine Testleri:" << std::endl;
  machine.print();
  machine.scan();
  machine.showPower();

  return 0;
}
```

► 05. Encapsulation Abstraction

module_02_oop/05_encapsulation_abstraction.cpp

Konu: OOP Part 5: Encapsulation & Abstraction / OOP Kısım 5: Kapsülleme ve Soyutlama

Teori / Theory

THEORY: The 4 Pillars of OOP

1. Inheritance -> Prevents code duplication (DRY). 2. Polymorphism -> Provides dynamic flexibility through the V-Table. 3. Encapsulation -> Data hiding and security shield. 4. Abstraction -> Hides the "How" and only shows the "What".

TEORİ: 4 Büyük OOP Sütunu

1. Kalıtım (Inheritance) -> Kod tekrarını önler (DRY). 2. Çok Biçimlilik (Polymorphism) -> V-Table ile dinamik esneklik sağlar. 3. Kapsülleme (Encapsulation) -> Veri gizleme ve güvenlik kalkanıdır. 4. Soyutlama (Abstraction) -> Sadece "Neyin" yapılacağını gösterir, "Nasıl" yapıldığını gizler.

THEORY: Why is Encapsulation INDISPENSABLE?

Imagine a 'BankAccount' class. If the 'balance' variable is public, ANYONE from anywhere in the code can do: `account.balance = 1000000;`. This breaks reality! Money should only change through `deposit()` or `withdraw()`. Encapsulation hides the data (`private`) and exposes safety checks (`public` methods).

TEORİ: Kapsülleme Neden Vazgeçilmezdir?

Bir 'BankaHesabi' sınıfı düşünün. Eğer 'bakiye' değişkeni 'public' (açık) olursa, kodun herhangi bir yerinden herhangi biri şunu yazabilir: `hesap.bakiye = 1000000;`. Mevcut gerçekliği yıkar! Para SADECE yasal yollarla `paraYatir()` veya `paraCek()` fonksiyonları üzerinden değişmelidir. Kapsülleme, veriyi (`private`) gizler ve dünyaya sadece bizim yazdığımız güvenlik fonksiyonlarını (`public`) sunar (Getter ve Setter mantığı).

THEORY: What is Abstraction?

When you drive a car, do you need to know the piston angles or spark plug timings? No. You just press the gas pedal (Interface). That's Abstraction. "Hide the complex internal mechanism, expose a simple interface." "Interface Classes" created with Pure Virtual Functions (= 0) are the peak of this.

TEORİ: Soyutlama Nedir?

Bir araba kullanırken "Motorun piston açılarını" veya "Bujilerin ateşleme milisaniyesini" bilmek zorunda mısınız? Hayır. Sadece gaza (Interface/Arayüz) basarsınız. Abstraction (Soyutlama) budur. "Karmaşık iç mekanizmayı gizle, sadece kullanımı basit olan dış yüzü göster." Saf Sanal Fonksiyonlar (= 0) ile yaratılan "Interface Sınıflar" bunun zirvesidir.

CPPREF Depth: Access Specifiers and Friend — Compiler Enforcement vs Runtime

Access specifiers (`public`, `private`, `protected`) are COMPILE-TIME ONLY checks. The compiler rejects `obj.privateVar` in source code, but at runtime ALL memory is flat and accessible — a raw pointer cast can read any byte. This is by design: C++ trusts the programmer after compilation. Security comes from the type system, not from memory hiding. `friend` is not a design flaw — it is a CONTROLLED GATE. When two classes are tightly coupled by design (e.g., an iterator and its container, or `operator<<` for a class), `friend` grants precise access without exposing internals to the entire world via public getters. The PIMPL (Pointer to IMPLementation) idiom achieves ABI-stable encapsulation: the public header contains only a forward-declared Impl struct and a `std::unique_ptr<Impl>`. Changing Impl's members does NOT change the public header's sizeof — downstream code need not recompile. This is critical in shared libraries and automotive ECU software where ABI breaks cause fleet-wide update cascades. Note: `protected` is weaker than `private` — any derived class in any translation unit can access protected members, making it a wider trust boundary than many developers realize.

CPPREF Derinlik: Erişim Belirteçleri ve Friend — Derleyici Zorlama vs Çalışma Zamanı

Erişim belirteçleri (`public`, `private`, `protected`) YALNIZCA DERLEME ZAMANI kontrolleridir. Derleyici kaynak kodda `obj.privateVar` erişimini reddeder, ancak çalışma zamanında TÜM bellek düzdür ve erişilebilirdir — ham bir işaretçi dönüşümü herhangi bir baytı okuyabilir. Bu tasarım gereğidir: C++ derleme sonrasında programcıya güvenir. `friend` bir tasarım hatası değildir — KONTROLLÜ bir GEÇİTTİR. İki sınıf tasarım gereği sıkı bağlıysa (örn. yineleyici ve konteyneri veya bir sınıf için `operator<<`), `friend` tüm dünyaya public getter'lar ile iç yapıyı açmadan kesin erişim sağlar. PIMPL deyimi ABI-kararlı kapsülleme sağlar: public başlık yalnızca ileri bildirilmiş bir Impl yapısı ve `std::unique_ptr<Impl>` içerir. Impl'in üyelerini değiştirmek public başlığın sizeof değerini DEĞİŞTİRMEZ — bağımlı kodun yeniden derlenmesi gerekmez. Not: `protected`, `private`'den daha zayıftır — herhangi bir çeviri birimindeki türetilmiş sınıf protected üyelere erişebilir.
→ Ref: https://en.cppreference.com/w/cpp/language/access

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

// EN: --- ENCAPSULATION EXAMPLE ---
// TR: --- KAPSÜLLEME ÖRNEĞİ ---
class BankAccount {
private: // DATA HIDING / VERİ GİZLEME
  std::string ownerName;
  double balance;

  // EN: Internal private helper function. CANNOT be called from outside. (Abstraction)
  // TR: Özel bir yardımcı fonksiyondur. Dışarıdan ÇAĞIRILAMAZ. (Soyutlama içerir)
  bool isTransactionValid(double amount) const { return amount > 0; }

public:
  BankAccount(std::string name, double initialDeposit)
      : ownerName(name), balance(initialDeposit) {}
```

```cpp

  // EN: GETTER method retrieving the encapsulated real data (Read-Only access):
  // TR: Kapsüllenen veriyi dışarı (sadece OKUMAK için) çıkaran GETTER metodu:
  double getBalance() const {
    // [Look, but don't touch! / Göster, ama dokundurma!]
    return balance;
  }

  // EN: SETTER method modifying the encapsulated data through rules:
  // TR: Kapsüllenen veriyi kurallarıyla değiştiren SETTER / İşlem metodu:
  void deposit(double amount) {
    if (isTransactionValid(amount)) {
      balance += amount;
      std::cout << "[Encapsulation] Safe deposit successful. Amount: " << amount <<
          " / Güvenli para yatırma işlemi başarılı." << std::endl;
    } else {
      std::cout << "[ERROR/HATA] Cannot deposit zero or negative funds! (Sıfır "
                    "veya eksi yatırılamaz!)"
                << std::endl;
    }
  }
};

// --- ABSTRACTION EXAMPLE / SOYUTLAMA ÖRNEĞİ ---

/**
 * @class IEspressoMachine
 * @brief EN: 'I' prefix indicates it is an Interface.
 * TR: 'I' ön eki Interface (Arayüz) olduğunu belirtir.
 *
 * EN: Contains a "Pure Virtual" function (= 0). This class has NO body. You CANNOT create an
 * object like 'new IEspressoMachine()'. It is just a "CONTRACT".
 *
 * TR: Saf Sanal Fonksiyon (Pure Virtual) içerir. Yani bu sınıfın BEDENİ yoktur. Ondan 'new
 * IEspressoMachine()' diye nesne ÜRETEMEZSİNİZ. O sadece bir "TAAHHÜTNAME"dir.
 */
class IEspressoMachine {
public:
  virtual ~IEspressoMachine() = default;

  // EN: '= 0' means just show the rule (interface), no body!
  // TR: '= 0' demek SADECE kuralı (arayüzü) göster, gövdesi yok demektir!
  virtual void makeCoffee() = 0;
};

// EN: The concrete class implementing the real mechanism (the 'How' details).
// TR: Gerçek mekanizmayı uygulayan (Nasıl çalıştığını yazan) Sınıf.
class ModernCoffeeMaker : public IEspressoMachine {
private:
  void grindBeans() {
    std::cout << "1. Grinding beans (0.3mm)... / Çekirdekler öğütülüyor..." << std::endl;
  }
  void boilWater() {
    std::cout << "2. Boiling water to 94C... / Su 94 dereceye isitiliyor..." << std::endl;
  }
  void filterEspresso() {
    std::cout << "3. Filtering with 9 Bars... / 9 Bar basınç ile süzülüyor!" << std::endl;
  }

public:
  void makeCoffee() override {
    // EN: The Core of Abstraction! User ONLY calls makeCoffee, they don't see the complex 3
    // steps.
    // TR: Soyutlamanın Çekirdeği! Kullanıcı SADECE makeCoffee çağırır, şu 3 karmaşık adımı
    // BİLMEZ/GÖRMEZ.
    grindBeans();
    boilWater();
    filterEspresso();
    std::cout << "[Result / Sonuç] Your espresso is ready! (Espresso hazır!)\n" << std::endl;
  }
};
```

```

int main() {
  std::cout << "=== MODULE 2: ENCAPSULATION & ABSTRACTION ===\n" << std::endl;

  // EN: 1. Encapsulation Test
  // TR: 1. Kapsülleme Testi
  std::cout << "--- 1. ENCAPSULATION (Kapsülleme) ---" << std::endl;
  BankAccount myAccount("Alice", 1000.0);

  // myAccount.balance = 9999999; // ERROR! private veriye asla dokunamazsın.
  std::cout << "Current Balance (Mevcut Para): " << myAccount.getBalance() << std::endl;

  // EN: Security shield engages!
  // TR: Sistem güvenlik kalkanını devreye sokar!
  myAccount.deposit(-500);

  // 2. Abstraction Test (Soyutlama Testi)
  std::cout << "\n--- 2. ABSTRACTION (Soyutlama) ---" << std::endl;

  // EN: We use the coffee machine through the IEspressoMachine interface. We don't worry about
  // pistons, pressure, or temp. We just press the button.
  // TR: IEspressoMachine arayüzü ile makineyi kullanırız. Pistonlar veya sıcaklık bizi
  // ilgilendirmez. Sadece düğmeye basarız.

  IEspressoMachine *myMachine = new ModernCoffeeMaker();
  myMachine->makeCoffee();

  delete myMachine;

  return 0;
}
```

► 06. Static And Friends

module_02_oop/06_static_and_friends.cpp

Konu: OOP Part 6: Static Members & Friend Functions / OOP Kısım 6: Statik Üyeler ve Friend

Teori / Theory

THEORY: Static Class Members

A `static` data member belongs to the CLASS itself, not to any individual object. All instances share the same copy. Think of a global configuration register inside an ECU — every software module reads and writes the same register, not a per-module copy. Declaring `static int errorCount;` in a class means there is exactly ONE `errorCount` for every object ever created. Static members must be defined (allocated) outside the class body unless they are `inline` or `constexpr`.

TEORİ: Statik Sınıf Üyeleri

Bir `static` veri üyesi, herhangi bir nesneye değil SINIFIN kendisine aittir. Tüm örnekler aynı kopyayı paylaşır. Bir ECU içindeki global konfigürasyon kaydını düşünün — her yazılım modülü aynı kaydı okur ve yazar, her modülün ayrı kopyası yoktur. Sınıf içinde `static int errorCount;` tanımladığınızda, oluşturulan tüm nesneler için TAM OLARAK BİR `errorCount` vardır. Statik üyeler, `inline` veya `constexpr` olmadıkça sınıf gövdesinin dışında tanımlanmalıdır (bellek ayrılmalıdır).

THEORY: Static Member Functions

A `static` member function can be called WITHOUT creating an object: `VehicleTelemetry::getFleetCount()`. It has NO `this` pointer, so it can only access other static members. Use it for factory methods, counters, or utility operations that logically belong to the class but need no instance.

TEORİ: Statik Üye Fonksiyonlar

Bir `static` üye fonksiyon, nesne oluşturmadan çağrılabilir: `VehicleTelemetry::getFleetCount()`. `this` işaretçisi YOKTUR, bu yüzden yalnızca diğer statik üyelere erişebilir. Fabrika metotları, sayaçlar veya mantıksal olarak sınıfa ait olan ancak bir örneğe ihtiyaç duymayan yardımcı işlemler için kullanılır.

THEORY: Friend Functions

A `friend` function is declared inside a class but is NOT a member. It can access all private and protected members. Think of a diagnostic tool that plugs into an ECU's OBD-II port — it needs direct access to private sensor readings and DTC logs that normal external code cannot see. The class explicitly grants trust by declaring `friend void diagnosticDump(...)`.

TEORİ: Friend (Arkadaş) Fonksiyonlar

Bir `friend` fonksiyon sınıf içinde bildirilir ancak üye DEĞİLDİR. Tüm private ve protected üyelere erişebilir. Bir ECU'nun OBD-II portuna takılan bir teşhis aracını düşünün — normal harici kodun göremeyeceği özel sensör okumalarına ve DTC kayıtlarına doğrudan erişmesi gerekir. Sınıf, `friend void diagnosticDump(...)` bildirerek güveni açıkça verir.

THEORY: Friend Classes

When you write `friend class CalibrationTool;` inside `Engine`, EVERY member function of `CalibrationTool` can access `Engine`'s private data. This is like a factory calibration station that must tune private engine parameters (fuel maps, timing offsets) — access that regular drivers never have. Friendship is NOT symmetric: Engine trusts CalibrationTool, but NOT vice versa. Friendship is NOT inherited.

TEORİ: Friend (Arkadaş) Sınıflar

`Engine` içinde `friend class CalibrationTool;` yazdığınızda, `CalibrationTool`'un TÜM üye fonksiyonları `Engine`'in private verilerine erişebilir. Bu, özel motor parametrelerini (yakıt haritaları, zamanlama ofsetleri) ayarlaması gereken bir fabrika kalibrasyon istasyonu gibidir — normal sürücülerin asla sahip olmadığı bir erişim. Arkadaşlık SİMETRİK DEĞİLDİR: Engine CalibrationTool'a güvenir, ama tersi geçerli değildir. Arkadaşlık miras ALINMAZ.

THEORY: When to Use

- USE `friend` when two classes are tightly coupled BY DESIGN (e.g., an iterator and its container, a serializer and its data class). - DO NOT use `friend` to bypass poor encapsulation. If you need many friends, your design is probably wrong — add proper public/protected API. - `static` is great for counters, caches, and singletons. Beware of initialization order across translation units (the "static init fiasco").

Abuse Warning / TEORİ: Ne Zaman Kullanılır / Kötüye Kullanım Uyarısı

- `friend` kullanın: İki sınıf TASARIM GEREĞİ sıkı bağlıysa (örn. bir yineleyici ve konteyneri, bir serileştirici ve veri sınıfı). - `friend` KULLANMAYIN: Zayıf kapsüllemeyi atlatmak için. Çok fazla friend'e ihtiyacınız varsa tasarımınız muhtemelen yanlıştır — uygun public/protected API ekleyin. - `static`, sayaçlar, önbellekler ve tekil nesneler (singleton) için harikadır. Çeviri birimleri arasında başlatma sırası konusunda dikkatli olun ("static init fiasco" sorunu).

CPPREF Depth: Static Members — Linkage, Initialization Order, and Thread Safety

Static class data members have EXTERNAL LINKAGE by default — the linker must find exactly one definition across all translation units (ODR). If you declare `static int count;` in a header included by 3 .cpp files without `inline`, you get a linker error: multiple definitions. The "Static Initialization Order Fiasco": globals and namespace-scope statics across different TUs have NO GUARANTEED construction order. If static A in file1.cpp depends on static B in file2.cpp, B might not be initialized yet → undefined behavior. The classic fix: use a local static inside a function ("Construct On First Use" idiom). C++11 §6.7 guarantees that local static variables are initialized in a THREAD-SAFE manner — the compiler inserts a hidden guard variable and mutex-like mechanism (often a spinlock or CAS). This makes the Meyer's Singleton pattern safe: `static T& instance() { static T t; return t; }` C++17 introduced `inline static` members, allowing in-class definition of static data members without a separate .cpp definition. This also guarantees a single definition across TUs — the linker deduplicates. `constexpr static` members are implicitly inline since C++17.

CPPREF Derinlik: Statik Üyeler — Bağlantı, Başlatma Sırası ve İş Parçacığı Güvenliği

Statik sınıf veri üyeleri varsayılan olarak DIŞ BAĞLANTIYA sahiptir — bağlayıcı tüm çeviri birimleri arasında tam olarak bir tanım bulmalıdır (ODR). Bir başlıkta `static int count;` bildirip 3 .cpp dosyasına dahil ederseniz `inline` olmadan bağlayıcı hatası alırsınız. "Statik Başlatma Sırası Felaketi": farklı çeviri birimlerindeki global ve namespace kapsamlı statikler GUARANTEED yapılandırma sırasına sahip DEĞİLDİR. Dosya1'deki statik A, dosya2'deki statik B'ye bağlıysa, B henüz başlatılmamış olabilir → tanımsız davranış. Klasik çözüm: fonksiyon içi yerel statik kullanmak ("İlk Kullanımda Oluştur" deyimi). C++11 §6.7, yerel statik değişkenlerin İŞ PARÇACIĞI GÜVENLİ biçimde başlatılacağını garanti eder — derleyici gizli bir koruma değişkeni ve mutex benzeri mekanizma ekler. Bu, Meyer's Singleton kalıbını güvenli kılar: `static T& instance() { static T t; return t; }` C++17, `inline static` üyeleri tanıttı ve sınıf içi tanıma izin verdi. `constexpr static` üyeler C++17'den beri örtük olarak inline'dır.
→ Ref: https://en.cppreference.com/w/cpp/language/static

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for VIN and identifiers
#include <cassert>    // EN: Runtime assertions for fleet-size guard

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: VehicleTelemetry — static members & friend function
// ═══════════════════════════════════════════════════════════════════════════

// EN: Forward declaration of friend function so the class can reference it.
// TR: Sınıfın referans verebilmesi için friend fonksiyonun önceden bildirimi.
class VehicleTelemetry;
void diagnosticDump(const VehicleTelemetry& vt);

class VehicleTelemetry {
public:
    // EN: Static constexpr — compile-time constant shared by all instances.
    // TR: Static constexpr — tüm örnekler tarafından paylaşılan derleme zamanı sabiti.
    static constexpr int MAX_FLEET_SIZE = 100;

    // EN: Constructor registers vehicle into the fleet.
    // TR: Yapıcı, aracı filoya kaydeder.
    explicit VehicleTelemetry(std::string vin, double speed = 0.0)
        : vin_(std::move(vin)), lastSpeed_(speed) {
        assert(totalVehiclesTracked_ < MAX_FLEET_SIZE &&
               "Fleet capacity exceeded!");
        ++totalVehiclesTracked_;
    }
```

```cpp

    // EN: Destructor unregisters vehicle from the fleet.
    // TR: Yıkıcı, aracı filodan çıkarır.
    ~VehicleTelemetry() {
        --totalVehiclesTracked_;
    }

    // EN: Static member function — callable without an object.
    // TR: Statik üye fonksiyon — nesne olmadan çağrılabilir.
    static int getFleetCount() {
        return totalVehiclesTracked_;
    }

    // EN: Regular member to update speed telemetry.
    // TR: Hız telemetrisini güncellemek için normal üye fonksiyon.
    void updateSpeed(double newSpeed) {
        lastSpeed_ = newSpeed;
    }

    // EN: Grant friend access to the free function diagnosticDump.
    // TR: diagnosticDump serbest fonksiyonuna friend erişimi ver.
    friend void diagnosticDump(const VehicleTelemetry& vt);

private:
    // EN: Shared counter — one copy for all VehicleTelemetry objects.
    // TR: Paylaşılan sayaç — tüm VehicleTelemetry nesneleri için tek kopya.
    static int totalVehiclesTracked_;

    // EN: Per-instance private data: VIN and last recorded speed (km/h).
    // TR: Örneğe özel private veri: VIN ve son kaydedilen hız (km/sa).
    std::string vin_;
    double lastSpeed_;
};

// EN: Out-of-class definition of the static member (required for non-inline statics).
// TR: Statik üyenin sınıf dışı tanımı (inline olmayan statikler için gereklidir).
int VehicleTelemetry::totalVehiclesTracked_ = 0;

// ─── 1.1 Friend Function Implementation ──────────────────────────────────────────────

// EN: diagnosticDump can read private vin_ and lastSpeed_ because it is a friend.
// TR: diagnosticDump, friend olduğu için private vin_ ve lastSpeed_'e erişebilir.
void diagnosticDump(const VehicleTelemetry& vt) {
    std::cout << "[DIAG] VIN: " << vt.vin_
              << " | Speed: " << vt.lastSpeed_ << " km/h"
              << " | Fleet size: " << VehicleTelemetry::totalVehiclesTracked_
              << "\n";
}

// ═══════════════════════════════════════════════════════════════════════════════════
// SECTION 2: Engine + CalibrationTool — friend class
// ═══════════════════════════════════════════════════════════════════════════════════

// EN: Forward declaration so Engine can declare CalibrationTool as a friend.
// TR: Engine'in CalibrationTool'u friend olarak bildirebilmesi için önceden bildirim.
class CalibrationTool;

class Engine {
public:
    // EN: Construct with default factory parameters.
    // TR: Varsayılan fabrika parametreleriyle oluştur.
    explicit Engine(std::string ecuId)
        : ecuId_(std::move(ecuId)),
          fuelMapOffset_(0.0),
          timingAdvanceDeg_(12.0),
          boostPressureBar_(1.0) {}
```

```cpp

    // EN: Public read-only interface — safe for any caller.
    // TR: Genel salt-okunur arayüz — herhangi bir çağrıcı için güvenli.
    void printStatus() const {
        std::cout << "[ENGINE " << ecuId_ << "] "
                  << "FuelOffset=" << fuelMapOffset_
                  << " | Timing=" << timingAdvanceDeg_ << "deg"
                  << " | Boost=" << boostPressureBar_ << "bar\n";
    }

    // EN: Grant full private access to CalibrationTool.
    // TR: CalibrationTool'a tam private erişim ver.
    friend class CalibrationTool;

private:
    std::string ecuId_;
    double fuelMapOffset_;     // EN: fuel-map trim offset
    double timingAdvanceDeg_;  // EN: ignition timing in degrees
    double boostPressureBar_;  // EN: turbo boost pressure in bar
};

class CalibrationTool {
public:
    // EN: Adjust fuel map offset — accesses Engine's private member directly.
    // TR: Yakıt haritası ofsetini ayarla — Engine'in private üyesine doğrudan erişir.
    static void adjustFuelMap(Engine& engine, double offset) {
        std::cout << "[CAL] Adjusting fuel map offset from "
                  << engine.fuelMapOffset_ << " to " << offset << "\n";
        engine.fuelMapOffset_ = offset;
    }

    // EN: Adjust ignition timing — accesses Engine's private member directly.
    // TR: Ateşleme zamanlamasını ayarla — Engine'in private üyesine doğrudan erişir.
    static void adjustTiming(Engine& engine, double degrees) {
        std::cout << "[CAL] Adjusting timing advance from "
                  << engine.timingAdvanceDeg_ << " to " << degrees << "deg\n";
        engine.timingAdvanceDeg_ = degrees;
    }

    // EN: Adjust boost pressure — accesses Engine's private member directly.
    // TR: Turbo basıncını ayarla — Engine'in private üyesine doğrudan erişir.
    static void adjustBoost(Engine& engine, double bar) {
        std::cout << "[CAL] Adjusting boost pressure from "
                  << engine.boostPressureBar_ << " to " << bar << "bar\n";
        engine.boostPressureBar_ = bar;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== Module 02 | Lesson 06: Static Members & Friends ===\n\n";

    // ─── Demo 1: Static counter increments with each vehicle ──────────────────
    // EN: Each VehicleTelemetry construction bumps the shared counter.
    // TR: Her VehicleTelemetry oluşturulduğunda paylaşılan sayaç artar.
    std::cout << "--- Demo 1: Static counter ---\n";
    std::cout << "Fleet count before any vehicle: "
              << VehicleTelemetry::getFleetCount() << "\n";

    VehicleTelemetry v1("WBA3A5C55DF123456", 85.0);
    std::cout << "After v1: fleet count = "
              << VehicleTelemetry::getFleetCount() << "\n";

    VehicleTelemetry v2("WVWZZZ3CZWE987654", 120.5);
    std::cout << "After v2: fleet count = "
              << VehicleTelemetry::getFleetCount() << "\n";

    {
        VehicleTelemetry v3("TMBJB9NE1L0123456");
        std::cout << "After v3 (scoped): fleet count = "
                  << VehicleTelemetry::getFleetCount() << "\n";
    }
```

```
169     // EN: v3 destroyed when scope ends — counter decrements.
170     // TR: Kapsam sona erdiğinde v3 yok edilir — sayaç azalır.
171     std::cout << "After v3 destroyed: fleet count = "
172               << VehicleTelemetry::getFleetCount() << "\n\n";
173
174     // ─── Demo 2: Static method called without an object ──────────────────────────
175     // EN: No object required — call directly on the class.
176     // TR: Nesne gerekmez — doğrudan sınıf üzerinden çağrılır.
177     std::cout << "--- Demo 2: Static method without object ---\n";
178     std::cout << "VehicleTelemetry::getFleetCount() = "
179               << VehicleTelemetry::getFleetCount() << "\n\n";
180
181     // ─── Demo 3: Friend function accessing private data ──────────────────────────
182     // EN: diagnosticDump reads private vin_ and lastSpeed_ directly.
183     // TR: diagnosticDump, private vin_ ve lastSpeed_'i doğrudan okur.
184     std::cout << "--- Demo 3: Friend function ---\n";
185     v1.updateSpeed(92.3);
186     diagnosticDump(v1);
187     diagnosticDump(v2);
188     std::cout << "\n";
189
190     // ─── Demo 4: Friend class CalibrationTool modifying Engine ────────────────────
191     // EN: CalibrationTool accesses Engine's private internals to tune params.
192     // TR: CalibrationTool, parametreleri ayarlamak için Engine'in private içeriğine erişir.
193     std::cout << "--- Demo 4: Friend class CalibrationTool ---\n";
194     Engine myEngine("ECU-2026-TDI");
195     std::cout << "Before calibration:\n";
196     myEngine.printStatus();
197
198     CalibrationTool::adjustFuelMap(myEngine, 2.5);
199     CalibrationTool::adjustTiming(myEngine, 14.0);
200     CalibrationTool::adjustBoost(myEngine, 1.8);
201
202     std::cout << "After calibration:\n";
203     myEngine.printStatus();
204     std::cout << "\n";
205
206     // ─── Demo 5: constexpr static constant usage ──────────────────────────────────
207     // EN: MAX_FLEET_SIZE is a compile-time constant available without an instance.
208     // TR: MAX_FLEET_SIZE, bir örneğe ihtiyaç duymadan kullanılabilen derleme zamanı sabitidir.
209     std::cout << "--- Demo 5: constexpr static constant ---\n";
210     std::cout << "Maximum fleet capacity: "
211               << VehicleTelemetry::MAX_FLEET_SIZE << " vehicles\n";
212     std::cout << "Current fleet utilization: "
213               << VehicleTelemetry::getFleetCount() << " / "
214               << VehicleTelemetry::MAX_FLEET_SIZE << "\n\n";
215
216     std::cout << "=== All demos completed successfully. ===\n";
217     return 0;
218 }
```

► 07. Nested And Local Classes

module_02_oop/07_nested_and_local_classes.cpp

Konu: OOP Part 7: Nested & Local Classes / OOP Kısım 7: İç İçe ve Yerel Sınıflar

Teori / Theory

THEORY: Nested Classes

A nested class is a class defined inside another class. The inner class logically belongs to the outer class and is scoped within it. Think of a CAN message: the Header (arbitration ID, DLC, flags) only makes sense as part of a CanMessage. Defining Header inside CanMessage keeps it tightly coupled and prevents polluting the global namespace.

TEORİ: İç İçe Sınıflar (Nested Classes)

İç içe sınıf (nested class), başka bir sınıfın içinde tanımlanan sınıftır. İç sınıf, mantıksal olarak dış sınıfa aittir ve onun kapsamında yer alır. Bir CAN mesajı düşünün: Header (arbitrasyon ID, DLC, bayraklar) yalnızca bir CanMessage'ın parçası olarak anlam taşır. Header'ı CanMessage içinde tanımlamak, sıkı bağlılığı korur ve global isim alanını kirletmez.

THEORY: Access Rules

Since C++11, a nested class can access the private and protected members of its enclosing class (given an instance of the outer class). The outer class, however, has NO special access to the nested class's private members. This asymmetry is important for encapsulation.

TEORİ: Erişim Kuralları

C++11'den beri, iç içe sınıf, dış sınıfın private ve protected üyelerine erişebilir (dış sınıfın bir örneği verildiğinde). Ancak dış sınıfın, iç sınıfın private üyelerine özel bir erişimi YOKTUR. Bu asimetri, kapsülleme açısından önemlidir.

THEORY: Local Classes

A local class is defined inside a function body. Its scope is limited to that function — it cannot be used outside. Local classes cannot access the function's automatic (local) variables but can access static locals and global variables. Use cases are rare; lambdas often replace them.

TEORİ: Yerel Sınıflar (Local Classes)

Yerel sınıf, bir fonksiyon gövdesi içinde tanımlanan sınıftır. Kapsamı o fonksiyonla sınırlıdır — dışarıda kullanılamaz. Yerel sınıflar, fonksiyonun otomatik (yerel) değişkenlerine erişemez ama statik yerel ve global değişkenlere erişebilir. Kullanım alanı nadirdir; lambdalar genellikle onların yerini alır.

THEORY: PIMPL Idiom

PIMPL (Pointer to IMPLementation) hides implementation details behind a pointer to a private nested class/struct (often called Impl). The public class header only forward-declares Impl, keeping compile-time dependencies minimal. This is widely used in automotive ECU codebases to stabilize ABIs.

TEORİ: PIMPL Kalıbı

PIMPL (Pointer to IMPLementation), uygulama detaylarını özel bir iç sınıf/yapıya (genellikle Impl olarak adlandırılır) yönelik bir işaretçinin arkasına gizler. Public sınıf başlığı yalnızca Impl'i ileri bildirir ve derleme zamanı bağımlılıklarını minimum tutar. Otomotiv ECU kod tabanlarında ABI kararlılığı sağlamak için yaygın olarak kullanılır.

CPPREF Depth: Nested Class Access Rules and the Pimpl Compilation Firewall

Before C++11, nested classes had NO special access to the enclosing class's private members — they were treated like any other class. The C++11 standard (§11.7) changed this: a nested class is now a MEMBER of the enclosing class and can access all private/protected names (given an instance of the outer class). This is what makes the Pimpl idiom clean. The PIMPL compilation firewall works because the public header only contains `class Impl;` (forward declaration) and `std::unique_ptr<Impl>`. Since unique_ptr's destructor requires Impl to be COMPLETE (it calls `delete`), you MUST define the outer class's destructor in the .cpp file where Impl is fully defined. Forgetting this causes a compile error: "incomplete type" in the unique_ptr destructor instantiation. PIMPL reduces transitive #include dependencies dramatically. A change to Impl's data members or private methods does NOT change the public header → no recompilation of dependent translation units. In large automotive codebases (millions of LOC), this saves hours of build time. Performance cost: every call to an Impl method is an extra pointer indirection, and Impl lives on the heap (cache-unfriendly). For hot paths, consider "fast Pimpl" with aligned storage or a fixed buffer.

CPPREF Derinlik: İç İçe Sınıf Erişim Kuralları ve Pimpl Derleme Güvenlik Duvarı

C++11 öncesinde iç içe sınıflar, çevreleyen sınıfın private üyelerine ÖZEL bir erişime sahip değildi. C++11 standardı (§11.7) bunu değiştirdi: iç içe sınıf artık çevreleyen sınıfın bir ÜYESİDİR ve tüm private/protected isimlere erişebilir (dış sınıfın bir örneği verildiğinde). PIMPL derleme güvenlik duvarı şöyle çalışır: public başlık yalnızca `class Impl;` (ileri bildirim) ve `std::unique_ptr<Impl>` içerir. unique_ptr'nin yıkıcısı Impl'in TAM olmasını gerektirir (`delete` çağırır), bu yüzden dış sınıfın yıkıcısını Impl'in tamamen tanımlandığı .cpp dosyasında tanımlamanız GEREKİR. Bunu unutmak derleme hatasına neden olur. PIMPL, geçişli #include bağımlılıklarını dramatik biçimde azaltır. Impl'in veri üyelerindeki değişiklik public başlığı DEĞİŞTİRMEZ → bağımlı çeviri birimlerinin yeniden derlenmesi gerekmez. Büyük otomotiv kod tabanlarında bu, saatlerce derleme süresinden tasarruf sağlar. Performans maliyeti: her Impl metodu çağrısı ekstra bir işaretçi yönlendirmesidir ve Impl heap'te yaşar (önbellek dostu değil).
→ Ref: https://en.cppreference.com/w/cpp/language/nested_types

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: String class for text handling
#include <vector>     // EN: Dynamic array container
#include <array>      // EN: Fixed-size array container
#include <cstdint>    // EN: Fixed-width integer types (uint32_t, uint8_t)
#include <memory>     // EN: Smart pointers (unique_ptr) for PIMPL
#include <algorithm>  // EN: std::count_if for validation demo

// ═══════════════════════════════════════════════════════════════════════════════
// 1. CanMessage with Nested Header and Payload Classes
// ═══════════════════════════════════════════════════════════════════════════════

// EN: A CAN message class with nested Header and Payload — mimics a real CAN 2.0 frame.
// TR: İç içe Header ve Payload içeren bir CAN mesaj sınıfı — gerçek bir CAN 2.0 çerçevesini
// taklit eder.
class CanMessage {
public:
    // ─── 1a. Nested Header ──────────────────────────────────────────────────
    class Header {
    public:
        Header() = default;
        Header(uint32_t id, uint8_t dlc, bool is_extended, bool is_rtr)
            : arb_id_(id), dlc_(dlc), extended_frame_(is_extended), rtr_(is_rtr) {}

        uint32_t getArbId()       const { return arb_id_; }
        uint8_t  getDlc()         const { return dlc_; }
        bool     isExtended()     const { return extended_frame_; }
        bool     isRtr()          const { return rtr_; }

        void setArbId(uint32_t id)      { arb_id_ = id; }
        void setDlc(uint8_t dlc)        { dlc_ = (dlc <= 8) ? dlc : 8; }
        void setExtended(bool ext)      { extended_frame_ = ext; }
        void setRtr(bool rtr)           { rtr_ = rtr; }

    private:
        uint32_t arb_id_        = 0x000;
        uint8_t  dlc_           = 0;
        bool     extended_frame_ = false;
        bool     rtr_           = false;
    };

    // ─── 1b. Nested Payload ─────────────────────────────────────────────────
    class Payload {
    public:
        Payload() { data_.fill(0x00); }

        void setByte(size_t index, uint8_t value) {
            if (index < 8) data_[index] = value;
        }
        uint8_t getByte(size_t index) const {
            return (index < 8) ? data_[index] : 0;
        }
        const std::array<uint8_t, 8>& raw() const { return data_; }

    private:
        std::array<uint8_t, 8> data_;
    };

    // ─── 1c. CanMessage public interface ────────────────────────────────────
    CanMessage() = default;
    CanMessage(uint32_t id, uint8_t dlc) {
        header_.setArbId(id);
        header_.setDlc(dlc);
    }

    Header&       header()        { return header_; }
    const Header& header()  const { return header_; }
    Payload&      payload()       { return payload_; }
    const Payload& payload() const { return payload_; }
```

```cpp

    void print() const {
        std::cout << "CAN Frame [ID=0x" << std::hex << header_.getArbId()
                  << std::dec << " DLC=" << static_cast<int>(header_.getDlc())
                  << " EXT=" << header_.isExtended()
                  << " RTR=" << header_.isRtr() << "] Data: ";
        for (uint8_t i = 0; i < header_.getDlc(); ++i) {
            std::cout << "0x" << std::hex
                      << static_cast<int>(payload_.getByte(i)) << " ";
        }
        std::cout << std::dec << "\n";
    }

private:
    Header  header_;
    Payload payload_;
};

// ═══════════════════════════════════════════════════════════════════════════════
// 2. ECUDiagnosticSession with PIMPL-like Nested Impl
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Diagnostic session class hiding internal state via a nested Impl struct.
// TR: İç içe Impl yapısı ile iç durumu gizleyen teşhis oturumu sınıfı.
class ECUDiagnosticSession {
public:
    ECUDiagnosticSession();
    ~ECUDiagnosticSession();

    // EN: Public interface — delegates to Impl
    // TR: Public arayüz — Impl'e delege eder
    void startSession(uint16_t session_id);
    bool authenticate(const std::string& key);
    void storeDtc(const std::string& dtc_code);
    std::vector<std::string> readDtcs() const;
    void printStatus() const;

private:
    // EN: Forward-declared nested struct holds all private state
    // TR: İleri bildirimli iç yapı, tüm özel durumu tutar
    struct Impl;
    std::unique_ptr<Impl> pimpl_;
};

// ─── 2a. Impl definition ─────────────────────────────────────────────────────
struct ECUDiagnosticSession::Impl {
    uint16_t session_id       = 0;
    bool     authenticated    = false;
    std::vector<std::string> dtc_buffer;

    static constexpr const char* SECRET_KEY = "ECU_AUTH_2026";
};

ECUDiagnosticSession::ECUDiagnosticSession()
    : pimpl_(std::make_unique<Impl>()) {}

ECUDiagnosticSession::~ECUDiagnosticSession() = default;

void ECUDiagnosticSession::startSession(uint16_t session_id) {
    pimpl_->session_id    = session_id;
    pimpl_->authenticated = false;
    pimpl_->dtc_buffer.clear();
    std::cout << "[ECU] Session 0x" << std::hex << session_id
              << std::dec << " started.\n";
}

bool ECUDiagnosticSession::authenticate(const std::string& key) {
    pimpl_->authenticated = (key == Impl::SECRET_KEY);
    std::cout << "[ECU] Authentication "
              << (pimpl_->authenticated ? "SUCCESS" : "FAILED") << "\n";
    return pimpl_->authenticated;
}

void ECUDiagnosticSession::storeDtc(const std::string& dtc_code) {
```

```
144     if (!pimpl_->authenticated) {
145         std::cout << "[ECU] ERROR: Not authenticated. Cannot store DTC.\n";
146         return;
147     }
148     pimpl_->dtc_buffer.push_back(dtc_code);
149 }
150
151 std::vector<std::string> ECUDiagnosticSession::readDtcs() const {
152     if (!pimpl_->authenticated) {
153         std::cout << "[ECU] ERROR: Not authenticated. Cannot read DTCs.\n";
154         return {};
155     }
156     return pimpl_->dtc_buffer;
157 }
158
159 void ECUDiagnosticSession::printStatus() const {
160     std::cout << "[ECU] Session=0x" << std::hex << pimpl_->session_id
161               << std::dec
162               << " Auth=" << (pimpl_->authenticated ? "YES" : "NO")
163               << " DTCs=" << pimpl_->dtc_buffer.size() << "\n";
164 }
165
166 // ═══════════════════════════════════════════════════════════════════════════════
167 // 3. Local Class inside a Function
168 // ═══════════════════════════════════════════════════════════════════════════════
169
170 // EN: Demonstrates a local class used for single-purpose CAN traffic validation.
171 // TR: Tek amaçlı CAN trafik doğrulaması için kullanılan yerel sınıfı gösterir.
172 void validateCanTraffic(const std::vector<CanMessage>& traffic) {
173     // EN: Local struct — only visible within this function
174     // TR: Yerel yapı — yalnızca bu fonksiyon içinde görünür
175     struct FrameValidator {
176         static bool isValid(const CanMessage& msg) {
177             // EN: Standard CAN ID must be <= 0x7FF, DLC must be 1-8
178             // TR: Standart CAN ID <= 0x7FF olmalı, DLC 1-8 arasında olmalı
179             if (msg.header().isExtended()) return true;  // skip extended check
180             return msg.header().getArbId() <= 0x7FF &&
181                    msg.header().getDlc() >= 1 &&
182                    msg.header().getDlc() <= 8;
183         }
184     };
185
186     auto valid_count = std::count_if(traffic.begin(), traffic.end(),
187                                      FrameValidator::isValid);
188     std::cout << "[Validator] " << valid_count << " / " << traffic.size()
189               << " frames are valid.\n";
190
191     for (size_t i = 0; i < traffic.size(); ++i) {
192         std::cout << "  Frame " << i << ": "
193                   << (FrameValidator::isValid(traffic[i]) ? "OK" : "INVALID")
194                   << "\n";
195     }
196 }
197
198 // ═══════════════════════════════════════════════════════════════════════════════
199 // 4. Nested Class Accessing Outer's Private Members
200 // ═══════════════════════════════════════════════════════════════════════════════
201
202 // EN: Demonstrates that a nested class can access private members of the outer class.
203 // TR: İç sınıfın, dış sınıfın private üyelerine erişebildiğini gösterir.
204 class SensorHub {
205 public:
```

```
206     // EN: Nested Inspector can read SensorHub's private calibration_offset_
207     // TR: İç Inspector, SensorHub'ın private calibration_offset_ alanını okuyabilir
208     class Inspector {
209     public:
210         static void diagnose(const SensorHub& hub) {
211             std::cout << "[Inspector] Hub '" << hub.name_
212                       << "' calibration offset = " << hub.calibration_offset_
213                       << ", sensor count = " << hub.readings_.size() << "\n";
214         }
215     };
216 
217     explicit SensorHub(std::string name, double offset = 0.0)
218         : name_(std::move(name)), calibration_offset_(offset) {}
219 
220     void addReading(double value) { readings_.push_back(value + calibration_offset_); }
221 
222     void printReadings() const {
223         std::cout << "[SensorHub:" << name_ << "] Readings: ";
224         for (double r : readings_) std::cout << r << " ";
225         std::cout << "\n";
226     }
227 
228 private:
229     std::string name_;
230     double calibration_offset_;
231     std::vector<double> readings_;
232 };
233 
234 // ═══════════════════════════════════════════════════════════════════════════
235 // main() — Demos
236 // ═══════════════════════════════════════════════════════════════════════════
237 int main() {
238     std::cout << "=== Demo 1: CanMessage with Nested Header & Payload ===\n";
239     // EN: Construct a CAN message, set header fields, fill payload, print
240     // TR: Bir CAN mesajı oluştur, başlık alanlarını ayarla, veriyi doldur, yazdır
241     CanMessage msg(0x1A3, 4);
242     msg.payload().setByte(0, 0xDE);
243     msg.payload().setByte(1, 0xAD);
244     msg.payload().setByte(2, 0xBE);
245     msg.payload().setByte(3, 0xEF);
246     msg.print();
247 
248     // EN: Access nested type from outside via scope resolution
249     // TR: İç türe dışarıdan kapsam çözücü ile erişim
250     CanMessage::Header standalone_hdr(0x7DF, 8, false, false);
251     std::cout << "Standalone Header ID: 0x" << std::hex
252               << standalone_hdr.getArbId() << std::dec << "\n\n";
253 
254     std::cout << "=== Demo 2: ECUDiagnosticSession (PIMPL) ===\n";
255     // EN: Create session, authenticate, store and read DTCs
256     // TR: Oturum oluştur, kimlik doğrula, DTC kaydet ve oku
257     ECUDiagnosticSession diag;
258     diag.startSession(0x01);
259     diag.printStatus();
260 
261     diag.storeDtc("P0301");   // EN: Should fail — not authenticated
262     diag.authenticate("WRONG_KEY");
263     diag.authenticate("ECU_AUTH_2026");
264     diag.printStatus();
265 
266     diag.storeDtc("P0301");
267     diag.storeDtc("P0420");
268     diag.storeDtc("U0100");
269 
270     auto dtcs = diag.readDtcs();
271     std::cout << "[ECU] Stored DTCs: ";
272     for (const auto& d : dtcs) std::cout << d << " ";
273     std::cout << "\n\n";
274 
275     std::cout << "=== Demo 3: Local Class — validateCanTraffic() ===\n";
```

```
    // EN: Build a small traffic sample with valid and invalid frames
    // TR: Geçerli ve geçersiz çerçevelerden oluşan küçük bir trafik örneği oluştur
    std::vector<CanMessage> traffic;
    traffic.emplace_back(0x100, 8);   // valid
    traffic.emplace_back(0x7FF, 1);   // valid (max standard ID)
    traffic.emplace_back(0x800, 4);   // invalid (ID > 0x7FF for standard)
    traffic.emplace_back(0x050, 0);   // invalid (DLC = 0)
    validateCanTraffic(traffic);
    std::cout << "\n";

    std::cout << "=== Demo 4: Nested Class Accessing Outer's Private ===\n";
    // EN: Inspector accesses SensorHub's private name_ and calibration_offset_
    // TR: Inspector, SensorHub'ın private name_ ve calibration_offset_ alanlarına erişir
    SensorHub hub("WheelSpeed_FL", 0.05);
    hub.addReading(1200.0);
    hub.addReading(1205.5);
    hub.addReading(1198.3);
    hub.printReadings();
    SensorHub::Inspector::diagnose(hub);

    return 0;
}
```

► 08. Rtti Typeid

module_02_oop/08_rtti_typeid.cpp

Konu: OOP Part 8: RTTI & typeid / OOP Kısım 8: RTTI ve typeid

Teori / Theory

THEORY: What is RTTI?

RTTI (Run-Time Type Information) is a C++ mechanism that allows the program to determine the actual (dynamic) type of a polymorphic object at runtime. When you have a base-class pointer or reference, RTTI lets you ask: "What is the REAL type hiding behind this pointer?" Two primary tools enable this: the `typeid` operator and `dynamic_cast`.

TEORİ: RTTI Nedir?

RTTI (Çalışma Zamanı Tür Bilgisi), C++'ın polimorfik bir nesnenin gerçek (dinamik) türünü çalışma zamanında belirlemeye yarayan mekanizmasıdır. Elinizde bir temel sınıf işaretçisi veya referansı varsa, RTTI size "Bu işaretçinin arkasında gizlenen GERÇEK tür nedir?" sorusunu sorma imkânı tanır. Bunu sağlayan iki temel araç vardır: `typeid` operatörü ve `dynamic_cast`.

THEORY: typeid Operator

`typeid(expr)` returns a `const std::type_info&`. For polymorphic types (classes with at least one virtual function), it evaluates the runtime type. For non-polymorphic types, it returns the static (compile-time) type. You can compare two type_info objects with `==` and `!=`, and call `.name()` to get an implementation-defined mangled name, or `.hash_code()` for fast comparisons in hash tables.

TEORİ: typeid Operatörü

`typeid(ifade)` bir `const std::type_info&` döndürür. Polimorfik türlerde (en az bir virtual fonksiyona sahip sınıflarda) çalışma zamanı türünü verir. Polimorfik olmayan türlerde ise derleme zamanı (statik) türünü döndürür. İki type_info nesnesini `==` ve `!=` ile karşılaştırabilir, `.name()` ile platforma göre değişen bir isim, `.hash_code()` ile hızlı hash değeri alabilirsiniz.

THEORY: dynamic_cast & Performance

`dynamic_cast<Derived*>(basePtr)` is the safe downcast mechanism. It returns nullptr on failure (for pointers) or throws `std::bad_cast` (for references). RTTI adds a small cost: each polymorphic class carries extra type metadata in its vtable. Compilers offer `-fno-rtti` to disable it entirely (saves binary size, but typeid and dynamic_cast become unavailable).

TEORİ: dynamic_cast ve Performans

`dynamic_cast<Türetilmiş*>(temelPtr)` güvenli alt-türe dönüşüm (downcast) mekanizmasıdır. Başarısız olursa pointer'larda nullptr, referanslarda `std::bad_cast` istisnası döndürür. RTTI küçük bir maliyet ekler: her polimorfik sınıf, vtable'ında ekstra tür bilgisi taşır. Derleyiciler `-fno-rtti` bayrağıyla tamamen devre dışı bırakmaya izin verir (binary boyutu küçülür ama typeid ve dynamic_cast kullanılamaz hale gelir).

THEORY: When to Use RTTI

Prefer virtual dispatch or std::variant/std::visit over RTTI when possible. RTTI is appropriate for plugin systems, serialization, debug logging, or when the set of derived types is open-ended. Overusing typeid checks is often a code smell — it bypasses polymorphism entirely.

TEORİ: RTTI Ne Zaman Kullanılmalı

Mümkün olduğunda RTTI yerine sanal fonksiyon dağıtımını (virtual dispatch) veya std::variant/std::visit tercih edin. RTTI; eklenti sistemleri, serileştirme, hata ayıklama günlüğü veya türetilmiş tür kümesinin açık uçlu olduğu durumlarda uygundur. typeid kontrollerini aşırı kullanmak genellikle bir tasarım hatasıdır — polimorfizmi tamamen devre dışı bırakır.

CPPREF Depth: RTTI Cost and Alternatives — When dynamic_cast Hurts Performance

RTTI requires a vtable — it ONLY works on polymorphic types (classes with at least one virtual function). Applying `typeid` to a non- polymorphic type yields the static (compile-time) type, not the runtime type — a subtle but critical distinction. `dynamic_cast` traverses the inheritance graph at runtime. The cost is O(depth) in the inheritance tree — in deep hierarchies with virtual bases, this involves following vbase offsets through the vtable. In tight loops processing millions of objects, this becomes measurable overhead. The `-fno-rtti` compiler flag (GCC/Clang) disables RTTI entirely, removing type_info structures from every polymorphic class. This typically saves ~5% binary size. However, `typeid` and `dynamic_cast` become compile errors, and libraries compiled WITH rtti cannot be mixed safely with code compiled WITHOUT it (ODR violations in type_info). Alternatives to RTTI-based dispatch: 1. Enum-based type tags — manual but fast O(1) switch dispatch. 2. Visitor pattern (double dispatch) — type-safe, no RTTI needed. 3. std::variant + std::visit — compile-time checked, zero overhead. 4. CRTP (Curiously Recurring Template Pattern) — static polymorphism. In automotive (MISRA C++), real-time, and embedded domains, RTTI is often BANNED due to unpredictable runtime cost and binary bloat.

CPPREF Derinlik: RTTI Maliyeti ve Alternatifleri — dynamic_cast Ne Zaman Performansı Düşürür

RTTI, vtable gerektirir — YALNIZCA polimorfik türlerde (en az bir virtual fonksiyona sahip sınıflarda) çalışır. Polimorfik olmayan bir türe `typeid` uygulamak statik (derleme zamanı) türünü verir — ince ama kritik bir fark. `dynamic_cast` çalışma zamanında kalıtım grafiğini dolaşır. Maliyet, kalıtım ağacında O(derinlik) kadardır — sanal tabanlı derin hiyerarşilerde vtable üzerinden vbase ofsetlerini takip etmeyi içerir. Milyonlarca nesneyi işleyen sıkı döngülerde bu ölçülebilir bir ek yük olur. `-fno-rtti` derleyici bayrağı (GCC/Clang) RTTI'yi tamamen devre dışı bırakır ve her polimorfik sınıftan type_info yapılarını kaldırır. Bu tipik olarak ~%5 binary boyutu tasarrufu sağlar. Ancak `typeid` ve `dynamic_cast` derleme hatasına dönüşür. RTTI tabanlı dağıtıma alternatifler: 1. Enum tabanlı tür etiketleri — manuel ama hızlı O(1) switch. 2. Visitor kalıbı (çift dağıtım) — tür güvenli, RTTI gerekmez. 3. std::variant + std::visit — derleme zamanında kontrollü, sıfır ek yük. 4. CRTP — statik polimorfizm. Otomotivde (MISRA C++), gerçek zamanlı ve gömülü alanlarda RTTI, öngörülemeyen çalışma zamanı maliyeti ve binary şişmesi nedeniyle sıklıkla YASAKLANIR.
→ Ref: https://en.cppreference.com/w/cpp/language/typeid

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for type descriptions
#include <typeinfo>   // EN: typeid, std::type_info, std::bad_cast
#include <memory>     // EN: std::unique_ptr for polymorphic ownership
#include <vector>     // EN: std::vector for sensor collection

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Polymorphic Sensor Hierarchy (Automotive Domain)
// ═══════════════════════════════════════════════════════════════════════════

// EN: Base class with virtual functions — makes the hierarchy polymorphic.
// TR: Sanal fonksiyonlara sahip temel sınıf — hiyerarşiyi polimorfik yapar.
class Sensor {
public:
    virtual ~Sensor() = default;

    virtual std::string getType() const { return "GenericSensor"; }

    virtual double read() const = 0;
};

// ─── 1.1 TemperatureSensor ─────────────────────────────────────────────────
class TemperatureSensor : public Sensor {
    double celsius_;
public:
    explicit TemperatureSensor(double c) : celsius_(c) {}

    std::string getType() const override { return "TemperatureSensor"; }
    double read() const override { return celsius_; }

    // EN: Derived-specific method — only accessible after downcast.
    // TR: Türetilmiş sınıfa özgü metot — yalnızca downcast sonrası erişilir.
    double toFahrenheit() const { return celsius_ * 9.0 / 5.0 + 32.0; }
};

// ─── 1.2 PressureSensor ────────────────────────────────────────────────────
class PressureSensor : public Sensor {
    double kpa_;
public:
    explicit PressureSensor(double kpa) : kpa_(kpa) {}

    std::string getType() const override { return "PressureSensor"; }
    double read() const override { return kpa_; }
```

```cpp

    // EN: Convert kilopascals to PSI (pounds per square inch).
    // TR: Kilopascal değerini PSI birimine dönüştürür.
    double toPSI() const { return kpa_ * 0.145038; }
};

// ─── 1.3 OxygenSensor ──────────────────────────────────────────────────────
class OxygenSensor : public Sensor {
    double voltage_;
public:
    explicit OxygenSensor(double v) : voltage_(v) {}

    std::string getType() const override { return "OxygenSensor"; }
    double read() const override { return voltage_; }

    // EN: Check if the air-fuel mixture is running lean (voltage < 0.45V).
    // TR: Hava-yakıt karışımının fakir (lean) olup olmadığını kontrol eder.
    bool isLean() const { return voltage_ < 0.45; }
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: Non-Polymorphic Type (for comparison)
// ═══════════════════════════════════════════════════════════════════════════

// EN: A simple struct without virtual functions — typeid returns static type.
// TR: Virtual fonksiyonu olmayan basit bir yapı — typeid statik tür döndürür.
struct DTCCode {
    int code;
    std::string description;
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3: Production Functions Using RTTI
// ═══════════════════════════════════════════════════════════════════════════

// ─── 3.1 identifySensor ────────────────────────────────────────────────────
// EN: Uses typeid to print the actual runtime type of a sensor.
// TR: Sensörün gerçek çalışma zamanı türünü yazdırmak için typeid kullanır.
void identifySensor(const Sensor& s) {
    std::cout << "  typeid name   : " << typeid(s).name() << "\n";
    std::cout << "  hash_code     : " << typeid(s).hash_code() << "\n";
    std::cout << "  getType()     : " << s.getType() << "\n";
}

// ─── 3.2 processSpecificSensor ─────────────────────────────────────────────
// EN: Uses dynamic_cast to safely downcast and call derived-specific methods.
// TR: Güvenli downcast yapıp türetilmiş sınıfa özgü metotları çağırır.
void processSpecificSensor(Sensor* s) {
    if (auto* temp = dynamic_cast<TemperatureSensor*>(s)) {
        std::cout << "  [TemperatureSensor] " << temp->read() << " °C = "
                  << temp->toFahrenheit() << " °F\n";
    } else if (auto* pres = dynamic_cast<PressureSensor*>(s)) {
        std::cout << "  [PressureSensor] " << pres->read() << " kPa = "
                  << pres->toPSI() << " PSI\n";
    } else if (auto* oxy = dynamic_cast<OxygenSensor*>(s)) {
        std::cout << "  [OxygenSensor] " << oxy->read() << " V -> "
                  << (oxy->isLean() ? "Lean" : "Rich") << " mixture\n";
    } else {
        std::cout << "  [Unknown sensor type]\n";
    }
}
```

```cpp

// ─── 3.3 demonstrateTypeComparison ──────────────────────────────────────────
// EN: Shows typeid equality comparison between objects.
// TR: Nesneler arasında typeid eşitlik karşılaştırmasını gösterir.
void demonstrateTypeComparison(const Sensor& a, const Sensor& b) {
    if (typeid(a) == typeid(b)) {
        std::cout << "  Result: SAME runtime type ("
                  << typeid(a).name() << ")\n";
    } else {
        std::cout << "  Result: DIFFERENT runtime types ("
                  << typeid(a).name() << " vs " << typeid(b).name() << ")\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "=== Module 02 OOP | 08: RTTI & typeid ===\n\n";

    // ─── Demo 1: typeid on different sensor types through base pointer ──
    std::cout << "--- Demo 1: typeid through base pointer ---\n";
    std::vector<std::unique_ptr<Sensor>> sensors;
    sensors.push_back(std::make_unique<TemperatureSensor>(92.5));
    sensors.push_back(std::make_unique<PressureSensor>(101.3));
    sensors.push_back(std::make_unique<OxygenSensor>(0.72));

    for (const auto& s : sensors) {
        std::cout << "Sensor [" << s->getType() << "]:\n";
        identifySensor(*s);
        std::cout << "\n";
    }

    // ─── Demo 2: typeid equality comparison ──────────────────────────────
    std::cout << "--- Demo 2: typeid equality comparison ---\n";
    TemperatureSensor coolantTemp(88.0);
    TemperatureSensor intakeTemp(35.0);
    PressureSensor boostPressure(150.0);

    std::cout << "Comparing coolantTemp vs intakeTemp:\n";
    demonstrateTypeComparison(coolantTemp, intakeTemp);

    std::cout << "Comparing coolantTemp vs boostPressure:\n";
    demonstrateTypeComparison(coolantTemp, boostPressure);
    std::cout << "\n";

    // ─── Demo 3: dynamic_cast safe downcast ──────────────────────────────
    std::cout << "--- Demo 3: dynamic_cast safe downcast ---\n";
    Sensor* engineTemp = &coolantTemp;
    processSpecificSensor(engineTemp);

    Sensor* manifold = &boostPressure;
    processSpecificSensor(manifold);

    OxygenSensor lambdaProbe(0.38);
    Sensor* exhaust = &lambdaProbe;
    processSpecificSensor(exhaust);
    std::cout << "\n";

    // ─── Demo 4: dynamic_cast failure (returns nullptr) ──────────────────
    std::cout << "--- Demo 4: dynamic_cast failure ---\n";
    Sensor* sensorPtr = &boostPressure;
    auto* asTemp = dynamic_cast<TemperatureSensor*>(sensorPtr);
    if (asTemp == nullptr) {
        std::cout << "  dynamic_cast<TemperatureSensor*> on PressureSensor "
                  << "returned nullptr (safe failure)\n";
    }
```

```cpp

    // EN: Reference cast throws std::bad_cast on failure.
    // TR: Referans dönüşümü başarısızlıkta std::bad_cast fırlatır.
    try {
        [[maybe_unused]] auto& asOxy =
            dynamic_cast<OxygenSensor&>(*sensorPtr);
    } catch (const std::bad_cast& e) {
        std::cout << "  dynamic_cast<OxygenSensor&> threw: "
                  << e.what() << "\n";
    }
    std::cout << "\n";

    // ─── Demo 5: Polymorphic vs non-polymorphic typeid ──────────────────────────
    std::cout << "--- Demo 5: Polymorphic vs non-polymorphic typeid ---\n";

    // EN: Polymorphic — typeid resolves actual runtime type.
    // TR: Polimorfik — typeid gerçek çalışma zamanı türünü çözer.
    Sensor* basePtr = &coolantTemp;
    std::cout << "  Declared type : Sensor*\n";
    std::cout << "  typeid(*basePtr): " << typeid(*basePtr).name()
              << "  (runtime type)\n";

    // EN: Non-polymorphic — typeid always returns static (declared) type.
    // TR: Polimorfik olmayan — typeid her zaman statik (bildirilen) türü döner.
    DTCCode dtc{420, "Catalyst Temp Below Threshold"};
    const DTCCode& dtcRef = dtc;
    std::cout << "  Declared type : const DTCCode&\n";
    std::cout << "  typeid(dtcRef)  : " << typeid(dtcRef).name()
              << "  (static type — no virtual)\n";
    std::cout << "\n";

    // ─── Demo 6: type_info::name() and type_info::hash_code() ───────────────────
    std::cout << "--- Demo 6: type_info name() & hash_code() ---\n";
    const std::type_info& ti1 = typeid(TemperatureSensor);
    const std::type_info& ti2 = typeid(PressureSensor);
    const std::type_info& ti3 = typeid(TemperatureSensor);

    std::cout << "  TemperatureSensor name : " << ti1.name() << "\n";
    std::cout << "  PressureSensor name    : " << ti2.name() << "\n";
    std::cout << "  TemperatureSensor hash : " << ti1.hash_code() << "\n";
    std::cout << "  PressureSensor hash    : " << ti2.hash_code() << "\n";
    std::cout << "  ti1 == ti3 (same type) : "
              << (ti1 == ti3 ? "true" : "false") << "\n";
    std::cout << "  ti1 == ti2 (diff type) : "
              << (ti1 == ti2 ? "true" : "false") << "\n";

    std::cout << "\n=== RTTI & typeid demonstration complete ===\n";
    return 0;
}
```

Modül 3: Modern C++ — Güvenlik ve Performans

Güvenlik ve Performans

5 kaynak dosya · Standart: C++17

C++17

► 01. Smart Pointers

module_03_modern_cpp/01_smart_pointers.cpp

Konu: Modern C++: Smart Pointers and RAII / Akıllı İşaretçiler ve RAII (Resource Acquisition

Teori / Theory

THEORY: Why Do We Need Smart Pointers?

In old C++ (before C++11), developers managed memory dynamically using `new` and `delete`. This was a nightmare. If a function threw an exception or returned early, `delete` was skipped -> Memory Leak! If we deleted a pointer twice -> Double Free Crash!

TEORİ: Neden Akıllı İşaretçilere İhtiyacımız Var?

Eski C++'da (C++11 öncesi), geliştiriciler belleği `new` ve `delete` ile tamamen manuel yönetirdi. Bu büyük bir kabustu. Bir fonksiyon hata fırlatır (exception) veya erken `return` ederse, `delete` komutu atlanırdı -> Bellek Sızıntısı (Memory Leak). Aynı pointer'ı yanlışlıkla iki defa silersek -> Çift Silme Çöküşü (Double Free Crash).

THEORY: What is RAII?

The backbone principle of C++. "When a resource (memory, file, socket) is Acquired, bind it to a local/stack Object. When that Object goes out of scope ({...}), its Destructor automatically runs and safely frees the resource." Smart Pointers are pure RAII. They silently call `delete` for you.

TEORİ: RAII Nedir?

C++'ın belkemiği prensibi. "Bir kaynak (bellek, dosya, ağ soketi) alındığı anda (Acquisition) onu yerel/stack'te yaşayan bir nesneye bağla. O nesne yaşam alanından ({...} scope) çıktığında otomatik olarak Yıkıcısı (Destructor) çalışsın ve kaynağı güvenle serbest bıraksın."
Akıllı işaretçiler (Smart Pointers) tam olarak RAII'dir. Arka planda senin için sessizce `delete` çağırırlar.

this memory. If I die, it dies." (Sahipliği TEKTİR. Kopyalanamaz, sadece taşınabilir). 2. std::shared_ptr: SHARED Ownership. Counts how many people hold the memory (Reference Counting). When the count hits 0, the memory vanishes. (Sahipliği PAYLAŞILIR. Referans sayacı 0 olunca belleği uçurur). 3. std::weak_ptr: A "watcher" for shared_ptr that doesn't increment the count, preventing circular reference loops! (shared_ptr sayacını artırmadan gözcülük yapan, dairesel referans kilitlenmelerini çözen türdür).

CPPREF Depth: `std::make_shared` vs `new` & The Control Block

1. std::unique_ptr: UNIQUE Ownership. Cannot be copied (only moved). "I am the sole owner of

CppReference dictates: Always use `std::make_shared` instead of `std::shared_ptr<T>(new T())`. Why? Because `shared_ptr` needs TWO things: 1. The actual object. 2. A "Control Block" (holds the counter). Using `new` allocates these separately (2 RAM jumps). `make_shared` allocates them in ONE single contiguous memory block (Faster Cache Hits, No memory leaps).

CPPREF Derinlik: `std::make_shared` vs `new` ve Kontrol Bloğu

CppReference emreder: Daima `std::make_shared` kullanın. Çünkü `shared_ptr` RAM'de 2 şeye ihtiyaç duyar: 1. Asıl Nesne, 2. Sayaç Bloğu (Control Block). Eğer `new` derseniz işletim sistemi iki farklı yere gider (Yavaş). `make_shared` ise Nesne'yi ve Sayacı BİTİŞİK TEK RAM BLOĞUNA yazar (CPU Önbellek, Cache Hit Hızı %100).

Kaynak Kod / Source Code

```cpp
#include <iostream>
// EN: Required for smart pointers
// TR: Akıllı işaretçiler için zorunlu
#include <memory>

/**
 * @class HeavyResource
 * @brief EN: Represents a memory-heavy object like a 4K texture or a gigabyte DB. TR: 4K doku
 * veya megabaytlık bir veritabanı gibi ağır nesneleri temsil eder.
 */
class HeavyResource {
public:
  HeavyResource() {
    std::cout << "[HeavyResource] Acquired from OS. (Kaynak İşletim "
                 "Sisteminden Alındı)"
              << std::endl;
  }
  ~HeavyResource() {
    std::cout << "[HeavyResource] Destroyed safely! (Kaynak Güvenle Yok "
                 "Edildi!) -> NO LEAK"
              << std::endl;
  }
  void render() {
    std::cout << "Rendering 4K Textures... / 4K Dokular İşleniyor..." << std::endl;
  }
};

int main() {
  std::cout << "=== MODERN C++: SMART POINTERS & RAII ===\n" << std::endl;

  std::cout << "--- 1. Raw Pointer (Legacy System - Bad Practice / Eski Sistem "
               "- Kötü Pratik) ---"
            << std::endl;
  HeavyResource *rawPointer = new HeavyResource();

  // EN: If an exception happens here, 'delete' is skipped -> Memory Leak!
  // TR: Eğer burada hata fırlayıp delete adımını atlarsak RAM dolar taşar!
  delete rawPointer;

  std::cout << "\n--- 2. std::unique_ptr (C++14 Way - Best Practice / C++14 "
               "Yolu - En İyi Pratik) ---"
            << std::endl;
  {
```

```
    // EN: Scope begins! `std::make_unique()` is faster and SAFER than calling `new`.
    // TR: Scope Başlıyor! `std::make_unique()` kullanmak, doğrudan `new` kullanmaktan çok daha
    // GÜVENLİDİR.
    std::unique_ptr<HeavyResource> uniquePtr = std::make_unique<HeavyResource>();
    uniquePtr->render();

    // auto uniquePtr2 = uniquePtr; // ERROR! Compiler forbids it. Ownership is unique. / HATA!
    // KOPYALANAMAZ.

    // EN: Allowed! Ownership transferred via Move Semantics.
    // TR: İzin verilir. Sahiplik devredildi (Taşıma Semantiği).
    std::unique_ptr<HeavyResource> movedPtr = std::move(uniquePtr);

  } // EN: Scope ends. movedPtr dies. Destructor AUTO-RUNS!
    // TR: Süslü parantez bitti. SCOPE DIŞINA çıktık. movedPtr YOK EDİLDİ. Yıkıcı OTO-ÇALIŞTI!

  std::cout << "\n--- 3. std::shared_ptr (Shared Ownership / Paylaşılan Sahiplik) ---" << std::endl;
  {
    // Count = 1 (Sayaç 1)
    std::shared_ptr<HeavyResource> shared1 = std::make_shared<HeavyResource>();
    std::cout << "Owners (Sahip Sayısı): " << shared1.use_count() << std::endl;

    {
      // EN: Copied! Count = 2
      // TR: Kopya ALINABİLİR! Sayaç 2
      std::shared_ptr<HeavyResource> shared2 = shared1;
      std::cout << "Owners in lower scope (Alt Scope Sahip Sayısı): " << shared1.use_count() <<
          std::endl;
    } // EN: shared2 dies. Count drops to 1. Object still lives.
      // TR: shared2 öldü. Sayaç tekrar 1'e düştü (Nesne halen yaşıyor).

    std::cout << "Owners after lower scope (Alt Scope Bittiğinde Sahip Sayısı): " <<
        shared1.use_count() << std::endl;
  } // EN: shared1 dies. Count becomes 0. OBJECT DESTROYED!
    // TR: shared1 öldü. Sayaç 0 oldu. NESNE TAMAMEN YOK EDİLDİ!

  return 0;
}
```

► 02. Lambdas

module_03_modern_cpp/02_lambdas.cpp

Konu: Modern C++: Lambda Expressions — From Zero to Mastery / Lambda İfadeleri — Sıfırdan

Teori / Theory

THEORY: What is a Lambda? (Lambdas for Dummies)

Imagine you need to tell the ECU diagnostic system: "Filter only the fault codes above severity 3." Before C++11 you had to write a separate named function somewhere far away and pass a pointer to it. With a Lambda you write the rule RIGHT WHERE you need it — inline, compact, readable.
A Lambda is an ANONYMOUS (unnamed) function object that the compiler generates for you behind the scenes. Think of it as a disposable mini-function you define on the spot. Lambdas are the cornerstone of modern STL algorithms (std::sort, std::for_each, std::find_if, std::transform, ...).

TEORİ: Lambda Nedir? (Sıfırdan Anlatım)

ECU arıza teşhis sistemine: "Sadece önem derecesi 3 üzeri hata kodlarını filtrele" demek istediğinizi düşünün. C++11 öncesi uzaklarda ayrı bir fonksiyon yazıp onun adresini (pointer) göndermeniz gerekirdi. Lambda ile kuralı TAM İHTİYAÇ DUYDUĞUNUZ YERE, satır içi, kompakt ve okunabilir şekilde yazarsınız.
Lambda, derleyicinin arka planda sizin için ürettiği İSİMSİZ (anonim) bir fonksiyon nesnesidir. Onu tek kullanımlık, yerinde tanımlanan mini-fonksiyon olarak düşünün. Lambdalar modern STL algoritmalarının (std::sort, std::for_each, std::find_if, std::transform, ...) temel taşıdır.

THEORY: Lambda Anatomy — The Full Syntax

The complete Lambda syntax is:

- [capture_list] : HOW to grab outside variables (see next theory). - (parameters) : Input arguments, just like a normal function. - mutable : (Optional) Lets you modify captured-by-value copies. - -> return_type : (Optional) Explicit return type. Usually auto-deduced. - { body } : The actual code.
Minimal Lambda: [](){} — captures nothing, takes nothing, does nothing. When there are no parameters you may even omit (): []{}.

TEORİ: Lambda Anatomisi — Tam Sözdizimi

Lambda'nın tam sözdizimi:

- [yakalama_listesi] : Dış değişkenleri NASIL yakalayacağınız. - (parametreler) : Girdi argümanları, normal fonksiyon gibi. - mutable : (Opsiyonel) Değer ile yakalanan kopyaları değiştirmenize izin verir. - -> dönüş_tipi : (Opsiyonel) Açık dönüş tipi. Genelde otomatik çıkarılır. - { gövde } : Gerçek kod.
En Basit Lambda: []{}() — hiçbir şey yakalamaz, almaz, yapmaz. Parametre yoksa () bile atlanabilir: []{}.

THEORY: Capture List Deep Dive

The capture list is what makes Lambdas special. It controls how outside variables enter the Lambda's body:
[] — Capture nothing. Pure function, no outside variables.

'expr' (C++14). Great for move-capturing!
Golden Rule: Capture only what you need. Prefer explicit captures [x, &y] over blanket [=] or

TEORİ: Yakalama Listesi Detaylı İnceleme

Yakalama listesi Lambda'ları özel kılan şeydir. Dış değişkenlerin Lambda gövdesine nasıl gireceğini kontrol eder:
[] — Hiçbir şey yakalama. Saf fonksiyon, dış değişken yok.

değişkeni oluştur (C++14). Move-capture için mükemmel!
Altın Kural: Sadece ihtiyacınız olanı yakalayın. Netlik ve güvenlik için toptan [=] veya [&] yerine açık yakalama [x, &y] tercih edin.

CPPREF Depth: Dangling Captures & Lifetime Traps

cppreference.com/w/cpp/language/lambda warns: If a Lambda captures local variables by REFERENCE [&] and then OUTLIVES the scope where those variables were defined (e.g., returned from a function, stored in a container, passed to std::thread), you get a DANGLING REFERENCE — Undefined Behavior! The Lambda holds an address to memory that no longer exists.
Safe rule: If the Lambda escapes the current scope, capture by VALUE [=] or use init-capture

CPPREF Derinlik: Sarkan Yakalamalar ve Ömür Tuzakları

cppreference.com/w/cpp/language/lambda uyarır: Lambda yerel değişkenleri REFERANS [&] ile yakalayıp, bu değişkenlerin tanımlandığı kapsamın dışına çıkarsa (fonksiyondan dönülürse, container'a saklanırsa, std::thread'e verilirse), SARKAN REFERANS (Dangling Reference) oluşur — Tanımsız Davranış! Lambda artık var olmayan belleğe işaret eder.
Güvenli kural: Lambda mevcut kapsamdan dışarı çıkıyorsa DEĞER [=] ile yakalayın veya init-capture [x = std::move(obj)] ile veriyi sahiplenin.

CPPREF Depth: std::function — Type-Erased Lambda Storage

cppreference.com/w/cpp/utility/functional/function documents: std::function<ReturnType(Args...)> is a general-purpose polymorphic function wrapper. It can store ANY callable: a Lambda, a function pointer, a functor, or std::bind result. Use it when you need to store a Lambda in a member variable, a container, or pass it across API boundaries. Be aware: it has overhead (heap allocation, virtual dispatch). For performance-critical automotive loops, prefer templates or auto.

CPPREF Derinlik: std::function — Tip Silmeli Lambda Saklama

cppreference.com/w/cpp/utility/functional/function belgeliyor: std::function<DönüşTipi(Argümanlar...)> genel amaçlı polimorfik fonksiyon sarmalayıcıdır. HERHANGİ bir çağrılabiliri saklayabilir: Lambda, fonksiyon işaretçisi, functor veya std::bind sonucu. Lambda'yı üye değişkende, container'da saklamanız veya API sınırları arasında geçirmeniz gerektiğinde kullanın. Dikkat: maliyeti vardır (heap tahsisi, sanal dağıtım). Performans-kritik otomotiv döngülerinde template veya auto tercih edin.

THEORY: Generic Lambdas (C++14) & Template Lambdas (C++20)

C++14 introduced generic Lambdas: use 'auto' in the parameter list and the compiler generates a templated operator() internally. This is perfect for writing ECU-agnostic utility functions.
C++20 goes further: you can write explicit template parameter lists on Lambdas: []<typename T>(T value){ ... }. This gives you SFINAE/concepts power directly inside a Lambda — useful for compile-time filtering of sensor types in embedded systems.

TEORİ: Jenerik Lambdalar (C++14) ve Şablon Lambdalar (C++20)

C++14 jenerik Lambda'ları tanıttı: parametre listesinde 'auto' kullanın, derleyici içeride şablonlu operator() üretir. Bu, ECU-bağımsız yardımcı fonksiyonlar yazmak için mükemmeldir.
C++20 daha ileri gider: Lambda'larda açık şablon parametre listesi yazabilirsiniz: []<typename T>(T value){ ... }. Bu size doğrudan Lambda içinde SFINAE/konsept gücü verir — gömülü sistemlerde sensör tiplerini derleme zamanında filtrelemek için kullanışlıdır.

CPPREF Depth: Lambda as Constexpr (C++17) & Consteval (C++20)

cppreference.com/w/cpp/language/lambda documents: Since C++17, Lambdas are implicitly constexpr if their body satisfies constexpr requirements. You can compute values at COMPILE TIME using Lambdas — zero runtime cost. In automotive ECUs with limited flash, this means look-up tables, CRC checksums, and protocol constants can be Lambda-computed at compile time.
C++20 allows `consteval` Lambdas that MUST execute at compile time.

CPPREF Derinlik: constexpr Lambda (C++17) ve consteval (C++20)

cppreference.com/w/cpp/language/lambda belgeliyor: C++17'den beri Lambda'lar gövdesi constexpr gereksinimlerini karşılıyorsa otomatik olarak constexpr'dir. Lambda'lar ile DERLEME ZAMANINDA değer hesaplayabilirsiniz — sıfır çalışma zamanı maliyeti. Sınırlı flash'e sahip otomotiv ECU'larında bu, arama tabloları, CRC sağlama toplamları ve protokol sabitlerinin derleme zamanında Lambda ile hesaplanabileceği anlamına gelir.
C++20, derleme zamanında çalışMASI GEREKEN consteval Lambda'lara izin verir.

Kaynak Kod / Source Code

```cpp
#include <algorithm>  // std::sort, std::for_each, std::find_if, std::transform
#include <cstdint>    // uint8_t, uint16_t
#include <functional> // std::function
#include <iostream>
#include <numeric>    // std::accumulate
#include <string>
#include <vector>

// ─── Automotive Helper Structures ──────────────────────────────────────────────

// EN: Diagnostic Trouble Code (DTC) — core data in any automotive ECU.
// TR: Arıza Teşhis Kodu (DTC) — her otomotiv ECU'sunun temel verisi.
struct DTC {
  uint16_t code;        // e.g. 0x0171
  std::string name;     // e.g. "EGT Sensor Open Circuit"
  uint8_t severity;     // 1 = info, 2 = warning, 3 = critical, 4 = fatal
  bool active;          // currently present?
};

// EN: Sensor reading from a vehicle bus (CAN/LIN).
// TR: Araç veri yolundan (CAN/LIN) gelen sensör okuması.
struct SensorReading {
  std::string sensor;
  double value;
  std::string unit;
};

int main() {
  std::cout << "=== MODULE 3/02: LAMBDA EXPRESSIONS — FULL GUIDE ===\n"
            << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // 1. THE SIMPLEST LAMBDA / EN BASIT LAMBDA
  // ═══════════════════════════════════════════════════════════════════════════
  // EN: A Lambda with no capture, no parameters. Just prints a message.
  // TR: Yakalama yok, parametre yok. Sadece bir mesaj yazdırır.

  auto greet = []() {
    std::cout << "1. ECU Boot: Lambda initialized successfully!" << std::endl;
  };
  // EN: Call the Lambda
  // TR: Lambda'yı çağır
  greet();
```

```cpp

    // EN: Even shorter — omit () when there are no parameters:
    // TR: Daha kısa — parametre yoksa () atlanabilir:
    auto greetShort = [] {
      std::cout << "   (Same Lambda without parentheses)\n" << std::endl;
    };
    greetShort();

    // ═══════════════════════════════════════════════════════════════════════════════
    // 2. LAMBDA WITH PARAMETERS / PARAMETRELI LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Lambdas accept parameters just like regular functions. Here we convert a raw ADC value
    // to a temperature (automotive formula).
    // TR: Lambda'lar normal fonksiyonlar gibi parametre alır. Ham ADC değerini sıcaklığa çeviren
    // otomotiv formülü.

    auto adcToTemperature = [](int rawAdc) -> double {
      // EN: Simplified EGT sensor formula: T = (raw * 0.1) - 40.0
      // TR: Basitleştirilmiş EGT sensör formülü: T = (raw * 0.1) - 40.0
      return (rawAdc * 0.1) - 40.0;
    };

    int rawValue = 850;
    std::cout << "2. ADC Raw=" << rawValue
              << " -> Temperature=" << adcToTemperature(rawValue)
              << " C\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 3. CAPTURE BY VALUE [=] / DEGER ILE YAKALAMA [=]
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: The Lambda copies the outside variable's VALUE at the moment of creation. Changes to the
    // original do NOT affect the Lambda's copy.
    // TR: Lambda oluşturulma anında dış değişkenin DEĞERİNİ kopyalar. Orijinaldeki değişiklikler
    // Lambda'nın kopyasını ETKİLEMEZ.

    // EN: Critical and above
    // TR: Kritik ve üzeri
    int severityThreshold = 3;
    auto isCritical = [=](const DTC& dtc) {
      return dtc.severity >= severityThreshold && dtc.active;
    };

    // EN: Even if we change threshold AFTER creating the Lambda, the Lambda still sees the OLD
    // value (3), because it captured a COPY.
    // TR: Lambda'yı oluşturduktan SONRA threshold'u değiştirsek bile, Lambda HÂL ESKİ değeri (3)
    // görür, çünkü KOPYAyı yakaladı.
    severityThreshold = 999;

    DTC testDtc{0x0171, "EGT Sensor Open", 4, true};
    std::cout << "3. Capture by value: DTC 0x0171 critical? "
              << (isCritical(testDtc) ? "YES" : "NO")
              << " (threshold was 3 at capture time)\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 4. CAPTURE BY REFERENCE [&] / REFERANS ILE YAKALAMA [&]
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: The Lambda gets a REFERENCE to the outside variable. Any change the Lambda makes
    // modifies the ORIGINAL. Also, if the outside changes, the Lambda sees the new value.
    // TR: Lambda dış değişkene REFERANS alır. Lambda'nın yaptığı her değişiklik ORİJİNALİ
    // değiştirir. Dışarıda değişiklik olursa Lambda yeni değeri görür.

    int activeFaultCount = 0;
    std::vector<DTC> faultLog = {
        {0x0100, "Coolant Temp High",   3, true},
        {0x0200, "Oil Pressure Low",    4, true},
        {0x0300, "Battery Voltage",     2, false},
        {0x0400, "Throttle Pos Sensor", 3, true},
        {0x0500, "O2 Sensor Circuit",   2, true},
        {0x0600, "Knock Sensor Range",  4, false},
        {0x0700, "Misfire Cylinder 2",  3, true},
    };
```

```cpp

    // EN: Count active faults by capturing counter BY REFERENCE.
    // TR: Sayacı REFERANS ile yakalayarak aktif arızaları say.
    std::for_each(faultLog.begin(), faultLog.end(),
                  [&activeFaultCount](const DTC& dtc) {
      if (dtc.active) {
        activeFaultCount++;
      }
    });
    std::cout << "4. Capture by reference: Active faults = "
              << activeFaultCount << "\n" << std::endl;

    // ==========================================================================================
    // 5. MIXED CAPTURE [=, &x] / KARISIK YAKALAMA [=, &x]
    // ==========================================================================================
    // EN: Best practice — capture everything by value for safety, but explicitly capture specific
    // variables by reference when you need to modify them.
    // TR: En iyi pratik — güvenlik için her şeyi değer ile yakala, ama değiştirmen gereken belirli
    // değişkenleri açıkça referans ile yakala.

    int criticalCount = 0;
    int warningCount = 0;
    int critThreshold = 3;

    std::for_each(faultLog.begin(), faultLog.end(),
                  [=, &criticalCount, &warningCount](const DTC& dtc) {
      if (!dtc.active) return;
      if (dtc.severity >= critThreshold) {
        // EN: Modified via reference
        // TR: Referansla değiştirilir
        criticalCount++;
      } else {
        warningCount++;
      }
    });
    std::cout << "5. Mixed capture: Critical=" << criticalCount
              << ", Warning=" << warningCount << "\n" << std::endl;

    // ==========================================================================================
    // 6. MUTABLE LAMBDA / DEGISTIRILEBILIR LAMBDA
    // ==========================================================================================
    // EN: By default, a Lambda captured by VALUE cannot modify its copies (they are const). The
    // 'mutable' keyword lifts this restriction. Note: It modifies the COPY, not the original!
    // TR: Varsayılan olarak DEĞER ile yakalanan kopya const'tır, değiştirilemez. 'mutable' bu
    // kısıtlamayı kaldırır. Not: KOPYAyı değiştirir, orijinali DEĞİL!

    int sequenceId = 1000;
    auto generateId = [sequenceId]() mutable -> int {
      // EN: Each call increments the Lambda's internal copy.
      // TR: Her çağrıda Lambda'nın iç kopyası artar.
      return sequenceId++;
    };

    std::cout << "6. Mutable Lambda (sequence generator):" << std::endl;
    std::cout << "   ID: " << generateId() << std::endl; // 1000
    std::cout << "   ID: " << generateId() << std::endl; // 1001
    std::cout << "   ID: " << generateId() << std::endl; // 1002
    std::cout << "   Original sequenceId still = " << sequenceId
              << "\n" << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // 7. INIT CAPTURE / MOVE CAPTURE (C++14) / BASLATICI YAKALAMA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: C++14 allows creating new variables inside the capture list itself. This is essential
    // for MOVE SEMANTICS — you can move a unique_ptr or a large buffer into the Lambda without
    // copying.
    // TR: C++14 yakalama listesi içinde yeni değişken oluşturmaya izin verir. Bu, TAŞIMA SEMANTİĞİ
    // için kritiktir — unique_ptr veya büyük bir tampon belleği kopyalamadan Lambda'ya
    // taşıyabilirsiniz.

    std::string rawFrame = "CAN_ID:0x7E8 DATA:41 0C 1A F0";
    auto processFrame = [frame = std::move(rawFrame)]() {
      std::cout << "7. Init-capture (moved): Processing frame: "
                << frame << std::endl;
    };
    processFrame();
    std::cout << "   Original rawFrame is now empty: \""
              << rawFrame << "\"\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 8. LAMBDA WITH STL ALGORITHMS / STL ALGORITMALARI ILE LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: This is where Lambdas truly shine in automotive software.
    // TR: Lambda'ların otomotiv yazılımında gerçekten parladığı yer.

    // --- 8a. std::sort — Sort DTCs by severity (descending) ---
    // EN: Custom sorting: highest severity first for prioritized display.
    // TR: Özel sıralama: öncelikli görüntüleme için en yüksek önem derecesi önce.
    std::vector<DTC> sortedFaults = faultLog;
    std::sort(sortedFaults.begin(), sortedFaults.end(),
              [](const DTC& a, const DTC& b) {
      return a.severity > b.severity;
    });
    std::cout << "8a. std::sort by severity (descending):" << std::endl;
    for (const auto& dtc : sortedFaults) {
      std::cout << "    [" << static_cast<int>(dtc.severity) << "] 0x"
                << std::hex << dtc.code << std::dec
                << " " << dtc.name << std::endl;
    }

    // --- 8b. std::find_if — Find first fatal DTC ---
    // EN: Search for the first DTC with severity == 4 (fatal).
    // TR: Önem derecesi == 4 (ölümcül) olan ilk DTC'yi ara.
    auto it = std::find_if(faultLog.begin(), faultLog.end(),
                           [](const DTC& dtc) {
      return dtc.severity == 4;
    });
    if (it != faultLog.end()) {
      std::cout << "\n8b. std::find_if: First fatal DTC -> 0x"
                << std::hex << it->code << std::dec
                << " " << it->name << std::endl;
    }

    // --- 8c. std::transform — Extract fault names into a string vector ---
    // EN: Map DTCs to their names (functional-style projection).
    // TR: DTC'leri isimlerine dönüştür (fonksiyonel-stil projeksiyon).
    std::vector<std::string> faultNames(faultLog.size());
    std::transform(faultLog.begin(), faultLog.end(), faultNames.begin(),
                   [](const DTC& dtc) { return dtc.name; });
    std::cout << "\n8c. std::transform: DTC names extracted ("
              << faultNames.size() << " items)" << std::endl;

    // --- 8d. std::accumulate — Sum all severity scores ---
    // EN: Total severity score across all DTCs (risk assessment).
    // TR: Tüm DTC'lerin toplam önem puanı (risk değerlendirmesi).
    int totalSeverity = std::accumulate(
        faultLog.begin(), faultLog.end(), 0,
        [](int sum, const DTC& dtc) {
      return sum + static_cast<int>(dtc.severity);
    });
    std::cout << "8d. std::accumulate: Total severity score = "
              << totalSeverity << "\n" << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // 9. GENERIC LAMBDA (C++14 auto) / JENERIK LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Use 'auto' parameters to create Lambdas that work with ANY type. The compiler generates
    // a templated operator() behind the scenes.
    // TR: HERHANGİ bir tip ile çalışan Lambda'lar oluşturmak için 'auto' parametre kullanın.
    // Derleyici arka planda şablonlu operator() üretir.

    auto printReading = [](const auto& r) {
      std::cout << "   Sensor: " << r.sensor
                << " = " << r.value << " " << r.unit << std::endl;
    };

    std::vector<SensorReading> readings = {
        {"Coolant Temp",  92.5, "C"},
        {"Engine RPM",  3500.0, "rpm"},
        {"Battery",      13.8, "V"},
        {"Fuel Pressure", 3.2, "bar"},
    };

    std::cout << "9. Generic Lambda (auto) — Sensor Readings:" << std::endl;
    std::for_each(readings.begin(), readings.end(), printReading);
    std::cout << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 10. std::function — STORING LAMBDAS / LAMBDA SAKLAMA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: auto is great for local use, but if you need to store a Lambda in a class member,
    // container, or pass it through an API, use std::function. Trade-off: small overhead (type
    // erasure + possible heap allocation).
    // TR: auto yerel kullanım için harikadır, ama Lambda'yı sınıf üyesinde, container'da
    // saklamanız veya API'den geçirmeniz gerekirse std::function kullanın. Takas: küçük maliyet
    // (tip silme + olası heap tahsisi).

    // EN: ECU callback registry pattern — store different validation rules.
    // TR: ECU geri-çağırma kayıt deseni — farklı doğrulama kurallarını sakla.
    std::vector<std::function<bool(const DTC&)>> validators;

    validators.push_back([](const DTC& d) { return d.active; });
    validators.push_back([](const DTC& d) { return d.severity >= 3; });
    validators.push_back([](const DTC& d) { return d.code < 0x0500; });

    DTC sample{0x0400, "Throttle Pos Sensor", 3, true};
    std::cout << "10. std::function validators for DTC 0x0400:" << std::endl;
    int passCount = 0;
    for (size_t i = 0; i < validators.size(); ++i) {
      bool result = validators[i](sample);
      std::cout << "    Validator " << (i + 1) << ": "
                << (result ? "PASS" : "FAIL") << std::endl;
      if (result) passCount++;
    }
    std::cout << "    Passed " << passCount << "/"
              << validators.size() << "\n" << std::endl;
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// 11. LAMBDA AS FUNCTION PARAMETER (HIGHER-ORDER FUNCTIONS) LAMBDA'YI FONKSIYON PARAMETRESI
// OLARAK GECIRME
// ═══════════════════════════════════════════════════════════════════════════════
// EN: You can write your own functions that ACCEPT Lambdas as parameters. This is the
// "Higher-Order Function" pattern — a function that takes another function as an argument.
// Using templates avoids std::function overhead.
// TR: Lambda'ları parametre olarak KABUL EDEN kendi fonksiyonlarınızı yazabilirsiniz. Bu
// "Yüksek Dereceli Fonksiyon (Higher-Order Function)" desenidir. Template kullanmak
// std::function maliyetinden kaçınır.

// EN: Using a Lambda with std::count_if for filtered counting.
// TR: Filtrelenmiş sayma için std::count_if ile Lambda kullanımı.
auto activeCount = std::count_if(faultLog.begin(), faultLog.end(),
                                 [](const DTC& d) { return d.active; });
auto severeActive = std::count_if(faultLog.begin(), faultLog.end(),
                                  [](const DTC& d) {
  return d.active && d.severity >= 3;
});
std::cout << "11. Higher-order: Active DTCs = " << activeCount
          << ", Severe+Active = " << severeActive << "\n" << std::endl;

// ═══════════════════════════════════════════════════════════════════════════════
// 12. IMMEDIATELY INVOKED LAMBDA EXPRESSION (IILE) ANINDA CAGIRILAN LAMBDA IFADESI
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Call a Lambda immediately at the point of definition. Useful for complex initialization
// of const variables.
// TR: Lambda'yı tanımlandığı noktada hemen çağırın. const değişkenlerin karmaşık başlatılması
// için kullanışlıdır.

const std::string ecuStatus = [&]() -> std::string {
  int fatal = 0;
  for (const auto& d : faultLog) {
    if (d.active && d.severity == 4) fatal++;
  }
  if (fatal > 0) return "LIMP MODE";
  if (activeFaultCount > 3) return "DEGRADED";
  return "NORMAL";
}(); // EN: Note the () at the end — immediate invocation!
     // TR: Sondaki () — hemen çağırma!

std::cout << "12. IILE: ECU Status = " << ecuStatus << "\n" << std::endl;

// ═══════════════════════════════════════════════════════════════════════════════
// 13. CONSTEXPR LAMBDA (C++17) / DERLEME-ZAMANI LAMBDA
// ═══════════════════════════════════════════════════════════════════════════════
// EN: Since C++17, Lambdas can be constexpr — computed at compile time. Perfect for automotive
// look-up tables and protocol constants.
// TR: C++17'den beri Lambda'lar constexpr olabilir — derleme zamanında hesaplanır. Otomotiv
// arama tabloları ve protokol sabitleri için ideal.

constexpr auto crcByte = [](uint8_t data) constexpr -> uint8_t {
  // EN: Simplified CRC-8 step for demonstration.
  // TR: Gösterim amaçlı basitleştirilmiş CRC-8 adımı.
  uint8_t crc = data;
  for (int i = 0; i < 8; ++i) {
    if (crc & 0x80U) {
      crc = static_cast<uint8_t>((crc << 1U) ^ 0x07U);
    } else {
      crc = static_cast<uint8_t>(crc << 1U);
    }
  }
  return crc;
};

constexpr uint8_t checksum = crcByte(0xA5);
std::cout << "13. constexpr Lambda: CRC-8(0xA5) = 0x"
          << std::hex << static_cast<int>(checksum) << std::dec
          << " (computed at compile time!)\n" << std::endl;
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // 14. RECURSIVE LAMBDA (via std::function) / OZYINELEMELI LAMBDA
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Lambdas cannot directly call themselves (they have no name). To make a recursive Lambda,
    // wrap it in std::function so it can reference itself through the captured variable.
    // TR: Lambda'lar kendilerini doğrudan çağıramaz (isimleri yoktur). Özyinelemeli Lambda yapmak
    // için std::function ile sarmalayın, böylece yakalanan değişken üzerinden kendine referans
    // verebilir.

    // EN: Compute checksum of a DTC code tree (recursive severity sum).
    // TR: DTC kodu ağacının sağlama toplamını hesapla (özyinelemeli önem toplamı).
    std::function<int(const std::vector<DTC>&, size_t)> recursiveSum =
        [&recursiveSum](const std::vector<DTC>& dtcs, size_t idx) -> int {
      if (idx >= dtcs.size()) return 0;
      return static_cast<int>(dtcs[idx].severity) +
             recursiveSum(dtcs, idx + 1);
    };

    std::cout << "14. Recursive Lambda: Total severity (recursive) = "
              << recursiveSum(faultLog, 0) << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // 15. LAMBDA RETURNING A LAMBDA (FACTORY PATTERN) LAMBDA DONDUREN LAMBDA (FABRIKA DESENI)
    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: A Lambda can return ANOTHER Lambda. This is a powerful functional programming pattern —
    // creating specialized filters at runtime.
    // TR: Bir Lambda BAŞKA bir Lambda döndürebilir. Bu güçlü bir fonksiyonel programlama desenidir
    // — çalışma zamanında özelleşmiş filtreler oluşturur.

    auto makeSeverityFilter = [](uint8_t minSev) {
      return [minSev](const DTC& dtc) {
        return dtc.active && dtc.severity >= minSev;
      };
    };

    auto criticalFilter = makeSeverityFilter(4); // fatal only
    auto warningFilter = makeSeverityFilter(2);  // warning and above

    std::cout << "15. Lambda factory:" << std::endl;
    std::cout << "    Oil Pressure Low fatal? "
              << (criticalFilter(faultLog[1]) ? "YES" : "NO") << std::endl;
    std::cout << "    Battery Voltage warning+? "
              << (warningFilter(faultLog[2]) ? "YES" : "NO")
              << " (inactive, so NO)\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // SUMMARY / OZET
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "=== LAMBDA CHEAT SHEET ===" << std::endl;
    std::cout << "  [](){}        — Simplest Lambda" << std::endl;
    std::cout << "  [=]           — Capture all by value" << std::endl;
    std::cout << "  [&]           — Capture all by reference" << std::endl;
    std::cout << "  [x, &y]       — Explicit mixed capture" << std::endl;
    std::cout << "  [x=move(v)]   — Init/move capture (C++14)" << std::endl;
    std::cout << "  mutable       — Modify value-captured copies" << std::endl;
    std::cout << "  auto params   — Generic Lambda (C++14)" << std::endl;
    std::cout << "  constexpr     — Compile-time Lambda (C++17)" << std::endl;
    std::cout << "  std::function — Type-erased storage" << std::endl;
    std::cout << "  IILE [](){}() — Immediate invocation" << std::endl;
    std::cout << "\nAll examples use Automotive/Embedded domain data." << std::endl;

    return 0;
}
```

► 03. Move Semantics Rvalue

module_03_modern_cpp/03_move_semantics_rvalue.cpp

Konu: Modern C++: Move Semantics & R-Values / Taşıma Semantiği ve L-Value vs R-Value (C++11

Teori / Theory

THEORY: Why are C++03 and C++11 practically "different languages"?

Before C++11, returning a HUGE object (like a 4GB Vector) from a function forced the compiler to perfectly COPY all 4GB of data byte by byte to the new variable, and then immediately DESTROY the old 4GB vector inside the function.
C++11 introduced "Move Semantics" (std::move). Instead of copying 4GB, it steals the internal pointer of the old vector, gives it to the new one, and leaves the old one empty. Cost: O(N) -> O(1). 4GB data copied in 1 nanosecond!

TEORİ: Büyük C++ Atılımı

C++11 öncesinde, bir fonksiyonun içinden devasa büyüklükte (örneğin 4 GB'lik) bir `vector` döndürdüğünüzde, derleyici bu 4 GB'ı bayt bayt yeni değişkene KOPYALAR (RAM ikiye katlanır), sonra fonksiyon içindeki o orijinal 4 GB'ı ÇÖPE ATARDI. Bu tam bir performans katliamıydı.
C++11 "Move Semantics" (Taşıma Semantiği) getirdi. Artık 4 GB'ı kopyalamıyoruz. "Madem fonksiyonun içindeki nesne birazdan ölecek, onun içindeki işçiyi (pointer'ı) ÇALALIM (Steal), yeni değişkene teslim edelim, ölecek olanın içini boşaltıp (null) cenazesini kaldıralım." Maliyet: O(N)'den -> O(1)'e inmiştir. 1 Nanosaniyede 4 GB veri taşınır!

THEORY: L-Value vs R-Value

- L-Value (Left Value): Has a permanent memory address, a name, and can sit on the left of an equal sign. (`int x = 5;` -> 'x' is an L-Value). - R-Value (Right Value): A temporary, nameless object born to die. (`5` or the unassigned return of `factory()` is an R-Value). - L-Value Reference (`&`): Classic reference. - R-Value Reference (`&&`): C++11 feature. Specifically catches TEMPORARY, DYING objects so you can "Steal/Move" their internals!

TEORİ: Sol Taraf vs Sağ Taraf Değeri

- L-Value (Left Value): RAM'de yer edinen, bir ismi (adresi) olan ve eşittirin solunda olabilen nesne. (`int x = 5;` -> 'x' bir L-Value'dir). - R-Value (Right Value): Geçici, ismi olmayan, şipşak yaratılıp ölen nesne. (`5` veya `factory()` fonksiyonunun havada kalan dönüşü R-Value'dir). - L-Value Reference (`&`): Klasik C++ referansı (Sadece kalıcıları tutar). - R-Value Reference (`&&`): C++11 Devrimi! GEÇİCİ, ÖLECEK olan nesneleri yakalar ve onların donanımlarını "çalmanızı/taşımanızı" sağlar!

CPPREF Depth: The Secret of `std::move` - It DOES NOT move anything!

CppReference warns developers continuously: `std::move()` DOES NOT ACTUALLY MOVE DATA. Yes, really. It is just an unconditional `static_cast<T&&>(obj)`. It merely "re-labels" an L-Value into an R-Value explicitly, so the compiler is "tricked" into calling the Move Constructor instead of the Copy Constructor. The actual moving/stealing is done inside the Move Constructor!

CPPREF Derinlik: `std::move`'un Sırrı - Aslında Hiçbir Şeyi TAŞIMAZ!

CppReference'in en büyük sırrı: `std::move()` ASLA BİR ŞEY TAŞIMAZ! Evet.. Okyanus kadar veriniz olsa da, o sadece şipşak bir `static_cast<T&&>(obje)` çevirimidir. Yaptığı tek iş bir L-Value nesnesinin etiketini söküp "Bu artık R-Value (Ölecek) bir nesnedir" yazmaktır. Böylece derleyici kandırılır ve (Kopya) yerine (Taşıma) fonksiyonunu çağırır. Asıl hırsızlık/taşıma Move Constructor'ın süslü parantezlerinin içinde yapılır!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class HeavyBuffer {
private:
  int size;
  int *data;

public:
  // [1] Normal Constructor / Normal Yapıcı
  HeavyBuffer(int bufferSize) : size(bufferSize) {
    data = new int[size];
    std::cout << "[Constructor] Allocated heavy data of size: " << size << " (Ağır veri üretildi.)"
        << std::endl;
  }

  // [2] DESTRUCTOR / YIKICI
  ~HeavyBuffer() {
    std::cout << "[Destructor] Memory wiped. Size: " << size << " (Bellek yıkıldı.)" << std::endl;
    delete[] data;
  }

  // [3] COPY CONSTRUCTOR (The Old, Slow, Expensive Way / Eski Yöntem - Yavaş ve
  // Pahalı)
  HeavyBuffer(const HeavyBuffer &other) : size(other.size) {
    std::cout << "\033[31m[COPY Constructor]\033[0m COPYING BYTE BY BYTE! Too "
                 "heavy... (Satır satır kopyalanıyor!)"
              << std::endl;
    data = new int[size];
    for (int i = 0; i < size; ++i)
      data[i] = other.data[i];
  }

  // [4] MOVE CONSTRUCTOR (The C++11 Revolution using R-Value Reference '&&')
  // EN: "other" is a temporary object. So we DO NOT make it "const". We will empty its guts
  // (Steal).
  // TR: "other" geçici bir nesnedir. O yüzden "const" YAPMIYORUZ, içini boşaltacağız
  // (Hırsızlık).
  HeavyBuffer(HeavyBuffer &&other) noexcept
      : size(other.size), data(other.data) {

    // [STEP 1]: We stole 'other's internal pointer and adopted it! (Zero
    // Copying) [ADIM 1]: 'other'ın pointerını ('data') çaldık ve kendimize verdik. O(1) hız!
    std::cout << "\033[32m[MOVE Constructor]\033[0m DATA STOLEN (Moved)! O(1) "
                 "Speed... (Veri çalındı!)"
              << std::endl;

    // [STEP 2]: POST-THEFT. We MUST nullify 'other' so when it dies, it doesn't
    // delete OUR stolen memory. [ADIM 2]: HIRSIZLIK SONRASI. 'other'ı nullptr yapmalıyız ki, o
    // yıkılırken bizim çalınan RAM'imizi çökertmesin.
    other.data = nullptr;
    other.size = 0;
  }
};
```

```cpp

/**
 * @brief EN: A factory function creating a massive temporary memory chunk.
 * TR: Devasa geçici bir bellek üreten fonksiyon.
 */
HeavyBuffer factory() {
  HeavyBuffer temp(1000000); // 1 Million elements / 1 Milyon eleman
  // EN: Temp is an R-Value! Returns a dying object.
  // TR: Temp fonksiyondan çıkarken ölür! (R-Value)
  return temp;
}

// ═══════════════════════════════════════════════════════════════════════════════
// EN: PERFECT FORWARDING — Passing arguments without losing their value category
// TR: MÜKEMMEL YÖNLENDİRME — Argümanları değer kategorisini kaybetmeden aktarma
// ═══════════════════════════════════════════════════════════════════════════════

// ─── 1. Helper Overloads for Forwarding Demo ─────────────────────────────────
// EN: Two overloads that reveal whether the caller passed an L-value or R-value.
// TR: Çağıranın L-value mi yoksa R-value mi gönderdiğini ortaya çıkaran iki aşırı yükleme.

void process(const std::string& signal) {
    // EN: Binds to L-values (named, persistent objects).
    // TR: L-value'lere (isimli, kalıcı nesnelere) bağlanır.
    std::cout << "  [process] L-value received: " << signal << std::endl;
}

void process(std::string&& signal) {
    // EN: Binds to R-values (temporaries, dying objects).
    // TR: R-value'lere (geçici, ölecek nesnelere) bağlanır.
    std::cout << "  [process] R-value received: " << signal << std::endl;
}

// ─── 2. Forwarding Reference (Universal Reference) ───────────────────────────
// EN: T&& in a template context is NOT an rvalue reference — it is a "forwarding reference"
// (also called universal reference). It deduces T as lvalue-ref or rvalue-ref depending on what
// the caller passes. std::forward<T>(arg) preserves the original value category.
// TR: Şablon bağlamında T&& bir rvalue referansı DEĞİLDİR — "yönlendirme referansı" (evrensel
// referans) adını alır. T, çağıranın gönderdiğine göre lvalue-ref veya rvalue-ref olarak
// çıkarılır. std::forward<T>(arg) orijinal değer kategorisini korur.

template <typename T>
void relay(T&& arg) {
    // EN: std::forward keeps lvalue as lvalue, rvalue as rvalue.
    // TR: std::forward, lvalue'yi lvalue, rvalue'yi rvalue olarak korur.
    process(std::forward<T>(arg));
}

// ─── 3. SensorPacket — Variadic Perfect Forwarding ───────────────────────────
// EN: Automotive ECU sensor data packet built via factory with zero unnecessary copies.
// TR: Otomotiv ECU sensör veri paketi — fabrika ile sıfır gereksiz kopya ile oluşturulur.

class SensorPacket {
private:
    std::string sensor_id_;
    double value_;
    std::string unit_;

public:
    SensorPacket(std::string id, double val, std::string unit)
        : sensor_id_(std::move(id)), value_(val), unit_(std::move(unit)) {
        std::cout << "  [SensorPacket] Constructed: " << sensor_id_
                  << " = " << value_ << " " << unit_ << std::endl;
    }

    friend std::ostream& operator<<(std::ostream& os, const SensorPacket& sp) {
        os << "[" << sp.sensor_id_ << ": " << sp.value_ << " " << sp.unit_ << "]";
        return os;
    }
};
```

```cpp

// EN: Variadic template factory — every argument is perfectly forwarded into the SensorPacket
// constructor. No extra copy or move happens.
// TR: Değişken sayıda argümanlı şablon fabrikası — her argüman SensorPacket yapıcısına mükemmel
// şekilde yönlendirilir. Fazladan kopya/taşıma olmaz.

template <typename... Args>
SensorPacket createSensorPacket(Args&&... args) {
    return SensorPacket(std::forward<Args>(args)...);
}

// ═══════════════════════════════════════════════════════════════════════════════

int main() {
  std::cout << "=== MODULE 3: MOVE SEMANTICS & R-VALUE (&&) ===\n" << std::endl;

  std::cout << "\n--- TEST 1: THE OLD C++ COPY (Wasted Power/Sokağa Atılan Güç) ---" << std::endl;
  HeavyBuffer bufferOne(50000);

  // EN: bufferOne is an L-Value (has a name). The compiler IS FORCED to perform a deep COPY.
  // TR: bufferOne bir L-Value'dir (ismi var). Derleyici MECBUREN derin Satır Satır Kopyalama
  // Yapar.
  HeavyBuffer copyBuffer = bufferOne;

  std::cout << "\n--- TEST 2: USING STD::MOVE (Forcible move / Zorla Taşıma) ---" << std::endl;
  // EN: Sometimes, even though bufferOne is an L-Value, we want to say "I'm done with you,
  // transfer to someone else". `std::move(bufferOne)` casts it to an R-Value (`&&`), triggering
  // the MOVE Constructor!
  // TR: Bazen bufferOne L-Value olsa da "Öl, donanımını yeniye aktar" demek isteriz.
  // `std::move(bufferOne)`, objeyi zorla R-Value'ye (`&&`) dönüştürür.
  HeavyBuffer movedBuffer = std::move(bufferOne);
  // WARNING (DİKKAT)! bufferOne is now an empty "shell". Its data is stolen. Do not use it
  // again!

  std::cout << "\n--- TEST 3: FACTORY (R-Value Passage / Fabrika R-Value Geçişi) ---" << std::endl;
  // EN: factory() returns a nameless (R-Value) object. Compiler detects this and triggers Move!
  // TR: factory() isimsiz geçici (R-Value) üretir. Derleyici bunu görür ve hemen Move (Taşıma)
  // tetikler!
  HeavyBuffer fastBuffer = factory();

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: Demo 4 — Perfect Forwarding: relay() dispatches to correct overload
  // TR: Demo 4 — Mükemmel Yönlendirme: relay() doğru aşırı yüklemeyi çağırır
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- DEMO 4: PERFECT FORWARDING (Mükemmel Yönlendirme) ---" << std::endl;

  std::string canSignal = "CAN_SPEED_0x1A3";

  // EN: canSignal is an L-value → relay deduces T = std::string&
  // TR: canSignal bir L-value → relay, T = std::string& olarak çıkarır
  std::cout << "  Passing L-value:" << std::endl;
  relay(canSignal);

  // EN: std::string{...} produces a temporary R-value → relay deduces T = std::string
  // TR: std::string{...} geçici R-value üretir → relay, T = std::string olarak çıkarır
  std::cout << "  Passing R-value:" << std::endl;
  relay(std::string{"CAN_RPM_0x0C6"});
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 5 — Universal Reference vs Rvalue Reference
    // TR: Demo 5 — Evrensel Referans ile Rvalue Referansı Farkı
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 5: UNIVERSAL REF vs RVALUE REF ---" << std::endl;

    // EN: KEY DISTINCTION: template<typename T> void foo(T&& x);   // <-- UNIVERSAL (forwarding)
    // ref void bar(std::string&& x);               // <-- plain RVALUE reference
    //
    // Universal ref (T&&) can bind to BOTH lvalue AND rvalue. Plain rvalue ref (Type&&) can ONLY
    // bind to rvalue.
    //
    // TR: KRİTİK FARK: template<typename T> void foo(T&& x);   // <-- EVRENSEL referans void
    // bar(std::string&& x);               // <-- düz RVALUE referansı
    //
    // Evrensel referans (T&&) hem lvalue hem rvalue'ya bağlanabilir. Düz rvalue ref (Tip&&)
    // YALNIZCA rvalue'ya bağlanabilir.

    // EN: Demonstrating that relay (universal ref) accepts both:
    // TR: relay (evrensel ref) her ikisini de kabul eder:
    relay(canSignal);                          // L-value OK
    relay(std::string{"TEMP_COOLANT_0xF4"});   // R-value OK

    // EN: A plain rvalue ref function would REJECT canSignal (l-value).
    // TR: Düz rvalue ref fonksiyon canSignal'i (l-value) REDDEDERDİ.
    std::cout << "  (Plain rvalue ref cannot accept L-values — compile error!)" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 6 — Variadic Perfect Forwarding: createSensorPacket factory
    // TR: Demo 6 — Değişken Argümanlı Mükemmel Yönlendirme: Fabrika
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 6: VARIADIC PERFECT FORWARDING (Fabrika) ---" << std::endl;

    // EN: All arguments are forwarded directly into the SensorPacket constructor.
    // TR: Tüm argümanlar doğrudan SensorPacket yapıcısına yönlendirilir.
    auto throttle  = createSensorPacket("THROTTLE_POS", 73.5, "percent");
    auto oilTemp   = createSensorPacket(std::string{"OIL_TEMP"}, 92.1, std::string{"celsius"});

    std::cout << "  Packet 1: " << throttle << std::endl;
    std::cout << "  Packet 2: " << oilTemp  << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 7 — std::forward vs std::move: when to use which?
    // TR: Demo 7 — std::forward vs std::move: hangisi ne zaman kullanılır?
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 7: std::forward vs std::move ---" << std::endl;

    // EN: RULE OF THUMB: • std::move  → Use when you OWN the object and are DONE with it.
    // Unconditionally casts to rvalue. The source becomes empty. • std::forward → Use ONLY inside
    // templates with forwarding references (T&&). Conditionally casts: preserves lvalue/rvalue
    // category.
    //
    // TR: ALTIN KURAL: • std::move  → Nesneye SAHİPSENİZ ve artık İŞİNİZ BİTTİYSE kullanın.
    // Koşulsuz olarak rvalue'ya dönüştürür. Kaynak boşalır. • std::forward → YALNIZCA T&&
    // yönlendirme referanslı şablonlarda kullanın. Koşullu dönüştürme yapar: lvalue/rvalue
    // kategorisini korur.

    std::string ecu_msg = "ECU_DIAG_REQUEST";
```

```cpp

    // EN: std::move — we are transferring ownership deliberately.
    // TR: std::move — sahipliği kasıtlı olarak devrediyoruz.
    std::string transferred = std::move(ecu_msg);
    std::cout << "  After std::move, original is empty: '" << ecu_msg << "'" << std::endl;
    std::cout << "  Transferred holds: '" << transferred << "'" << std::endl;

    // EN: std::forward — only meaningful inside relay() / template context. Outside templates,
    // prefer std::move for owned objects.
    // TR: std::forward — yalnızca relay() / şablon bağlamında anlamlıdır. Şablonların dışında,
    // sahip olduğunuz nesneler için std::move tercih edin.
    std::cout << "  (std::forward is used inside relay() — see Demo 4 output)" << std::endl;

    std::cout << "\n=== Program Ending (Destructors Will Run / Yıkıcılar "
                 "Çalışacak) ===\n"
              << std::endl;

    return 0;
}
```

► 04. Modern Types

module_03_modern_cpp/04_modern_types.cpp

Konu: Modern C++: New Data Types (C++17) / Modern Veri Tipleri: optional, variant, any,

Teori / Theory

THEORY: Why Do We Lose Points in Code Reviews?

In old C++ code, if a function failed, it usually returned "-1", "NULL" or "nullptr" (Magic Numbers). This is dangerous because what if "-1" is the ACTUAL valid answer? With C++17, things became extremely safe, similar to Rust.

TEORİ: Neden Puan Kaybederiz?

Eski C++ kodlarında bir fonksiyon başarılı olamazsa genellikle "-1", "NULL" veya "nullptr" döndürülürdü (Magic Numbers/Sihirli Sayılar). Bu tehlikelidir çünkü kişi gerçekten -1 değerini bulmuş olabilir! C++17 ile beraber işler "Rust" dillerine benzer şekilde muhteşem güvenli hale geldi.

bir değer YA VARDIR YA DA YOKTUR - nullopt). 2. std::variant<T, Y> : A type-safe version of C's "union". Holds only ONE type at a time but allocates memory equal to the BIGGEST mapped type (RAM saver). (C'deki "union"ın güvenli halidir. Aynı anda tek tip tutar, RAM cimrisidir.)

atarsan at" tipidir. Maliyetlidir, zorda kalınmadıkça kullanılmaz.) 4. std::string_view : A lightweight purely observational "window" to look at a string WITHOUT copying it! (Bir metni kopyalamadan -yeni RAM tüketmeden- SADECE BAKMA penceresidir.)

CPPREF Depth: auto Type Deduction Rules — Template Argument Deduction in Disguise

3. std::any : Holds "anything" (Like variables in Python or JS). Heavy overhead. ("İçine ne

`auto` uses the same rules as template argument deduction (§7.1.7.4 / §13.10.2). When you write `auto x = expr;`, the compiler deduces the type as if `x` were a function template parameter: references and top-level const/volatile are stripped. This means: • `const int& r = n; auto a = r;` → a is `int`, NOT `const int&`. • `auto x = {1,2,3};` deduces `std::initializer_list<int>` (special rule). • `auto x{42};` in C++17 deduces `int` (changed from init-list in C++11). `decltype(auto)` preserves the EXACT type including references: • `decltype(auto) y = r;` → y is `const int&`. Parenthesized id-expressions change decltype semantics: • `decltype(x)` → declared type of x. • `decltype((x))` → always an lvalue reference (T&), because (x) is an lvalue expression, not an id-expression! This subtle difference is a common source of bugs in return type deduction with `decltype(auto)` — returning `(localVar)` gives a dangling reference.

CPPREF Derinlik: auto Tür Çıkarım Kuralları — Şablon Argüman Çıkarımının Gizli Yüzü

`auto` şablon argüman çıkarımıyla (template argument deduction) aynı kuralları kullanır. `auto x = expr;` yazıldığında derleyici, x'i bir şablon parametresiymiş gibi çıkarır: referanslar ve üst-düzey const/volatile atılır. • `const int& r = n; auto a = r;` → a tipi `int`'tir, `const int&` DEĞİL. • `auto x = {1,2,3};` → `std::initializer_list<int>` çıkarılır.
• `auto x{42};` → C++17'de `int` (C++11'de init-list'ti, değişti). `decltype(auto)` referanslar dahil TAM türü korur: • `decltype(auto) y = r;` → y tipi `const int&`. Parantezli ifadeler decltype semantiğini değiştirir: • `decltype(x)` → x'in bildirim türü. • `decltype((x))` → her zaman lvalue referans (T&)! Çünkü (x) bir lvalue ifadesidir, id-expression değildir. Bu ince fark, `decltype(auto)` dönüş türüyle `(localVar)` döndürürken sarkan referans (dangling reference) hatasına yol açar.

Kaynak Kod / Source Code

```cpp
#include <any>
#include <iostream>
#include <optional>
#include <string>
#include <string_view>
#include <variant>

// 1. std::optional (C++17) Example: How to securely handle "Data Not Found" without MAGIC
// NUMBERS? (Veri bulamama durumu "-1" kullanılmadan nasıl çözülür?)
std::optional<int> findAgeByName(const std::string &name) {
  if (name == "Alice")
    return 25;
  if (name == "Bob")
    return 30;

  // EN: Instead of -1, we return 'nullopt' (no value)!
  // TR: Bulamadıysak -1 döndürmek yerine C++17'deki harika "nullopt" döneriz!
  return std::nullopt;
}

// 2. std::string_view (C++17) Example: Preventing massive RAM copies line by line!
// TR: Satır satır metin kopyalamayı önlemek! Eğer std::string alsaydık, o devasa metni RAM'e
// kopyalardık.
void printFirst5Chars(std::string_view text) {
  if (text.length() >= 5)
    std::cout << "[StringView] " << text.substr(0, 5) << "..." << std::endl;
}

int main() {
  std::cout << "=== MODULE 3: MODERN TYPES (C++17) ===\n" << std::endl;
```

```cpp

  // --- TEST 1: std::optional ---
  std::cout << "--- 1. std::optional ---" << std::endl;
  // Not in database / Veritabanında yok.
  auto ageResult = findAgeByName("Charlie");

  if (ageResult.has_value()) { // or just: if (ageResult)
    std::cout << "Age Found (Bulunan Yaş): " << ageResult.value() << std::endl;
  } else {
    std::cout << "Person DOES NOT EXIST (Bu isimde biri YOK. '-1' hatası "
                  "yapmadan güvenle çözüldü.)"
              << std::endl;
  }

  // --- TEST 2: std::variant ---
  std::cout << "\n--- 2. std::variant ---" << std::endl;
  // EN: An automotive diagnostic result. Can hold EITHER an int error code OR a string
  // description.
  // TR: Bir otomotiv teşhis sonucu. SADECE Ya hata kodu (int) ya açıklama (string) taşır!
  std::variant<int, std::string> diagnosticResult;

  diagnosticResult = 300; // Int Mode (DTC numeric code)
  std::cout << "DTC Code (Hata Kodu): " << std::get<int>(diagnosticResult) << std::endl;

  // Switch to String immediately (Old data vanishes/Yok olur)
  diagnosticResult = "P0300 - Random Misfire";
  std::cout << "DTC Description (Açıklama): " << std::get<std::string>(diagnosticResult) <<
      std::endl;

  // --- TEST 3: std::any ---
  std::cout << "\n--- 3. std::any ---" << std::endl;
  // EN: Like 'var' in JS. Heavy performance hit. DO NOT USE unless desperate.
  // TR: Dinamik tür! Performansı çok kötüdür, ZORUNDA KALMADIKÇA KULLANILMAZ.
  std::any anything = 3.14;
  anything = std::string("Dynamic Word / Dinamik Kelime");
  std::cout << "Content (Dinamik Yapı İçeriği): " << std::any_cast<std::string>(anything) <<
      std::endl;

  // --- TEST 4: std::string_view ---
  std::cout << "\n--- 4. std::string_view ---" << std::endl;
  const char *massiveText = "This is a very large text block. Copying it is suicide.";
  // TR: C Tipi metin veya C++ string fark etmez, RAM'de KOPYA ÜRETMEDEN okur!
  printFirst5Chars(massiveText);

  return 0;
}
```

► 05. Structured Bindings Optional

module_03_modern_cpp/05_structured_bindings_optional.cpp

Konu: EN: Structured Bindings, std::optional, std::variant, and std::any — Modern C++17

Kaynak Kod / Source Code

```cpp
#include <iostream>     // EN: Standard I/O
#include <string>       // EN: std::string
#include <map>          // EN: std::map for sensor readings
#include <optional>     // EN: std::optional — nullable value wrapper
#include <variant>      // EN: std::variant — type-safe union
#include <any>          // EN: std::any — type-erased container
#include <tuple>        // EN: std::tuple for structured bindings demo
#include <array>        // EN: std::array for structured bindings demo
#include <vector>       // EN: std::vector for parameter storage
#include <stdexcept>    // EN: std::runtime_error, std::bad_any_cast

// ═══════════════════════════════════════════════════════════════════════════
// 1. STRUCTURED BINDINGS WITH std::map — SENSOR READINGS
// ═══════════════════════════════════════════════════════════════════════════

// EN: Build a map of automotive sensor names to their current readings (volts).
// TR: Otomotiv sensör adlarını mevcut okumalarına (volt) eşleyen bir harita oluşturur.
std::map<std::string, double> getEngineSensorReadings() {
    return {
        {"CoolantTemp",    92.3},
        {"OilPressure",     3.8},
        {"ThrottlePos",    45.0},
        {"IntakeAirTemp",  38.7},
        {"BatteryVoltage", 13.9}
    };
}

// EN: Print all sensor readings using structured bindings in a range-for loop.
// TR: Tüm sensör okumalarını yapısal bağlamalar ile range-for döngüsünde yazdırır.
void printSensorMap(const std::map<std::string, double>& sensorMap) {
    std::cout << "  Sensor readings:\n";
    for (const auto& [name, value] : sensorMap) {
        std::cout << "    " << name << " = " << value << "\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════
// 2. STRUCTURED BINDINGS WITH STRUCT — ECU DIAGNOSTIC RESULT
// ═══════════════════════════════════════════════════════════════════════════

// EN: Diagnostic Trouble Code result from an ECU self-test.
// TR: ECU öz-test sonucundan gelen Arıza Teşhis Kodu (DTC) sonucu.
struct ECUDiagResult {
    int         dtcCode;        // EN: DTC numeric code (e.g., 301 = misfire cyl 1)
    std::string description;    // EN: Human-readable fault description
    bool        isPending;      // EN: true if fault is intermittent / not confirmed
};
```

```cpp

// EN: Simulate reading a diagnostic result from ECU.
// TR: ECU'dan teşhis sonucu okumayı simüle eder.
ECUDiagResult getLatestDiagResult() {
    return {301, "Misfire detected — Cylinder 1", true};
}

// ═══════════════════════════════════════════════════════════════════════════════
// 3. std::optional — SENSOR READING THAT MAY BE ABSENT
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Represents a single sensor reading with metadata.
// TR: Meta verili tek bir sensör okumasını temsil eder.
struct SensorReading {
    std::string name;
    double      value;
    std::string unit;
};

// EN: Read sensor by ID. Returns std::nullopt if the sensor is offline.
// TR: Sensörü ID ile okur. Sensör çevrimdışıysa std::nullopt döndürür.
std::optional<SensorReading> readSensor(int sensorId) {
    // EN: Simulate: even IDs are online, odd IDs are offline.
    // TR: Simülasyon: çift ID'ler çevrimiçi, tek ID'ler çevrimdışı.
    if (sensorId % 2 == 0) {
        return SensorReading{"Sensor_" + std::to_string(sensorId),
                             25.0 + sensorId * 1.5,
                             "°C"};
    }
    return std::nullopt;
}

// ─── 3a. Optional Chaining — Multiple Sensors Must Be Online ─────────────────

// EN: Perform a combined diagnostic only if ALL required sensors are online.
// TR: Birleşik teşhisi yalnızca TÜM gerekli sensörler çevrimiçiyse gerçekleştirir.
void combinedDiagnostic(int id1, int id2, int id3) {
    auto s1 = readSensor(id1);
    auto s2 = readSensor(id2);
    auto s3 = readSensor(id3);

    if (s1 && s2 && s3) {
        // EN: All sensors online — proceed with combined check.
        // TR: Tüm sensörler çevrimiçi — birleşik kontrole devam et.
        std::cout << "  Combined diagnostic OK:\n";
        std::cout << "    " << s1->name << " = " << s1->value << " " << s1->unit << "\n";
        std::cout << "    " << s2->name << " = " << s2->value << " " << s2->unit << "\n";
        std::cout << "    " << s3->name << " = " << s3->value << " " << s3->unit << "\n";
    } else {
        // EN: At least one sensor is offline — report which ones.
        // TR: En az bir sensör çevrimdışı — hangilerini raporla.
        std::cout << "  Combined diagnostic FAILED — offline sensors:";
        if (!s1) std::cout << " Sensor_" << id1;
        if (!s2) std::cout << " Sensor_" << id2;
        if (!s3) std::cout << " Sensor_" << id3;
        std::cout << "\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// 4. std::variant — POLYMORPHIC CAN MESSAGE PAYLOADS
// ═══════════════════════════════════════════════════════════════════════════════

// EN: CAN bus messages can carry different payload types depending on the signal.
// TR: CAN veri yolu mesajları, sinyale bağlı olarak farklı yük türleri taşıyabilir.
using CANPayload = std::variant<int, double, std::string>;
```

```cpp

// EN: Overloaded helper — combines multiple lambdas into a single visitor.
// TR: Overloaded yardımcısı — birden fazla lambdayı tek bir ziyaretçide birleştirir.
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

// EN: Decode and display a CAN payload using std::visit + overloaded lambdas.
// TR: CAN yükünü std::visit + overloaded lambdalar ile çöz ve görüntüle.
void decodeCANPayload(const std::string& signalName, const CANPayload& payload) {
    std::cout << "    Signal [" << signalName << "]: ";
    std::visit(overloaded{
        [](int val) {
            // EN: Integer payload — e.g., gear position, RPM count.
            // TR: Tam sayı yükü — ör. vites konumu, RPM sayısı.
            std::cout << "INT = " << val << "\n";
        },
        [](double val) {
            // EN: Floating-point payload — e.g., temperature, voltage.
            // TR: Kayan nokta yükü — ör. sıcaklık, voltaj.
            std::cout << "DOUBLE = " << val << "\n";
        },
        [](const std::string& val) {
            // EN: String payload — e.g., VIN, diagnostic text.
            // TR: Metin yükü — ör. VIN, teşhis metni.
            std::cout << "STRING = \"" << val << "\"\n";
        }
    }, payload);
}

// ═══════════════════════════════════════════════════════════════════════════════
// 5. std::any — DYNAMIC ECU PARAMETER STORAGE
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Generic parameter store for ECU calibration — keys are string names, values can be any
// type (int thresholds, double gains, string labels).
// TR: ECU kalibrasyon için genel parametre deposu — anahtarlar string adlar, değerler herhangi
// bir tür olabilir (int eşikler, double kazançlar, string etiketler).
using ECUParamStore = std::map<std::string, std::any>;

// EN: Safely retrieve a parameter from the store with type checking.
// TR: Parametre deposundan tür kontrolüyle güvenli bir şekilde parametre alır.
template<typename T>
std::optional<T> getParam(const ECUParamStore& store, const std::string& key) {
    auto it = store.find(key);
    if (it == store.end()) {
        return std::nullopt;
    }
    try {
        return std::any_cast<T>(it->second);
    } catch (const std::bad_any_cast&) {
        // EN: Type mismatch — requested type doesn't match stored type.
        // TR: Tür uyumsuzluğu — istenen tür depolanan türle eşleşmiyor.
        return std::nullopt;
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// main()
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== Module 03 | 05: Structured Bindings, optional, variant, any ===\n\n";

    // ─── Demo 1: Structured Bindings — pair, tuple, struct, map ──────────────────

    std::cout << "[Demo 1] Structured Bindings\n";
```

```cpp

    // EN: 1a — Decompose an std::pair (e.g., map insert result).
    // TR: 1a — std::pair ayrıştırma (ör. map ekleme sonucu).
    std::map<std::string, int> dtcCounts;
    auto [iter, inserted] = dtcCounts.emplace("P0301", 1);
    std::cout << "  Pair: key=" << iter->first
              << ", inserted=" << std::boolalpha << inserted << "\n";

    // EN: 1b — Decompose an std::tuple (e.g., ECU firmware version).
    // TR: 1b — std::tuple ayrıştırma (ör. ECU yazılım sürümü).
    std::tuple<int, int, int> firmwareVersion{2, 14, 7};
    auto [major, minor, patch] = firmwareVersion;
    std::cout << "  Tuple: firmware v" << major << "." << minor << "." << patch << "\n";

    // EN: 1c — Decompose an std::array.
    // TR: 1c — std::array ayrıştırma.
    std::array<double, 3> pidGains{0.8, 0.1, 0.05};
    auto [kp, ki, kd] = pidGains;
    std::cout << "  Array: PID gains Kp=" << kp << " Ki=" << ki << " Kd=" << kd << "\n";

    // EN: 1d — Decompose a user-defined struct.
    // TR: 1d — Kullanıcı tanımlı struct ayrıştırma.
    auto [code, desc, pending] = getLatestDiagResult();
    std::cout << "  Struct: DTC " << code << " — \"" << desc
              << "\" pending=" << std::boolalpha << pending << "\n";

    // EN: 1e — Iterate over a map with structured bindings.
    // TR: 1e — Yapısal bağlamalarla harita üzerinde dolaşma.
    auto sensors = getEngineSensorReadings();
    printSensorMap(sensors);

    std::cout << "\n";

    // ─── Demo 2: std::optional — Sensor Read (present / absent) ──────────────────────

    std::cout << "[Demo 2] std::optional — Sensor Read\n";

    for (int id : {2, 3, 6, 7}) {
        auto reading = readSensor(id);
        if (reading.has_value()) {
            // EN: Sensor is online — display value.
            // TR: Sensör çevrimiçi — değeri göster.
            std::cout << "  Sensor " << id << ": " << reading->name
                      << " = " << reading->value << " " << reading->unit << "\n";
        } else {
            // EN: Sensor is offline — use value_or as fallback demonstration.
            // TR: Sensör çevrimdışı — value_or ile varsayılan değer gösterimi.
            SensorReading fallback{"N/A", 0.0, ""};
            auto safe = reading.value_or(fallback);
            std::cout << "  Sensor " << id << ": OFFLINE (fallback name=\""
                      << safe.name << "\")\n";
        }
    }

    std::cout << "\n";

    // ─── Demo 3: Optional Chaining — Multiple Sensors ────────────────────────────────

    std::cout << "[Demo 3] Optional Chaining — Combined Diagnostic\n";

    // EN: All even IDs → all online → success.
    // TR: Tüm çift ID'ler → hepsi çevrimiçi → başarılı.
    combinedDiagnostic(2, 4, 6);

    // EN: Mixed IDs → some offline → failure with report.
    // TR: Karışık ID'ler → bazıları çevrimdışı → raporlu başarısızlık.
    combinedDiagnostic(2, 3, 6);

    std::cout << "\n";

    // ─── Demo 4: std::variant — CAN Payload with std::visit ──────────────────────────

    std::cout << "[Demo 4] std::variant — CAN Message Payloads\n";
```

```cpp

    // EN: Simulate CAN bus messages with different payload types.
    // TR: Farklı yük türleriyle CAN veri yolu mesajlarını simüle eder.
    std::vector<std::pair<std::string, CANPayload>> canMessages = {
        {"GearPosition",    3},
        {"CoolantTemp",     91.7},
        {"VIN",             std::string("WVWZZZ3CZWE123456")},
        {"EngineRPM",       3200},
        {"BatteryVoltage",  13.8}
    };

    for (const auto& [signal, payload] : canMessages) {
        decodeCANPayload(signal, payload);
    }

    // EN: Check which type is currently held using index().
    // TR: index() kullanarak şu anda hangi türün tutulduğunu kontrol et.
    CANPayload sample = 42;
    std::cout << "  Variant index for int(42): " << sample.index() << "\n";
    sample = 3.14;
    std::cout << "  Variant index for double(3.14): " << sample.index() << "\n";
    sample = std::string("test");
    std::cout << "  Variant index for string: " << sample.index() << "\n";

    std::cout << "\n";

    // ─── Demo 5: std::any — Generic Parameter Map ──────────────────────────────

    std::cout << "[Demo 5] std::any — ECU Parameter Store\n";

    ECUParamStore ecuParams;
    ecuParams["MaxRPM"]           = 7000;
    ecuParams["IdleTargetRPM"]    = 800;
    ecuParams["FuelTrimGain"]     = 1.05;
    ecuParams["CalibrationLabel"] = std::string("CAL_2026_Q1");

    // EN: Safe retrieval with correct type.
    // TR: Doğru türle güvenli erişim.
    if (auto rpm = getParam<int>(ecuParams, "MaxRPM")) {
        std::cout << "  MaxRPM = " << *rpm << "\n";
    }

    if (auto gain = getParam<double>(ecuParams, "FuelTrimGain")) {
        std::cout << "  FuelTrimGain = " << *gain << "\n";
    }

    if (auto label = getParam<std::string>(ecuParams, "CalibrationLabel")) {
        std::cout << "  CalibrationLabel = \"" << *label << "\"\n";
    }

    // EN: Wrong type → returns nullopt instead of crashing.
    // TR: Yanlış tür → çökmek yerine nullopt döndürür.
    if (auto bad = getParam<std::string>(ecuParams, "MaxRPM")) {
        std::cout << "  MaxRPM as string = " << *bad << "\n";
    } else {
        std::cout << "  MaxRPM as string: type mismatch (nullopt)\n";
    }

    // EN: Missing key → returns nullopt.
    // TR: Eksik anahtar → nullopt döndürür.
    if (auto missing = getParam<int>(ecuParams, "NonExistentParam")) {
        std::cout << "  NonExistentParam = " << *missing << "\n";
    } else {
        std::cout << "  NonExistentParam: not found (nullopt)\n";
    }

    std::cout << "\n";

    // ─── Demo 6: Comparison — When to Use optional vs variant vs any ──────────

    std::cout << "[Demo 6] Comparison — optional vs variant vs any\n\n";
```

```cpp

    // EN: optional — "might not exist": a sensor reading that could be absent.
    // TR: optional — "olmayabilir": mevcut olmayabilecek bir sensör okuması.
    std::optional<double> oilTemp = 105.3;
    std::optional<double> exhaustTemp = std::nullopt;
    std::cout << "  optional<double> oilTemp: "
              << oilTemp.value_or(-1.0) << "\n";
    std::cout << "  optional<double> exhaustTemp: "
              << exhaustTemp.value_or(-1.0) << " (default = sensor offline)\n";

    // EN: variant — "one of several known types": a CAN signal value.
    // TR: variant — "bilinen birkaç türden biri": bir CAN sinyal değeri.
    std::variant<int, double, std::string> canSignal = 42;
    std::cout << "  variant<int,double,string>: holds index "
              << canSignal.index() << " (int)\n";
    canSignal = std::string("DIAGNOSTIC_OK");
    std::cout << "  variant<int,double,string>: holds index "
              << canSignal.index() << " (string)\n";

    // EN: any — "truly dynamic type": use when type set is open/unknown.
    // TR: any — "gerçekten dinamik tür": tür kümesi açık/bilinmediğinde kullanılır.
    std::any dynParam = 3.14;
    std::cout << "  any: type = " << dynParam.type().name()
              << ", value = " << std::any_cast<double>(dynParam) << "\n";
    dynParam = std::string("runtime_label");
    std::cout << "  any: type = " << dynParam.type().name()
              << ", value = \"" << std::any_cast<std::string>(dynParam) << "\"\n";

    std::cout << "\n  Rule of thumb:\n";
    std::cout << "    optional  → value may be absent (nullable semantics)\n";
    std::cout << "    variant   → value is one of N known types (closed set)\n";
    std::cout << "    any       → value is truly dynamic (open set, use sparingly)\n";

    std::cout << "\n=== End of Module 03 | 05 ===\n";
    return 0;
}
```

Modül 4: Templates & STL — Şablonlar ve STL

Şablonlar ve STL

8 kaynak dosya · Standart: C++17

C++17

► 01. Templates Stl

module_04_advanced/01_templates_stl.cpp

Konu: Advanced: Templates and STL / İleri Seviye: Şablonlar (Derleme Zamanı Büyüsü) ve STL

Teori / Theory

THEORY: Why Were Templates Invented?

Imagine you want to find the maximum of two numbers. You write a function `max(int a, int b)`. But tomorrow, you need `max(double a, double b)`. Next week, `max(string a, string b)`. Should you write 100 overloaded functions? NO! In C++, Templates are "Code that writes code". You write ONE template, and the compiler generates the specific versions (int, float, custom class) FOR YOU at Compile-Time. This is called "Template Instantiation". The beauty is that the same logic applies to ANY type that supports the operations used inside the template — this is called "Structural Typing" or "Duck Typing at Compile-Time".
There are two kinds of templates: 1. Function Templates: Generic functions like `findMax<T>(T a, T b)`. 2. Class Templates: Generic classes like `Box<T>`, `std::vector<T>`.
The compiler performs "Type Deduction" — if you call `findMax(3, 5)` it automatically deduces `T = int`. For class templates (before C++17 CTAD), you must specify the type explicitly: `Box<int> b(42);`

TEORİ: Şablonlar (Templates) Neden Yaratıldı?

İki sayının maksimumunu bulan bir `max(int a, int b)` yazdın. Yarın `max(double, double)` gerekti, haftaya `max(string, string)`. 100 farklı fonksiyon mu yazacaksın? HAYIR! C++'da Şablonlar (Templates) "Kod Yazan Kodlardır". Sen sadece tek bir T (Zaman/Tip) şablonu yazarsın, DERLEYİCİ program derlenirken (Compile-Time) arka planda senin için o tiplere özel fonksiyonları (veya sınıfları) fiziksel olarak üretir. Buna "Şablon Somutlaştırma (Template Instantiation)" denir. Şablon içinde kullanılan operasyonları destekleyen HER tip ile çalışır — buna derleme zamanı "Ördek Tipleme" denir (Duck Typing at Compile-Time).
İki çeşit şablon vardır: 1. Fonksiyon Şablonları: `findMax<T>(T a, T b)` gibi jenerik fonksiyonlar. 2. Sınıf Şablonları: `Box<T>`, `std::vector<T>` gibi jenerik sınıflar.
Derleyici "Tip Çıkarımı (Type Deduction)" yapar — `findMax(3, 5)` çağrısında otomatik olarak `T = int` anlar. Sınıf şablonlarında (C++17 CTAD öncesi) tipi açıkça belirtmelisin: `Box<int> b(42);`

THEORY: The Standard Template Library (STL)

The STL is one of the greatest engineering marvels. It has three pillars: 1. Containers (vector, map, set, list, deque) -> Store data. 2. Iterators (begin(), end(), rbegin()) -> Object-oriented pointers. 3. Algorithms (std::sort, std::find, std::transform) -> Process data.
The key insight: Algorithms don't know about containers. Containers don't know about algorithms. They communicate ONLY through iterators! This decoupling means `std::sort` works on vector, array, deque — anything with random-access iterators. And `std::find` works on ANY container with forward iterators.
Common STL containers and their use cases: - `std::vector<T>`: Dynamic array, contiguous memory, fast random access. Best for: most general-purpose sequential data. - `std::map<K,V>`: Red-black tree, sorted key-value pairs, O(log n) lookup. Best for: ordered dictionaries. - `std::unordered_map<K,V>`: Hash table, O(1) average lookup. Best for: fast key-value lookups without ordering. - `std::set<T>`: Sorted unique elements. Best for: membership testing. - `std::deque<T>`: Double-ended queue, fast push/pop at both ends.

TEORİ: Standart Kütüphane ve Algoritması

STL (Standard Template Library) dünyanın en harika mühendislik ürünlerinden biridir. Üç temel bacağı vardır: 1. Konteynerler (Containers: vector, map, set, list, deque -> Veriyi depolar) 2. İteratörler (Iterators: begin(), end(), rbegin() -> Nesne odaklı ptr) 3. Algoritmalar (Algorithms: std::sort, std::find, std::transform)
Kilit bilgi: Algoritmalar konteynerleri bilmez. Konteynerler algoritmaları bilmez. Sadece İteratörler aracılığıyla iletişim kurarlar! Bu ayrışma sayesinde `std::sort` vector, array, deque üzerinde — random-access iteratörü olan her şeyde çalışır. `std::find` ise forward iteratörü olan HER konteynerde çalışır.
Yaygın STL konteynerleri ve kullanım alanları: - `std::vector<T>`: Dinamik dizi, bitişik bellek, hızlı rastgele erişim. - `std::map<K,V>`: Kırmızı-siyah ağaç, sıralı anahtar-değer, O(log n). - `std::unordered_map<K,V>`: Hash tablo, ortalama O(1) arama. - `std::set<T>`: Sıralı benzersiz elemanlar. Üyelik testi için ideal. - `std::deque<T>`: Çift uçlu kuyruk, her iki uçta hızlı push/pop.

THEORY: Template Specialization

Sometimes a generic template doesn't work well for a specific type. For example, comparing C-strings with `>` compares pointer addresses, not content! You can provide a "Template Specialization" — a custom version for a specific type. The compiler will prefer the specialized version when that type is used. This is explicit (full) specialization. There's also "Partial Specialization" for class templates where you specialize some parameters.

TEORİ: Şablon Özelleştirme

Bazen jenerik bir şablon belirli bir tip için düzgün çalışmaz. Örneğin, C-string'leri `>` ile karşılaştırmak pointer adreslerini karşılaştırır, içeriği değil! "Şablon Özelleştirme (Template Specialization)" ile belirli bir tipe özel versiyon yazabilirsin. Derleyici o tip kullanıldığında özel versiyonu tercih eder. Bu "tam (full) özelleştirme"dir. Sınıf şablonlarında bazı parametreleri özelleştiren "Kısmi Özelleştirme (Partial Specialization)" de vardır.

CPPREF Depth: Template Instantiation & Code Bloat

Templates are NOT real functions. They are blueprints. If you call `findMax` with 50 different data types `(int, float, char, SensorData, ECUConfig...)`, the compiler physically generates 50 DIFFERENT functions in the final `.exe` binary. This causes "Code Bloat" (Massive file size). Use them wisely! You can mitigate bloat with: - `extern template` declarations (C++11): Tell the compiler NOT to instantiate a template in this translation unit. - Thin template wrappers over non-template implementations. - Limiting the number of distinct type arguments.

CPPREF Derinlik: Şablon Somutlaştırma ve Kod Şişmesi

Şablonlar (Templates) gerçek fonksiyon DEĞİLDİR. Eğer siz `findMax` metodunu 50 farklı veri tipiyle çağırırsanız, derleyici arkaplanda 50 FARKLI fiziksel fonksiyon kopyası (Instantiations) oluşturur ve `.exe` dosyanızın boyutu devasa şişer! (Buna Code Bloat denir). Şişmeyi azaltmak için: - `extern template` bildirimleri (C++11): Derleyiciye bu çeviri biriminde şablonu somutlaştırma demeyi sağlar. - Şablon olmayan uygulamalar üzerine ince şablon sarmalayıcılar yazmak. - Farklı tip argüman sayısını sınırlamak.

Kaynak Kod / Source Code

```cpp
#include <algorithm> // EN: std::sort, std::find, std::transform
#include <cstring>   // EN: strcmp for C-string specialization
#include <iostream>
#include <map>
#include <numeric> // EN: std::accumulate
#include <string>
#include <vector>

// ─── 1. Function Template (Fonksiyon Şablonu) ──────────────────────────────────────

/**
 * @brief Generic Template Function / Jenerik Şablon Fonksiyonu
 *
 * EN: A template parameter `typename T` stands in for "any data type". The compiler deduces T
 * from the arguments automatically.
 * TR: T, "herhangi bir veri tipi" anlamına gelen kılıftır (Placeholder). Derleyici T'yi
 * argümanlardan otomatik çıkarır.
 */
template <typename T> T findMax(T a, T b) {
  return (a > b) ? a : b;
}

// ─── 2. Template Specialization (Şablon Özelleştirme) ──────────────────────────────

/**
 * @brief Specialization for C-strings / C-string İçin Özelleştirme
 *
 * EN: Without this, `findMax("apple", "zebra")` compares pointer addresses! This specialization
 * uses strcmp to compare actual string content.
 * TR: Bu olmazsa `findMax("apple", "zebra")` pointer adreslerini karşılaştırır! Bu özelleştirme
 * strcmp ile gerçek string içeriğini karşılaştırır.
 */
template <> const char *findMax<const char *>(const char *a, const char *b) {
  return (std::strcmp(a, b) > 0) ? a : b;
}

// ─── 3. Class Template (Sınıf Şablonu) ─────────────────────────────────────────────

/**
 * @class Box
 * @brief Generic Class Template with Multiple Operations
 *
 * EN: A generic box that can hold any type. Demonstrates class templates with member functions,
 * operator overloading, and type-safe containers.
 * TR: Herhangi bir tipi tutabilen jenerik bir kutu. Sınıf şablonlarını üye fonksiyonlar,
 * operatör yükleme ve tip-güvenli konteynerlerle gösterir.
 */
template <typename DataType> class Box {
private:
  DataType content;

public:
  explicit Box(DataType initData) : content(initData) {}
  DataType peek() const { return content; }
```

```cpp

  // EN: Comparison operator so we can compare Box objects
  // TR: Box nesnelerini karşılaştırabilmek için karşılaştırma operatörü
  bool operator>(const Box &other) const { return content > other.content; }

  // EN: Stream output operator (friend function)
  // TR: Akış çıktı operatörü (arkadaş fonksiyon)
  friend std::ostream &operator<<(std::ostream &os, const Box &b) {
    os << "Box[" << b.content << "]";
    return os;
  }
};

// ─── 4. Multi-Parameter Template (Çok Parametreli Şablon) ──────────────────────────

/**
 * @brief Template with multiple type parameters / Çoklu tip parametreli şablon
 *
 * EN: Templates can have more than one type parameter. This Pair class holds two values of
 * potentially different types, like std::pair.
 * TR: Şablonlar birden fazla tip parametresine sahip olabilir. Bu Pair sınıfı, std::pair gibi
 * farklı tipte iki değer tutar.
 */
template <typename First, typename Second> struct Pair {
  First first;
  Second second;

  Pair(First f, Second s) : first(f), second(s) {}

  void print() const {
    std::cout << "(" << first << ", " << second << ")";
  }
};

int main() {
  std::cout << "=== ADVANCED C++: TEMPLATES & STL DEEP DIVE ===\n"
            << std::endl;

  // ─── 1. Function Templates with Type Deduction ──────────────────────────────────────
  std::cout << "--- 1. Function Templates (Sablon Fonksiyonlar) ---"
            << std::endl;

  // EN: Explicit type specification vs auto deduction
  // TR: Açık tip belirtme vs otomatik çıkarım
  std::cout << "Max of (10, 20): " << findMax<int>(10, 20)
            << "  [explicit <int>]" << std::endl;
  std::cout << "Max of (3.14, 2.71): " << findMax(3.14, 2.71)
            << "  [auto deduced double]" << std::endl;
  std::cout << "Max of strings: "
            << findMax(std::string("Apple"), std::string("Zebra"))
            << "  [auto deduced string]" << std::endl;

  // EN: Template specialization for C-strings in action
  // TR: C-string şablon özelleştirmesi iş başında
  const char *result = findMax("apple", "zebra");
  std::cout << "Max of C-strings: " << result
            << "  [specialization with strcmp]" << std::endl;
  std::cout << std::endl;

  // ─── 2. Class Templates ─────────────────────────────────────────────────────────────
  std::cout << "--- 2. Class Templates (Sablon Siniflar) ---" << std::endl;

  Box<int> intBox(999);
  Box<std::string> strBox("Secret Message / Gizli Mesaj");
  Box<double> dblBox(3.14159);

  std::cout << "Int box: " << intBox << std::endl;
  std::cout << "String box: " << strBox << std::endl;
  std::cout << "Double box: " << dblBox << std::endl;
```

```
124
125     // EN: Using findMax with Box objects (operator> is defined)
126     // TR: Box nesneleriyle findMax kullanımı (operator> tanımlı)
127     Box<int> boxA(42);
128     Box<int> boxB(99);
129     std::cout << "Max box: " << findMax(boxA, boxB) << std::endl;
130     std::cout << std::endl;
131
132     // ─── 3. Multi-Parameter Templates ──────────────────────────────────────────
133     std::cout << "--- 3. Multi-Parameter Pair (Coklu Tip Sablonu) ---"
134               << std::endl;
135
136     Pair<std::string, int> student("Ali", 95);
137     Pair<int, double> measurement(42, 98.6);
138     std::cout << "Student: ";
139     student.print();
140     std::cout << std::endl;
141     std::cout << "Measurement: ";
142     measurement.print();
143     std::cout << "\n" << std::endl;
144
145     // ─── 4. STL Container + Iterator + Algorithm Combo ─────────────────────────
146     std::cout << "--- 4. STL: Container + Iterator + Algorithm ---" << std::endl;
147
148     // EN: Vector container with algorithms
149     // TR: Vector konteyneri ve algoritmalar
150     std::vector<int> data = {42, 1, 99, 15, -3, 77, 23};
151     std::cout << "Original: ";
152     for (int v : data) {
153       std::cout << v << " ";
154     }
155     std::cout << std::endl;
156
157     // EN: std::sort uses iterators to work with ANY random-access container
158     // TR: std::sort iteratörleri kullanarak rastgele erişimli HER konteynerde çalışır
159     std::sort(data.begin(), data.end());
160     std::cout << "Sorted:   ";
161     for (int v : data) {
162       std::cout << v << " ";
163     }
164     std::cout << std::endl;
165
166     // EN: std::find returns an iterator to the found element or end()
167     // TR: std::find bulunan elemana iterator döndürür, yoksa end()
168     auto it = std::find(data.begin(), data.end(), 42);
169     if (it != data.end()) {
170       std::cout << "Found 42 at index: "
171                 << std::distance(data.begin(), it) << std::endl;
172     }
173
174     // EN: std::accumulate sums all elements (from <numeric>)
175     // TR: std::accumulate tüm elemanları toplar (<numeric>'ten)
176     int sum = std::accumulate(data.begin(), data.end(), 0);
177     std::cout << "Sum of all elements: " << sum << std::endl;
178
179     // EN: std::transform applies a function to each element
180     // TR: std::transform her elemana bir fonksiyon uygular
181     std::vector<int> doubled(data.size());
182     std::transform(data.begin(), data.end(), doubled.begin(),
183                    [](int x) { return x * 2; });
184     std::cout << "Doubled:  ";
185     for (int v : doubled) {
186       std::cout << v << " ";
187     }
188     std::cout << "\n" << std::endl;
189
190     // ─── 5. STL Map (Associative Container) ────────────────────────────────────
191     std::cout << "--- 5. STL Map (Iliskisel Konteyner) ---" << std::endl;
```

```cpp

    // EN: std::map stores sorted key-value pairs (Red-Black Tree)
    // TR: std::map sıralı anahtar-değer çiftleri saklar (Kırmızı-Siyah Ağaç)
    std::map<std::string, int> scores;
    scores["Alice"] = 95;
    scores["Bob"] = 87;
    scores["Charlie"] = 92;
    scores["Diana"] = 98;

    // EN: Iterate over map (automatically sorted by key)
    // TR: Map üzerinde döngü (anahtara göre otomatik sıralı)
    std::cout << "Student scores (sorted by name):" << std::endl;
    for (const auto &[name, score] : scores) {
      std::cout << "  " << name << ": " << score << std::endl;
    }

    // EN: Find a specific key
    // TR: Belirli bir anahtarı bul
    auto mapIt = scores.find("Bob");
    if (mapIt != scores.end()) {
      std::cout << "Bob's score: " << mapIt->second << std::endl;
    }
    std::cout << std::endl;

    // ─── 6. STL Algorithm Chaining (Algoritma Zincirleme) ──────────────────────────────────────
    std::cout << "--- 6. Algorithm Chaining (Algoritma Zincirleme) ---"
              << std::endl;

    // EN: Count elements greater than 20 using std::count_if with a lambda
    // TR: Lambda ile 20'den büyük elemanları say (std::count_if)
    std::vector<int> values = {5, 12, 35, 8, 42, 99, 3, 17, 50};
    auto countAbove20 = std::count_if(values.begin(), values.end(),
                                      [](int x) { return x > 20; });
    std::cout << "Elements > 20: " << countAbove20 << std::endl;

    // EN: Partition — move elements satisfying a predicate to the front
    // TR: Partition — koşulu sağlayan elemanları öne taşı
    std::partition(values.begin(), values.end(),
                   [](int x) { return x % 2 == 0; });
    std::cout << "After partition (evens first): ";
    for (int v : values) {
      std::cout << v << " ";
    }
    std::cout << std::endl;

    // EN: Find min and max in one pass
    // TR: Tek geçişte min ve max bul
    auto [minIt, maxIt] = std::minmax_element(values.begin(), values.end());
    std::cout << "Min: " << *minIt << ", Max: " << *maxIt << std::endl;

    return 0;
  }
```

► 02. Functors Callbacks

module_04_advanced/02_functors_callbacks.cpp

Konu: Advanced STL: Functors and Callbacks / İleri STL: Obje Fonksiyonlar (Functors) ve Geri

Teori / Theory

THEORY: Why Are Standard Functions Insufficient? (What is a Functor?)

A regular function holds NO state (no memory of past calls). A "Functor" (Function Object) is a Class that acts like a function by overloading `operator()`. It can have internal variables (state), a Constructor, and can be passed to STL algorithms like `std::sort` or `std::for_each`.

TEORİ: Neden Sıradan Fonksiyonlar Yetersiz Kalır?

Alışageldiğimiz sıradan fonksiyonların "Hafızası" (State/Durum) yoktur. Kaç kere çağırıldığı, içine konan parametrelerin geçmişi vs silinir. "Functor" (Fonksiyon Objesi), sanki bir fonksiyonmuş gibi davranan ama aslında bir "Sınıf" (Class) olan yapılardır. Bunu `operator()` ezerek (overload) yapar. İçinde kendi "private" hafızasını tutabilir! Özel bir devrimdir.

THEORY: Callbacks and std::function

In HMI (Head-Up Display) or Networking, when a button is pressed, you don't know exactly which logic should run at compile time. You "donate" a function to that button. This is called a "Callback". Using `#include <functional>`, `std::function` acts as a universal vessel capable of storing ANY callable (Lambdas, Functors, raw functions) like a variable!

TEORİ: Callbacks ve std::function

Bir butona tıklandığında (UI programlama) HANGİ fonksiyonun çalışacağını önceden bilemezsiniz. O butona bir fonksiyon "bağışlarsınız". Buna "Callback" (Geri Çağırma) denir. `<functional>` kütüphanesi ile `std::function` kullanarak her türlü fonksiyonu (lambda, functor, normal fonk) bir C++ değişkeni gibi saklayabilirsiniz.

CPPREF Depth: `std::function` CPU Penalty & Type Erasure (Tip Silinmesi)

`std::function` achieves this magic using "Type Erasure". However, if the lambda/functor you store inside it is too large (usually > 16-32 bytes), it FAILS the "Small Object Optimization (SOO)" and silently calls `new` to allocate heap memory! In a rendering loop (60 FPS), this silent allocation can murder system performance!

CPPREF Derinlik: `std::function` CPU Cezası ve Tip Silinmesi

`std::function` bu esnekliği "Type Erasure (Tip Silinmesi)" ile yapar. Fakat içine koyduğunuz Lambda objesi çok büyükse (Örneğin bir sürü şeyi [=] ile hapsettiyse), "Küçük Nesne Optimizasyonunu (SOO)" aşar ve o okyanus kadar objeyi saklamak için arkaplanda gizlice `new` ile Heap(RAM) ayırır! Saniyede 60 kere çalışan bir ECU kontrol döngüsünde `std::function` yüzünden gizlice RAM alınıp silinmesi performansı FELÇ eder!

Kaynak Kod / Source Code

```cpp
#include <algorithm>
#include <functional> // EN: std::function
#include <iostream>
#include <vector>

/**
 * @class DeductorFunctor
 * @brief EN: A Stateful Functor evaluating ongoing calculations.
 * TR: Özel bir durum (State) tutan Obje Fonksiyon (Functor). Her çağırılışta kendisinden önceki
 * bakiyeyi HATIRLAR (Stateful)!
 */
class DeductorFunctor {
private:
  // EN: STATE (Memory of past calls)
  // TR: STATE (Durum - Fonksiyon Hafızası)
  int balance;
public:
  DeductorFunctor(int initialBalance) : balance(initialBalance) {}

  // EN: Overloading the parenthesis () operator! Now this CLASS can be called like a FUNCTION!
  // TR: Parantez () operatörü aşırı yüklendi! (Overloaded). Artık bu SINIF, bir FONKSİYON gibi
  // `banka(10);` şeklinde ÇAĞRILABİLİR!
  void operator()(int amount) {
    balance -= amount;
    std::cout << amount << " spent (harcandı). Remaining Balance (Kalan bakiye): " << balance <<
        std::endl;
  }
};

/**
 * @brief EN: GUI or Networking Simulation. "Give me a function to execute later!"
 * TR: GUI veya Ağ simülasyonu. "Bana çalışması bitince bir fonksiyon ver, çağıracağım!"
 * @param callback Çalıştırılacak olay. `std::function` sayesinde Lambda, Functor fark etmez!
 */
void executeEvent(std::function<void(int)> callback) {
  std::cout << "\n[SYSTEM] Event triggered! Starting Callback... (Olay "
                "tetiklendi! Geri çağırma başlıyor...)"
            << std::endl;
  // EN: Running the injected function internally.
  // TR: Bize verilen fonksiyonu kendi içimizde çalıştırdık.
  callback(50);
}

int main() {
  std::cout << "=== MODULE 4: FUNCTORS & CALLBACKS (std::function) ===\n" << std::endl;

  // 1. Functor Test (Stateful Function Magic / Hafızalı Fonksiyon Büyüsü)
  std::cout << "--- 1. FUNCTORS (Stateful Functions) ---" << std::endl;
  DeductorFunctor bankSystem(100);

  std::cout << ">> Used exactly like a function, but it's an Object! "
                "(Fonksiyon gibi çağrılıyor):"
            << std::endl;
  // EN: 1st Call: Balance is 80. (It remembers!)
  // TR: 1. Çağrı: Bakiye 80. (Hatırlıyor!)
  bankSystem( 20);
  // EN: 2nd Call: Balance is 65.
  // TR: 2. Çağrı: Bakiye 65.
  bankSystem(15);

  // 2. Callbacks Test
  std::cout << "\n--- 2. CALLBACKS (std::function) ---" << std::endl;

  // EN: Lambda Carrier
  // TR: Sıradan fonk yerine LAMBDA taşıyıcısı (std::function) kullanıyoruz.
  std::function<void(int)> myLambda = [](int x) {
    std::cout << "Lambda executed! Given value (Lambda çalıştı, değer): " << x << std::endl;
  };

  executeEvent(myLambda);
```

```

  // EN: Callback via Functor
  // TR: Functor ile callback (Sınıfı veriyoruz, operator() sayesinde çağırılıyor)
  executeEvent(bankSystem);

  return 0;
}
```

► 03. Stl Containers Deepdive

module_04_advanced/03_stl_containers_deepdive.cpp

Konu: Advanced STL: Container Architectures & Big-O / İleri STL: Konteyner Mimarileri ve

Teori / Theory

THEORY: When to Use Which Container? (Software Engineering Interviews)

Choosing the wrong STL container can make a program 10,000x slower. - std::vector : Fast random access [O(1)]. Slow to insert at the front [O(N)]. Under the hood, it's just a dynamic array. Best default choice because of CPU CACHE. - std::list : Doubly Linked List. Fast to insert anywhere [O(1)]. TERRIBLE at searching or random access [O(N)]. Nodes are scattered in RAM. - std::map / unordered_map : Key-Value stores. map uses a Red-Black Tree [O(logN)]. unordered_map uses a Hash Table [O(1)].

TEORİ: Hangi Konteyner Ne Zaman?

Yazılım Mimarisi Mülakatlarında asıl soru "Neyi yazdığın" değil, "Veriyi nereye koyduğun"dur. - std::vector : %90 ihtimalle doğru cevaptır. RAM'de yan yana (Contiguous) durur. CPU Cache (Önbellek) dostudur. Okuması roket hızındadır O(1). Ama dizinin başına eleman İTERSENİZ tüm dizi kayacağı için çok yavaştır O(N). - std::list : Bağlı Listelerdir (Linked List). Elemanlar RAM'de darmadağın durur. Araya eleman sokuşturmak harikadır O(1). Ama 50. elemanı bulmak O(N)'dir. - std::map vs std::unordered_map : Sözlük yapıları (Key-Value/Anahtar-Değer). map: Veriyi sürekli alfabetik/sayısal sıralayarak (Kırmızı-Siyah Ağaç) tutar O(logN). unordered_map: Sırasız, Hash (Şifreleme) tablosudur. Rokettir O(1).

THEORY: Vector Capacity vs Size Confusion

`size` is the actual elements inside (e.g., 5 elements). `capacity` is the pre-allocated RAM hardware given by OS (e.g., 8 spaces). If capacity fills up, Vector doubles it `(8*2 = 16) ` and physically MOVES to a new RAM address (Reallocation). This is costly! Solution: `reserve()`.

TEORİ: Kapasite vs Boyut Kargaşası

Vector'ün Boyutu (Size) içindeki gerçek eleman sayısıdır. Capacity (Kapasite) ise işletim sisteminden çekrilmiş RAM potansiyelidir. Eğer Kapasite dolar ve siz itmeye devam ederseniz, Vector kendi kapasitesini 2 KATINA ÇIKARARAK yep yeni bir fiziksel RAM adresine kopyalanır ve Taşınır (Reallocation). Bu korkunç yavaştır! Çözüm: `reserve()` fonksiyonu ile baştan yer ayırtmak.

CPPREF Depth: The Dreaded Iterator Invalidation HACK!

What happens if you have a pointer (Iterator) looking at `vector[5]`, but you push a new element and the vector reallocates to a new RAM address? CppReference says: ALL ITERATORS ARE INVALIDATED! Your pointer is now looking at dead memory (Garbage). Same happens if you `erase()` from the middle of a Vector. ALWAYS check cppreference for the "Iterator invalidation" table before modifying a container while looping over it!

CPPREF Derinlik: Korkunç İteratör Geçersizleşmesi (İteratör Çöküşü)!

Ya eğer elinizde `vector`'ün 5. elemanını gösteren bir pointer/İteratör varsa, ve siz yeni eleman ekleyip vector'ün kapasitesini doldurduğunuz için vector komple YENİ BİR RAM ADRESİNE taşınırsa? CppReference açıkça yazar: "TÜM İTERATÖRLER GEÇERSİZ KALIR (Invalidated)". Sizin o

içinde döngü yaparken eleman siliyorsanız, daima CppRef Invalidation tablosuna bakmalısınız!

Kaynak Kod / Source Code

```cpp
#include <algorithm>
#include <array>
#include <iostream>
#include <map>
#include <tuple>
#include <unordered_map>
#include <vector>

int main() {
  std::cout << "=== MODULE 4: STL CONTAINERS DEEP DIVE ===\n" << std::endl;

  // --- 1. VECTOR CAPACITY & REALLOCATION COST (Vektor Kapasite Çilesi) ---
  std::cout << "--- 1. std::vector Capacity & Reallocation ---" << std::endl;
  std::vector<int> numbers;

  // EN: Cost of not using reserve():
  // TR: reserve() kullanmamanın "Yeniden Taşınma (Realloc)" maliyeti:
  int migrationCount = 0;
  size_t oldCapacity = numbers.capacity();

  for (int i = 0; i < 20; i++) {
    numbers.push_back(i);
    // EN: If capacity changes, it means the entire vector moved to a new memory block!
    // TR: Eğer kapasite değişirse, tüm vector donanımsal olarak yeni bir RAM alanına fırlatıldı
    // demektir!
    if (numbers.capacity() != oldCapacity) {
      std::cout << "Reallocation (Veri Taşındı)! New Capacity (Yeni Kapasite): " <<
          numbers.capacity() << std::endl;
      oldCapacity = numbers.capacity();
      migrationCount++;
    }
  }

  std::cout << "For just 20 elements, Vector physically moved in RAM " << migrationCount <<
      " times!" << std::endl;
  std::cout << "> SOLUTION (ÇÖZÜM): `numbers.reserve(20);` to prepare RAM at "
              "the start.\n"
            << std::endl;

  // --- 2. MAP (Tree) vs UNORDERED_MAP (Hash Table) ---
  std::cout << "--- 2. Dictionary (Key-Value) Selection ---" << std::endl;

  // EN: Backed by Red-Black Tree. Maintains order. O(logN).
  // TR: Arkasında Kırmızı-Siyah Ağaç yatar. Sürekli veriyi sıralı tutar. O(logN).
  std::map<std::string, int> orderedDictionary;
  orderedDictionary["Zebra"] = 1;
  orderedDictionary["Bear"] = 2;
  orderedDictionary["Cat"] = 3;

  std::cout << "std::map (Ordered Tree / Sıralı Ağaç - O(logN) ) :" << std::endl;
  for (auto const &[key, val] : orderedDictionary) {
```

```
52        // EN: Output will AUTOMATICALLY be Alphabetical! (Bear, Cat, Zebra)
53        // TR: Çıktı OTOMATİK olarak Alfabetik sıralıdır! (Bear, Cat, Zebra)
54        std::cout << key << ":" << val << " ";
55      }
56
57      // EN: Backed by Hash Table. Unpredictable order. O(1) Speed!
58      // TR: Arkasında Hash C-Tabloları yatar. Sıralama garantisi YOKTUR. O(1) Roket Hızı!
59      std::cout << "\n\nstd::unordered_map (Hash Table / Sırasız - O(1) ) :" << std::endl;
60      std::unordered_map<std::string, int> hashDictionary;
61      hashDictionary["Zebra"] = 1;
62      hashDictionary["Bear"] = 2;
63      hashDictionary["Cat"] = 3;
64
65      for (auto const &[key, val] : hashDictionary) {
66        // EN: Output order is NOT guaranteed. It only focuses on speed.
67        // TR: Çıktı sıralama garantisi VERMEZ. Sadece hıza odaklanır.
68        std::cout << key << ":" << val << " ";
69      }
70      std::cout << "\n" << std::endl;
71
72      // ═══════════════════════════════════════════════════════════════════════════════
73      // EN: Demo 3 — std::array: Fixed-Size Wheel Speed Sensors
74      // TR: Demo 3 — std::array: Sabit Boyutlu Tekerlek Hız Sensörleri
75      // ═══════════════════════════════════════════════════════════════════════════════
76      std::cout << "--- 3. std::array — Fixed-Size Wheel Speed Sensors ---" << std::endl;
77
78      // EN: std::array<T, N> lives entirely on the stack — zero heap allocation.
79      // TR: std::array<T, N> tamamen stack üzerinde yaşar — heap tahsisi sıfırdır.
80      std::array<double, 4> wheelSpeedKmh = {82.5, 81.9, 83.1, 80.7};
81
82      // ── 3a. Compile-Time Size ──────────────────────────────────────────────────
83      // EN: Size is known at compile time — no runtime overhead.
84      // TR: Boyut derleme zamanında bilinir — çalışma zamanı ek maliyeti yoktur.
85      std::cout << "Wheel count (compile-time): " << wheelSpeedKmh.size() << std::endl;
86      static_assert(wheelSpeedKmh.size() == 4, "ECU expects exactly 4 wheel sensors");
87
88      // ── 3b. Bounds Checking with .at() ─────────────────────────────────────────
89      // EN: .at() throws std::out_of_range if index is invalid — safer than [].
90      // TR: .at() geçersiz indekste std::out_of_range fırlatır — []'den güvenlidir.
91      std::cout << "Front-Left  (FL) speed: " << wheelSpeedKmh.at(0) << " km/h" << std::endl;
92      std::cout << "Front-Right (FR) speed: " << wheelSpeedKmh.at(1) << " km/h" << std::endl;
93      std::cout << "Rear-Left   (RL) speed: " << wheelSpeedKmh.at(2) << " km/h" << std::endl;
94      std::cout << "Rear-Right  (RR) speed: " << wheelSpeedKmh.at(3) << " km/h" << std::endl;
95
96      // ── 3c. STL Algorithm Compatibility ────────────────────────────────────────
97      // EN: std::array is fully compatible with STL algorithms (sort, find, etc.).
98      // TR: std::array, STL algoritmalarıyla (sort, find vb.) tam uyumludur.
99      std::sort(wheelSpeedKmh.begin(), wheelSpeedKmh.end());
100     std::cout << "Sorted speeds (ascending): ";
101     for (const auto &spd : wheelSpeedKmh) {
102       std::cout << spd << " ";
103     }
104     std::cout << std::endl;
105
106     auto it = std::find(wheelSpeedKmh.begin(), wheelSpeedKmh.end(), 83.1);
107     if (it != wheelSpeedKmh.end()) {
108       std::cout << "Found 83.1 km/h at index: "
109                 << std::distance(wheelSpeedKmh.begin(), it) << std::endl;
110     }
111     std::cout << std::endl;
112
113     // ═══════════════════════════════════════════════════════════════════════════════
114     // EN: Demo 4 — std::tuple: Heterogeneous ECU Diagnostic Data
115     // TR: Demo 4 — std::tuple: Heterojen ECU Arıza Teşhis Verisi
116     // ═══════════════════════════════════════════════════════════════════════════════
117     std::cout << "--- 4. std::tuple — ECU Diagnostic Packet ---" << std::endl;
```

```cpp

    // EN: Tuple groups different types together — perfect for multi-field ECU frames.
    // TR: Tuple farklı tipleri bir arada gruplar — çoklu alan içeren ECU çerçeveleri için
    // idealdir.
    std::tuple<int, std::string, double> ecuDiag(0x1A3F, "Throttle Sensor Drift", 4.87);

    // ─── 4a. Access with std::get<> ──────────────────────────────────────────────────────
    // EN: std::get<Index> retrieves the element at compile-time index.
    // TR: std::get<Index> derleme zamanı indeksiyle elemana erişir.
    std::cout << "Error Code : 0x" << std::hex << std::get<0>(ecuDiag) << std::dec << std::endl;
    std::cout << "Description: " << std::get<1>(ecuDiag) << std::endl;
    std::cout << "Voltage    : " << std::get<2>(ecuDiag) << " V" << std::endl;

    // ─── 4b. Unpacking with std::tie ─────────────────────────────────────────────────────
    // EN: std::tie binds tuple elements to existing variables by reference.
    // TR: std::tie, tuple elemanlarını mevcut değişkenlere referansla bağlar.
    int errCode{};
    std::string errDesc;
    double voltage{};
    std::tie(errCode, errDesc, voltage) = ecuDiag;
    std::cout << "[tie] Code=0x" << std::hex << errCode << std::dec
              << "  Desc=" << errDesc << "  V=" << voltage << std::endl;

    // ─── 4c. Structured Bindings (C++17) ─────────────────────────────────────────────────
    // EN: Structured bindings provide the cleanest syntax — one-liner unpacking.
    // TR: Yapısal bağlamalar en temiz sözdizimini sunar — tek satırda açma.
    auto [code, desc, volt] = ecuDiag;
    std::cout << "[structured binding] Code=0x" << std::hex << code << std::dec
              << "  Desc=" << desc << "  V=" << volt << "\n" << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════════════
    // EN: Demo 5 — Container Selection Guide (Automotive Context)
    // TR: Demo 5 — Konteyner Seçim Rehberi (Otomotiv Bağlamı)
    // ═══════════════════════════════════════════════════════════════════════════════════════
    std::cout << "--- 5. Container Selection Guide ---" << std::endl;
    std::cout << R"(
    +-------------------+------------------+-------------------------------------+
    | Container         | Complexity       | Automotive Use Case                 |
    +-------------------+------------------+-------------------------------------+
    | std::vector       | O(1) access      | Sensor stream buffers, CAN frames   |
    | std::array        | O(1), stack only | Fixed wheel/axle sensor arrays      |
    | std::map          | O(logN) ordered  | DTC code registry (sorted by ID)    |
    | std::unordered_map| O(1) hash        | Real-time signal lookup by name     |
    | std::tuple        | compile-time     | Grouping heterogeneous ECU fields   |
    +-------------------+------------------+-------------------------------------+
  )" << std::endl;

    // ─── Note: std::span (C++20) ─────────────────────────────────────────────────────────
    // EN: std::span<T> is a non-owning, lightweight view over contiguous memory (e.g., a raw
    // array, std::vector, or std::array). It does NOT copy data. Think of it as a "window" into an
    // existing buffer — ideal for passing sensor data slices to processing functions without
    // ownership transfer. Requires C++20: compile with -std=c++20.
    // TR: std::span<T>, bitişik bellekteki veriye sahiplenmeyen hafif bir görünümdür (raw dizi,
    // std::vector veya std::array üzerinde). Veriyi KOPYALAMAZ. Mevcut bir tampona açılan bir
    // "pencere" gibi düşünün — sensör veri dilimlerini sahiplik devretmeden işleme fonksiyonlarına
    // geçirmek için idealdir. C++20 gerektirir: -std=c++20 ile derleyin.

    return 0;
  }
```

► 04. Iterators And Adaptors

module_04_advanced/04_iterators_and_adaptors.cpp

Konu: Templates & STL: Iterators, Categories and Adaptors / Şablonlar ve STL: İteratörler,

Teori / Theory

THEORY: Iterator Categories — The 5 Ranks of STL Access

1. Input Iterator (Giriş İteratörü)

Read-only, single-pass, forward-only. Like reading sensor data from a CAN bus — once you read a frame, it's gone. You can only move forward and read each value exactly once. Supports: ++it, *it (read), it == it2, it != it2

TEORİ: İteratör Kategorileri — STL Erişiminin 5 Kademesi

Salt okunur, tek geçişli, sadece ileri. CAN bus'tan sensör verisi okumak gibi — bir kare okunduktan sonra gider. Sadece ileri gidip her değeri tam olarak bir kez okuyabilirsiniz. Destekler: ++it, *it (okuma), it == it2, it != it2

2. Output Iterator (Çıkış İteratörü)

Write-only, single-pass, forward-only. Like writing commands to an actuator — you push data out, you can't read it back. Supports: ++it, *it = val (write)

Salt yazılır, tek geçişli, sadece ileri. Bir aktüatöre komut yazmak gibi — veri gönderirsiniz, geri okuyamazsınız. Destekler: ++it, *it = val (yazma)

3. Forward Iterator (İleri İteratörü)

Read+Write, multi-pass, forward-only. Like iterating over a linked list of ECU diagnostic trouble codes (DTCs). Supports: Everything Input + multi-pass guarantee

Okuma+Yazma, çok geçişli, sadece ileri. ECU arıza kodları (DTC) bağlı listesinde gezinmek gibi. Destekler: Input'un tamamı + çok geçiş garantisi

4. Bidirectional Iterator (Çift Yönlü İteratör)

Forward + backward traversal. Like scrolling through a vehicle's event log — you can go forward and backward through entries. Supports: Everything Forward + --it Used by: std::list, std::set, std::map

İleri + geri gezinme. Araç olay günlüğünde gezinmek gibi — ileri ve geri gidebilirsiniz. Destekler: Forward'ın tamamı + --it Kullanan: std::list, std::set, std::map

5. Random Access Iterator (Rastgele Erişim İteratörü)

Full pointer arithmetic. Like accessing any sensor value in an array by index — instant O(1) jump to any position. Supports: Everything Bidirectional + it+n, it-n, it[n], it<it2 Used by: std::vector, std::deque, std::array, raw pointers

Tam pointer aritmetiği. Bir dizideki herhangi bir sensör değerine indeksle erişmek gibi — herhangi bir konuma anında O(1) atlama. Destekler: Bidirectional'ın tamamı + it+n, it-n, it[n], it<it2 Kullanan: std::vector, std::deque, std::array, ham pointerlar

Random Access ⊃ Bidirectional ⊃ Forward ⊃ Input ⊃ Output
Her üst kategori, altındakilerin tüm yeteneklerini içerir. std::sort → Random Access gerektirir std::find → Input yeterlidir std::reverse → Bidirectional gerektirir

CPPREF Depth: Iterator Categories and Their Guarantees

The C++ standard defines 5 iterator categories, each a superset of the previous: 1. InputIterator — single-pass read (==, !=, *, ++) 2. OutputIterator — single-pass write (*it = val, ++) 3. ForwardIterator — multi-pass read/write (can revisit elements) 4. BidirectionalIterator — adds backward traversal (--) 5. RandomAccessIterator — O(1) jump: it+n, it-n, it[n], <, > `std::distance(first, last)` is O(1) for RandomAccess iterators (pointer subtraction) but O(n) for all others (incremental ++). `std::advance(it, n)` similarly dispatches on category. Iterator invalidation rules vary critically by container: • vector: insert/push_back may invalidate ALL iterators (realloc). • deque: insert at front/back invalidates iterators but not refs. • list/set/map: insert NEVER invalidates existing iterators. C++20 introduces `std::contiguous_iterator` — guarantees elements are adjacent in memory (e.g., vector, array, string).

CPPREF Derinlik: İteratör Kategorileri ve Garantileri

C++ standardı 5 iteratör kategorisi tanımlar, her biri öncekinin üst kümesidir: 1. InputIterator — tek geçişli okuma (==, !=, *, ++) 2. OutputIterator — tek geçişli yazma (*it = val, ++) 3. ForwardIterator — çok geçişli okuma/yazma (tekrar ziyaret edilebilir) 4. BidirectionalIterator — geri gidiş eklenir (--) 5. RandomAccessIterator — O(1) atlama: it+n, it-n, it[n], <, > `std::distance(first, last)` RandomAccess için O(1)'dir (işaretçi çıkarma), diğerleri için O(n)'dir (sıralı ++). İteratör geçersizleşme kuralları konteynere göre değişir:
• vector: insert/push_back TÜM iteratörleri geçersiz kılabilir (realloc). • deque: baş/sona ekleme iteratörleri geçersiz kılar ama referansları değil. • list/set/map: ekleme mevcut iteratörleri ASLA geçersiz kılmaz. C++20 `std::contiguous_iterator` ekler — elemanların bellekte bitişik olduğunu garanti eder (ör. vector, array, string).

Kaynak Kod / Source Code

```cpp
#include <algorithm>    // EN: STL algorithms
#include <deque>        // EN: Double-ended queue
#include <forward_list> // EN: Singly-linked list
#include <iostream>     // EN: Standard I/O
#include <iterator>     // EN: Iterator adaptors
#include <list>         // EN: Doubly-linked list
#include <numeric>      // EN: std::iota, std::accumulate
#include <set>          // EN: Ordered set
#include <sstream>      // EN: String stream
#include <string>       // EN: std::string
#include <vector>       // EN: Dynamic array

// ═══════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. Demonstrating Iterator Categories ──────────────────────────────

// EN: A function that REQUIRES Random Access Iterators (like std::sort).
// TR: Random Access İteratör GEREKTİREN bir fonksiyon (std::sort gibi).
template <typename RandomIt>
void printSorted(RandomIt first, RandomIt last) {
    std::vector<typename std::iterator_traits<RandomIt>::value_type> temp(first, last);
    std::sort(temp.begin(), temp.end());
    for (const auto& val : temp) {
        std::cout << val << " ";
    }
    std::cout << "\n";
}

// EN: A function that works with ANY Input Iterator (minimum requirement).
// TR: Herhangi bir Input İteratör ile çalışan fonksiyon (minimum gereksinim).
template <typename InputIt>
typename std::iterator_traits<InputIt>::value_type
sumRange(InputIt first, InputIt last) {
    typename std::iterator_traits<InputIt>::value_type total{};
    while (first != last) {
        total += *first;
        ++first;
    }
    return total;
}

// ─── 2. Custom Iterator — SensorRange ──────────────────────────────────

// EN: A custom forward iterator that generates sensor readings in a range. Simulates reading
// sequential sensor addresses (e.g., temperature registers 0x100, 0x104, 0x108 ... on an ECU
// memory map).
// TR: Belirli bir aralıkta sensör okumaları üreten özel forward iterator. Sıralı sensör
// adreslerini okumayı simüle eder (örn. ECU bellek haritasında 0x100, 0x104, 0x108 ... sıcaklık
// kaydedicileri).
class SensorRange {
   public:
    // EN: Forward Iterator implementation
    // TR: Forward İteratör uygulaması
    class Iterator {
       public:
        // EN: Required type aliases for std::iterator_traits
        // TR: std::iterator_traits için gerekli tip tanımları
        using iterator_category = std::forward_iterator_tag;
        using value_type = int;
        using difference_type = std::ptrdiff_t;
        using pointer = const int*;
        using reference = const int&;

        explicit Iterator(int val) : current_(val) {}

        reference operator*() const { return current_; }
        pointer operator->() const { return &current_; }

        Iterator& operator++() {
```

```
            // EN: 4-byte aligned sensor addresses
            // TR: 4-byte hizalı sensör adresleri
            current_ += 4;
            return *this;
        }

        Iterator operator++(int) {
            Iterator tmp = *this;
            ++(*this);
            return tmp;
        }

        bool operator==(const Iterator& other) const {
            return current_ >= other.current_;
        }

        bool operator!=(const Iterator& other) const {
            return !(*this == other);
        }

       private:
        int current_;
    };

    SensorRange(int start, int end) : start_(start), end_(end) {}

    Iterator begin() const { return Iterator(start_); }
    Iterator end() const { return Iterator(end_); }

   private:
    int start_;
    int end_;
};

// ─── 3. Container Adaptor Demonstrations ──────────────────────────────────────

// EN: Collect diagnostic trouble codes (DTCs) using back_inserter. back_inserter automatically
// calls push_back on the target container.
// TR: Arıza kodlarını (DTC) back_inserter ile topla. back_inserter hedef konteynerde otomatik
// push_back çağırır.
std::vector<std::string> collectActiveDTCs(
    const std::vector<std::string>& allCodes,
    const std::string& prefix) {
    std::vector<std::string> result;
    std::copy_if(allCodes.begin(), allCodes.end(),
                 std::back_inserter(result),
                 [&prefix](const std::string& code) {
                     return code.substr(0, prefix.size()) == prefix;
                 });
    return result;
}

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: ITERATORS & ADAPTORS ===\n"
              << "=== MODÜL 4: İTERATÖRLER VE ADAPTÖRLER ===\n\n";

    // ─── 1. Iterator Categories in Action ──────────────────────────────────────
    {
        std::cout << "--- 1. Iterator Categories ---\n";

        std::vector<int> speeds = {120, 85, 200, 60, 150};
```

```cpp

        // EN: vector iterators are Random Access — supports std::sort
        // TR: vector iteratörleri Random Access — std::sort destekler
        std::cout << "Sorted speeds: ";
        printSorted(speeds.begin(), speeds.end());

        // EN: sumRange works with ANY iterator (Input is enough)
        // TR: sumRange herhangi bir iteratör ile çalışır (Input yeterli)
        std::cout << "Sum of speeds: " << sumRange(speeds.begin(), speeds.end()) << "\n";

        // EN: list iterators are Bidirectional — can reverse but NOT random access
        // TR: list iteratörleri Bidirectional — ters gidebilir ama rastgele erişim YOK
        std::list<int> rpms = {3000, 1500, 4500, 2200};
        // EN: list has its own sort (merge sort) — doesn't need Random Access
        // TR: list'in kendi sıralaması var (merge sort) — Random Access gerektirmez
        rpms.sort();
        std::cout << "Sorted RPMs (list::sort): ";
        for (const auto& r : rpms) std::cout << r << " ";
        std::cout << "\n";

        // EN: forward_list is Forward only — single direction, no size()
        // TR: forward_list sadece Forward — tek yön, size() yok
        std::forward_list<int> temps = {72, 88, 65, 91};
        temps.sort();
        std::cout << "Sorted temps (forward_list): ";
        for (const auto& t : temps) std::cout << t << " ";
        std::cout << "\n\n";
    }

    // ── 2. Custom Iterator — SensorRange ──────────────────────────────────────
    {
        std::cout << "--- 2. Custom SensorRange Iterator ---\n";

        // EN: Iterate over sensor addresses 0x100 to 0x120 (4-byte steps)
        // TR: 0x100'den 0x120'ye sensör adresleri üzerinde gezin (4-byte adım)
        SensorRange sensors(0x100, 0x120);

        std::cout << "Sensor addresses: ";
        for (int addr : sensors) {
            std::cout << "0x" << std::hex << addr << " ";
        }
        std::cout << std::dec << "\n";

        // EN: Works with STL algorithms because it satisfies ForwardIterator
        // TR: ForwardIterator'ı karşıladığı için STL algoritmaları ile çalışır
        std::cout << "Count of addresses: "
                  << std::distance(sensors.begin(), sensors.end()) << "\n\n";
    }

    // ── 3. back_inserter — Dynamic Collection ─────────────────────────────────
    {
        std::cout << "--- 3. back_inserter ---\n";

        std::vector<int> source = {10, 20, 30, 40, 50};
        std::vector<int> dest;

        // EN: back_inserter creates an Output Iterator that calls push_back
        // TR: back_inserter push_back çağıran bir Output İteratör oluşturur
        std::copy(source.begin(), source.end(), std::back_inserter(dest));
        std::cout << "Copied via back_inserter: ";
        for (int v : dest) std::cout << v << " ";
        std::cout << "\n";

        // EN: Transform + back_inserter — double each value into a new vector
        // TR: Transform + back_inserter — her değeri iki katına çıkar
        std::vector<int> doubled;
        std::transform(source.begin(), source.end(),
                       std::back_inserter(doubled),
                       [](int x) { return x * 2; });
        std::cout << "Doubled: ";
        for (int v : doubled) std::cout << v << " ";
        std::cout << "\n\n";
    }
```

```cpp

    // ─── 4. inserter — Middle Insertion ──────────────────────────────────────
    {
        std::cout << "--- 4. inserter (mid-point insertion) ---\n";

        std::vector<int> data = {1, 2, 5, 6};
        std::vector<int> toInsert = {3, 4};

        // EN: Insert {3,4} at position 2 (before element '5')
        // TR: {3,4}'ü 2. pozisyona ekle ('5' elemanından önce)
        auto insertPos = data.begin() + 2;
        std::copy(toInsert.begin(), toInsert.end(),
                  std::inserter(data, insertPos));

        std::cout << "After inserter: ";
        for (int v : data) std::cout << v << " ";
        std::cout << "\n\n";
    }

    // ─── 5. ostream_iterator — Direct Output ─────────────────────────────────
    {
        std::cout << "--- 5. ostream_iterator ---\n";

        std::vector<std::string> ecuNames = {"ECM", "TCM", "BCM", "ABS", "TPMS"};

        // EN: Write directly to cout using an output iterator — no loop needed!
        // TR: Output iterator ile doğrudan cout'a yaz — döngü gerekmez!
        std::cout << "ECU modules: ";
        std::copy(ecuNames.begin(), ecuNames.end(),
                  std::ostream_iterator<std::string>(std::cout, ", "));
        std::cout << "\n\n";
    }

    // ─── 6. istream_iterator — Parse from String Stream ──────────────────────
    {
        std::cout << "--- 6. istream_iterator (from stringstream) ---\n";

        // EN: Simulate reading space-separated sensor values from a data stream
        // TR: Veri akışından boşlukla ayrılmış sensör değerlerini okumayı simüle et
        std::istringstream sensorData("72 85 91 68 77");
        std::vector<int> readings{
            std::istream_iterator<int>(sensorData),
            std::istream_iterator<int>()};

        std::cout << "Parsed readings: ";
        for (int r : readings) std::cout << r << " ";
        std::cout << "\n";
        std::cout << "Average: "
                  << sumRange(readings.begin(), readings.end()) /
                         static_cast<int>(readings.size())
                  << "\n\n";
    }

    // ─── 7. Reverse Iterators ────────────────────────────────────────────────
    {
        std::cout << "--- 7. Reverse Iterators ---\n";

        std::vector<int> faultLog = {101, 203, 305, 407, 509};

        // EN: rbegin()/rend() traverse the container backwards
        // TR: rbegin()/rend() konteyneri geriye doğru gezinir
        std::cout << "Fault log (newest first): ";
        for (auto it = faultLog.rbegin(); it != faultLog.rend(); ++it) {
            std::cout << *it << " ";
        }
        std::cout << "\n";
```

```

        // EN: Find last element > 300 using reverse iterators
        // TR: Ters iteratör ile > 300 olan son elemanı bul
        auto rit = std::find_if(faultLog.rbegin(), faultLog.rend(),
                                [](int code) { return code > 300; });
        if (rit != faultLog.rend()) {
            std::cout << "Last fault > 300: " << *rit << "\n";
        }
        std::cout << "\n";
    }

    // ─── 8. std::move_iterator ──────────────────────────────────────────────
    {
        std::cout << "--- 8. move_iterator ---\n";

        std::vector<std::string> logs = {"[INFO] Start", "[WARN] Temp high",
                                         "[ERR] Overheat", "[INFO] Shutdown"};

        // EN: Move (not copy!) strings from logs into archive
        // TR: Stringleri logs'tan archive'a taşı (kopyalama değil!)
        std::vector<std::string> archive(
            std::make_move_iterator(logs.begin()),
            std::make_move_iterator(logs.end()));

        std::cout << "Archived " << archive.size() << " entries\n";
        std::cout << "Original logs[0] after move: \""
                  << logs[0] << "\" (empty — moved!)\n\n";
    }

    // ─── 9. DTC Filter with back_inserter ───────────────────────────────────
    {
        std::cout << "--- 9. Automotive DTC Filter ---\n";

        std::vector<std::string> allDTCs = {
            "P0301", "P0420", "B0100", "C0035",
            "P0171", "U0100", "P0442", "B0200"};

        // EN: Filter only Powertrain codes (start with 'P')
        // TR: Sadece Motor/Şanzıman kodlarını filtrele ('P' ile başlayan)
        auto powertrainDTCs = collectActiveDTCs(allDTCs, "P");

        std::cout << "Powertrain DTCs: ";
        std::copy(powertrainDTCs.begin(), powertrainDTCs.end(),
                  std::ostream_iterator<std::string>(std::cout, " "));
        std::cout << "\n";

        // EN: Filter Body codes (start with 'B')
        // TR: Gövde kodlarını filtrele ('B' ile başlayan)
        auto bodyDTCs = collectActiveDTCs(allDTCs, "B");
        std::cout << "Body DTCs: ";
        std::copy(bodyDTCs.begin(), bodyDTCs.end(),
                  std::ostream_iterator<std::string>(std::cout, " "));
        std::cout << "\n\n";
    }

    // ─── 10. std::iota + Iterators ──────────────────────────────────────────
    {
        std::cout << "--- 10. std::iota + Iterator Arithmetic ---\n";

        std::vector<int> sensorIDs(10);
        // EN: Fill with sequential IDs: 100, 101, 102, ...
        // TR: Sıralı ID'ler ile doldur: 100, 101, 102, ...
        std::iota(sensorIDs.begin(), sensorIDs.end(), 100);
```

```cpp

        // EN: Random Access: jump to 5th element directly
        // TR: Random Access: doğrudan 5. elemana atla
        auto it = sensorIDs.begin() + 4;
        std::cout << "5th sensor ID: " << *it << "\n";

        // EN: Iterator arithmetic: distance between two positions
        // TR: İteratör aritmetiği: iki pozisyon arası mesafe
        auto first = std::find(sensorIDs.begin(), sensorIDs.end(), 103);
        auto last = std::find(sensorIDs.begin(), sensorIDs.end(), 108);
        std::cout << "Distance 103→108: "
                  << std::distance(first, last) << " elements\n";

        // EN: advance() moves an iterator by N steps
        // TR: advance() bir iteratörü N adım ilerletir
        auto walker = sensorIDs.begin();
        std::advance(walker, 7);
        std::cout << "After advance(7): " << *walker << "\n";

        // EN: next()/prev() return a NEW iterator without modifying the original
        // TR: next()/prev() orijinali değiştirmeden YENİ iteratör döndürür
        auto n = std::next(sensorIDs.begin(), 3);
        auto p = std::prev(sensorIDs.end(), 2);
        std::cout << "next(begin,3)=" << *n << "  prev(end,2)=" << *p << "\n";
    }

    std::cout << "\n=== ITERATORS & ADAPTORS COMPLETE ===\n";
    return 0;
}
```

► 05. Algorithms Masterclass

module_04_advanced/05_algorithms_masterclass.cpp

Konu: Templates & STL: Algorithm Masterclass / Şablonlar ve STL: Algoritma Ustalık Rehberi

Teori / Theory

THEORY: STL Algorithms — The Power of Generic Programming

The STL provides 100+ algorithms in <algorithm> and <numeric>. They operate on ITERATORS, not containers — making them universal. This is the "separation of concerns" principle: containers STORE data, algorithms PROCESS data, iterators CONNECT them.

Key algorithm families: ┌──┐ │ Non-modifying │ find, count, all_of, any_of, none_of, │ │ │ for_each, mismatch, equal, search │ ├──┤ │ Modifying │ copy, move, transform, fill, generate, │ │ │ replace, remove, unique, reverse, rotate │ ├──┤ │ Sorting │ sort, stable_sort, partial_sort, │ │ │ nth_element, is_sorted │ ├──┤ │ Binary Search │ lower_bound, upper_bound, equal_range, │ │ │ binary_search (requires sorted range!) │ ├──┤ │ Partitioning │ partition, stable_partition, is_partitioned│ ├──┤ │ Numeric │ accumulate, inner_product, partial_sum, │ │ │ iota, reduce (C++17), transform_reduce │ ├──┤ │ Min/Max │ min, max, minmax, min_element, │ │ │ max_element, minmax_element, clamp (C++17) │ └──┘

TEORİ: STL Algoritmaları — Jenerik Programlamanın Gücü

STL, <algorithm> ve <numeric> başlıklarında 100'den fazla algoritma sunar. Konteynerler üzerinde değil, İTERATÖRLER üzerinde çalışırlar — bu onları evrensel kılar. Bu "sorumlulukların ayrılması" ilkesidir: konteynerler veri DEPOLAR, algoritmalar veri İŞLER, iteratörler onları BAĞLAR.

CPPREF Depth: Algorithm Complexity Guarantees and Exception Safety

STL algorithms provide strict worst-case complexity guarantees: • `std::sort` — O(n log n) comparisons. Typically introsort (quicksort + heapsort fallback), guaranteed worst-case O(n log n). • `std::stable_sort` — O(n log²n) in-place, or O(n log n) if extra memory is available (merge sort with buffer). • `std::nth_element` — O(n) average (introselect). After the call, element at nth position is what it would be in a sorted range.
• `std::partial_sort` — O(n log k) where k = middle - first. Exception safety: algorithms use `std::move_if_noexcept` internally. If the element's move constructor is noexcept, it moves; otherwise it copies to maintain the strong exception guarantee. This is why marking move constructors `noexcept` is critical for performance. Algorithms that write to output iterators (e.g., `std::transform`) provide the basic guarantee: no leaks, but output may be partial. `std::for_each` returns its function object — useful for stateful functors accumulating results during traversal.

CPPREF Derinlik: Algoritma Karmaşıklık Garantileri ve İstisna Güvenliği

STL algoritmaları katı en-kötü-durum karmaşıklık garantisi sunar: • `std::sort` — O(n log n) karşılaştırma. Genelde introsort (quicksort + heapsort yedek), en kötü durum O(n log n) garantili. • `std::stable_sort` — Yerinde O(n log²n), veya ek bellekle O(n log n) (tamponlu merge sort). • `std::nth_element` — ortalama O(n) (introselect). Çağrı sonrası n. pozisyondaki eleman, sıralı dizideki yerindedir. • `std::partial_sort` — O(n log k), k = middle - first. İstisna güvenliği: algoritmalar dahili olarak `std::move_if_noexcept` kullanır. Elemanın taşıma yapıcısı noexcept ise taşır; değilse güçlü istisna garantisini korumak için kopyalar. Bu nedenle taşıma yapıcılarını `noexcept` işaretlemek performans için kritiktir. Çıkış iteratörlerine yazan algoritmalar (ör. `std::transform`) temel garanti sunar: sızıntı olmaz, ama çıktı kısmi kalabilir.

Kaynak Kod / Source Code

```cpp
#include <algorithm>   // EN: STL algorithms
#include <cmath>       // EN: std::abs
#include <functional>  // EN: std::greater, std::multiplies
#include <iostream>    // EN: Standard I/O
#include <iterator>    // EN: ostream_iterator, back_inserter
#include <numeric>     // EN: accumulate, iota, inner_product, partial_sum, reduce
#include <string>      // EN: std::string
#include <vector>      // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Automotive sensor reading structure
// TR: Otomotiv sensör okuma yapısı
struct SensorReading {
    std::string name;
    double value;
    // EN: 1=critical, 5=info
    // TR: 1=kritik, 5=bilgi
    int priority;

    friend std::ostream& operator<<(std::ostream& os, const SensorReading& s) {
        return os << s.name << "(" << s.value << ",P" << s.priority << ")";
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: STL ALGORITHMS MASTERCLASS ===\n"
              << "=== MODÜL 4: STL ALGORİTMA USTALIK REHBERİ ===\n\n";
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════
    // [SORTING FAMILY / SIRALAMA AİLESİ]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 1. std::sort — Introsort O(N log N) ─────────────────────────────────
    {
        std::cout << "--- 1. std::sort ---\n";

        std::vector<int> speeds = {120, 85, 200, 60, 150, 95, 180};

        // EN: Default ascending sort
        // TR: Varsayılan artan sıralama
        std::sort(speeds.begin(), speeds.end());
        std::cout << "Ascending: ";
        for (int s : speeds) std::cout << s << " ";

        // EN: Descending sort with std::greater
        // TR: std::greater ile azalan sıralama
        std::sort(speeds.begin(), speeds.end(), std::greater<int>());
        std::cout << "\nDescending: ";
        for (int s : speeds) std::cout << s << " ";
        std::cout << "\n\n";
    }

    // ─── 2. std::stable_sort — Preserves Equal Order ─────────────────────────
    {
        std::cout << "--- 2. std::stable_sort (preserves relative order) ---\n";

        std::vector<SensorReading> alerts = {
            {"OilTemp", 95.0, 2},    {"Coolant", 102.0, 1},
            {"TireFL", 32.0, 3},     {"BrakeFL", 450.0, 1},
            {"Battery", 12.1, 3},    {"FuelLvl", 25.0, 2}};

        // EN: stable_sort by priority — equal priorities keep original order
        // TR: Önceliğe göre stable_sort — eşit öncelikler sırasını korur
        std::stable_sort(alerts.begin(), alerts.end(),
                         [](const SensorReading& a, const SensorReading& b) {
                             return a.priority < b.priority;
                         });

        std::cout << "By priority (stable): ";
        for (const auto& s : alerts) std::cout << s << " ";
        std::cout << "\n\n";
    }

    // ─── 3. std::partial_sort — Top-N ────────────────────────────────────────
    {
        std::cout << "--- 3. std::partial_sort (Top-3 fastest) ---\n";

        std::vector<int> lapTimes = {92, 88, 95, 85, 91, 87, 98, 83};

        // EN: Sort only the first 3 elements — rest is unspecified order More efficient than
        // full sort when you only need top-N!
        // TR: Sadece ilk 3 elemanı sırala — geri kalan belirsiz sırada Sadece ilk-N gerektiğinde
        // tam sıralamadan daha verimli!
        std::partial_sort(lapTimes.begin(), lapTimes.begin() + 3,
                          lapTimes.end());

        std::cout << "Top 3 fastest laps: ";
        for (int i = 0; i < 3; ++i) std::cout << lapTimes[static_cast<size_t>(i)] << "s ";
        std::cout << "\n\n";
    }

    // ─── 4. std::nth_element — Selection ─────────────────────────────────────
    {
        std::cout << "--- 4. std::nth_element (median finder) ---\n";

        std::vector<int> temps = {72, 88, 65, 91, 78, 85, 70};
        size_t mid = temps.size() / 2;
```

```cpp

        // EN: After nth_element: temps[mid] is the median. Elements before it are ≤ it, after
        // are ≥ it (unordered). O(N) average — much faster than O(N log N) full sort!
        // TR: nth_element sonrası: temps[mid] medyan. Öncesindekiler ≤, sonrasındakiler ≥
        // (sırasız). Ortalama O(N) — tam sıralamadan çok daha hızlı!
        std::nth_element(temps.begin(), temps.begin() + static_cast<long>(mid),
                         temps.end());
        std::cout << "Median temperature: " << temps[mid] << "°C\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [SEARCHING FAMILY / ARAMA AİLESİ]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 5. std::find / std::find_if ──────────────────────────────────────────
    {
        std::cout << "--- 5. std::find / std::find_if ---\n";

        std::vector<int> errorCodes = {0, 0, 301, 0, 420, 0, 171};

        // EN: find first non-zero error code
        // TR: ilk sıfır olmayan hata kodunu bul
        auto it = std::find_if(errorCodes.begin(), errorCodes.end(),
                               [](int code) { return code != 0; });
        if (it != errorCodes.end()) {
            std::cout << "First error: P0" << *it
                      << " at index " << std::distance(errorCodes.begin(), it)
                      << "\n\n";
        }
    }

    // ─── 6. Binary Search Family (SORTED ranges only!) ────────────────────────
    {
        std::cout << "--- 6. Binary Search (lower/upper_bound, equal_range) ---\n";

        // EN: MUST be sorted for binary search algorithms!
        // TR: Binary search algoritmaları için SIRALANMIŞ olmalı!
        std::vector<int> sortedRPMs = {1000, 1500, 2000, 2500, 3000,
                                       3000, 3000, 3500, 4000, 5000};

        // EN: lower_bound — first element ≥ value (insertion point)
        // TR: lower_bound — değere ≥ ilk eleman (ekleme noktası)
        auto lo = std::lower_bound(sortedRPMs.begin(), sortedRPMs.end(), 3000);
        std::cout << "lower_bound(3000): index "
                  << std::distance(sortedRPMs.begin(), lo) << "\n";

        // EN: upper_bound — first element > value
        // TR: upper_bound — değerden > ilk eleman
        auto up = std::upper_bound(sortedRPMs.begin(), sortedRPMs.end(), 3000);
        std::cout << "upper_bound(3000): index "
                  << std::distance(sortedRPMs.begin(), up) << "\n";

        // EN: equal_range — returns [lower_bound, upper_bound) pair Count of 3000s = upper -
        // lower
        // TR: equal_range — [lower_bound, upper_bound) çifti döndürür
        auto [eqLo, eqUp] = std::equal_range(sortedRPMs.begin(),
                                             sortedRPMs.end(), 3000);
        std::cout << "Count of 3000 RPM readings: "
                  << std::distance(eqLo, eqUp) << "\n";
```

```cpp

        // EN: binary_search — just returns true/false (exists or not)
        // TR: binary_search — sadece true/false döndürür (var mı yok mu)
        bool has4500 = std::binary_search(sortedRPMs.begin(),
                                          sortedRPMs.end(), 4500);
        std::cout << "Has 4500 RPM? " << (has4500 ? "yes" : "no") << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════
    // [MODIFYING ALGORITHMS / DEĞİŞTİRİCİ ALGORİTMALAR]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 7. std::transform — Map Operation ─────────────────────────────────
    {
        std::cout << "--- 7. std::transform (map operation) ---\n";

        std::vector<double> celsiusReadings = {22.5, 18.0, 35.2, 0.0, -5.0};
        std::vector<double> fahrenheit;

        // EN: Convert Celsius → Fahrenheit using transform
        // TR: Transform ile Celsius → Fahrenheit dönüşümü
        std::transform(celsiusReadings.begin(), celsiusReadings.end(),
                       std::back_inserter(fahrenheit),
                       [](double c) { return c * 9.0 / 5.0 + 32.0; });

        std::cout << "Fahrenheit: ";
        for (double f : fahrenheit) std::cout << f << " ";
        std::cout << "\n\n";
    }

    // ─── 8. std::remove_if + erase (Erase-Remove Idiom) ────────────────────
    {
        std::cout << "--- 8. Erase-Remove Idiom ---\n";

        std::vector<int> readings = {85, -1, 72, -1, 91, 68, -1, 77};

        // EN: remove_if moves "removed" elements to the end, returns new logical end. You MUST
        // call erase() to actually shrink! -1 represents invalid/error readings.
        // TR: remove_if "silinen" elemanları sona taşır, yeni mantıksal sonu döndürür. Gerçekten
        // küçültmek için erase() ÇAĞIRMALISIN!
        auto newEnd = std::remove_if(readings.begin(), readings.end(),
                                     [](int r) { return r < 0; });
        readings.erase(newEnd, readings.end());

        std::cout << "After removing invalid (-1): ";
        for (int r : readings) std::cout << r << " ";
        std::cout << " (size=" << readings.size() << ")\n\n";
    }

    // ─── 9. std::unique — Remove Consecutive Duplicates ────────────────────
    {
        std::cout << "--- 9. std::unique (consecutive duplicates) ---\n";

        std::vector<int> canFrames = {100, 100, 200, 200, 200, 300, 100, 100};

        // EN: unique only removes CONSECUTIVE duplicates — sort first for all!
        // TR: unique sadece ARDIŞIK tekrarları siler — hepsi için önce sırala!
        auto it = std::unique(canFrames.begin(), canFrames.end());
        canFrames.erase(it, canFrames.end());

        std::cout << "After unique: ";
        for (int f : canFrames) std::cout << f << " ";
        std::cout << "\n\n";
    }

    // ─── 10. std::rotate — Circular Shift ──────────────────────────────────
    {
        std::cout << "--- 10. std::rotate ---\n";

        std::vector<int> buffer = {1, 2, 3, 4, 5, 6, 7, 8};
```

```cpp

        // EN: Rotate left by 3 positions: element at position 3 becomes first
        // TR: 3 pozisyon sola döndür: 3. pozisyondaki eleman ilk olur
        std::rotate(buffer.begin(), buffer.begin() + 3, buffer.end());

        std::cout << "After rotate left 3: ";
        for (int b : buffer) std::cout << b << " ";
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [PARTITIONING / BÖLÜMLEME]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 11. std::partition / std::stable_partition ──────────────────────────────
    {
        std::cout << "--- 11. std::partition ---\n";

        std::vector<SensorReading> data = {
            {"OilTemp", 95.0, 2},  {"Coolant", 110.0, 1},
            {"TireFL", 32.0, 3},   {"BrakeFL", 500.0, 1},
            {"Battery", 12.1, 3},  {"FuelLvl", 15.0, 2}};

        // EN: Partition into critical (priority ≤ 2) and non-critical
        // TR: Kritik (öncelik ≤ 2) ve kritik olmayan olarak böl
        auto boundary = std::stable_partition(
            data.begin(), data.end(),
            [](const SensorReading& s) { return s.priority <= 2; });

        std::cout << "Critical alerts: ";
        for (auto it = data.begin(); it != boundary; ++it) {
            std::cout << it->name << " ";
        }
        std::cout << "\nInfo alerts: ";
        for (auto it = boundary; it != data.end(); ++it) {
            std::cout << it->name << " ";
        }
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [NUMERIC ALGORITHMS / SAYISAL ALGORİTMALAR]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 12. std::accumulate, std::reduce (C++17) ────────────────────────────────
    {
        std::cout << "--- 12. std::accumulate & std::reduce ---\n";

        std::vector<double> fuelConsumption = {7.2, 8.1, 6.5, 9.0, 7.8};

        // EN: accumulate — sequential fold (guarantees left-to-right)
        // TR: accumulate — sıralı katlama (soldan sağa garanti)
        double totalFuel = std::accumulate(fuelConsumption.begin(),
                                           fuelConsumption.end(), 0.0);
        std::cout << "Total fuel: " << totalFuel << " L\n";
        std::cout << "Average: " << totalFuel / static_cast<double>(fuelConsumption.size())
                  << " L/100km\n";

        // EN: reduce (C++17) — may reorder operations (parallelizable!) For floating point,
        // reduce may give slightly different results due to reordering. For integers, identical
        // to accumulate.
        // TR: reduce (C++17) — işlemleri yeniden sıralayabilir (paralelleştirilebilir!)
        double totalReduce = std::reduce(fuelConsumption.begin(),
                                         fuelConsumption.end(), 0.0);
        std::cout << "reduce total: " << totalReduce << " L\n\n";
    }

    // ─── 13. std::inner_product — Dot Product ────────────────────────────────────
    {
        std::cout << "--- 13. std::inner_product ---\n";
```

```

        // EN: Weighted sensor scores: score = Σ(value × weight)
        // TR: Ağırlıklı sensör skorları: skor = Σ(değer × ağırlık)
        std::vector<double> values = {95.0, 78.0, 88.0, 92.0};
        std::vector<double> weights = {0.3, 0.2, 0.25, 0.25};

        double weightedScore = std::inner_product(
            values.begin(), values.end(), weights.begin(), 0.0);
        std::cout << "Weighted health score: " << weightedScore << "\n\n";
    }

    // ─── 14. std::partial_sum — Running Total ──────────────────────────────────────
    {
        std::cout << "--- 14. std::partial_sum (cumulative) ---\n";

        std::vector<int> dailyKm = {120, 85, 200, 150, 95};
        std::vector<int> cumulative;

        // EN: Cumulative distance: [120, 205, 405, 555, 650]
        // TR: Kümülatif mesafe: [120, 205, 405, 555, 650]
        std::partial_sum(dailyKm.begin(), dailyKm.end(),
                         std::back_inserter(cumulative));

        std::cout << "Daily km: ";
        for (int d : dailyKm) std::cout << d << " ";
        std::cout << "\nCumulative: ";
        for (int c : cumulative) std::cout << c << " ";
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════════
    // [PREDICATES & CONDITIONS / KOŞULLAR]
    // ═══════════════════════════════════════════════════════════════════════════════

    // ─── 15. all_of / any_of / none_of ─────────────────────────────────────────────
    {
        std::cout << "--- 15. all_of / any_of / none_of ---\n";

        std::vector<int> tirePressures = {32, 33, 31, 34};

        bool allSafe = std::all_of(tirePressures.begin(), tirePressures.end(),
                                   [](int p) { return p >= 28 && p <= 36; });
        bool anyLow = std::any_of(tirePressures.begin(), tirePressures.end(),
                                  [](int p) { return p < 28; });
        bool noneFlat = std::none_of(tirePressures.begin(), tirePressures.end(),
                                     [](int p) { return p == 0; });

        std::cout << "All tires safe (28-36 PSI)? " << (allSafe ? "yes" : "no") << "\n";
        std::cout << "Any tire low (<28 PSI)? " << (anyLow ? "yes" : "no") << "\n";
        std::cout << "No flat tires? " << (noneFlat ? "yes" : "no") << "\n\n";
    }

    // ─── 16. std::count / std::count_if ────────────────────────────────────────────
    {
        std::cout << "--- 16. std::count_if ---\n";

        std::vector<int> speedLog = {60, 85, 120, 130, 95, 140, 110, 70};
        int speedLimit = 120;

        auto violations = std::count_if(
            speedLog.begin(), speedLog.end(),
            [speedLimit](int s) { return s > speedLimit; });
        std::cout << "Speed violations (>" << speedLimit << " km/h): "
                  << violations << "\n\n";
    }
```

```cpp

    // ═══════════════════════════════════════════════════════════════════════════
    // [MIN/MAX / MİN/MAKS]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 17. min_element / max_element / minmax_element / clamp ──────────────
    {
        std::cout << "--- 17. min/max/clamp ---\n";

        std::vector<double> voltages = {12.4, 11.8, 13.2, 12.9, 11.5, 13.8};

        auto [minIt, maxIt] = std::minmax_element(voltages.begin(),
                                                   voltages.end());
        std::cout << "Min voltage: " << *minIt << "V\n";
        std::cout << "Max voltage: " << *maxIt << "V\n";

        // EN: clamp (C++17) — restrict a value to [lo, hi] range
        // TR: clamp (C++17) — değeri [lo, hi] aralığına kısıtla
        double rawThrottle = 1.15;
        double clamped = std::clamp(rawThrottle, 0.0, 1.0);
        std::cout << "Throttle " << rawThrottle << " clamped to [0,1]: "
                  << clamped << "\n\n";
    }

    // ─── 18. std::for_each with Side Effects ─────────────────────────────────
    {
        std::cout << "--- 18. std::for_each ---\n";

        std::vector<std::string> modules = {"ECM", "TCM", "BCM", "ABS"};

        int index = 0;
        std::for_each(modules.begin(), modules.end(),
                      [&index](const std::string& m) {
                          std::cout << "  [" << index++ << "] " << m << "\n";
                      });
        std::cout << "\n";
    }

    // ─── 19. std::generate — Fill with Computed Values ───────────────────────
    {
        std::cout << "--- 19. std::generate ---\n";

        std::vector<int> sensorIDs(8);
        int nextID = 1000;

        // EN: Generate sequential sensor IDs starting from 1000
        // TR: 1000'den başlayarak sıralı sensör ID'leri üret
        std::generate(sensorIDs.begin(), sensorIDs.end(),
                      [&nextID]() { return nextID++; });

        std::cout << "Generated IDs: ";
        for (int id : sensorIDs) std::cout << id << " ";
        std::cout << "\n\n";
    }

    // ─── 20. std::mismatch — Find First Difference ───────────────────────────
    {
        std::cout << "--- 20. std::mismatch ---\n";

        std::vector<int> expected = {100, 200, 300, 400, 500};
        std::vector<int> actual   = {100, 200, 350, 400, 500};

        // EN: Find where calibration data diverges from expected values
        // TR: Kalibrasyon verilerinin beklenen değerlerden nerede saptığını bul
        auto [expIt, actIt] = std::mismatch(expected.begin(), expected.end(),
                                            actual.begin());
        if (expIt != expected.end()) {
            std::cout << "First mismatch at index "
                      << std::distance(expected.begin(), expIt)
                      << ": expected=" << *expIt
                      << " actual=" << *actIt << "\n";
        }
    }

    std::cout << "\n=== ALGORITHMS MASTERCLASS COMPLETE ===\n";
    return 0;
}
```

► 06. Associative Containers

module_04_advanced/06_associative_containers.cpp

Konu: Templates & STL: Associative & Unordered Containers / Şablonlar ve STL: İlişkisel ve

Teori / Theory

——— Ordered Containers (Sıralı Konteynerler) ———

THEORY: Ordered vs Unordered — Red-Black Tree vs Hash Table

std::set, std::map, std::multiset, std::multimap Backed by Red-Black Tree (self-balancing BST). ✓ Always sorted by key ✓ O(log N) insert, find, erase ✓ Supports ordered traversal, lower_bound, upper_bound ✗ Slower than hash for pure lookup

TEORİ: Sıralı vs Sırasız — Kırmızı-Siyah Ağaç vs Hash Tablosu

std::set, std::map, std::multiset, std::multimap Kırmızı-Siyah Ağaç (kendi kendini dengeleyen BST) ile desteklenir. ✓ Her zaman anahtara göre sıralı ✓ O(log N) ekleme, bulma, silme ✓ Sıralı gezinme, lower_bound, upper_bound destekler ✗ Saf arama için hash'ten yavaş
——— Unordered Containers (Sırasız Konteynerler) ———

std::unordered_set, std::unordered_map, std::unordered_multiset/map Backed by Hash Table (bucket array + linked lists). ✓ O(1) average insert, find, erase ✓ Fastest for pure key lookup ✗ No ordering guarantee ✗ Worst case O(N) if many hash collisions ✗ Requires hash function + equality operator

std::unordered_set, std::unordered_map, std::unordered_multiset/map Hash Tablosu (kova dizisi + bağlı listeler) ile desteklenir. ✓ Ortalama O(1) ekleme, bulma, silme ✓ Saf anahtar araması için en hızlı ✗ Sıralama garantisi yok ✗ Çok çarpışma varsa en kötü O(N) ✗ Hash fonksiyonu + eşitlik operatörü gerektirir
——— Container Adaptors (Konteyner Adaptörleri) ———

std::stack (LIFO), std::queue (FIFO), std::priority_queue (max-heap). These are NOT real containers — they WRAP an underlying container (default: deque for stack/queue, vector for priority_queue) and restrict its interface.

std::stack (LIFO), std::queue (FIFO), std::priority_queue (max-heap). Bunlar gerçek konteyner DEĞİL — alttaki bir konteyneri SARARLAR (varsayılan: stack/queue için deque, priority_queue için vector) ve arayüzünü kısıtlarlar.

CPPREF Depth: Red-Black Trees vs Hash Tables — When Order Matters

`std::map` / `std::set` are red-black trees (self-balancing BST): • All operations O(log n) — insert, find, erase. • Iteration yields elements in SORTED order by key. • Iterators are stable: insert/erase never invalidates other iterators. `std::unordered_map` / `std::unordered_set` use hash tables: • Average O(1) for insert/find/erase, worst-case O(n) (all collide). • No ordering guarantee — iteration order is implementation-defined. • Load factor = size / bucket_count. When load_factor > max_load_factor (default 1.0), a REHASH occurs: new bucket array allocated, all elements re-hashed — O(n) and ALL iterators are invalidated! Custom hash: specialize `std::hash<T>` or pass as template argument: `std::unordered_map<K, V, MyHash, MyEqual>` A good hash distributes uniformly. Combine fields with: `seed ^= std::hash<T>{}(val) + 0x9e3779b9 + (seed<<6) + (seed>>2);` C++17 `try_emplace()` avoids constructing the value if key exists — more efficient than `emplace()` for expensive-to-construct values.

CPPREF Derinlik: Kırmızı-Siyah Ağaç vs Hash Tablosu — Sıra Ne Zaman Önemli

`std::map` / `std::set` kırmızı-siyah ağaçtır (dengeli BST): • Tüm işlemler O(log n) — ekleme, arama, silme. • İterasyon elemanları anahtara göre SIRALI verir. • İteratörler kararlıdır: ekleme/silme diğer iteratörleri bozmaz. `std::unordered_map` / `std::unordered_set` hash tablosu kullanır: • Ortalama O(1), en kötü O(n) (tüm anahtarlar çakışırsa). • Sıralama garantisi YOKTUR — gezinme sırası uygulama tanımlıdır. • Yük faktörü = size / bucket_count. max_load_factor (varsayılan 1.0) aşılınca REHASH olur: yeni kova dizisi, tüm elemanlar yeniden hash'lenir — O(n) ve TÜM iteratörler geçersiz olur! Özel hash: `std::hash<T>` özelleştirmesi veya şablon argümanı: `std::unordered_map<K, V, MyHash, MyEqual>` İyi bir hash fonksiyonu düzgün dağılım sağlar. Alanları birleştirmek için: `seed ^= hash(val) + 0x9e3779b9 + (seed<<6) + (seed>>2);` C++17 `try_emplace()` anahtar varsa değer oluşturmaktan kaçınır — pahalı nesnelerde `emplace()`'ten daha verimlidir.

Kaynak Kod / Source Code

```cpp
#include <algorithm>       // EN: STL algorithms
#include <functional>      // EN: std::greater
#include <iostream>        // EN: Standard I/O
#include <map>             // EN: std::map, std::multimap
#include <queue>           // EN: std::queue, std::priority_queue
#include <set>             // EN: std::set, std::multiset
#include <stack>           // EN: std::stack
#include <string>          // EN: std::string
#include <unordered_map>   // EN: std::unordered_map
#include <unordered_set>   // EN: std::unordered_set
#include <vector>          // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. Custom Comparator for std::set ─────────────────────────────────

// EN: Diagnostic Trouble Code (DTC) with severity and timestamp.
// TR: Arıza Kodu (DTC) — şiddet derecesi ve zaman damgası ile.
struct DTC {
    std::string code;     // EN: e.g. "P0301"
    int severity;         // EN: 1=critical..5=info
    int timestampMs;      // EN: milliseconds since boot

    friend std::ostream& operator<<(std::ostream& os, const DTC& d) {
        return os << d.code << "(sev=" << d.severity
                  << ",t=" << d.timestampMs << "ms)";
    }
};
```

```cpp

// EN: Custom comparator: sort DTCs by severity first, then by time. This is a FUNCTOR — std::set
// stores it internally.
// TR: Özel karşılaştırıcı: DTC'leri önce şiddete, sonra zamana göre sırala. Bu bir FUNCTOR —
// std::set bunu dahili olarak saklar.
struct DTCSeverityOrder {
    bool operator()(const DTC& a, const DTC& b) const {
        if (a.severity != b.severity) return a.severity < b.severity;
        return a.timestampMs < b.timestampMs;
    }
};

// ─── 2. Custom Hash for unordered_map ──────────────────────────────────────────────────

// EN: Sensor key: ECU ID + Sensor Index. We need a custom hash because std::pair doesn't have a
// default hash in C++.
// TR: Sensör anahtarı: ECU ID + Sensör İndeksi. std::pair'in C++'ta varsayılan hash'i olmadığı
// için özel hash gerekir.
struct SensorKey {
    int ecuId;
    int sensorIndex;

    bool operator==(const SensorKey& other) const {
        return ecuId == other.ecuId && sensorIndex == other.sensorIndex;
    }
};

struct SensorKeyHash {
    size_t operator()(const SensorKey& k) const {
        // EN: Combine two hashes using XOR + bit shift (a common technique)
        // TR: İki hash'i XOR + bit kaydırma ile birleştir (yaygın teknik)
        size_t h1 = std::hash<int>{}(k.ecuId);
        size_t h2 = std::hash<int>{}(k.sensorIndex);
        return h1 ^ (h2 << 1);
    }
};

// ─── 3. Priority Queue for Alert Processing ────────────────────────────────────────────

// EN: Vehicle alert with priority level.
// TR: Öncelik seviyeli araç uyarısı.
struct VehicleAlert {
    std::string message;
    int priority;  // EN: lower = more urgent

    // EN: priority_queue is max-heap by default, so we invert: higher priority number = LESS
    // urgent → goes to bottom
    // TR: priority_queue varsayılan max-heap, bu yüzden ters çeviriyoruz: yüksek öncelik
    // numarası = DAHA AZ acil → alta gider
    bool operator<(const VehicleAlert& other) const {
        return priority > other.priority;  // min-heap behavior
    }
};

// ═══════════════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: ASSOCIATIVE & UNORDERED CONTAINERS ===\n"
              << "=== MODÜL 4: İLİŞKİSEL VE SIRASIZ KONTEYNERLER ===\n\n";

    // ═══════════════════════════════════════════════════════════════════════════════════
    // [ORDERED CONTAINERS / SIRALI KONTEYNERLER]
    // ═══════════════════════════════════════════════════════════════════════════════════

    // ─── 1. std::map — Sorted Key-Value ────────────────────────────────────────────────
    {
        std::cout << "--- 1. std::map (sorted key-value) ---\n";

        std::map<std::string, double> sensorData;
```

```cpp

        // EN: Insert via [] operator (creates if not exists)
        // TR: [] operatörü ile ekle (yoksa oluşturur)
        sensorData["OilTemp"] = 92.5;
        sensorData["Coolant"] = 88.0;
        sensorData["Battery"] = 12.6;

        // EN: Insert via insert() — does NOT overwrite existing!
        // TR: insert() ile ekle — mevcut olanın üzerine YAZMAZ!
        auto [it1, inserted1] = sensorData.insert({"OilTemp", 999.0});
        std::cout << "OilTemp insert: " << (inserted1 ? "new" : "exists")
                  << " val=" << it1->second << "\n";

        // EN: insert_or_assign() (C++17) — inserts or overwrites
        // TR: insert_or_assign() (C++17) — ekler veya üzerine yazar
        sensorData.insert_or_assign("OilTemp", 95.0);

        // EN: Traversal is ALWAYS sorted by key (alphabetical for string)
        // TR: Gezinme HER ZAMAN anahtara göre sıralı (string için alfabetik)
        std::cout << "All sensors (sorted):\n";
        for (const auto& [key, val] : sensorData) {
            std::cout << "  " << key << " = " << val << "\n";
        }

        // EN: find() returns iterator — O(log N)
        // TR: find() iterator döndürür — O(log N)
        auto found = sensorData.find("Coolant");
        if (found != sensorData.end()) {
            std::cout << "Found: " << found->first << " = " << found->second << "\n";
        }

        // EN: count() — 0 or 1 for map (useful for existence check)
        // TR: count() — map için 0 veya 1 (varlık kontrolü için kullanışlı)
        std::cout << "Has 'Turbo'? " << sensorData.count("Turbo") << "\n\n";
    }

    // ─── 2. std::multimap — Multiple Values Per Key ──────────────────────────
    {
        std::cout << "--- 2. std::multimap (multiple values per key) ---\n";

        std::multimap<int, std::string> errorsByHour;

        // EN: Same key (hour) can have multiple error entries
        // TR: Aynı anahtar (saat) birden fazla hata girişine sahip olabilir
        errorsByHour.insert({9, "P0301 — Cylinder misfire"});
        errorsByHour.insert({9, "P0420 — Catalyst efficiency"});
        errorsByHour.insert({10, "P0171 — Lean mixture"});
        errorsByHour.insert({9, "C0035 — ABS sensor"});

        // EN: equal_range returns [first, last) for a given key
        // TR: equal_range verilen anahtar için [ilk, son) döndürür
        auto [lo, hi] = errorsByHour.equal_range(9);
        std::cout << "Errors at hour 9:\n";
        for (auto it = lo; it != hi; ++it) {
            std::cout << "  " << it->second << "\n";
        }
        std::cout << "Total at hour 9: " << errorsByHour.count(9) << "\n\n";
    }

    // ─── 3. std::set with Custom Comparator ──────────────────────────────────
    {
        std::cout << "--- 3. std::set<DTC, DTCSeverityOrder> ---\n";

        std::set<DTC, DTCSeverityOrder> dtcQueue;

        dtcQueue.insert({"P0301", 3, 1500});
        dtcQueue.insert({"P0420", 2, 800});
        dtcQueue.insert({"C0035", 1, 200});
        dtcQueue.insert({"B0100", 4, 3000});
        dtcQueue.insert({"P0171", 1, 1200});
```

```cpp

        // EN: Iteration is in severity order (1=critical first)
        // TR: Gezinme şiddet sırasında (1=kritik ilk)
        std::cout << "DTCs by severity:\n";
        for (const auto& dtc : dtcQueue) {
            std::cout << "  " << dtc << "\n";
        }
        std::cout << "\n";
    }

    // ─── 4. std::set — Set Operations ──────────────────────────────────────────
    {
        std::cout << "--- 4. Set Operations (union, intersection, diff) ---\n";

        std::set<std::string> mondayDTCs = {"P0301", "P0420", "C0035", "P0171"};
        std::set<std::string> tuesdayDTCs = {"P0420", "B0100", "P0171", "U0100"};

        // EN: set_intersection — DTCs that appeared BOTH days
        // TR: set_intersection — HER İKİ gün görülen DTC'ler
        std::vector<std::string> persistent;
        std::set_intersection(mondayDTCs.begin(), mondayDTCs.end(),
                              tuesdayDTCs.begin(), tuesdayDTCs.end(),
                              std::back_inserter(persistent));
        std::cout << "Persistent DTCs: ";
        for (const auto& d : persistent) std::cout << d << " ";

        // EN: set_difference — Monday only (cleared by Tuesday)
        // TR: set_difference — Sadece Pazartesi (Salı'ya kadar temizlendi)
        std::vector<std::string> clearedDTCs;
        std::set_difference(mondayDTCs.begin(), mondayDTCs.end(),
                            tuesdayDTCs.begin(), tuesdayDTCs.end(),
                            std::back_inserter(clearedDTCs));
        std::cout << "\nCleared DTCs: ";
        for (const auto& d : clearedDTCs) std::cout << d << " ";

        // EN: set_union — all unique DTCs across both days
        // TR: set_union — iki gündeki tüm benzersiz DTC'ler
        std::set<std::string> allDTCs;
        std::set_union(mondayDTCs.begin(), mondayDTCs.end(),
                       tuesdayDTCs.begin(), tuesdayDTCs.end(),
                       std::inserter(allDTCs, allDTCs.begin()));
        std::cout << "\nAll unique DTCs: ";
        for (const auto& d : allDTCs) std::cout << d << " ";
        std::cout << "\n\n";
    }

    // ═══════════════════════════════════════════════════════════════════════════
    // [UNORDERED CONTAINERS / SIRASIZ KONTEYNERLER]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 5. std::unordered_map — O(1) Lookup ───────────────────────────────────
    {
        std::cout << "--- 5. std::unordered_map (hash table) ---\n";

        std::unordered_map<std::string, int> ecuModules;
        // EN: Engine Control Module address
        // TR: Motor Kontrol Modülü adresi
        ecuModules["ECM"] = 0x7E0;
        // EN: Transmission Control Module
        // TR: Şanzıman Kontrol Modülü
        ecuModules["TCM"] = 0x7E1;
        // EN: Anti-lock Braking System
        // TR: Kilitlenme Önleyici Fren Sistemi
        ecuModules["ABS"] = 0x7B0;
        // EN: Body Control Module
        // TR: Gövde Kontrol Modülü
        ecuModules["BCM"] = 0x740;
```

```cpp

        // EN: O(1) average lookup — much faster than map for large datasets
        // TR: Ortalama O(1) arama — büyük veri setleri için map'ten çok daha hızlı
        std::cout << "ECM address: 0x" << std::hex << ecuModules["ECM"]
                  << std::dec << "\n";

        // EN: Bucket information — shows hash table internals
        // TR: Kova bilgisi — hash tablosu iç yapısını gösterir
        std::cout << "Buckets: " << ecuModules.bucket_count()
                  << ", Load factor: " << ecuModules.load_factor()
                  << "\n\n";
    }

    // ─── 6. unordered_map with Custom Hash ──────────────────────────────────────
    {
        std::cout << "--- 6. unordered_map with Custom Hash ---\n";

        std::unordered_map<SensorKey, double, SensorKeyHash> sensorMap;

        // EN: Key = {ECU_ID, SensorIndex}, Value = reading
        // TR: Anahtar = {ECU_ID, SensörIndeks}, Değer = okuma
        sensorMap[{0x7E0, 0}] = 92.5;   // ECM, OilTemp
        sensorMap[{0x7E0, 1}] = 88.0;   // ECM, Coolant
        sensorMap[{0x7B0, 0}] = 32.0;   // ABS, WheelSpeedFL
        sensorMap[{0x7B0, 1}] = 33.0;   // ABS, WheelSpeedFR

        SensorKey query{0x7B0, 0};
        auto it = sensorMap.find(query);
        if (it != sensorMap.end()) {
            std::cout << "ABS WheelSpeedFL: " << it->second << "\n\n";
        }
    }

    // ─── 7. std::unordered_set — Fast Membership Test ───────────────────────────
    {
        std::cout << "--- 7. std::unordered_set (fast contains) ---\n";

        // EN: Blacklisted VIN prefixes (known recall vehicles)
        // TR: Kara listedeki VIN ön ekleri (bilinen geri çağırma araçları)
        std::unordered_set<std::string> recallVINs = {
            "1G1YY", "2T1BU", "3FA6P", "5YJ3E", "JN1TB"};

        std::string testVIN = "3FA6P";
        // EN: O(1) average — contains() is C++20, use count() for C++17
        // TR: Ortalama O(1) — contains() C++20, C++17 için count() kullan
        if (recallVINs.count(testVIN) > 0) {
            std::cout << "⚠ VIN " << testVIN << " is in recall list!\n\n";
        }
    }

    // ═══════════════════════════════════════════════════════════════════════════
    // [CONTAINER ADAPTORS / KONTEYNER ADAPTÖRLERİ]
    // ═══════════════════════════════════════════════════════════════════════════

    // ─── 8. std::stack — LIFO (Undo History) ────────────────────────────────────
    {
        std::cout << "--- 8. std::stack (LIFO — undo history) ---\n";

        std::stack<std::string> undoHistory;

        // EN: Each user action is pushed onto the stack
        // TR: Her kullanıcı aksiyonu yığına itilir
        undoHistory.push("Set RPM limit to 6000");
        undoHistory.push("Enable traction control");
        undoHistory.push("Set drive mode to Sport");

        std::cout << "Undo sequence:\n";
        while (!undoHistory.empty()) {
            std::cout << "  Undo: " << undoHistory.top() << "\n";
            undoHistory.pop();
        }
        std::cout << "\n";
    }
```

```cpp

    // ─── 9. std::queue — FIFO (Message Queue) ──────────────────────────────────────
    {
        std::cout << "--- 9. std::queue (FIFO — message processing) ---\n";

        std::queue<std::string> canBusQueue;

        // EN: CAN bus messages arrive in order and must be processed FIFO
        // TR: CAN bus mesajları sırayla gelir ve FIFO işlenmelidir
        canBusQueue.push("0x7E0: Engine RPM = 3200");
        canBusQueue.push("0x7B0: Wheel Speed = 85");
        canBusQueue.push("0x740: Door Status = Closed");

        std::cout << "Processing CAN messages:\n";
        while (!canBusQueue.empty()) {
            std::cout << "  → " << canBusQueue.front() << "\n";
            canBusQueue.pop();
        }
        std::cout << "\n";
    }

    // ─── 10. std::priority_queue — Alert Dispatcher ────────────────────────────────
    {
        std::cout << "--- 10. std::priority_queue (alert dispatcher) ---\n";

        std::priority_queue<VehicleAlert> alertQueue;

        // EN: Push alerts with different priorities
        // TR: Farklı önceliklerle uyarılar ekle
        alertQueue.push({"Low washer fluid", 5});
        alertQueue.push({"ENGINE OVERHEATING!", 1});
        alertQueue.push({"Tire pressure low", 3});
        alertQueue.push({"BRAKE FAILURE!", 1});
        alertQueue.push({"Service due in 500 km", 4});

        // EN: Dequeue processes the most urgent (lowest number) first!
        // TR: Kuyruktan çıkarma en acil olanı (en düşük numara) ilk işler!
        std::cout << "Alert processing order:\n";
        while (!alertQueue.empty()) {
            const auto& a = alertQueue.top();
            std::cout << "  [P" << a.priority << "] " << a.message << "\n";
            alertQueue.pop();
        }
        std::cout << "\n";
    }

    // ─── 11. std::map — Structured Bindings & try_emplace ──────────────────────────
    {
        std::cout << "--- 11. C++17: try_emplace & extract ---\n";

        std::map<int, std::string> ecuRegistry;
        ecuRegistry[0x7E0] = "ECM";
        ecuRegistry[0x7B0] = "ABS";

        // EN: try_emplace (C++17) — only constructs value if key doesn't exist More efficient
        // than insert for expensive-to-construct values
        // TR: try_emplace (C++17) — anahtar yoksa değeri oluşturur Pahalı oluşturma maliyetli
        // değerler için insert'ten verimli
        auto [it, ok] = ecuRegistry.try_emplace(0x7E0, "DUPLICATE");
        std::cout << "try_emplace 0x7E0: " << (ok ? "inserted" : "exists")
                  << " -> " << it->second << "\n";
```

```

        // EN: extract (C++17) — remove a node without destroying it Can be re-inserted with a
        // different key!
        // TR: extract (C++17) — düğümü yok etmeden çıkar Farklı bir anahtarla tekrar
        // eklenebilir!
        auto node = ecuRegistry.extract(0x7B0);
        if (!node.empty()) {
            // EN: Change key!
            // TR: Anahtarı değiştir!
            node.key() = 0x7B1;
            ecuRegistry.insert(std::move(node));
        }

        std::cout << "After extract+reinsert:\n";
        for (const auto& [addr, name] : ecuRegistry) {
            std::cout << "  0x" << std::hex << addr << std::dec
                      << " → " << name << "\n";
        }
    }

    std::cout << "\n=== ASSOCIATIVE CONTAINERS COMPLETE ===\n";
    return 0;
}
```

► 07. String Operations

module_04_advanced/07_string_operations.cpp

Konu: Templates & STL: String Operations & string_view / Şablonlar ve STL: String İşlemleri

Teori / Theory

——— std::string Memory Model (Bellek Modeli) ———

THEORY: std::string Internals & string_view

std::string is a dynamic char array with: - Heap-allocated buffer (for strings > SSO threshold) - size(): number of characters - capacity(): allocated buffer size (can be > size)
- SSO (Small String Optimization): strings ≤ ~15-22 chars (implementation-dependent) are stored INSIDE the object itself — no heap allocation! This makes short strings very fast.

TEORİ: std::string İç Yapısı ve string_view

std::string, şu özelliklere sahip dinamik bir char dizisidir: - Heap tahsisli tampon (SSO eşiğinden büyük stringler için) - size(): karakter sayısı - capacity(): tahsis edilmiş tampon boyutu (size'dan büyük olabilir) - SSO (Küçük String Optimizasyonu): ≤ ~15-22 karakter stringler (uygulamaya bağlı) nesnenin KENDİSİNDE depolanır — heap tahsisi yok! Bu kısa stringleri çok hızlı yapar.
——— std::string_view (C++17) ———

A non-owning, read-only view into a string. Like a "window" that points to someone else's data without copying it. ✓ No allocation, no copy — just pointer + size ✓ Can view std::string, const char*, string literals ✓ Perfect for function parameters that only READ strings ✗ DANGER: The viewed string must outlive the string_view! If the original string is destroyed → dangling view!

Stringe sahiplik almayan, salt okunur bir görünüm. Kopyalamadan başkasının verisine işaret eden bir "pencere" gibi. ✓ Tahsis yok, kopya yok — sadece pointer + boyut ✓ std::string, const char*, string sabitleri görüntüleyebilir ✓ Sadece string OKUYAN fonksiyon parametreleri için mükemmel ✗ TEHLİKE: Görüntülenen string, string_view'dan uzun yaşamalı! Orijinal string yok edilirse → sarkan (dangling) görünüm!

CPPREF Depth: std::string SSO — Small String Optimization

Most standard library implementations use SSO (Small String Optimization): strings up to ~15-22 characters (implementation- dependent) are stored IN-PLACE within the std::string object itself — no heap allocation at all. Only when the string exceeds this threshold does a heap allocation occur. Typical layout: `sizeof(std::string)` is 32 bytes (GCC/libstdc++) with SSO buffer of 15 chars + null terminator. `std::string_view` (C++17) is a non-owning view: just a pointer + length. O(1) construction, never allocates, never copies. DANGER: the viewed data MUST outlive the string_view! Returning a string_view to a local std::string → dangling view → UB. This is one of the most common C++17 bugs. `std::string::reserve(n)` pre-allocates n bytes of capacity. `std::string::shrink_to_fit()` is a non-binding request to free excess capacity. C++20 adds `constexpr std::string` — compile-time string operations with transient allocation.

CPPREF Derinlik: std::string SSO — Küçük String Optimizasyonu

Çoğu standart kütüphane SSO (Small String Optimization) kullanır: ~15-22 karaktere kadar olan stringler std::string nesnesinin KENDİSİNDE saklanır — hiç heap tahsisi yapılmaz. Yalnızca bu eşik aşıldığında heap'ten bellek ayrılır. Tipik düzen: `sizeof(std::string)` GCC/libstdc++ ile 32 bayttır, 15 karakter + null sonlandırıcı SSO tamponuyla. `std::string_view` (C++17) sahipsiz bir görünümdür: sadece bir işaretçi + uzunluk. O(1) inşa, asla tahsis etmez, asla kopyalamaz. TEHLİKE: görüntülenen veri, string_view'dan UZUN YAŞAMALI! Yerel bir std::string'e string_view döndürmek → sarkan görünüm → tanımsız davranış. Bu, en yaygın C++17 hatalarından biridir. `reserve(n)` n bayt kapasite önceden ayırır. `shrink_to_fit()` fazla kapasiteyi serbest bırakma isteğidir (bağlayıcı değil). C++20 `constexpr std::string` ekler — derleme zamanı string işlemleri.

Kaynak Kod / Source Code

```cpp
#include <algorithm>    // EN: STL algorithms
#include <cctype>       // EN: std::toupper, std::tolower, std::isdigit
#include <iostream>     // EN: Standard I/O
#include <sstream>      // EN: std::istringstream, std::ostringstream
#include <string>       // EN: std::string
#include <string_view>  // EN: std::string_view (C++17)
#include <vector>       // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// [PRODUCTION CODE / ÜRETİM KODU]
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. String Splitting (Tokenization) ──────────────────────────────────

// EN: Split a CAN bus log line by delimiter. No regex needed — just std::string::find() and
// substr(). This is the most common string operation in embedded log parsing.
// TR: CAN bus log satırını ayırıcıya göre böl. Regex gerekmez — sadece std::string::find() ve
// substr(). Gömülü log ayrıştırmada en yaygın string işlemidir.
std::vector<std::string> split(const std::string& str, char delimiter) {
    std::vector<std::string> tokens;
    std::string::size_type start = 0;
    std::string::size_type end = str.find(delimiter);

    while (end != std::string::npos) {
        tokens.push_back(str.substr(start, end - start));
        start = end + 1;
        end = str.find(delimiter, start);
    }
    tokens.push_back(str.substr(start));
    return tokens;
}
```

```cpp

// ─── 2. String Replace All ──────────────────────────────────────────────────

// EN: Replace all occurrences of a substring. Useful for sanitizing diagnostic messages before
// logging.
// TR: Bir alt dizenin tüm tekrarlarını değiştir. Tanılama mesajlarını loglama öncesi temizlemek
// için kullanışlı.
std::string replaceAll(std::string str, const std::string& from,
                       const std::string& to) {
    if (from.empty()) return str;
    size_t pos = 0;
    while ((pos = str.find(from, pos)) != std::string::npos) {
        str.replace(pos, from.length(), to);
        pos += to.length();
    }
    return str;
}

// ─── 3. string_view Utility Functions ───────────────────────────────────────

// EN: Extract the DTC category from a code like "P0301". Uses string_view to avoid any
// allocation — just creates a view into the first character.
// TR: "P0301" gibi bir koddan DTC kategorisini çıkar. Herhangi bir tahsitten kaçınmak için
// string_view kullanır — sadece ilk karaktere bir görünüm oluşturur.
std::string_view getDTCCategory(std::string_view code) {
    // EN: P=Powertrain, B=Body, C=Chassis, U=Network
    // TR: P=Motor/Şanzıman, B=Gövde, C=Şasi, U=Ağ
    if (code.empty()) return "Unknown";
    switch (code[0]) {
        case 'P': return "Powertrain";
        case 'B': return "Body";
        case 'C': return "Chassis";
        case 'U': return "Network";
        default:  return "Unknown";
    }
}

// EN: Trim whitespace from both sides (using string_view — zero copy!)
// TR: Her iki taraftan boşlukları kes (string_view ile — sıfır kopya!)
std::string_view trim(std::string_view sv) {
    while (!sv.empty() && std::isspace(static_cast<unsigned char>(sv.front()))) {
        sv.remove_prefix(1);
    }
    while (!sv.empty() && std::isspace(static_cast<unsigned char>(sv.back()))) {
        sv.remove_suffix(1);
    }
    return sv;
}

// EN: Check if a VIN (Vehicle Identification Number) is valid format. Uses string_view as
// parameter — accepts string, const char*, literal.
// TR: VIN (Araç Kimlik Numarası) formatının geçerli olup olmadığını kontrol et. Parametre olarak
// string_view kullanır — string, const char*, sabit kabul eder.
bool isValidVIN(std::string_view vin) {
    if (vin.size() != 17) return false;
    // EN: VIN must be alphanumeric and not contain I, O, Q
    // TR: VIN alfanümerik olmalı ve I, O, Q içermemeli
    return std::all_of(vin.begin(), vin.end(), [](char c) {
        return std::isalnum(static_cast<unsigned char>(c)) &&
               c != 'I' && c != 'O' && c != 'Q';
    });
}

// ═══════════════════════════════════════════════════════════════════════════════
// [EXAMPLES / ÖRNEKLER]
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 4: STRING OPERATIONS & STRING_VIEW ===\n"
              << "=== MODÜL 4: STRİNG İŞLEMLERİ VE STRİNG_VIEW ===\n\n";
```

```cpp

    // ─── 1. Basic String Operations ──────────────────────────────────────────
    {
        std::cout << "--- 1. Basic String Operations ---\n";

        std::string vin = "1G1YY22G965";

        // EN: Append, concatenate
        // TR: Ekleme, birleştirme
        vin += "123456";
        std::cout << "VIN: " << vin << " (length=" << vin.size() << ")\n";

        // EN: substr — extract a portion
        // TR: substr — bir bölüm çıkar
        std::string wmi = vin.substr(0, 3);     // World Manufacturer ID
        std::string vds = vin.substr(3, 5);     // Vehicle Descriptor Section
        std::cout << "WMI: " << wmi << ", VDS: " << vds << "\n";

        // EN: compare
        // TR: karşılaştır
        std::string other = "1G1YY22G965123456";
        std::cout << "Equal? " << (vin == other ? "yes" : "no") << "\n";
        std::cout << "Compare: " << vin.compare(other) << "\n\n";
    }

    // ─── 2. Search Operations ────────────────────────────────────────────────
    {
        std::cout << "--- 2. String Search ---\n";

        std::string logLine = "[2026-03-06 14:22:05] ECM P0301: Cylinder 1 Misfire Detected";

        // EN: find — returns index of first occurrence (or npos)
        // TR: find — ilk bulunma indeksini döndürür (veya npos)
        auto dtcPos = logLine.find("P0");
        if (dtcPos != std::string::npos) {
            std::string dtcCode = logLine.substr(dtcPos, 5);
            std::cout << "Found DTC: " << dtcCode << " at position " << dtcPos << "\n";
        }

        // EN: rfind — search backwards (find LAST occurrence)
        // TR: rfind — geriye doğru ara (SON tekrarı bul)
        auto lastColon = logLine.rfind(':');
        std::cout << "Text after last colon: " << logLine.substr(lastColon + 2) << "\n";

        // EN: find_first_of — find any character from a set
        // TR: find_first_of — bir kümeden herhangi bir karakteri bul
        auto firstDigit = logLine.find_first_of("0123456789");
        std::cout << "First digit at index: " << firstDigit << "\n";

        // EN: find_first_not_of — find first char NOT in set
        // TR: find_first_not_of — kümede OLMAYAN ilk karakteri bul
        std::string padded = "   trimmed text   ";
        auto contentStart = padded.find_first_not_of(' ');
        auto contentEnd = padded.find_last_not_of(' ');
        std::cout << "Trimmed: \"" << padded.substr(contentStart, contentEnd - contentStart + 1)
                  << "\"\n\n";
    }

    // ─── 3. Modification Operations ──────────────────────────────────────────
    {
        std::cout << "--- 3. String Modification ---\n";

        std::string msg = "Engine temperature is CRITICAL at 120C";
```

```

        // EN: replace a range
        // TR: bir aralığı değiştir
        msg.replace(msg.find("CRITICAL"), 8, "WARNING");
        std::cout << "After replace: " << msg << "\n";

        // EN: erase characters
        // TR: karakterleri sil
        std::string serial = "SN-12345-AB";
        serial.erase(0, 3);  // Remove "SN-"
        std::cout << "Erased prefix: " << serial << "\n";

        // EN: insert at position
        // TR: pozisyona ekle
        serial.insert(0, "VH-");
        std::cout << "Inserted prefix: " << serial << "\n";

        // EN: Transform to uppercase using std::transform
        // TR: std::transform ile büyük harfe dönüştür
        std::string ecuName = "engine_control_module";
        std::transform(ecuName.begin(), ecuName.end(), ecuName.begin(),
                       [](unsigned char c) { return std::toupper(c); });
        std::cout << "Uppercase: " << ecuName << "\n\n";
    }

    // ─── 4. Split / Tokenize ──────────────────────────────────────────────────
    {
        std::cout << "--- 4. Split (tokenize) ---\n";

        // EN: Parse a CAN log line: "timestamp|ECU_ID|data_bytes"
        // TR: CAN log satırını ayrıştır: "zaman_damgası|ECU_ID|veri_baytları"
        std::string canLog = "14:22:05.123|0x7E0|03 41 0D 7A";

        auto parts = split(canLog, '|');
        std::cout << "Timestamp: " << parts[0] << "\n";
        std::cout << "ECU ID:    " << parts[1] << "\n";
        std::cout << "Data:      " << parts[2] << "\n";

        // EN: Alternative: split with std::istringstream
        // TR: Alternatif: std::istringstream ile böl
        std::string csv = "85,120,72,91,68";
        std::istringstream iss(csv);
        std::string token;
        std::cout << "CSV values: ";
        while (std::getline(iss, token, ',')) {
            std::cout << token << " ";
        }
        std::cout << "\n\n";
    }

    // ─── 5. Replace All ───────────────────────────────────────────────────────
    {
        std::cout << "--- 5. Replace All ---\n";

        std::string template_msg = "Vehicle {VIN} has {DTC_COUNT} active DTCs";
        std::string result = replaceAll(template_msg, "{VIN}", "1G1YY22G965123456");
        result = replaceAll(result, "{DTC_COUNT}", "3");
        std::cout << "Formatted: " << result << "\n\n";
    }

    // ─── 6. std::string_view — Zero-Copy Reading ──────────────────────────────
    {
        std::cout << "--- 6. std::string_view (zero-copy) ---\n";
```

```

        // EN: string_view doesn't allocate — just points to existing data
        // TR: string_view tahsis etmez — mevcut veriye işaret eder
        std::string fullLog = "[2026-03-06 14:22:05] P0301 Cylinder 1 Misfire";

        // EN: Create views into different parts WITHOUT copying
        // TR: Kopyalamadan farklı bölümlere görünümler oluştur
        std::string_view dateView(fullLog.data() + 1, 10);   // "2026-03-06"
        std::string_view timeView(fullLog.data() + 12, 8);   // "14:22:05"
        std::string_view codeView(fullLog.data() + 22, 5);   // "P0301"

        std::cout << "Date: " << dateView << "\n";
        std::cout << "Time: " << timeView << "\n";
        std::cout << "Code: " << codeView << "\n";
        std::cout << "Category: " << getDTCCategory(codeView) << "\n\n";

        // EN: string_view works with string literals too (no std::string needed!)
        // TR: string_view string sabitleri ile de çalışır (std::string gerekmez!)
        std::cout << "P0420 category: " << getDTCCategory("P0420") << "\n";
        std::cout << "B0100 category: " << getDTCCategory("B0100") << "\n";
        std::cout << "U0100 category: " << getDTCCategory("U0100") << "\n\n";
    }

    // ─── 7. string_view — trim and validate ──────────────────────────────────────
    {
        std::cout << "--- 7. string_view trim & validate ---\n";

        std::string_view padded = "   Hello World   ";
        std::cout << "Before trim: \"" << padded << "\"\n";
        std::cout << "After trim:  \"" << trim(padded) << "\"\n";

        // EN: VIN validation using string_view (no copies!)
        // TR: string_view ile VIN doğrulama (kopya yok!)
        std::cout << "VIN 'WBA3A5G59DNP12345' valid? "
                  << (isValidVIN("WBA3A5G59DNP12345") ? "yes" : "no") << "\n";
        std::cout << "VIN 'INVALID_SHORT' valid? "
                  << (isValidVIN("INVALID_SHORT") ? "yes" : "no") << "\n";
        std::cout << "VIN 'WBAOI5G59DNP12345' valid? "  // Contains 'O' and 'I'
                  << (isValidVIN("WBAOI5G59DNP12345") ? "yes" : "no") << "\n\n";
    }

    // ─── 8. std::ostringstream — String Building ─────────────────────────────────
    {
        std::cout << "--- 8. std::ostringstream (string builder) ---\n";

        // EN: Build a formatted diagnostic report without repeated concatenation
        // TR: Tekrarlanan birleştirme olmadan biçimlendirilmiş teşhis raporu oluştur
        std::ostringstream report;
        report << "=== DIAGNOSTIC REPORT ===\n";
        report << "VIN: 1G1YY22G965123456\n";
        report << "DTCs: P0301, P0420\n";
        report << "Battery: " << 12.6 << "V\n";
        report << "Oil Temp: " << 92 << "°C\n";

        std::string finalReport = report.str();
        std::cout << finalReport;
        std::cout << "Report size: " << finalReport.size() << " bytes\n\n";
    }

    // ─── 9. SSO (Small String Optimization) Proof ────────────────────────────────
    {
        std::cout << "--- 9. SSO (Small String Optimization) ---\n";
```

```cpp

        // EN: Likely SSO (no heap)
        // TR: Muhtemelen SSO (heap yok)
        std::string shortStr = "Hi";
        std::string longStr = "This is a much longer string that exceeds SSO threshold";

        std::cout << "Short string:\n";
        std::cout << "  size=" << shortStr.size()
                  << " capacity=" << shortStr.capacity()
                  << " sizeof=" << sizeof(shortStr) << "\n";

        std::cout << "Long string:\n";
        std::cout << "  size=" << longStr.size()
                  << " capacity=" << longStr.capacity()
                  << " sizeof=" << sizeof(longStr) << "\n";

        // EN: sizeof is always the SAME (typically 32 bytes on 64-bit). Short strings fit inside
        // those 32 bytes (SSO). Long strings allocate on the heap — capacity > sizeof.
        // TR: sizeof HER ZAMAN AYNI (genellikle 64-bit'te 32 byte). Kısa stringler bu 32 byte'a
        // sığar (SSO). Uzun stringler heap'te tahsis edilir — capacity > sizeof.
        std::cout << "\n";
    }

    // ─── 10. String Comparison and starts_with (C++20 backport) ──────────────────────────
    {
        std::cout << "--- 10. Prefix/Suffix Check ---\n";

        std::string dtc = "P0301";

        // EN: C++17 approach: compare substr
        // TR: C++17 yaklaşımı: substr karşılaştır
        bool isPowertrain = (dtc.compare(0, 1, "P") == 0);
        bool isMisfire = (dtc.compare(dtc.size() - 2, 2, "01") == 0);

        std::cout << dtc << " isPowertrain? " << (isPowertrain ? "yes" : "no") << "\n";
        std::cout << dtc << " isMisfire? " << (isMisfire ? "yes" : "no") << "\n";

        // EN: Using string_view for efficient prefix matching
        // TR: Verimli önek eşleştirme için string_view kullanma
        std::vector<std::string> allDTCs = {"P0301", "P0420", "B0100", "C0035", "U0100"};
        std::cout << "Powertrain codes: ";
        for (const auto& code : allDTCs) {
            std::string_view sv(code);
            if (!sv.empty() && sv[0] == 'P') {
                std::cout << code << " ";
            }
        }
        std::cout << "\n";
    }

    std::cout << "\n=== STRING OPERATIONS COMPLETE ===\n";
    return 0;
}
```

► 08. Ranges And Views

module_04_advanced/08_ranges_and_views.cpp

Konu: C++20 Ranges & Views — Educational C++17 Implementation / C++20 Aralıklar ve Görünümler

Teori / Theory

THEORY: C++20 Ranges & Views — The Pipeline Revolution

1. What Are Ranges?

A Range is any type that exposes begin() and end(). C++20 formalises this with concepts like std::ranges::range and std::ranges::random_access_range. Ranges replace the old iterator-pair idiom with a single object that *is* the sequence.

TEORİ : C++20 Aralıklar ve Görünümler — Boru Hattı Devrimi

Aralık Nedir?

Bir Aralık (Range), begin() ve end() sunan herhangi bir tiptir. C++20 bunu std::ranges::range ve std::ranges::random_access_range gibi kavramlarla (concepts) resmîleştirir. Aralıklar, eski yineleyici-çifti deyimini, dizinin *kendisi* olan tek bir nesneyle değiştirir.

2. Views — Lazy, Non-Owning Wrappers

A View is a lightweight, non-owning wrapper around a range. Views are cheap to copy and never own the underlying data. Key views: • filter(pred) — keep elements satisfying predicate • transform(fn) — apply function to each element • take(n) — first N elements • drop(n) — skip first N elements • reverse — reverse iteration order

Görünümler

Bir Görünüm (View), aralık etrafındaki hafif, sahiplik almayan bir sarmalayıcıdır. Kopyalaması ucuzdur ve alttaki veriye sahip olmaz: • filter(pred) — koşulu sağlayan elemanları tut • transform(fn) — her elemana fonksiyon uygula • take(n) — ilk N eleman • drop(n) — ilk N elemanı atla • reverse — ters sırada yinele

3. Pipe Syntax

The pipe operator `|` chains views left-to-right, reading like a processing pipeline on a factory floor: sensorData | views::filter(isValid) | views::transform(calibrate) Each stage lazily yields elements to the next — no intermediate containers are created.

Boru Sözdizimi

Boru operatörü `|`, görünümleri soldan sağa zincirler; fabrika bandındaki bir işleme hattı gibi okunur: sensorData | views::filter(isValid) | views::transform(calibrate) Her aşama tembel (lazy) olarak bir sonrakine eleman verir — ara konteyner oluşturulmaz.

4. Why Ranges Matter

• Eliminate temporary containers between algorithm stages • Improve readability — declarative, not imperative • Lazy evaluation — compute only what is consumed • Composable — small pieces snap together like LEGO bricks

Aralıklar Neden Önemli

• Algoritma aşamaları arasındaki geçici konteynerleri ortadan kaldırır • Okunabilirliği artırır — bildirimsel, zorunlu değil • Tembel değerlendirme — yalnızca tüketilen hesaplanır • Bileşenleştirilebilir — küçük parçalar LEGO gibi birleşir

5. C++17 Workaround Strategy

Under C++17 we lack <ranges>, so we emulate the pattern: 1. Template helper functions: filterAndTransform<Container, Pred, Fn> 2. Iterator-based LazyRange<T> that defers computation

3. Pipeline builder class that chains stages via std::function All code below compiles with

-std=c++17 -Wall -Wextra -Wpedantic.

C++17'de <ranges> olmadığından, aşağıdaki stratejiyle öykünüyoruz: 1. Şablon yardımcı fonksiyonlar: filterAndTransform<Container, Pred, Fn> 2. Hesaplamayı erteleyen yineleyici tabanlı LazyRange<T> 3. std::function ile aşamaları zincirleyen boru hattı oluşturucu sınıf Aşağıdaki tüm kod -std=c++17 -Wall -Wextra -Wpedantic ile derlenir.
Standart: C++17 (C++20 sections in comments only) Derleme : g++ -std=c++17 -Wall -Wextra -Wpedantic 08_ranges_and_views.cpp

CPPREF Depth: Lazy Evaluation and View Composition — Zero-Overhead Abstraction

C++20 views are lazy — they produce elements on demand with NO intermediate containers allocated. `views::filter | views::transform` creates a composed view object at COMPILE TIME; the pipeline is a single type encoding the entire chain of operations. Views satisfy O(1) construction and move — they store only the source range reference plus the predicate/transform objects. `views::iota(0)` is an INFINITE range — safe because laziness means elements are generated only when consumed (e.g., via `take(10)`). Borrowed range vs owning range: a borrowed range's iterators remain valid even after the range object is destroyed (e.g., `std::span`, `std::string_view`). Functions like `ranges::find` return `std::ranges::dangling` if given a non-borrowed temporary — compile- time dangling protection! In C++17 (pre-ranges), similar laziness can be achieved with: • Custom iterator pairs wrapping filter/transform logic • Pipeline builder classes chaining `std::function` stages • Coroutine-like generators (Boost.Coroutine2 or custom)

CPPREF Derinlik: Tembel Değerlendirme ve Görünüm Bileşimi — Sıfır Maliyetli Soyutlama

C++20 view'ları tembeldir — elemanları talep üzerine üretir, ara konteyner OLUŞTURMAZ. `views::filter | views::transform` DERLEME ZAMANINDA bileşik bir view nesnesi oluşturur; boru hattı tüm işlem zincirini kodlayan tek bir tiptir. View'lar O(1) inşa ve taşıma garantisi sunar — yalnızca kaynak aralık referansı ile yüklem/dönüşüm nesnelerini saklar. `views::iota(0)` SONSUZ bir aralıktır — tembellik sayesinde elemanlar yalnızca tüketildiğinde üretilir (ör. `take(10)` ile). Ödünç alınmış aralık (borrowed range) vs sahipli aralık: ödünç alınmış aralığın iteratörleri, aralık nesnesi yok edildikten sonra bile geçerli kalır (ör. `std::span`, `std::string_view`). `ranges::find` gibi fonksiyonlar, ödünç alınmamış geçici nesne verilirse `std::ranges::dangling` döner — derleme zamanında sarkan iteratör koruması! C++17'de (ranges öncesi) benzer tembellik şu yollarla sağlanabilir: • Filtre/dönüşüm mantığını saran özel iteratör çiftleri • `std::function` aşamalarını zincirleyen boru hattı sınıfları • Coroutine benzeri üreteçler (Boost.Coroutine2 veya özel)

Kaynak Kod / Source Code

```cpp
#include <algorithm>   // EN: STL algorithms (copy_if, transform)
#include <cstdint>     // EN: Fixed-width integers
#include <functional>  // EN: std::function for pipeline stages
#include <iomanip>     // EN: Output formatting
#include <iostream>    // EN: Console I/O
#include <iterator>    // EN: Iterator utilities
#include <numeric>     // EN: Accumulate, iota
#include <sstream>     // EN: String stream for lazy range
#include <string>      // EN: String operations
#include <vector>      // EN: Dynamic array container

// #include <ranges>  // C++20 — not used; all code below is C++17-compatible

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1 — filterAndTransform: C++17 Pipeline Helper
// ═══════════════════════════════════════════════════════════════════════════

// EN: Generic filter + transform pipeline — C++17 equivalent of:
// TR: Genel filtre + dönüştürme boru hattı — C++17 karşılığı: C++20: container |
// views::filter(pred) | views::transform(fn)
template <typename Container, typename Predicate, typename Transform>
auto filterAndTransform(const Container& input, Predicate pred, Transform fn) {
    using OutputType = decltype(fn(*input.begin()));
    std::vector<OutputType> result;
    result.reserve(input.size());
    for (const auto& elem : input) {
        if (pred(elem)) {
            result.push_back(fn(elem));
        }
    }
    return result;
}

// EN: Take the first N elements from a container
// TR: Bir konteynerden ilk N elemanı al
template <typename Container>
auto takeN(const Container& input, std::size_t n) {
    using ValueType = typename Container::value_type;
    std::vector<ValueType> result;
    std::size_t count = 0;
    for (const auto& elem : input) {
        if (count >= n) break;
        result.push_back(elem);
        ++count;
    }
    return result;
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2 — LazyRange: Deferred Computation (C++17 Iterator Approach)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: LazyRange defers filter + transform until iteration — mimics view laziness
// TR: LazyRange, filtre + dönüşümü yinelemeye kadar erteler — görünüm tembelliğini taklit eder
template <typename T>
class LazyRange {
public:
    using FilterFn    = std::function<bool(const T&)>;
    using TransformFn = std::function<T(const T&)>;

    LazyRange(const std::vector<T>& source, FilterFn filter, TransformFn transform)
        : source_(source), filter_(std::move(filter)), transform_(std::move(transform)) {}

    // ── Inner Iterator ──────────────────────────────────────────────────────
    class Iterator {
    public:
        using iterator_category = std::input_iterator_tag;
        using value_type        = T;
        using difference_type   = std::ptrdiff_t;
        using pointer           = const T*;
        using reference         = const T&;

        Iterator(const LazyRange* owner, std::size_t pos)
            : owner_(owner), pos_(pos) { advance(); }

        T operator*() const { return owner_->transform_(owner_->source_[pos_]); }

        Iterator& operator++() { ++pos_; advance(); return *this; }
        Iterator operator++(int) { auto tmp = *this; ++(*this); return tmp; }

        bool operator==(const Iterator& o) const { return pos_ == o.pos_; }
        bool operator!=(const Iterator& o) const { return !(*this == o); }

    private:
        void advance() {
            while (pos_ < owner_->source_.size() &&
                   !owner_->filter_(owner_->source_[pos_])) {
                ++pos_;
            }
        }
        const LazyRange* owner_;
        std::size_t pos_;
    };

    Iterator begin() const { return Iterator(this, 0); }
    Iterator end()   const { return Iterator(this, source_.size()); }

    // EN: Materialise lazily-computed results into a vector
    // TR: Tembel hesaplanmış sonuçları bir vektöre topla
    std::vector<T> collect() const {
        return std::vector<T>(begin(), end());
    }

private:
    const std::vector<T>& source_;
    FilterFn              filter_;
    TransformFn           transform_;
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3 — SensorPipeline: Multi-Stage Processing Builder
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Represents a raw automotive sensor reading (e.g., throttle position)
// TR: Ham bir otomotiv sensör okumasını temsil eder (ör. gaz kelebeği konumu)
struct SensorReading {
    int      sensorId;    // EN: Unique sensor identifier / TR: Benzersiz sensör tanımlayıcı
    double   rawValue;    // EN: Raw ADC value / TR: Ham ADC değeri
    bool     valid;       // EN: Validity flag / TR: Geçerlilik bayrağı
};
```

```cpp

// EN: SensorPipeline chains: filter invalid → scale → clamp → collect
// TR: SensorPipeline zinciri: geçersiz filtrele → ölçekle → sınırla → topla
class SensorPipeline {
public:
    using Stage = std::function<std::vector<double>(const std::vector<double>&)>;

    // EN: Add a processing stage to the pipeline
    // TR: Boru hattına bir işleme aşaması ekle
    SensorPipeline& addStage(Stage stage) {
        stages_.push_back(std::move(stage));
        return *this;
    }

    // EN: Execute all stages sequentially on the input
    // TR: Tüm aşamaları girdi üzerinde sıralı çalıştır
    [[nodiscard]] std::vector<double> execute(const std::vector<double>& input) const {
        auto data = input;
        for (const auto& stage : stages_) {
            data = stage(data);
        }
        return data;
    }

private:
    std::vector<Stage> stages_;
};

// ─── Pipeline Stage Factories ──────────────────────────────────────────────────

// EN: Create a filter stage that keeps elements satisfying pred
// TR: Koşulu sağlayan elemanları tutan bir filtre aşaması oluştur
inline auto makeFilterStage(std::function<bool(double)> pred) {
    return [pred = std::move(pred)](const std::vector<double>& in) {
        std::vector<double> out;
        std::copy_if(in.begin(), in.end(), std::back_inserter(out), pred);
        return out;
    };
}

// EN: Create a transform stage that applies fn to each element
// TR: Her elemana fn uygulayan bir dönüşüm aşaması oluştur
inline auto makeTransformStage(std::function<double(double)> fn) {
    return [fn = std::move(fn)](const std::vector<double>& in) {
        std::vector<double> out(in.size());
        std::transform(in.begin(), in.end(), out.begin(), fn);
        return out;
    };
}

// EN: Create a take-N stage that keeps only the first N elements
// TR: Yalnızca ilk N elemanı tutan bir alma aşaması oluştur
inline auto makeTakeStage(std::size_t n) {
    return [n](const std::vector<double>& in) {
        auto count = std::min(n, in.size());
        return std::vector<double>(in.begin(), in.begin() + static_cast<std::ptrdiff_t>(count));
    };
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4 — Classic STL vs Range-Style Comparison
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Classic STL approach — requires intermediate containers
// TR: Klasik STL yaklaşımı — ara konteynerler gerektirir
inline std::vector<double> classicApproach(const std::vector<SensorReading>& readings) {
```

```
    // EN: Step 1 — filter valid readings
    // TR: Adım 1 — geçerli okumaları filtrele
    std::vector<SensorReading> validReadings;
    std::copy_if(readings.begin(), readings.end(),
                 std::back_inserter(validReadings),
                 [](const SensorReading& r) { return r.valid; });

    // EN: Step 2 — extract and calibrate raw values
    // TR: Adım 2 — ham değerleri çıkar ve kalibre et
    std::vector<double> calibrated(validReadings.size());
    std::transform(validReadings.begin(), validReadings.end(),
                   calibrated.begin(),
                   [](const SensorReading& r) { return r.rawValue * 0.01; });
    return calibrated;
}

// EN: Range-style approach in C++17 — single pass with filterAndTransform
// TR: C++17'de aralık tarzı yaklaşım — filterAndTransform ile tek geçiş
inline std::vector<double> rangeStyleApproach(const std::vector<SensorReading>& readings) {
    /*  C++20 equivalent / C++20 karşılığı:
     *  auto result = readings
     *      | std::views::filter([](const SensorReading& r) { return r.valid; })
     *      | std::views::transform([](const SensorReading& r) { return r.rawValue * 0.01; });
     */
    return filterAndTransform(
        readings,
        [](const SensorReading& r) { return r.valid; },
        [](const SensorReading& r) { return r.rawValue * 0.01; }
    );
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 5 — Utility Helpers
// ═══════════════════════════════════════════════════════════════════════════

// EN: Print a vector with a label
// TR: Bir vektörü etiketle yazdır
template <typename T>
void printVec(const std::string& label, const std::vector<T>& v) {
    std::cout << label << ": [";
    for (std::size_t i = 0; i < v.size(); ++i) {
        if (i > 0) std::cout << ", ";
        std::cout << std::fixed << std::setprecision(2) << v[i];
    }
    std::cout << "]\n";
}

// ═══════════════════════════════════════════════════════════════════════════
// main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " C++20 Ranges & Views — C++17 Educational Demos\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ── Demo 1: Filter + Transform Pipeline (C++17 Working) ──────────────
    // EN: Filter valid sensor readings, then calibrate
    // TR: Geçerli sensör okumalarını filtrele, ardından kalibre et
    {
        std::cout << "── Demo 1: Filter + Transform Pipeline ──\n";

        std::vector<SensorReading> readings = {
            {1, 512.0, true},  {2, -10.0, false}, {3, 1023.0, true},
            {4, 0.0, true},    {5, 750.0, false},  {6, 200.0, true}
        };

        auto calibrated = filterAndTransform(
            readings,
            [](const SensorReading& r) { return r.valid; },
            [](const SensorReading& r) { return r.rawValue * 0.01; }
        );

        /*  C++20 equivalent / C++20 karşılığı:
         *  auto calibrated = readings
         *      | std::views::filter([](auto& r){ return r.valid; })
```

```
         *      | std::views::transform([](auto& r){ return r.rawValue * 0.01; });
         */

        printVec("  Calibrated values", calibrated);
        std::cout << '\n';
    }

    // ─── Demo 2: C++20 Equivalent (Comment Only) ──────────────────────────────────────
    // EN: The following C++20 code is shown for reference only
    // TR: Aşağıdaki C++20 kodu yalnızca referans amaçlıdır
    {
        std::cout << "── Demo 2: C++20 Equivalent (in comments) ──\n";
        std::cout << "  // #include <ranges>\n";
        std::cout << "  // auto result = sensorData\n";
        std::cout << "  //     | std::views::filter(isValid)\n";
        std::cout << "  //     | std::views::transform(calibrate)\n";
        std::cout << "  //     | std::views::take(5);\n";
        std::cout << "  (See source comments for full C++20 syntax)\n\n";

        /*  Full C++20 pipeline / Tam C++20 boru hattı:
         *
         *  #include <ranges>
         *  namespace views = std::views;
         *
         *  auto isValid   = [](const SensorReading& r) { return r.valid; };
         *  auto calibrate = [](const SensorReading& r) { return r.rawValue * 0.01; };
         *
         *  std::vector<SensorReading> sensorData = { ... };
         *
         *  auto result = sensorData
         *      | views::filter(isValid)
         *      | views::transform(calibrate)
         *      | views::take(5);
         *
         *  for (double v : result) std::cout << v << '\n';
         */
    }

    // ─── Demo 3: Multi-Stage Pipeline — filter → transform → take N ──────────────────
    // EN: Build a pipeline: discard negatives → multiply by 2 → take first 3
    // TR: Boru hattı kur: negatifleri at → 2 ile çarp → ilk 3'ü al
    {
        std::cout << "── Demo 3: Multi-Stage Pipeline ──\n";

        std::vector<double> rawSignals = {
            -5.0, 12.0, 3.5, -1.0, 8.8, 22.1, 6.0, -0.5, 15.3
        };

        /*  C++20 equivalent / C++20 karşılığı:
         *  auto result = rawSignals
         *      | views::filter([](double v){ return v >= 0.0; })
         *      | views::transform([](double v){ return v * 2.0; })
         *      | views::take(3);
         */

        SensorPipeline pipeline;
        pipeline
            .addStage(makeFilterStage([](double v) { return v >= 0.0; }))
            .addStage(makeTransformStage([](double v) { return v * 2.0; }))
            .addStage(makeTakeStage(3));

        auto result = pipeline.execute(rawSignals);
        printVec("  Pipeline output (first 3 non-negative × 2)", result);
        std::cout << '\n';
    }
```

```cpp

    // ─── Demo 4: Lazy Range vs Eager Evaluation ──────────────────────────────────────
    // EN: Compare lazy (deferred) vs eager (immediate) computation
    // TR: Tembel (ertelenmiş) ile istekli (anlık) hesaplamayı karşılaştır
    {
        std::cout << "── Demo 4: Lazy vs Eager Evaluation ──\n";

        std::vector<double> temperatures = {
            -40.0, 25.5, 105.3, 72.0, -10.0, 88.9, 150.0, 36.6
        };

        // EN: Predicate — keep temperatures in valid ECU operating range [-30, 125]
        // TR: Koşul — geçerli ECU çalışma aralığındaki sıcaklıkları tut [-30, 125]
        auto inRange = [](const double& t) { return t >= -30.0 && t <= 125.0; };

        // EN: Transform — Celsius to Fahrenheit
        // TR: Dönüşüm — Celsius'tan Fahrenheit'a
        auto toFahrenheit = [](const double& t) { return t * 9.0 / 5.0 + 32.0; };

        // --- Eager approach (C++17 filterAndTransform) ---
        // EN: Computes all results immediately into a new vector
        // TR: Tüm sonuçları hemen yeni bir vektöre hesaplar
        auto eagerResult = filterAndTransform(temperatures, inRange, toFahrenheit);
        printVec("  Eager (all at once)", eagerResult);

        // --- Lazy approach (LazyRange) ---
        // EN: Nothing is computed until we iterate or collect
        // TR: Yineleyinceye veya toplayana kadar hiçbir şey hesaplanmaz
        LazyRange<double> lazyView(temperatures, inRange, toFahrenheit);

        std::cout << "  Lazy  (first 3)  : [";
        int count = 0;
        for (auto it = lazyView.begin(); it != lazyView.end() && count < 3; ++it, ++count) {
            if (count > 0) std::cout << ", ";
            std::cout << std::fixed << std::setprecision(2) << *it;
        }
        std::cout << "]\n";

        auto lazyAll = lazyView.collect();
        printVec("  Lazy  (collected)", lazyAll);
        std::cout << '\n';
    }

    // ─── Demo 5: Custom Pipeline Builder Pattern ─────────────────────────────────────
    // EN: Demonstrates the builder pattern for composing processing stages
    // TR: İşleme aşamalarını birleştirmek için inşaatçı kalıbını gösterir
    {
        std::cout << "── Demo 5: Custom Pipeline Builder ──\n";

        // EN: Simulated throttle position sensor data (0–4095 ADC range)
        // TR: Simüle edilmiş gaz kelebeği konum sensörü verisi (0–4095 ADC aralığı)
        std::vector<double> throttleADC = {
            150.0, 4200.0, 2048.0, -50.0, 3500.0, 0.0, 1500.0, 4096.0
        };

        /*  C++20 equivalent / C++20 karşılığı:
         *  auto processed = throttleADC
         *      | views::filter([](double v){ return v >= 0.0 && v <= 4095.0; })
         *      | views::transform([](double v){ return (v / 4095.0) * 100.0; })
         *      | views::transform([](double v){ return std::clamp(v, 0.0, 100.0); });
         */

        SensorPipeline throttlePipeline;
        throttlePipeline
            // EN: Stage 1 — filter out-of-range ADC values
            // TR: Aşama 1 — aralık dışı ADC değerlerini filtrele
            .addStage(makeFilterStage([](double v) {
                return v >= 0.0 && v <= 4095.0;
            }))
```

```
            // EN: Stage 2 — normalize to percentage (0–100%)
            // TR: Aşama 2 — yüzdeye normalleştir (0–100%)
            .addStage(makeTransformStage([](double v) {
                return (v / 4095.0) * 100.0;
            }))
            // EN: Stage 3 — clamp to safe throttle range
            // TR: Aşama 3 — güvenli gaz kelebeği aralığına sınırla
            .addStage(makeTransformStage([](double v) {
                return std::max(0.0, std::min(100.0, v));
            }));

        auto throttlePercent = throttlePipeline.execute(throttleADC);
        printVec("  Throttle %", throttlePercent);

        // EN: Compare classic vs range-style on SensorReading data
        // TR: SensorReading verisi üzerinde klasik ile aralık tarzını karşılaştır
        std::vector<SensorReading> mixedReadings = {
            {10, 800.0, true}, {11, -5.0, false}, {12, 400.0, true},
            {13, 950.0, true}, {14, 0.0, false},  {15, 620.0, true}
        };

        auto classicResult = classicApproach(mixedReadings);
        auto rangeResult   = rangeStyleApproach(mixedReadings);

        printVec("  Classic STL result", classicResult);
        printVec("  Range-style result", rangeResult);

        // EN: Verify both approaches produce identical output
        // TR: Her iki yaklaşımın aynı çıktıyı ürettiğini doğrula
        bool match = (classicResult == rangeResult);
        std::cout << "  Results match: " << (match ? "YES" : "NO") << "\n\n";
    }

    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " All demos completed successfully.\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

Modül 5: Certification Prep — Sertifikasyon Hazırlığı

Sertifikasyon Hazırlığı

6 kaynak dosya · Standart: C++17

C++17

► 01. Undefined Behavior

module_05_certification/01_undefined_behavior.cpp

Konu: CPA/CPP Prep: Undefined Behavior & Tricky Questions / Sertifikasyon Hazırlığı: Tanımsız

Teori / Theory

THEORY: Why do C++ Institute and CppQuiz ask such hard questions?

In C++, unlike managed languages like Java or C#, the compiler assumes YOU know exactly what you are doing. If you access memory out of bounds or use a pointer after it's deleted, C++ DOES NOT stop you. It results in "Undefined Behavior" (UB). Certification exams test your ability to spot these hidden traps.

TEORİ: Sertifika Tuzakları

C++, Java veya C# gibi dillerin aksine kontrolü tamamen size bırakır. Derleyici sizin NE YAPTIĞINIZI KESİN OLARAK BİLDİĞİNİZİ varsayar. Dizi boyutunu aşarsanız veya silinmiş bir belleği okumaya çalışırsanız, C++ sizi durdurmaz (hata vermez). Buna "Undefined Behavior" (Tanımsız Davranış - UB) denir. CPA/CPP ve CppQuiz sınavları tam olarak bu gizli tuzakları görme yeteneğinizi test eder.

THEORY: Examples of Undefined Behavior

1. Dangling Pointers: Pointing to an address that has already been freed. 2. Out of Bounds: Reading the 10th element of a 5-element array. 3. Uninitialized Variables: Reading a local variable containing garbage memory. 4. Signed Integer Overflow: Adding 1 to the absolute MAX value of an `int`.

TEORİ: Tanımsız Davranış Örnekleri

1. Dangling (Sarkan) Pointers: Silinmiş bir adresi işaret eden pointer (Use after free).

Değişken (Uninitialized): İçine değer atanmamış çöp belleği okumak. 4. İşaretli Taşma (Signed Overflow): Maksimum sınırındaki int değerine 1 eklemek (C++'ta UB'dir).

CPPREF Depth: Unsequenced Modification HACK!

2. Sınır Aşımı (Out of Bounds): 5 elemanlı dizinin 10. elemanını okumak. 3. İlklenmemiş

CppReference explicitly forbids modifying a variable and reading it twice in the same sequence point (e.g., `cout << i++ << ++i;`). The C++ standard DOES NOT GUARANTEE which one executes first. Depending on the compiler (GCC vs Clang), it might output completely different numbers, or crash!

CPPREF Derinlik: Sırasız Değişiklik (Sırasız Değişim)!

CppReference, bir değişkeni aynı satırda (sekans noktasında) hem okuyup hem de değiştirmeyi (Örn: `cout << i++ << ++i;`) kesinlikle YASAKLAR (UB'dir). C++ standardı derleyicilere sağdan mı yoksa soldan mı hesaplayacaklarını söylemez! Mac'te farklı, Windows'ta farklı sonuç çıkar. Oyun state'leriniz darmadağın olur!

Kaynak Kod / Source Code

```cpp
#include <iostream>

void testDanglingPointer() {
  std::cout << "\n--- Danger Zone: Dangling Pointer (Sarkan İşaretçi) ---" << std::endl;
  int *ptr = new int(100);
  std::cout << "Original Value (Orijinal Değer): " << *ptr << std::endl;

  // EN: Memory is freed.
  // TR: Belleği iade ettik.
  delete ptr;

  // DANGER (TEHLİKE)!
  // EN: 'ptr' still points to that address, but it belongs to the OS now.
  // TR: 'ptr' hala o adresi gösteriyor ama adres artık bize ait değil.
  //
  // *ptr = 500; // UNDEFINED BEHAVIOR! (Do not uncomment! / Yorumdan çıkarmayın!)

  // Correct Way (Doğru Yönem):
  // EN: Safer!
  // TR: Artık hiçbir yeri göstermiyor. Güvenli!
  ptr = nullptr;
  std::cout << "Pointer safely nullified. (İşaretçi sıfırlandı.)" << std::endl;
}

void testUninitializedVariable() {
  std::cout << "\n--- Danger Zone: Uninitialized Variable (İlklenmemiş Değişken) ---" << std::endl;
  // EN: Memory allocated, but holds generic GARBAGE.
  // TR: Sadece hafızada yer ayrıldı, içi çöp.
  int mysteriousNumber;

  // std::cout << "Value: " << mysteriousNumber << std::endl; // UNDEFINED BEHAVIOR: Just reading
  // it is UB!
  // EN: Now initialized.
  // TR: Ancak değer atandıktan sonra okunabilir.
  mysteriousNumber = 42;
  std::cout << "Initialized safely (Güvenle ilk değer atandı): " << mysteriousNumber << std::endl;
}

int main() {
  std::cout << "=== MODULE 5: CERTIFICATION & CPPQUIZ TRAPS ===\n" << std::endl;
  std::cout << "EN: Exams ask you to READ code and spot hidden DANGER ZONES, "
               "not write them!"
            << std::endl;
  std::cout << "TR: Sertifika sınavları kodu YAZMANIZI değil, OKUMANIZI ve "
               "gizli tehlikeleri bulmanızı ister!\n"
            << std::endl;

  testDanglingPointer();
  testUninitializedVariable();

  // 3. Trap: Object Slicing (Nesne Dilimlenmesi)
  class Base {
  public:
    virtual void print() { std::cout << "Base"; }
  };
```

```
56    class Derived : public Base {
57    public:
58      void print() override { std::cout << "Derived"; }
59    };
60
61    Derived myDerived;
62
63    // EN: ERROR! Specific traits were sliced off! (Object Slicing)
64    // TR: HATA! Özel olan kısımlar "dilimlenip" kesilip atıldı. (Nesne Dilimlenmesi)
65    Base slicedBase = myDerived;
66
67    std::cout << "\nObject Slicing Test: ";
68    // EN: Prints "Base"! Polymorphism FAILED.
69    // TR: "Base" yazar! Polimorfizm çalışmaz!
70    slicedBase.print();
71
72    std::cout << " (Polymorphism FAILED due to Value Copy. Reference was needed!)" << std::endl;
73
74    return 0;
75  }
```

► 02. Object Slicing Virtual

module_05_certification/02_object_slicing_virtual.cpp

Konu: CPA/CPP Prep: Object Slicing and Virtual Tables / Nesne Dilimlenmesi ve Sanal Tablolar

Teori / Theory

THEORY: What is Object Slicing?

Polymorphism (Dynamic Binding) ONLY works over Pointers (*) or References (&). If you directly assign a Derived object to a Base object (Pass by Value), the compiler literally "slices off" all the new variables and overridden functions that belong to the Derived class. The object becomes a pure Base object!

TEORİ: Nesne Dilimlenmesi Nedir?

Çok biçimlilik (Polimorfizm) SADECE İşaretçiler (*) veya Referanslar (&) üzerinden çalışır. Eğer türetilmiş (Derived) bir nesneyi doğrudan temel (Base) bir nesneye ("Değer ile", yani Pass by Value) atarsanız, derleyici Alt sınıfa ait olan TÜM yeni değişkenleri ve ezilmiş (overridden) fonksiyonları kelimenin tam anlamıyla "bıçakla kesip / dilimleyip" çöpe atar. O nesne artık saf bir Base nesnesine dönüşür!

THEORY: Why does it slice? (Memory Architecture)

In C++, objects are contiguous blocks of memory. - Base Class: 4 Bytes - Derived Class: 8 Bytes If you say `Base b = Derived();`, the compiler cannot fit 8 Bytes into a 4 Byte box. It HAS to SLICE the overflowing parts off.

TEORİ: Neden Dilimleniyor?

C++'ta nesneler RAM'de yan yana duran kutucuklardır. - Base Sınıf: 4 Bayt (sadece yaş) - Derived Sınıf: 8 Bayt (hem yaş hem maaş) Eğer `Base b = Derived();` derseniz, derleyici 4 baytlık küçük kutunun içine, 8 baytlık dev nesneyi sığdıramaz. Mecburen taflan kısmı kesip atar (Dilimler)!

CPPREF Depth: Slicing Destroys RTTI

Slicing doesn't just cut off memory; it permanently DESTROYS the object's dynamic type identity (typeid / RTTI pointer). The sliced object's hidden `vptr` is forcibly rewritten to point to the Base class's `vtable`. It is completely unrecoverable!

CPPREF Derinlik: Dilimleme RTTI'yı Yok Eder (Yazılımsal Kimlik Katliamı)

Dilimleme sadece fazla byte'ları kesmez; nesnenin dinamik RTTI kimliğini (typeid) SONSUZA DEK YOK EDER. Gizli Sanal İşaretçi (`vptr`), zorla Base sınıfın `vtable`'ına (sanal tablosuna) bağlanacak şekilde değiştirilir. Orijinal nesneyi bir daha asla geri getiremezsiniz!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>

class Document {
public:
  std::string name;
  Document(std::string docName) : name(docName) {}

  // EN: Virtual function is the key to polymorphism.
  // TR: Sanal (Virtual) fonksiyon, polimorfizmin anahtarıdır.
  virtual void print() const {
    std::cout << "[Base] Printing generic Document: " << name << std::endl;
  }
};

class PDFDocument : public Document {
public:
  // EN: Specific to subclass.
  // TR: Alt sınıfa Özel bir değişken.
  int securityProtocol;

  PDFDocument(std::string docName, int proto)
      : Document(docName), securityProtocol(proto) {}

  // EN: Overriding the base function.
  // TR: Fonksiyonu ezmek (Override).
  void print() const override {
    std::cout << "[Derived] Printing PDF: " << name << " | Security Protocol (Güvenlik Protokolü): "
        << securityProtocol << std::endl;
  }
};

// EN: 1. UNSAFE: Pass by Value -> SLICED!
// TR: 1. GÜVENSİZ: Değer ile Geçiş -> DİLİMLENİR!
void sloppyPrint(Document doc) {
  // EN: This object is no longer a PDF. It is just a regular Document.
  // TR: Bu nesne artık bir PDF değil. Sadece sıradan bir Document (Dilimlenmiş).
  doc.print();
}

// EN: 2. SAFE: Pass by Reference -> NO SLICING!
// TR: 2. GÜVENLİ: Referans ile Geçiş -> Polimorfizm!
void securePrint(const Document &doc) {
  // EN: The V-Table is consulted. The true owner (PDF) method is called.
  // TR: V-Table'a (Sanal Tabloya) bakılarak asıl sahibin (PDF) metodu çağırılır.
  doc.print();
}

int main() {
  std::cout << "=== MODULE 5: OBJECT SLICING (NESNE DİLİMLENMESİ) ===\n" << std::endl;

  PDFDocument topSecretFile("NuclearCodes.pdf", 256);
```

```

    // --- TEST 1: THE SLICING DISASTER / DİLİMLENME FELAKETİ ---
    std::cout << "1. Pass by Value (Sliced Object / Dilimlenen Nesne):" << std::endl;
    std::cout << "-> INTERVIEW TRAP: Compiler gives NO ERROR, but calls the "
                 "wrong function!"
              << std::endl;
    std::cout << "-> MÜLAKAT TUZAĞI: Derleyici hata (Error) VERMEZ ancak yanlış "
                 "fonksiyonu çağırır!"
              << std::endl;
    sloppyPrint(topSecretFile);

    std::cout << "\n-------------------------------------------------\n" << std::endl;

    // --- TEST 2: CORRECT & SAFE POLYMORPHISM / DOĞRU POLİMORFİZM ---
    std::cout << "2. Pass by Reference (Real Polymorphism / Gerçek Polimorfizm):" << std::endl;
    std::cout << "-> INTACT OBJECT: The V-Table recognized the true owner!" << std::endl;
    std::cout << "-> NESNE BÜTÜN: Sanal Tablo (V-Table) asıl sahibini tanıdı!" << std::endl;
    securePrint(topSecretFile);

    std::cout << "\n-------------------------------------------------\n" << std::endl;

    // EN: A classic CppQuiz Question:
    // TR: Klasik CppQuiz Sorusu:
    std::cout << "3. Direct Assignment (Doğrudan Atama / Slicing):" << std::endl;
    Document standardDoc = topSecretFile;
    standardDoc.print();
    // std::cout << standardDoc.securityProtocol; // ERROR! Won't compile because it was sliced
    // off! (Erişilemez)

    return 0;
}
```

► 03. Type Casting

module_05_certification/03_type_casting.cpp

Konu: CPA/CPP Prep: Type Casting / Sertifikasyon: C++ Tip Dönüşüm Operatörleri

Teori / Theory

THEORY: Why are C-Style Casts (int) banned in Modern C++?

In C, you could force any type into any other type using `(int) x` or `(char*) ptr`. This tells the compiler: "Shut up, I know what I'm doing." But it's violent. It breaks constness, converts pointers blindly, and causes severe run-time crashes. C++ introduced 4 safe, intentional, and searchable cast operators.

TEORİ: Eski Cast Neden Yasak?

C dilinde, `(int)x` diyerek her türlü veriyi (örneğin bir kedi nesnesini integer'a) zorla dönüştürebilirsiniz. Derleyiciye "Sus, ne yaptığımı biliyorum" demektir. Ama bu bir felakettir; nesne dilimlenmelerine, `const` zırhının kırılmasına yol açar. C++, derleyicinin "niyetinizi anlaması" ve sizi koruması için 4 ayrı dönüştürücü çıkardı. C-Stili dönüşüm kullananlar mülakatlarda genellikle anında eksi puan alır!

THEORY: The 4 Modern C++ Casts

C++ replaced the dangerous C-style `(int)x` cast with 4 explicit, searchable, and intentional cast operators. Each has a specific purpose:

TEORİ: 4 Modern C++ Dönüştürücü

C++, tehlikeli C-stili `(int)x` dönüşümünü 4 açık, aranabilir ve niyeti belirten dönüştürücü operatörle değiştirdi. Her birinin amacı farklıdır:

---

1. static_cast<Type>(val)

The "logical/ideal" cast. Used for well-defined conversions like float → int, int → enum, base* → derived* (when you know the type). Checked at COMPILE-TIME. Zero runtime cost. This is the cast you use 95% of the time.

"Mantıklı/ideal" dönüşüm. float → int, int → enum, base* → derived* gibi iyi tanımlanmış dönüşümler için kullanılır (tipi bildiğinizde). DERLEME ZAMANINDA kontrol edilir. Çalışma zamanı maliyeti sıfırdır. Zamanın %95'inde bu cast'i kullanırsınız.

---

2. dynamic_cast<Type>(val)

The "safe polymorphism" cast. Used to safely downcast a base class pointer/reference to a derived class. Checks the V-Table at RUN-TIME (RTTI). If the cast is invalid, returns nullptr (pointer) or throws std::bad_cast (reference). Slower than static_cast but SAFE.

"Güvenli polimorfizm" dönüşümü. Üst sınıf pointer/referansını alt sınıfa güvenle dönüştürmek için kullanılır. ÇALIŞMA ZAMANINDA V-Table üzerinden kontrol eder (RTTI). Geçersiz dönüşümde nullptr (pointer) veya std::bad_cast (referans) döner. static_cast'ten yavaş ama GÜVENLİDİR.

---

3. const_cast<Type>(val)

Strips away `const` or `volatile` qualifiers. Used ONLY when interfacing with old C APIs that forgot to mark parameters as `const`. Modifying a truly const object through const_cast is UNDEFINED BEHAVIOR. Use with extreme caution!

`const` veya `volatile` niteleyicilerini kaldırır. SADECE `const` parametreyi unutmuş eski C API'leriyle çalışırken kullanılır. Gerçekten const olan bir nesneyi const_cast ile değiştirmek TANIMSIZ DAVRANISTIR (UB). Son derece dikkatli kullanın!

---

4. reinterpret_cast<Type>(val)

The "brute force" cast. Reinterprets the raw bit pattern of one type as another. No checks, no safety net. Used almost exclusively in low-level hardware/driver programming and memory-mapped I/O. Casting unrelated pointer types and dereferencing them triggers Strict Aliasing UB. Only safe targets: `char*`, `unsigned char*`, `std::byte*`. FORBIDDEN in application-level code!

"Kaba kuvvet" dönüşümü. Bir tipin ham bit desenini başka bir tip olarak yorumlar. Kontrol yok, güvenlik ağı yok. Neredeyse yalnızca düşük seviye donanım/sürücü programlama ve bellek-eşlemeli I/O'da kullanılır. İlişkisiz pointer tiplerini dönüştürüp okumak Strict Aliasing UB tetikler. Güvenli hedefler: `char*`, `unsigned char*`, `std::byte*`. Uygulama seviyesi kodda YASAKTIR!

CPPREF Depth: `dynamic_cast` Cost & `reinterpret_cast` Strict Aliasing UB

`dynamic_cast` performs a string-search on the Run-Time Type Information (RTTI) tree, making it EXTREMELY SLOW compared to `static_cast`. Use it sparingly! Also, `reinterpret_cast` frequently triggers "Strict Aliasing" Undefined Behavior if you cast two unrelated pointers and dereference them. Only cast to `char*` or `std::byte*`.

CPPREF Derinlik: `dynamic_cast` Maliyeti ve `reinterpret_cast` Sıkı Örtüşme UB

`dynamic_cast`, asıl kimliği bulmak için RTTI ağacında text araması yapar. Bu yüzden `static_cast`e kıyasla DEHŞET YAVAŞTIR (Oyun döngüsünde anlık FPS düşüşü yaratır). Ek olarak, `reinterpret_cast` ile dönüştürülmüş uyumsuz tipleri `*ptr` okumak Strict Aliasing kuralını kırıp UB (Tanımsız Davranış) çökmelerine sebep olur. Sadece `char*` veya `std::byte*` ile güvenle donanım okuyabilirsiniz.

Kaynak Kod / Source Code

```cpp
#include <iostream>

class Organism {
public:
  virtual ~Organism() {}
// EN: Virtual Destructor REQUIRED for V-Table (Polymorphism)
// TR: V-Table için şart!
};

class Human : public Organism {
public:
  void run() { std::cout << "Human is running! (İnsan koşuyor!)" << std::endl; }
};

class Plant : public Organism {};

void cStyle_VS_staticCast() {
  double pi = 3.14159;

  // EN: Old C-way (Bad Habit) - Unclear intention and unsearchable.
  // TR: Eski C Yöntemi (Kötü Alışkanlık) - Güvenliği belirsiz.
  int oldInt = (int)pi;
```

```cpp

  // EN: Modern C++ way (static_cast) - Explicit intention. Compiler verifies it.
  // TR: Modern C++ yöntemi (static_cast) - Niyet belli. Derleyici onaylar.
  int modernInt = static_cast<int>(pi);

  std::cout << "C-Style cast: " << oldInt << " | static_cast: " << modernInt << std::endl;
}

void unlockDynamicCast() {
  Organism *unknownEntity1 = new Human();
  Organism *unknownEntity2 = new Plant();

  std::cout << "\n--- DYNAMIC CAST (Polymorphic Safety / Güvenlik) ---" << std::endl;

  // EN: [INTERVIEW QUESTION]: I have an Organism*. If it's truly a Human, I want to make it run.
  // If it's a Plant, I do nothing. How do I do this without crashing?
  //
  // TR: [MÜLAKAT SORUSU]: Elimde Organism* var, ama "İnsan" ise koşturacağım. "Bitki" ise hiçbir
  // şey yapmayacağım. Nasıl güvenle kontrol ederim?

  // EN: dynamic_cast queries the V-Table! If it matches `Human*`, it returns the pointer. Else,
  // `nullptr`.
  // TR: dynamic_cast gider V-Table'dan sorar! İnsansa verir, değilse "nullptr" döndürür! Çökme
  // engellenir.

  Human *isReallyHuman1 = dynamic_cast<Human *>(unknownEntity1);
  // EN: C++17 allows "if (auto p = dynamic_cast...)"
  // TR: C++17, "if (auto p = dynamic_cast...)" kullanımına izin verir.
  if (isReallyHuman1 != nullptr) {
    std::cout << "[SUCCESS / BAŞARI] 1st Entity is Human!" << std::endl;
    isReallyHuman1->run();
  }

  Human *isReallyHuman2 = dynamic_cast<Human *>(unknownEntity2);
  if (isReallyHuman2 == nullptr) {
    std::cout << "[REJECTED / REDDEDİLDİ] 2nd Entity is NOT Human (Probably Plant). "
            "CRASH PREVENTED! (Çöküş önlendi!)"
        << std::endl;
  }

  delete unknownEntity1;
  delete unknownEntity2;
}

int main() {
  std::cout << "=== MODULE 5: MODERN TYPE CASTING ===\n" << std::endl;

  unlockDynamicCast();

  std::cout << "\n--- reinterpret_cast (Danger Zone / Tehlikeli Bölge) ---" << std::endl;
  // EN: Do not use this unless hacking hardware/drivers. It tells the OS to treat a meaningless
  // number as a RAM address.
  // TR: Donanım programlama harici YASAKTIR. Integer bir sayıyı, RAM adresi imiş gibi yutturmaya
  // denir.

  long hardwareAddress = 0x7FFAABBCCDD;
  int *fakePointer = reinterpret_cast<int *>(hardwareAddress);

  std::cout << "Integer masked as a Pointer (Sayı Pointer yapıldı): " << fakePointer <<
      " (If you dereference this, PROGRAM CRASHES! / İçini okursanız ÇÖKER!)" << std::endl;

  return 0;
}
```

► 04. Exceptions Handling

module_05_certification/04_exceptions_handling.cpp

Konu: CPA/CPP Prep: Exception Handling in C++ / Sertifikasyon: İstisnai Durum (Hata) Yönetimi

Teori / Theory

THEORY: if-else Error Checking vs Exceptions

In old C systems, when an error occurred, functions returned `-1` or `false`. The danger was "Ignoring the Error Value". A developer might forget to check it.
In C++, the `try / catch / throw` mechanism exists. If an error is thrown (`throw`), the program starts tearing itself back down the Stack (Stack Unwinding) until it finds a `catch` block. If you never catch it, it CRASHES (`std::terminate`).

TEORİ: if-else Hata Kontrolü vs İstisnalar

Eski C sistemlerinde bir hata oluştuğunda fonksiyon `-1` veya `false` döndürürdü. Bu yöntem "Hata değerinin yoksayılması / unutulması (Ignore)" riskini taşırdı.
C++'ta `try / catch / throw` mekanizması vardır. Bir hata fırlatılırsa (`throw`), o hatayı Tutan (`catch`) bir yer bulana kadar program kendi kendini Stack'ten (Yığın) geriye doğru söker (Buna "Stack Unwinding" denir). Yakalamazsanız program tamamen GÖÇER (std::terminate).

THEORY: The `noexcept` Keyword (Interview Question)

C++11 introduced `noexcept` (The guarantee of NOT throwing). It tells the compiler: "I swear this function is perfect. It will NEVER throw." The compiler therefore removes all background try-catch safety netting, making the code INCREDIBLY FAST! Often used in 'Move Constructors'.

TEORİ: noexcept Anahtar Kelimesi

C++11 ile `noexcept` (Exception fırlatmama sözü/garantisi) geldi. Derleyiciye: "Söz veriyorum, bu fonksiyonum hatasız çalışacak, hiç hata fırlatmayacak" der. Derleyici bunu görünce tüm hata kontrol (try-catch) koruma zırhlarını kaldırır ve KODU İNANILMAZ HIZLANDIRIR! Genellikle 'Move Constructor' (Taşıma Yıkıcıları) ve 'Destructor' (Yıkıcılar) için yazılır.
DİKKAT: Eğer `noexcept` verip içinde gizlice `throw` yaparsanız (sözünüzü bozduğunuz an), C++ programı ANINDA infaz eder (terminate).

CPPREF Depth: The 4 Levels of Exception Safety Guarantees

CppReference expects professional C++ code to provide one of these guarantees: 1. Nothrow Guarantee: Will NEVER throw (like `noexcept`). 2. Strong Guarantee: If an operation fails, the state is rolled back to exactly what it was before the call (Commit or Rollback - like Databases). 3. Basic Guarantee: If it fails, no memory is leaked, but the object might be in an altered (but valid) state. 4. No Guarantee: Fails and corrupts/leaks memory. (Unacceptable in modern C++).

CPPREF Derinlik: 4 İstisna Güvenliği Garanti Seviyesi

CppReference, profesyonel kodun şu 4 Özel Güvenlik Seviyesinden birini vadetmesini bekler:

olursa, her şey işlem öncesi haline geri döner. Hiç dokunulmamış gibi olur (Veritabanlarındaki Rollback gibi). 3. Temel Teminat (Basic): Hata olursa RAM sızdırmaz, çökmeleri engeller ama nesne o anki bozuk (fakat güvenli) bir halde kalabilir. 4. Garantisiz: Hata olursa RAM sızdırır ve programı çökertir (Modern C++'ta Yasak!)

Kaynak Kod / Source Code

```cpp
#include <iostream>
// EN: Contains many built-in standard exceptions
#include <stdexcept>

// EN: 1. Custom Exception Class
// TR: 1. Özel Hata Sınıfı
// EN: Usually inherited from std::exception.
// TR: Genelde std::exception'dan miras alınarak yapılır.
class BankException : public std::exception {
public:
  // EN: Overriding what() to provide a custom message. Must be `noexcept`.
  // TR: what() fonksiyonunu ezerek kendi mesajımızı veriyoruz. (`noexcept` zorunludur).
  const char *what() const noexcept override {
    return "Insufficient Funds! (Bakiye Yetersiz!)";
  }
};

class BankAccount {
private:
  int balance;

public:
  BankAccount(int initialBalance) : balance(initialBalance) {}

  // EN: 2. A function that THROWS an Exception
  // TR: 2. Hata fırlatan fonksiyon.
  void deposit(int amount) {
    if (amount < 0) {
      // EN: Throw! Execution stops instantly. Program searches for 'catch'.
      // TR: Fırlat (throw)! Program akışı ANINDA KESİLİR, en yakın 'catch' bloğu aranır.
      throw std::invalid_argument( "Cannot deposit negative money! (Negatif para yatırılamaz!)");
    }
    balance += amount;
  }

  void withdraw(int amount) {
    if (amount > balance) {
      // EN: Throwing our CUSTOM Object!
      // TR: Kendi Özel Objemizi fırlattık.
      throw BankException();
    }
    balance -= amount;
  }

  // EN: 3. NOEXCEPT GUARANTEE (Performance Boost)
  // TR: 3. PERFORMANS ARTIŞI
  int getBalance() const noexcept {
    // EN: So simple, it's impossible to throw. Compiler optimizes it to O(1) raw speed.
    // TR: Bu o kadar basit bír işlem ki, hata fırlatması ihtimali SIFIR. Derleyici güvenlik
    // ağlarını siler, en hızlı şekilde (O(1)) C gibi derler.
    return balance;
  }
};

int main() {
  std::cout << "=== MODULE 5: EXCEPTION HANDLING (try/catch/noexcept) ===\n" << std::endl;

  BankAccount myAccount(100);
```

```

  // EN: RULE: Trap risky zones (where a throw might happen) inside a 'try' block!
  // TR: KURAL: Riskli bölgeleri (throw olma ihtimali olanları) 'try' içine hapset!
  try {
    std::cout << "Transaction 1 in progress (İşlem 1 yazılıyor)..." << std::endl;
    myAccount.deposit(50); // Balance: 150

    std::cout << "Transaction 2 in progress (İşlem 2 yapılıyor)..." << std::endl;
    // CRASH! / HATA FIRLIYOR! (Aşağıdaki işlemler iptal)
    myAccount.withdraw( 1000);

    std::cout << "You will never see this line! (Bu satırı göremeyeceksiniz! "
                 "Kod kırıldı.)"
              << std::endl;
    myAccount.deposit(-5);

  }
  // EN: catch blocks are Polymorphic. Put the most specific ones FIRST, the generic ones LAST.
  // TR: catch blokları 'Polimorfik'tir. En spesifik (özel) hatayı başa, en geneli sona
  // (Fallback) yazın.

  catch (const BankException &e) {
    // EN: We caught our custom exception by reference (&).
    // TR: Özel Banka sınıfımızı referans(&) ile yakaladık.
    std::cout << "\n[ERROR CAUGHT - BANK]: " << e.what() << std::endl;
  } catch (const std::invalid_argument &e) {
    // EN: Standard argument error.
    // TR: std içinden gelen Argüman hatası.
    std::cout << "\n[ERROR CAUGHT - ARGUMENT]: " << e.what() << std::endl;
  } catch (const std::exception &e) {
    // EN: The ultimate fallback catcher for any other std::exception.
    // TR: Yukarıdaki özel durumlara uymayan başka BİLİNMEYEN std hataları EN SON BURAYA düşer.
    std::cout << "\n[ERROR CAUGHT - GENERIC]: " << e.what() << std::endl;
  }

  std::cout << "\nProgram did not crash! Life goes on! (Program çökmedi!)" << std::endl;
  std::cout << "Current Balance [noexcept] (Mevcut Bakiye): " << myAccount.getBalance() <<
      std::endl;

  return 0;
}
```

► 05. Const Correctness

module_05_certification/05_const_correctness.cpp

Konu: CPA/CPP Prep: const Correctness in C++ / Sertifikasyon: const Doğruluğu (Sabitlik

Teori / Theory

THEORY: const Member Functions

A const member function promises NOT to modify the object's state. When you mark a method as `const`, the compiler enforces that no data member is modified through `this`. Only const member functions can be called on const objects — calling a non-const method on a const object is a compile-time error.

TEORİ: const Üye Fonksiyonlar

Bir const üye fonksiyon, nesnenin durumunu DEĞİŞTİRMEYECEĞİNE söz verir. Bir metodu `const` olarak işaretlediğinizde, derleyici `this` üzerinden hiçbir veri üyesinin değiştirilmediğini garanti eder. const nesneler üzerinde yalnızca const üye fonksiyonlar çağrılabilir — const bir nesne üzerinde non-const metot çağırmak derleme hatasıdır.

THEORY: The mutable Keyword

`mutable` allows a data member to be modified even inside a const member function. This is used for caches, mutexes, and lazy initialization — things that do NOT change the logical/observable state of the object.

TEORİ: mutable Anahtar Kelimesi

`mutable`, bir veri üyesinin const üye fonksiyon içinde bile değiştirilmesine izin verir. Önbellekler (cache), mutex'ler ve tembel başlatma (lazy init) gibi nesnenin mantıksal/gözlemlenebilir durumunu DEĞİŞTİRMEYEN öğeler için kullanılır.

THEORY: const_cast Dangers

`const_cast` can remove the const qualifier, but removing const from data that was ORIGINALLY declared const is Undefined Behavior! The compiler may place truly-const data in read-only memory. The only legitimate use is interfacing with legacy C APIs that forgot const.

TEORİ: const_cast Tehlikeleri

`const_cast`, const niteleyicisini kaldırabilir, ancak ASLINDA const olarak tanımlanmış veriden const'u kaldırmak Tanımsız Davranıştır (UB)! Derleyici gerçek const verileri salt-okunur belleğe yerleştirebilir. Tek meşru kullanımı, const'u unutan eski C API'leriyle arayüzlemedir.

THEORY: East-const vs West-const

`const int x` (west-const) and `int const x` (east-const) are identical. East-const reads naturally right-to-left: "x is a const int". With pointers it matters: `int const* p` (pointer to const int) vs `int* const p` (const pointer to int). East-const is consistent.

TEORİ: Doğu-const vs Batı-const

`const int x` (batı-const) ile `int const x` (doğu-const) aynıdır. Doğu-const sağdan sola doğal okunur: "x bir const int'tir". Pointer'larda fark önemlidir: `int const* p` (const int'e pointer) vs `int* const p` (int'e const pointer). Doğu-const tutarlıdır.

THEORY: const Propagation & API Design

Passing by `const&` avoids copies while guaranteeing immutability. A well-designed API marks every parameter and method const unless mutation is required. "const is a contract" — it documents intent and enables compiler optimizations. const propagation means if you receive a const ref, everything you pass it to must also accept const.

TEORİ: const Yayılımı ve API

`const&` ile geçiş, kopyalamayı önlerken değişmezliği garanti eder. İyi tasarlanmış bir API, mutasyon gerektirmedikçe her parametreyi ve metodu const olarak işaretler. "const bir sözleşmedir" — niyeti belgeler ve derleyici optimizasyonlarını mümkün kılar. const yayılımı, bir const ref aldığınızda, onu aktardığınız her yerin de const kabul etmesi gerektiği anlamına gelir.

CPPREF Depth: Const Propagation Through Pointers and the Mutable Escape Hatch

`const` on a member function means `this` is `const T*`. BUT: pointers held inside the class are only "shallow const" — `const` on `this` does NOT propagate through `T* member_`. The member pointer itself cannot be reseated, but the pointee remains fully mutable. This is a well-known hole in the const system. `std::experimental::propagate_const<T>` wraps a pointer/smart-pointer and makes `operator*` / `operator->` propagate the constness of the wrapper to the pointee, closing the gap. `mutable` bypasses const entirely — it was designed for mutexes, caches, and lazy initialization (things that do not affect observable state). Overusing `mutable` silently violates the const-correctness contract. Thread-safe const (§17.6.5.9): the standard library assumes `const` member functions are safe to call concurrently. If your `const` methods mutate shared mutable state without synchronization, you have broken this contract and invoked undefined behavior in any STL algorithm.

CPPREF Derinlik: Pointer'lar Üzerinden Const Yayılımı ve Mutable Kaçış Yolu

Bir üye fonksiyondaki `const`, `this`'in `const T*` olduğu anlamına gelir. ANCAK: sınıf içindeki pointer'lar yalnızca "sığ const"tur — `this` üzerindeki `const`, `T* member_` boyunca YAYILMAZ. Pointer'ın kendisi yeniden atanamaz ama gösterdiği veri tamamen değiştirilebilir. `std::experimental::propagate_const<T>` bu açığı kapatır — wrapper'ın const'luğunu pointee'ye yayar. `mutable` ise const'u tamamen atlar; mutex, önbellek ve tembel başlatma için tasarlanmıştır (gözlemlenebilir durumu etkilemeyen şeyler). Aşırı kullanımı const sözleşmesini ihlal eder. İş parçacığı güvenli const (§17.6.5.9): standart kütüphane, `const` üye fonksiyonların eşzamanlı çağrılmasının güvenli olduğunu varsayar. `const` metodlarınız senkronizasyon olmadan paylaşılan mutable durumu değiştiriyorsa, bu sözleşmeyi bozmuş ve herhangi bir STL algoritmasında tanımsız davranışı başlatmış olursunuz.

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: String class for text data
#include <mutex>      // EN: Mutex for thread-safe const access pattern
#include <cmath>      // EN: Math functions for expensive computation demo

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: const and non-const Overloads — SensorCalibration
// EN: Demonstrates overload resolution based on const-ness of the object
// TR: Nesnenin const olup olmadığına göre aşırı yükleme çözünürlüğünü gösterir
// ═══════════════════════════════════════════════════════════════════════════

class SensorCalibration {
private:
    double offset_ = 0.0;
    // EN: Sensor name used in automotive ECU calibration
    // TR: Otomotiv ECU kalibrasyon sensör adı
    std::string sensorName_ = "DefaultSensor";

public:
    explicit SensorCalibration(std::string name, double offset)
        : offset_(offset), sensorName_(std::move(name)) {}

    // EN: const overload — returns by const reference (read-only access)
    // TR: const aşırı yükleme — const referans döndürür (salt okunur erişim)
    const double& getValue() const {
        std::cout << "  [const getValue() called]\n";
        return offset_;
    }

    // EN: non-const overload — returns by reference (allows modification)
    // TR: non-const aşırı yükleme — referans döndürür (değiştirmeye izin verir)
    double& getValue() {
        std::cout << "  [non-const getValue() called]\n";
        return offset_;
    }

    const std::string& getName() const { return sensorName_; }
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: mutable Cache — ExpensiveCalculation
// EN: mutable allows caching inside a const method (logical constness)
// TR: mutable, const metot içinde önbelleklemeye izin verir (mantıksal sabitlik)
// ═══════════════════════════════════════════════════════════════════════════

class ExpensiveCalculation {
private:
    double input_;
    // EN: mutable — these can change even in a const context
    // TR: mutable — const bağlamda bile değişebilir
    mutable double cachedResult_ = 0.0;
    mutable bool dirty_ = true;

    // EN: Simulates a heavy computation (e.g., aerodynamic drag coefficient)
    // TR: Ağır bir hesaplamayı simüle eder (ör. aerodinamik sürtünme katsayısı)
    double computeExpensive() const {
        std::cout << "  [Performing expensive calculation...]\n";
        return std::sin(input_) * std::cos(input_) * 42.0;
    }

public:
    explicit ExpensiveCalculation(double input) : input_(input) {}

    // EN: const method but updates cache via mutable members
    // TR: const metot ama mutable üyeler ile önbelleği günceller
    double getResult() const {
        if (dirty_) {
            cachedResult_ = computeExpensive();
            dirty_ = false;
        }
        return cachedResult_;
    }
```

```cpp

    void setInput(double input) {
        input_ = input;
        dirty_ = true;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: ThreadSafeCounter with mutable std::mutex
// EN: mutable mutex allows locking in const methods for thread-safe reads
// TR: mutable mutex, const metotlarda thread-güvenli okuma için kilitlemeye izin verir
// ═══════════════════════════════════════════════════════════════════════════════

class ThreadSafeCounter {
private:
    int count_ = 0;
    // EN: mutable so we can lock even in const member functions
    // TR: mutable, böylece const üye fonksiyonlarda bile kilitleyebiliriz
    mutable std::mutex mtx_;

public:
    // EN: const method — reads the counter (thread-safe pattern)
    // TR: const metot — sayacı okur (thread-güvenli desen)
    int getCount() const {
        std::lock_guard<std::mutex> lock(mtx_);
        return count_;
    }

    void increment() {
        std::lock_guard<std::mutex> lock(mtx_);
        ++count_;
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: const Propagation Chain — VehicleConfig
// EN: Demonstrates const flowing through multiple function layers
// TR: const'un birden fazla fonksiyon katmanından geçişini gösterir
// ═══════════════════════════════════════════════════════════════════════════════

struct VehicleConfig {
    std::string model = "EV-500";
    int maxSpeedKmh = 250;
    double batteryCapacityKwh = 75.0;
};

// ─── 4a. Low-level reader ──────────────────────────────────────────────────
// EN: Accepts const ref — cannot modify the config
// TR: const ref kabul eder — yapılandırmayı değiştiremez
void printMaxSpeed(const VehicleConfig& config) {
    std::cout << "  Max speed: " << config.maxSpeedKmh << " km/h\n";
}

// ─── 4b. Mid-level validator ───────────────────────────────────────────────
// EN: Also takes const ref — propagates const guarantee downward
// TR: O da const ref alır — const garantisini aşağıya yayar
void validateConfig(const VehicleConfig& config) {
    std::cout << "  Validating config for: " << config.model << "\n";
    printMaxSpeed(config);
}
```

```cpp

// ─── 4c. Top-level entry point ──────────────────────────────────────────────
// EN: Entire chain preserves const — no accidental mutation possible
// TR: Tüm zincir const'u korur — kazara değişiklik imkansız
void processConfig(const VehicleConfig& config) {
    std::cout << "  Processing vehicle configuration...\n";
    validateConfig(config);
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: const_cast Danger (demonstrated safely in comments)
// EN: Removing const from originally-const data is Undefined Behavior!
// TR: Aslında const olan veriden const kaldırmak Tanımsız Davranıştır!
// ═══════════════════════════════════════════════════════════════════════════════

// EN: BAD PRACTICE — DO NOT DO THIS IN PRODUCTION:
// TR: KÖTÜ PRATİK — BUNU ÜRETİMDE YAPMAYIN:
//
// const int engineCode = 42; int& ref = const_cast<int&>(engineCode);   // UB if we write to
// ref! ref = 99;  // UNDEFINED BEHAVIOR — compiler may have placed 42 in ROM
//
// EN: The only acceptable use: calling a C API that takes non-const but promises not to modify:
// TR: Tek kabul edilebilir kullanım: non-const alan ama değiştirmeyeceğine söz veren bir C
// API'sini çağırmak:
//
// void legacy_c_api(char* str);   // C function, doesn't modify str const std::string msg =
// "ECU_OK"; legacy_c_api(const_cast<char*>(msg.c_str()));  // OK if API truly read-only

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " const Correctness — C++ Certification Prep\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ─── Demo 1: const vs non-const overload resolution ────────────────────
    // EN: The compiler picks the overload based on the const-ness of the object
    // TR: Derleyici, nesnenin const olup olmadığına göre aşırı yüklemeyi seçer
    std::cout << "--- Demo 1: const vs non-const overload ---\n";
    {
        SensorCalibration sensor("TirePressure", 2.5);
        const SensorCalibration constSensor("Throttle", 1.0);

        // EN: Non-const object → non-const overload chosen
        // TR: Non-const nesne → non-const aşırı yükleme seçilir
        std::cout << "Mutable sensor:\n";
        sensor.getValue() = 3.0;
        std::cout << "  Updated value: " << sensor.getValue() << "\n";

        // EN: const object → const overload chosen (cannot assign!)
        // TR: const nesne → const aşırı yükleme seçilir (atama yapılamaz!)
        std::cout << "Const sensor:\n";
        double val = constSensor.getValue();
        std::cout << "  Read value: " << val << "\n";
        // constSensor.getValue() = 5.0;  // ERROR: assignment to const ref
    }

    // ─── Demo 2: mutable cache — lazy computation ──────────────────────────
    // EN: const method computes on first call, returns cache on second
    // TR: const metot ilk çağrıda hesaplar, ikincide önbellekten döndürür
    std::cout << "\n--- Demo 2: mutable cache (lazy computation) ---\n";
    {
        const ExpensiveCalculation calc(1.5);
        std::cout << "First call:\n";
        std::cout << "  Result: " << calc.getResult() << "\n";
        std::cout << "Second call (cached):\n";
        std::cout << "  Result: " << calc.getResult() << "\n";
    }
```

```

    // ─── Demo 3: const reference chains ──────────────────────────────────────
    // EN: const flows from top-level to every nested function call
    // TR: const, üst seviyeden her iç içe fonksiyon çağrısına akar
    std::cout << "\n--- Demo 3: const propagation chain ---\n";
    {
        const VehicleConfig config{"EV-900", 300, 100.0};
        processConfig(config);
    }

    // ─── Demo 4: const object restrictions ───────────────────────────────────
    // EN: Shows what you CAN and CANNOT do with a const object
    // TR: const bir nesne ile yapabileceklerinizi ve yapamayacaklarınızı gösterir
    std::cout << "\n--- Demo 4: const object restrictions ---\n";
    {
        const ThreadSafeCounter counter;
        std::cout << "  Count (const access): " << counter.getCount() << "\n";
        // counter.increment();  // ERROR: cannot call non-const on const object

        ThreadSafeCounter mutableCounter;
        mutableCounter.increment();
        mutableCounter.increment();
        mutableCounter.increment();
        std::cout << "  Count (after 3 increments): "
                  << mutableCounter.getCount() << "\n";
    }

    // ─── Demo 5: East-const vs west-const ────────────────────────────────────
    // EN: Both styles are identical — east-const reads right-to-left
    // TR: Her iki stil de aynıdır — doğu-const sağdan sola okunur
    std::cout << "\n--- Demo 5: East-const vs West-const ---\n";
    {
        // EN: West-const style (traditional)
        // TR: Batı-const stili (geleneksel)
        const int westSpeed = 120;

        // EN: East-const style (reads right-to-left: "x is a const int")
        // TR: Doğu-const stili (sağdan sola okunur: "x bir const int'tir")
        int const eastSpeed = 120;

        std::cout << "  West-const: " << westSpeed << "\n";
        std::cout << "  East-const: " << eastSpeed << "\n";
        std::cout << "  Identical? " << (westSpeed == eastSpeed ? "YES" : "NO")
                  << "\n";

        // EN: Where it really matters — pointers
        // TR: Asıl farkın önemli olduğu yer — pointer'lar
        int speed = 200;
        int const* ptrToConst = &speed;      // pointer to const int
        int* const constPtr   = &speed;      // const pointer to int

        // *ptrToConst = 300;  // ERROR: can't modify through pointer-to-const
        *constPtr = 300;       // OK: the pointer is const, not the data
        std::cout << "  constPtr modification: speed = " << speed << "\n";
        (void)ptrToConst;
    }

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << " All const correctness demos completed successfully!\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

► 06. Interview Gotchas

module_05_certification/06_interview_gotchas.cpp

Konu: CPA/CPP Prep: C++ Interview Gotchas & Tricky Behavior / Sertifikasyon: C++ Mülakat

Teori / Theory

THEORY: Most Vexing Parse

`SensorReading sr(TemperatureSource());` looks like object construction, but the compiler parses it as a FUNCTION DECLARATION: a function `sr` that takes a function-pointer parameter and returns `SensorReading`. This is the "Most Vexing Parse" — the C++ grammar always prefers a declaration over an expression. Fix: use brace initialization `{}`.

TEORİ: En Sinir Bozucu Ayrıştırma

`SensorReading sr(TemperatureSource());` nesne oluşturma gibi görünür, ama derleyici bunu bir FONKSİYON BİLDİRİMİ olarak ayrıştırır: `sr` adında, fonksiyon-pointer parametresi alan ve `SensorReading` döndüren bir fonksiyon. Bu "Most Vexing Parse"tır — C++ grameri bildirim olabilecek her şeyi bildirim olarak yorumlar. Çözüm: süslü parantez `{}`.

THEORY: RVO/NRVO (Return Value Optimization)

When a function returns a local object by value, the compiler can eliminate the copy/move entirely — constructing the object directly in the caller's memory. Named RVO (NRVO) applies to named local variables. C++17 mandates RVO for prvalues (unnamed temporaries). NRVO is not guaranteed by the standard but virtually all major compilers apply it.

TEORİ: Dönüş Değeri Opt.

Bir fonksiyon yerel bir nesneyi değer ile döndürdüğünde, derleyici kopyalama/taşımayı tamamen ortadan kaldırabilir — nesneyi doğrudan çağıranın belleğinde oluşturur. İsimli RVO (NRVO) isimli yerel değişkenlere uygulanır. C++17, prvalue'lar (isimsiz geçiciler) için RVO'yu zorunlu kılar. NRVO standart tarafından garanti edilmez ama neredeyse tüm büyük derleyiciler uygular.

THEORY: Dangling References

Returning a reference to a local variable is ALWAYS dangling — the local is destroyed when the function returns. Returning a reference to a temporary has the same problem. Temporaries die at end of the full expression, EXCEPT when bound to a `const&` or `&&` in the same scope, which extends their lifetime to match the reference's scope.

TEORİ: Sarkan (Dangling) Referanslar

Yerel bir değişkene referans döndürmek HER ZAMAN sarkan (dangling) olur — yerel, fonksiyon döndüğünde yok edilir. Geçiciye referans döndürmek de aynı sorundur. Geçiciler tam ifadenin sonunda ölür, ANCAK aynı kapsamda `const&` veya `&&`'ye bağlanırlarsa ömürleri referansın kapsamına kadar uzatılır.

THEORY: Evaluation Order

Before C++17, in `f(a(), b())` the order of evaluating `a()` and `b()` was UNSPECIFIED — either could run first! C++17 guarantees: chained function calls (e.g., `a.b(x).c(y)`) are evaluated left-to-right, and the object expression is evaluated before its arguments.

TEORİ: Değerlendirme Sırası

C++17 öncesinde, `f(a(), b())` ifadesinde `a()` ve `b()` değerlendirme sırası BELİRTİLMEMİŞTİ — hangisi önce çalışır bilinmezdi! C++17 garantisi: zincirleme fonksiyon çağrılarında (ör. `a.b(x).c(y)`) soldan sağa değerlendirilir ve nesne ifadesi argümanlarından önce değerlendirilir.

THEORY: Implicit Conversions & explicit

A single-argument constructor acts as an implicit conversion operator! `Voltage v = 3.3;` silently constructs a Voltage from a double. This causes subtle bugs. Mark constructors `explicit` to prevent this. `explicit` is one of the most important defensive keywords in C++.

TEORİ: Örtülü Dönüşümler

Tek argümanlı yapıcı, örtülü bir dönüşüm operatörü gibi davranır! `Voltage v = 3.3;` sessizce bir double'dan Voltage oluşturur. Bu sinsi hatalara yol açar. Yapıcıları `explicit` olarak işaretleyerek önleyin. `explicit`, C++'taki en önemli savunma anahtar kelimelerden biridir.

CPPREF Depth: RVO, NRVO and Copy Elision — Guaranteed vs Optional

C++17 guarantees RVO for prvalue returns — the copy/move constructor need not even exist! The object is materialized directly in the caller's storage. NRVO (Named Return Value Optimization) applies to named local variables returned by value; it is NOT guaranteed by the standard but every major compiler (GCC, Clang, MSVC) performs it. CRITICAL PITFALL: writing `return std::move(local);` PREVENTS NRVO because the expression is no longer an lvalue naming a local — the compiler sees an xvalue and must use the move constructor instead of eliding entirely. Never `std::move` a return value of a local object. Copy elision also applies to throw/catch: throwing a local by value can elide the copy into the exception object. In practice, returning by value is often FASTER than returning through an output parameter or pointer, because elision eliminates all copies while an output param forces at least one construction + assignment.

CPPREF Derinlik: RVO, NRVO ve Kopya Eleme — Garantili vs İsteğe Bağlı

C++17, prvalue dönüşleri için RVO'yu garanti eder — kopya/taşıma yapıcısının var olması bile gerekmez! Nesne doğrudan çağıranın belleğinde oluşturulur. NRVO (İsimli Dönüş Değeri Optimizasyonu) değer ile döndürülen isimli yerel değişkenlere uygulanır; standart tarafından garanti EDİLMEZ ama tüm büyük derleyiciler uygular. KRİTİK TUZAK: `return std::move(local);` yazmak NRVO'yu ENGELLER çünkü ifade artık bir yerel'i isimlendiren lvalue değildir — derleyici bir xvalue görür ve tamamen elemek yerine taşıma yapıcısını kullanmak zorunda kalır. Asla yerel nesnenin dönüş değerini `std::move` ile sarmalamayın. Kopya eleme throw/catch için de geçerlidir. Pratikte, değer ile dönüş genellikle çıktı parametresi veya pointer ile dönüşten DAHA HIZLIDIR çünkü eleme tüm kopyaları ortadan kaldırır.

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: String class for text data
#include <vector>     // EN: Dynamic array for sensor data

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Most Vexing Parse — ECUConfig
// EN: Parentheses cause the compiler to see a function declaration, not a ctor
// TR: Parantezler derleyicinin nesne değil fonksiyon bildirimi görmesine yol açar
// ═══════════════════════════════════════════════════════════════════════════

struct DataSource {
    // EN: Simulates an automotive data bus (CAN/LIN)
    // TR: Otomotiv veri yolunu simüle eder (CAN/LIN)
    int channelId = 1;
};

class ECUConfig {
    std::string name_;
    int channel_;
public:
    // EN: Default constructor
    // TR: Varsayılan yapıcı
    ECUConfig() : name_("DefaultECU"), channel_(0) {
        std::cout << "  ECUConfig() default ctor\n";
    }

    // EN: Constructor taking a DataSource
    // TR: DataSource alan yapıcı
    explicit ECUConfig(DataSource src)
        : name_("ECU_CH" + std::to_string(src.channelId)),
          channel_(src.channelId) {
        std::cout << "  ECUConfig(DataSource) ctor — channel "
                  << channel_ << "\n";
    }

    const std::string& getName() const { return name_; }
};

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: RVO/NRVO Demo — SensorPacket with Noisy Ctors
// EN: Tracks every copy/move to prove they are elided by the compiler
// TR: Derleyicinin kopyalama/taşımayı ortadan kaldırdığını kanıtlamak için izler
// ═══════════════════════════════════════════════════════════════════════════

class SensorPacket {
    std::string tag_;
    double value_;
public:
    explicit SensorPacket(std::string tag, double val)
        : tag_(std::move(tag)), value_(val) {
        std::cout << "  [SensorPacket CONSTRUCTED: " << tag_ << "]\n";
    }

    SensorPacket(const SensorPacket& other)
        : tag_(other.tag_), value_(other.value_) {
        std::cout << "  [SensorPacket COPY ctor: " << tag_ << "]\n";
    }

    SensorPacket(SensorPacket&& other) noexcept
        : tag_(std::move(other.tag_)), value_(other.value_) {
        std::cout << "  [SensorPacket MOVE ctor: " << tag_ << "]\n";
    }

    SensorPacket& operator=(const SensorPacket&) = default;
    SensorPacket& operator=(SensorPacket&&) noexcept = default;
    ~SensorPacket() = default;

    friend std::ostream& operator<<(std::ostream& os, const SensorPacket& sp) {
        return os << sp.tag_ << " = " << sp.value_;
    }
};
```

```cpp

// EN: Returns a local object by value — NRVO should elide the copy/move
// TR: Yerel nesneyi değer ile döndürür — NRVO kopyayı/taşımayı ortadan kaldırır
SensorPacket createPacket() {
    SensorPacket packet("EngineTemp", 92.5);
    return packet;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: explicit Constructor — Voltage
// EN: Prevents dangerous implicit double → Voltage conversions
// TR: Tehlikeli örtülü double → Voltage dönüşümlerini engeller
// ═══════════════════════════════════════════════════════════════════════════════

class Voltage {
    double volts_;
public:
    // EN: explicit prevents: Voltage v = 3.3; (implicit conversion)
    // TR: explicit şunu engeller: Voltage v = 3.3; (örtülü dönüşüm)
    explicit Voltage(double v) : volts_(v) {}

    double get() const { return volts_; }

    friend std::ostream& operator<<(std::ostream& os, const Voltage& v) {
        return os << v.volts_ << "V";
    }
};

// EN: Without explicit, this function could be called with a plain double!
// TR: explicit olmasaydı, bu fonksiyona düz bir double ile çağrı yapılabilirdi!
void checkVoltage(const Voltage& v) {
    std::cout << "  Checking voltage: " << v << "\n";
    if (v.get() > 14.0) {
        std::cout << "  WARNING: Over-voltage detected!\n";
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: Dangling Reference Anti-Pattern (safe demonstration)
// EN: Shows the WRONG way and the RIGHT way to return data from functions
// TR: Fonksiyonlardan veri döndürmenin YANLIŞ ve DOĞRU yolunu gösterir
// ═══════════════════════════════════════════════════════════════════════════════

// EN: WRONG — returns reference to a local (dangling after return!)
// TR: YANLIŞ — yerel değişkene referans döndürür (dönüşten sonra sarkık!) const std::string&
// getBrandDangling() { std::string brand = "AutomotiveCorp"; return brand;  // DANGLING: brand
// is destroyed here! }

// EN: RIGHT — return by value (RVO will optimize it anyway)
// TR: DOĞRU — değer ile döndür (RVO zaten optimize edecek)
std::string getBrandSafe() {
    std::string brand = "AutomotiveCorp";
    return brand;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: Evaluation Order Demo (C++17)
// EN: Demonstrates guaranteed left-to-right evaluation in chained calls
// TR: Zincirleme çağrılarda garantili soldan sağa değerlendirmeyi gösterir
// ═══════════════════════════════════════════════════════════════════════════════

class DiagnosticChain {
    std::vector<std::string> log_;
public:
```

```cpp
    // EN: Each method returns *this for chaining, logs order of calls
    // TR: Her metot zincirleme için *this döndürür, çağrı sırasını loglar
    DiagnosticChain& checkEngine(int code) {
        log_.push_back("Engine[" + std::to_string(code) + "]");
        return *this;
    }

    DiagnosticChain& checkBrakes(int code) {
        log_.push_back("Brakes[" + std::to_string(code) + "]");
        return *this;
    }

    DiagnosticChain& checkTransmission(int code) {
        log_.push_back("Transmission[" + std::to_string(code) + "]");
        return *this;
    }

    void printLog() const {
        std::cout << "  Evaluation order: ";
        for (size_t i = 0; i < log_.size(); ++i) {
            if (i > 0) std::cout << " -> ";
            std::cout << log_[i];
        }
        std::cout << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════\n";
    std::cout << " Interview Gotchas — C++ Certification Prep\n";
    std::cout << "═══════════════════════════════════════════════════\n\n";

    // ─── Demo 1: Most Vexing Parse ─────────────────────────────────────────
    // EN: Parentheses = function declaration! Braces = correct construction.
    // TR: Parantez = fonksiyon bildirimi! Süslü parantez = doğru oluşturma.
    std::cout << "--- Demo 1: Most Vexing Parse ---\n";
    {
        // EN: BROKEN — this declares a FUNCTION, not an object:
        // TR: BOZUK — bu bir NESNE değil, FONKSİYON bildirir: ECUConfig config(DataSource());
        // // Function declaration!

        // EN: FIX 1 — brace initialization (preferred in modern C++)
        // TR: ÇÖZÜM 1 — süslü parantez başlatma (modern C++'ta tercih edilir)
        ECUConfig config1{DataSource()};
        std::cout << "  config1 name: " << config1.getName() << "\n";

        // EN: FIX 2 — extra parentheses around argument
        // TR: ÇÖZÜM 2 — argüman etrafında ekstra parantez
        ECUConfig config2((DataSource()));
        std::cout << "  config2 name: " << config2.getName() << "\n";

        // EN: FIX 3 — named variable for the argument
        // TR: ÇÖZÜM 3 — argüman için isimli değişken
        DataSource src;
        ECUConfig config3(src);
        std::cout << "  config3 name: " << config3.getName() << "\n";
    }

    // ─── Demo 2: RVO — zero copies ─────────────────────────────────────────
    // EN: Noisy constructors prove that no copy or move happens
    // TR: Gürültülü yapıcılar kopya veya taşıma olmadığını kanıtlar
    std::cout << "\n--- Demo 2: RVO/NRVO (Return Value Optimization) ---\n";
    {
        std::cout << "Creating packet via function return:\n";
        SensorPacket pkt = createPacket();
        std::cout << "  Packet: " << pkt << "\n";
        std::cout << "  (If no COPY/MOVE printed above, NRVO worked!)\n";
    }
```

```

    // ─── Demo 3: explicit prevents implicit conversion ──────────────────────────
    // EN: Without explicit, checkVoltage(3.3) would silently compile
    // TR: explicit olmasaydı, checkVoltage(3.3) sessizce derlenir
    std::cout << "\n--- Demo 3: explicit constructor ---\n";
    {
        Voltage batteryVoltage(12.6);
        checkVoltage(batteryVoltage);

        // EN: This would fail to compile — no implicit double → Voltage:
        // TR: Bu derlenmez — örtülü double → Voltage dönüşümü yok: checkVoltage(3.3);  // ERROR:
        // no implicit conversion

        // EN: Must be explicit:
        // TR: Açık olmalı:
        checkVoltage(Voltage(14.5));
    }

    // ─── Demo 4: Temporary lifetime extension ────────────────────────────────────
    // EN: Binding a temporary to const& extends its lifetime
    // TR: Bir geçiciyi const&'ye bağlamak ömrünü uzatır
    std::cout << "\n--- Demo 4: Temporary lifetime extension ---\n";
    {
        // EN: Safe — return by value, stored in local variable
        // TR: Güvenli — değer ile dönüş, yerel değişkende saklanır
        std::string brand = getBrandSafe();
        std::cout << "  Brand (by value): " << brand << "\n";

        // EN: Safe — const& extends the temporary's lifetime to this scope
        // TR: Güvenli — const& geçicinin ömrünü bu kapsama kadar uzatır
        const std::string& tempRef = getBrandSafe();
        std::cout << "  Brand (const& to temp): " << tempRef << "\n";

        // EN: Safe — auto&& (forwarding ref) also extends lifetime
        // TR: Güvenli — auto&& (yönlendirme ref) de ömrü uzatır
        auto&& fwdRef = getBrandSafe();
        std::cout << "  Brand (auto&& to temp): " << fwdRef << "\n";
    }

    // ─── Demo 5: Evaluation order — C++17 left-to-right ──────────────────────────
    // EN: Chained member function calls are evaluated left-to-right in C++17
    // TR: Zincirleme üye fonksiyon çağrıları C++17'de soldan sağa değerlendirilir
    std::cout << "\n--- Demo 5: Evaluation order (C++17) ---\n";
    {
        DiagnosticChain diag;
        // EN: C++17 guarantees: checkEngine → checkBrakes → checkTransmission
        // TR: C++17 garantisi: checkEngine → checkBrakes → checkTransmission
        diag.checkEngine(101)
            .checkBrakes(202)
            .checkTransmission(303);
        diag.printLog();
    }

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << " All interview gotcha demos completed successfully!\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

Modül 6: Industry Standards — Endüstri Standartları

Endüstri Standartları

6 kaynak dosya · Standart: C++17

C++17

► 01. Concurrency Threads

module_06_industry_standards/01_concurrency_threads.cpp

Konu: Industry Standards: Threading and Concurrency / Endüstri Standartları: İş Parçacıkları

Teori / Theory

THEORY: Why Multithreading?

Single-threaded programs execute one line at a time. If a video is rendering and taking 5 minutes, the entire UI freezes. Modern CPUs have 8, 16, or 32 cores. `std::thread` allows C++ to split work across these cores simultaneously.

TEORİ: Neden Çoklu İş Parçacığı?

Tek iş parçacıklı (single-threaded) programlar aynı anda tek satır okur. Eğer bir video render ediliyorsa ve 5 dakika sürüyorsa, bütün kullanıcı arayüzü (UI) donar. Modern işlemciler 16-32 çekirdeklidir. C++11 ile gelen `std::thread`, donanımın tüm gücünü sömürerek işleri yan yana (aynı anda) fiziksel olarak çalıştırmamızı sağlar.

THEORY: Race Condition and Mutex

Two threads trying to write to the SAME variable at the EXACT same nanosecond causes a "Race Condition" (corruption). We must protect shared data. `std::mutex` acts like a bathroom key. Only one thread can hold the key, modify data, and return the key. Others wait at the door.

TEORİ: Yarış Durumu ve Mutex Kilidi

İki ayrı Thread'in saniyenin milyarda biri hızla aynı değişkene (aynı RAM adresine) YAZMAYA çalışmasına 'Race Condition' (Yarış Durumu) denir. Veri anında bozulur. Bunu çözmek için `std::mutex` (Mutual Exclusion / Karşılıklı Dışlama) kullanırız. Mutex bir "Kilitli Kapı Anahtarı" gibidir. Bir Thread anahtarı alır (`lock()`), veriyi değiştirir ve işi bitince anahtarı bırakır (`unlock()`). Diğer Thread'ler kapıda beklemek zorundadır.

THEORY: std::atomic — Lock-Free Thread Safety

For simple types (int, bool, pointers), `std::atomic<T>` provides thread-safe reads and writes WITHOUT needing a mutex. The CPU guarantees the operation completes in a single indivisible step. This is dramatically faster than mutex for counters, flags, and simple shared state. Operations like `fetch_add()`, `compare_exchange_strong()`, and `store()/load()` are all atomic.

TEORİ: std::atomic — Kilitsiz Thread Güvenliği

Basit tipler (int, bool, pointer) için `std::atomic<T>`, mutex'e gerek kalmadan thread-güvenli okuma/yazma sağlar. CPU, işlemin TEK bölünmez adımda tamamlanacağını garanti eder. Sayaçlar, bayraklar ve basit paylaşımlı durum için mutex'ten DRAMATIK şekilde hızlıdır. `fetch_add()`, `compare_exchange_strong()`, `store()/load()` gibi işlemler atomiktir.

THEORY: std::condition_variable — Thread Signaling

Sometimes a thread needs to WAIT until a certain condition is met (e.g., "wait until the queue has data"). Busy-waiting (spin loop) wastes CPU. `std::condition_variable` lets a thread sleep (zero CPU) until another thread calls `notify_one()` or `notify_all()`. This is the foundation of Producer-Consumer patterns.

TEORİ: std::condition_variable — Thread Sinyalleşmesi

Bazen bir thread belirli bir koşul gerçekleşene kadar BEKLEMELİDİR (örn. "kuyrukta veri olana kadar bekle"). Sürekli döngüde kontrol etmek (spin loop) CPU israfıdır. `std::condition_variable`, bir thread'in sıfır CPU ile uyumasına ve başka bir thread `notify_one()` veya `notify_all()` çağırana kadar beklemesine olanak tanır. Producer-Consumer kalıbının temelidir.

CPPREF Depth: Data Race is PURE Undefined Behavior!

CppReference explicitly states: A Data Race isn't just "getting the wrong math result". It is strictly Undefined Behavior (UB)! The compiler is allowed to completely optimize away your loop, crash the program, or print garbage. Even READING a variable while another thread is WRITING to it is a Data Race!

CPPREF Derinlik: Veri Yarışı SAF Tanımsız Davranıştır!

Data Race (Veri Yarışı) C++'ta sadece "Yanlış matematiksel sonuç çıkması" demek değildir. CppReference'a göre BU KESİN BİR UB'DİR (Tanımsız Davranış)! Derleyici kodun o kısmını tamamen silebilir, çökertebilir. Bir Thread yazarken diğerinin sadece OKUMASI bile UB'dir (Yarı yazılmış bit'leri -Torn Reads- okur).

CPPREF Depth: std::atomic Memory Ordering — The Hidden Complexity

By default, `std::atomic` uses `memory_order_seq_cst` (sequentially consistent), the STRONGEST and slowest ordering. For performance-critical code, weaker orderings like `memory_order_relaxed` (no ordering guarantees), `memory_order_acquire/release` (one-directional fence) exist. Using the wrong ordering leads to subtle, platform-specific bugs that only appear on ARM/weak-memory architectures. cppreference.com/w/cpp/atomic/memory_order

CPPREF Derinlik: std::atomic Bellek Sıralaması — Gizli Karmaşıklık

Varsayılan olarak `std::atomic`, `memory_order_seq_cst` (sıralı tutarlı) kullanır — en GÜÇLÜ ve en yavaş sıralama. Performans-kritik kodlar için `memory_order_relaxed` (sıralama garantisi yok), `memory_order_acquire/ release` (tek yönlü çit) gibi zayıf sıralamalar vardır. Yanlış sıralama, yalnızca ARM/zayıf-bellek mimarilerinde ortaya çıkan ince, platforma özgü hatalara yol açar. cppreference.com/w/cpp/atomic/memory_order

CPPREF Depth: Spurious Wakeups in condition_variable

`condition_variable::wait()` can wake up WITHOUT anyone calling `notify_*()`. This is called a "Spurious Wakeup" and is allowed by the POSIX and C++ standards. Always use the predicate overload: `cv.wait(lock, [&]{ return !queue.empty(); })` to handle this. The lambda is re-checked every time the thread wakes, spurious or not. cppreference.com/w/cpp/thread/condition_variable/wait

CPPREF Derinlik: condition_variable'da Sahte Uyanmalar

`condition_variable::wait()`, kimse `notify_*()` çağırmadan UYANABİLİR. Buna "Sahte Uyanma (Spurious Wakeup)" denir ve POSIX ile C++ standartları tarafından izin verilir. Her zaman predicate (koşul) aşırı yüklemesini kullanın: `cv.wait(lock, [&]{ return !queue.empty(); })`. Lambda, thread her uyandığında (sahte veya değil) yeniden kontrol edilir. cppreference.com/w/cpp/thread/condition_variable/wait

Kaynak Kod / Source Code

```cpp
#include <atomic>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

// EN: Shared Key
// TR: Ortak Kilit
std::mutex mtx;
int sharedBankAccount = 0;

/**
 * @brief EN: A task to be run by multiple threads at the same time.
 * TR: Aynı anda birden fazla iş parçacığı tarafından koşturulacak görev.
 */
void depositMoney(int id, int amount) {
  // EN: Thread ID for identification
  // TR: Thread kimliği
  (void)id;
  for (int i = 0; i < 10000; i++) {
    // [THEORY: std::lock_guard & RAII]
    // EN: lock_guard works like unique_ptr (RAII). It locks the mutex when entering the scope
    // {...} and AUTOMATICALLY unlocks it when the scope ends. No risk of Deadlocks even if an
    // exception is thrown!
    //
    // TR: lock_guard tıpkı unique_ptr gibidir (RAII prensibi). Süslü parantez {...} içine
    // girerken Mutex'i otomatik kilitler (lock). Süslü parantez bitince otomatik olarak kilidi
    // AÇAR (unlock). Exception fırlasa bile sistem kilitlenmez (Deadlock olmaz!)

    std::lock_guard<std::mutex> lock(mtx);

    // EN: CRITICAL SECTION. Only ONE thread is allowed here at any given time.
    // TR: KRİTİK BÖLGE. Burada aynı anda sadece TEK bir thread islem yapabilir.
    sharedBankAccount += amount;

  // EN: lock_guard dies here and returns the key.
  // TR: lock_guard burada yıkılır ve anahtarı devreder.
  }
}

// ==========================================================================================
// [DEMO 2: std::atomic - Lock-Free Counter / Kilitsiz Sayaç]
// ==========================================================================================
std::atomic<int> atomicCounter{0};
```

```cpp

void atomicIncrement(int iterations) {
  for (int i = 0; i < iterations; i++) {
    // EN: fetch_add is ONE atomic CPU instruction — no mutex needed!
    // TR: fetch_add TEK atomik CPU komutu — mutex gerekmez!
    atomicCounter.fetch_add(1, std::memory_order_relaxed);
  }
}

// ==========================================================================================
// [DEMO 3: Producer-Consumer with condition_variable]
// ==========================================================================================
std::queue<int> taskQueue;
std::mutex queueMtx;
std::condition_variable cv;
bool producerDone = false;

/**
 * @brief EN: Producer pushes tasks into the queue and notifies consumers.
 * TR: Üretici, kuyruğa görev ekler ve tüketicileri bilgilendirir.
 */
void producer(int taskCount) {
  for (int i = 1; i <= taskCount; i++) {
    {
      std::lock_guard<std::mutex> lock(queueMtx);
      taskQueue.push(i);
      std::cout << "[PRODUCER] Pushed task #" << i << std::endl;
    }
    // EN: Wake ONE waiting consumer thread
    // TR: Bekleyen BİR tüketiciyi uyandır
    cv.notify_one();
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
  }
  {
    std::lock_guard<std::mutex> lock(queueMtx);
    producerDone = true;
  }
  // EN: Wake ALL remaining consumers to let them exit.
  // TR: Kalan TÜM tüketicileri çıkabilmeleri için uyandır.
  cv.notify_all();
}

/**
 * @brief EN: Consumer waits for tasks and processes them.
 * TR: Tüketici görev bekler ve işler.
 */
void consumer(int id) {
  while (true) {
    std::unique_lock<std::mutex> lock(queueMtx);
    // EN: Wait with predicate to handle SPURIOUS WAKEUPS safely!
    // TR: Sahte uyanmalara karşı predicate ile bekle!
    cv.wait(lock, [] { return !taskQueue.empty() || producerDone; });

    if (taskQueue.empty() && producerDone) {
      // EN: No more work
      // TR: İş kalmadı
      break;
    }

    int task = taskQueue.front();
    taskQueue.pop();
    // EN: Release lock before processing
    // TR: İşlemden önce kilidi bırak
    lock.unlock();

    std::cout << "[CONSUMER " << id << "] Processing task #" << task << std::endl;
  }
}

int main() {
  std::cout << "=== MODULE 6: INDUSTRY STANDARDS - CONCURRENCY ===\n" << std::endl;
```

```

  // ==========================================================================================
  // PART 1: Mutex-Protected Bank Account (Original Demo)
  // ==========================================================================================
  std::cout << "--- PART 1: std::mutex + std::lock_guard ---\n" << std::endl;
  std::cout << "Main Thread started (Ana cekirdek acildi).\n" << std::endl;

  std::vector<std::thread> workers;
  for (int i = 0; i < 10; i++) {
    workers.push_back(std::thread(depositMoney, i, 1));
  }

  std::cout << "10 Threads LAUNCHED! Running massively in the background...\n"
               "(10 Thread Firlatildi! Arka planda devasa hizla calisiyorlar...)\n"
            << std::endl;

  for (auto &w : workers) {
    w.join();
  }

  std::cout << "Expected Bank Account: 100000" << std::endl;
  std::cout << "Actual Bank Account:   " << sharedBankAccount << std::endl;

  if (sharedBankAccount == 100000) {
    std::cout << "=> [SUCCESS] std::mutex prevented the Race Condition!\n" << std::endl;
  }

  // ==========================================================================================
  // PART 2: std::atomic — Lock-Free Counter
  // ==========================================================================================
  // EN: 10 threads each increment an atomic counter 10000 times. No mutex needed! The CPU
  // handles atomicity at the hardware level.
  // TR: 10 thread her biri atomik sayacı 10000 kez artırır. Mutex gerekmez! CPU atomikliği
  // donanım seviyesinde halleder.
  std::cout << "--- PART 2: std::atomic (Lock-Free Counter) ---\n" << std::endl;

  std::vector<std::thread> atomicWorkers;
  for (int i = 0; i < 10; i++) {
    atomicWorkers.emplace_back(atomicIncrement, 10000);
  }
  for (auto &w : atomicWorkers) {
    w.join();
  }

  std::cout << "Expected atomic counter: 100000" << std::endl;
  std::cout << "Actual atomic counter:   " << atomicCounter.load() << std::endl;

  if (atomicCounter.load() == 100000) {
    std::cout << "=> [SUCCESS] std::atomic is lock-free and correct!\n" << std::endl;
  }

  // ==========================================================================================
  // PART 3: Producer-Consumer with condition_variable
  // ==========================================================================================
  // EN: Classic multi-threaded pattern: 1 producer pushes tasks, 2 consumers process them.
  // condition_variable coordinates without busy-waiting.
  // TR: Klasik çoklu-thread kalıbı: 1 üretici görev ekler, 2 tüketici işler. condition_variable
  // meşgul-bekleme olmadan koordine eder.
  std::cout << "--- PART 3: Producer-Consumer (condition_variable) ---\n"
            << std::endl;

  std::thread prod(producer, 6);
  std::thread cons1(consumer, 1);
  std::thread cons2(consumer, 2);

  prod.join();
  cons1.join();
  cons2.join();

  std::cout << "\n=> Producer-Consumer pattern completed with zero busy-waiting!"
            << std::endl;

  return 0;
}
```

► 02. Cmake Basics.Txt

module_06_industry_standards/02_cmake_basics.txt

Kaynak Kod / Source Code

```
# CMakeLists.txt (Example / Ornek)
# ------------------------------------------------------------------
# [THEORY: Why Do We Use CMake? / TEORI: Neden CMake Kullanıyoruz?]
# EN:
# Compiling C++ code manually like `g++ a.cpp b.cpp -o main` works for small tasks.
# But what if you have hundreds of files, or external libraries written by someone
# else like "nlohmann_json" or "Google_Test"? It becomes impossible!
# CMake is a PERFECT "meta-build" tool. It allows your project to be compiled with
# the EXACT SAME STANDARD on Windows (Visual Studio), Linux (Make/Ninja), or Mac (Xcode).
#
# TR:
# C++ projelerinde her dosyayi "g++ a.cpp b.cpp -o main" seklinde derlemek kücük
# projelerde calisir. Ancak yüzlerce dosyaniz ve baska birinin yazdigi "Google_Test"
# gibi kütüphaneleriniz varsa bu mümkün degildir!
# CMake, projeyi her isletim sisteminde ayni standartta hatasız derlememizi
# saglayan MUKEMMEL bir meta-derleme aracidir. Endüstri standardidir.
# ------------------------------------------------------------------

# 1. Minimum CMake Version Required
# EN: Tells the system the lowest acceptable CMake version.
# TR: Projenin gerektirdigi minimum CMake sürümü.
cmake_minimum_required(VERSION 3.10)

# 2. Project Declaration
# EN: Setting the Project Name, Version, and the Language (CXX = C++).
# TR: Proje Adi, Versiyonu ve Dil Belirleme (CXX).
project(ModernCppMastery VERSION 1.0.0 LANGUAGES CXX)

# 3. Setting C++ Standards (Crucial!)
# EN: Forcing the compiler to use C++17 rules strictly.
# TR: Hangi C++ Standardini kullanacagimizi derleyiciye dayatiyoruz (Zorunlu kiliyoruz).
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED True)

# 4. Source Files & Executable Target
# EN: "Create an application named MainApp utilizing these source files."
# TR: Kodlarimizin bulundugu baglantilari ve olusacak calistirilabilir dosyanin adini belirliyoruz.
# add_executable(MainApp src/main.cpp src/player.cpp)

# 5. External Libraries (Dependencies)
# EN: If we downloaded a package via vcpkg or conan, we bind it here.
# TR: Eger disaridan (vcpkg / conan yardimiyla) bir kütüphane indirilmisse onu kodumuzla birlestiririz.
# find_package(GTest REQUIRED)
# target_link_libraries(MainApp PRIVATE GTest::gtest_main)
```

► 03. Unit Testing Gtest

module_06_industry_standards/03_unit_testing_gtest.cpp

Konu: Industry Standards: Unit Testing (TDD) with Google Test Simulation / Endüstri

Teori / Theory

THEORY: Why is Unit Testing the Peak of Professionalism?

Writing code is easy. Proving that it works perfectly under every condition is engineering. If you change a function today, how do you know you didn't break something you wrote 3 months ago? "Test-Driven Development (TDD)" and frameworks like "Google Test (GTest)" automate this. The cycle: 1) Write a FAILING test, 2) Write minimal code to pass, 3) Refactor.

TEORİ: TDD Nedir?

Kod yazmak kolaydir. O kodun her kosulda (%100) calistigini "kanitlamak" ise yazılım mühendisligidir. Bugün bir fonksiyonda degisiklik yaptiginizda, 3 ay önce yazilan modulleri bozmadiginizi nereden biliyorsunuz? İste "Test-Driven Development (TDD)" yani Test Güdümlü Gelistirme burada devreye girer. "Google Test" (GTest), cogu savunma sanayi, otomotiv veya HFT (Finans) sirketinin C++ kodunu test etmek icin kullandigi standard kütüphanedir. Döngü: 1) BAŞARISIZ test yaz, 2) Geçecek minimum kodu yaz, 3) Refactor et.

THEORY: TEST() and TEST_F() Macros

In real GTest, `TEST(SuiteName, TestName)` defines a standalone test case. `TEST_F(FixtureName, TestName)` defines a test that uses a "Test Fixture" — a class with `SetUp()` and `TearDown()` methods that run before and after every test. Fixtures share setup logic (like creating database connections) across multiple tests without code duplication.

TEORİ: TEST() ve TEST_F() Makroları

Gerçek GTest'te `TEST(SüitAdı, TestAdı)` bağımsız bir test tanımlar. `TEST_F(FixtureAdı, TestAdı)` ise bir "Test Fixture" kullanan test tanımlar — her testten önce ve sonra çalışan `SetUp()` ve `TearDown()` metodlarına sahip bir sınıf. Fixture'lar kurulum mantığını (veritabanı bağlantısı oluşturma gibi) birden fazla test arasında kod tekrarı olmadan paylaşır.

THEORY: EXPECT vs ASSERT

`ASSERT_EQ(a, b)` — if fails, test STOPS immediately. Use for critical preconditions. `EXPECT_EQ(a, b)` — if fails, test CONTINUES running remaining checks (reports all failures at end). In production, `EXPECT_*` is preferred so you see ALL failures in one run, not just the first one.

TEORİ: EXPECT vs ASSERT Farkı

`ASSERT_EQ(a, b)` — başarısız olursa test ANINDA durur. Kritik ön koşullar için kullanılır. `EXPECT_EQ(a, b)` — başarısız olursa test çalışmaya DEVAM eder (tüm hataları sonda raporlar). Üretimde `EXPECT_*` tercih edilir çünkü tek çalıştırmada TÜM hataları görürsünüz.

THEORY: Dependency Injection for Testability

Tightly coupled code (class directly calls database/network) is UNTESTABLE because you can't run the test without an actual server. Solution: inject dependencies through interfaces (abstract classes). In tests, you pass a "Mock" or "Stub" instead of the real implementation. The class under test doesn't know the difference!

TEORİ: Test Edilebilirlik için Bağımlılık Enjeksiyonu

Sıkı bağlı kod (sınıf doğrudan veritabanı/ağ çağırır) TEST EDİLEMEZDİR çünkü gerçek sunucu olmadan test çalıştıramazsınız. Çözüm: bağımlılıkları arayüzler (soyut sınıflar) aracılığıyla enjekte edin. Testlerde gerçek uygulama yerine "Mock" veya "Stub" geçirin. Test edilen sınıf farkı bilmez!

CPPREF Depth: GTest Assertion Macros Reference

Key GTest macros from the official documentation: - `EXPECT_EQ(a, b)` / `ASSERT_EQ(a, b)` — equality - `EXPECT_NE(a, b)` — not equal - `EXPECT_LT(a, b)` / `EXPECT_GT(a, b)` — less/greater - `EXPECT_TRUE(cond)` / `EXPECT_FALSE(cond)` — boolean - `EXPECT_THROW(stmt, ExType)` — expects exception - `EXPECT_NEAR(a, b, eps)` — floating point comparison - `EXPECT_THAT(val, matcher)` — GMock matchers google.github.io/googletest/reference/assertions.html

CPPREF Derinlik: GTest Assertion Makroları Referansı

Resmi dokümantasyondan temel GTest makroları: - `EXPECT_EQ(a, b)` / `ASSERT_EQ(a, b)` — eşitlik - `EXPECT_NE(a, b)` — eşit değil - `EXPECT_LT(a, b)` / `EXPECT_GT(a, b)` — küçük/büyük
- `EXPECT_TRUE(cond)` / `EXPECT_FALSE(cond)` — boolean - `EXPECT_THROW(stmt, ExType)` — istisna beklentisi - `EXPECT_NEAR(a, b, eps)` — kayan nokta karşılaştırma - `EXPECT_THAT(val, matcher)` — GMock eşleştiriciler google.github.io/googletest/reference/assertions.html

CPPREF Depth: GMock — Interface Mocking for Isolated Tests

Google Mock (GMock) allows creating mock classes that override virtual methods with `MOCK_METHOD(ReturnType, Name, (Args), (Qualifiers))`. You can then set expectations: `EXPECT_CALL(mock, method()).Times(1)`. GMock verifies that the expected calls were made. This enables testing code WITHOUT any real external dependencies (network, database, filesystem). google.github.io/googletest/gmock_for_dummies.html

CPPREF Derinlik: GMock — İzole Testler için Arayüz Taklit Etme

Google Mock (GMock), sanal metodları `MOCK_METHOD(DönüşTipi, Ad, (Argümanlar), (Nitelikler))` ile geçersiz kılan taklit sınıflar oluşturmaya olanak tanır. Beklentiler ayarlayabilirsiniz: `EXPECT_CALL(mock, metod()).Times(1)`. GMock beklenen çağrıların yapılıp yapılmadığını doğrular. Bu, gerçek harici bağımlılıklar olmadan (ağ, veritabanı, dosya sistemi) kod test etmeyi mümkün kılar. google.github.io/googletest/gmock_for_dummies.html
@warning EN: Never test your own code by "just running and looking". Tests MUST be automated (CI/CD)! TR: Asla kendi yazdiginiz kodu sadece "calistirarak ve gözle bakarak" test etmeyin. Testler OTOMATIK kosmalidir!

Kaynak Kod / Source Code

```cpp
#include <cmath>
#include <functional>
#include <iostream>
#include <memory>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

// ==========================================================================================
// [GTest SIMULATION FRAMEWORK / GTest SİMÜLASYON ÇATISI]
// ==========================================================================================
// EN: We simulate GTest macros (TEST, EXPECT_EQ, EXPECT_TRUE, EXPECT_THROW, EXPECT_NEAR) without
// needing the actual library.
// TR: Gerçek kütüphane gerekmeden GTest makrolarını simüle ediyoruz.

namespace MiniGTest {

struct TestResult {
  std::string suiteName;
  std::string testName;
  bool passed = true;
  std::vector<std::string> failures;
};

std::vector<TestResult> allResults;
TestResult *currentTest = nullptr;

void registerTest(const std::string &suite, const std::string &name,
                  std::function<void()> fn) {
  allResults.push_back({suite, name, true, {}});
  currentTest = &allResults.back();
  try {
    fn();
  } catch (const std::exception &e) {
    currentTest->passed = false;
    currentTest->failures.push_back(std::string("EXCEPTION: ") + e.what());
  }
}

void expectEq(int expected, int actual, const char *file, int line) {
  if (expected != actual) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_EQ failed: expected=" << expected
        << " actual=" << actual;
    currentTest->failures.push_back(oss.str());
  }
}

void expectTrue(bool cond, const char *file, int line) {
  if (!cond) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_TRUE failed";
    currentTest->failures.push_back(oss.str());
  }
}

void expectFalse(bool cond, const char *file, int line) {
  if (cond) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_FALSE failed";
    currentTest->failures.push_back(oss.str());
  }
}
```

```

void expectNear(double a, double b, double eps, const char *file, int line) {
  if (std::fabs(a - b) > eps) {
    currentTest->passed = false;
    std::ostringstream oss;
    oss << file << ":" << line << " EXPECT_NEAR failed: |" << a << " - " << b
        << "| > " << eps;
    currentTest->failures.push_back(oss.str());
  }
}

void printSummary() {
  int passed = 0;
  int failed = 0;
  for (const auto &r : allResults) {
    std::cout << (r.passed ? "\033[32m[ PASSED ]\033[0m "
                           : "\033[31m[ FAILED ]\033[0m ")
              << r.suiteName << "." << r.testName << std::endl;
    if (!r.passed) {
      for (const auto &f : r.failures) {
        std::cout << "           " << f << std::endl;
      }
      failed++;
    } else {
      passed++;
    }
  }
  std::cout << "\n========================================" << std::endl;
  std::cout << "[ SUMMARY ] " << passed << " passed, " << failed << " failed, "
            << (passed + failed) << " total." << std::endl;
}

} // namespace MiniGTest

// Macros simulating real GTest interface
#define TEST(suite, name)                                                    \
  void suite##_##name();                                                     \
  struct suite##_##name##_reg {                                              \
    suite##_##name##_reg() {                                                 \
      MiniGTest::registerTest(#suite, #name, suite##_##name);                \
    }                                                                        \
  } suite##_##name##_instance;                                               \
  void suite##_##name()

#define EXPECT_EQ(expected, actual)                                          \
  MiniGTest::expectEq(expected, actual, __FILE__, __LINE__)
#define EXPECT_TRUE(cond) MiniGTest::expectTrue(cond, __FILE__, __LINE__)
#define EXPECT_FALSE(cond) MiniGTest::expectFalse(cond, __FILE__, __LINE__)
#define EXPECT_NEAR(a, b, eps)                                              \
  MiniGTest::expectNear(a, b, eps, __FILE__, __LINE__)

#define EXPECT_THROW(stmt, ExType)                                          \
  do {                                                                       \
    bool caught = false;                                                     \
    try {                                                                    \
      stmt;                                                                  \
    } catch (const ExType &) {                                               \
      caught = true;                                                         \
    } catch (...) {                                                          \
    }                                                                        \
    if (!caught) {                                                           \
      MiniGTest::currentTest->passed = false;                                \
      MiniGTest::currentTest->failures.push_back(                            \
          std::string(__FILE__) + ":" + std::to_string(__LINE__) +           \
          " EXPECT_THROW: no " #ExType " thrown");                           \
    }                                                                        \
  } while (0)
```

```cpp

// ==========================================================================================
// [PRODUCTION CODE / ÜRETİM KODU]
// ==========================================================================================

// --- Brake Force Calculator (Automotive) ---
int calculateBrakeForce(int pedalPressure, int brakeWear) {
  int force = pedalPressure - brakeWear;
  return (force < 0) ? 0 : force;
}

// --- Temperature Converter ---
double celsiusToFahrenheit(double c) { return c * 9.0 / 5.0 + 32.0; }

// --- Safe Division with Exception ---
double safeDivide(double a, double b) {
  if (b == 0.0) {
    throw std::invalid_argument("Division by zero!");
  }
  return a / b;
}

// ==========================================================================================
// [DEPENDENCY INJECTION / BAĞIMLILIK ENJEKSİYONU]
// ==========================================================================================
// EN: Interface (abstract class) for a payment gateway. Production code uses RealPaymentGateway;
// tests use MockPaymentGateway.
// TR: Ödeme ağ geçidi için arayüz (soyut sınıf). Üretim kodu RealPaymentGateway kullanır;
// testler MockPaymentGateway kullanır.

class IPaymentGateway {
public:
  virtual ~IPaymentGateway() = default;
  virtual bool processPayment(double amount) = 0;
  virtual double getBalance() const = 0;
};

// EN: Mock implementation for testing — no real payment processing!
// TR: Test için taklit uygulama — gerçek ödeme işlemi yok!
class MockPaymentGateway : public IPaymentGateway {
  double balance_;
  int callCount_ = 0;

public:
  explicit MockPaymentGateway(double balance) : balance_(balance) {}
  bool processPayment(double amount) override {
    callCount_++;
    if (amount > balance_) return false;
    balance_ -= amount;
    return true;
  }
  double getBalance() const override { return balance_; }
  int getCallCount() const { return callCount_; }
};

// EN: The class under test — takes IPaymentGateway via constructor injection.
// TR: Test edilen sınıf — IPaymentGateway'i yapıcı enjeksiyonu ile alır.
class OrderProcessor {
  IPaymentGateway &gateway_;

public:
  explicit OrderProcessor(IPaymentGateway &gw) : gateway_(gw) {}
  bool placeOrder(double price) {
    if (price <= 0) return false;
    return gateway_.processPayment(price);
  }
};
```

```

// ==========================================================================================
// [TEST CASES — Simulating Real GTest TEST() Macros]
// ==========================================================================================

// --- Suite: BrakeForceTest ---
TEST(BrakeForceTest, NormalBraking) {
  EXPECT_EQ(20, calculateBrakeForce(50, 30));
}

TEST(BrakeForceTest, WornBrakesAbsorbFully) {
  EXPECT_EQ(0, calculateBrakeForce(10, 50));
}

TEST(BrakeForceTest, NewBrakePads) {
  EXPECT_EQ(100, calculateBrakeForce(100, 0));
}

TEST(BrakeForceTest, EqualPressureAndWear) {
  EXPECT_EQ(0, calculateBrakeForce(30, 30));
}

// --- Suite: TemperatureTest ---
TEST(TemperatureTest, BoilingPoint) {
  EXPECT_NEAR(212.0, celsiusToFahrenheit(100.0), 0.001);
}

TEST(TemperatureTest, FreezingPoint) {
  EXPECT_NEAR(32.0, celsiusToFahrenheit(0.0), 0.001);
}

TEST(TemperatureTest, BodyTemperature) {
  EXPECT_NEAR(98.6, celsiusToFahrenheit(37.0), 0.1);
}

// --- Suite: DivisionTest ---
TEST(DivisionTest, NormalDivision) {
  EXPECT_NEAR(5.0, safeDivide(10.0, 2.0), 0.001);
}

TEST(DivisionTest, DivisionByZeroThrows) {
  EXPECT_THROW(safeDivide(10.0, 0.0), std::invalid_argument);
}

// --- Suite: PaymentTest (Dependency Injection + Mock) ---
TEST(PaymentTest, SuccessfulOrder) {
  MockPaymentGateway mock(100.0);
  OrderProcessor processor(mock);
  EXPECT_TRUE(processor.placeOrder(30.0));
  EXPECT_NEAR(70.0, mock.getBalance(), 0.001);
}

TEST(PaymentTest, InsufficientFunds) {
  MockPaymentGateway mock(20.0);
  OrderProcessor processor(mock);
  EXPECT_FALSE(processor.placeOrder(50.0));
  EXPECT_NEAR(20.0, mock.getBalance(), 0.001);
}

TEST(PaymentTest, InvalidPrice) {
  MockPaymentGateway mock(100.0);
  OrderProcessor processor(mock);
  EXPECT_FALSE(processor.placeOrder(-10.0));
  EXPECT_EQ(0, mock.getCallCount());
}

TEST(PaymentTest, MockCallCount) {
  MockPaymentGateway mock(1000.0);
  OrderProcessor processor(mock);
  processor.placeOrder(10.0);
  processor.placeOrder(20.0);
  processor.placeOrder(30.0);
  EXPECT_EQ(3, mock.getCallCount());
}

```

```
// ==========================================================================================
// [MAIN — Run All Tests / Tüm Testleri Çalıştır]
// ==========================================================================================
int main() {
  std::cout << "=== MODULE 6: GOOGLE TEST (GTEST) SIMULATION ===\n"
            << std::endl;
  std::cout << "Running " << MiniGTest::allResults.size() << " tests...\n"
            << std::endl;
  MiniGTest::printSummary();
  return 0;
}
```

► 04. Valgrind Profiling

module_06_industry_standards/04_valgrind_profiling.cpp

Konu: Industry Standards: Memory Profiling and Valgrind / Profiling ve Valgrind ile Bellek

Teori / Theory

THEORY: Why Use Valgrind and Profiling?

Writing code that compiles without errors is NOT enough. If your program consumes 1 Megabyte of RAM extra every hour, it will crash your server after a week (OOM - Out of Memory). 'Valgrind' is a magical Linux tool. You run your program "inside" Valgrind, and it tracks EVERY SINGLE BYTE of memory you allocated and checks if you freed it.

TEORİ: Neden Valgrind Kullanılır?

Hata vermeden (0 error) derlenen bir kod yazmak YETERLİ DEĞİLDİR! C++ programınız her saat fazladan 1 MB RAM tüketirse (Memory Leak - Bellek Sızıntısı), o program 1 hafta sonra sunucuyu patlatır (Out of Memory - OOM).
Linux tarafında (Kurumsal firmalarda %99 Unix tabanlı iş yapılır) "Valgrind" isimli inanılmaz bir dedektif programı vardır. C++ executable (çalıştırılabilir) dosyanızı normal `./main` diye çalıştırmak yerine `valgrind ./main` diye çalıştırırsınız. Valgrind, programınızın etrafını sarı şeritlerle çevirir ve istediğiniz HER BİR BAYT belleği not alır. Program kapanırken "Sen işletim sisteminden şu kadar bellek aldın ama şu kadarını geri iade etmemişsin, RAM'in kanayan yeri şurasıdır!" der.

HOW TO APPLY? / NASIL UYGULANIR?

Add the "-g" flag while compiling to inject line numbers into the binary.

Derlerken (Compile) kod satır numaralarını içine gömmek için "-g" bayrağı eklenir:

--leak-check=full ./leak_test`

CPPREF Depth: Sanitizer Internals — How ASan, TSan, UBSan Actually Work

1. Compile: `g++ -g -o leak_test 04_valgrind_profiling.cpp` 2. Execute: `valgrind

ASan (AddressSanitizer) uses shadow memory with a 1:8 mapping — every 8 bytes of application memory have 1 byte of shadow tracking whether that region is fully accessible, partially accessible, or poisoned. Typical slowdown is ~2x with ~3x memory overhead. TSan (ThreadSanitizer) uses happens-before analysis backed by vector clocks to detect data races; it incurs ~5-15x slowdown and 5-10x memory overhead. UBSan (UndefinedBehaviorSanitizer) instruments arithmetic and type operations at compile time (signed overflow, null dereference, misaligned access) with minimal runtime overhead. ASan and TSan CANNOT be combined in the same binary — they use incompatible shadow memory layouts. MSan (MemorySanitizer) detects reads of uninitialized memory; also incompatible with ASan. In CI pipelines, run separate builds for each sanitizer: one ASan+UBSan build and one TSan build.

CPPREF Derinlik: Sanitizer İç Yapısı — ASan, TSan, UBSan Nasıl Çalışır?

ASan (AddressSanitizer) 1:8 eşlemeli gölge bellek kullanır — uygulama belleğinin her 8 baytı için 1 bayt gölge, o bölgenin tamamen erişilebilir, kısmen erişilebilir veya zehirlenmiş olduğunu izler. Tipik yavaşlama ~2x, bellek ek yükü ~3x'tir. TSan (ThreadSanitizer) veri yarışlarını tespit etmek için vektör saatleriyle desteklenen happens-before analizi kullanır; ~5-15x yavaşlama ve 5-10x bellek ek yükü getirir. UBSan (UndefinedBehavior Sanitizer) aritmetik ve tür işlemlerini derleme zamanında enstrümante eder (işaretli taşma, null referans, hizasız erişim) — minimal çalışma zamanı ek yükü vardır. ASan ve TSan aynı binary'de BİRLEŞTİRİLEMEZ — uyumsuz gölge bellek düzenleri kullanırlar. MSan (MemorySanitizer) başlatılmamış bellek okumalarını tespit eder; o da ASan ile uyumsuzdur. CI pipeline'larında her sanitizer için ayrı derleme çalıştırın: bir ASan+UBSan, bir TSan.

Kaynak Kod / Source Code

```cpp
#include <cstring>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ═══════════════════════════════════════════════════════════════════════════════
// EN: AddressSanitizer (ASan) — Demonstrates what ASan targets.
// TR: AddressSanitizer (ASan) — ASan'ın tespit ettiği hataları gösterir.
// ═══════════════════════════════════════════════════════════════════════════════
// EN: ASan is a compile-time instrumentation flag: -fsanitize=address. It detects
// heap-buffer-overflow, stack-buffer-overflow, use-after-free, and memory leaks at ~2x runtime
// cost (vs Valgrind's ~20x slowdown). Compile: g++ -g -fsanitize=address -o asan_test file.cpp
// TR: ASan, derleme zamanı enstrümantasyon bayrağıdır: -fsanitize=address. Heap tampon taşması,
// stack tampon taşması, serbest bırakılmış bellek erişimi ve bellek sızıntılarını ~2x çalışma
// maliyetiyle tespit eder (Valgrind'in ~20x yavaşlamasına karşılık). Derleme: g++ -g
// -fsanitize=address -o asan_test dosya.cpp
void demonstrateASanTargets() {
  std::cout << "\n--- ASan Target Patterns (Commented — triggering would crash!) ---\n";

  // ─── Heap-Buffer-Overflow ──────────────────────────────────────────────────
  // EN: Writing past allocated heap memory. ASan catches this immediately.
  // TR: Tahsis edilen heap belleğinin ötesine yazma. ASan bunu anında yakalar. DANGEROUS — DO
  // NOT UNCOMMENT: int *buf = new int[10]; buf[10] = 0xDEAD;  // heap-buffer-overflow! delete[]
  // buf;
  std::cout << "[ASan] Heap-buffer-overflow: writing buf[10] on a 10-element array\n";
```

```cpp

  // ─── Stack-Buffer-Overflow ──────────────────────────────────────────────────
  // EN: Overrunning a local stack array. ASan instruments stack frames too.
  // TR: Yerel stack dizisini taşırma. ASan stack çerçevelerini de izler. DANGEROUS — DO NOT
  // UNCOMMENT: int stack_arr[5]; stack_arr[5] = 42;  // stack-buffer-overflow!
  std::cout << "[ASan] Stack-buffer-overflow: stack_arr[5] on a 5-element array\n";

  // ─── Use-After-Free ─────────────────────────────────────────────────────────
  // EN: Accessing memory after delete. Classic dangling pointer.
  // TR: Delete sonrası belleğe erişim. Klasik sarkan işaretçi. DANGEROUS — DO NOT UNCOMMENT: int
  // *p = new int(7); delete p; *p = 99;  // use-after-free!
  std::cout << "[ASan] Use-after-free: dereferencing deleted pointer\n";
}

// ═══════════════════════════════════════════════════════════════════════════════
// EN: ThreadSanitizer (TSan) — Data Race Detection
// TR: ThreadSanitizer (TSan) — Veri Yarışı Tespiti
// ═══════════════════════════════════════════════════════════════════════════════
// EN: TSan is enabled with -fsanitize=thread. It detects data races where two threads access the
// same memory without synchronization and at least one is a write. CANNOT be used simultaneously
// with ASan. Compile: g++ -g -fsanitize=thread -o tsan_test file.cpp
// TR: TSan, -fsanitize=thread ile etkinleştirilir. İki iş parçacığının senkronizasyon olmadan
// aynı belleğe eriştiği ve en az birinin yazma yaptığı veri yarışlarını tespit eder. ASan ile
// aynı anda KULLANILAMAZ. Derleme: g++ -g -fsanitize=thread -o tsan_test dosya.cpp
//
// EN: Example data race pattern (DO NOT run without TSan):
// TR: Örnek veri yarışı deseni (TSan olmadan ÇALIŞTIRMAYIN): int sharedCounter = 0; // Thread 1:
// sharedCounter++;  // WRITE — no lock! // Thread 2:  sharedCounter++;  // WRITE — no lock! =>
// DATA RACE
//
// EN: Fix: protect with std::mutex:
// TR: Çözüm: std::mutex ile koruma: std::mutex mtx; // Thread 1:  { std::lock_guard<std::mutex>
// lk(mtx); sharedCounter++; } // Thread 2:  { std::lock_guard<std::mutex> lk(mtx);
// sharedCounter++; }

// ═══════════════════════════════════════════════════════════════════════════════
// EN: UndefinedBehaviorSanitizer (UBSan) — Catches Silent Killers
// TR: UndefinedBehaviorSanitizer (UBSan) — Sessiz Katilleri Yakalar
// ═══════════════════════════════════════════════════════════════════════════════
// EN: UBSan is enabled with -fsanitize=undefined. It catches signed integer overflow, null
// pointer dereference, misaligned access, and more. These are "undefined behavior" — compiler
// may do ANYTHING. Compile: g++ -g -fsanitize=undefined -o ubsan_test file.cpp
// TR: UBSan, -fsanitize=undefined ile etkinleştirilir. İşaretli tamsayı taşması, null işaretçi
// referansı, hizalama hataları vb. yakalar. Bunlar "tanımsız davranış"tır — derleyici HERHANGİ
// bir şey yapabilir. Derleme: g++ -g -fsanitize=undefined -o ubsan_test dosya.cpp
int potentialSignedOverflow(int a, int b) {
  // EN: If a + b exceeds INT_MAX, this is undefined behavior. UBSan will report: "signed integer
  // overflow: X + Y".
  // TR: Eğer a + b, INT_MAX'ı aşarsa bu tanımsız davranıştır. UBSan şunu raporlar: "signed
  // integer overflow: X + Y".
  return a + b;
}

// ═══════════════════════════════════════════════════════════════════════════════
// EN: RAII Approach — Valgrind Reports 0 Leaks
// TR: RAII Yaklaşımı — Valgrind 0 Sızıntı Raporlar
// ═══════════════════════════════════════════════════════════════════════════════
void properRAII_VectorApproach() {
  // EN: std::vector manages heap memory automatically. When the function returns, the vector
  // destructor frees all memory — zero leaks.
  // TR: std::vector heap belleği otomatik yönetir. Fonksiyon döndüğünde vector yıkıcısı tüm
  // belleği serbest bırakır — sıfır sızıntı.
  std::vector<int> sensorBuffer(1000000);
  sensorBuffer[0] = 42;
  std::cout << "[RAII Vector] Data loaded: " << sensorBuffer[0]
            << " — memory auto-freed on scope exit" << std::endl;
}
```

```cpp

void properRAII_UniquePtrApproach() {
  // EN: std::unique_ptr owns the resource exclusively. Automatically calls delete[] when it goes
  // out of scope — impossible to leak.
  // TR: std::unique_ptr kaynağa münhasır sahiptir. Kapsam dışına çıkınca otomatik delete[]
  // çağırır — sızıntı imkansız.
  auto canBuffer = std::make_unique<int[]>(1000000);
  canBuffer[0] = 0xCAFE;
  std::cout << "[RAII unique_ptr] CAN frame loaded: 0x" << std::hex
            << canBuffer[0] << std::dec
            << " — unique_ptr frees on scope exit" << std::endl;
}

void theLeakyFunction() {
  // EN: Allocating 1 million integers (approx 4 MB of RAM).
  // TR: 1 Milyon tam sayı (yaklasık 4 MB RAM tüketimi) istiyoruz.
  int *databaseSimulation = new int[1000000];

  databaseSimulation[0] = 42;
  std::cout << "[Valgrind Test] Data loaded (Veri Yüklendi): " << databaseSimulation[0] <<
      std::endl;

  // EN: OH NO! We exit the function WITHOUT calling 'delete[]'! This is a disaster. When the
  // function ends, the pointer 'databaseSimulation' vanishes, but the 4MB memory it grabbed
  // remains occupied forever as a ZOMBIE.
  //
  // TR: AMAN TANRIM! DELETE YAPMADAN FONKSİYONDAN CIKIYORUZ! İşte burası bir felakettir. Zira
  // fonksiyon kapandığında 'databaseSimulation' isimli işaretçi YOK OLUR ama RAM'de ayırdığı o
  // devasa 4MB alan sahipsiz (Zombi) şekilde sistem yeniden başlayana kadar havada kalır!
  // (MEMORY LEAK).
}

int main() {
  std::cout << "=== MODULE 6: VALGRIND & MEMORY PROFILING ===\n" << std::endl;

  std::cout << "[SYSTEM] WARNING: Triggering 4MB Memory Leak intentionally...\n";
  std::cout << "[SYSTEM] DİKKAT: Bilerek 4MB Bellek sızıntısı tetikleniyor...\n";

  theLeakyFunction();

  std::cout << "\n[SYSTEM] Function ended. But 4 Megabytes of garbage remains in RAM." << std::endl;
  std::cout << "[SYSTEM] Fonksiyon bitti ama 4MB çöp kaldı. Bunu bulan "
               "dedektif Valgrind aracıdır!"
            << std::endl;

  // ==========================================================================================
  // EN: How to fix this in Modern C++? NEVER use RAW POINTERS inside 'theLeakyFunction'. Use
  // `std::vector<int>` or `std::unique_ptr` instead! When they go out of scope, they instantly
  // wipe the 4MB data (RAII rules supreme).
  //
  // TR: Modern C++ ile Nasıl Düzeltilir? 'theLeakyFunction' içerisinde ASLA ÇIPLAK İŞARETÇİ
  // KULLANMA. `std::vector<int>` veya akıllı işaretçi kullan. Cünkü std::vector kapsam dışına
  // cıktığında içindeki o 4MB datayı saniyesinde sisteme iade eder! (RAII)
  // ==========================================================================================

  // ==========================================================================================
  // EN: Demo 2 — ASan Target Patterns
  // TR: Demo 2 — ASan Hedef Desenleri
  // ==========================================================================================
  std::cout << "\n--- Demo 2: AddressSanitizer (ASan) Targets ---" << std::endl;
  demonstrateASanTargets();
```

```cpp

    // ==========================================================================
    // EN: Demo 3 — Proper RAII: Vector vs Raw Pointer
    // TR: Demo 3 — Doğru RAII: Vector vs Ham İşaretçi
    // ==========================================================================
    std::cout << "\n--- Demo 3: RAII Vector Approach (0 Leaks Under Valgrind) ---" << std::endl;
    properRAII_VectorApproach();

    // ==========================================================================
    // EN: Demo 4 — unique_ptr Automatic Cleanup
    // TR: Demo 4 — unique_ptr Otomatik Temizlik
    // ==========================================================================
    std::cout << "\n--- Demo 4: unique_ptr Automatic Cleanup ---" << std::endl;
    properRAII_UniquePtrApproach();

    // ==========================================================================
    // EN: Demo 5 — UBSan: Signed Overflow Example
    // TR: Demo 5 — UBSan: İşaretli Taşma Örneği
    // ==========================================================================
    std::cout << "\n--- Demo 5: UBSan Signed Overflow ---" << std::endl;
    // EN: Safe call — no overflow here. Compile with -fsanitize=undefined to instrument and catch
    // overflows automatically.
    // TR: Güvenli çağrı — burada taşma yok. Taşmaları otomatik yakalamak için -fsanitize=undefined
    // ile derleyin.
    int safeResult = potentialSignedOverflow(100, 200);
    std::cout << "100 + 200 = " << safeResult << " (safe, no overflow)" << std::endl;
    std::cout << "Compile with -fsanitize=undefined to catch cases like INT_MAX + 1\n";

    // ==========================================================================
    // EN: Demo 6 — Sanitizer & Profiling Compilation Flags Reference
    // TR: Demo 6 — Sanitizer & Profiling Derleme Bayrakları Referansı
    // ==========================================================================
    std::cout << "\n--- Demo 6: Sanitizer Compilation Flags Reference ---" << std::endl;
    std::cout << R"(
    +------+-------------------------------+-------------------------------------+
    | Tool | Compile Flag                  | Detects                             |
    +------+-------------------------------+-------------------------------------+
    | ASan | -fsanitize=address            | Buffer overflow, use-after-free, leak|
    | TSan | -fsanitize=thread             | Data races between threads          |
    | UBSan| -fsanitize=undefined          | Signed overflow, null deref, align  |
    | MSan | -fsanitize=memory (clang)     | Uninitialized memory reads          |
    +------+-------------------------------+-------------------------------------+
    | Valgrind | valgrind --leak-check=full ./bin | All leaks, ~20x slower      |
    | Perf     | perf record ./bin; perf report   | CPU hotspots, cache misses  |
    | Callgrind| valgrind --tool=callgrind ./bin  | Function-level profiling    |
    +----------+-------------------------------+----------------------------+
  )" << std::endl;

    // ==========================================================================
    // EN: Practical Profiling Workflow (Automotive ECU Software)
    // TR: Pratik Profiling İş Akışı (Otomotiv ECU Yazılımı)
    // ==========================================================================
    // EN: Step 1 — Compile with debug symbols: g++ -g -O0 -o ecu_test main.cpp
    // TR: Adım 1 — Debug sembolleriyle derle: g++ -g -O0 -o ecu_test main.cpp
    //
    // EN: Step 2 — Run under Valgrind Memcheck for full leak report:
    // TR: Adım 2 — Tam sızıntı raporu için Valgrind Memcheck altında çalıştır: valgrind
    // --leak-check=full --show-leak-kinds=all ./ecu_test
    //
    // EN: Step 3 — Compile with ASan for faster turnaround (~2x vs 20x):
    // TR: Adım 3 — Daha hızlı geri dönüş için ASan ile derle (~2x vs 20x): g++ -g
    // -fsanitize=address -o ecu_test main.cpp && ./ecu_test
    //
    // EN: Step 4 — Profile CPU hotspots with perf:
    // TR: Adım 4 — CPU darboğazlarını perf ile profilleyin: perf record -g ./ecu_test && perf
    // report
    //
    // EN: Step 5 — Function-level profiling with Callgrind:
    // TR: Adım 5 — Fonksiyon düzeyinde profil için Callgrind: valgrind --tool=callgrind ./ecu_test
    // kcachegrind callgrind.out.*   (GUI visualization)

    return 0;
  }
```

► 05. Async Futures

module_06_industry_standards/05_async_futures.cpp

Konu: Industry Standards / Asynchronous Code: std::async, std::future, std::promise,

Teori / Theory

THEORY: Why std::async instead of std::thread?

When you launch `std::thread`, it runs completely parallel, but it does NOT easily RETURN a value back to the main thread. Also, managing raw threads can be messy. `std::async` is a higher-level tool. It runs a task in the background and returns a "TICKET" called `std::future`. You can continue doing your work on the main thread, and when you finally need the answer, you cash in the ticket (`ticket.get()`).

TEORİ: std::async vs std::thread

`std::thread` ile boyut açtığımızda harika bir şekilde paralel çalışır (ANCAK), işi bitirdiğinde Ana Thread'e (Main) "Sonucu (Return değeri)" geri yollaması büyük bir eziyettir. C++11 ile `std::async` geldi.
`std::async` arka planda bir thread acar, işi o thread'e verir ve sana garantör olarak bir "Fiş / Bilet / Kupon" (`std::future`) uzatır. "Sen git kendi isine bak, işlemler bitince bana bu biletle geleceksin ve ben sana hesapladığım değeri geri vereceğim." der. Eğer sen bileti tahsil etmeye `ticket.get()` ile gidersen ve arka plandaki is HENÜZ BİTMEMİŞSE, Main Thread o noktada işlemi beklemeye (BLOKE OLMAYA) gecer!

THEORY: std::promise — The Manual Push Channel

`std::async` automatically decides which thread runs the work and delivers the result. But sometimes YOU want to control when and what value gets pushed. `std::promise<T>` is a write-end channel: you call `promise.set_value(42)` from any thread, and the paired `std::future<T>` (obtained via `promise.get_future()`) receives it. Think of it as a one-shot mailbox: the sender drops a letter, the receiver picks it up.

TEORİ: std::promise — Manuel Değer İtme Kanalı

`std::async` işi otomatik planlar. Ancak bazen HANGİ değerin, HANGİ anda gönderileceğini SİZ kontrol etmek istersiniz. `std::promise<T>` bir "yazma ucu" kanalıdır. Herhangi bir thread'den `promise.set_value(42)` dersiniz, eşleştirilmiş `std::future<T>` (promise.get_future() ile alınan) o değeri alır. Tek kullanımlık posta kutusu gibidir: gönderici mektubu atar, alıcı kutudan çeker.

THEORY: std::packaged_task — Callable Wrapper with Future

`std::packaged_task<R(Args...)>` wraps ANY callable (function, lambda, functor) and automatically connects it to a `std::future`. When you invoke the packaged_task (via `operator()`), the result is stored in the future. Unlike `std::async`, YOU decide exactly when and on which thread the task executes — giving you full scheduling control (perfect for thread pools!).

TEORİ: std::packaged_task — Future Bağlantılı Çağrılabilir Sarmalayıcı

`std::packaged_task<R(Args...)>` herhangi bir çağrılabilir nesneyi (fonksiyon, lambda, functor) sarar ve otomatik olarak bir `std::future` ile eşleştirir. packaged_task'ı `operator()` ile çağırdığınızda sonuç future'a yazılır. `std::async`'den farkı: görevi NE ZAMAN ve HANGİ thread'de çalıştıracağınıza SİZ karar verirsiniz — Thread Pool mimarisi için ideal!

THEORY: std::shared_future — Multiple Readers

`std::future::get()` can only be called ONCE (the ticket is torn). `std::shared_future<T>` allows MULTIPLE threads to call `.get()` and each receives a copy of the result. Created from `future.share()`. Useful when a single computation feeds many consumers (e.g., config loading shared by 10 worker threads).

TEORİ: std::shared_future — Çoklu Okuyucu

`std::future::get()` yalnızca BİR KEZ çağrılabilir (bilet yırtılır). `std::shared_future<T>` ise BİRDEN FAZLA thread'in `.get()` çağırmasına izin verir ve her biri sonucun bir kopyasını alır. `future.share()` ile oluşturulur. Tek bir hesaplamanın birçok tüketiciye dağıtılması gerektiğinde kullanılır (örneğin, 10 işçi thread'e paylaşılan config yüklemesi).

CPPREF Depth: The Silent Block of `std::future` Destructor

A massive trap! If you use `std::async(std::launch::async, ...)` but don't assign the returned ticket to a variable, the temporary `std::future` dies immediately at the semicolon. Its Destructor WILL BLOCK THE MAIN THREAD until the async task finishes! So it completely defeats the purpose of async!

CPPREF Derinlik: `std::future` Yıkıcısının Sessiz Bloğu

Devasa bir mülakat tuzağı! Eğer `std::async(std::launch::async, ...)` ile asenkron iş başlatır ama dönen o Bileti (future) bir değişkene atamazsanız, isimsiz geçici future nesnesi o satır sonunda anında ölür. CppRef kurallarına göre o Future'ın Destructor'ı, arka plandaki iş bitene kadar ANA THREAD'İ KİLİTLER VE BEKLER! Yani asenkron yaptığınızı sanarken kodu tamamen senkronize bloke etmiş olursunuz!

CPPREF Depth: std::promise::set_exception — Error Propagation Across Threads

If a worker thread encounters an error, it can call `promise.set_exception(std::current_exception())`. The consumer thread calling `future.get()` will then receive the exception as if it was thrown locally. This is the ONLY safe way to propagate exceptions across thread boundaries. cppreference.com/w/cpp/thread/promise/set_exception

CPPREF Derinlik: std::promise::set_exception — Thread'ler Arası Hata İletimi

Eğer işçi thread bir hatayla karşılaşırsa, `promise.set_exception(std::current_exception())` çağırabilir. Tüketici thread `future.get()` çağırdığında, istisna sanki yerel olarak fırlatılmış gibi alınır. Bu, thread sınırları arasında istisna iletmenin TEK güvenli yoludur. cppreference.com/w/cpp/thread/promise/set_exception

CPPREF Depth: std::packaged_task is MoveOnly — No Copying!

`std::packaged_task` is move-only, just like `std::unique_ptr`. You cannot copy it. To pass it into a `std::thread`, you MUST use `std::move(task)`. Attempting to copy triggers a deleted copy-constructor error at compile time. cppreference.com/w/cpp/thread/packaged_task

CPPREF Derinlik: std::packaged_task MoveOnly'dir — Kopyalanamaz!

`std::packaged_task`, tıpkı `std::unique_ptr` gibi yalnızca taşınabilir (move-only). Kopyalanamaz. Bir `std::thread`'e geçirmek için `std::move(task)` kullanmanız ZORUNLUDUR. Kopyalama girişimi derleme zamanında silinmiş kopya yapıcı hatası verir. cppreference.com/w/cpp/thread/packaged_task

Kaynak Kod / Source Code

```cpp
#include <chrono> // EN: Time simulations
#include <functional>
#include <future>
#include <iostream>
#include <thread>
#include <vector>

/**
 * @brief EN: Heavy function simulating a network/database request.
 * TR: Ağdan / veritabanından veri ceker gibi bekleyen agir fonksiyon.
 * @return 200 (OK status) after 3 seconds.
 */
int heavyServerRequest() {
  std::cout << "[WORKER THREAD] Accessing Server... ETA: 3 Seconds (Sunucuya "
               "erisiliyor...)"
            << std::endl;
  std::this_thread::sleep_for(std::chrono::seconds(3));

  std::cout << "[WORKER THREAD] Server Responded / Sunucu yanit verdi: [OK - 200]" << std::endl;

  // EN: We are actually RETURNING a value! You can't easily do this with std::thread.
  // TR: Dikkat, değer DONDURUYORUZ! std::thread ile bunu basitce yakalayamazsiniz.
  return 200;
}

// ==========================================================================================
// [DEMO 2: std::promise - Manual Value Push / Elle Değer İtme]
// ==========================================================================================
/**
 * @brief EN: Worker receives a promise and pushes the result manually.
 * TR: İşçi bir promise alır ve sonucu elle gönderir.
 */
void workerWithPromise(std::promise<std::string> prom) {
  std::cout << "[PROMISE WORKER] Computing secret key..." << std::endl;
  std::this_thread::sleep_for(std::chrono::seconds(1));
  // EN: Push the value into the channel — the future on the other side wakes.
  // TR: Değeri kanala it — diğer uçtaki future uyanır.
  prom.set_value("SECRET-KEY-XJ9-ALPHA");
}

// ==========================================================================================
// [DEMO 3: std::packaged_task - Deferred Callable / Ertelenmiş Görev]
// ==========================================================================================
/**
 * @brief EN: A pure function to be wrapped in packaged_task.
 * TR: packaged_task içine sarılacak saf fonksiyon.
 */
int computeChecksum(int a, int b) {
  std::this_thread::sleep_for(std::chrono::milliseconds(500));
  return (a * 31) ^ (b * 17);
}

int main() {
  std::cout << "=== MODULE 6: ADVANCED CONCURRENCY (ASYNC / FUTURE / "
               "PROMISE / PACKAGED_TASK / SHARED_FUTURE) ===\n"
            << std::endl;

  // ==========================================================================================
  // PART 1: std::async + std::future (Original Demo)
  // ==========================================================================================
  std::cout << "--- PART 1: std::async + std::future ---\n" << std::endl;

  auto startTime = std::chrono::high_resolution_clock::now();
```

```cpp

    // EN: std::launch::async -> Forces it to run strictly on a new separate thread. "Work in the
    // background and give me a ticket!"
    // TR: "Arka planda çalış, bana bilet (future) fırlat!" std::launch::async -> Kesinlikle yeni
    // bir çekirdekte çalıştır demektir.
    std::future<int> ticket = std::async(std::launch::async, heavyServerRequest);

    // EN: Main Thread continues working!
    // TR: Ana döngü bos durmuyor!
    std::cout << "\n[MAIN] Task sent asynchronously! Meanwhile, UI is rendering "
                 "frames..."
              << std::endl;

    for (int i = 1; i <= 3; i++) {
      std::cout << "[MAIN] Drawing Frame (Kare cizimi...) " << i << std::endl;
      std::this_thread::sleep_for(std::chrono::seconds(1));
    }

    // EN: Once `.get()` is called: if the heavy code isn't finished yet, MAIN FREEZES and WAITS!
    // TR: ".get()" cagirildiginda eğer ağır iş henuz isini bitirmemisse, ANA PROGRAM DONAR!
    int result = ticket.get();

    auto endTime = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> totalTime = endTime - startTime;

    std::cout << "\n--- ASYNC RESULTS / SONUCLAR ---" << std::endl;
    std::cout << "Received Server Code (Gelen Kod): " << result << std::endl;
    std::cout << "Total Elapsed Time: " << totalTime.count() << " seconds.\n"
              << std::endl;

    // ==========================================================================================
    // PART 2: std::promise + std::future (Manual Push)
    // ==========================================================================================
    // EN: Create a promise-future pair. The promise is the "write end", the future is the "read
    // end". We send the promise to a worker thread.
    // TR: Bir promise-future çifti oluştur. Promise "yazma ucu", future "okuma ucu". Promise'i
    // işçi thread'e gönderiyoruz.
    std::cout << "--- PART 2: std::promise (Manual Channel) ---\n" << std::endl;

    std::promise<std::string> secretPromise;
    std::future<std::string> secretFuture = secretPromise.get_future();

    // EN: We MUST move the promise into the thread (it's move-only).
    // TR: Promise taşınabilir-only; std::move ZORUNLU.
    std::thread promThread(workerWithPromise, std::move(secretPromise));

    std::cout << "[MAIN] Waiting for secret key from worker..." << std::endl;
    std::string key = secretFuture.get(); // Blocks until set_value()
    std::cout << "[MAIN] Received key: " << key << "\n" << std::endl;

    promThread.join();

    // ==========================================================================================
    // PART 3: std::packaged_task (Deferred Execution)
    // ==========================================================================================
    // EN: Wrap computeChecksum in a packaged_task. We get the future BEFORE running the task, then
    // move the task into a thread.
    // TR: computeChecksum'u packaged_task ile sar. Görevi çalıştırmadan ÖNCE future'ı al, ardından
    // görevi thread'e taşı.
    std::cout << "--- PART 3: std::packaged_task (Deferred Callable) ---\n"
              << std::endl;

    std::packaged_task<int(int, int)> task(computeChecksum);
    std::future<int> checksumFuture = task.get_future();
```

```cpp

    // EN: Move the task into a thread (packaged_task is move-only like unique_ptr).
    // TR: Görevi thread'e taşı (packaged_task unique_ptr gibi move-only).
    std::thread taskThread(std::move(task), 42, 99);

    std::cout << "[MAIN] packaged_task dispatched. Doing other work..." << std::endl;
    int checksum = checksumFuture.get(); // Blocks until task completes
    std::cout << "[MAIN] Checksum result: " << checksum << "\n" << std::endl;

    taskThread.join();

    // ==========================================================================================
    // PART 4: std::shared_future (Multiple Readers)
    // ==========================================================================================
    // EN: One computation, many consumers. shared_future allows multiple .get() calls from
    // different threads.
    // TR: Tek hesaplama, çok tüketici. shared_future birden fazla thread'in .get() çağırmasına
    // izin verir.
    std::cout << "--- PART 4: std::shared_future (Multiple Readers) ---\n"
              << std::endl;

    std::shared_future<int> sharedResult =
        std::async(std::launch::async, []() -> int {
          std::cout << "[SHARED WORKER] Loading config..." << std::endl;
          std::this_thread::sleep_for(std::chrono::milliseconds(500));
          // EN: Config value
          // TR: Ayar değeri
          return 9999;
        // EN: .share() converts future -> shared_future
        // TR: .share() future'ı shared_future'a dönüştürür
        }).share();

    // EN: Multiple threads read the same result simultaneously.
    // TR: Birden fazla thread aynı sonucu eşzamanlı okur.
    std::vector<std::thread> readers;
    for (int i = 0; i < 3; i++) {
      readers.emplace_back([sharedResult, i]() {
        int val = sharedResult.get(); // All 3 threads can call get()!
        std::cout << "[READER " << i << "] Config loaded: " << val << std::endl;
      });
    }

    for (auto &r : readers) {
      r.join();
    }

    std::cout << "\n=> std::shared_future allows MULTIPLE .get() calls — unlike "
                 "std::future which tears the ticket on first use!"
              << std::endl;

    return 0;
  }
```

► 06. Debugging Techniques

module_06_industry_standards/06_debugging_techniques.cpp

Konu: Industry Standards: Debugging Techniques & Defensive Programming / Endüstri

Teori / Theory

THEORY: Debugging, Assertions & Defensive Programming

1. GDB Basics / GDB Temelleri

GDB is the GNU Debugger — the primary debugging tool on Linux. Compile with `-g` to include debug symbols: g++ -g -std=c++17 -o prog 06_debugging_techniques.cpp Key commands: break main — set breakpoint at main() run — start execution next (n) — step over to next line step (s) — step into function call print var — inspect variable value backtrace (bt)— show call stack watch var — break when variable changes (watchpoint)

TEORİ : Hata Ayıklama, Onaylamalar ve Savunmacı Programlama

GDB, GNU Hata Ayıklayıcısıdır — Linux'ta temel hata ayıklama aracıdır. Hata ayıklama sembollerini eklemek için `-g` ile derleyin: g++ -g -std=c++17 -o prog 06_debugging_techniques.cpp Temel komutlar: break main — main()'de kırılma noktası koy run — çalıştırmayı başlat next (n) — sonraki satıra geç step (s) — fonksiyon çağrısına gir print var — değişken değerini incele backtrace (bt)— çağrı yığınını göster watch var — değişken değişince dur (izleme noktası)

2. Core Dumps

Enable core dumps: `ulimit -c unlimited` When a program crashes (SIGSEGV, SIGABRT), the OS writes a core file. Analyze: `gdb ./program core` — see exact crash location and variables.

Çekirdek Dökümleri

Çekirdek dökümünü etkinleştir: `ulimit -c unlimited` Program çöktüğünde (SIGSEGV, SIGABRT), işletim sistemi bir core dosyası yazar. Analiz: `gdb ./program core` — tam çökme noktasını ve değişkenleri gör.

3. Static Analysis & Sanitizers / Statik Analiz ve Temizleyiciler

Compiler warnings (-Wall -Wextra -Wpedantic) are the first line of defense. Address/Thread/UB sanitizers catch bugs at runtime: -fsanitize=address — buffer overflow, use-after-free -fsanitize=undefined — signed overflow, null deref, alignment -fsanitize=thread — data races in multithreaded code

Derleyici uyarıları (-Wall -Wextra -Wpedantic) ilk savunma hattıdır. Adres/İş parçacığı/TB temizleyicileri çalışma zamanında hata yakalar: -fsanitize=address — tampon taşması, serbest bırakılmış kullanımı -fsanitize=undefined — işaretli taşma, null ref, hizalama -fsanitize=thread — çok iş parçacıklı kodda veri yarışları

4. assert() vs static_assert()

assert(expr) — runtime check; aborts if expr is false. Disabled by defining NDEBUG. static_assert(expr, msg) — compile-time check; zero runtime cost. Use for type traits, sizes, alignment guarantees.

assert(ifade) — çalışma zamanı kontrolü; ifade false ise iptal eder. NDEBUG tanımlanarak devre dışı bırakılır. static_assert(ifade, msg) — derleme zamanı kontrolü; çalışma zamanı maliyeti sıfır. Tip özellikleri, boyutlar, hizalama garantileri için kullanılır.

5. Defensive Programming

Preconditions — validate inputs before use. Postconditions — verify outputs before returning. Invariants — maintain class state consistency across all methods.

Savunmacı Programlama

Ön koşullar — kullanmadan önce girdileri doğrula. Son koşullar — dönmeden önce çıktıları doğrula. Değişmezler — tüm yöntemlerde sınıf durum tutarlılığını koru.

[nodiscard]] — çağıranları dönüş değerini kontrol etmeye zorla

6. Logging Levels

TRACE — finest granularity, every function entry/exit DEBUG — diagnostic information for developers INFO — general operational messages WARN — potentially harmful situations ERROR — error events, program can continue FATAL — severe error, program must abort

Günlük Seviyeleri

TRACE — en ince ayrıntı, her fonksiyon giriş/çıkış DEBUG — geliştiriciler için tanılama bilgisi INFO — genel operasyonel mesajlar WARN — potansiyel zararlı durumlar ERROR — hata olayları, program devam edebilir FATAL — ağır hata, program iptal etmeli
Standart: C++17 Derleme : g++ -std=c++17 -Wall -Wextra -Wpedantic 06_debugging_techniques.cpp

CPPREF Depth: GDB Advanced — Conditional Breakpoints, Watchpoints, and Reverse Debugging

`break func if x > 100` sets a conditional breakpoint — execution stops only when the condition is true, invaluable for debugging iteration N of a million-iteration loop. `watch variable` sets a hardware watchpoint; the CPU halts when the watched memory location changes value. `rwatch` triggers on read access, `awatch` on any access. GDB reverse debugging: `record` starts recording, then `reverse-next`, `reverse-step`, and `reverse-continue` replay execution backwards — you can step BACK to find where a variable was corrupted. `gdb -tui` launches a terminal UI with source code pane. Core dump analysis: `thread apply all bt` prints all thread backtraces — essential for multithreaded crash investigation. Python scripting inside GDB lets you write custom pretty-printers for complex types (e.g., printing a red-black tree as a sorted list). `set pagination off` and `set logging on` for scripted batch debugging.

CPPREF Derinlik: İleri GDB — Koşullu Kırılma Noktaları, İzleme Noktaları ve Geri Hata Ayıklama

`break func if x > 100` koşullu kırılma noktası koyar — yürütme yalnızca koşul doğru olduğunda durur; milyon iterasyonlu bir döngüde N. iterasyonu hata ayıklamak için paha biçilmezdir. `watch variable` donanım izleme noktası koyar; izlenen bellek konumunun değeri değiştiğinde CPU durur. `rwatch` okuma erişiminde, `awatch` herhangi bir erişimde tetiklenir. GDB geri hata ayıklama: `record` kaydı başlatır, ardından `reverse-next`, `reverse-step` ve `reverse-continue` yürütmeyi GERİYE doğru oynatır — bir değişkenin nerede bozulduğunu bulmak için GERİ adım atabilirsiniz. `gdb -tui` kaynak kodlu terminal arayüzü başlatır. Çekirdek dökümü analizi: `thread apply all bt` tüm iş parçacığı geri izlerini yazdırır — çok iş parçacıklı çökme araştırması için zorunludur. GDB içinde Python betikleme, karmaşık tipler için özel pretty-printer yazmanızı sağlar.

Kaynak Kod / Source Code

```cpp
#include <cassert>     // EN: Runtime assertions
#include <chrono>      // EN: Timestamp generation
#include <cstdint>     // EN: Fixed-width integers
#include <functional>  // EN: std::function for invariant checks
#include <iomanip>     // EN: Output formatting
#include <iostream>    // EN: Console I/O
#include <sstream>     // EN: String stream for log formatting
#include <string>      // EN: String operations
#include <type_traits> // EN: Type traits for static_assert

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1 — ECULogger: Singleton Logger with Levels
// ═══════════════════════════════════════════════════════════════════════════

// EN: Log severity levels for automotive ECU diagnostics
// TR: Otomotiv ECU tanılaması için günlük önem seviyeleri
enum class LogLevel : std::uint8_t {
    TRACE = 0,
    DEBUG = 1,
    INFO  = 2,
    WARN  = 3,
    ERROR = 4,
    FATAL = 5
};

// EN: Convert LogLevel enum to string representation
// TR: LogLevel enum'ını metin temsiline dönüştür
inline const char* logLevelToString(LogLevel level) {
    switch (level) {
        case LogLevel::TRACE: return "TRACE";
        case LogLevel::DEBUG: return "DEBUG";
        case LogLevel::INFO:  return "INFO ";
        case LogLevel::WARN:  return "WARN ";
        case LogLevel::ERROR: return "ERROR";
        case LogLevel::FATAL: return "FATAL";
    }
    return "?????";
}

// EN: Singleton logger — centralized logging for ECU software
// TR: Tekil günlükçü — ECU yazılımı için merkezi günlükleme
class ECULogger {
public:
    // EN: Get the singleton instance
    // TR: Tekil örneği al
    static ECULogger& instance() {
        static ECULogger logger;
        return logger;
    }

    // EN: Set the minimum log level — messages below this are suppressed
    // TR: Minimum günlük seviyesini ayarla — altındaki mesajlar bastırılır
    void setLevel(LogLevel level) { minLevel_ = level; }

    // EN: Log a message with level, file and line information
    // TR: Seviye, dosya ve satır bilgisiyle bir mesaj günlükle
    void log(LogLevel level, const char* file, int line, const std::string& msg) {
        if (level < minLevel_) return;

        auto now = std::chrono::system_clock::now();
        auto epoch = now.time_since_epoch();
        auto millis = std::chrono::duration_cast<std::chrono::milliseconds>(epoch).count();

        std::cout << "[" << std::setw(13) << millis << "ms] "
                  << "[" << logLevelToString(level) << "] "
                  << file << ":" << line << " — " << msg << "\n";
    }
```

```cpp

    // EN: Deleted copy/move — singleton pattern
    // TR: Silinen kopya/taşıma — tekil kalıp
    ECULogger(const ECULogger&)            = delete;
    ECULogger& operator=(const ECULogger&) = delete;
    ECULogger(ECULogger&&)                 = delete;
    ECULogger& operator=(ECULogger&&)      = delete;

private:
    ECULogger() = default;
    LogLevel minLevel_ = LogLevel::TRACE;
};

// ─── Convenience Macros ──────────────────────────────────────────────────────

// EN: LOG macro captures __FILE__ and __LINE__ automatically
// TR: LOG makrosu __FILE__ ve __LINE__ bilgisini otomatik yakalar
#define ECU_LOG(level, msg) \
    ECULogger::instance().log(level, __FILE__, __LINE__, msg)

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2 — ECU_ASSERT: Enhanced Assertion with Logging
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Enhanced assert macro — logs the failure before aborting
// TR: Geliştirilmiş onaylama makrosu — iptal etmeden önce hatayı günlükler
#define ECU_ASSERT(cond, msg)                                                  \
    do {                                                                       \
        if (!(cond)) {                                                         \
            ECU_LOG(LogLevel::FATAL,                                           \
                    std::string("ASSERTION FAILED: ") + #cond + " — " + msg); \
            assert((cond) && (msg));                                           \
        }                                                                      \
    } while (false)

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3 — static_assert Examples: Compile-Time Guarantees
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Automotive CAN frame — strict layout required for hardware DMA
// TR: Otomotiv CAN çerçevesi — donanım DMA için katı yerleşim gerekli
struct CANFrame {
    std::uint32_t id;         // EN: CAN identifier (11 or 29 bit)
    std::uint8_t  dlc;        // EN: Data length code (0–8)
    std::uint8_t  data[8];    // EN: Payload bytes
    std::uint8_t  padding[3]; // EN: Alignment padding
};

// EN: Compile-time checks — zero runtime cost
// TR: Derleme zamanı kontrolleri — sıfır çalışma zamanı maliyeti
static_assert(sizeof(CANFrame) == 16, "CANFrame must be exactly 16 bytes for DMA");
static_assert(std::is_trivially_copyable_v<CANFrame>,
              "CANFrame must be trivially copyable for memcpy/DMA");
static_assert(alignof(CANFrame) <= 4, "CANFrame alignment must be at most 4 bytes");

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4 — Precondition / Postcondition Pattern
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Compute throttle percentage from raw ADC value with pre/post checks
// TR: Ham ADC değerinden gaz kelebeği yüzdesini ön/son kontrollerle hesapla
[[nodiscard]] inline double computeThrottlePercent(int rawADC) {
```

```
    // EN: Precondition — raw ADC must be in valid 12-bit range
    // TR: Ön koşul — ham ADC geçerli 12-bit aralığında olmalı
    ECU_ASSERT(rawADC >= 0 && rawADC <= 4095,
               "rawADC out of 12-bit range [0, 4095]");

    double percent = (static_cast<double>(rawADC) / 4095.0) * 100.0;

    // EN: Postcondition — result must be in [0, 100]
    // TR: Son koşul — sonuç [0, 100] aralığında olmalı
    ECU_ASSERT(percent >= 0.0 && percent <= 100.0,
               "Throttle percent out of [0, 100] range");

    return percent;
}

// EN: Validate engine RPM — returns clamped value with [[nodiscard]]
// TR: Motor devrini doğrula — [[nodiscard]] ile sınırlanmış değer döndürür
[[nodiscard]] inline int clampRPM(int rpm) {
    // EN: Precondition — RPM should be non-negative from sensor
    // TR: Ön koşul — sensörden gelen RPM negatif olmamalı
    ECU_ASSERT(rpm >= 0, "RPM cannot be negative");

    constexpr int kMaxRPM = 8000;
    if (rpm > kMaxRPM) {
        ECU_LOG(LogLevel::WARN,
                "RPM " + std::to_string(rpm) + " exceeds max, clamping to " +
                std::to_string(kMaxRPM));
        rpm = kMaxRPM;
    }
    return rpm;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5 — InvariantChecker: RAII Guard for Class Invariants
// ═══════════════════════════════════════════════════════════════════════════════

// EN: RAII guard that checks a class invariant when the scope exits
// TR: Kapsam çıkışında sınıf değişmezini kontrol eden RAII koruyucusu
class InvariantChecker {
public:
    using CheckFn = std::function<bool()>;

    // EN: Construct with an invariant check function and description
    // TR: Bir değişmez kontrol fonksiyonu ve açıklama ile oluştur
    InvariantChecker(CheckFn check, std::string description)
        : check_(std::move(check)), desc_(std::move(description)) {}

    // EN: On destruction, verify the invariant holds
    // TR: Yıkımda, değişmezin geçerli olduğunu doğrula
    ~InvariantChecker() {
        if (!check_()) {
            ECU_LOG(LogLevel::ERROR,
                    "INVARIANT VIOLATED: " + desc_);
        }
    }

    InvariantChecker(const InvariantChecker&)            = delete;
    InvariantChecker& operator=(const InvariantChecker&) = delete;

private:
    CheckFn     check_;
    std::string desc_;
};
```

```cpp

// EN: Simple battery monitor class that must maintain voltage invariant
// TR: Voltaj değişmezini koruması gereken basit akü izleme sınıfı
class BatteryMonitor {
public:
    explicit BatteryMonitor(double voltage) : voltage_(voltage) {}

    // EN: Update voltage with automatic invariant checking on scope exit
    // TR: Kapsam çıkışında otomatik değişmez kontrolüyle voltajı güncelle
    void updateVoltage(double newVoltage) {
        InvariantChecker guard(
            [this]() { return voltage_ >= 0.0 && voltage_ <= 16.0; },
            "Battery voltage must be in [0.0, 16.0]V"
        );
        voltage_ = newVoltage;
    }

    [[nodiscard]] double voltage() const { return voltage_; }

private:
    double voltage_;
};

// ═══════════════════════════════════════════════════════════════════════════════
// main() — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════
int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " Debugging Techniques & Defensive Programming Demos\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ─── Demo 1: ECULogger — Log at Different Levels ─────────────────────────
    // EN: Demonstrate logging at various severity levels
    // TR: Çeşitli önem seviyelerinde günlüklemeyi göster
    {
        std::cout << "── Demo 1: ECULogger — Log Levels ──\n";

        auto& logger = ECULogger::instance();
        logger.setLevel(LogLevel::TRACE);

        ECU_LOG(LogLevel::TRACE, "Engine control loop iteration 42");
        ECU_LOG(LogLevel::DEBUG, "Throttle ADC raw = 2048");
        ECU_LOG(LogLevel::INFO,  "ECU initialised — firmware v3.2.1");
        ECU_LOG(LogLevel::WARN,  "Coolant temperature approaching limit: 98°C");
        ECU_LOG(LogLevel::ERROR, "CAN bus timeout on node 0x1A — retrying");

        // EN: Raise minimum level to suppress TRACE and DEBUG
        // TR: TRACE ve DEBUG'ı bastırmak için minimum seviyeyi yükselt
        logger.setLevel(LogLevel::INFO);
        ECU_LOG(LogLevel::TRACE, "This TRACE message will be suppressed");
        ECU_LOG(LogLevel::DEBUG, "This DEBUG message will be suppressed");
        ECU_LOG(LogLevel::INFO,  "Only INFO and above are visible now");
        std::cout << '\n';
    }

    // ─── Demo 2: ECU_ASSERT — Precondition Checking ──────────────────────────
    // EN: Validate inputs using ECU_ASSERT macro
    // TR: Girdileri ECU_ASSERT makrosuyla doğrula
    {
        std::cout << "── Demo 2: ECU_ASSERT — Precondition Checking ──\n";

        // EN: Valid input — should pass assertions
        // TR: Geçerli girdi — onaylamaları geçmeli
        double throttle = computeThrottlePercent(2048);
        std::cout << "  Throttle at ADC 2048: "
                  << std::fixed << std::setprecision(1)
                  << throttle << "%\n";

        throttle = computeThrottlePercent(0);
        std::cout << "  Throttle at ADC 0:    " << throttle << "%\n";

        throttle = computeThrottlePercent(4095);
        std::cout << "  Throttle at ADC 4095: " << throttle << "%\n";
```

```

        // EN: Would trigger assertion (uncomment to test):
        // TR: Onaylamayı tetikler (test etmek için yorum işaretini kaldırın): double bad =
        // computeThrottlePercent(-100);  // ASSERTION FAILED
        std::cout << '\n';
    }

    // ─── Demo 3: static_assert — Compile-Time Guarantees ──────────────────────────────
    // EN: These checks happen at compile time; if they fail, build stops
    // TR: Bu kontroller derleme zamanında olur; başarısız olursa derleme durur
    {
        std::cout << "── Demo 3: static_assert — Compile-Time Checks ──\n";

        std::cout << "  sizeof(CANFrame)  = " << sizeof(CANFrame)  << " bytes\n";
        std::cout << "  alignof(CANFrame) = " << alignof(CANFrame) << " bytes\n";
        std::cout << "  is_trivially_copyable: "
                  << std::boolalpha
                  << std::is_trivially_copyable_v<CANFrame> << "\n";

        // EN: Additional compile-time checks
        // TR: Ek derleme zamanı kontrolleri
        static_assert(sizeof(int) >= 4, "int must be at least 32 bits");
        static_assert(std::is_same_v<decltype(CANFrame::id), std::uint32_t>,
                      "CAN ID must be uint32_t");

        std::cout << "  All static_assert checks passed at compile time.\n\n";
    }

    // ─── Demo 4: InvariantChecker — Automatic Postcondition Verify ────────────────────
    // EN: RAII guard automatically checks invariant when scope exits
    // TR: RAII koruyucusu kapsam çıkışında otomatik olarak değişmezi kontrol eder
    {
        std::cout << "── Demo 4: InvariantChecker — RAII Guard ──\n";

        BatteryMonitor battery(12.6);
        std::cout << "  Initial voltage: " << battery.voltage() << "V\n";

        // EN: Normal update — invariant holds
        // TR: Normal güncelleme — değişmez geçerli
        battery.updateVoltage(14.2);
        std::cout << "  After charge:    " << battery.voltage() << "V\n";

        // EN: Edge-case update — invariant still holds
        // TR: Sınır değer güncelleme — değişmez hâlâ geçerli
        battery.updateVoltage(0.0);
        std::cout << "  Deep discharge:  " << battery.voltage() << "V\n";

        // EN: Violation — InvariantChecker logs error on scope exit
        // TR: İhlal — InvariantChecker kapsam çıkışında hata günlükler
        battery.updateVoltage(18.5);
        std::cout << "  Over-voltage:    " << battery.voltage()
                  << "V (invariant violated!)\n\n";
    }

    // ─── Demo 5: Defensive Coding Patterns ────────────────────────────────────────────
    // EN: Demonstrate [[nodiscard]], clamp, and safe parameter handling
    // TR: [[nodiscard]], sınırlama ve güvenli parametre işlemeyi göster
    {
        std::cout << "── Demo 5: Defensive Coding Patterns ──\n";

        // EN: [[nodiscard]] ensures the caller uses the return value
        // TR: [[nodiscard]] çağıranın dönüş değerini kullanmasını sağlar
        int safeRPM = clampRPM(6500);
        std::cout << "  clampRPM(6500)  = " << safeRPM << "\n";

        safeRPM = clampRPM(9500);
        std::cout << "  clampRPM(9500)  = " << safeRPM << " (clamped)\n";
```

```

        // EN: Trying to ignore [[nodiscard]] result causes compiler warning:
        // TR: [[nodiscard]] sonucunu yok saymak derleyici uyarısına neden olur: clampRPM(5000);
        // // WARNING: ignoring return value with 'nodiscard'

        // EN: Defensive null/range checking pattern
        // TR: Savunmacı null/aralık kontrol kalıbı
        auto safeDivide = [](double a, double b) -> double {
            ECU_ASSERT(b != 0.0, "Division by zero in safeDivide");
            return a / b;
        };

        double ratio = safeDivide(100.0, 3.0);
        std::cout << "  safeDivide(100, 3) = "
                  << std::fixed << std::setprecision(4)
                  << ratio << "\n";

        std::cout << '\n';
    }

    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " All demos completed successfully.\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

Modül 7: Under the Hood — OS Derinlikleri

OS Derinlikleri

6 kaynak dosya · Standart: C++17

C++17

► 01. Vtable Memory Layout

module_07_under_the_hood/01_vtable_memory_layout.cpp

Konu: Under the Hood: V-Table (Virtual Table) Memory Cost Proof / Donanım Seviyesi: V-Table

Teori / Theory

THEORY: Polymorphism is NOT Free! (What is a Vptr?)

Polymorphism requires a mechanism to look up the correct function at RUNTIME. This is done using a "Virtual Table" (V-Table). When a class has AT LEAST ONE `virtual` function, the compiler silently inserts a completely hidden pointer into your class called `vptr` (Virtual Pointer). This pointer points to the V-Table. Since it is a pointer, it adds exactly 8 Bytes (on a 64-bit OS) to the size of EVERY SINGLE OBJECT you create!

TEORİ: Polymorphism Bedava Değildir!

Polimorfizm ("Sanal" metodlar), C++'ın doğru fonksiyonu ÇALIŞMA ZAMANINDA donanımsal Gerçek: 64-bit bir işletim sisteminde pointer'lar TAM 8 BAYT yer kaplar. Yere düşen bir yağmur damlası (Particle) sınıfında `virtual` kullanırsanız, milyonlarca damlanın her biri fazladan 8 bayt şişerek RAM'i bitirir!

CPPREF Depth: The Devirtualization Optimization

CppReference notes that modern compilers (GCC/Clang) try to perform "Devirtualization". If the compiler can PROVE at compile-time the exact type of the polymorphic object, it bypasses the V-Table lookup entirely and calls the function directly! (Massive speed boost). Using the `final` keyword helps the compiler prove this.

CPPREF Derinlik: Sanallaştırma Giderme Optimizasyonu (Sanal Çözme)

CppReference'a göre modern derleyiciler (GCC/Clang) "Sanal Çözme (Devirtualization)" adı verilen bir optimizasyon dener. Eğer derleyici, o nesnenin kimliğini derleme anında (Compile-Time) KESİN olarak kanıtlarsa, o yavaş V-Table aramasını tamamen çöpe atar ve doğrudan düz fonksiyon gibi çağırır! (Devasa FPS artışı). Sınıflara veya fonksiyonlara `final` kelimesini yazmak derleyiciye bu kanıtı sunar.

Kaynak Kod / Source Code

```cpp
#include <iostream>

// EN: 1. Plain Old Data Class. NO virtual keywords here.
// TR: Düz (Plain Old Data) Sınıf. İçinde virtual HİÇBİR ŞEY yok.
class CheapCharacter {
public:
  int health;  // 4 Bytes
  float speed; // 4 Bytes
  // Expected Total (Beklenen Toplam): 8 Bytes.

  // EN: Regular function. Does NOT take up RAM space inside the object. Resolved at
  // Compile-Time.
  // TR: Sıradan fonksiyon. RAM'de (nesne içinde) yer KAPLAMAZ. Derleme zamanında çözülür.
  void jump() { std::cout << "Jump! (Zıpla!)" << std::endl; }
};

// EN: 2. Polymorphic Class (Contains a V-Table!).
// TR: Polimorfik (Sanal Tablolu) Sınıf. Sadece bir 'virtual' eklendiği için...
class ExpensiveCharacter {
public:
  int health;  // 4 Bytes
  float speed; // 4 Bytes
  // Expected Total: 8 Bytes...

  // EN: BUT! The moment we say "virtual", the compiler secretly adds a `void* vptr;` behind the
  // scenes! vptr = 8 Bytes (on 64-bit systems). Total: 8 + 8 = 16 Bytes! (100% RAM overhead!)
  //
  // TR: ANCAK! "virtual" dendiği an, derleyici gizli bir 'void* vptr;' satırını GİZLİCE EKLER!
  // vptr = 8 Bayt (64-bit sistemlerde). Toplam: 8 + 8 = 16 Bayt bekliyoruz! (%100 RAM israfı!)
  virtual void jump() {
    std::cout << "Flying Jump! (Uçarak Zıpla!)" << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 7: UNDER THE HOOD (V-TABLE PROOF) ===\n" << std::endl;

  std::cout << "[64-Bit OS Memory Estimation]" << std::endl;
  std::cout << "1 int + 1 float = Normally occupies 8 Bytes." << std::endl;

  std::cout << "\n----------------------------------------" << std::endl;
  std::cout << "1. CheapCharacter (NO Polymorphism): " << sizeof(CheapCharacter) << " Bytes." <<
      std::endl;
  std::cout << "-> Only Health (4) + Speed (4). Functions are free." << std::endl;
  std::cout << "-> (Sadece Health ve Speed. Fonksiyon bellekte bedavadır.)" << std::endl;

  std::cout << "\n----------------------------------------" << std::endl;
  std::cout << "2. ExpensiveCharacter (HAS Polymorphism): " << sizeof(ExpensiveCharacter) <<
      " Bytes!" << std::endl;
  std::cout << "-> WHY 16 BYTES? Because an 8-Byte 'vptr' (Virtual Pointer) "
              "was injected!"
            << std::endl;
  std::cout << "-> (NEDEN 16 BAYT OLDU? Çünkü 8 bayt 'vptr' pointer eklendi!)" << std::endl;

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "EN: If you instantiate 1 Million sensor objects in an ECU "
            << std::endl;
  std::cout << "    DO NOT use 'virtual' in the base class. It wastes RAM and "
              "causes Cache Misses!"
            << std::endl;
  std::cout << "TR: Eğer bir ECU'da 1 milyon sensör nesnesi üreteceksiniz," << std::endl;
  std::cout << "    Merminin Base sınıfında 'virtual' KULLANMAYIN donanımı yorar." << std::endl;

  return 0;
}
```

► 02. Sigsegv And Raii

module_07_under_the_hood/02_sigsegv_and_raii.cpp

Konu: Under the Hood: Segmentation Fault (SIGSEGV) & Prevention via RAII / İşletim Sistemi

Teori / Theory

THEORY: Why do we get SIGSEGV (Crash)?

The OS allocates specific RAM pages for your C++ program. If you try to read or write to a memory address that the OS DID NOT GIVE YOU (like `nullptr`, or an array index that is out of bounds), the hardware's MMU (Memory Management Unit) sends an interrupt to the OS. The OS immediately ASSASSINATES your program with a signal called `SIGSEGV` (Segmentation Violation).

TEORİ: Neden SIGSEGV (Çöktü) Hatası Alırız?

İşletim Sistemi (OS), C++ programınıza sadece belirli bir RAM hafızası (Sayfa) verir. Eğer size ait OLMAYAN bir adrese (Örn: Donanımın adresine veya 'nullptr' içi boş adrese) dokunmaya, okumaya veya yazmaya çalışırsanız, işlemcinin Donanımsal Bellek Yöneticisi (MMU) bunu yakalar. İşletim sistemi anında programınıza suikast düzenler ve `SIGSEGV` (Segmentation Violation) sinyali yollayarak UYGULAMAYI ZORLA KAPATIR!

THEORY: RAII - How it saves lives!

RAII = "Resource Acquisition Is Initialization". Manual `new` and `delete` leads to 90% of human errors. If you forget `delete`, you get a Memory Leak. If you delete twice, you get a Double Free Crash. RAII dictates: "Entrust the Resource (RAM/File/Mutex) to a C++ Object. When the object goes out of scope {...}, its Destructor AUTOMATICALLY CLEANS IT UP." (e.g., std::unique_ptr, std::vector).

TEORİ: RAII Nedir ve Hayatımızı Nasıl Kurtarır?

RAII = "Kaynak Edinimi Başlatmadır". Manuel `new` ve `delete` kullanımı hataların %90'ına yol açar. `delete` yapmayı unutursanız Bellek Sızıntısı (Memory Leak) olur. İki kere silerseniz Çift Silme Hatası (Double Free/SIGSEGV) olur. RAII der ki: "RAM/Dosya/Kilit kaynağını C++ Objelerine emanet et. Nesne süslü parantez dışına çıkıp öldüğünde, KENDİ KENDİNİ OTOMATİK SİLSİN." (Örn: std::unique_ptr, std::vector).

CPPREF Depth: OS Core Dumps (Program Cesetleri)

When SIGSEGV happens, Linux doesn't just kill the program. It violently dumps the entire RAM state (Memory map, registers) into a massive file called a "Core Dump" (The corpse of your program). You can later load this Core Dump into GDB (`gdb ./app core`) to do an autopsy and see exactly which line of C++ triggered the assassination!

CPPREF Derinlik: İşletim Sistemi Bellek Dökümleri (Program Cesetleri)

SIGSEGV olduğunda Linux sadece programı kapatıp geçmez. Çöktüğü andaki tüm İşlemci yazmaçlarını (Registers) ve RAM haritasını "Core Dump" (Çekirdek Dökümü - Programın Cesedi) denen devasa bir dosyaya yazar. Daha sonra GDB hata ayıklayıcısına bu cesedi verip (`gdb ./uygulama core`) otopsi yapabilir, programınızın tam olarak hangi C++ satırından vurulduğunu görebilirsiniz!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <vector>

// EN: Dangerous legacy C-style function. WARNING: SIGSEGV RISK!
// TR: Tehlikeli (Eski C Tarzi) bir fonksiyon. DİKKAT: SIGSEGV RİSKİ!
void dangerousMemoryAccess() {
  int *garbageAddress = nullptr; // Address 0x0000000
  // EN: Suppress warning — demo only
  // TR: Uyarıyı bastır — yalnızca demo
  (void)garbageAddress;

  // EN: BURIED SIGSEGV TRAP: If you uncomment the line below, the program is ASSASSINATED
  // INSTANTLY!
  // TR: GÖMÜLÜ SIGSEGV TUZAĞI: Aşağıdaki satır açılırsa Program ANINDA İNFAZ EDİLİR!
  //
  // *garbageAddress = 55; // SIGSEGV (Segmentation Fault: 11)!!!

  // EN: Why? Because the OS did not grant us permission to write to Address 0.
  // TR: Neden? Cünkü İşletim Sistemi bize 0. (Yokluk) adresine yazmamız için HAK VERMEDİ.
}

// EN: Memory Leak Trap (What happens without RAII?)
// TR: Memory Leak Tuzağı. (RAII kullanılmazsa ne olur?)
void createAndForget() {
  // EN: Stole 2000 Bytes from OS!
  // TR: OS'den 2000 Bayt çaldık!
  int *array = new int[500];

  // EN: Imagine an error in the control loop
  // TR: Kontrol döngüsünde hata çıktı.
  bool randomErrorOccurred = true;

  if (randomErrorOccurred) {
    // EN: WARNING! We escape before reaching 'delete[] array'. RAM is lost forever (Leak).
    // TR: DİKKAT! 'delete[] array;' koduna asla gelemeden fonksiyondan kaçtık! RAM sızdı.
    return;
  }

  // EN: Would have freed it if no error happened...
  // TR: Hata olmasaydı silinecekti...
  delete[] array;
}

// EN: ------ THE ARCHITECTURE OF MODERN C++: RAII ------
// TR: ------ MÜKEMMEL C++ MİMARİSİ: RAII ------
class ActuatorController {
public:
  ActuatorController() {
    std::cout << "[RAII] Actuator initialized (Memory Acquired). / Aktüatör başlatıldı." <<
        std::endl;
  }
  ~ActuatorController() {
    std::cout << "[RAII] Actuator went out of scope! AUTO-RELEASED!!" <<
        "\n       (Aktüatör kapsam dışına çıktı! OTOMATIK SERBEST BIRAKILIYOR!)" << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 7: SIGSEGV (SEGMENTATION FAULT) & RAII ===\n" << std::endl;

  std::cout << "\n1. Risk of SIGSEGV (SIGSEGV Riski ve Neden Çıkar?)" << std::endl;
  std::cout << "EN: The OS won't let you write to unowned RAM. It shoots you "
               "with SIGSEGV."
            << std::endl;
  std::cout << "TR: İşletim Sistemi, hakkiniz olmayan RAM adresine yazmaniza "
               "anında SIGSEGV vurur."
            << std::endl;

  dangerousMemoryAccess();

  std::cout << "\n2. The RAII Solution (RAII Çözümü)" << std::endl;
```

```cpp

  // EN: Creating a Scope { ... }
  // TR: Süslü parantez ile (Scope / Kapsam) yaratıyoruz.
  {
    // EN: STEP 1: Smart Pointer is the backbone of C++14 RAII. NO NEW, NO DELETE.
    // TR: ADIM 1: "Smart Pointer", C++14'ün RAII silahidir. Asla `new` veya `delete`
    // kullanılmaz.
    std::unique_ptr<ActuatorController> safeObject = std::make_unique<ActuatorController>();

    // EN: STEP 2: std::vector is historically a RAII container itself.
    // TR: ADIM 2: std::vector doğuştan bir RAII'dir.
    std::vector<int> smartArray(500, 10); // 2000 Bytes managed automatically!

    std::cout << "  -> Inside the Scope, everything is fine... (Kapsam içindeyiz...)" << std::endl;

    bool suddenCodeCrash = true;
    if (suddenCodeCrash) {
      std::cout << "  -> ERROR BROKE THE FLOW! ESCAPING! (HATA İLE KOD "
                   "KIRILDI! KACIYORUZ!)"
                << std::endl;
      // EN: Without RAII, breaking here would mean severe leaking...
      // TR: Eğer RAII olmasaydı 'return/throw' yüzünden RAM sızacaktı...
    }

  // EN: THIS IS RAII! The moment this bracket closes, EVERYTHING is returned to OS
  // automatically!
  // TR: İşTE RAII! Parantez kapandığı an, İşletim Sisteminden alınan HDD/RAM her şey İADE
  // EDİLİR!
  }

  std::cout << "  -> Returned outside safely with 0 Bytes leaked. (Dışarıya "
               "Sifir Cöple dönüldü.)"
            << std::endl;

  return 0;
}
```

► 03. Fork Vs Thread Pool

module_07_under_the_hood/03_fork_vs_thread_pool.cpp

Konu: Under the Hood: Posix fork() vs std::thread & Thread Pooling / İşletim Sistemi: Process

Teori / Theory

THEORY: Process vs Thread

In Linux (POSIX), `fork()` creates a completely new PROCESS. It CLONES the entire memory space of the parent program. Processes do not share memory. If Process A changes a variable, Process B does not see it! (High security, HIGH cost, communication requires "Pipes/IPC").
`std::thread` creates a THREAD within the same Process. All threads SHARE the exact same RAM (Heap, Global vars). If Thread A changes a variable, Thread B sees it immediately! (Low cost, HIGH DANGER of Race Conditions -> Requires Mutex).

TEORİ: İşlem (Process) vs İş Parçacığı (Thread)

İşletim Sistemi Mimarisinde `fork()` (Linux'a özeldir) kocaman bir ağaçtır. Sen `fork` çağırdığında programın RAM'deki her şeyinin (3 GB bile olsa) İşletim Sistemi tarafından İKİNCİ BİR KOPYASI yaratılır! İki paralel evren oluşur. Birbirlerinin değişkenlerini GÖREMEZLER! (Güvenlik devasa, Bedel devasa).
`std::thread` (C++11) ise aynı evin içinde yeni bir odadır. Tüm Thread'ler AYNI SALONU (RAM / Heap Bellek) paylaşırlar. Bir Thread masadaki (RAM) değeri değiştirirse, diğeri anında görür! (Bedel yok denecek kadar azdır ama Race Condition (Yarış Durumu) çıkmaması için `std::mutex` Kilidi mecburidir).

THEORY: Thread Pooling Architecture

Constantly "Creating a Thread, assigning work, and Destroying the Thread" is slow even at the OS level. Professional automotive middleware (AUTOSAR) or servers (Node.js) create a "Thread Pool" on startup. They keep heavily sleeping workers alive. When there is work, they wake them up, no creation cost overhead!

TEORİ: İş Parçacığı Havuzu

Sürekli "Thread Yarat, iş ver, Thread'i Yık" döngüsü işletim sistemi seviyesinde bile yavaştır. Profesyonel otomotiv ara katmanları (AUTOSAR) veya sunucular (Node.js) uygulama açılışında bir "Thread Pool" (İş Parçacığı Havuzu) açar. 8 boş çekirdek için 8 tane "aç uyuyan işçi" Thread yaratılır. İş oldukça onlara yollanır, bitince uyumaya geri dönerler. ASLA baştan oluşturulmaz (Zero overhead).

CPPREF Depth: Context Switching Penalty

The OS Scheduler switches between running threads 1000 times a second. This is called a "Context Switch" and it is EXPENSIVE! A thread has to save all its CPU registers to RAM, and load the next thread's registers. If you spawn 1000 threads on an 8-core CPU, the OS spends more time Context Switching than actually executing your code (Thrashing)!

CPPREF Derinlik: Bağlam Değişimi Cezası

İşletim Sistemi Zamanlayıcısı (OS Scheduler) saniyede 1000 kere koşan Thread'ler arasında geçiş yapar. Buna "Context Switch (Bağlam Değiştirme)" denir ve ÇOK PAHALIDIR! Bir Thread tüm işlemci yazmaçlarını RAM'e kaydeder, diğeri de RAM'den işlemciye yüklenir. 8 Core bir CPU'da 1000 tane Thread açarsanız, donanım kodunuzu çalıştırmaktan çok sadece Thread'ler arası geçiş yaparak alev alır ve sistem kitlenir (Thrashing/Donma)! Thread Pool bu yüzden şarttır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
// EN: For waiting on Linux processes
#include <sys/wait.h>
#include <thread>
// EN: POSIX standard (for fork). Does NOT exist on Windows!
#include <unistd.h>

// EN: A variable sitting in the global heap space.
// TR: Global uzayda oturan ortak bir değişken.
int sharedVariable = 100;

void threadWorkerTask(int id) {
```

```
13    // EN: All Threads SEE and MODIFY the EXACT SAME variable!
14    // TR: Tüm İplikler (Threads) Aynı "sharedVariable"ı GÖRÜR VE DEĞİŞTİRİR!
15    sharedVariable += 50;
16    std::cout << "[THREAD " << id << "] I am a C++ Thread! Shared RAM value is now: " <<
17        sharedVariable << std::endl;
18  }
19
20  int main() {
21    std::cout << "=== MODULE 7: FORK (PROCESS) VS THREAD (IS PARCACIGI) ===\n" << std::endl;
22
23    std::cout << "--- 1. C++11 std::thread (Lightweight, Shared Memory / Hafif "
24                "ve Ortak) ---"
25              << std::endl;
26    sharedVariable = 100;
27
28    std::thread t1(threadWorkerTask, 1);
29    std::thread t2(threadWorkerTask, 2);
30
31    t1.join();
32    t2.join();
33
34    std::cout << "[MAIN] Threads finished. Did the shared variable TRULY change "
35                "globally?\n"
36              << "[MAIN] Degisken sistemde GERCEKTEN DEĞİŞTİ Mİ?: "
37              << sharedVariable << std::endl;
38    // EN: Output will be 200! (Shared Memory)
39    // TR: Çıktı 200 olacaktır! (Ortak Bellek)
40
41    std::cout << "\n-----------------------------------------------------\n";
42
43    std::cout << "--- 2. Linux POSIX fork() (Heavy, Cloned Universes / Ağır, "
44                "Paralel Evrenler) ---"
45              << std::endl;
46    sharedVariable = 100; // Reset / Sıfırlıyoruz.
47
48    // EN: Fork SPLITS the OS execution in two. It returns once as the Parent, once as the Child.
49    // TR: Fork işletim sistemini 2'ye yarar. İşletim sistemi aynı satırda ikiz kod döndürür.
50    pid_t processID = fork();
51
52    if (processID < 0) {
53      std::cerr << "Fork failed! (Fork fiyaskosu!)" << std::endl;
54      return 1;
55    } else if (processID == 0) {
56      // === CHILD PROCESS ZONE (THE PARALLEL UNIVERSE) / ÇOCUK BÖLGESİ ===
57      // EN: The child is born with an exact 1:1 photostatic copy of the RAM.
58      // TR: Çocuk, babasının anılarının (RAM'in) 1'e 1 fotokopisine sahip başlar!
59      sharedVariable += 500;
60      std::cout << "[FORK: CHILD PROCESS / ÇOCUK] My pID: " << getpid() <<
61          ". In my parallel universe, sharedVariable is (Benim "
62                  "evrenimde değer): "
63                << sharedVariable << std::endl;
64      // EN: Child commits suicide.
65      // TR: Çocuk işini zorla bitirip "exit" yapar.
66      exit(0);
67
68    } else {
69      // === PARENT PROCESS ZONE (THE ORIGINAL UNIVERSE) / BABA (ASIL) BÖLGE ===
70      // EN: processID > 0 means it's the ID of the son that was just born.
71      // TR: processID > 0 demek, Oğlumun doğduğu pID numarasıdır.
72
73      // EN: MUST WAIT for the child to die, otherwise it becomes a Zombie Process.
74      // TR: Çocuğun paralel evrendeki işini bitirmesini Bekliyoruz (Zombi Proces olmasın diye).
75      wait(NULL);
76
77      std::cout << "[FORK: PARENT PROCESS / BABA] Child finished, I woke up." << std::endl;
78      std::cout << "[FORK: PARENT PROCESS / BABA] Let's see if the child's +500 "
79                  "affected MY UNIVERSE?"
80                << std::endl;
81      std::cout << "[RESULT / SONUÇ] Parent's sharedVariable is (Babanın "
82                  "evrenindeki değer): "
83                << sharedVariable << " !!!\n"
84                << std::endl;
```

```cpp

    // EN: Output will STILL be 100! The OS blocked the memory via Virtual Memory walls.
    // TR: Cıktı Hala 100 kalıcaktır! İşletim Sistemi aralarına Duvar (Virtual Memory
    // Segregation) cektì!
  }

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "A-) 'fork()' (Process) is ultra safe but HEAVY. Data exchange "
               "(Pipes) is a nightmare."
            << std::endl;
  std::cout << "B-) 'std::thread' runs extremely FAST but causes Race Conditions." << std::endl;
  std::cout << "C-) 'THREAD POOL' (Havuz) is the modern industry secret. Keep "
               "threads alive and waiting!"
            << std::endl;

  return 0;
}
```

► 04. Memory Model And Cache

module_07_under_the_hood/04_memory_model_and_cache.cpp

Konu: EN: CPU cache hierarchy, memory layout, false sharing, and cache-friendly design

Teori / Theory

THEORY: CPU Cache Hierarchy & Memory Model

Modern CPUs do not access main memory (RAM) directly for every read/write. Instead, a hierarchy of small, fast caches sits between the core and RAM: - L1 cache — per-core, ~32-64 KB, ~1-4 cycle latency - L2 cache — per-core, ~256 KB-1 MB, ~10-20 cycles - L3 cache — shared across cores, ~4-64 MB, ~30-50 cycles - Main RAM — ~100-300 cycles latency Data is transferred in cache lines, typically 64 bytes at a time.

TEORİ: CPU Önbellek Hiyerarşisi ve Bellek Modeli

Modern CPU'lar her okuma/yazma için doğrudan ana belleğe (RAM) erişmez. Bunun yerine, çekirdek ile RAM arasında küçük ve hızlı önbellekler bulunur: - L1 önbellek — çekirdek başına, ~32-64 KB, ~1-4 döngü gecikme - L2 önbellek — çekirdek başına, ~256 KB-1 MB, ~10-20 döngü - L3 önbellek — çekirdekler arası paylaşımlı, ~4-64 MB, ~30-50 döngü - Ana RAM — ~100-300 döngü gecikme Veriler önbellek satırları halinde, genellikle 64 bayt olarak aktarılır.

THEORY: Spatial & Temporal Locality

Spatial Locality — When data is accessed, the entire cache line (64B) is fetched. Contiguous memory (std::vector) is fast because the next element is already in cache. Scattered memory (std::list, pointer-chasing) causes cache misses.
Temporal Locality — Recently accessed data stays in cache and is fast to re-read. Hot loops over small data sets benefit enormously from temporal locality.

TEORİ: Uzamsal ve Zamansal Yerellik

Uzamsal Yerellik — Veriye erişildiğinde, tüm önbellek satırı (64B) getirilir. Bitişik bellek (std::vector) hızlıdır çünkü sonraki eleman zaten önbellektedir. Dağınık bellek (std::list, işaretçi takibi) önbellek kaçırmalarına neden olur.
Zamansal Yerellik — Son erişilen veri önbellekte kalır ve tekrar okumak hızlıdır. Küçük veri setleri üzerindeki sıcak döngüler zamansal yerellikten büyük fayda görür.

THEORY: False Sharing & SoA vs AoS

False Sharing — When two threads write to different variables on the SAME cache line, hardware forces constant invalidation ("cache line ping-pong"). Fix: pad each variable to its own 64-byte cache line with alignas(64).
SoA vs AoS — AoS (Array of Structs): struct { float x,y,z; } arr[N]; Nice OOP, but iterating one field loads unused fields into cache. SoA (Struct of Arrays): struct { float x[N], y[N], z[N]; }; Better cache use when iterating one field across all elements.
alignas keyword — alignas(N) forces a variable/struct to start at an N-byte boundary. Use alignas(64) to ensure struct occupies its own cache line.

TEORİ: Yanlış Paylaşım ve SoA vs AoS

Yanlış Paylaşım — İki iş parçacığı AYNI önbellek satırındaki farklı değişkenlere yazarsa, donanım sürekli geçersiz kılma yapar ("önbellek satırı ping-pong"). Çözüm: alignas(64) ile her değişkeni kendi 64 baytlık satırına hizalayın.
SoA vs AoS — AoS (Yapı Dizisi): struct { float x,y,z; } arr[N]; Güzel OOP, ama tek alan iterasyonunda kullanılmayan alanlar da önbelleğe yüklenir. SoA (Dizi Yapısı): struct { float x[N], y[N], z[N]; }; Tüm elemanlar üzerinde tek alan iterasyonunda daha iyi önbellek kullanımı.
alignas anahtar kelimesi — alignas(N) bir değişkeni/yapıyı N-bayt sınırına hizalamaya zorlar. Yapının kendi önbellek satırını kaplaması için alignas(64) kullanın.

CPPREF Depth: C++ Memory Model — Sequential Consistency and Acquire-Release Semantics

C++11 defines a formal memory model (§1.10) that governs how threads observe writes to shared variables. `memory_order_seq_cst` (the default for std::atomic operations) provides a total order visible to all threads — the simplest mental model but the most expensive on weakly-ordered architectures (ARM, POWER). `memory_order_acquire` / `memory_order_release` establish a happens-before relationship between a store-release and a load-acquire on the SAME atomic variable, without imposing a full fence. `memory_order_relaxed` guarantees only atomicity — no ordering at all — useful for pure counters. False sharing occurs when two independent atomics (or hot variables) reside on the same cache line (typically 64 bytes), forcing cores to bounce the line back and forth. C++17 exposes `std::hardware_destructive_interference_size` so you can pad/align to avoid it. Always profile before weakening memory orders; incorrect relaxed code creates data races that surface only under heavy load.

CPPREF Derinlik: C++ Bellek Modeli — Sıralı Tutarlılık ve Acquire-Release Semantiği

C++11 iş parçacıklarının paylaşılan değişkenlere yapılan yazmaları nasıl gözlemlediğini yöneten resmi bir bellek modeli tanımlar (§1.10). `memory_order_seq_cst` tüm iş parçacıklarına görünür toplam sıra sağlar — en basit model ama zayıf sıralı mimarilerde (ARM, POWER) en pahalısı. `memory_order_acquire/release` AYNI atomik değişken üzerinde store-release ile load-acquire arasında happens-before ilişkisi kurar; tam çit gerektirmez. `memory_order_relaxed` yalnızca atomiklik garanti eder, sıralama yoktur — saf sayaçlar için uygundur. Yanlış paylaşım (false sharing), iki bağımsız atomik aynı önbellek satırında (genellikle 64 bayt) bulunduğunda çekirdeklerin satırı sürekli birbirine göndermesine yol açar. C++17 bunu önlemek için `std::hardware_destructive_interference_size` sunar.

Kaynak Kod / Source Code

```cpp
#include <iostream>      // EN: Standard I/O
#include <vector>        // EN: Contiguous dynamic array
#include <list>          // EN: Doubly-linked list for comparison
#include <chrono>        // EN: High-resolution timing
#include <thread>        // EN: Thread support for false sharing demo
#include <numeric>       // EN: std::accumulate, std::iota
#include <algorithm>     // EN: std::generate
#include <cstdint>       // EN: Fixed-width integer types
#include <cstddef>       // EN: std::size_t, alignof

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Data Structures — AoS vs SoA (Automotive Sensors)
// ═══════════════════════════════════════════════════════════════════════════

// EN: AoS — each sensor reading is one struct with all fields together
// TR: AoS — her sensör okuması tüm alanları bir arada tutan bir yapıdır
struct SensorAoS {
    float temp;
    float pressure;
    float voltage;
};

// EN: SoA — separate arrays for each field, better cache line utilization
// TR: SoA — her alan için ayrı dizi, daha iyi önbellek satırı kullanımı
struct SensorSoA {
    std::vector<float> temp;
    std::vector<float> pressure;
    std::vector<float> voltage;

    // EN: Resize all channels at once
    // TR: Tüm kanalları aynı anda yeniden boyutlandır
    void resize(std::size_t n) {
        temp.resize(n);
        pressure.resize(n);
        voltage.resize(n);
    }
};

// ─── 1. False-Sharing Prevention with alignas ──────────────────────────

// EN: Without padding — both counters likely share a cache line
// TR: Dolgu olmadan — iki sayaç muhtemelen aynı önbellek satırını paylaşır
struct NaiveCounter {
    std::uint64_t counter_a = 0;
    std::uint64_t counter_b = 0;
};

// EN: With padding — each counter occupies its own 64-byte cache line
// TR: Dolgu ile — her sayaç kendi 64 baytlık önbellek satırını kaplar
struct alignas(64) PaddedCounter {
    std::uint64_t value = 0;
};

// ─── 2. ECU Configuration with Cache-Line Alignment ────────────────────

// EN: ECU config aligned to cache line boundary for DMA / shared-memory use
// TR: DMA / paylaşımlı bellek kullanımı için önbellek satırına hizalı ECU yapılandırması
struct alignas(64) EcuConfig {
    std::uint32_t ecu_id;
    float max_rpm;
    float idle_rpm;
    float fuel_trim;
    std::uint8_t  cylinder_count;
    bool          turbo_enabled;
```

```
    // EN: Padding is implicit — alignas ensures 64-byte total boundary
    // TR: Dolgu örtüktür — alignas 64 baytlık toplam sınırı garanti eder
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: Benchmark Utilities
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Simple RAII timer that prints elapsed time on destruction
// TR: Yıkımda geçen süreyi yazdıran basit RAII zamanlayıcı
class ScopedTimer {
public:
    explicit ScopedTimer(const char* label)
        : label_(label), start_(std::chrono::high_resolution_clock::now()) {}

    ~ScopedTimer() {
        auto end = std::chrono::high_resolution_clock::now();
        auto us  = std::chrono::duration_cast<std::chrono::microseconds>(end - start_).count();
        std::cout << "  [" << label_ << "] " << us << " us\n";
    }

    // EN: Non-copyable, non-movable
    // TR: Kopyalanamaz, taşınamaz
    ScopedTimer(const ScopedTimer&)            = delete;
    ScopedTimer& operator=(const ScopedTimer&) = delete;

private:
    const char* label_;
    std::chrono::high_resolution_clock::time_point start_;
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: Demonstration Functions
// ═══════════════════════════════════════════════════════════════════════════════

// ─── 3. AoS vs SoA Benchmark ─────────────────────────────────────────────────

// EN: Sum all temperature readings — AoS version (loads pressure & voltage too)
// TR: Tüm sıcaklık okumalarını topla — AoS versiyonu (basınç ve voltajı da yükler)
double sumTemperaturesAoS(const std::vector<SensorAoS>& sensors) {
    double total = 0.0;
    for (const auto& s : sensors) {
        total += static_cast<double>(s.temp);
    }
    return total;
}

// EN: Sum all temperature readings — SoA version (only touches temp array)
// TR: Tüm sıcaklık okumalarını topla — SoA versiyonu (sadece temp dizisine dokunur)
double sumTemperaturesSoA(const SensorSoA& sensors) {
    double total = 0.0;
    for (const auto& t : sensors.temp) {
        total += static_cast<double>(t);
    }
    return total;
}

// ─── 4. Vector vs List Iteration ─────────────────────────────────────────────

// EN: Iterate and sum elements — tests spatial locality difference
// TR: Elemanları dolaş ve topla — uzamsal yerellik farkını test eder
void benchmarkVectorVsList(std::size_t n) {
    std::cout << "\n--- Vector vs List Iteration (n=" << n << ") ---\n";
```

```cpp

    // EN: Fill a vector with sequential values
    // TR: Vektörü sıralı değerlerle doldur
    std::vector<int> vec(n);
    std::iota(vec.begin(), vec.end(), 0);

    // EN: Fill a list with the same values
    // TR: Listeyi aynı değerlerle doldur
    std::list<int> lst(vec.begin(), vec.end());

    volatile long long sink = 0;

    {
        ScopedTimer t("vector sum");
        long long sum = 0;
        for (const auto& v : vec) { sum += v; }
        sink = sum;
    }

    {
        ScopedTimer t("list   sum");
        long long sum = 0;
        for (const auto& v : lst) { sum += v; }
        sink = sum;
    }

    (void)sink;
}

// ─── 5. False Sharing Demonstration ──────────────────────────────────────────

static constexpr std::size_t kIncrements = 50'000'000;

// EN: Two threads hammer the same NaiveCounter — false sharing likely
// TR: İki iş parçacığı aynı NaiveCounter'ı döver — yanlış paylaşım olası
void falseSharingNaive() {
    NaiveCounter nc;

    auto inc_a = [&]() {
        for (std::size_t i = 0; i < kIncrements; ++i) { ++nc.counter_a; }
    };
    auto inc_b = [&]() {
        for (std::size_t i = 0; i < kIncrements; ++i) { ++nc.counter_b; }
    };

    ScopedTimer t("naive  (shared line)");
    std::thread t1(inc_a);
    std::thread t2(inc_b);
    t1.join();
    t2.join();
}

// EN: Two threads on separate PaddedCounters — no false sharing
// TR: İki iş parçacığı ayrı PaddedCounter'larda — yanlış paylaşım yok
void falseSharingPadded() {
    PaddedCounter pa;
    PaddedCounter pb;

    auto inc_a = [&]() {
        for (std::size_t i = 0; i < kIncrements; ++i) { ++pa.value; }
    };
    auto inc_b = [&]() {
        for (std::size_t i = 0; i < kIncrements; ++i) { ++pb.value; }
    };

    ScopedTimer t("padded (own line)   ");
    std::thread t1(inc_a);
    std::thread t2(inc_b);
    t1.join();
    t2.join();
}
```

```cpp

// ─── 6. Master Demonstration ──────────────────────────────────────────────────

void demonstrateCacheEffects() {
    constexpr std::size_t N = 2'000'000;

    // EN: Prepare AoS data
    // TR: AoS verisini hazırla
    std::vector<SensorAoS> aos(N);
    for (std::size_t i = 0; i < N; ++i) {
        aos[i].temp     = static_cast<float>(i) * 0.01f;
        aos[i].pressure = static_cast<float>(i) * 0.02f;
        aos[i].voltage  = static_cast<float>(i) * 0.03f;
    }

    // EN: Prepare SoA data
    // TR: SoA verisini hazırla
    SensorSoA soa;
    soa.resize(N);
    for (std::size_t i = 0; i < N; ++i) {
        soa.temp[i]     = static_cast<float>(i) * 0.01f;
        soa.pressure[i] = static_cast<float>(i) * 0.02f;
        soa.voltage[i]  = static_cast<float>(i) * 0.03f;
    }

    std::cout << "--- AoS vs SoA Temperature Sum (N=" << N << ") ---\n";

    volatile double result = 0.0;

    {
        ScopedTimer t("AoS temp sum");
        result = sumTemperaturesAoS(aos);
    }
    {
        ScopedTimer t("SoA temp sum");
        result = sumTemperaturesSoA(soa);
    }

    (void)result;

    // EN: Vector vs list comparison
    // TR: Vektör ile liste karşılaştırması
    benchmarkVectorVsList(N);

    // EN: False sharing comparison
    // TR: Yanlış paylaşım karşılaştırması
    std::cout << "\n--- False Sharing: Naive vs Padded ---\n";
    falseSharingNaive();
    falseSharingPadded();
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: main() — All Demos
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "============================================\n";
    std::cout << " Module 07 — Memory Model & Cache Effects\n";
    std::cout << "============================================\n\n";

    // ─── Demo 1: sizeof and alignment checks ──────────────────────────────────
    std::cout << "--- Demo 1: sizeof / alignof ---\n";

    std::cout << "  sizeof(SensorAoS)   = " << sizeof(SensorAoS)   << " bytes\n";
    std::cout << "  alignof(SensorAoS)  = " << alignof(SensorAoS)  << " bytes\n";
    std::cout << "  sizeof(NaiveCounter) = " << sizeof(NaiveCounter) << " bytes\n";
    std::cout << "  sizeof(PaddedCounter)= " << sizeof(PaddedCounter)<< " bytes\n";
    std::cout << "  alignof(PaddedCounter)=" << alignof(PaddedCounter)<< " bytes\n";
    std::cout << "  sizeof(EcuConfig)   = " << sizeof(EcuConfig)   << " bytes\n";
    std::cout << "  alignof(EcuConfig)  = " << alignof(EcuConfig)  << " bytes\n\n";
```

```

    // EN: Verify PaddedCounter is on its own cache line
    // TR: PaddedCounter'ın kendi önbellek satırında olduğunu doğrula
    static_assert(sizeof(PaddedCounter) >= 64,
                  "PaddedCounter must be at least 64 bytes for cache-line isolation");
    static_assert(alignof(PaddedCounter) == 64,
                  "PaddedCounter must be aligned to 64-byte boundary");

    // ── Demo 2: AoS vs SoA benchmark ──────────────────────────────────────
    std::cout << "--- Demo 2 & 3 & 4: Cache Effect Benchmarks ---\n";
    demonstrateCacheEffects();

    // ── Demo 5: alignas applied to ECU config struct ──────────────────────
    std::cout << "\n--- Demo 5: ECU Config Alignment ---\n";

    EcuConfig ecu{};
    ecu.ecu_id         = 0xA0B1;
    ecu.max_rpm        = 7500.0f;
    ecu.idle_rpm       = 800.0f;
    ecu.fuel_trim      = 1.02f;
    ecu.cylinder_count = 4;
    ecu.turbo_enabled  = true;

    // EN: Show that the struct address is 64-byte aligned
    // TR: Yapı adresinin 64 bayta hizalı olduğunu göster
    auto addr = reinterpret_cast<std::uintptr_t>(&ecu);
    std::cout << "  EcuConfig address  : 0x" << std::hex << addr << std::dec << "\n";
    std::cout << "  64-byte aligned?   : " << ((addr % 64 == 0) ? "YES" : "NO") << "\n";
    std::cout << "  ecu_id             : 0x" << std::hex << ecu.ecu_id << std::dec << "\n";
    std::cout << "  max_rpm            : " << ecu.max_rpm << "\n";
    std::cout << "  idle_rpm           : " << ecu.idle_rpm << "\n";
    std::cout << "  fuel_trim          : " << ecu.fuel_trim << "\n";
    std::cout << "  cylinder_count     : " << static_cast<int>(ecu.cylinder_count) << "\n";
    std::cout << "  turbo_enabled      : " << std::boolalpha << ecu.turbo_enabled << "\n";

    std::cout << "\n============================================\n";
    std::cout << " All demos complete.\n";
    std::cout << "============================================\n";

    return 0;
}
```

► 05. Compilation Pipeline

module_07_under_the_hood/05_compilation_pipeline.cpp

Konu: EN: C++ Compilation Pipeline — From Source to Executable

Teori / Theory

THEORY

The C++ build process transforms human-readable source into machine code through four distinct stages. Understanding this pipeline is critical for diagnosing linker errors, managing build times, and writing robust firmware for automotive ECUs.

C++ derleme süreci, insan tarafından okunabilir kaynağı makine koduna dört ayrı aşamada dönüştürür. Bu hattı anlamak, bağlayıcı hatalarını teşhis etmek, derleme sürelerini yönetmek ve otomotiv ECU'ları için sağlam yazılım geliştirmek açısından kritiktir.

── Stage 1: PREPROCESSING ── The preprocessor (cpp) runs before actual compilation. It handles: • #include — textual inclusion of header files • #define / #ifdef / #ifndef — macro expansion & conditional compilation • #pragma — compiler-specific directives • __FILE__, __LINE__, __func__ — predefined macros In automotive, conditional compilation selects HW-specific code paths (e.g., ECU_VARIANT_A vs ECU_VARIANT_B) from one codebase.

── Aşama 1: ÖN İŞLEME ── Ön işlemci (cpp), gerçek derlemeden önce çalışır: • #include — başlık dosyalarının metin olarak dahil edilmesi • #define / #ifdef / #ifndef — makro açılımı ve koşullu derleme • #pragma — derleyiciye özel direktifler • __FILE__, __LINE__, __func__ — önceden tanımlı makrolar Otomotivde koşullu derleme, tek kod tabanından donanıma özel yolları seçer.

── Stage 2: COMPILATION ── The compiler (cc1plus) translates each preprocessed translation unit (TU) into an object file (.o). Each .cpp file is ONE translation unit. Templates are instantiated per-TU, which can cause code bloat if many TUs instantiate the same template with the same types.

── Aşama 2: DERLEME ── Derleyici, ön işlenmiş her çeviri birimini (TU) nesne dosyasına (.o) dönüştürür. Her .cpp dosyası BİR çeviri birimidir. Şablonlar her TU'da ayrı ayrı örneklenir; bu, aynı şablonun birden fazla TU'da kullanılması durumunda kod şişmesine yol açabilir.

── Stage 3: LINKING ── The linker (ld) merges object files and resolves symbol references. The One Definition Rule (ODR) states: every used entity must have exactly one definition across all TUs. Violations cause undefined behavior or linker errors. `extern` declares a symbol without defining it; the definition lives elsewhere. `inline` functions/variables are ODR-exempt — the linker picks one copy. C++17 inline variables enable header-only definitions of global state. `static` at file/namespace scope gives internal linkage — the symbol is invisible outside its TU.

── Aşama 3: BAĞLAMA ── Bağlayıcı (ld), nesne dosyalarını birleştirir ve sembol referanslarını çözümler. Tek Tanım Kuralı (ODR): her kullanılan varlığın tüm TU'lar boyunca tam olarak bir tanımı olmalıdır. `extern` bir sembolü tanımlamadan bildirir. `inline` fonksiyonlar/değişkenler ODR'den muaftır — bağlayıcı tek bir kopya seçer. C++17 inline değişkenler, başlık-yalnız tanımları mümkün kılar. Dosya/ad-alanı kapsamında `static`, iç bağlantı sağlar.

── Stage 4: LOADING ── The OS loader maps the executable into memory, resolves dynamic library symbols, sets up the stack, and transfers control to _start → main(). In automotive, ECU firmware is often flashed directly to ROM; the "loader" is the bootloader that copies code to execution addresses.

── Aşama 4: YÜKLEME ── İşletim sistemi yükleyicisi çalıştırılabilir dosyayı belleğe eşler, dinamik kütüphane sembollerini çözümler, yığını kurar ve kontrolü _start → main()'e aktarır. Otomotivde ECU yazılımı genellikle doğrudan ROM'a yazılır; "yükleyici" kodu çalıştırma adreslerine kopyalayan önyükleyicidir.

── Header-Only Libraries ── Placing all code in headers is convenient (no separate .cpp to compile) but can increase build times and binary size (code bloat) since every TU that includes the header gets its own copy of non-inline definitions.

── Başlık-Yalnız Kütüphaneler ── Tüm kodu başlık dosyalarına koymak pratiktir ancak derleme süresini ve ikili boyutunu artırabilir; başlığı dahil eden her TU, inline olmayan tanımların kendi kopyasını alır.

CPPREF Depth: One Definition Rule (ODR) — The Silent Killer of Large Projects

The One Definition Rule states every entity must have exactly one definition across all Translation Units. An ODR violation is undefined behavior — and no diagnostic is required! Inline functions must have identical token-for-token definitions in every TU; otherwise the linker silently picks one and discards the rest. Header-only libraries rely on the inline/template ODR exemption: the definition may appear in multiple TUs provided every copy is identical. In C++17, `inline` on a variable or function means "allow multiple definitions" — it does NOT mean "please inline this call". Link-Time Optimization (LTO) can detect some ODR violations that normal linking misses. C++20 Modules eliminate the textual inclusion model entirely, making ODR violations structurally impossible for module-exported entities.

CPPREF Derinlik: Tek Tanım Kuralı (ODR) — Büyük Projelerin Sessiz Katili

Tek Tanım Kuralı her varlığın tüm Çeviri Birimlerinde tam olarak bir tanımı olmasını gerektirir. ODR ihlali tanımsız davranıştır — ve derleyicinin uyarı vermesi zorunlu DEĞİLDİR! Inline fonksiyonlar her TU'da birebir aynı tanıma sahip olmalıdır; aksi halde bağlayıcı sessizce birini seçer. Başlık-yalnız kütüphaneler inline/template ODR muafiyetine dayanır. C++17'de `inline` bir değişken veya fonksiyonda "birden fazla tanıma izin ver" demektir — "bu çağrıyı satır içi yap" değil. LTO bazı ODR ihlallerini yakalayabilir. C++20 Modülleri dâhil etme modelini tamamen kaldırarak modül-dışa-aktarılan varlıklar için ODR ihlallerini yapısal olarak imkânsız kılar.

Kaynak Kod / Source Code

```cpp
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for text handling
#include <sstream>    // EN: std::ostringstream for string building
#include <cstdint>    // EN: Fixed-width integer types (uint16_t, etc.)
#include <vector>     // EN: Dynamic array container
#include <array>      // EN: Fixed-size array container
#include <typeinfo>   // EN: typeid for RTTI demonstration

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 1: INLINE CONSTEXPR AUTOMOTIVE CONFIGURATION
// ═══════════════════════════════════════════════════════════════════════════════

// EN: inline constexpr values can be defined in headers safely (ODR-exempt).
// TR: inline constexpr değerler, başlık dosyalarında güvenle tanımlanabilir (ODR muaf).

// ─── 1.1 Engine Parameters ─────────────────────────────────────────────────

inline constexpr uint16_t MAX_RPM           = 7500;
// EN: Maximum engine revolutions per minute
// TR: Maksimum motor devir sayısı (d/dk)

inline constexpr uint16_t IDLE_RPM          = 800;
// EN: Idle engine speed target
// TR: Rölanti devir hedefi

inline constexpr float    MIN_VOLTAGE       = 9.0f;
// EN: Minimum battery voltage before ECU triggers low-voltage mode
// TR: ECU'nun düşük gerilim moduna geçtiği minimum akü voltajı

inline constexpr float    MAX_COOLANT_TEMP  = 110.0f;
// EN: Coolant temperature threshold for overheating alarm (°C)
// TR: Aşırı ısınma alarmı için soğutma suyu sıcaklık eşiği (°C)

inline constexpr uint8_t  NUM_CYLINDERS     = 4;
// EN: Number of engine cylinders
// TR: Motor silindir sayısı

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: PREPROCESSOR — CONDITIONAL COMPILATION
// ═══════════════════════════════════════════════════════════════════════════════

// EN: ECU_DEBUG enables verbose diagnostics; undef for production builds.
// TR: ECU_DEBUG ayrıntılı tanılamayı etkinleştirir; üretim derlemelerinde kaldırılır.

#define ECU_DEBUG

#ifdef ECU_DEBUG
inline constexpr bool IS_DEBUG_BUILD = true;
#else
inline constexpr bool IS_DEBUG_BUILD = false;
#endif

// ─── 2.1 ECU_LOG Macro ─────────────────────────────────────────────────────

// EN: Logging macro using __FILE__, __LINE__, __func__ predefined macros.
// TR: __FILE__, __LINE__, __func__ önceden tanımlı makrolarını kullanan günlük makrosu.

#define ECU_LOG(msg)                                                        \
    do {                                                                    \
        if constexpr (IS_DEBUG_BUILD) {                                     \
            std::cout << "[ECU_LOG] " << __FILE__                           \
                      << ":" << __LINE__                                    \
                      << " (" << __func__ << ") — " << msg << "\n";        \
        }                                                                   \
    } while (0)
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: STATIC AT NAMESPACE SCOPE — INTERNAL LINKAGE
// ═══════════════════════════════════════════════════════════════════════════════

// EN: 'static' at namespace scope gives internal linkage — the variable exists only in this
// translation unit. Another TU can define its own copy without causing an ODR violation.
// TR: Ad-alanı kapsamında 'static', iç bağlantı sağlar — değişken yalnızca bu çeviri biriminde
// bulunur. Başka bir TU, ODR ihlali olmadan kendi kopyasını tanımlayabilir.

static int s_ecu_boot_count = 0;

// EN: Each TU that includes such a definition gets its own independent copy.
// TR: Böyle bir tanımı içeren her TU, kendi bağımsız kopyasını alır.

static void increment_boot_count() {
    ++s_ecu_boot_count;
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: TEMPLATE INSTANTIATION — CODE GENERATION & BLOAT
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Each unique template instantiation generates its own copy of code. If SensorBuffer<float,
// 64> and SensorBuffer<double, 64> are both used, the compiler emits TWO complete class
// implementations.
// TR: Her benzersiz şablon örneklemesi kendi kod kopyasını üretir. SensorBuffer<float, 64> ve
// SensorBuffer<double, 64> birlikte kullanılırsa, derleyici İKİ tam sınıf gerçeklemesi üretir.

template <typename T, std::size_t Capacity>
class SensorBuffer {
public:
    // EN: Push a sensor reading into the circular buffer.
    // TR: Dairesel tampona bir sensör okuma değeri ekler.
    void push(T value) {
        buffer_[write_idx_ % Capacity] = value;
        ++write_idx_;
    }

    // EN: Get the most recent reading.
    // TR: En son okuma değerini döndürür.
    T latest() const {
        return (write_idx_ > 0)
            ? buffer_[(write_idx_ - 1) % Capacity]
            : T{};
    }

    // EN: Return how many elements have been pushed (may exceed Capacity).
    // TR: Kaç öğe eklendiğini döndürür (Capacity'yi aşabilir).
    std::size_t total_writes() const { return write_idx_; }

    // EN: Return the fixed capacity of this buffer.
    // TR: Bu tamponun sabit kapasitesini döndürür.
    static constexpr std::size_t capacity() { return Capacity; }

private:
    std::array<T, Capacity> buffer_{};
    std::size_t write_idx_ = 0;
};

// ─── 4.1 Extern Declaration Example ──────────────────────────────────────────

// EN: 'extern' keyword declares a variable/function defined elsewhere. In a multi-TU project,
// the definition would be in exactly one .cpp file. Here we provide the definition immediately
// (single-TU demo).
// TR: 'extern' anahtar kelimesi, başka yerde tanımlanan bir değişkeni bildirir. Çok TU'lu bir
// projede tanım tam olarak bir .cpp dosyasında olur. Burada tanımı hemen sağlıyoruz (tek TU
// demosu).

extern const char* const ECU_FIRMWARE_VERSION;
const char* const ECU_FIRMWARE_VERSION = "v3.2.1-AUTOSAR";
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 5: MAIN — DEMONSTRATIONS
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════╗\n";
    std::cout << "│  Module 07 — 05: C++ Compilation Pipeline             │\n";
    std::cout << "│  EN: From Source Code to Running Binary               │\n";
    std::cout << "│  TR: Kaynak Koddan Çalışan İkili Dosyaya              │\n";
    std::cout << "╚═══════════════════════════════════════════════════════╝\n\n";

    // ── Demo 1: Preprocessor Macros & Conditional Compilation ──────────────

    std::cout << "── Demo 1: Preprocessor & Conditional Compilation ──\n";
    // EN: Show whether ECU_DEBUG is active at compile time.
    // TR: Derleme zamanında ECU_DEBUG'ın aktif olup olmadığını gösterir.

    std::cout << "  IS_DEBUG_BUILD = " << std::boolalpha << IS_DEBUG_BUILD << "\n";

#ifdef ECU_DEBUG
    std::cout << "  [ECU_DEBUG] Diagnostic mode ACTIVE — extra checks enabled.\n";
    // EN: In production, #undef ECU_DEBUG removes this entire block.
    // TR: Üretimde, #undef ECU_DEBUG bu bloğun tamamını kaldırır.
#endif

    std::cout << "  Firmware: " << ECU_FIRMWARE_VERSION << "\n\n";

    // ── Demo 2: ECU_LOG Macro with File/Line/Func Info ─────────────────────

    std::cout << "── Demo 2: ECU_LOG Macro ──\n";
    // EN: ECU_LOG expands __FILE__, __LINE__, __func__ at the call site.
    // TR: ECU_LOG, çağrı noktasında __FILE__, __LINE__, __func__ değerlerini açar.

    ECU_LOG("Engine control module initialized");
    ECU_LOG("CAN bus ready — baud rate 500kbps");
    ECU_LOG("Sensor calibration complete");
    std::cout << "\n";

    // ── Demo 3: inline constexpr Configuration Values ──────────────────────

    std::cout << "── Demo 3: Inline Constexpr Configuration ──\n";
    // EN: These values are compile-time constants with no storage overhead.
    // TR: Bu değerler, depolama maliyeti olmayan derleme zamanı sabitleridir.

    std::cout << "  MAX_RPM          = " << MAX_RPM << " rpm\n";
    std::cout << "  IDLE_RPM         = " << IDLE_RPM << " rpm\n";
    std::cout << "  MIN_VOLTAGE      = " << MIN_VOLTAGE << " V\n";
    std::cout << "  MAX_COOLANT_TEMP = " << MAX_COOLANT_TEMP << " °C\n";
    std::cout << "  NUM_CYLINDERS    = " << static_cast<int>(NUM_CYLINDERS) << "\n";

    // EN: Simulate an RPM range check using the constants.
    // TR: Sabitler kullanarak bir RPM aralığı denetimi simüle eder.
    uint16_t current_rpm = 3200;
    if (current_rpm >= IDLE_RPM && current_rpm <= MAX_RPM) {
        std::cout << "  RPM " << current_rpm << " is within operating range.\n";
    }
    std::cout << "\n";

    // ── Demo 4: Static Namespace-Scope Variable Behavior ───────────────────

    std::cout << "── Demo 4: Static Internal Linkage ──\n";
    // EN: s_ecu_boot_count has internal linkage — it only exists in this TU. In a multi-file
    // project, each TU would have its own independent copy.
    // TR: s_ecu_boot_count iç bağlantıya sahiptir — yalnızca bu TU'da bulunur. Çok dosyalı bir
    // projede her TU kendi bağımsız kopyasına sahip olur.

    std::cout << "  Boot count before: " << s_ecu_boot_count << "\n";
    increment_boot_count();
    increment_boot_count();
    increment_boot_count();
    std::cout << "  Boot count after 3 increments: " << s_ecu_boot_count << "\n";
    std::cout << "  (Each TU would maintain its own static copy)\n\n";
```

```

    // ─── Demo 5: Template Instantiation & Code Bloat ──────────────────────────────

    std::cout << "── Demo 5: Template Instantiation Sizes ──\n";
    // EN: Different instantiations create separate code — observe sizes.
    // TR: Farklı örneklemeler ayrı kod oluşturur — boyutları gözlemleyin.

    SensorBuffer<float, 32>    rpm_buffer;
    SensorBuffer<double, 32>   temp_buffer;
    SensorBuffer<uint16_t, 64> voltage_buffer;
    SensorBuffer<float, 128>   accel_buffer;

    // EN: Push some sample automotive sensor data.
    // TR: Örnek otomotiv sensör verisi ekler.
    rpm_buffer.push(3200.0f);
    rpm_buffer.push(3450.5f);
    temp_buffer.push(92.3);
    voltage_buffer.push(1380);
    accel_buffer.push(0.98f);

    std::cout << "  sizeof SensorBuffer<float,   32>  = "
              << sizeof(rpm_buffer)     << " bytes\n";
    std::cout << "  sizeof SensorBuffer<double,  32>  = "
              << sizeof(temp_buffer)    << " bytes\n";
    std::cout << "  sizeof SensorBuffer<uint16_t,64>  = "
              << sizeof(voltage_buffer) << " bytes\n";
    std::cout << "  sizeof SensorBuffer<float,  128>  = "
              << sizeof(accel_buffer)   << " bytes\n";

    std::cout << "\n  Latest RPM reading:     " << rpm_buffer.latest() << " rpm\n";
    std::cout << "  Latest coolant temp:    " << temp_buffer.latest() << " °C\n";
    std::cout << "  Latest voltage (raw):   " << voltage_buffer.latest() << "\n";
    std::cout << "  Accel buffer capacity:  " << accel_buffer.capacity() << "\n";

    // EN: Each unique <T, N> combination generates a full class in the object file. With 4
    // instantiations, 4 copies of push(), latest(), etc. exist.
    // TR: Her benzersiz <T, N> kombinasyonu nesne dosyasında tam bir sınıf üretir. 4 örnekleme
    // ile push(), latest() vb. fonksiyonların 4 kopyası bulunur.

    std::cout << "\n  4 instantiations = 4× full class code in the object file.\n";
    std::cout << "  This is why header-only template libraries can cause bloat.\n";

    std::cout << "\n═══════════════════════════════════════════════════════════\n";
    std::cout << "  Pipeline: .cpp → Preprocessor → Compiler → Linker → Binary\n";
    std::cout << "                        ↑ macros     ↑ .o files  ↑ executable\n";
    std::cout << "═══════════════════════════════════════════════════════════\n";

    return 0;
}
```

► 06. Abi And Name Mangling

module_07_under_the_hood/06_abi_and_name_mangling.cpp

Konu: EN: ABI (Application Binary Interface) and C++ Name Mangling — binary contracts, symbol

Teori / Theory

THEORY

ABI (Application Binary Interface) — The Binary-Level Contract

ABI (Uygulama İkili Arayüzü) — İkili Düzey Sözleşme

An ABI defines how compiled code interacts at the binary level: calling conventions, register usage, struct layout, vtable ordering, and how function names become symbols. Linked object files must agree on the ABI or linker errors / crashes will follow.

ABI, derlenmiş kodun ikili düzeyde nasıl etkileştiğini tanımlar: çağrı kuralları, yazmaç kullanımı, struct düzeni, vtable sıralaması ve fonksiyon adlarının sembollere dönüşümü. Bağlanan nesne dosyaları ABI üzerinde anlaşmalıdır.

Name Mangling — Encoding Overloaded Signatures

İsim Karıştırma — Aşırı Yüklenmiş İmzaları Kodlama

The compiler "mangles" each function into a unique symbol. Under the Itanium ABI: foo(int) → _Z3fooi | foo(int,float) → _Z3fooif Each overload gets a distinct mangled name so the linker resolves the correct one.

Derleyici her fonksiyonu benzersiz bir sembole "karıştırır". Itanium ABI altında: foo(int) → _Z3fooi | foo(int,float) → _Z3fooif Her aşırı yükleme farklı bir karıştırılmış isim alır; bağlayıcı doğru olanı çözümler.

extern "C" — Disabling Name Mangling for Interoperability

extern "C" — Birlikte Çalışabilirlik İçin İsim Karıştırmayı Devre Dışı Bırakma

`extern "C" { ... }` tells the compiler to use C linkage: no mangling. Essential for C interop, shared libraries (.so/.dll), and dlopen/dlsym plugin APIs.

`extern "C" { ... }` derleyiciye C bağlantısı kullanmasını söyler: karıştırma yok. C birlikte çalışma, paylaşımlı kütüphaneler ve dlopen/dlsym eklenti API'leri için gerekli.

Compiler ABI Incompatibility

Derleyici ABI Uyumsuzluğu

MSVC, GCC, and Clang each use different mangling schemes. Mixing compilers (or major versions) can cause linker failures. In automotive ECU software, the entire toolchain must be locked to ensure ABI consistency.

MSVC, GCC ve Clang farklı karıştırma şemaları kullanır. Derleyici karışımı bağlayıcı hatalarına neden olabilir. Otomotiv ECU yazılımında araç zinciri kilitlenmelidir.

Tools: `c++filt` demangles symbols (echo _Z3fooi | c++filt → foo(int)). `nm` lists symbols in object files. Combine: nm mylib.so | c++filt

Araçlar: `c++filt` sembolleri çözümler. `nm` sembolleri listeler.

Shared Libraries: exposing a C ABI (extern "C") for .so files ensures any compiler version can link your functions — standard for UDS, bootloaders, CAN layers.

Paylaşımlı Kütüphaneler: .so dosyaları için C ABI sunmak, herhangi bir derleyici sürümünün fonksiyonlarınızı bağlamasını sağlar — UDS, önyükleyiciler, CAN katmanları.

CPPREF Depth: ABI Stability and the Itanium C++ ABI

Most Linux/Unix compilers (GCC, Clang) follow the Itanium C++ ABI, which standardises name mangling, vtable layout, exception handling, and RTTI representation. Name mangling encodes: namespace, enclosing class, function name, parameter types (including cv-qualifiers and references), and template arguments. `extern "C"` suppresses mangling so C code can link the symbol directly. ABI breaks happen when: struct/class layout changes (adding or reordering members), virtual function order changes, or the exception model changes. Compiling with `-fvisibility=hidden` and explicitly exporting only public symbols (`__attribute__((visibility("default")))`) produces smaller and faster shared libraries — fewer relocations, faster `dlopen()`. This is the standard practice for plugin architectures.

CPPREF Derinlik: ABI Kararlılığı ve Itanium C++ ABI'si

Çoğu Linux/Unix derleyicisi (GCC, Clang) isim bozma, vtable düzeni, istisna işleme ve RTTI temsilini standartlaştıran Itanium C++ ABI'sini takip eder. İsim bozma şunu kodlar: ad alanı, kapsayan sınıf, fonksiyon adı, parametre tipleri (cv-niteleyiciler ve referanslar dahil) ve şablon argümanları. `extern "C"` bozulmayı bastırır, böylece C kodu sembolü doğrudan bağlayabilir. ABI kırılmaları şu durumlarda olur: yapı/sınıf düzeni değiştiğinde, sanal fonksiyon sırası değiştiğinde veya istisna modeli değiştiğinde. `-fvisibility=hidden` ile derleyip yalnızca genel sembolleri açıkça dışa aktarmak daha küçük ve hızlı paylaşımlı kütüphaneler üretir — daha az yer değiştirme, daha hızlı `dlopen()`.

Kaynak Kod / Source Code

```cpp
#include <iostream>    // EN: Standard I/O for console output
#include <string>      // EN: std::string for sensor name lookups
#include <functional>  // EN: std::function for modern callback wrappers
#include <typeinfo>    // EN: typeid for RTTI-based type name inspection
#include <cstdio>      // EN: C-style printf for extern "C" callback demos
#include <array>       // EN: std::array for plugin dispatch table

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: C++ Overloaded Functions — Name Mangling in Action
// ═══════════════════════════════════════════════════════════════════════════

// EN: Three overloads share the name "readSensor" — the compiler mangles each into a unique
// symbol.
// TR: Üç aşırı yükleme "readSensor" adını paylaşır — derleyici her birini benzersiz bir sembole
// karıştırır.

// ── 1a. Read sensor by numeric ID ──────────────────────────────────────
int readSensor(int id) {
    // EN: Simulates reading raw ADC value from sensor channel
    // TR: Sensör kanalından ham ADC değeri okumayı simüle eder
    int raw_value = 1000 + id * 42;
    std::cout << "  [readSensor(int)] Channel " << id
              << " → raw value: " << raw_value << "\n";
    return raw_value;
}

// ── 1b. Read sensor by ID with calibration factor ──────────────────────
float readSensor(int id, float calibration) {
    // EN: Applies calibration offset to the sensor reading
    // TR: Sensör okumasına kalibrasyon ofseti uygular
    float calibrated = static_cast<float>(1000 + id * 42) * calibration;
    std::cout << "  [readSensor(int, float)] Channel " << id
              << " with cal=" << calibration
              << " → calibrated: " << calibrated << "\n";
    return calibrated;
}

// ── 1c. Read sensor by name (string lookup) ────────────────────────────
int readSensor(const std::string& name) {
    // EN: Simulates named sensor lookup (e.g., from config database)
    // TR: İsimli sensör aramasını simüle eder (ör. yapılandırma veritabanından)
    int resolved_value = static_cast<int>(name.length()) * 100 + 55;
    std::cout << "  [readSensor(string)] Sensor \"" << name
              << "\" → resolved value: " << resolved_value << "\n";
    return resolved_value;
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: extern "C" Block — C-Compatible ECU API Stubs
// ═══════════════════════════════════════════════════════════════════════════

// EN: extern "C" prevents name mangling — symbols are exported as plain C names.
// TR: extern "C" isim karıştırmayı engeller — semboller düz C adları olarak dışa aktarılır.

extern "C" {

int ecu_init() {
    // EN: Initialize ECU hardware abstraction layer (stub)
    // TR: ECU donanım soyutlama katmanını başlat (saplama)
    std::cout << "  [ecu_init] ECU initialized (firmware v2.4.1)\n";
    return 0;
}

void ecu_shutdown() {
```

```
    // EN: Graceful ECU shutdown — flush CAN buffers, store NVM
    // TR: Düzgün ECU kapatma — CAN arabelleklerini boşalt, NVM'yi kaydet
    std::cout << "  [ecu_shutdown] ECU shutdown complete\n";
}

int ecu_read_sensor(int channel) {
    // EN: Read raw sensor value via C-linkage API (no mangling)
    // TR: C bağlantılı API ile ham sensör değerini oku (karıştırma yok)
    int value = 2048 + channel * 17;
    std::cout << "  [ecu_read_sensor] Channel " << channel
              << " → value: " << value << "\n";
    return value;
}

} // extern "C"

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 3: C-Style Callback Pattern — Diagnostic Service
// ═══════════════════════════════════════════════════════════════════════════

// EN: Function pointer typedef for diagnostic callbacks (C-compatible signature)
// TR: Tanılama geri çağrıları için fonksiyon işaretçisi typedef'i (C uyumlu imza)
typedef void (*DiagCallback)(int code, const char* msg);

// EN: Global callback storage (simulates registration in a diagnostic manager)
// TR: Global geri çağrı deposu (tanı yöneticisinde kayıt işlemini simüle eder)
static DiagCallback g_diag_callback = nullptr;

void registerCallback(DiagCallback cb) {
    // EN: Register a diagnostic callback function pointer
    // TR: Bir tanılama geri çağrı fonksiyon işaretçisini kaydet
    g_diag_callback = cb;
    std::cout << "  [registerCallback] Callback registered at address: "
              << reinterpret_cast<void*>(cb) << "\n";
}

void triggerDiagnostic(int fault_code, const char* description) {
    // EN: Trigger the registered diagnostic callback if available
    // TR: Varsa kayıtlı tanılama geri çağrısını tetikle
    if (g_diag_callback) {
        g_diag_callback(fault_code, description);
    } else {
        std::cout << "  [triggerDiagnostic] No callback registered!\n";
    }
}

// EN: Sample callback implementations (would be in separate modules in production)
// TR: Örnek geri çağrı uygulamaları (üretimde ayrı modüllerde olurdu)
void onDtcDetected(int code, const char* msg) {
    std::printf("  [DTC Handler] Fault 0x%04X: %s\n", code, msg);
}

void onDtcLogged(int code, const char* msg) {
    std::printf("  [DTC Logger]  Fault 0x%04X stored to NVM: %s\n", code, msg);
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 4: Simulated Plugin System — Function Pointer Dispatch Table
// ═══════════════════════════════════════════════════════════════════════════

// EN: Plugin entry point signature — compatible with dlsym/GetProcAddress patterns
// TR: Eklenti giriş noktası imzası — dlsym/GetProcAddress kalıplarıyla uyumlu
using PluginFunc = int (*)(const char* command);
```

```cpp

// EN: Simulated plugin implementations (in real systems, loaded from .so files)
// TR: Simüle edilmiş eklenti uygulamaları (gerçek sistemlerde .so dosyalarından yüklenir)
int pluginDiagnostics(const char* command) {
    std::cout << "  [Plugin:Diagnostics] Executing: " << command << "\n";
    return 0;
}

int pluginFlasher(const char* command) {
    std::cout << "  [Plugin:Flasher] Executing: " << command << "\n";
    return 0;
}

int pluginCalibration(const char* command) {
    std::cout << "  [Plugin:Calibration] Executing: " << command << "\n";
    return 0;
}

// EN: Plugin dispatch table — maps slot indices to function pointers
// TR: Eklenti dağıtım tablosu — yuva indekslerini fonksiyon işaretçilerine eşler
struct PluginEntry {
    const char*  name;
    PluginFunc   func;
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstration Scenarios
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════════\n";
    std::cout << " ABI & Name Mangling — Automotive ECU Examples\n";
    std::cout << "═══════════════════════════════════════════════════════\n\n";

    // ─── Demo 1: Overloaded Function Calls ─────────────────────────────────
    // EN: Each call resolves to a different mangled symbol at link time
    // TR: Her çağrı bağlama zamanında farklı bir karıştırılmış sembole çözümlenir
    std::cout << "── Demo 1: Overloaded readSensor() calls ──\n";
    readSensor(3);
    readSensor(3, 1.05f);
    readSensor(std::string("throttle_position"));
    std::cout << "\n";

    // ─── Demo 2: extern "C" Function Calls ─────────────────────────────────
    // EN: These symbols have no mangling — plain C names in the symbol table
    // TR: Bu sembollerin karıştırması yok — sembol tablosunda düz C adları
    std::cout << "── Demo 2: extern \"C\" ECU API calls ──\n";
    int init_status = ecu_init();
    std::cout << "  Init status: " << init_status << "\n";
    ecu_read_sensor(0);
    ecu_read_sensor(5);
    ecu_shutdown();
    std::cout << "\n";

    // ─── Demo 3: Register and Trigger C-Style Callbacks ────────────────────
    // EN: Function pointer callbacks — standard pattern for C-compatible diagnostic interfaces
    // TR: Fonksiyon işaretçisi geri çağrıları — C uyumlu tanı arayüzleri için standart kalıp
    std::cout << "── Demo 3: Diagnostic callback registration ──\n";
    registerCallback(onDtcDetected);
    triggerDiagnostic(0xC410, "Catalyst temperature above threshold");

    registerCallback(onDtcLogged);
    triggerDiagnostic(0xB112, "CAN bus off — ECU communication lost");
    std::cout << "\n";
```

```cpp

    // ─── Demo 4: Function Pointer Table (Simulated Plugin Dispatch) ──────────────────────────
    // EN: In real systems, these pointers come from dlsym() on loaded .so files.
    // TR: Gerçek sistemlerde bu işaretçiler yüklenen .so dosyalarındaki dlsym()'den gelir.
    std::cout << "── Demo 4: Plugin dispatch table ──\n";
    std::array<PluginEntry, 3> plugins = {{
        {"Diagnostics",  pluginDiagnostics},
        {"Flasher",      pluginFlasher},
        {"Calibration",  pluginCalibration}
    }};

    for (const auto& plugin : plugins) {
        std::cout << "  Dispatching to plugin [" << plugin.name << "]:\n";
        int result = plugin.func("self_test");
        std::cout << "    → Return code: " << result << "\n";
    }
    std::cout << "\n";

    // ─── Demo 5: typeid Names of Different Overloads ─────────────────────────────────────────
    // EN: typeid reveals the compiler's internal type representation for each overload.
    // TR: typeid her aşırı yükleme için derleyicinin dahili tür temsilini gösterir.
    std::cout << "── Demo 5: typeid of overloaded function pointers ──\n";
    int   (*fp_int)(int)                    = readSensor;
    float (*fp_int_float)(int, float)       = readSensor;
    int   (*fp_string)(const std::string&)  = readSensor;

    std::cout << "  readSensor(int)          → typeid: "
              << typeid(fp_int).name() << "\n";
    std::cout << "  readSensor(int, float)   → typeid: "
              << typeid(fp_int_float).name() << "\n";
    std::cout << "  readSensor(string)       → typeid: "
              << typeid(fp_string).name() << "\n";
    std::cout << "\n  Addresses (all distinct due to mangling):\n";
    std::cout << "    readSensor(int):        " << reinterpret_cast<void*>(fp_int) << "\n";
    std::cout << "    readSensor(int, float): " << reinterpret_cast<void*>(fp_int_float) << "\n";
    std::cout << "    readSensor(string):     " << reinterpret_cast<void*>(fp_string) << "\n";

    std::cout << "\n═══════════════════════════════════════════════════════\n";
    std::cout << " Tip: compile and inspect mangled symbols with:\n";
    std::cout << "   nm ./06_abi_and_name_mangling | grep readSensor\n";
    std::cout << "   nm ./06_abi_and_name_mangling | grep ecu_ \n";
    std::cout << "   nm ./06_abi_and_name_mangling | c++filt | grep readSensor\n";
    std::cout << "═══════════════════════════════════════════════════════\n";

    return 0;
}
```

Modül 8: C++ Core Guidelines — Üst Düzey Kurallar

Üst Düzey Kurallar

6 kaynak dosya · Standart: C++17

C++17

► 01. Philosophy And Interfaces

module_08_core_guidelines/01_philosophy_and_interfaces.cpp

Konu: C++ Core Guidelines: Philosophy & Interfaces / C++ Çekirdek Kuralları: Felsefe ve

Teori / Theory

INTRODUCTION: What are C++ Core Guidelines?

The "C++ Core Guidelines" are a set of tried-and-true rules about coding in C++. Written by Bjarne Stroustrup (creator of C++) and Herb Sutter, they aim to help developers write MUCH Safer, Cleaner, and Faster modern C++.

GİRİŞ: C++ Core Guidelines Nedir?

"C++ Core Guidelines", C++'ın yaratıcısı Bjarne Stroustrup ve ISO C++ Komitesi tarafından yazılan, Modern C++'ın nasıl DOĞRU, GÜVENLİ ve PERFORMANSLI yazılacağını anlatan resmi bir anayasadır. Sektördeki en iyi Mimarlar bu kurallara uyar.

P.1: Express ideas directly in code

Compilers don't read comments. If you have to write a comment to explain WHAT a loop does, name your variables better or use Standard Library (STL) algorithms.

P.1: Fikirlerinizi Koda Doğrudan Dökün

Derleyiciler yorum satırlarını (comments) okumazlar. Bir döngünün NE yaptığını anlatmak için satırlarca yoruma ihtiyaç duyuyorsanız, kodun iskeleti kötüdür. Standart Kütüphane (STL) metodlarını (Örn: std::find_if) kullanın. İşin NİYETİNİ belli edin!

P.4 & P.5: Compile-Time Checking

"Prefer compile-time checking to run-time checking." If you can calculate or check a rule during COMPILATION, do it! Use `constexpr` and `static_assert()`. A bug caught at compile-time costs $0. At runtime, it causes a crash.

Derleme Zamanı Kontrolleri

"Derleme zamanını Çalışma zamanına (Runtime) tercih edin." Eğer bir değeri oyun çalışmadan (Build alırken) absolut hesaplayabiliyorsan, `constexpr` ve `static_assert()` kullan! Derlemede yakalanan bir hatanın maliyeti SIFIRDIR, çalışırken yakalanan bir hata programı (veya sunucuyu) çökertir.

CPPREF Depth: Type Safety as Zero-Cost Abstraction — Strong Typedefs

C++ Core Guidelines I.4: "Make interfaces precisely and strongly typed". Primitive obsession — using bare `int` for meters, seconds, and IDs — lets the compiler silently accept nonsensical expressions like `meters + seconds`. Strong typedefs (a thin wrapper class with explicit constructors) enforce correctness at ZERO runtime cost because the optimizer elides the wrapper entirely. `enum class` prevents implicit conversions between unrelated enumerations. `[[nodiscard]]` forces callers to use (not ignore) a return value — essential for error codes. In C++20, Concepts formalize interface contracts: `template<Sortable T>` replaces unnamed SFINAE with readable, diagnosable constraints.

CPPREF Derinlik: Sıfır Maliyetli Soyutlama Olarak Tip Güvenliği — Güçlü Typedef'ler

C++ Çekirdek Kılavuzu I.4: "Arayüzleri kesin ve güçlü tipli yapın". İlkel tip takıntısı — metre, saniye ve kimlik için çıplak `int` kullanmak — derleyicinin `metre + saniye` gibi anlamsız ifadeleri sessizce kabul etmesine yol açar. Güçlü typedef'ler (açık kuruculu ince sarmalayıcı sınıf) SIFIR çalışma zamanı maliyetiyle doğruluğu zorlar çünkü optimizer sarmalayıcıyı tamamen eler. `enum class` ilgisiz numaralandırmalar arası örtük dönüşümü engeller. `[[nodiscard]]` çağıranı dönüş değerini kullanmaya zorlar. C++20'de Kavramlar arayüz sözleşmelerini resmileştirir.

Kaynak Kod / Source Code

```cpp
#include <algorithm> // for std::find_if
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------------------------
// [P.1] Express Intent (Kötü vs İyi Örnek)
// ---------------------------------------------------------------------------------------------
class ECU {
public:
  std::string name;
  bool isOnline = true;
};

void checkECUStatus_Bad(const std::vector<ECU> &ecus, const std::string &targetName) {
  // BAD (Kötü): C-Style manual loop. Hard to catch the intent at a glance.
  // TR: KÖTÜ: Geleneksel döngü. Okuması ve arkasındaki "Niyeti" anlamak vakit kaybettirir.
  bool found = false;
  for (size_t i = 0; i < ecus.size(); ++i) {
    if (ecus[i].name == targetName) {
      found = true;
      break;
    }
  }
  (void)found; // EN: Suppress — intentional bad example
}
```

```cpp

void checkECUStatus_Good(const std::vector<ECU> &ecus, const std::string &targetName) {
  // GOOD (İyi - C++ Core Guidelines): Use STL algorithm! Intent is 100% clear immediately: "Find
  // if any ECU matches".
  // TR: İYİ: STL Algoritması kullanımı. Niyet çok açıktır: "Bir Koşula Göre Ara ve Bul
  // (find_if)".
  auto it = std::find_if(ecus.begin(), ecus.end(),
      [&](const ECU &e) { return e.name == targetName; });

  if (it != ecus.end()) {
    std::cout << "[Intent/Niyet: STL] ECU '" << targetName << "' found!" << std::endl;
  }
}

// ---------------------------------------------------------------------------------------------
// [P.4 & P.5] Compile-Time Checking (`constexpr` and `static_assert`)
// ---------------------------------------------------------------------------------------------
// EN: Because of `constexpr`, compiler calculates this during BUILD time (0 CPU cost at
// runtime). TR: `constexpr` sayesinde CPU bu fonksiyonun hesabını BUILD (Derleme) anında
// tamamlar.
constexpr int calculateMaxSensorChannels(int baseChannels, int expansionModules) {
  return baseChannels + (expansionModules * 2);
}

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - PHILOSOPHY & INTERFACES ===\n" << std::endl;

  // 1. Express Intent demonstration
  std::vector<ECU> vehicleECUs = {
      {"BCM", true}, {"Powertrain", false}, {"HVAC", true}};
  checkECUStatus_Good(vehicleECUs, "Powertrain");

  // 2. Compile-Time Magic
  // EN: This is NOT calculated at runtime! Replaced directly with `20` in the ASM binary. TR: Bu
  // işlem çalışırken (Runtime) YAPILMAZ! Derleyici satırı Assembly kodunda doğrudan `20` yapar.
  constexpr int maxChannels = calculateMaxSensorChannels(10, 5);

  // EN: static_assert checks rules ALWAYS DURING COMPILE. If false -> Red Compiler Error! TR:
  // static_assert kuralları DERLEME AŞAMASINDA kontrol eder. Yanlışsa IDE direkt kırmızıyı
  // basar, Build alınmaz.
  static_assert(maxChannels >= 10, "[FATAL ERROR] Sensor channels cannot be less than 10!");

  std::cout << "\nMax Sensor Channels (Calculated purely at Compile Time!): " << maxChannels <<
      std::endl;

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "- EN: Write code for the COMPILER to optimize and HUMANS to read!" << std::endl;
  std::cout << "- TR: İnsanların rahat okuyacağı (STL kullanarak niyet belli eden), "
         "Derleyicinin uçuracağı (constexpr) mimariler kurun!"
      << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════════════════
  // EN: I.4 — Make interfaces precisely and strongly typed
  // TR: I.4 — Arayüzleri kesin ve güçlü-tipli yapın
  // ═══════════════════════════════════════════════════════════════════════════════════════

  // ─── 1. BAD: Weakly typed interface ──────────────────────────────────────────────────
  // EN: What does 'unit' mean? 0 = KPH? 1 = MPH? Magic numbers everywhere.
  // TR: 'unit' ne demek? 0 = KPH mi? 1 = MPH mi? Anlaşılmaz büyü sayıları.
  //
  // void setSpeed(int speed, int unit);   // BAD — caller can pass anything setSpeed(120, 3);
  // // What is 3? Compiles fine!

  // ─── 2. GOOD: Strongly typed interface ───────────────────────────────────────────────
  // EN: Type-safe enum makes invalid calls impossible at compile time.
  // TR: Tip-güvenli enum, geçersiz çağrıları derleme zamanında engeller.
  enum class SpeedUnit { KPH, MPH };

  auto setSpeed = [](double speed, SpeedUnit unit) {
    const char* label = (unit == SpeedUnit::KPH) ? "km/h" : "mph";
    std::cout << "[I.4 GOOD] Speed set to " << speed << " " << label << std::endl;
  };

  setSpeed(120.0, SpeedUnit::KPH);
```

```
  // setSpeed(120.0, 2);  // EN: Won't compile! Type safety enforced.
                          // TR: Derlenmez! Tip güvenliği sağlandı.

  // ═══════════════════════════════════════════════════════════════════════════
  // EN: I.11 — Never transfer ownership by a raw pointer
  // TR: I.11 — Ham işaretçi ile sahiplik aktarımı ASLA yapılmaz
  // ═══════════════════════════════════════════════════════════════════════════

  // ─── 3. BAD: Raw pointer ownership transfer ──────────────────────────────
  // EN: Who owns this pointer? Caller? Callee? Nobody knows → leak.
  // TR: Bu işaretçinin sahibi kim? Çağıran mı? Çağrılan mı? → sızıntı.
  //
  // ECU* createECU() { return new ECU{"Raw", true}; }  // BAD

  // ─── 4. GOOD: unique_ptr ownership transfer ──────────────────────────────
  // EN: Ownership is explicit. unique_ptr is the ONLY owner, auto-deletes.
  // TR: Sahiplik açıkça bellidir. unique_ptr TEK sahiptir, otomatik siler.
  auto createECU_Safe = []() -> std::unique_ptr<ECU> {
    return std::make_unique<ECU>(ECU{"SmartECU", true});
  };

  auto safeEcu = createECU_Safe();
  std::cout << "[I.11 GOOD] Owned ECU: " << safeEcu->name
            << " (unique_ptr, no leak possible)" << std::endl;

  return 0;
}
```

► 02. Resource Management Rule Of Zero

module_08_core_guidelines/02_resource_management_rule_of_zero.cpp

Konu: C++ Core Guidelines: Resource Management & Rule of Zero / Kaynak Yönetimi Mimarisi

Teori / Theory

INTRODUCTION: Rule of Zero & RAII

The Guidelines say: "Manage resources automatically using resource handles and RAII."
What is the "Rule of Zero"? If your class only manages standard modern resources (like `std::string`, `std::vector`, `std::unique_ptr`), you SHOULD NOT write a Destructor (`~MyClass()`), Copy Constructor, or Assignment Operator. The compiler's default automatically handles memory perfectly. ZERO custom memory-management code = ZERO memory leak bugs!

GİRİŞ: Sıfır Kuralı ve RAII

Çekirdek Kurallar der ki: "Kaynakları (Belleği) otomatik olarak RAII kılıflarıyla yönet!"
"Sıfır Kuralı (Rule of Zero)" Nedir? Eğer sizin yazdığınız sınıf, belleği kendi yöneten akıllı nesneler (std::vector, std::string) kuruyorsa, BAŞKA HİÇBİR ŞEY YAZMAYIN! Derleyicinin üreteceği otomatik Yıkıcı/Kurucu kodlar zaten KUSURSUZ çalışır. O (Sıfır) kod yazarsanız = 0 (Sıfır) Bellek Kaçağı üretirsiniz!

R.10: Avoid malloc() and free()

`malloc()` does not call Constructors! It grabs raw, dumb bytes from RAM. `free()` does not call Destructors! It leaves sub-resources orphaned. NEVER use them.

R.10: Malloc ve Free KESİNLİKLE YASAK

Eski C fonksiyonu `malloc()`, C++ Sınıflarını "Constructor" çağırmadan ucube gibi yaşama döndürür. `free()` ise Sınıf içindeki işaretçileri umursamadan öldürür ve RAM çöp olur.

CPPREF Depth: Rule of Zero — Let the Compiler Write Your Special Members

If every data member is an RAII type (std::string, std::vector, std::unique_ptr, std::shared_ptr), the compiler-generated destructor, copy constructor, copy assignment, move constructor, and move assignment are all correct — you need to write NONE of them. This is the Rule of Zero. The moment you define ANY special member (even a destructor), the compiler suppresses implicit generation of the move operations. Writing `= default` explicitly requests compiler generation and documents your intent. The Rule of Zero dramatically reduces bugs because hand-written special members are a top source of resource leaks, double-frees, and slicing errors. Reserve the Rule of Five for low-level resource handles only.

CPPREF Derinlik: Sıfır Kuralı — Özel Üyeleri Derleyiciye Bırakın

Tüm veri üyeleri RAII tipi (std::string, std::vector, std::unique_ptr, std::shared_ptr) ise derleyicinin ürettiği yıkıcı, kopyalama kurucusu, kopyalama ataması, taşıma kurucusu ve taşıma ataması zaten doğrudur — HİÇBİRİNİ yazmanız gerekmez (Sıfır Kuralı). HERHANGİ bir özel üye tanımladığınız anda (yıkıcı bile olsa) derleyici taşıma işlemlerini otomatik üretmez. `= default` derleyici üretimini açıkça talep eder. Sıfır Kuralı hataları büyük ölçüde azaltır çünkü elle yazılan özel üyeler kaynak sızıntısı, çift serbest bırakma ve dilimleme hatalarının en büyük kaynağıdır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------------------------
// [C.20, R.1] THE RULE OF ZERO (Sıfır Kuralı Mimarisi)
// ---------------------------------------------------------------------------------------------
// EN: This class flawlessly manages its own memory thanks to the Rule of Zero.
// TR: Bu sınıf, 'Sıfır Kuralı' sayesinde kendi belleğini kusursuz yönetir.
class VehicleConfigManager {
private:
  std::string vehicleId;                  // EN: Self-managed (RAII)
  std::vector<std::string> ecuModuleNames; // EN: Self-managed (RAII)

public:
  // Simple constructor to initialize data.
  // TR: Constructor sadece ilk değer ataması içindir.
  VehicleConfigManager(const std::string &id) : vehicleId(id) {
    std::cout << "[VehicleConfig] Initialized: " << vehicleId << std::endl;
  }

  void addECU(const std::string &name) { ecuModuleNames.push_back(name); }
```

```cpp

  // EN: NOTICE WHAT IS MISSING? - NO `~VehicleConfigManager()` Destructor! - NO `new` or
  // `delete` anywhere! -> When this object naturally dies, `std::vector` and `std::string` clean
  // themselves up. 100% Leak-Proof!
  //
  // TR: FARK ETTİNİZ Mİ NE EKSİK? - Yıkıcı `~VehicleConfigManager()` fonksiyonu YOK! - El ile
  // yazılmış `new` veya `delete` YOK! -> Obje ölürken, içindeki objeler (vector, string) C++
  // mantığıyla OS'ye iade edilir.
};

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - RULE OF ZERO & RESOURCE "
               "MGMT ===\n"
            << std::endl;

  {
    std::cout << "-> Entering local scope... (Kapsama giriliyor...)" << std::endl;

    // 1. Applying Rule of Zero
    VehicleConfigManager myCar("VIN_WBAPH5C55BA271234");
    myCar.addECU("BCM");
    myCar.addECU("Powertrain");

    // 2. Applying R.11 (Avoid new and delete explicitly)
    // EN: NEVER write -> VehicleConfigManager* v = new VehicleConfigManager(...);
    // TR: ASLA yazma -> VehicleConfigManager* v = new VehicleConfigManager(...);

    std::unique_ptr<VehicleConfigManager> smartConfig =
        std::make_unique<VehicleConfigManager>("VIN_JH4DB7660SS001234");

  } // EN: BOTH 'myMatch' and 'smartMatchLocation' auto-destruct HERE safely.
    // TR: Buradaki '}' anında, akıllı işaretçi ve stack objesi muazzam bir sessizlikle iade
    // edilir.

  std::cout << "-> Left local scope safely. No manual memory management required!\n"
            "-> (Sıfır sızıntı! Amale gibi delete[] kontrolü yapmaya gerek "
            "kalmadı!)\n"
        << std::endl;

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "- EN: C++ Core Guidelines states: 'Rule of Zero'. Fall back "
               "entirely on std::vector and Smart Pointers."
            << std::endl;
  std::cout << "- TR: (Manuel bellek işine hiç girmemeyi (Rule of Zero) emreder!)" << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: R.1 — Manage resources automatically using RAII
  // TR: R.1 — Kaynakları RAII kullanarak otomatik yönet
  // ═══════════════════════════════════════════════════════════════════════════════

  // ─── 1. RAII FileHandle wrapper ──────────────────────────────────────────────
  // EN: Constructor acquires, Destructor releases. No manual close needed.
  // TR: Constructor edinir, Destructor serbest bırakır. Manuel kapatma yok.
  struct FileHandle {
    std::string filename;
    bool isOpen = false;

    explicit FileHandle(const std::string& fname)
        : filename(fname), isOpen(true) {
      std::cout << "[R.1 RAII] Opened file: " << filename << std::endl;
    }

    ~FileHandle() {
      if (isOpen) {
        isOpen = false;
        std::cout << "[R.1 RAII] Auto-closed file: " << filename << std::endl;
      }
    }
```

```

    // EN: Delete copy to enforce single ownership (move-only resource).
    // TR: Tek sahiplik için kopyalamayı sil (sadece taşınabilir kaynak).
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
  };

  // ─── 2. Demo: automatic cleanup even on early return ──────────────────────────
  // EN: Even if we exit the scope early, destructor is guaranteed to run.
  // TR: Kapsamdan erken çıksak bile, yıkıcı çalışması garanti edilir.
  {
    FileHandle logFile("telemetry_log.csv");
    std::cout << "[R.1 Demo] Processing file: " << logFile.filename << std::endl;

    bool earlyExit = true;
    if (earlyExit) {
      std::cout << "[R.1 Demo] Early return from scope!" << std::endl;
      // EN: ~FileHandle() runs automatically RIGHT HERE at '}'
      // TR: ~FileHandle() otomatik olarak '}' noktasında çalışır
    }
  } // EN: Destructor fires here — file is auto-closed, no leak.
    // TR: Yıkıcı burada çalışır — dosya otomatik kapatılır, sızıntı yok.

  std::cout << "[R.1 Demo] Scope exited — FileHandle cleaned up automatically!" << std::endl;

  return 0;
}
```

► 03. Error Handling

module_08_core_guidelines/03_error_handling.cpp

Konu: C++ Core Guidelines: Error Handling / Hata Yönetimi

Teori / Theory

E.2 & E.3: Use Exceptions for Errors

"Throw an exception to signal that a function can't perform its assigned task." C++ Core Guidelines heavily prefers Exceptions (`throw std::runtime_error`) over returning Error Codes (like `-1` or `false`). Error codes get ignored, leading to silent bugs. Exceptions CANNOT be ignored unless explicitly caught (`catch`).

Hatalar için Exception Kullanın

"Eğer bir fonksiyon işini yapamıyorsa (Örn: Dosya bulunamadı), Exception (İstisna) fırlat." C++ Mimarisi, eski C-tarzı "-1" veya "false" döndüren hata kodlarından nefret eder. Çünkü hata kodlarını yazılımcılar kolayca görmezden gelir. Exception (`throw`) ise yakalanmazsa (`catch`) programı durdurur. Sessiz (Zombie) bug'lara izin vermez!

E.15: Throw by value, catch by reference

ALWAYS throw standard exceptions by value (`throw MyError();`) and catch them by const reference (`catch (const MyError& e)`). This prevents "Object Slicing".

Değer ile Fırlat, Referans ile Yakala

Exception fırlatırken obje oluşturarak (değer) fırlat: `throw std::invalid_argument("hata");` Yakalarken ASLA değer olarak yakalama, her zaman **Referans** ile yakala: `catch(const std::exception& e)` Bu sayede "Object Slicing" engellenir ve C++ polimorfizmi çalışır.

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>

// EN: BAD PRACTICE: Using Error Codes (C-Style)
// TR: KÖTÜ PRATİK: C-Tarzı Hata Kodları (Return ile kontrol). Yazılımcı bunu unutabilir!
int openDatabaseOldWay(const std::string &address) {
  if (address.empty())
    return -1; // -1 means FATAL ERROR (Bulunamadı)
  return 0;    // 0 means Success
}

// EN: GOOD PRACTICE: Using Exceptions (C++ Core Guidelines E.2)
// TR: MÜKEMMEL PRATİK: Exception! Sessizce geçilmesine / unutulmasına C++ derleyicisi asla izin
// vermez!
void openDatabaseModern(const std::string &address) {
  if (address.empty()) {
    // EN: E.15: Throw by value! (Mimar kuralı: Sadece objeyi fırlat)
    throw std::invalid_argument(
        "FATAL: Database address cannot be empty!"
        " (Db adresi bos olamaz!)");
  }
  std::cout << "[Database] Successfully connected to: " << address << std::endl;
}

// EN: [[nodiscard]] makes the compiler WARN if the return value is ignored.
// TR: [[nodiscard]] dönüş değeri göz ardı edilirse derleyiciyi UYARIR.
[[nodiscard]] int getErrorCode() {
  return -1; // EN: Simulated error code
             // TR: Simüle edilmiş hata kodu
}

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - ERROR HANDLING ===\n" << std::endl;

  // 1. DANGER OF ERROR CODES (Sessiz Zombileşen Hatalar)
  // EN: I forgot to check the return value (-1). My program continues
  openDatabaseOldWay( "");
           // BROKEN! TR: Gelen -1 değerini `if` içine almayı unuttum. Program ZOMBİ olarak devam
           // ediyor!

  // 2. SAFETY OF EXCEPTIONS (Exception Güvenliği, Fail-Fast Yaklasimi)
  try {
    // EN: This will immediately blast an error and JUMP to catch! TR: Bu fonksiyon anında
    // fırlatma yapar ve Alt satırları okumadan 'catch' bloğuna DÜŞER!
    openDatabaseModern("");
    std::cout << "This line will NEVER be printed." << std::endl;
  }
```

```
50    // EN: CABLE-RULE (E.15): CATCH BY CONST REFERENCE! (Referans ile Yakala)
51    // TR: Object Slicing olmasın diye her zaman 'const &' ile polimorfik olarak yakalıyoruz!
52    catch (const std::invalid_argument &e) {
53      std::cerr << "[Core Guidelines Exception Catcher] -> " << e.what() << std::endl;
54    }
55
56    // ==========================================================================================
57    // [E.12 & E.16: Destructors and noexcept / Yıkıcılar ve noexcept Güvencesi]
58    // ==========================================================================================
59    // EN: Destructors (`~MyClass()`) and memory deallocation functions MUST NEVER THROW
60    // EXCEPTIONS! If an exception happens inside a destructor during another exception unwinding,
61    // the C++ runtime immediately terminates (`std::terminate`).
62    //
63    // TR: Yıkıcı fonksiyonlarda (`~MyClass`), ASLA Exception fırlatılamaz. Eğer fırlatılırsa C++
64    // sistemi o saniye İntihar/İptal eder (`std::terminate`). Modern C++'da tüm yıkıcılar
65    // varsayılan olarak `noexcept` (Hata Fırlatmaz) güvencesindedir.
66    // ==========================================================================================
67    //
68    // ==========================================================================================
69      // PPREF DEPTH: The Brutality of `std::terminate()` / CPPREF DERİNLİK: `std::terminate()`
70    // Acımasızlığı]
71    // ==========================================================================================
72    // EN: `std::terminate()` is the absolute end of the line. It doesn't call destructors, it
73    // doesn't free memory, it doesn't close files. It violently kills the process from the OS
74    // level (`abort()`). This happens if: 1. You throw an exception and no `catch` block exists
75    // anywhere to catch it. 2. An exception throws INSIDE a destructor while another exception is
76    // currently flying (Stack Unwinding collision). 3. A `noexcept` function explicitly throws.
77    //
78    // TR: `std::terminate()` bir programın yaşayabileceği en vahşi ölümdür. Açık olan dosyaları
79    // kapatmaz, RAM'i temizlemez (Destructor çağırmadan fişi çeker). 3 durumda ortaya çıkar: 1.
80    // Fırlattığınız hatayı kimse (catch ile) yakalamazsa, 2. Bir hata havada uçarken (Stack geri
81    // sarılırken), Destructor'ın içinden İKİNCİ bir hata daha fırlatırsanız (Çarpışma), 3.
82    // `noexcept` verdiğiniz fonksiyonun içinde gizlice hata fırlatırsanız. Program anında OS
83    // seviyesinde vurulur (`abort()`).
84    // ==========================================================================================
85
86    // ==========================================================================================
87    // EN: E.6 — Use RAII to prevent leaks in error paths
88    // TR: E.6 — Hata yollarında sızıntıları önlemek için RAII kullanın
89    // ==========================================================================================
90
91    // ─── 1. BAD: Raw pointer leaks on exception ──────────────────────────────────────────
92    // EN: If process() throws, the delete never runs → memory leak.
93    // TR: process() hata fırlatırsa delete çalışmaz → bellek sızıntısı.
94    //
95    // int* data = new int[1024]; process(data);      // If this throws... delete[] data;      //
96    // ...this line is NEVER reached! LEAK!
97
98    // ─── 2. GOOD: unique_ptr + exception = no leak ───────────────────────────────────────
99    // EN: unique_ptr guarantees cleanup even when exceptions fly.
100   // TR: unique_ptr, exception fırlatılsa bile temizliği garanti eder.
101   try {
102     auto safeData = std::make_unique<int[]>(1024);
103     safeData[0] = 42;
104     std::cout << "[E.6 GOOD] safeData[0] = " << safeData[0] << std::endl;
105
106     // EN: Simulate an error after allocation
107     // TR: Tahsis sonrası bir hata simüle et
108     throw std::runtime_error("Simulated processing error!");
```

```cpp

      // EN: This line is skipped, but unique_ptr still frees memory.
      // TR: Bu satır atlanır ama unique_ptr yine de belleği serbest bırakır.
    } catch (const std::runtime_error& e) {
      std::cerr << "[E.6 Caught] " << e.what()
                << " (unique_ptr freed memory automatically!)" << std::endl;
    }

    // ─── 3. [[nodiscard]]: Force callers to check return values ──────────────────────────────
    // EN: [[nodiscard]] makes the compiler WARN if the return value is ignored.
    // TR: [[nodiscard]] dönüş değeri göz ardı edilirse derleyiciyi UYARIR. (getErrorCode() defined
    // above main as a [[nodiscard]] free function)

    // EN: Good — caller checks the result. Compiler is happy.
    // TR: İyi — çağıran sonucu kontrol eder. Derleyici mutludur.
    int errCode = getErrorCode();
    if (errCode != 0) {
      std::cout << "[nodiscard Demo] Error code received: " << errCode << std::endl;
    }

    // EN: Bad — ignoring the return would trigger a compiler warning:
    // TR: Kötü — dönüşü yok saymak derleyici uyarısı tetikler: getErrorCode();  // WARNING:
    // ignoring return value with [[nodiscard]]

    return 0;
  }
```

► 04. Concurrency And Performance

module_08_core_guidelines/04_concurrency_and_performance.cpp

Konu: C++ Core Guidelines: Concurrency & Performance Rules / Eşzamanlılık ve Performans

Teori / Theory

CP.20: Use RAII Locks, Never touch raw mutexes

"Never use plain lock() and unlock(). Use RAII." If you manually call `mtx.lock()` and subsequently throw an exception before reaching `mtx.unlock()`, your underlying mutex NEVER unlocks. Your server will FREEZE FOREVER (Deadlock). Always use `std::lock_guard` or `std::scoped_lock` (C++17).

Çıplak Mutex yasaktır, RAII kullan!

"ASLA çıplak mtx.lock() ve mtx.unlock() KULLANMAYIN." Eğer manuel `lock()` yaparsanız ve hemen alt satırda bir Exception (Hata) fırlatılırsa kod `unlock()` kısmına asırlar geçse varamaz. Tüm sunucunuz sonsuza dek kitlenir (Deadlock). Kilit mekanizmasını bir Scope (Kapsam) içine kilitleyen `std::lock_guard` kullanın.

Per.1 & Per.2: Premature Optimization

"Don't optimize without reason." & "Don't optimize prematurely." C++ Programmers love doing bitwise magic or raw pointer arithmetic to save 2 nanoseconds. 99% of the time, the compiler's optimizer flag (`-O3`) does it much better and safer. First, write CLEAN, INTENTIONAL, READABLE CODE. Second, use a PROFILER (like Valgrind/Callgrind). Third, ONLY optimize the bottleneck!

Erken Optimizasyon (Şeytanın İşi)

"Sebepsiz yere optimizasyon yapma!" ve "Erken optimizasyon bütün kötülüklerin anasıdır." C++ Mimarisi: 3 nanosaniye kazanmak için okunması imkansız Bitwise kodlar YAZMAYIN! * Halbuki `-O3` (Optimizasyon Bayrağı) ile çalışan C++ Derleyicisi sizden 100 kat akıllıdır. Önce TEMİZ, NİYETİ BELLİ OLAN KOD yaz (STL kullan). Sonra Valgrind Profiler ile ölç. Ve SADECE Oyunun Cidden Kasma Yaptığı yerleri (Bottleneck) optimize et!

CPPREF Depth: Hardware Concurrency limit & Cache Coherence!

CppReference warns against creating 1000 threads for performance. Use `std::thread::hardware_concurrency()` to find the exact number of physical/ logical cores you have. Spawning more threads than CPU cores causes extreme lag. Also, multiple threads modifying variables physically close together in RAM causes "False Sharing" (L1/L2 Cache Coherence ping-pong), drastically hurting performance!

CPPREF Derinlik: Donanım Eşzamanlılık Sınırı ve Önbellek Tutarlılığı!

CppReference'in dev uyarısı: Performans için "1000 tane Thread açayım" derseniz sistem çöker. Gerçek fiziksel ve mantıksal çekirdek sayısını öğrenmek için `std::thread::hardware_concurrency()` kullanılmalıdır (Maksimum thread limiti). İkinci Felaket: Birden çok Thread, RAM'de birbirine çok yakın duran iki farklı değişkene sürekli yazıp bozarsa, İşlemci Ön Bellekleri (L1/L2 Cache) çıldırır. Sürekli birbirlerinin verisini geçersiz kılarlar ("False Sharing" / Cache Coherence). Performans 10 kat düşer!

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

std::mutex databaseLock;
int globalOnlineECUs = 0;

void registerECU_Bad() {
  // EN: CP.20 VIOLATION: Manual Lock/Unlock. VERY DANGEROUS!
  // TR: CORE GUIDELINE İHLALİ: Manuel Kilit. EĞER ARADA HATA ÇIKARSA SİSTEM ÇÖKER!
  databaseLock.lock();

  globalOnlineECUs++;
  // If a 'throw' happens exactly here, unlock() is silently skipped -> FATAL DEADLOCK!

  databaseLock.unlock();
}

void registerECU_Good() {
  // EN: C++ CORE GUIDELINES (CP.20): Use RAII (std::lock_guard explicitly)
  // TR: MÜKEMMEL KOD (RAII): Kapsama girince Mutex Kilitlenir, süslü parantez kapanınca Otomatik
  // Açılır!
  std::lock_guard<std::mutex> lock(databaseLock); // Automatic Safe Locker!

  globalOnlineECUs++;
  // EN: Even if an Exception throws here, RAII strictly un-winds the lock back to OS. No
  // deadlocks. TR: Burada Hata/Exception atılsa dahi C++ sistemi kilidi garantili olarak İşletim
  // sistemine iade eder.
}

int main() {
  std::cout << "=== MODULE 8: C++ CORE GUIDELINES - CONCURRENCY & PERFORMANCE ===\n" << std::endl;

  std::vector<std::thread> workers;
```

```cpp

  // EN: Launch 10 Threads simultaneously to register ECU modules.
  // TR: 10 Thread Aynı Anda ECU Kayıt (Simülasyon). Race Condition'ı RAII Lock Guard ile
  // engelliyoruz.
  for (int i = 0; i < 10; ++i) {
    workers.push_back(std::thread(registerECU_Good));
  }

  // EN: Join all threads (Wait for them to finish properly).
  // TR: Tüm çalışma grupları birleşene (Bitiş) kadar ana sunucuyu bekle.
  for (auto &w : workers) {
    w.join();
  }

  std::cout << "All ECUs Registered Safely! Total Online: " << globalOnlineECUs << std::endl;

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "1. Threads + Shared Global Data = CATASTROPHE. Always use "
               "std::lock_guard / std::scoped_lock."
            << std::endl;
  std::cout << "2. PERFORMANCE MYTH: Let the '-O3' generic compiler flags do "
               "the 'clever' micro-optimizations!"
            << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════
  // EN: Per.7 — Design to enable optimization (data layout matters)
  // TR: Per.7 — Optimizasyonu mümkün kılacak şekilde tasarla (veri düzeni önemlidir)
  // ═══════════════════════════════════════════════════════════════════════════

  // EN: C++ has no standard `restrict` keyword (C99 feature). Some compilers offer
  // `__restrict__` as an extension, but it's non-portable.
  // TR: C++'da standart bir `restrict` anahtar kelimesi yoktur (C99 özelliği). Bazı derleyiciler
  // `__restrict__` uzantısını sunar ama taşınabilir değildir.

  // ─── 1. Avoid unnecessary copies in range-for loops ──────────────────────
  // EN: Using `auto` (value) copies every element. Use `const auto&` instead.
  // TR: `auto` (değer) her elemanı kopyalar. Bunun yerine `const auto&` kullanın.

  std::vector<std::string> sensorNames = {
      "Temperature", "Pressure", "Humidity", "Altitude", "Speed"};

  // EN: BAD — copies every string (expensive for large objects).
  // TR: KÖTÜ — her string'i kopyalar (büyük nesneler için pahalıdır). for (auto name :
  // sensorNames) { /* copy per iteration */ }

  // EN: GOOD — zero-copy read, compiler can auto-vectorize.
  // TR: İYİ — sıfır kopyalı okuma, derleyici otomatik vektörleştirebilir.
  std::cout << "\n[Per.7] Sensor names (const auto& — zero copy):" << std::endl;
  for (const auto& name : sensorNames) {
    std::cout << "  -> " << name << std::endl;
  }

  std::cout << "[Per.7] Always use 'const auto&' in range-for when you only read!"
            << std::endl;

  return 0;
}
```

► 05. Smart Pointer Guidelines

module_08_core_guidelines/05_smart_pointer_guidelines.cpp

Konu: C++ Core Guidelines: Smart Pointer Ownership Rules / Akıllı İşaretçi Sahiplik Kuralları

Teori / Theory

[R.20: Use unique_ptr to represent exclusive ownership / Özel sahiplik için unique_ptr kullanın]

When exactly ONE entity owns a resource, use `std::unique_ptr`. It has ZERO runtime overhead compared to a raw pointer — the compiler generates identical machine code. The destructor automatically frees the resource when the owning scope exits, preventing every possible memory leak.

Bir kaynağın yalnızca TEK bir sahibi varsa `std::unique_ptr` kullanın. Ham işaretçiye (raw pointer) kıyasla SIFIR çalışma zamanı maliyeti vardır — derleyici birebir aynı makine kodunu üretir. Sahip olan kapsam çıkınca yıkıcı kaynağı otomatik serbest bırakır, bellek sızıntısı imkansızlaşır.

[R.21: Prefer unique_ptr over shared_ptr unless you need shared ownership / Paylaşımlı sahiplik gerekmiyorsa unique_ptr tercih edin]

`shared_ptr` carries atomic reference-count overhead (control block + two atomic increments/decrements per copy). Default to `unique_ptr`; promote to `shared_ptr` ONLY when multiple owners genuinely need the same resource.

`shared_ptr` atomik referans sayacı yükü taşır (kontrol bloğu + her kopyada iki atomik artış/azalış). Varsayılan olarak `unique_ptr` kullanın; SADECE birden fazla sahip aynı kaynağa gerçekten ihtiyaç duyduğunda `shared_ptr`'ye terfi ettirin.

R.22: Use make_shared / make_unique — Exception safety + single allocation

`std::make_unique<T>(args)` and `std::make_shared<T>(args)` prevent memory leaks in complex expressions and, for shared_ptr, fuse the control block with the object into ONE allocation (cache-friendly, fewer calls to the allocator).

`make_unique` ve `make_shared` karmaşık ifadelerde bellek sızıntısını önler. `make_shared` ayrıca kontrol bloğu ile nesneyi TEK bir bellek tahsisinde birleştirir (önbellek dostu, daha az tahsis çağrısı).

R.24: Aliasing constructor — shared_ptr to a sub-object

The aliasing constructor `shared_ptr<T>(owner, &owner->member)` lets you hand out a pointer to a MEMBER while keeping the PARENT alive. No extra allocation. The control block is shared with the parent.

Aliasing constructor `shared_ptr<T>(sahip, &sahip->uye)` ile ANA nesneyi canlı tutarken bir ÜYE'ye işaretçi verebilirsiniz. Ekstra tahsis yok, kontrol bloğu ana nesneyle paylaşılır.

R.30: Take smart pointers as parameters ONLY to express lifetime semantics

If a function only READS or USES an object, take `const T&` or `T*`. Accepting `unique_ptr<T>` means "I will take ownership." Accepting `shared_ptr<T>` means "I may share ownership." Passing smart pointers when you don't need ownership semantics pollutes the API and forces callers into a specific allocation strategy.

Bir fonksiyon yalnızca nesneyi OKUYORSA veya KULLANIYORSA `const T&` veya `T*` alın. `unique_ptr<T>` kabul etmek "sahipliği devralıyorum" demek, `shared_ptr<T>` kabul etmek "sahipliği paylaşabilirim" demektir. Sahiplik semantiği gerekmediğinde akıllı işaretçi geçmek API'yi kirletir.

CPPREF Depth: shared_ptr Control Block and Aliasing Constructor Internals

`std::shared_ptr<T>` internally holds TWO pointers: one to the managed object and one to a control block. The control block stores the strong reference count, the weak reference count, the deleter, and (optionally) the allocator. `std::make_shared<T>(args...)` performs a SINGLE heap allocation for both the object and the control block — more cache-friendly and fewer allocator calls versus `shared_ptr<T>(new T)`. The aliasing constructor `shared_ptr<U>(shared_ptr<T>, U*)` shares the control block of T but points to a different object U — useful for pointing to a member of a shared object without extra ref counting. `enable_shared_from_this<T>` embeds a hidden `weak_ptr<T>`; calling `shared_from_this()` on an object NOT managed by a `shared_ptr` is UB.

CPPREF Derinlik: shared_ptr Kontrol Bloğu ve Aliasing Kurucu İç Yapısı

`std::shared_ptr<T>` dahili olarak İKİ işaretçi tutar: biri yönetilen nesneye, biri kontrol bloğuna. Kontrol bloğu güçlü referans sayısını, zayıf referans sayısını, siliciyi ve (isteğe bağlı) ayırıcıyı saklar. `std::make_shared<T>(args...)` nesne ve kontrol bloğu için TEK yığın tahsisi yapar — `shared_ptr<T>(new T)`'ye kıyasla daha önbellek dostu. Aliasing kurucu `shared_ptr<U>(shared_ptr<T>, U*)` T'nin kontrol bloğunu paylaşır ama farklı bir U nesnesine işaret eder. `enable_shared_from_this` gizli bir `weak_ptr<T>` gömer; `shared_ptr` tarafından yönetilmeyen bir nesnede `shared_from_this()` çağırmak tanımsız davranıştır.

Kaynak Kod / Source Code

```cpp
#include <chrono>   // EN: high_resolution_clock for timing benchmarks
#include <iostream> // EN: standard I/O
#include <memory>   // EN: smart pointers (unique_ptr, shared_ptr, weak_ptr)
#include <string>   // EN: std::string
#include <vector>   // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// PRODUCTION CODE — Automotive Smart Pointer Architecture
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. unique_ptr Factory Pattern ─────────────────────────────────────

enum class ModuleType { BCM, Powertrain, ADAS, Infotainment };

class ECUModule {
public:
    std::string name;
    int firmwareVersion;

    ECUModule(std::string n, int fw) : name(std::move(n)), firmwareVersion(fw) {}
    virtual ~ECUModule() = default;
```

```

    virtual void selfTest() const {
        std::cout << "  [ECU] " << name << " v" << firmwareVersion
                  << " — generic self-test PASS" << std::endl;
    }
};

class ADASModule : public ECUModule {
public:
    ADASModule() : ECUModule("ADAS", 3) {}
    void selfTest() const override {
        std::cout << "  [ADAS] Camera + LiDAR calibration — PASS" << std::endl;
    }
};

// EN: R.20 + R.22 — factory returns unique_ptr (exclusive ownership)
// TR: R.20 + R.22 — fabrika unique_ptr döndürür (özel sahiplik)
std::unique_ptr<ECUModule> createModule(ModuleType type) {
    switch (type) {
        case ModuleType::BCM:
            return std::make_unique<ECUModule>("BCM", 7);
        case ModuleType::Powertrain:
            return std::make_unique<ECUModule>("Powertrain", 12);
        case ModuleType::ADAS:
            return std::make_unique<ADASModule>();
        case ModuleType::Infotainment:
            return std::make_unique<ECUModule>("Infotainment", 5);
    }
    return nullptr;
}

// ─── 2. shared_ptr — Shared Sensor Data Pool ──────────────────────────────────

struct SensorDataPool {
    std::vector<double> temperatures;
    std::vector<double> pressures;

    SensorDataPool() : temperatures(128, 0.0), pressures(64, 0.0) {
        std::cout << "  [SensorDataPool] Allocated (128 temp + 64 pressure slots)"
                  << std::endl;
    }
    ~SensorDataPool() {
        std::cout << "  [SensorDataPool] Released — all consumers done" << std::endl;
    }
};

// ─── 3. Aliasing Constructor — Engine / Turbo ─────────────────────────────────

struct Turbo {
    int boostPSI = 22;
};

struct Engine {
    std::string model = "V6-TwinTurbo";
    Turbo turbo;
};

// ─── 4. R.30 — Correct function signatures ────────────────────────────────────

// EN: GOOD — function only reads the object, takes const reference
// TR: İYİ — fonksiyon sadece nesneyi okur, const referans alır
void diagnose(const ECUModule& module) {
    std::cout << "  [Diag] Running diagnostics on: " << module.name << std::endl;
    module.selfTest();
}
```

```cpp

// EN: GOOD — function takes ownership, accepts unique_ptr by value
// TR: İYİ — fonksiyon sahipliği devralır, unique_ptr değer olarak kabul eder
void installModule(std::unique_ptr<ECUModule> module) {
    std::cout << "  [Install] Installed " << module->name
              << " (fw v" << module->firmwareVersion << ")" << std::endl;
    // EN: module is automatically destroyed at end of scope
    // TR: module kapsam sonunda otomatik yok edilir
}

// ─── 5. weak_ptr Observer — Cycle Breaking ──────────────────────────────────

class SensorObserver {
    std::weak_ptr<SensorDataPool> pool_;
    std::string observerName_;

public:
    SensorObserver(std::shared_ptr<SensorDataPool> pool, std::string name)
        : pool_(pool), observerName_(std::move(name)) {}

    void readLatest() const {
        // EN: lock() promotes weak_ptr to shared_ptr if the object is alive
        // TR: lock() nesne hayattaysa weak_ptr'yi shared_ptr'ye yükseltir
        if (auto sp = pool_.lock()) {
            std::cout << "  [" << observerName_ << "] Pool alive — temp slots: "
                      << sp->temperatures.size() << std::endl;
        } else {
            std::cout << "  [" << observerName_ << "] Pool EXPIRED — nothing to read"
                      << std::endl;
        }
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 8: C++ CORE GUIDELINES — SMART POINTER OWNERSHIP ===\n"
              << std::endl;

    // ─── Demo 1: unique_ptr Factory & Ownership Transfer ────────────────────
    std::cout << "[Demo 1] unique_ptr Factory & std::move\n";
    {
        auto bcm = createModule(ModuleType::BCM);
        auto adas = createModule(ModuleType::ADAS);

        // EN: R.30 — diagnose does NOT take ownership, receives const ref
        // TR: R.30 — diagnose sahiplik almaz, const referans alır
        diagnose(*bcm);
        diagnose(*adas);

        // EN: Transfer ownership via std::move — bcm is now nullptr
        // TR: std::move ile sahiplik devri — bcm artık nullptr
        installModule(std::move(bcm));
        if (!bcm) {
            std::cout << "  bcm is nullptr after std::move (ownership transferred)\n";
        }
    }

    std::cout << std::endl;

    // ─── Demo 2: shared_ptr Reference Counting ──────────────────────────────
    std::cout << "[Demo 2] shared_ptr Reference Counting\n";
    {
        auto pool = std::make_shared<SensorDataPool>();
        std::cout << "  use_count after creation  : " << pool.use_count() << std::endl;

        {
```

```
            // EN: Second consumer shares ownership
            // TR: İkinci tüketici sahipliği paylaşır
            auto consumer2 = pool;
            std::cout << "  use_count with 2 consumers: " << pool.use_count() << std::endl;

            auto consumer3 = pool;
            std::cout << "  use_count with 3 consumers: " << pool.use_count() << std::endl;
        }
        // EN: consumer2 and consumer3 destroyed — count drops back to 1
        // TR: consumer2 ve consumer3 yok edildi — sayaç 1'e düştü
        std::cout << "  use_count after inner scope: " << pool.use_count() << std::endl;
    }

    std::cout << std::endl;

    // ─── Demo 3: Aliasing Constructor ──────────────────────────────────────────────────────
    std::cout << "[Demo 3] Aliasing Constructor (R.24)\n";
    {
        auto engine = std::make_shared<Engine>();

        // EN: Aliasing — turboPtr points to engine->turbo but shares ownership
        // TR: Aliasing — turboPtr engine->turbo'yu gösterir ama sahipliği paylaşır
        std::shared_ptr<Turbo> turboPtr(engine, &engine->turbo);

        std::cout << "  engine use_count : " << engine.use_count() << std::endl;
        std::cout << "  turbo boost (PSI): " << turboPtr->boostPSI << std::endl;

        engine.reset();
        // EN: Engine object still alive because turboPtr holds shared ownership
        // TR: Engine nesnesi hâlâ hayatta çünkü turboPtr paylaşımlı sahiplik tutuyor
        std::cout << "  turbo boost after engine.reset(): " << turboPtr->boostPSI
                  << " (engine kept alive by alias)" << std::endl;
    }

    std::cout << std::endl;

    // ─── Demo 4: weak_ptr Observer — Detect Expired Objects ────────────────────────────────
    std::cout << "[Demo 4] weak_ptr Observer Pattern\n";
    {
        SensorObserver observer(nullptr, "LateBinder");

        {
            auto pool = std::make_shared<SensorDataPool>();
            observer = SensorObserver(pool, "TempMonitor");

            // EN: Pool is alive — observer can read
            // TR: Pool hayatta — gözlemci okuyabilir
            observer.readLatest();
        }
        // EN: Pool is destroyed — weak_ptr detects expiration
        // TR: Pool yok edildi — weak_ptr süresinin dolduğunu tespit eder
        observer.readLatest();
    }

    std::cout << std::endl;

    // ─── Demo 5: Performance — raw vs unique vs shared creation timing ─────────────────────
    std::cout << "[Demo 5] Performance Comparison (creation timing)\n";
    {
        constexpr int N = 500000;

        // EN: Raw pointer allocation
        // TR: Ham işaretçi tahsisi
        auto t0 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            auto* p = new ECUModule("raw", i);
            delete p;
        }
        auto t1 = std::chrono::high_resolution_clock::now();
```

```

        // EN: unique_ptr allocation (make_unique)
        // TR: unique_ptr tahsisi (make_unique)
        auto t2 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            auto p = std::make_unique<ECUModule>("unique", i);
        }
        auto t3 = std::chrono::high_resolution_clock::now();

        // EN: shared_ptr allocation (make_shared)
        // TR: shared_ptr tahsisi (make_shared)
        auto t4 = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < N; ++i) {
            auto p = std::make_shared<ECUModule>("shared", i);
        }
        auto t5 = std::chrono::high_resolution_clock::now();

        auto rawMs = std::chrono::duration_cast<std::chrono::milliseconds>(t1 - t0).count();
        auto uniqMs = std::chrono::duration_cast<std::chrono::milliseconds>(t3 - t2).count();
        auto shrMs = std::chrono::duration_cast<std::chrono::milliseconds>(t5 - t4).count();

        std::cout << "  " << N << " allocations each:\n";
        std::cout << "    raw new/delete : " << rawMs << " ms\n";
        std::cout << "    make_unique    : " << uniqMs << " ms\n";
        std::cout << "    make_shared    : " << shrMs << " ms\n";
        std::cout << "  (unique ≈ raw; shared slightly slower due to control block)"
                  << std::endl;
    }

    std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
    std::cout << "1. Default to unique_ptr (zero overhead, exclusive ownership)." << std::endl;
    std::cout << "2. Use shared_ptr ONLY when multiple owners genuinely exist." << std::endl;
    std::cout << "3. Never pass smart pointers to functions that don't need ownership."
              << std::endl;

    return 0;
}
```

► 06. Class Design Guidelines

module_08_core_guidelines/06_class_design_guidelines.cpp

Konu: C++ Core Guidelines: Class Design Rules / Sınıf Tasarım Kuralları

Teori / Theory

C.20: Rule of Zero — Avoid defining default operations if you can

If your class members are ALL RAII types (std::string, std::vector, std::unique_ptr), the compiler-generated destructor, copy, and move operations are ALREADY CORRECT. Writing them manually is error-prone and redundant. Let the compiler do its job.

Sınıfınızın tüm üyeleri RAII türlerinden (std::string, std::vector, std::unique_ptr) oluşuyorsa, derleyicinin otomatik ürettiği yıkıcı, kopya ve taşıma işlemleri ZATEN DOĞRUDUR. Manuel yazmak hata kaynağıdır ve gereksizdir. Derleyicinin işini yapmasına izin verin.

C.21: Define or =delete ALL special members if you define any

If you write a custom destructor, copy constructor, or copy assignment, you almost certainly need to define (or delete) ALL of them. This is the "Rule of Five" in modern C++. Partial definitions lead to subtle bugs like double-free or missing move semantics.

Özel bir yıkıcı, kopya kurucu veya kopya atama yazıyorsanız, neredeyse kesinlikle HEPSİNİ tanımlamanız (veya silmeniz) gerekir. Bu modern C++'da "Beşli Kuralı"dır. Kısmi tanımlamalar çift serbest bırakma veya eksik taşıma semantiği gibi ince hatalara yol açar.

C.35: A base class destructor should be public+virtual or protected+non-virtual

If you delete through a base pointer, the destructor MUST be virtual. Otherwise the derived destructor never runs — resource leak. If deletion through base is not intended, make the destructor protected and non-virtual.

Bir temel sınıf işaretçisi üzerinden silme yapıyorsanız yıkıcı KESİNLİKLE sanal (virtual) olmalıdır. Aksi halde türetilmiş yıkıcı asla çalışmaz — kaynak sızıntısı. Temel sınıf üzerinden silme amaçlanmıyorsa yıkıcıyı korumalı (protected) ve sanal olmayan yapın.

C.67: Polymorphic class should suppress copying

Copying a polymorphic object through a base pointer causes "Object Slicing" — the derived part is silently chopped off. Delete copy operations or make them protected. Use clone() (virtual copy) if deep copying is needed.

Polimorfik bir nesneyi temel işaretçi üzerinden kopyalamak "Object Slicing"e neden olur — türetilmiş kısım sessizce kesilir. Kopya işlemlerini silin veya korumalı yapın. Derin kopya gerekiyorsa clone() kullanın.

C.128 / C.129: virtual → override → final & Interface vs Implementation

Mark every overriding function `override` so the compiler catches typos. Use `final` to prevent further overriding of a specific method or to seal an entire class. Separate INTERFACE inheritance (pure virtuals) from IMPLEMENTATION inheritance (shared base logic) — Liskov Substitution Principle: every subtype must be usable wherever the base type is expected.

Her geçersiz kılan fonksiyonu `override` ile işaretleyin, derleyici yazım hatalarını yakalasın. Belirli bir metodu veya tüm sınıfı mühürlemek için `final` kullanın. ARAYÜZ kalıtımını (saf sanal) UYGULAMA kalıtımından (ortak temel mantık) ayırın — Liskov İkame Prensibi: her alt tür, temel türün beklendiği her yerde kullanılabilir olmalıdır.

CPPREF Depth: Liskov Substitution Principle in C++ — Beyond Theory

LSP states that if S is a subtype of T, objects of type T may be replaced with objects of type S without altering program correctness. Common violations include: strengthening preconditions (derived demands more than base promised), weakening postconditions (derived delivers less than base guaranteed), and changing exception specifications. The `override` keyword (C++11) enforces exact signature matching with the base — a misspelled or mistyped override becomes a compile error. `final` prevents further overriding of a particular virtual method or prevents deriving from a class entirely. Composition over inheritance: prefer has-a (member) relationships to is-a (inheritance) except when runtime polymorphism through interfaces (pure virtual base) is needed. A well-designed hierarchy has a thin interface base and concrete leaves.

CPPREF Derinlik: Liskov İkame Prensibi — C++'ta Teorinin Ötesi

LSP, S tipi T'nin alt tipi ise T nesnelerinin S ile yer değiştirilebileceğini ve program doğruluğunun bozulmaması gerektiğini belirtir. Yaygın ihlaller: ön-koşulları güçlendirmek (türetilmiş sınıf tabandan fazlasını talep eder), son-koşulları zayıflatmak (türetilmiş sınıf tabandan azını garanti eder), istisna belirtimlerini değiştirmek. `override` (C++11) taban ile tam imza eşleşmesini zorlar. `final` belirli bir sanal fonksiyonun daha fazla geçersiz kılınmasını veya sınıftan türetilmesini engeller. Kalıtım yerine bileşim: çalışma zamanı çok biçimliliği (saf sanal taban) dışında has-a (üye) ilişkilerini is-a (kalıtım) ilişkisine tercih edin.

Kaynak Kod / Source Code

```cpp
#include <iostream> // EN: standard I/O
#include <memory>   // EN: smart pointers for RAII members
#include <string>   // EN: std::string
#include <vector>   // EN: std::vector

// ═══════════════════════════════════════════════════════════════════════════
// PRODUCTION CODE — Automotive Class Design
// ═══════════════════════════════════════════════════════════════════════════

// ─── 1. Rule of Zero: SafeVehicleData ──────────────────────────────────

// EN: C.20 — all members are RAII types; no user-defined special members needed
// TR: C.20 — tüm üyeler RAII türlerinden; kullanıcı tanımlı özel üye gerekmez
class SafeVehicleData {
public:
    std::string vin;
    std::vector<double> sensorReadings;
    std::unique_ptr<std::string> diagnosticLog;

    // EN: A regular constructor is fine — Rule of Zero still applies
    // TR: Normal bir kurucu sorun değil — Sıfır Kuralı hâlâ geçerli
    SafeVehicleData(std::string v, std::vector<double> readings)
        : vin(std::move(v)),
          sensorReadings(std::move(readings)),
          diagnosticLog(std::make_unique<std::string>("Initial log entry")) {}

    void print() const {
        std::cout << "  VIN: " << vin
                  << " | Sensors: " << sensorReadings.size()
                  << " | Log: " << (diagnosticLog ? *diagnosticLog : "null")
                  << std::endl;
    }
};

// ─── 2. Abstract Interface: IDiagnosticProtocol ────────────────────────

// EN: C.129 — pure interface (no data, all pure virtual)
// TR: C.129 — saf arayüz (veri yok, hepsi saf sanal)
class IDiagnosticProtocol {
public:
    virtual ~IDiagnosticProtocol() = default;

    // EN: C.67 — suppress copying on polymorphic base
    // TR: C.67 — polimorfik temel sınıfta kopyalamayı engelle
    IDiagnosticProtocol(const IDiagnosticProtocol&) = delete;
    IDiagnosticProtocol& operator=(const IDiagnosticProtocol&) = delete;

    virtual std::string protocolName() const = 0;
    virtual bool connect(const std::string& target) = 0;
    virtual std::string readDTC() const = 0;
    virtual void clearDTC() = 0;

protected:
    IDiagnosticProtocol() = default;
    IDiagnosticProtocol(IDiagnosticProtocol&&) = default;
    IDiagnosticProtocol& operator=(IDiagnosticProtocol&&) = default;
};
```

```
58
59  // ─── 3. Concrete: OBD2Protocol ──────────────────────────────────────────────
60
61  class OBD2Protocol : public IDiagnosticProtocol {
62      std::string connectedECU_;
63      bool connected_ = false;
64
65  public:
66      OBD2Protocol() = default;
67
68      // EN: C.128 — every override explicitly marked
69      // TR: C.128 — her geçersiz kılma açıkça işaretlendi
70      std::string protocolName() const override { return "OBD-II (ISO 15765)"; }
71
72      bool connect(const std::string& target) override {
73          connectedECU_ = target;
74          connected_ = true;
75          std::cout << "  [OBD2] Connected to " << target << std::endl;
76          return true;
77      }
78
79      std::string readDTC() const override {
80          if (!connected_) return "NOT_CONNECTED";
81          return "P0301 — Cylinder 1 Misfire Detected";
82      }
83
84      void clearDTC() override {
85          std::cout << "  [OBD2] DTCs cleared on " << connectedECU_ << std::endl;
86      }
87  };
88
89  // ─── 4. Concrete: CANProtocol ───────────────────────────────────────────────
90
91  class CANProtocol : public IDiagnosticProtocol {
92      std::string busName_;
93      bool connected_ = false;
94
95  public:
96      CANProtocol() = default;
97
98      std::string protocolName() const override { return "CAN 2.0B (ISO 11898)"; }
99
100     bool connect(const std::string& target) override {
101         busName_ = target;
102         connected_ = true;
103         std::cout << "  [CAN] Joined bus: " << target << std::endl;
104         return true;
105     }
106
107     std::string readDTC() const override {
108         if (!connected_) return "NOT_CONNECTED";
109         return "U0100 — Lost Communication with ECM/PCM";
110     }
111
112     void clearDTC() override {
113         std::cout << "  [CAN] DTCs cleared on bus " << busName_ << std::endl;
114     }
115 };
116
117 // ─── 5. Final Class: HardcodedConfig ────────────────────────────────────────
118
119 // EN: C.128 final — sealed class, no further derivation allowed
120 // TR: C.128 final — mühürlü sınıf, daha fazla türetme yasak
121 class HardcodedConfig final {
122     int baudRate_;
123     std::string protocol_;
124
125 public:
126     HardcodedConfig(int baud, std::string proto)
127         : baudRate_(baud), protocol_(std::move(proto)) {}
```

```cpp

    void print() const {
        std::cout << "  [Config] " << protocol_ << " @ " << baudRate_
                  << " baud (FINAL — immutable design)" << std::endl;
    }
};

// ─── 6. Deleted Copy: SingletonECU ──────────────────────────────────────────

// EN: C.21 + C.67 — singleton must not be copied or moved
// TR: C.21 + C.67 — tekil nesne kopyalanamaz veya taşınamaz
class SingletonECU {
    std::string ecuId_;

    // EN: Private constructor — only accessible via instance()
    // TR: Özel kurucu — yalnızca instance() üzerinden erişilebilir
    explicit SingletonECU(std::string id) : ecuId_(std::move(id)) {}

public:
    // EN: Delete ALL copy and move operations (C.21)
    // TR: TÜM kopya ve taşıma işlemlerini sil (C.21)
    SingletonECU(const SingletonECU&) = delete;
    SingletonECU& operator=(const SingletonECU&) = delete;
    SingletonECU(SingletonECU&&) = delete;
    SingletonECU& operator=(SingletonECU&&) = delete;

    static SingletonECU& instance() {
        static SingletonECU inst("MasterECU-001");
        return inst;
    }

    void identify() const {
        std::cout << "  [Singleton] I am: " << ecuId_
                  << " (one and only instance)" << std::endl;
    }
};

// ═══════════════════════════════════════════════════════════════════════════
// Helper: Polymorphic dispatch through interface pointer
// ═══════════════════════════════════════════════════════════════════════════

// EN: Liskov Substitution — works with ANY IDiagnosticProtocol subtype
// TR: Liskov İkamesi — HERHANGİ bir IDiagnosticProtocol alt türüyle çalışır
void runDiagnosticSession(IDiagnosticProtocol& proto, const std::string& target) {
    std::cout << "  Protocol : " << proto.protocolName() << std::endl;
    proto.connect(target);
    std::cout << "  DTC Read : " << proto.readDTC() << std::endl;
    proto.clearDTC();
}

// ═══════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "=== MODULE 8: C++ CORE GUIDELINES — CLASS DESIGN ===\n"
              << std::endl;

    // ─── Demo 1: Rule of Zero ──────────────────────────────────────────────
    std::cout << "[Demo 1] Rule of Zero — SafeVehicleData\n";
    {
        SafeVehicleData car1("WBA12345678901234", {36.5, 37.1, 38.0});
        car1.print();
```

```cpp

        // EN: Move semantics work automatically — no user-defined move ctor
        // TR: Taşıma semantiği otomatik çalışır — kullanıcı tanımlı taşıma yok
        SafeVehicleData car2 = std::move(car1);
        std::cout << "  After move:\n";
        std::cout << "    car2 -> ";
        car2.print();
        std::cout << "    car1 -> ";
        car1.print();
    }

    std::cout << std::endl;

    // ─── Demo 2: Polymorphism through Interface ──────────────────────────────────────
    std::cout << "[Demo 2] IDiagnosticProtocol — Polymorphic Dispatch\n";
    {
        OBD2Protocol obd2;
        CANProtocol can;

        std::cout << "  --- OBD2 Session ---\n";
        runDiagnosticSession(obd2, "EngineCU");

        std::cout << "  --- CAN Session ---\n";
        runDiagnosticSession(can, "CAN-HS-Bus");
    }

    std::cout << std::endl;

    // ─── Demo 3: Final Class ─────────────────────────────────────────────────────────
    std::cout << "[Demo 3] final class — HardcodedConfig\n";
    {
        HardcodedConfig cfg(500000, "CAN-FD");
        cfg.print();

        // EN: Attempting `class Derived : public HardcodedConfig {}` would be a compile error
        // because HardcodedConfig is final.
        // TR: `class Turetilmis : public HardcodedConfig {}` denemek derleme hatası verir çünkü
        // HardcodedConfig final'dır.
        std::cout << "  (Any attempt to inherit from HardcodedConfig → compile error)"
                  << std::endl;
    }

    std::cout << std::endl;

    // ─── Demo 4: Deleted Copy Operations ─────────────────────────────────────────────
    std::cout << "[Demo 4] Deleted Copy — SingletonECU\n";
    {
        SingletonECU& ecu = SingletonECU::instance();
        ecu.identify();

        SingletonECU& ecu2 = SingletonECU::instance();
        ecu2.identify();

        // EN: Both references point to the exact same object
        // TR: Her iki referans da tam olarak aynı nesneyi gösterir
        std::cout << "  Same instance? "
                  << (&ecu == &ecu2 ? "YES" : "NO") << std::endl;

        // EN: Uncommenting the next line would cause a compile error: SingletonECU copy = ecu;
        // // DELETED — C.21 enforced
        // TR: Aşağıdaki satırı açmak derleme hatası verir: SingletonECU kopya = ecu; // SİLİNMİŞ
        // — C.21 uygulanmış
    }

    std::cout << std::endl;

    // ─── Demo 5: override and final on Virtual Methods ───────────────────────────────
    std::cout << "[Demo 5] override / final Keywords\n";
    {
```

```
        // EN: Demonstrate that override catches errors at compile time
        // TR: override'ın derleme zamanında hataları yakaladığını göster
        std::unique_ptr<IDiagnosticProtocol> proto =
            std::make_unique<OBD2Protocol>();
        std::cout << "  Protocol via base ptr: " << proto->protocolName() << std::endl;

        auto proto2 = std::make_unique<CANProtocol>();
        std::cout << "  Protocol via base ptr: " << proto2->protocolName() << std::endl;

        // EN: Both correctly dispatch through virtual table thanks to override
        // TR: override sayesinde her ikisi de sanal tablo aracılığıyla doğru gönderilir
        std::cout << "  (Removing 'override' keyword would hide typo-bugs silently)"
                  << std::endl;
    }

    std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
    std::cout << "1. Rule of Zero: Let RAII members handle resource management." << std::endl;
    std::cout << "2. Pure interfaces + override/final = safe polymorphic hierarchies." << std::endl;
    std::cout << "3. Delete copy/move on singletons and polymorphic bases." << std::endl;

    return 0;
}
```

Modül 9: Advanced Mechanics — İleri Mekanikler

İleri Mekanikler

7 kaynak dosya · Standart: C++20

C++20

► 01. Bit Manipulation And Alignment

module_09_advanced_mechanics/01_bit_manipulation_and_alignment.cpp

Konu: Advanced Mechanics: Bit Manipulation & Memory Alignment / İleri Teknikler: Bit

Teori / Theory

1. Bitwise Manipulation

Hardware interfaces, embedded systems, and automotive ECU firmware rely on Bit Masking. Doing math at the bit level (`&`, `|`, `^`, `<<`, `>>`) is the absolute fastest way a CPU can operate. Standard C++ also provides `<bitset>` to make handling flags easier.

Bit Düzeyinde İşlemler

Donanım arayüzleri, gömülü sistemler ve otomotiv ECU firmware'leri "Bit Maskeleme" (Bit Masking) kurallarına dayanır. Bitlerle çalışmak (`VE`, `VEYA`, `XOR`, `Kaydırma`) CPU'nun dünyadaki en hızlı yaptığı matematiktir. Saf matematiksel hesapları bir kenara bırakıp bit mantığıyla bellek veya hız kazanımı sağlanır.

2. Memory Alignment & Padding (alignas)

Data structures are automatically "padded" by the compiler to align perfectly within the CPU's cache lines. If you organize variables poorly inside a `struct`, you can waste up to 50% of your RAM due to invisible Padding Bytes! We can control this using `alignas` or by ordering members largest-to-smallest.

Bellek Hizalaması

Sınıf veya Yapılar (Struct) bellekte rastgele yan yana durmazlar. C++ Derleyicisi verileri işlemcinin L1/L2 önbelleğine tam uyumlu oturtmak için aralarına gizli boşluklar (Padding) serpiştirir. Değişkenleri kötü sıralarsanız RAM israfı yaparsınız. Bunu kontrol etmek için `sizeof` bilincine ve `alignas` kuralına ihtiyacımız var.

CPPREF Depth: alignas, alignof, and the Strict Aliasing Rule

`alignof(T)` returns the alignment requirement of type T in bytes. `alignas(N)` overrides the default alignment, forcing a variable or type to be aligned to at least N bytes. The Strict Aliasing Rule (§6.10) states: accessing an object through a pointer of incompatible type is Undefined Behavior. Exception: `char*`, `unsigned char*`, and `std::byte*` may alias any type. GCC's `-fno-strict-aliasing` disables this optimization at the cost of performance. `std::aligned_storage` is deprecated in C++23 — prefer `alignas` with a `std::byte` array.

CPPREF Derinlik: alignas, alignof ve Katı Örtüşme Kuralı

`alignof(T)`, T türünün hizalama gereksinimini bayt olarak döndürür. `alignas(N)` varsayılan hizalamayı geçersiz kılarak değişkeni en az N bayta hizalar. Katı Örtüşme Kuralı (§6.10): bir nesneye uyumsuz türde bir işaretçi üzerinden erişmek Tanımsız Davranıştır. İstisna: `char*`, `unsigned char*` ve `std::byte*` her türe örtüşebilir. GCC'nin `-fno-strict-aliasing` seçeneği bu optimizasyonu performans pahasına devre dışı bırakır. `std::aligned_storage` C++23'te kullanımdan kaldırıldı — `alignas` + `std::byte` dizisi tercih edin.

Kaynak Kod / Source Code

```cpp
#include <bitset>
#include <iostream>

// ---------------------------------------------------------------------------------------------
// [SECTION 1] RAM WASTING vs OPTIMAL STRUCT (Padding/Hizalama İsrafı)
// ---------------------------------------------------------------------------------------------
// EN: Order: 1 byte, 8 bytes, 4 bytes.
// TR: Derleyici aralara boşluk (Padding) ekleneceği için 13 Byte yerine 24 Byte kaplar!
struct WastedMemoryStruct {
  char a;   // 1 byte (+7 bytes of hidden padding!) -> 8 byte block
  double b; // 8 bytes                              -> 8 byte block
  int c;    // 4 bytes (+4 bytes of hidden padding!)-> 8 byte block
}; // Total Size = 24 Bytes (Almost 50% Memory Wasted!)

// EN: Order: Largest to Smallest (8, 4, 1).
// TR: En büyükten en küçüğe sıralandığı için derleyici boşluk koymaz! İşlemci rahat eder.
struct OptimizedStruct {
  double b; // 8 bytes
  int c;    // 4 bytes
  char a;   // 1 byte (+3 bytes padding at the end)
}; // Total Size = 16 Bytes (Much better!)

// EN: Forcing absolute cache-line alignment (e.g. 64-bytes for AVX/SIMD instructions) TR:
// Gelişmiş donanım hızlandırmaları için objeyi kesinlikle 64-Byte katsayısına zorlamak:
struct alignas(64) SimdVector {
  float x, y, z, w; // Genelde matematik veya SIMD motorları için kullanılır.
};

// ---------------------------------------------------------------------------------------------
// [SECTION 2] BITWISE FLAGS vs BOOLEANS
// ---------------------------------------------------------------------------------------------
// EN: Instead of using 8 `bool` variables (which take 8 Full Bytes), use 8 Bits (1 Byte). TR: 8
// tane `bool` değişkeni açmak 8 Byte yer kaplar. 8 biti olan tek 1 Byte ile hepsini kontrol
// etmek:
const unsigned char FLAG_RUNNING = 1 << 0;    // 00000001 (Engine Running)
const unsigned char FLAG_ABS_ACTIVE = 1 << 1; // 00000010 (ABS Active)
const unsigned char FLAG_LIGHTS_ON = 1 << 2;  // 00000100 (Headlights On)
const unsigned char FLAG_DOOR_OPEN = 1 << 3;  // 00001000 (Door Open)

int main() {
  std::cout << "=== MODULE 9: BIT MANIPULATION & ALIGNMENT ===\n" << std::endl;

  // --- ALIGNMENT TESTS ---
  std::cout << "[MEMORY ALIGNMENT]" << std::endl;
  std::cout << "Size of WastedMemoryStruct: " << sizeof(WastedMemoryStruct) << " bytes" <<
      std::endl;
  std::cout << "Size of OptimizedStruct   : " << sizeof(OptimizedStruct) << " bytes" << std::endl;
  std::cout << "Size of alignas(64) Vector: " << sizeof(SimdVector) << " bytes\n" << std::endl;

  // --- BITWISE TESTS (C-Style) ---
  std::cout << "[BIT MANIPULATION (C-Style)]" << std::endl;
  unsigned char ecuStatusFlags = 0; // 00000000

  // Set flags (OR `|`) / Bayrakları Yak (VEYA)
  ecuStatusFlags |= FLAG_RUNNING;
  ecuStatusFlags |= FLAG_LIGHTS_ON;
```

```cpp

  // Check flags (AND `&`) / Bayrakları Kontrol Et (VE)
  if (ecuStatusFlags & FLAG_RUNNING) {
    std::cout << "- ECU Engine is Running!" << std::endl;
  }

  // Toggle flags (XOR `^`) / Bayrakları Tersine Çevir (XOR)
  ecuStatusFlags ^= FLAG_LIGHTS_ON; // Turns headlights flag OFF

  // --- BITWISE TESTS (Modern C++ std::bitset) ---
  std::cout << "\n[BIT MANIPULATION (Modern C++ <bitset>)]" << std::endl;
  // EN: Modern C++ provides a safe object wrapper around bits.
  // TR: C++ bitset ile manuel 0000'larla uğraşmanızı engeller, OOP yapısına çeker.
  std::bitset<8> modernECUStatus;
  modernECUStatus.set(0); // Bit 0 (Engine Running) -> True
  modernECUStatus.set(3); // Bit 3 (Brake Active) -> True

  std::cout << "Current 8-Bit Status: " << modernECUStatus << std::endl;
  std::cout << "Is brake active? : " << (modernECUStatus.test(3) ? "Yes" : "No") << std::endl;

  return 0;
}
```

► 02. Operator Overloading

module_09_advanced_mechanics/02_operator_overloading.cpp

Konu: Advanced Mechanics: Operator Overloading / İleri Teknikler: Operatör Aşırı Yükleme

Teori / Theory

Operator Overloading (Operatörlerin Ezilmesi

C++ allows us to modify what standard operators (`+`, `-`, `==`, `<<`, `()`) actually do when used with our custom Classes. This allows our custom Types to feel and act exactly like built-in native C++ types (`int`, `float`).

Aşırı Yüklenmesi)

C++ dili, kendi yarattığımız devasa Sınıfları (Objects) tıpkı bir `int` veya `float` gibi toplama `+`, çıkarma `-`, karılaştırma `==` operatörleriyle kullanabilmemiz için Operatör Ezme (Aşırı Yükleme) yetkisi verir.

Why overload `<<`?

If you type `std::cout << myObject;`, the compiler throws a FATAL ERROR. Because it doesn't know how to "Print" your custom object. By overloading `operator<<` as a `friend` function, we teach `std::ostream` how to read your object.

Neden Sol Kaydırma `<<` ezilir?

Eğer bir objeye `std::cout << obje` derseniz derleyici küfür eder. Çünkü objeyi ekrana nasıl basacağını bilemez. `operator<<` fonksiyonunu sınıfın içine bir `friend` (dost fonksiyon) olarak aşılarsak kütüphane objemizi şakır şakır okur.

CPPREF Depth: Operator Overloading — ADL, Hidden Friends, and the Spaceship

Argument-Dependent Lookup (ADL) finds `operator<<` in the namespace of the operand type, even without a `using` directive. The "hidden friend" idiom defines `friend operator==` inline inside the class — it is found only via ADL and prevents implicit conversions on the LHS operand. C++20 `operator<=>` (the "spaceship operator") auto-generates all six comparison operators (`<`, `>`, `<=`, `>=`, `==`, `!=`). Return types: `std::strong_ordering` (total order, substitutable), `std::weak_ordering` (total order, not substitutable), and `std::partial_ordering` (permits incomparable values, e.g., NaN).

CPPREF Derinlik: Operatör Aşırı Yükleme — ADL, Gizli Dostlar ve Uzay Gemisi

Argümana Bağlı Arama (ADL), `using` direktifi olmadan bile `operator<<` fonksiyonunu operandın ad alanında bulur. "Gizli dost" deyimi, sınıf içinde `friend operator==` tanımlar — yalnızca ADL ile bulunur ve sol operandda örtük dönüşümleri engeller. C++20 `operator<=>` ("uzay gemisi operatörü") altı karşılaştırma operatörünü otomatik üretir. Dönüş türleri: `std::strong_ordering` (tam sıralama, ikame edilebilir), `std::weak_ordering` (tam sıralama, ikame edilemez) ve `std::partial_ordering` (karşılaştırılamaz değerlere izin verir, ör: NaN).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <string>
#include <tuple>

// EN: A 2D Math Vector (Not std::vector array, but a mathematical Coordinate System) TR: 2D
// Matematik Vektörü (Dizilerdeki vector değil, fizikteki XYZ vektörü).
class Vector2D {
private:
  float x;
  float y;

public:
  Vector2D(float inputX = 0, float inputY = 0) : x(inputX), y(inputY) {}

  // ------------------------------------------------------------------------------------------
  // [1] Operator Overloading for Addition `+` (Toplama Operatörü)
  // ------------------------------------------------------------------------------------------
  // EN: Tells C++ what happens when we do: vectorA + vectorB
  // TR: vecA + vecB yaparsak arka planda tam olarak ne olacağını C++'a öğretiyoruz.
  Vector2D operator+(const Vector2D &other) const {
    return Vector2D(this->x + other.x, this->y + other.y);
  }

  // ------------------------------------------------------------------------------------------
  // [2] Operator Overloading for Equality `==` (Eşitlik Operatörü)
  // ------------------------------------------------------------------------------------------
  bool operator==(const Vector2D &other) const {
    // EN: Note: Float comparison requires an epsilon gap. simplified here.
    // TR: Normalde float kıyası risklidir (1.000001), ama eğitim için basit tutuyoruz.
    return (this->x == other.x && this->y == other.y);
  }
```

```cpp

  // ---------------------------------------------------------------------------------------------
  // [3] Operator Overloading for Function Call `()` (Functor Mimarisinin
  // Temeli)
  // ---------------------------------------------------------------------------------------------
  // EN: Allows an OBJECT to be called exactly like a FUNCTION. `myObject()`
  // TR: Bir OBJEYİ, sanki sıradan bir fonksiyonmuş gibi çağırmayı (`obje()`) sağlar.
  void operator()() const {
    std::cout << "[Functor Call] Vector coordinates: (" << x << ", " << y << ")" << std::endl;
  }

  // ---------------------------------------------------------------------------------------------
  // [4] STREAM OPERATOR OVERLOADING (`<<`) -> Using `friend`
  // ---------------------------------------------------------------------------------------------
  // EN: Stream operators CANNOT be normal member functions! They must be global or friends. TR:
  // Cout/Ostream fonksiyonları dışarıdan geldiği için `friend` anahtar kelimesi ile Sınıfımızın
  // 'Mahrem (Private)' bölgelerine erişim izni vermemiz gerekir!
  friend std::ostream &operator<<(std::ostream &out, const Vector2D &vec) {
    out << "[" << vec.x << ", " << vec.y << "]";
    // EN: We return out to chain calls: `cout << v1 << v2;` (Zincirleme)
    return out;
  }
};

// ═══════════════════════════════════════════════════════════════════════════════════════════
// EN: SensorReading — All 6 comparison operators via std::tie idiom
// TR: SensorReading — std::tie deyimiyle 6 karşılaştırma operatörünün tamamı
// ═══════════════════════════════════════════════════════════════════════════════════════════

class SensorReading {
private:
    double timestamp_;
    double value_;
    std::string sensor_id_;

public:
    SensorReading(double ts, double val, std::string id)
        : timestamp_(ts), value_(val), sensor_id_(std::move(id)) {}

    // ─── 1. Equality == ──────────────────────────────────────────────────────────────────
    // EN: Two readings are equal when both timestamp and value match.
    // TR: Zaman damgası ve değer eşleştiğinde iki okuma eşittir.
    bool operator==(const SensorReading& o) const {
        return std::tie(timestamp_, value_) == std::tie(o.timestamp_, o.value_);
    }

    // ─── 2. Inequality != ────────────────────────────────────────────────────────────────
    // EN: Defined in terms of == for consistency.
    // TR: Tutarlılık için == ile tanımlanır.
    bool operator!=(const SensorReading& o) const {
        return !(*this == o);
    }

    // ─── 3. Less-than < ──────────────────────────────────────────────────────────────────
    // EN: Lexicographic comparison via std::tie — first by timestamp, then value.
    // TR: std::tie ile sözlüksel karşılaştırma — önce zaman damgası, sonra değer.
    bool operator<(const SensorReading& o) const {
        return std::tie(timestamp_, value_) < std::tie(o.timestamp_, o.value_);
    }

    // ─── 4. Greater-than > ───────────────────────────────────────────────────────────────
    // EN: Flip the operands of < for > .
    // TR: > için < operandları ters çevrilir.
    bool operator>(const SensorReading& o) const {
        return o < *this;
    }
```

```cpp

    // ─── 5. Less-or-equal <= ──────────────────────────────────────────────────────
    // EN: Not greater means less-or-equal.
    // TR: Büyük değilse küçük-eşittir.
    bool operator<=(const SensorReading& o) const {
        return !(o < *this);
    }

    // ─── 6. Greater-or-equal >= ───────────────────────────────────────────────────
    // EN: Not less means greater-or-equal.
    // TR: Küçük değilse büyük-eşittir.
    bool operator>=(const SensorReading& o) const {
        return !(*this < o);
    }

    // ─── Stream output operator ───────────────────────────────────────────────────
    // EN: Prints sensor reading in automotive diagnostic format.
    // TR: Sensör okumasını otomotiv teşhis formatında yazdırır.
    friend std::ostream& operator<<(std::ostream& os, const SensorReading& sr) {
        os << "[" << sr.sensor_id_ << " @" << sr.timestamp_
           << "s : " << sr.value_ << "]";
        return os;
    }

    // ─── C++20 Spaceship Operator (Reference Only) ────────────────────────────────
    // EN: In C++20, a SINGLE declaration replaces all 6 comparison operators: auto
    // operator<=>(const SensorReading&) const = default; The compiler auto-generates ==, !=, <,
    // >, <=, >= from this one line. It performs member-wise lexicographic comparison in
    // declaration order.
    // TR: C++20'de TEK bir bildirim 6 karşılaştırma operatörünün hepsini değiştirir: auto
    // operator<=>(const SensorReading&) const = default; Derleyici bu tek satırdan ==, !=, <, >,
    // <=, >= operatörlerini üretir. Üye bazında, bildirim sırasına göre sözlüksel karşılaştırma
    // yapar.
};

// ═══════════════════════════════════════════════════════════════════════════════
// EN: EventCounter — Prefix and Postfix increment operators
// TR: EventCounter — Önek ve Sonek artırma operatörleri
// ═══════════════════════════════════════════════════════════════════════════════

class EventCounter {
private:
    int count_;
    std::string event_name_;

public:
    EventCounter(std::string name, int initial = 0)
        : count_(initial), event_name_(std::move(name)) {}

    // ─── Prefix ++counter (returns reference to incremented self) ─────────────────
    // EN: Prefix increment — increments first, then returns the new value.
    // TR: Önek artırma — önce artırır, sonra yeni değeri döndürür.
    EventCounter& operator++() {
        ++count_;
        return *this;
    }

    // ─── Postfix counter++ (int dummy distinguishes from prefix) ──────────────────
    // EN: Postfix increment — the dummy 'int' parameter tells the compiler this is postfix.
    // Returns the OLD value, then increments.
    // TR: Sonek artırma — 'int' kukla parametresi derleyiciye sonek olduğunu bildirir. ESKİ
    // değeri döndürür, sonra artırır.
    EventCounter operator++(int) {
        EventCounter old = *this;
        ++count_;
        return old;
    }

    friend std::ostream& operator<<(std::ostream& os, const EventCounter& ec) {
        os << "[" << ec.event_name_ << ": " << ec.count_ << "]";
        return os;
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════

int main() {
  std::cout << "=== MODULE 9: OPERATOR OVERLOADING ===\n" << std::endl;

  Vector2D pointVelocity_A(5.5f, -2.0f);
  Vector2D pointVelocity_B(2.0f, 4.0f);

  // 1. Addition Overload (O-Overload of +)
  // TR: Sanki int veya float imiş gibi doğal toplama yapar!
  Vector2D result = pointVelocity_A + pointVelocity_B;

  // 2. Stream Overload (O-Overload of <<)
  // TR: `<<` ezildiği (overload edildiği) için obje anında string algısı yaratır!
  std::cout << "Velocity A : " << pointVelocity_A << std::endl;
  std::cout << "Velocity B : " << pointVelocity_B << std::endl;
  std::cout << "Result (A+B): " << result << std::endl;

  // 3. Equality Overload (O-Overload of ==)
  if (pointVelocity_A == pointVelocity_B) {
    std::cout << "Both vectors are totally equal!" << std::endl;
  } else {
    std::cout << "Vectors are pointing to different directions." << std::endl;
  }

  // 4. Function Overload / Functor Mimari Testi (O-Overload of ())
  std::cout << "\nTriggering Functor Object Call:" << std::endl;
  // EN: Looks like a function, but it is actually the OBJECT acting like a function.
  result();

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: Demo 5 — All 6 comparison operators on SensorReading
  // TR: Demo 5 — SensorReading üzerinde 6 karşılaştırma operatörü
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- DEMO 5: SENSOR READING COMPARISONS ---" << std::endl;

  SensorReading rpm_early(1.00, 3200.0, "RPM_SENSOR");
  SensorReading rpm_late (2.50, 3200.0, "RPM_SENSOR");
  SensorReading rpm_same (1.00, 3200.0, "RPM_SENSOR");

  std::cout << "  A: " << rpm_early << std::endl;
  std::cout << "  B: " << rpm_late  << std::endl;
  std::cout << "  C: " << rpm_same  << std::endl;

  // EN: Testing all 6 comparison operators.
  // TR: 6 karşılaştırma operatörünün tümünü test ediyoruz.
  std::cout << "  A == C : " << (rpm_early == rpm_same  ? "true" : "false") << std::endl;
  std::cout << "  A != B : " << (rpm_early != rpm_late  ? "true" : "false") << std::endl;
  std::cout << "  A <  B : " << (rpm_early <  rpm_late  ? "true" : "false") << std::endl;
  std::cout << "  B >  A : " << (rpm_late  >  rpm_early ? "true" : "false") << std::endl;
  std::cout << "  A <= C : " << (rpm_early <= rpm_same  ? "true" : "false") << std::endl;
  std::cout << "  B >= A : " << (rpm_late  >= rpm_early ? "true" : "false") << std::endl;

  // ═══════════════════════════════════════════════════════════════════════════════
  // EN: Demo 6 — Prefix vs Postfix increment on EventCounter
  // TR: Demo 6 — EventCounter üzerinde önek vs sonek artırma
  // ═══════════════════════════════════════════════════════════════════════════════
  std::cout << "\n--- DEMO 6: PREFIX vs POSTFIX INCREMENT ---" << std::endl;

  EventCounter brakeEvents("BRAKE_PRESS", 0);
  std::cout << "  Initial : " << brakeEvents << std::endl;

  // EN: Prefix — increments then returns the NEW value.
  // TR: Önek — artırır, sonra YENİ değeri döndürür.
  std::cout << "  ++counter: " << ++brakeEvents << std::endl;
  std::cout << "  After    : " << brakeEvents   << std::endl;
```

```cpp

    // EN: Postfix — returns OLD value, then increments.
    // TR: Sonek — ESKİ değeri döndürür, sonra artırır.
    std::cout << "  counter++: " << brakeEvents++ << std::endl;
    std::cout << "  After    : " << brakeEvents   << std::endl;

    // ═══════════════════════════════════════════════════════════════════════════════
    // EN: Demo 7 — Stream output operator for SensorReading
    // TR: Demo 7 — SensorReading için akış çıkış operatörü
    // ═══════════════════════════════════════════════════════════════════════════════
    std::cout << "\n--- DEMO 7: STREAM OUTPUT FOR SENSOR READING ---" << std::endl;

    SensorReading coolant(0.50, 87.3,  "COOLANT_TEMP");
    SensorReading battery(0.50, 13.8,  "BATTERY_VOLT");
    SensorReading oil    (1.20, 102.5, "OIL_PRESSURE");

    // EN: The overloaded << operator prints in automotive diagnostic format.
    // TR: Aşırı yüklenmiş << operatörü otomotiv teşhis formatında yazdırır.
    std::cout << "  " << coolant << std::endl;
    std::cout << "  " << battery << std::endl;
    std::cout << "  " << oil     << std::endl;

    // EN: Chained stream output — works because operator<< returns ostream&.
    // TR: Zincirleme akış çıkışı — operator<< ostream& döndürdüğü için çalışır.
    std::cout << "  Chained: " << coolant << " | " << battery << " | " << oil << std::endl;

    return 0;
}
```

► 03. Advanced Pointers

module_09_advanced_mechanics/03_advanced_pointers.cpp

Konu: Advanced Mechanics: Pointers to Member Functions & std::invoke / Üye Fonksiyon ve

Teori / Theory

Pointers to Data Members

Regular pointers (`int*`) point to a specific memory address AT RUNTIME. Pointers to class members (`int ECUModule::*`) don't point to RAM until they are combined with an actual instantiated Object! They just hold an "offset" inside.

Bir Sınıfın Değişkenini İşaret Etmek

Normal Pointerlar (`int*`) çalışma zamanında RAM'de kesin bir adres tutar. (0x1A2...) Ama Sınıf Üye Pointerları (`int ECUModule::*`), RAM adresi DEĞİL, Değişkenin veya Fonksiyonun Sınıfın içindeki göreceli (Offset) indeksini/formülünü tutar. Bunu çağırmak için GERÇEK BİR OBJEYE (instance) ihtiyaç duyarız.

Pointers to Member Functions

Used massively in Automotive Middleware (like AUTOSAR Adaptive) or UI frameworks (Qt Signals/Slots) to store a "Function call to be executed later on an object". Modern C++17 provides `std::invoke` which makes the ugly syntax beautiful.

Otomotiv Middleware Delegasyonu İçin

AUTOSAR Adaptive içindeki Delegate'ler veya Qt UI sinyalleri bu prensiple çalışır. Bir ECU modülünün bir komutu tetiklemesi "işlemini" bir Pointer içine kaydederiz ve olay tetiklendiğinde "O objenin üzerindeki kaydettiğimiz bu fonksiyonu" çalıştırırız.

CPPREF Depth: Pointer Arithmetic, Provenance, and std::launder

Pointer arithmetic is defined only within an array (including one-past- the-end). Dereferencing a pointer outside these bounds is UB. Pointer provenance: even if two pointers hold the same numeric address, they may not be interchangeable — the optimizer tracks "where" a pointer came from. `std::launder` (C++17) creates a new pointer to an object at a given address, bypassing provenance tracking. It is needed after placement-new overwrites an object whose type contains non-trivial `const` or reference members, because the compiler may cache the old values through the original pointer.

CPPREF Derinlik: İşaretçi Aritmetiği, Köken ve std::launder

İşaretçi aritmetiği yalnızca bir dizi içinde (sondan-bir-sonrası dahil) tanımlıdır. Bu sınırlar dışındaki bir işaretçinin referansını kaldırmak UB'dir. İşaretçi kökeni: iki işaretçi aynı sayısal adresi taşısa bile birbirinin yerine kullanılamayabilir — derleyici işaretçinin "nereden geldiğini" izler. `std::launder` (C++17), verilen bir adresteki nesneye yeni bir işaretçi oluşturarak köken takibini atlar. Placement-new ile `const` veya referans üyeler içeren bir nesne üzerine yazıldığında gereklidir, çünkü derleyici eski değerleri orijinal işaretçi üzerinden önbelleğe almış olabilir.

Kaynak Kod / Source Code

```cpp
#include <functional> // for std::invoke (C++17)
#include <iostream>
#include <string>

class ECUModule {
public:
  std::string name;
  int messageBudget;
  int uptimeHours;

  // Constructor
  ECUModule(std::string n, int budget, int uptime)
      : name(std::move(n)), messageBudget(budget), uptimeHours(uptime) {}

  // Target Member Function 1
  void sendCANFrame(int priority) {
    if (messageBudget >= priority) {
      messageBudget -= priority;
      std::cout << "[" << name << "] CAN frame sent (Priority: " << priority
                << "). Budget left: " << messageBudget << std::endl;
    } else {
      std::cout << "[" << name << "] CAN budget exhausted!" << std::endl;
    }
  }

  // Target Member Function 2
  void activateDiagMode(int durationSecs) {
    std::cout << "[" << name << "] Diagnostic mode activated for "
              << durationSecs << " seconds." << std::endl;
  }
};

int main() {
  std::cout << "=== MODULE 9: POINTERS TO MEMBERS (ADVANCED) ===\n" << std::endl;

  ECUModule bcm("BCM_BodyControl", 50, 100);
  ECUModule hvac("HVAC_Climate", 10, 50);
```

```cpp

    // ==========================================================================================
    // 1. Pointer to a DATA MEMBER (Veri değişkeni işaretçisi)
    // ==========================================================================================
    // EN: The ugly C++ syntax: Define a pointer that CAN hold an 'int' belonging to 'ECUModule'.
    // TR: Ucube Klasik C++ Syntaxı: 'ECUModule'in içindeki bir 'int' değişkeni tutabilen formül.
    int ECUModule::*pDataMetric = &ECUModule::messageBudget;

    // Accesses messageBudget!
    std::cout << "[DATA POINTER]: BCM Budget:  " << bcm.*pDataMetric << std::endl;
    // Same pointer, different object!
    std::cout << "[DATA POINTER]: HVAC Budget: " << hvac.*pDataMetric << std::endl;

    // EN: Change the pointer to point to 'uptimeHours' instead of 'messageBudget'.
    // TR: Formülü değiştir. Artık Mesaj Bütçesi değil, 'Çalışma Süresi' değerini tutar.
    pDataMetric = &ECUModule::uptimeHours;
    std::cout << "[DATA POINTER]: BCM Uptime:  " << bcm.*pDataMetric << "\n" << std::endl;

    // ==========================================================================================
    // 2. Pointer to a MEMBER FUNCTION (Üye Fonksiyon işaretçisi + Delegate Mantığı)
    // ==========================================================================================
    // EN: The UGLIEST Syntax known to mankind. -> return_type (Class::*pointerName)(args); TR: C++
    // insanlığa karşı bir suç olan Syntax: void (ECUModule::*pAction)(int);
    void (ECUModule::*pAction)(int) = &ECUModule::sendCANFrame;

    // EN: Native execution (Requires the painful `.*` or `->*` operators)
    // TR: Geleneksel/Korkunç yürütme şekli (Parantez cehennemi!)
    (bcm.*pAction)(10); // BCM sends a CAN frame!

    // EN: Wait! Let's change the "Action" mapping to diagnostic mode instead!
    // TR: Dur! Eylem atamasını değiştir, aynı pointer artık 'Diagnostic' fonksiyonunu kaydetsin!
    pAction = &ECUModule::activateDiagMode;
    (hvac.*pAction)(5); // HVAC activates diagnostic mode!

    // ==========================================================================================
    // 3. MODERN C++17 `std::invoke` (The Hero / Kurtarıcı)
    // ==========================================================================================
    // EN: std::invoke completely solves the horrible syntax above.
    // TR: std::invoke, C++ Komitesinin bu korkunç `.*` saçmalığını yok etmek için C++17'de yazdığı
    // çözümdür.
    std::cout << "\n[C++17 std::invoke MAGIC]" << std::endl;

    // Just invoke (Execute) the functionPointer ON the specific object WITH its arguments! TR:
    // Çağır (Invoke et) -> Neyi? pAction'ı -> Neyin Üstünde? bcm -> Hangi Argümanla? 15 BCM
    // activates diagnostic mode using modern magic.
    std::invoke(pAction, bcm, 15);

    // Also works naturally with Member Data (Değişkenlerde de aynı temizlik geçerlidir) Gets
    // HVAC's uptime smoothly!
    int currentParam = std::invoke(pDataMetric, hvac);
    std::cout << "Invoked HVAC Uptime: " << currentParam << std::endl;

    return 0;
}
```

► 04. Constant Evaluation

module_09_advanced_mechanics/04_constant_evaluation.cpp

Konu: Advanced Mechanics: C++20 Constants (constexpr vs consteval vs constinit) / Sabit

Teori / Theory

1. constexpr

A `constexpr` function CAN be evaluated at compile-time (if all inputs are known). But if inputs are dynamic (from user), it naturally degrades to a normal run-time function! It is basically saying: "Compiler, please do this beforehand IF you can."

Derleme Zamanı VEYA Çalışma Zamanı Mimarisi (C++11/14)

`constexpr` fonksiyon, derleme (Build) anında işlemcinin hesaplayabileceği bir emirdir. ANCAK değişkenler kullanıcı elinden dinamik olarak gelirse, derleyici mırın kırın etmez, "Tamam bunu oyun içerisinde Runtime'da normal fonksiyon gibi yapacağım" der. Yani hem derlemede hem oyun anında esnek çalışır.

2. consteval

A `consteval` function (Immediate Function) MUST strictly be evaluated at compile-time! If the compiler cannot determine the inputs beforehand, it throws a HARD BUILD ERROR. Use this when you absolutely CANNOT afford calculation overhead in the final binary execution.

YALNIZCA Derleme Zamanı Şartı (C++20)

`consteval` "Hemen Hesapla!" (C++20) anlamına gelir. Eğer bu fonksiyon derleme aşamasında bitmezse Oyun asla derlenemez. Askeri/Cerrahi robot yazılımlarında, hızın sıfır kayıp (0 overhead) yaşaması istenen kriptolu hash / konfigürasyon işlemlerinde mecburidir.

3. constinit

The `constinit` keyword guarantees that a static/global variable is fully initialized during compilation (Constant Initialization). It solves the infamous "Static Initialization Order Fiasco" in massive C++ engines. The variable CAN be mutable (changeable) later!

Statik Değerlerin Temiz Başlangıcı (C++20)

`constinit` kelimesi Global veya Static değişkenleri hedefler. Sadece "Oyun açılmadan önce ilk değerini" derleme aşamasında sabitler. Ama `const` DEĞİLDİR! Yani oyun akarken sonradan değiştirilebilir! C++ motorlarındaki meşhur Global çökme bug'larını kökünden engeller.

CPPREF Depth: constexpr vs consteval vs constinit — Compile-Time Guarantees

`constexpr` (C++11): function/variable CAN be evaluated at compile time if all inputs are constant; otherwise degrades to runtime. `consteval` (C++20): "immediate function" — MUST be evaluated at compile time; runtime call is a hard error. `constinit` (C++20): the variable must be initialized at compile time, but may be modified at runtime — solves the static initialization order fiasco. `if consteval` (C++23) tests whether execution is currently in a constexpr evaluation context. Constexpr dynamic allocations (C++20): `new`/`delete` are allowed inside constexpr if fully deallocated by the end of evaluation.

CPPREF Derinlik: constexpr vs consteval vs constinit — Derleme Zamanı Garantileri

`constexpr` (C++11): tüm girdiler sabit ise derleme zamanında değerlendirilebilir; aksi halde çalışma zamanına düşer. `consteval` (C++20): "anlık fonksiyon" — derleme zamanında hesaplanmak ZORUNDADIR; çalışma zamanı çağrısı derleme hatasıdır. `constinit` (C++20): değişken derleme zamanında başlatılmalıdır ama çalışma zamanında değiştirilebilir — statik başlatma sırası fiyaskosunu çözer. `if consteval` (C++23), yürütmenin şu anda constexpr bağlamında olup olmadığını test eder. Constexpr dinamik tahsisler (C++20): `new`/`delete` değerlendirme sonunda tam serbest bırakılırsa constexpr içinde kullanılabilir.

Kaynak Kod / Source Code

```cpp
#include <iostream>

// ==========================================================================================
// [constexpr] function (Flex / Esnek)
// ==========================================================================================
constexpr int squareConstexpr(int n) { return n * n; }

// ==========================================================================================
// [consteval] function (Strict Compile-Time / ASLA Çalışma Zamanına Kalmaz)
// ==========================================================================================
// C++20 Feature! Only available if compiled with -std=c++20
consteval int squareConsteval(int n) {
  return n * n; // Strictly must be replaced by the answer before ECU boots!
}

// ==========================================================================================
// [constinit] Variable (No Runtime startup cost, but Mutable)
// ==========================================================================================
// Calculated at compile time (100).
constinit int GlobalConfigurationScale = squareConstexpr(10);
// Notice it is an `int` not `const int`. It can be mutated later!
// TR: Oyun başlamadan önce bu (100) oldu. Oyun oynanırken 500 yapılabilir!

int main() {
  std::cout << "=== MODULE 9: CONSTANTS DEEP DIVE (C++20) ===\n" << std::endl;

  // --- 1. constexpr (ESNEK TEST) ---
  // TR: Parametre sabit (5), o halde 25 olarak Derleme Zamaninda bitir ve Binaya gom!
  constexpr int compileTimeResult = squareConstexpr(5);

  int dynamicVar = 8;
  // TR: Parametre dışarıdan/dosyadan geldi, derleyici pes etti, bunu ÇALIŞMA (Run) zamanında
  // hesapladı!
  int runTimeResult = squareConstexpr(dynamicVar);

  // --- 2. consteval (KUSURSUZ ASKER TESTİ) ---
  // TR: Bu %100 Build aşamasında Assembly'e gömüldü. Maliyet 0 CPU.
  int immediateResult = squareConsteval(6); // 36 is baked directly!

  std::cout << "constexpr(5)=" << compileTimeResult << " | constexpr(" << dynamicVar << ")=" <<
      runTimeResult << " | consteval(6)=" << immediateResult << std::endl;
```

```cpp

    // [FATAL ERROR]: This would completely crash the compilation! int
    // failedAttempt = squareConsteval(dynamicVar);
    // EN: The compiler cannot know what `dynamicVar` is at compile time, so `consteval` throws an
    // error! TR: Derleyici dynamicVar'ın kaç olduğunu bilemez, C++20'ye göre cinayet!

    // --- 3. constinit (GLOBAL/STATIC TEST) ---
    std::cout << "Original constinit global Config Scale: " << GlobalConfigurationScale << std::endl;
    // EN: Still mutable! It just started without runtime cost.
    GlobalConfigurationScale = 450;
             // TR: Başlangıç değeri 100 olarak derlemeden geldi ama sonra değiştirebildik!
    std::cout << "Mutated Global Config Scale           : " << GlobalConfigurationScale << std::endl;

    std::cout << "\n[ARCHITECTURAL RECAP]:" << std::endl;
    std::cout << "1. Use 'constexpr' by default. (It falls back to runtime "
                 "gently). // Genelde bunu kullan."
              << std::endl;
    std::cout << "2. Use 'consteval' when overhead is forbidden. // Hız kaybı "
                 "ölümcül ise bunu kullan."
              << std::endl;
    std::cout << "3. Use 'constinit' on Globals/Statics to prevent startup "
                 "fiascos. // Globalleri güvene al."
              << std::endl;

    return 0;
  }
```

► 05. Type Traits And Concepts

module_09_advanced_mechanics/05_type_traits_and_concepts.cpp

Konu: Advanced Mechanics: Type Traits (SFINAE) & C++20 Concepts / Kısıtlandırılmış Şablonlar

Teori / Theory

1. Type Traits & SFINAE (Substitution Failure Is Not An Error) - Eski C++ (C++11/14)

When using C++ Templates (`template <typename T>`), `T` can be ANYTHING (an `int`, a `std::string`, or an `ECUModule`). To restrict `T` (e.g., "Only accept numbers!"), modern C++ uses `<type_traits>` like `std::is_arithmetic_v<T>` combined with `std::enable_if`. If a substitution fails, the compiler simply ignores the overload rather than throwing an error (SFINAE!).

C++ Şablonlarını (Templates) kullanırken, Giren 'T' (Type) kelimesi HER ŞEY olabilir. Bir adam oraya 'Helikopter' sınıfını bile atabilir. Derleyiciyi kısıtlamak ("Buraya sadece rakam girebilir!") için Type Traits ve SFINAE denilen ürkütücü derecede karmaşık, ve efsanevi `std::enable_if` mekanizması kullanılırdı.

2. Concepts: The Savior of C++20

SFINAE is notoriously unreadable and generates 500-page long compiler error logs. C++20 introduced `requires` and `Concepts` to completely replace SFINAE! Concepts allow us to write clean, extremely readable restrictions on Templates.

Kurtarıcı Kavramlar (Modern C++20)

SFINAE mekanizması korkunç uzun hatalar fırlatmasıyla meşhurdur. C++ Komitesi, C++20 ile birlikte "Oluşumları/Kavramları (Concepts)" piyasaya sürdü! Artık `requires` yazarak kodun tamı tamına ne kabul ettiğini %100 okunabilirlikle derleyiciye izah edebiliyoruz. 500 sayılık hatalar yerine "Bu tipe izin yok!" alırız.

CPPREF Depth: Type Traits Implementation — How is_same and conditional Work Under the Hood

`is_same<T,U>`: primary template inherits `false_type`; partial specialization `is_same<T,T>` inherits `true_type`. `conditional<B,T,F>`: primary template defines `type = T`; specialization for `false` defines `type = F`. `enable_if<true, T>` defines member `type = T`; `enable_if<false>` has no `type` — this triggers SFINAE when used in template parameter deduction. `void_t<Ts...>` equals `void` if all `Ts` are well-formed — this is the detection idiom for checking members. C++20 concepts replace most SFINAE patterns with cleaner syntax: `requires` clauses, concept definitions, and constrained `auto`.

CPPREF Derinlik: Tür Özelliklerinin İç Yüzü — is_same ve conditional Nasıl Çalışır

`is_same<T,U>`: birincil şablon `false_type`'dan türer; kısmi özelleştirme `is_same<T,T>` ise `true_type`'dan türer. `conditional<B,T,F>`: birincil şablon `type = T` tanımlar; `false` özelleştirmesi `type = F` tanımlar. `enable_if<true, T>` `type = T` üyesini tanımlar; `enable_if<false>` hiçbir `type` içermez — şablon parametre çıkarımında SFINAE'yi tetikler. `void_t<Ts...>`, tüm `Ts` geçerliyse `void`'e eşittir — üye tespit deyimi (detection idiom) budur. C++20 kavramları (concepts), SFINAE kalıplarının çoğunu daha temiz bir sözdizimi ile değiştirir: `requires` cümleleri, kavram tanımları ve kısıtlı `auto`.

Kaynak Kod / Source Code

```cpp
#include <concepts> // Modern C++20 Concepts!
#include <iostream>
#include <string>
#include <type_traits> // Old C++11 traits

class Helicopter {}; // Dummy object / Eğlence için ucube obje

// ==========================================================================================
// [APPROACH 1: The OLD SFINAE Way (C++14) - Extremely Ugly and Cryptic]
// ==========================================================================================
// EN: "This function only exists if T is an arithmetic (number) type."
// TR: "Bu fonksiyon SADECE 'T' bir sayı türüyse kabul edilsin, yoksa yok sayılsın."
template <typename T, typename std::enable_if_t<std::is_arithmetic_v<T>, bool> = true>
void addTaxes_SFINAE(T price) {
  std::cout << "[SFINAE (Old)] New Price: " << (price * 1.18) << std::endl;
}

// ==========================================================================================
// [APPROACH 2: The NEW C++20 Concept Way - Beautiful and Readable!]
// ==========================================================================================
// EN: "This function REQUIRES 'T' to be a floating-point number (double/float)." TR: "Bu şablon
// FONKSİYON, T'nin ONDALIKLI SAYI (float, double) olmasını EMREDER! (requires)"
template <typename T>
  requires std::floating_point<T>
void addTaxes_Concept(T price) {
  std::cout << "[C++20 Concept] Floating Tax Calculation: " << (price * 1.18) << std::endl;
}

// EN: We can even create our Custom Concepts!
// TR: Kendi emrimizi / kısıtlamamızı özel Concept kelimesiyle (Mimarisi) yaratabiliriz!
template <typename T>
concept MustBePrintable = requires(T x) {
  // If this code compiles, the concept is met! (Bu derlenirse kabul!)
  std::cout << x;
};

// Target structure that enforces OUR custom concept. (Bizim konseptimiz!)
template <MustBePrintable T> void customPrinter(T data) {
  std::cout << "[Custom Concept Printer]: > " << data << std::endl;
}
```

```cpp

int main() {
  std::cout << "=== MODULE 9: TYPE TRAITS & CONCEPTS (C++20) ===\n" << std::endl;

  Helicopter apache;
  (void)apache; // EN: Used only for SFINAE rejection demo

  // --- 1. SFINAE LOGIC (C++11/14) ---
  addTaxes_SFINAE(100);  // int is arithmetic. OK! (int izinlidir)
  addTaxes_SFINAE(45.5); // double is arithmetic. OK! (double izinlidir)
  // addTaxes_SFINAE("10");     // SFINAE REJECTS (String is not arithmetic)
  // addTaxes_SFINAE(apache);   // SFINAE REJECTS (Helicopter cannot pay taxes)

  // --- 2. C++20 CONCEPTS LOGIC --- addTaxes_Concept(100);    // CONCEPT REJECTS / BUILD ERROR!
  // (100 is int, NOT a float!)
  addTaxes_Concept(45.5f); // OK! (float is floating_point)
  addTaxes_Concept(20.90); // OK! (double is floating_point)

  // --- 3. CUSTOM CONCEPT IMPLEMENTATION ---
  customPrinter(999);           // Integers can be printed via std::cout
  customPrinter("Hello World"); // Strings can be printed via std::cout
  // customPrinter(apache);      // WILL CAUSE AN ERROR! (No overload for << on Helicopter)

  std::cout << "\n[ARCHITECTURAL CONCLUSION / MİMARİ SONUÇ]:" << std::endl;
  std::cout << "- SFINAE and enable_if are obsolete. Move purely to C++20 "
               "`requires` & `Concepts`."
            << std::endl;
  std::cout << "- C++20 Kavramları, karmaşık Template kodlarının "
               "okunabilirliğini C#'taki Generics seviyesine taşır!"
            << std::endl;

  return 0;
}
```

► 06. Sfinae And Tag Dispatch

module_09_advanced_mechanics/06_sfinae_and_tag_dispatch.cpp

Konu: SFINAE, Tag Dispatch & if constexpr — Compile-Time Overload Selection

Teori / Theory

THEORY

1. SFINAE — "Substitution Failure Is Not An Error"

When the compiler tries to instantiate a function template, it substitutes the provided type arguments into the template signature. If that substitution produces an invalid type or expression, the compiler does NOT emit a hard error — it simply removes that overload from the candidate set. This is the SFINAE principle. Combined with std::enable_if, we can conditionally enable/disable overloads based on type properties at compile time.

TEORİ

Derleyici bir fonksiyon şablonunu somutlaştırmaya çalışırken, sağlanan tür argümanlarını şablon imzasına yerleştirir. Bu yerleştirme geçersiz bir tür veya ifade üretirse, derleyici hata VERMEZ — o aşırı yüklemeyi (overload) aday kümesinden sessizce çıkarır. Bu SFINAE prensibidir.

2. std::enable_if<condition, T>

enable_if conditionally defines a nested type `type` only when `condition` is true. When false, the substitution fails (SFINAE) and the overload is discarded. This lets us create separate function overloads for integral vs floating-point vs custom types — all resolved at compile time.

enable_if, yalnızca koşul doğru olduğunda iç içe `type` türünü tanımlar. Yanlış olduğunda yerleştirme başarısız olur (SFINAE) ve aşırı yükleme atılır. Böylece tamsayı, ondalıklı ve özel türler için ayrı fonksiyon aşırı yüklemeleri — hepsi derleme zamanında çözülür.

3. Tag Dispatch

Tag dispatch uses empty struct "tags" as function parameters to guide overload resolution at compile time. The caller passes a tag object, and the compiler selects the matching overload. This gives clean, readable compile-time branching without any enable_if clutter.

Etiket dağıtımı, derleme zamanında aşırı yükleme çözümlemesini yönlendirmek için boş yapı "etiketlerini" fonksiyon parametresi olarak kullanır. Çağıran bir etiket nesnesi geçirir, derleyici eşleşen aşırı yüklemeyi seçer.

4. if constexpr (C++17)

C++17 introduced `if constexpr` — a compile-time if-statement. The branch not taken is completely discarded by the compiler (not even type-checked). This is a much cleaner alternative to SFINAE for simple compile-time branching inside a single function template.

C++17 ile gelen `if constexpr`, derleme zamanı if ifadesidir. Alınmayan dal derleyici tarafından tamamen atılır (tür kontrolü bile yapılmaz). Basit derleme zamanı dallanma için SFINAE'den çok daha temiz bir yoldur.

5. Type Traits Used: std::is_integral, std::is_floating_point, std::is_same

The <type_traits> header provides compile-time boolean queries about types. std::is_integral_v<T> is true for int, long, char, etc. std::is_floating_point_v<T> is true for float, double, long double. std::is_same_v<T, U> checks if T and U are the exact same type. These traits are the building blocks of all SFINAE-based dispatch.

<type_traits> başlığı, türler hakkında derleme zamanı sorgulamaları sağlar. Bu özellikler, tüm SFINAE tabanlı dağıtımın yapı taşlarıdır.

CPPREF Depth: SFINAE Contexts and the Expression SFINAE Revolution

SFINAE only operates in the "immediate context" of template argument substitution — errors inside a function body are hard compilation errors, NOT SFINAE. Expression SFINAE (C++11): placing `decltype(expr)` in the return type triggers SFINAE if `expr` is ill-formed. `std::void_t<>` simplifies member detection by mapping any well-formed type list to `void`. `if constexpr` (C++17) eliminates most SFINAE need for branching within a single function body. `requires` expressions (C++20) are the final evolution — readable, composable constraints with clear compiler error messages that replace the arcane enable_if chains.

CPPREF Derinlik: SFINAE Bağlamları ve İfade SFINAE Devrimi

SFINAE yalnızca şablon argüman yerleştirmesinin "yakın bağlamında" çalışır — fonksiyon gövdesindeki hatalar SFINAE değil, kesin derleme hatalarıdır. İfade SFINAE (C++11): dönüş türüne `decltype(expr)` yerleştirildiğinde, `expr` geçersizse SFINAE tetiklenir. `std::void_t<>`, herhangi bir geçerli tür listesini `void`'e eşleyerek üye tespitini kolaylaştırır. `if constexpr` (C++17), tek bir fonksiyon gövdesindeki dallanma için SFINAE ihtiyacını büyük ölçüde ortadan kaldırır. `requires` ifadeleri (C++20) son evrimdir — gizemli enable_if zincirlerinin yerini alan okunabilir, birleştirilebilir kısıtlamalar ve açık hata mesajları.

Kaynak Kod / Source Code

```cpp
#include <cstdint>      // EN: Fixed-width integer types (uint8_t)
#include <iostream>     // EN: Standard I/O for console output
#include <string>       // EN: std::string for text data
#include <type_traits>  // EN: Type traits (is_integral, enable_if, etc.)
#include <sstream>      // EN: String stream for serialization

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: SFINAE with std::enable_if — ECU Data Processing
// ═══════════════════════════════════════════════════════════════════════════

// EN: Process integral ECU data (raw ADC counts, register values, CAN IDs)
// TR: Tamsayı ECU verilerini işle (ham ADC sayıları, register değerleri, CAN ID'leri)
template <typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
void processECUData(T value) {
    std::cout << "  [INTEGRAL path] Raw ECU register value: 0x"
              << std::hex << value << std::dec
              << " (" << value << " decimal)" << std::endl;
}

// EN: Process floating-point ECU data (calibrated sensor readings, voltages)
// TR: Ondalıklı ECU verilerini işle (kalibre sensör okumaları, voltajlar)
template <typename T, std::enable_if_t<std::is_floating_point_v<T>, int> = 0>
void processECUData(T value) {
    std::cout << "  [FLOAT path] Calibrated sensor reading: "
              << value << " V" << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 2: Tag Dispatch — Sensor Data Pipeline
// ═══════════════════════════════════════════════════════════════════════════

// EN: Tag types — empty structs used solely for overload selection
// TR: Etiket türleri — yalnızca aşırı yükleme seçimi için kullanılan boş yapılar
struct RawSensorTag {};
struct CalibratedSensorTag {};

// ─── 2a. Implementation overloads selected by tag ──────────────────────────

// EN: Process raw sensor data (needs filtering, offset correction)
// TR: Ham sensör verisini işle (filtreleme, ofset düzeltmesi gerekir)
template <typename T>
void processSensorData(T value, RawSensorTag) {
    constexpr double offset = 0.33;
    constexpr double gain   = 1.05;
    double corrected = static_cast<double>(value) * gain - offset;
    std::cout << "  [RAW] Input: " << value
              << " -> After offset/gain correction: " << corrected << std::endl;
}

// EN: Process calibrated sensor data (already engineering units)
// TR: Kalibre sensör verisini işle (zaten mühendislik birimlerinde)
template <typename T>
void processSensorData(T value, CalibratedSensorTag) {
    std::cout << "  [CALIBRATED] Direct engineering value: " << value
              << " (no correction needed)" << std::endl;
}

// ─── 2b. Public dispatch interface ─────────────────────────────────────────

// EN: Caller passes a tag to choose the processing path at compile time
// TR: Çağıran, derleme zamanında işleme yolunu seçmek için bir etiket geçirir
template <typename T, typename Tag>
void dispatchSensor(T value, Tag tag) {
    processSensorData(value, tag);
}
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: if constexpr (C++17) — Modern Replacement
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Single function with compile-time branching — no SFINAE boilerplate
// TR: Derleme zamanı dallanmalı tek fonksiyon — SFINAE şablonu gerekmez
template <typename T>
void processModern(T value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "  [if constexpr → INTEGRAL] Register: 0x"
                  << std::hex << value << std::dec << std::endl;
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "  [if constexpr → FLOAT] Sensor voltage: "
                  << value << " V" << std::endl;
    } else {
        std::cout << "  [if constexpr → OTHER] Unknown data type, forwarding raw bytes"
                  << std::endl;
    }
}

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: Has-Member Detection via SFINAE — .toString() check
// ═══════════════════════════════════════════════════════════════════════════════

// ─── 4a. Detector trait ──────────────────────────────────────────────────────

// EN: Primary template — fallback when T does NOT have .toString()
// TR: Birincil şablon — T'de .toString() YOKSA geri dönüş
template <typename T, typename = void>
struct HasToString : std::false_type {};

// EN: Specialization — enabled only when T has a .toString() method
// TR: Uzmanlık — yalnızca T'de .toString() metodu olduğunda etkinleşir
template <typename T>
struct HasToString<T, std::void_t<decltype(std::declval<T>().toString())>>
    : std::true_type {};

// ─── 4b. Serialize function using the detector ───────────────────────────────

// EN: If T has .toString(), use it; otherwise fall back to stream insertion
// TR: T'de .toString() varsa onu kullan; yoksa akış ekleme operatörüne dön
template <typename T>
auto serialize(const T& val) -> std::string {
    if constexpr (HasToString<T>::value) {
        return val.toString();
    } else {
        std::ostringstream oss;
        oss << val;
        return oss.str();
    }
}

// ─── 4c. Test types ──────────────────────────────────────────────────────────

// EN: Automotive diagnostic trouble code — has .toString()
// TR: Otomotiv arıza kodu — .toString() metodu var
struct DiagnosticCode {
    int code;
    std::string module;

    std::string toString() const {
        return "DTC-" + module + "-" + std::to_string(code);
    }
};
```

```cpp

// EN: Plain voltage reading — no .toString(), relies on operator<<
// TR: Düz voltaj okuması — .toString() yok, operator<< kullanır
struct VoltageReading {
    double volts;

    friend std::ostream& operator<<(std::ostream& os, const VoltageReading& vr) {
        return os << vr.volts << "V";
    }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: Compile-Time Error Prevention Demo Types
// ═══════════════════════════════════════════════════════════════════════════════

// EN: A constrained function that only accepts pointer types (e.g., memory-mapped I/O)
// TR: Yalnızca işaretçi türlerini kabul eden kısıtlı fonksiyon (örn. bellek eşlemeli G/Ç)
template <typename T, std::enable_if_t<std::is_pointer_v<T>, int> = 0>
void writeMemoryMappedIO(T addr, uint8_t value) {
    // EN: In production, this would write to a hardware register
    // TR: Üretimde bu, bir donanım kaydına yazardı
    std::cout << "  [MMIO] Writing 0x" << std::hex << static_cast<int>(value)
              << std::dec << " to address " << static_cast<const void*>(addr)
              << std::endl;
}

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n";
    std::cout << "║  MODULE 09: SFINAE, Tag Dispatch & if constexpr           ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // ─── Demo 1: SFINAE enable_if — integral vs floating-point paths ──────────
    std::cout << "── Demo 1: SFINAE enable_if (ECU Data Processing) ──\n";
    // EN: Integral type → hex register path
    // TR: Tamsayı türü → hex register yolu
    processECUData(0x1A3F);
    // EN: Floating-point type → calibrated sensor path
    // TR: Ondalıklı tür → kalibre sensör yolu
    processECUData(3.28);
    processECUData(255);
    processECUData(1.5f);
    std::cout << std::endl;

    // ─── Demo 2: Tag dispatch — raw vs calibrated sensor data ─────────────────
    std::cout << "── Demo 2: Tag Dispatch (Sensor Pipeline) ──\n";
    // EN: Raw ADC reading needs offset/gain correction
    // TR: Ham ADC okuması ofset/kazanç düzeltmesi gerektirir
    dispatchSensor(1023, RawSensorTag{});
    dispatchSensor(2.48, RawSensorTag{});
    // EN: Calibrated value already in engineering units
    // TR: Kalibre değer zaten mühendislik birimlerinde
    dispatchSensor(98.6, CalibratedSensorTag{});
    dispatchSensor(14.7, CalibratedSensorTag{});
    std::cout << std::endl;

    // ─── Demo 3: if constexpr — modern compile-time branching ─────────────────
    std::cout << "── Demo 3: if constexpr (Modern Replacement) ──\n";
    // EN: Same function, different compile-time branches based on type
    // TR: Aynı fonksiyon, türe göre farklı derleme zamanı dalları
    processModern(0xFF);
    processModern(4.75);
    processModern("CAN_BUS_MSG");
    std::cout << std::endl;

    // ─── Demo 4: Has-member detection — .toString() check ─────────────────────
    std::cout << "── Demo 4: Has-Member Detection (.toString()) ──\n";
    DiagnosticCode dtc{4201, "ECM"};
    VoltageReading vr{12.6};
```

```

    // EN: DiagnosticCode has .toString() → uses it
    // TR: DiagnosticCode .toString() metoduna sahip → onu kullanır
    std::cout << "  DiagnosticCode serialized: " << serialize(dtc) << std::endl;
    // EN: VoltageReading has no .toString() → uses operator<<
    // TR: VoltageReading .toString() yok → operator<< kullanır
    std::cout << "  VoltageReading serialized: " << serialize(vr) << std::endl;
    // EN: Plain int has no .toString() → uses operator<<
    // TR: Düz int .toString() yok → operator<< kullanır
    std::cout << "  int serialized:            " << serialize(42) << std::endl;

    // EN: Compile-time trait verification
    // TR: Derleme zamanı özellik doğrulaması
    static_assert(HasToString<DiagnosticCode>::value,
                  "DiagnosticCode must have .toString()");
    static_assert(!HasToString<VoltageReading>::value,
                  "VoltageReading must NOT have .toString()");
    static_assert(!HasToString<int>::value,
                  "int must NOT have .toString()");
    std::cout << std::endl;

    // ── Demo 5: Compile-time error prevention ──────────────────────────────────
    std::cout << "── Demo 5: Compile-Time Error Prevention (MMIO) ──\n";
    uint8_t fake_register = 0;
    // EN: Pointer type → SFINAE allows the call
    // TR: İşaretçi türü → SFINAE çağrıya izin verir
    writeMemoryMappedIO(&fake_register, 0xAB);

    // EN: Uncommenting the line below would cause a compile error — SFINAE removes the overload
    // because int is not a pointer type.
    // TR: Aşağıdaki satırın yorumunu kaldırmak derleme hatasına neden olur — SFINAE, int bir
    // işaretçi türü olmadığı için aşırı yüklemeyi kaldırır. writeMemoryMappedIO(0x4000'0000,
    // 0xAB);  // ERROR: int is not a pointer!

    std::cout << "  (Passing a non-pointer to writeMemoryMappedIO is a compile error)\n";
    std::cout << std::endl;

    // ── Summary ────────────────────────────────────────────────────────────────
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n";
    std::cout << "│  SUMMARY / ÖZET                                           │\n";
    std::cout << "╠═══════════════════════════════════════════════════════════╣\n";
    std::cout << "│  1. SFINAE + enable_if: powerful but verbose              │\n";
    std::cout << "│  2. Tag dispatch: clean compile-time overload selection    │\n";
    std::cout << "│  3. if constexpr (C++17): simplest for inline branching   │\n";
    std::cout << "│  4. Has-member detection: SFINAE metadata introspection   │\n";
    std::cout << "│  5. For C++20 projects, prefer Concepts over SFINAE       │\n";
    std::cout << "╚═══════════════════════════════════════════════════════════╝\n";

    return 0;
}
```

► 07. Crtp Pattern

module_09_advanced_mechanics/07_crtp_pattern.cpp

Konu: CRTP — Curiously Recurring Template Pattern for Static Polymorphism

Teori / Theory

THEORY

1. CRTP — Curiously Recurring Template Pattern

CRTP is a C++ idiom where a class Derived inherits from a base class template parameterized by Derived itself: class Derived : public Base<Derived> The base class can then static_cast<Derived*>(this) to call methods on the derived class — all resolved at compile time with no virtual dispatch.

TEORİ

CRTP, bir Derived sınıfının kendisi ile parametrize edilmiş bir temel sınıf şablonundan türetildiği C++ deyimidir: class Derived : public Base<Derived> Temel sınıf, türetilmiş sınıftaki metotları çağırmak için static_cast<Derived*>(this) kullanabilir — hepsi derleme zamanında çözülür.

2. Static Polymorphism vs Virtual Dispatch

Virtual functions require a vtable pointer per object and an indirect call through the vtable at runtime. CRTP resolves the call at compile time: - No vtable pointer (8 bytes saved per object on 64-bit) - No indirect branch (better branch prediction, inlineable by compiler)
- No heap allocation required for polymorphic behavior Trade-off: CRTP types are not runtime-polymorphic (no base-class pointer).

Sanal fonksiyonlar nesne başına bir vtable işaretçisi ve çalışma zamanında vtable üzerinden dolaylı çağrı gerektirir. CRTP çağrıyı derleme zamanında çözer: - vtable işaretçisi yok (64-bit'te nesne başına 8 bayt tasarruf) - Dolaylı dal yok (daha iyi dal tahmini, derleyici tarafından inline edilebilir) - Polimorfik davranış için yığın tahsisi gerekmez Takas: CRTP türleri çalışma zamanında polimorfik değildir (temel sınıf pointer'ı yok).

3. Use Cases: Mixins, Static Interfaces, Expression Templates

- Mixins: inject reusable functionality (logging, counting, serialization) into unrelated classes without virtual overhead. - Static interfaces: enforce that derived classes implement certain methods, checked at compile time via static_cast. - Expression templates: build lazy evaluation trees (e.g., Eigen library).

- Mixin'ler: sanal maliyet olmadan ilgisiz sınıflara yeniden kullanılabilir işlevsellik (loglama, sayma, serileştirme) enjekte eder. - Statik arayüzler: türetilmiş sınıfların belirli metotları uygulamasını derleme zamanında zorlar. - İfade şablonları: tembel değerlendirme ağaçları oluşturur.

4. Common Pitfall: Calling Derived Methods from Base Constructor

When Base<Derived>'s constructor runs, the Derived object is NOT fully constructed yet. Calling static_cast<Derived*>(this)->method() from the base constructor is undefined behavior! Always call derived methods from regular member functions, never from constructors or destructors.

Base<Derived> yapıcısı çalıştığında, Derived nesnesi henüz tam olarak oluşturulmamıştır. Temel yapıcıdan static_cast<Derived*>(this)->method() çağırmak tanımsız davranıştır! Türetilmiş metotları her zaman normal üye fonksiyonlarından çağırın, asla yapıcı veya yıkıcılardan çağırmayın.

CPPREF Depth: CRTP Pitfalls and the Deducing-This Alternative (C++23)

Pitfall #1: calling `static_cast<Derived*>(this)` in the base constructor — Derived is not yet constructed, invoking its methods is UB. Pitfall #2: forgetting to pass the correct Derived type (copy-paste error such as inheriting from `Base<WrongClass>` instead of `Base<Derived>`). Pitfall #3: CRTP does not support runtime polymorphism — there is no common base-class pointer for heterogeneous collections. C++23 deducing-this (`this auto&& self`) eliminates CRTP for most use cases: the base class method deduces the exact derived type from the implicit object parameter — cleaner syntax, same zero-cost dispatch, and no inheritance required.

CPPREF Derinlik: CRTP Tuzakları ve Deducing-This Alternatifi (C++23)

Tuzak #1: temel yapıcıda `static_cast<Derived*>(this)` çağırmak — Derived henüz oluşturulmamıştır, metotlarını çağırmak UB'dir. Tuzak #2: doğru Derived türünü geçirmeyi unutmak (kopyala-yapıştır hatası, örneğin `Base<Derived>` yerine `Base<YanlışSınıf>`'tan türetme). Tuzak #3: CRTP çalışma zamanı polimorfizmini desteklemez — heterojen koleksiyonlar için ortak bir temel sınıf işaretçisi yoktur. C++23 deducing-this (`this auto&& self`) çoğu kullanım için CRTP'yi ortadan kaldırır: temel sınıf metodu, örtük nesne parametresinden tam türetilmiş türü çıkarır — daha temiz sözdizimi, aynı sıfır maliyetli dağıtım, kalıtım gerekmez.

Kaynak Kod / Source Code

```cpp
#include <chrono>    // EN: High-resolution clock for benchmarking
#include <cstdint>   // EN: Fixed-width integer types
#include <iostream>   // EN: Standard I/O for console output
#include <string>     // EN: std::string for log messages

// ═══════════════════════════════════════════════════════════════════════════
// SECTION 1: Static Polymorphism — SensorBase with CRTP
// ═══════════════════════════════════════════════════════════════════════════

// EN: CRTP base — defines the common algorithm, delegates specifics to Derived
// TR: CRTP tabanı — ortak algoritmayı tanımlar, özellikleri Derived'a devreder
template <typename Derived>
class SensorBase {
public:
    // EN: Template method: read → process → report (algorithm skeleton)
    // TR: Şablon metodu: oku → işle → raporla (algoritma iskeleti)
    void readAndProcess() {
        // EN: static_cast to Derived — resolved at compile time, zero overhead
        // TR: Derived'a static_cast — derleme zamanında çözülür, sıfır maliyet
        auto& self = *static_cast<Derived*>(this);
        double raw = self.doRead();
        std::cout << "  [" << self.name() << "] Raw: " << raw
                  << " → Processed: " << raw * self.scaleFactor() << std::endl;
    }
};
```

```cpp

// ─── 1a. TemperatureSensor (NTC thermistor on coolant line) ──────────────────────

// EN: Reads coolant temperature via ADC, applies NTC conversion factor
// TR: ADC üzerinden soğutma suyu sıcaklığını okur, NTC dönüşüm katsayısı uygular
class TemperatureSensor : public SensorBase<TemperatureSensor> {
public:
    double doRead() const { return 2048.0; }
    double scaleFactor() const { return 0.0489; }
    const char* name() const { return "CoolantTemp"; }
};

// ─── 1b. PressureSensor (MAP sensor on intake manifold) ──────────────────────────

// EN: Reads manifold absolute pressure, applies bar conversion
// TR: Manifold mutlak basıncını okur, bar dönüşümü uygular
class PressureSensor : public SensorBase<PressureSensor> {
public:
    double doRead() const { return 3200.0; }
    double scaleFactor() const { return 0.000305; }
    const char* name() const { return "MAP_Sensor"; }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 2: CRTP Mixin — Loggable (adds logging to any class)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Mixin that injects logging capability into any Derived class via CRTP
// TR: CRTP aracılığıyla herhangi bir Derived sınıfına loglama yeteneği enjekte eden mixin
template <typename Derived>
class Loggable {
public:
    void log(const std::string& message) const {
        // EN: Access derived class's identifier without virtual dispatch
        // TR: Sanal dağıtım olmadan türetilmiş sınıfın tanımlayıcısına erişim
        const auto& self = *static_cast<const Derived*>(this);
        std::cout << "  [LOG:" << self.moduleName() << "] " << message << std::endl;
    }
};

// EN: ECU module that gets logging functionality for free via CRTP mixin
// TR: CRTP mixin aracılığıyla ücretsiz loglama işlevselliği alan ECU modülü
class ECUModule : public Loggable<ECUModule> {
    std::string name_;
public:
    explicit ECUModule(std::string name) : name_(std::move(name)) {}
    const std::string& moduleName() const { return name_; }

    void initialize() {
        log("Initialization sequence started");
        log("Self-test PASSED, module ready");
    }

    void shutdown() {
        log("Graceful shutdown initiated");
    }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 3: CRTP Mixin — InstanceCounter (per-type instance tracking)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Each Derived type gets its OWN static counter (separate template instantiation)
// TR: Her Derived türü KENDİ statik sayacını alır (ayrı şablon somutlaştırması)
template <typename T>
class InstanceCounter {
    static inline int count_ = 0;  // EN: C++17 inline static
public:
    InstanceCounter()  { ++count_; }
    InstanceCounter(const InstanceCounter&) { ++count_; }
    InstanceCounter(InstanceCounter&&) noexcept { ++count_; }
    ~InstanceCounter() { --count_; }

    static int instanceCount() { return count_; }
};

// EN: Injector hardware driver — tracks how many instances are alive
// TR: Enjektör donanım sürücüsü — kaç örneğin canlı olduğunu izler
class InjectorDriver : public InstanceCounter<InjectorDriver> {
    int cylinder_;
public:
    explicit InjectorDriver(int cyl) : cylinder_(cyl) {}
    int cylinder() const { return cylinder_; }
};

// EN: Ignition coil driver — separate counter from InjectorDriver
// TR: Ateşleme bobini sürücüsü — InjectorDriver'dan ayrı sayaç
class IgnitionCoilDriver : public InstanceCounter<IgnitionCoilDriver> {
    int cylinder_;
public:
    explicit IgnitionCoilDriver(int cyl) : cylinder_(cyl) {}
    int cylinder() const { return cylinder_; }
};

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 4: Virtual Dispatch Baseline (for benchmark comparison)
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Abstract sensor using traditional virtual dispatch
// TR: Geleneksel sanal dağıtım kullanan soyut sensör
class VirtualSensorBase {
public:
    virtual ~VirtualSensorBase() = default;
    virtual double doRead() const = 0;
    virtual double scaleFactor() const = 0;

    double readAndProcess() const {
        return doRead() * scaleFactor();
    }
};

class VirtualTemperatureSensor : public VirtualSensorBase {
public:
    double doRead() const override { return 2048.0; }
    double scaleFactor() const override { return 0.0489; }
};
```

```cpp

// ═══════════════════════════════════════════════════════════════════════════════
// SECTION 5: Template Method Pattern via CRTP
// ═══════════════════════════════════════════════════════════════════════════════

// EN: Diagnostic routine with fixed steps, customizable per ECU subsystem
// TR: Sabit adımları olan, ECU alt sistemi başına özelleştirilebilir teşhis rutini
template <typename Derived>
class DiagnosticRoutine {
public:
    void runFullDiagnostic() {
        auto& self = *static_cast<Derived*>(this);
        std::cout << "  [DIAG] Starting diagnostic for: " << self.subsystemName() << std::endl;
        std::cout << "  [DIAG] Step 1 — Pre-check:  " << (self.preCheck()  ? "OK" : "FAIL") <<
            std::endl;
        std::cout << "  [DIAG] Step 2 — Core test:   " << (self.coreTest()  ? "OK" : "FAIL") <<
            std::endl;
        std::cout << "  [DIAG] Step 3 — Post-check:  " << (self.postCheck() ? "OK" : "FAIL") <<
            std::endl;
        std::cout << "  [DIAG] Diagnostic complete." << std::endl;
    }
};

// EN: Fuel system diagnostic — implements the three customizable steps
// TR: Yakıt sistemi teşhisi — üç özelleştirilebilir adımı uygular
class FuelSystemDiag : public DiagnosticRoutine<FuelSystemDiag> {
public:
    const char* subsystemName() const { return "FuelSystem"; }
    bool preCheck()  const { return true;  }
    bool coreTest()  const { return true;  }
    bool postCheck() const { return true;  }
};

// EN: Brake system diagnostic
// TR: Fren sistemi teşhisi
class BrakeSystemDiag : public DiagnosticRoutine<BrakeSystemDiag> {
public:
    const char* subsystemName() const { return "BrakeSystem"; }
    bool preCheck()  const { return true;  }
    bool coreTest()  const { return false; }
    bool postCheck() const { return true;  }
};

// ═══════════════════════════════════════════════════════════════════════════════
// MAIN — Demonstrations
// ═══════════════════════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════════════════╗\n";
    std::cout << "║  MODULE 09: CRTP — Curiously Recurring Template Pattern    ║\n";
    std::cout << "╚═══════════════════════════════════════════════════════════╝\n\n";

    // ── Demo 1: Static polymorphism via SensorBase CRTP ──────────────────────
    std::cout << "── Demo 1: Static Polymorphism (SensorBase CRTP) ──\n";
    TemperatureSensor tempSensor;
    PressureSensor    mapSensor;
    // EN: Same interface, different implementations — no vtable involved
    // TR: Aynı arayüz, farklı uygulamalar — vtable yok
    tempSensor.readAndProcess();
    mapSensor.readAndProcess();
    std::cout << std::endl;

    // ── Demo 2: Loggable mixin — ECUModule gets logging for free ─────────────
    std::cout << "── Demo 2: Loggable Mixin (ECU Module) ──\n";
    ECUModule engineControl("EngineCtrl");
    ECUModule absModule("ABS_Unit");
```

```cpp
    // EN: Logging injected via CRTP, no virtual dispatch overhead
    // TR: CRTP aracılığıyla enjekte edilen loglama, sanal dağıtım yükü yok
    engineControl.initialize();
    absModule.initialize();
    engineControl.shutdown();
    std::cout << std::endl;

    // ─── Demo 3: InstanceCounter — separate counts per type ──────────────────────────
    std::cout << "── Demo 3: InstanceCounter (Per-Type Tracking) ──\n";
    InjectorDriver inj1(1), inj2(2), inj3(3), inj4(4);
    IgnitionCoilDriver coil1(1), coil2(2);
    std::cout << "  InjectorDriver instances:     "
              << InjectorDriver::instanceCount() << std::endl;
    std::cout << "  IgnitionCoilDriver instances:  "
              << IgnitionCoilDriver::instanceCount() << std::endl;
    {
        // EN: Create temporary instances inside a scope
        // TR: Bir kapsam içinde geçici örnekler oluştur
        InjectorDriver temp5(5);
        IgnitionCoilDriver temp3(3);
        std::cout << "  [inner scope] Injectors: "
                  << InjectorDriver::instanceCount()
                  << ", Coils: " << IgnitionCoilDriver::instanceCount() << std::endl;
    }
    // EN: Temporaries destroyed — count decremented
    // TR: Geçiciler yok edildi — sayaç azaltıldı
    std::cout << "  [after scope] Injectors: "
              << InjectorDriver::instanceCount()
              << ", Coils: " << IgnitionCoilDriver::instanceCount() << std::endl;
    std::cout << std::endl;

    // ─── Demo 4: Performance comparison — CRTP vs virtual dispatch ────────────────────
    std::cout << "── Demo 4: CRTP vs Virtual Dispatch Benchmark ──\n";
    constexpr int iterations = 10'000'000;
    volatile double sink = 0.0;

    // EN: CRTP benchmark (compile-time dispatch, inlineable)
    // TR: CRTP karşılaştırması (derleme zamanı dağıtımı, satır içi yapılabilir)
    {
        TemperatureSensor crtp_sensor;
        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            auto& base = static_cast<SensorBase<TemperatureSensor>&>(crtp_sensor);
            // EN: Inline-able static dispatch
            // TR: Satır içi yapılabilen statik dağıtım
            sink = crtp_sensor.doRead() * crtp_sensor.scaleFactor();
            (void)base;
        }
        auto end = std::chrono::high_resolution_clock::now();
        auto crtp_us = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();
        std::cout << "  CRTP dispatch:    " << crtp_us << " µs (" << iterations << " iterations)"
            << std::endl;
    }

    // EN: Virtual dispatch benchmark (runtime indirect call)
    // TR: Sanal dağıtım karşılaştırması (çalışma zamanı dolaylı çağrı)
    {
        VirtualTemperatureSensor virt_sensor;
        VirtualSensorBase* base_ptr = &virt_sensor;
        auto start = std::chrono::high_resolution_clock::now();
        for (int i = 0; i < iterations; ++i) {
            // EN: Indirect call through vtable
            // TR: Vtable üzerinden dolaylı çağrı
            sink = base_ptr->readAndProcess();
        }
        auto end = std::chrono::high_resolution_clock::now();
        auto virt_us = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count();
        std::cout << "  Virtual dispatch: " << virt_us << " µs (" << iterations << " iterations)"
            << std::endl;
    }
    (void)sink;
    std::cout << std::endl;
```

```

    // ─── Demo 5: Template method pattern via CRTP ──────────────────────────────────────
    std::cout << "── Demo 5: Template Method Pattern via CRTP ──\n";
    FuelSystemDiag  fuelDiag;
    BrakeSystemDiag brakeDiag;
    // EN: Same runFullDiagnostic() algorithm, different per-subsystem steps
    // TR: Aynı runFullDiagnostic() algoritması, alt sisteme göre farklı adımlar
    fuelDiag.runFullDiagnostic();
    std::cout << std::endl;
    brakeDiag.runFullDiagnostic();
    std::cout << std::endl;

    // ─── Summary ───────────────────────────────────────────────────────────────────────
    std::cout << "╔═══════════════════════════════════════════════════════════════╗\n";
    std::cout << "│  SUMMARY / ÖZET                                               │\n";
    std::cout << "╟───────────────────────────────────────────────────────────────╢\n";
    std::cout << "│  1. CRTP: static polymorphism — zero vtable overhead          │\n";
    std::cout << "│  2. Mixin: inject reusable behavior via CRTP inheritance      │\n";
    std::cout << "│  3. InstanceCounter: per-type tracking with CRTP + static     │\n";
    std::cout << "│  4. Benchmark: CRTP enables inlining, virtual does not        │\n";
    std::cout << "│  5. Template method: CRTP enforces interface at compile       │\n";
    std::cout << "╚═══════════════════════════════════════════════════════════════╝\n";

    return 0;
}
```

Modül 10: Design Patterns — Tasarım Kalıpları

Tasarım Kalıpları

6 kaynak dosya · Standart: C++20

C++20

► 01. Solid Principles

module_10_design_patterns/01_solid_principles.cpp

Konu: Design Patterns & Architecture: SOLID Principles / Tasarım Kalıpları ve Mimari: SOLID

Teori / Theory

THEORY: What is SOLID?

SOLID is a set of five design principles that make software easier to maintain, extend, and test. Coined by Robert C. Martin ("Uncle Bob"), these principles are the FOUNDATION of professional OOP design. Every senior C++ developer is expected to know and apply them daily.

TEORİ: SOLID Nedir?

SOLID, yazılımı bakımı kolay, genişletilebilir ve test edilebilir kılan beş tasarım prensibidir. Robert C. Martin ("Bob Amca") tarafından formüle edilen bu prensipler profesyonel OOP tasarımının TEMELİDİR. Her kıdemli C++ geliştiricisinin bunları bilmesi ve günlük uygulaması beklenir.

THEORY: S — Single Responsibility Principle (SRP)

A class should have ONE and ONLY ONE reason to change. If a class handles file I/O, business logic, AND formatting — it has THREE reasons to change. Split it into three focused classes. The key question: "What is this class's single job?"

TEORİ: S — Tek Sorumluluk Prensibi

Bir sınıfın değişmek için TEK ve YALNIZCA TEK bir nedeni olmalıdır. Eğer bir sınıf dosya G/Ç, iş mantığı VE biçimlendirme yapıyorsa — üç değişim nedeni vardır. Üç odaklı sınıfa bölün. Anahtar soru: "Bu sınıfın tek görevi nedir?"

THEORY: O — Open/Closed Principle (OCP)

Software entities should be OPEN for extension but CLOSED for modification. Add new behavior by creating NEW classes (inheritance or composition), NOT by editing existing working code. This prevents introducing bugs into stable code while still allowing growth.

TEORİ: O — Açık/Kapalı Prensibi

Yazılım varlıkları genişlemeye AÇIK, değişikliğe KAPALI olmalıdır. Yeni davranış eklemek için mevcut çalışan kodu DEĞİŞTİRMEK yerine YENİ sınıflar (kalıtım veya bileşim) oluşturun. Bu, kararlı kodda hata tanıtmayı önlerken büyümeye izin verir.

THEORY: L — Liskov Substitution Principle (LSP)

Objects of a derived class MUST be substitutable for objects of its base class WITHOUT altering the correctness of the program. If `Square` inherits from `Rectangle` but breaks `setWidth()/setHeight()` behavior, it violates LSP. The derived class must honor the BASE class's contract.

TEORİ: L — Liskov Yerine Geçme Prensibi

Türetilmiş sınıf nesneleri, programın doğruluğunu bozmadan temel sınıf nesnelerinin yerine GEÇEBİLMELİDİR. Eğer `Kare` sınıfı `Dikdörtgen`'den miras alır ama `setWidth()/setHeight()` davranışını bozarsa, LSP'yi ihlal eder. Türetilmiş sınıf, TEMEL sınıfın sözleşmesine uymalıdır.

THEORY: I — Interface Segregation Principle (ISP)

No client should be FORCED to depend on methods it does not use. Instead of one "God Interface" with 20 methods, split into small focused interfaces. A `Printer` shouldn't be forced to implement `scan()` and `fax()` if it only prints. Each interface represents ONE capability.

TEORİ: I — Arayüz Ayrımı Prensibi

Hiçbir istemci, kullanmadığı yöntemlere bağımlı olmaya ZORLANMAMALIDIR. 20 metodlu tek bir "Tanrı Arayüzü" yerine küçük odaklı arayüzlere bölün. Bir `Yazıcı`, yalnızca yazdırıyorsa `tarama()` ve `faks()` uygulamaya zorlanmamalıdır. Her arayüz TEK bir yeteneği temsil eder.

THEORY: D — Dependency Inversion Principle (DIP)

High-level modules should NOT depend on low-level modules. Both should depend on ABSTRACTIONS (interfaces). A `GameEngine` shouldn't know about `OpenGLRenderer` directly — it should depend on an `IRenderer` interface. This makes swapping implementations trivial and enables testing with mocks.

TEORİ: D — Bağımlılık Tersine Çevirme Prensibi

Üst seviye modüller, alt seviye modüllere bağımlı OLMAMALIDIR. İkisi de SOYUTLAMALARA (arayüzler) bağımlı olmalıdır. Bir `OyunMotoru` doğrudan `OpenGLRenderer`'ı bilmemelidir — bir `IRenderer` arayüzüne bağımlı olmalıdır. Bu, uygulama değiştirmeyi kolay ve mock ile test etmeyi mümkün kılar.

CPPREF Depth: Pure Virtual Functions and Abstract Classes

SOLID heavily relies on abstract classes (interfaces). In C++, a class with at least one `= 0` pure virtual function is abstract and CANNOT be instantiated. The derived class MUST override all pure virtuals or it also becomes abstract. A virtual destructor is essential in base classes to avoid undefined behavior when deleting through base pointers. cppreference.com/w/cpp/language/abstract_class

CPPREF Derinlik: Saf Sanal Fonksiyonlar ve Soyut Sınıflar

SOLID, soyut sınıflara (arayüzler) yoğun şekilde dayanır. C++'ta en az bir `= 0` saf sanal fonksiyonu olan sınıf soyuttur ve ÖRNEKLENEMez. Türetilmiş sınıf tüm saf sanal fonksiyonları geçersiz kılMALIDIR aksi takdirde kendisi de soyut olur. Temel sınıflarda, temel işaretçi aracılığıyla silerken tanımsız davranıştan kaçınmak için sanal yıkıcı şarttır. cppreference.com/w/cpp/language/abstract_class

CPPREF Depth: std::unique_ptr and Ownership Transfer

DIP implementations typically use `std::unique_ptr<IInterface>` to hold injected dependencies. `unique_ptr` enforces single ownership — the object is destroyed when the pointer goes out of scope. Transfer ownership with `std::move()`. For shared ownership, use `std::shared_ptr`. Never use raw `new`/`delete` in modern C++ — smart pointers are the RAII standard. cppreference.com/w/cpp/memory/unique_ptr

CPPREF Derinlik: std::unique_ptr ve Sahiplik Devri

DIP uygulamaları tipik olarak enjekte edilen bağımlılıkları tutmak için `std::unique_ptr<IArayüz>` kullanır. `unique_ptr` tek sahipliği zorlar — nesne, işaretçi kapsam dışına çıktığında yok edilir. Sahipliği `std::move()` ile devredin. Paylaşımlı sahiplik için `std::shared_ptr` kullanın. Modern C++'ta asla ham `new`/`delete` kullanmayın — akıllı işaretçiler RAII standardıdır. cppreference.com/w/cpp/memory/unique_ptr

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ==========================================================================================
// [SRP DEMO: Single Responsibility Principle]
// ==========================================================================================
// EN: BAD: One class doing everything. GOOD: Split into focused classes.
// TR: KÖTÜ: Her şeyi yapan tek sınıf. İYİ: Odaklı sınıflara böl.

// --- BAD DESIGN (SRP violation) ---
class BadReport {
public:
  std::string generateContent() { return "Q4 Sales: $1,200,000"; }
  void saveToFile(const std::string &content) {
    // EN: Mixes business logic with I/O!
    (void)content;
    std::cout << "[BAD] Saving to file... (mixed responsibility)" << std::endl;
  }
};

// --- GOOD DESIGN (SRP applied) ---
class ReportGenerator {
public:
  std::string generate() { return "Q4 Sales: $1,200,000"; }
};

class ReportSaver {
public:
  void save(const std::string &content) {
    (void)content;
    std::cout << "[GOOD-SRP] Report saved by dedicated saver." << std::endl;
  }
};
```

```cpp

// ==========================================================================================
// [OCP DEMO: Open/Closed Principle]
// ==========================================================================================
// EN: Add new shapes WITHOUT modifying existing code.
// TR: Mevcut kodu DEĞİŞTİRMEDEN yeni şekiller ekle.

class Shape {
public:
  virtual ~Shape() = default;
  virtual double area() const = 0;
  virtual std::string name() const = 0;
};

class Circle : public Shape {
  double radius_;

public:
  explicit Circle(double r) : radius_(r) {}
  double area() const override { return 3.14159 * radius_ * radius_; }
  std::string name() const override { return "Circle"; }
};

class Rectangle : public Shape {
  double w_, h_;

public:
  Rectangle(double w, double h) : w_(w), h_(h) {}
  double area() const override { return w_ * h_; }
  std::string name() const override { return "Rectangle"; }
};

// EN: NEW shape added without touching Circle or Rectangle!
// TR: Circle veya Rectangle'a dokunmadan YENİ şekil eklendi!
class Triangle : public Shape {
  double base_, height_;

public:
  Triangle(double b, double h) : base_(b), height_(h) {}
  double area() const override { return 0.5 * base_ * height_; }
  std::string name() const override { return "Triangle"; }
};

// EN: This function works with ANY shape — open for extension.
// TR: Bu fonksiyon HERHANGİ bir şekille çalışır — genişlemeye açık.
void printArea(const Shape &s) {
  std::cout << "[OCP] " << s.name() << " area = " << s.area() << std::endl;
}

// ==========================================================================================
// [ISP DEMO: Interface Segregation Principle]
// ==========================================================================================
// EN: Split "God interface" into small focused interfaces.
// TR: "Tanrı arayüzü"nü küçük odaklı arayüzlere böl.

class IPrintable {
public:
  virtual ~IPrintable() = default;
  virtual void print() = 0;
};

class IScannable {
public:
  virtual ~IScannable() = default;
  virtual void scan() = 0;
};
```

```

// EN: SimplePrinter only implements IPrintable — not forced to implement scan!
// TR: SimplePrinter yalnızca IPrintable uygular — scan uygulamaya zorlanmaz!
class SimplePrinter : public IPrintable {
public:
  void print() override {
    std::cout << "[ISP] SimplePrinter printing..." << std::endl;
  }
};

class MultiFunctionDevice : public IPrintable, public IScannable {
public:
  void print() override {
    std::cout << "[ISP] MultiFunctionDevice printing..." << std::endl;
  }
  void scan() override {
    std::cout << "[ISP] MultiFunctionDevice scanning..." << std::endl;
  }
};

// ==========================================================================================
// [DIP DEMO: Dependency Inversion Principle]
// ==========================================================================================
// EN: High-level DiagnosticsEngine depends on ICommunicationBus interface, not a concrete
// protocol class.
// TR: Üst seviye DiagnosticsEngine, somut sınıfa değil ICommunicationBus arayüzüne bağımlıdır.

class ICommunicationBus {
public:
  virtual ~ICommunicationBus() = default;
  virtual void sendFrame(const std::string &frameData) = 0;
};

class CANBusProtocol : public ICommunicationBus {
public:
  void sendFrame(const std::string &frameData) override {
    std::cout << "[DIP] CAN Bus transmitting: " << frameData << std::endl;
  }
};

class FlexRayProtocol : public ICommunicationBus {
public:
  void sendFrame(const std::string &frameData) override {
    std::cout << "[DIP] FlexRay transmitting: " << frameData << std::endl;
  }
};

// EN: DiagnosticsEngine doesn't know or care which bus protocol it uses!
// TR: DiagnosticsEngine hangi veri yolu protokolünü kullandığını bilMEZ!
class DiagnosticsEngine {
  std::unique_ptr<ICommunicationBus> bus_;

public:
  explicit DiagnosticsEngine(std::unique_ptr<ICommunicationBus> b)
      : bus_(std::move(b)) {}
  void transmitDTC(const std::string &dtcCode) { bus_->sendFrame(dtcCode); }
};

int main() {
  std::cout << "=== MODULE 10: SOLID PRINCIPLES ===\n" << std::endl;

  // --- SRP Demo ---
  std::cout << "--- SRP: Single Responsibility ---\n" << std::endl;
  ReportGenerator gen;
  ReportSaver saver;
  std::string content = gen.generate();
  saver.save(content);
  std::cout << "Report content: " << content << "\n" << std::endl;
```

```
170
171     // --- OCP Demo ---
172     std::cout << "--- OCP: Open/Closed ---\n" << std::endl;
173     Circle c(5.0);
174     Rectangle r(4.0, 6.0);
175     Triangle t(3.0, 8.0);
176     printArea(c);
177     printArea(r);
178     printArea(t);
179     std::cout << std::endl;
180
181     // --- LSP Demo (via OCP shapes — all substitutable) ---
182     std::cout << "--- LSP: Liskov Substitution ---\n" << std::endl;
183     std::vector<std::unique_ptr<Shape>> shapes;
184     shapes.push_back(std::make_unique<Circle>(3.0));
185     shapes.push_back(std::make_unique<Rectangle>(2.0, 5.0));
186     shapes.push_back(std::make_unique<Triangle>(4.0, 6.0));
187     for (const auto &s : shapes) {
188       std::cout << "[LSP] " << s->name() << " -> " << s->area() << std::endl;
189     }
190     std::cout << std::endl;
191
192     // --- ISP Demo ---
193     std::cout << "--- ISP: Interface Segregation ---\n" << std::endl;
194     SimplePrinter sp;
195     MultiFunctionDevice mfd;
196     sp.print();
197     mfd.print();
198     mfd.scan();
199     std::cout << std::endl;
200
201     // --- DIP Demo ---
202     std::cout << "--- DIP: Dependency Inversion ---\n" << std::endl;
203     DiagnosticsEngine engine1(std::make_unique<CANBusProtocol>());
204     DiagnosticsEngine engine2(std::make_unique<FlexRayProtocol>());
205     engine1.transmitDTC("P0300 - Random Misfire Detected");
206     engine2.transmitDTC("C0035 - Left Front Wheel Speed Sensor");
207
208     std::cout << "\n=> All 5 SOLID principles demonstrated!" << std::endl;
209
210     return 0;
211 }
```

► 02. Creational Patterns

module_10_design_patterns/02_creational_patterns.cpp

Konu: Design Patterns: Creational — Singleton, Factory, Builder, Abstract Factory /

Teori / Theory

THEORY: Creational Patterns Overview

Creational Design Patterns control HOW objects are created. Instead of scattering `new` calls throughout your codebase, these patterns centralize and encapsulate object creation. This makes code more flexible, testable, and decoupled from concrete types.

TEORİ: Oluşturucu Kalıplar

Oluşturucu Tasarım Kalıpları nesnelerin NASIL oluşturulduğunu kontrol eder. Kod tabanı boyunca `new` çağrılarını dağıtmak yerine, bu kalıplar nesne oluşturmayı merkezileştirir ve kapsüller. Bu, kodu daha esnek, test edilebilir ve somut tiplerden bağımsız kılar.

THEORY: Singleton — One Instance, Global Access

The Singleton pattern ensures a class has EXACTLY ONE instance and provides a global point of access to it. Common use cases: logging systems, configuration managers, hardware drivers. The modern C++11 approach uses a static local variable inside a method — the compiler guarantees thread-safe initialization ("Magic Static" / Scott Meyers' Singleton).

TEORİ: Singleton — Tek Örnek, Global Erişim

Singleton kalıbı, bir sınıfın TAM OLARAK TEK bir örneğe sahip olmasını ve ona küresel erişim noktası sağlamasını garanti eder. Yaygın kullanım: loglama sistemleri, yapılandırma yöneticileri, donanım sürücüleri. Modern C++11 yaklaşımı, bir metodun içindeki statik yerel değişken kullanır — derleyici thread-güvenli başlatmayı garanti eder ("Sihirli Statik" / Scott Meyers' Singleton).

THEORY: Factory Method — Delegate Creation to Subclasses

The Factory Method defines an interface for creating objects but lets subclasses decide WHICH class to instantiate. The client code calls a factory function that returns a base class pointer — it never knows the concrete type. This follows the Open/Closed Principle: adding a new product requires only a new subclass, no modification of existing code.

TEORİ: Fabrika Metodu — Oluşturmayı Alt Sınıflara Devret

Fabrika Metodu, nesne oluşturmak için bir arayüz tanımlar ama HANGİ sınıfın örnekleneceğine alt sınıfların karar vermesine izin verir. İstemci kodu, temel sınıf işaretçisi döndüren bir fabrika fonksiyonu çağırır — somut tipi asla bilmez. Bu, Açık/Kapalı Prensibine uyar: yeni ürün eklemek yalnızca yeni bir alt sınıf gerektirir, mevcut kodda değişiklik gerekmez.

THEORY: Builder — Step-by-Step Complex Object Construction

The Builder pattern separates the CONSTRUCTION of a complex object from its REPRESENTATION. Instead of a constructor with 15 parameters, you chain `.setX().setY().setZ().build()` calls. Each method returns `*this` for fluent API chaining. The Builder validates the object before returning it.

TEORİ: Builder — Adım Adım Karmaşık Nesne İnşası

Builder kalıbı, karmaşık bir nesnenin İNŞASINI TEMSİLİNDEN ayırır. 15 parametreli yapıcı yerine `.setX().setY().setZ().build()` çağrılarını zincirlersiniz. Her metod akıcı API zincirleme için `*this` döndürür. Builder, nesneyi döndürmeden önce doğrular.

THEORY: Abstract Factory — Family of Related Objects

The Abstract Factory provides an interface for creating FAMILIES of related objects without specifying their concrete classes. Example: a GUI toolkit factory that creates buttons, checkboxes, and menus that all match the same theme (Windows vs. macOS). The client uses the factory interface and never sees concrete implementations.

TEORİ: Soyut Fabrika — İlişkili Nesneler Ailesi

Soyut Fabrika, somut sınıflarını belirtmeden İLİŞKİLİ nesneler AİLELERİ oluşturmak için bir arayüz sağlar. Örnek: hepsi aynı temaya uyan düğmeler, onay kutuları ve menüler oluşturan bir GUI araç takımı fabrikası (Windows vs. macOS). İstemci fabrika arayüzünü kullanır ve somut uygulamaları asla görmez.

CPPREF Depth: Static Local Variable Thread Safety (C++11)

Since C++11, initialization of a function-local static variable is guaranteed to be thread-safe. The compiler inserts a hidden guard variable to prevent double-initialization even if multiple threads call the function simultaneously. This is called "Magic Statics" and is the recommended way to implement Singleton in modern C++. cppreference.com/w/cpp/language/storage_duration#Static_local_variables

CPPREF Derinlik: Statik Yerel Değişken Thread Güvenliği (C++11)

C++11'den beri, fonksiyon-yerel statik değişkeninin başlatılması thread-güvenli olarak GARANTİ edilir. Derleyici, birden fazla thread fonksiyonu aynı anda çağırsa bile çift başlatmayı önlemek için gizli bir koruma değişkeni ekler. Buna "Sihirli Statikler" denir ve modern C++'ta Singleton uygulamanın önerilen yoludur. cppreference.com/w/cpp/language/storage_duration#Static_local_variables

CPPREF Depth: std::unique_ptr as Factory Return Type

Modern factories return `std::unique_ptr<Base>` instead of raw pointers. This enforces RAII: the caller automatically owns the object and it's destroyed when the unique_ptr goes out of scope. No manual `delete` needed. `std::make_unique<T>(args...)` is preferred over `new T(args...)` as it's exception-safe and clearer in intent. cppreference.com/w/cpp/memory/unique_ptr

CPPREF Derinlik: Fabrika Dönüş Tipi Olarak std::unique_ptr

Modern fabrikalar ham işaretçi yerine `std::unique_ptr<Temel>` döndürür. Bu RAII'yi zorlar: çağıran otomatik olarak nesneye sahip olur ve unique_ptr kapsam dışına çıktığında yok edilir. Manuel `delete` gerekmez. `std::make_unique<T>(args...)`, `new T(args...)` yerine tercih edilir çünkü istisna-güvenli ve niyette daha açıktır. cppreference.com/w/cpp/memory/unique_ptr

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <sstream>
#include <string>
#include <vector>

// ==========================================================================================
// [1. SINGLETON PATTERN — Thread-Safe "Magic Static"]
// ==========================================================================================

class Logger {
  Logger() { std::cout << "[Singleton] Logger created (only once)." << std::endl; }
  Logger(const Logger &) = delete;
  Logger &operator=(const Logger &) = delete;

public:
  // EN: C++11 guarantees thread-safe initialization of static locals.
  // TR: C++11, statik yerel değişkenlerin thread-güvenli başlatmasını garanti eder.
  static Logger &getInstance() {
    static Logger instance; // Created exactly ONCE ("Magic Static")
    return instance;
  }

  void log(const std::string &msg) {
    std::cout << "[LOG] " << msg << std::endl;
  }
};

// ==========================================================================================
// [2. FACTORY METHOD PATTERN]
// ==========================================================================================

// EN: Abstract ECU diagnostic module — each ECU type runs its own check.
// TR: Soyut ECU teşhis modülü — her ECU tipi kendi kontrolünü çalıştırır.
class DiagnosticModule {
public:
  virtual ~DiagnosticModule() = default;
  virtual void runDiagnostic() const = 0;
  virtual std::string moduleName() const = 0;
};

class BodyControlModule : public DiagnosticModule {
public:
  void runDiagnostic() const override {
    std::cout << "BCM: Checking door locks, windows, lighting relays..."
              << std::endl;
  }
  std::string moduleName() const override { return "BodyControlModule"; }
};

class PowertrainECU : public DiagnosticModule {
public:
  void runDiagnostic() const override {
    std::cout << "Powertrain: Verifying fuel injection, ignition timing..."
              << std::endl;
  }
  std::string moduleName() const override { return "PowertrainECU"; }
};
```

```cpp

class ClimateControlUnit : public DiagnosticModule {
public:
  void runDiagnostic() const override {
    std::cout << "HVAC: Testing compressor, blower motor, cabin temp sensor..."
              << std::endl;
  }
  std::string moduleName() const override { return "ClimateControlUnit"; }
};

// EN: Factory function — client never uses 'new' directly.
// TR: Fabrika fonksiyonu — istemci asla doğrudan 'new' kullanmaz.
std::unique_ptr<DiagnosticModule> createModule(const std::string &type) {
  if (type == "bcm") return std::make_unique<BodyControlModule>();
  if (type == "powertrain") return std::make_unique<PowertrainECU>();
  if (type == "hvac") return std::make_unique<ClimateControlUnit>();
  return nullptr;
}

// ==========================================================================================
// [3. BUILDER PATTERN — Fluent API for Complex Objects]
// ==========================================================================================

class HttpRequest {
  std::string method_;
  std::string url_;
  std::string body_;
  int timeout_;

  // EN: Private constructor — only Builder can create.
  // TR: Özel yapıcı — yalnızca Builder oluşturabilir.
  HttpRequest(std::string m, std::string u, std::string b, int t)
      : method_(std::move(m)), url_(std::move(u)), body_(std::move(b)),
        timeout_(t) {}

public:
  class Builder {
    std::string method_ = "GET";
    std::string url_;
    std::string body_;
    int timeout_ = 30;

  public:
    Builder &setMethod(const std::string &m) { method_ = m; return *this; }
    Builder &setUrl(const std::string &u) { url_ = u; return *this; }
    Builder &setBody(const std::string &b) { body_ = b; return *this; }
    Builder &setTimeout(int t) { timeout_ = t; return *this; }

    HttpRequest build() {
      return HttpRequest(method_, url_, body_, timeout_);
    }
  };

  void display() const {
    std::cout << "[Builder] " << method_ << " " << url_;
    if (!body_.empty()) std::cout << " | Body: " << body_;
    std::cout << " | Timeout: " << timeout_ << "s" << std::endl;
  }
};

// ==========================================================================================
// [4. ABSTRACT FACTORY PATTERN — Family of Related Objects]
// ==========================================================================================

// EN: Abstract product interfaces
// TR: Soyut ürün arayüzleri
class IButton {
public:
  virtual ~IButton() = default;
  virtual void render() const = 0;
};
```

```cpp

class ICheckbox {
public:
  virtual ~ICheckbox() = default;
  virtual void render() const = 0;
};

// --- Windows family ---
class WindowsButton : public IButton {
public:
  void render() const override {
    std::cout << "[Win] Rendering Windows-style button." << std::endl;
  }
};
class WindowsCheckbox : public ICheckbox {
public:
  void render() const override {
    std::cout << "[Win] Rendering Windows-style checkbox." << std::endl;
  }
};

// --- macOS family ---
class MacButton : public IButton {
public:
  void render() const override {
    std::cout << "[Mac] Rendering macOS-style button." << std::endl;
  }
};
class MacCheckbox : public ICheckbox {
public:
  void render() const override {
    std::cout << "[Mac] Rendering macOS-style checkbox." << std::endl;
  }
};

// EN: Abstract Factory interface
// TR: Soyut Fabrika arayüzü
class IGUIFactory {
public:
  virtual ~IGUIFactory() = default;
  virtual std::unique_ptr<IButton> createButton() const = 0;
  virtual std::unique_ptr<ICheckbox> createCheckbox() const = 0;
};

class WindowsFactory : public IGUIFactory {
public:
  std::unique_ptr<IButton> createButton() const override {
    return std::make_unique<WindowsButton>();
  }
  std::unique_ptr<ICheckbox> createCheckbox() const override {
    return std::make_unique<WindowsCheckbox>();
  }
};

class MacFactory : public IGUIFactory {
public:
  std::unique_ptr<IButton> createButton() const override {
    return std::make_unique<MacButton>();
  }
  std::unique_ptr<ICheckbox> createCheckbox() const override {
    return std::make_unique<MacCheckbox>();
  }
};

// EN: Client code works with ANY factory — no concrete types mentioned.
// TR: İstemci kodu HERHANGİ bir fabrikayla çalışır — somut tip bahsedilmez.
void buildUI(const IGUIFactory &factory) {
  auto btn = factory.createButton();
  auto chk = factory.createCheckbox();
  btn->render();
  chk->render();
}
```

```cpp

int main() {
  std::cout << "=== MODULE 10: CREATIONAL DESIGN PATTERNS ===\n" << std::endl;

  // --- 1. Singleton ---
  std::cout << "--- SINGLETON ---\n" << std::endl;
  Logger::getInstance().log("Application started.");
  Logger::getInstance().log("Processing data...");
  Logger::getInstance().log("Shutdown complete.");
  std::cout << std::endl;

  // --- 2. Factory Method ---
  std::cout << "--- FACTORY METHOD ---\n" << std::endl;
  std::vector<std::string> ecuList = {"bcm", "bcm", "powertrain", "hvac"};
  for (const auto &type : ecuList) {
    auto mod = createModule(type);
    if (mod) {
      std::cout << "Loaded: " << mod->moduleName() << " -> ";
      mod->runDiagnostic();
    }
  }
  std::cout << std::endl;

  // --- 3. Builder ---
  std::cout << "--- BUILDER ---\n" << std::endl;
  auto req1 = HttpRequest::Builder()
                  .setMethod("POST")
                  .setUrl("https://api.example.com/users")
                  .setBody(R"({"name":"Ahmet"})")
                  .setTimeout(10)
                  .build();
  auto req2 = HttpRequest::Builder()
                  .setUrl("https://api.example.com/health")
                  .build(); // Uses defaults: GET, timeout=30
  req1.display();
  req2.display();
  std::cout << std::endl;

  // --- 4. Abstract Factory ---
  std::cout << "--- ABSTRACT FACTORY ---\n" << std::endl;
  WindowsFactory wf;
  MacFactory mf;
  std::cout << "Building Windows UI:" << std::endl;
  buildUI(wf);
  std::cout << "Building macOS UI:" << std::endl;
  buildUI(mf);

  std::cout << "\n=> All creational patterns demonstrated!" << std::endl;

  return 0;
}
```

► 03. Behavioral Patterns

module_10_design_patterns/03_behavioral_patterns.cpp

Konu: Design Patterns: Behavioral — Observer, Strategy, Command, State / Davranışsal Kalıplar

Teori / Theory

THEORY: Behavioral Patterns Overview

Behavioral patterns focus on HOW objects communicate and distribute responsibilities. They define the protocols for interaction between objects, making systems more flexible and easier to extend. These patterns are the backbone of event-driven architectures, ECU communication lines, and UI frameworks.

TEORİ: Davranışsal Kalıplar

Davranışsal kalıplar, nesnelerin NASIL iletişim kurduğuna ve sorumlulukları nasıl dağıttığına odaklanır. Nesneler arasındaki etkileşim protokollerini tanımlar, sistemleri daha esnek ve genişletmesi kolay kılar. Bu kalıplar olay-güdümlü mimarilerin, ECU iletişim hatlarının ve HMI çerçevelerinin omurgasıdır.

THEORY: Observer — Publish/Subscribe Event System

The Observer pattern defines a one-to-many dependency: when one object (Subject) changes state, ALL registered observers are notified automatically. The subject doesn't know concrete observer types — it only knows the `IObserver` interface. Used in: ECU event buses, HMI notification systems, CAN signal dispatchers, sensor data pipelines.

TEORİ: Observer — Yayınla/Abone Ol Olay Sistemi

Observer kalıbı bir-çoğa bağımlılık tanımlar: bir nesne (Konu) durum değiştirdiğinde, TÜM kayıtlı gözlemciler otomatik bilgilendirilir. Konu, somut gözlemci tiplerini bilmez — yalnızca `IObserver` arayüzünü bilir. Kullanım: ECU olay veri yolları, HMI bildirim sistemleri, CAN sinyal dağıtıcıları, sensör veri hatları.

THEORY: Strategy — Swappable Algorithms at Runtime

The Strategy pattern encapsulates a family of algorithms behind an interface and makes them interchangeable at runtime. Instead of `if/else` chains selecting behavior, you inject the desired strategy object. The context delegates the algorithm call to the strategy. Adding new algorithms requires ZERO modification of existing code (Open/Closed Principle).

TEORİ: Strategy — Çalışma Zamanında Değiştirilebilir Algoritmalar

Strategy kalıbı, bir algoritma ailesini bir arayüz arkasında kapsüller ve çalışma zamanında değiştirilebilir kılar. Davranış seçen `if/else` zincirleri yerine istenen strateji nesnesi enjekte edilir. Bağlam, algoritma çağrısını stratejiye delege eder. Yeni algoritma eklemek mevcut kodda SIFIR değişiklik gerektirir (Açık/Kapalı Prensibi).

THEORY: Command — Encapsulate Request as Object

The Command pattern turns a request/action into a standalone object with all necessary information. This enables: queuing commands, logging them, implementing undo/redo, and executing them later. Each command implements an `execute()` method. The invoker doesn't know what the command does — it just calls `execute()`.

TEORİ: Command — İsteği Nesne Olarak Kapsülle

Command kalıbı, bir istek/eylemi gerekli tüm bilgilerle bağımsız bir nesneye dönüştürür. Bu şunları mümkün kılar: komutları kuyruğa alma, loglama, geri alma/yineleme uygulamak ve sonra çalıştırma. Her komut bir `execute()` metodu uygular. Çağırıcı komutun ne yaptığını bilmez — sadece `execute()` çağırır.

THEORY: State — Object Behavior Changes with Internal State

The State pattern lets an object change its behavior when its internal state changes. Instead of massive `switch/case` blocks, each state is a separate class with its own behavior. State transitions happen by replacing the current state object. The context object delegates all calls to the current state.

TEORİ: State — Nesne Davranışı İç Durumla Değişir

State kalıbı, bir nesnenin iç durumu değiştiğinde davranışını değiştirmesine olanak tanır. Devasa `switch/case` blokları yerine her durum, kendi davranışına sahip ayrı bir sınıftır. Durum geçişleri mevcut durum nesnesini değiştirerek gerçekleşir. Bağlam nesnesi tüm çağrıları mevcut duruma delege eder.

CPPREF Depth: std::function as Strategy Container

In modern C++, you don't always need a full class hierarchy for Strategy. `std::function<ReturnType(Args...)>` can hold any callable (function pointer, lambda, functor, member function via std::bind). This is lighter than virtual dispatch for simple strategies. However, `std::function` has type-erasure overhead (~40 bytes + potential heap allocation for large callables). cppreference.com/w/cpp/utility/functional/function

CPPREF Derinlik: Strateji Kapsayıcı Olarak std::function

Modern C++'ta Strategy için her zaman tam bir sınıf hiyerarşisine ihtiyacınız yok. `std::function<DönüşTipi(Argümanlar...)>` herhangi bir çağrılabiliryi tutabilir (fonksiyon işaretçisi, lambda, functor, std::bind ile üye fonksiyon). Basit stratejiler için sanal fonksiyon dağıtımından daha hafiftir. Ancak `std::function`'ın tip-silme maliyeti vardır (~40 bayt + büyük çağrılabilirler için potansiyel heap tahsisi). cppreference.com/w/cpp/utility/functional/function

CPPREF Depth: std::variant as Type-Safe State Machine

C++17's `std::variant` can replace virtual-dispatch State pattern with a zero-overhead, value-based approach. Each state is a struct, the variant holds one at a time. `std::visit` with a visitor dispatches to the correct state handler at compile-time. No heap allocation, no virtual table — pure stack-based state machine. cppreference.com/w/cpp/utility/variant

CPPREF Derinlik: Tip-Güvenli Durum Makinesi Olarak std::variant

C++17'nin `std::variant`'ı, sanal-dağıtımlı State kalıbını sıfır maliyetli, değer-tabanlı bir yaklaşımla değiştirebilir. Her durum bir struct, variant aynı anda birini tutar. Bir visitor ile `std::visit`, derleme zamanında doğru durum işleyicisine yönlendirir. Heap tahsisi yok, sanal tablo yok — tamamen yığın-tabanlı durum makinesi. cppreference.com/w/cpp/utility/variant

Kaynak Kod / Source Code

```cpp
#include <functional>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ==========================================================================================
// [1. OBSERVER PATTERN]
// ==========================================================================================

class IObserver {
public:
  virtual ~IObserver() = default;
  virtual void onNotify(const std::string &event) = 0;
};

class EventBus {
  std::vector<IObserver *> observers_;

public:
  void subscribe(IObserver *obs) { observers_.push_back(obs); }

  void notify(const std::string &event) {
    for (auto *obs : observers_) {
      obs->onNotify(event);
    }
  }
};

class FaultLogger : public IObserver {
public:
  void onNotify(const std::string &event) override {
    if (event == "ENGINE_OVERHEAT") {
      std::cout << "[FaultLog] Logging DTC P0217 — Engine Overheat."
                << std::endl;
    } else if (event == "MAINTENANCE_DUE") {
      std::cout << "[FaultLog] Logging scheduled maintenance reminder."
                << std::endl;
    }
  }
};

class ClusterDisplay : public IObserver {
public:
  void onNotify(const std::string &event) override {
    if (event == "ENGINE_OVERHEAT") {
      std::cout << "[Cluster] Showing RED overheat warning on dashboard!"
                << std::endl;
    } else if (event == "MAINTENANCE_DUE") {
      std::cout << "[Cluster] Showing wrench icon — service needed."
                << std::endl;
    }
  }
};

class TelematicsReporter : public IObserver {
public:
  void onNotify(const std::string &event) override {
    if (event == "ENGINE_OVERHEAT") {
      std::cout << "[Telematics] Uploading overheat snapshot to cloud."
                << std::endl;
    }
  }
};
```

```cpp

// ==========================================================================================
// [2. STRATEGY PATTERN]
// ==========================================================================================

class ISortStrategy {
public:
  virtual ~ISortStrategy() = default;
  virtual void sort(std::vector<int> &data) const = 0;
  virtual std::string name() const = 0;
};

class BubbleSort : public ISortStrategy {
public:
  void sort(std::vector<int> &data) const override {
    for (size_t i = 0; i < data.size(); i++) {
      for (size_t j = 0; j + 1 < data.size() - i; j++) {
        if (data[j] > data[j + 1]) std::swap(data[j], data[j + 1]);
      }
    }
  }
  std::string name() const override { return "BubbleSort"; }
};

class SelectionSort : public ISortStrategy {
public:
  void sort(std::vector<int> &data) const override {
    for (size_t i = 0; i < data.size(); i++) {
      size_t minIdx = i;
      for (size_t j = i + 1; j < data.size(); j++) {
        if (data[j] < data[minIdx]) minIdx = j;
      }
      std::swap(data[i], data[minIdx]);
    }
  }
  std::string name() const override { return "SelectionSort"; }
};

class InsertionSort : public ISortStrategy {
public:
  void sort(std::vector<int> &data) const override {
    for (size_t i = 1; i < data.size(); i++) {
      int key = data[i];
      size_t j = i;
      while (j > 0 && data[j - 1] > key) {
        data[j] = data[j - 1];
        j--;
      }
      data[j] = key;
    }
  }
  std::string name() const override { return "InsertionSort"; }
};

// EN: Context class — delegates sorting to injected strategy.
// TR: Bağlam sınıfı — sıralamayı enjekte edilen stratejiye delege eder.
class Sorter {
  std::unique_ptr<ISortStrategy> strategy_;

public:
  void setStrategy(std::unique_ptr<ISortStrategy> s) {
    strategy_ = std::move(s);
  }
  void doSort(std::vector<int> &data) {
    if (strategy_) {
      std::cout << "[Strategy] Using " << strategy_->name() << ": ";
      strategy_->sort(data);
      for (int v : data) std::cout << v << " ";
      std::cout << std::endl;
    }
  }
};
```

```cpp

// ==========================================================================================
// [3. COMMAND PATTERN — Undo/Redo Support]
// ==========================================================================================

class ICommand {
public:
  virtual ~ICommand() = default;
  virtual void execute() = 0;
  virtual void undo() = 0;
  virtual std::string description() const = 0;
};

class TextEditor {
  std::string text_;

public:
  void insert(const std::string &s) { text_ += s; }
  void removeLast(size_t n) {
    if (n <= text_.size()) text_.erase(text_.size() - n, n);
  }
  const std::string &getText() const { return text_; }
};

class InsertTextCommand : public ICommand {
  TextEditor &editor_;
  std::string text_;

public:
  InsertTextCommand(TextEditor &e, std::string t)
      : editor_(e), text_(std::move(t)) {}
  void execute() override { editor_.insert(text_); }
  void undo() override { editor_.removeLast(text_.size()); }
  std::string description() const override {
    return "Insert '" + text_ + "'";
  }
};

class CommandHistory {
  std::vector<std::unique_ptr<ICommand>> history_;

public:
  void executeCommand(std::unique_ptr<ICommand> cmd) {
    cmd->execute();
    std::cout << "[Command] Executed: " << cmd->description() << std::endl;
    history_.push_back(std::move(cmd));
  }
  void undoLast() {
    if (!history_.empty()) {
      auto &cmd = history_.back();
      cmd->undo();
      std::cout << "[Command] Undone: " << cmd->description() << std::endl;
      history_.pop_back();
    }
  }
};

// ==========================================================================================
// [4. STATE PATTERN — Traffic Light State Machine]
// ==========================================================================================

class TrafficLight; // Forward declaration

class ITrafficState {
public:
  virtual ~ITrafficState() = default;
  virtual void handle(TrafficLight &context) = 0;
  virtual std::string color() const = 0;
};
```

```cpp

class TrafficLight {
  std::unique_ptr<ITrafficState> state_;

public:
  explicit TrafficLight(std::unique_ptr<ITrafficState> initial)
      : state_(std::move(initial)) {}

  void setState(std::unique_ptr<ITrafficState> s) { state_ = std::move(s); }

  void next() {
    std::cout << "[State] Current: " << state_->color() << " -> ";
    state_->handle(*this);
  }
};

class GreenState : public ITrafficState {
public:
  void handle(TrafficLight &context) override;
  std::string color() const override { return "GREEN"; }
};

class YellowState : public ITrafficState {
public:
  void handle(TrafficLight &context) override;
  std::string color() const override { return "YELLOW"; }
};

class RedState : public ITrafficState {
public:
  void handle(TrafficLight &context) override {
    std::cout << "switching to GREEN" << std::endl;
    context.setState(std::make_unique<GreenState>());
  }
  std::string color() const override { return "RED"; }
};

void GreenState::handle(TrafficLight &context) {
  std::cout << "switching to YELLOW" << std::endl;
  context.setState(std::make_unique<YellowState>());
}

void YellowState::handle(TrafficLight &context) {
  std::cout << "switching to RED" << std::endl;
  context.setState(std::make_unique<RedState>());
}

int main() {
  std::cout << "=== MODULE 10: BEHAVIORAL DESIGN PATTERNS ===\n" << std::endl;

  // --- 1. Observer ---
  std::cout << "--- OBSERVER (Vehicle Event Bus) ---\n" << std::endl;
  EventBus bus;
  FaultLogger faultLog;
  ClusterDisplay cluster;
  TelematicsReporter telematics;
  bus.subscribe(&faultLog);
  bus.subscribe(&cluster);
  bus.subscribe(&telematics);
  bus.notify("ENGINE_OVERHEAT");
  std::cout << std::endl;
  bus.notify("MAINTENANCE_DUE");
  std::cout << std::endl;
```

```cpp

    // --- 2. Strategy ---
    std::cout << "--- STRATEGY (Sorting) ---\n" << std::endl;
    Sorter sorter;
    std::vector<int> data1 = {5, 2, 8, 1, 9};
    std::vector<int> data2 = {7, 3, 6, 4, 0};
    std::vector<int> data3 = {12, 11, 15, 13, 14};

    sorter.setStrategy(std::make_unique<BubbleSort>());
    sorter.doSort(data1);
    sorter.setStrategy(std::make_unique<SelectionSort>());
    sorter.doSort(data2);
    sorter.setStrategy(std::make_unique<InsertionSort>());
    sorter.doSort(data3);
    std::cout << std::endl;

    // --- 3. Command (Undo/Redo) ---
    std::cout << "--- COMMAND (Text Editor with Undo) ---\n" << std::endl;
    TextEditor editor;
    CommandHistory history;

    history.executeCommand(
        std::make_unique<InsertTextCommand>(editor, "Hello"));
    history.executeCommand(
        std::make_unique<InsertTextCommand>(editor, " World"));
    history.executeCommand(
        std::make_unique<InsertTextCommand>(editor, "!"));
    std::cout << "Text: \"" << editor.getText() << "\"" << std::endl;

    history.undoLast(); // Removes "!"
    std::cout << "After undo: \"" << editor.getText() << "\"" << std::endl;
    history.undoLast(); // Removes " World"
    std::cout << "After undo: \"" << editor.getText() << "\"\n" << std::endl;

    // --- 4. State (Traffic Light) ---
    std::cout << "--- STATE (Traffic Light) ---\n" << std::endl;
    TrafficLight light(std::make_unique<GreenState>());
    for (int i = 0; i < 6; i++) {
      light.next(); // GREEN->YELLOW->RED->GREEN->YELLOW->RED
    }

    std::cout << "\n=> All behavioral patterns demonstrated!" << std::endl;

    return 0;
  }
```

► 04. Structural Patterns

module_10_design_patterns/04_structural_patterns.cpp

Konu: Design Patterns: Structural — Adapter, Decorator, Facade, Composite, Proxy / Yapısal

Teori / Theory

THEORY: Structural Patterns Overview

Structural patterns deal with HOW classes and objects are COMPOSED to form larger structures. They simplify complex relationships, adapt incompatible interfaces, and add new behavior without altering existing code. Think of them as "architectural connectors" that glue components together.

TEORİ: Yapısal Kalıplar

Yapısal kalıplar, sınıfların ve nesnelerin daha büyük yapılar oluşturmak için NASIL BİRLEŞTİRİLDİĞİ ile ilgilenir. Karmaşık ilişkileri basitleştirir, uyumsuz arayüzleri uyarlar ve mevcut kodu değiştirmeden yeni davranış ekler. Bileşenleri birbirine yapıştıran "mimari bağlayıcılar" olarak düşünün.

THEORY: Adapter — Make Incompatible Interfaces Work Together

The Adapter pattern wraps an existing class with an incompatible interface so it matches the interface the client expects. Like a power adapter that converts EU plugs to US outlets. The adapted class doesn't need ANY modification — the adapter handles the translation.

TEORİ: Adapter — Uyumsuz Arayüzleri Birlikte Çalıştır

Adapter kalıbı, uyumsuz bir arayüze sahip mevcut sınıfı, istemcinin beklediği arayüze uyacak şekilde sarar. AB fişlerini ABD prizlerine dönüştüren güç adaptörü gibi. Uyarlanan sınıf HİÇBİR değişiklik gerektirmez — adaptör çeviriyi halleder.

THEORY: Decorator — Add Behavior Dynamically

The Decorator pattern wraps an object to ADD new behavior without modifying the original class. Decorators share the same interface as the wrapped object, so they can be stacked (chained). Example: adding encryption, compression, and logging to a data stream — each decorator wraps the previous one.

TEORİ: Decorator — Davranışı Dinamik Olarak Ekle

Decorator kalıbı, orijinal sınıfı değiştirmeden yeni davranış EKLEMEK için bir nesneyi sarar. Dekoratörler sarılan nesneyle aynı arayüzü paylaşır, böylece üst üste yığılabilir (zincirlenir). Örnek: bir veri akışına şifreleme, sıkıştırma ve loglama ekleme — her dekoratör öncekini sarar.

THEORY: Facade — Simplified Interface to Complex System

The Facade pattern provides a SIMPLIFIED interface to a complex subsystem consisting of many classes. Instead of clients managing 10 different objects, they talk to ONE facade that orchestrates everything behind the scenes. Common in automotive middleware, media frameworks, and library wrappers.

TEORİ: Facade — Karmaşık Sisteme Basitleştirilmiş Arayüz

Facade kalıbı, birçok sınıftan oluşan karmaşık bir alt sisteme BASİTLEŞTİRİLMİŞ bir arayüz sağlar. İstemciler 10 farklı nesneyi yönetmek yerine, perde arkasında her şeyi orkestra eden TEK bir facade ile konuşurlar. Oyun motorlarında, medya çerçevelerinde ve kütüphane sarmalayıcılarda yaygındır.

THEORY: Composite — Treat Individual and Group Objects Uniformly

The Composite pattern lets you compose objects into tree structures and treat individual objects and compositions UNIFORMLY through the same interface. A `File` and a `Folder` (containing files and subfolders) both implement `getSize()`. The client doesn't care if it's a leaf or a branch.

TEORİ: Composite — Tekil ve Grup Nesneleri Aynı Şekilde Ele Al

Composite kalıbı, nesneleri ağaç yapılarına birleştirmenize ve tekil nesnelerle kompozisyonları aynı arayüz aracılığıyla AYNI ŞEKİLDE ele almanıza olanak tanır. Bir `Dosya` ve bir `Klasör` (dosya ve alt klasörler içeren) her ikisi de `getSize()` uygular. İstemci yaprak mı dal mı olduğunu umursamaz.

CPPREF Depth: Multiple Inheritance for Mixin-Style Decorator Chains

C++ supports multiple inheritance, which enables mixin-style decorators where behavior is composed by inheriting from multiple small classes. Each mixin adds ONE capability. Virtual inheritance resolves the diamond problem when mixins share a common base. In practice, prefer composition-based decorators (wrapping via constructor) over inheritance-based ones for better flexibility. cppreference.com/w/cpp/language/derived_class

CPPREF Derinlik: Mixin-Tarzı Dekoratör Zincirleri için Çoklu Kalıtım

C++ çoklu kalıtımı destekler, bu da davranışın birden fazla küçük sınıftan kalıtımla birleştirildiği mixin-tarzı dekoratörleri mümkün kılar. Her mixin TEK bir yetenek ekler. Sanal kalıtım, mixin'ler ortak bir temeli paylaştığında elmas problemini çözer. Pratikte, daha iyi esneklik için kalıtım-tabanlı yerine bileşim-tabanlı dekoratörleri (yapıcı ile sarma) tercih edin. cppreference.com/w/cpp/language/derived_class

CPPREF Depth: std::shared_ptr for Shared Composite Ownership

In Composite pattern implementations, child nodes can be shared among multiple parents (e.g., symbolic links, DAG structures). Use `std::shared_ptr` for shared ownership where multiple owners keep the object alive. Use `std::weak_ptr` to break circular references. For strict tree hierarchies (single parent), `std::unique_ptr` is preferred for zero overhead. cppreference.com/w/cpp/memory/shared_ptr

CPPREF Derinlik: Paylaşımlı Composite Sahipliği için std::shared_ptr

Composite kalıp uygulamalarında, çocuk düğümler birden fazla ebeveyn arasında paylaşılabilir (örn. sembolik bağlantılar, DAG yapıları). Birden fazla sahibin nesneyi canlı tuttuğu paylaşımlı sahiplik için `std::shared_ptr` kullanın. Dairesel referansları kırmak için `std::weak_ptr` kullanın. Katı ağaç hiyerarşileri (tek ebeveyn) için sıfır maliyetli `std::unique_ptr` tercih edilir. cppreference.com/w/cpp/memory/shared_ptr

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <numeric>
#include <string>
#include <vector>

// ==========================================================================================
// [1. ADAPTER PATTERN]
// ==========================================================================================

// EN: Legacy library we CANNOT modify (third-party).
// TR: Değiştiremeyeceğimiz eski kütüphane (üçüncü taraf).
class LegacyXMLParser {
public:
  std::string parseXML(const std::string &xml) {
    return "[XML Parsed] " + xml;
  }
};

// EN: Interface our system expects.
// TR: Sistemimizin beklediği arayüz.
class IDataParser {
public:
  virtual ~IDataParser() = default;
  virtual std::string parse(const std::string &data) = 0;
};

// EN: Adapter wraps the legacy class to match our interface.
// TR: Adaptör, eski sınıfı bizim arayüzümüze uyacak şekilde sarar.
class XMLParserAdapter : public IDataParser {
  LegacyXMLParser legacy_;

public:
  std::string parse(const std::string &data) override {
    return legacy_.parseXML(data);
  }
};

class JSONParser : public IDataParser {
public:
  std::string parse(const std::string &data) override {
    return "[JSON Parsed] " + data;
  }
};

// ==========================================================================================
// [2. DECORATOR PATTERN — Stacking Behaviors]
// ==========================================================================================

class IMessage {
public:
  virtual ~IMessage() = default;
  virtual std::string getContent() const = 0;
};

class PlainMessage : public IMessage {
  std::string text_;

public:
  explicit PlainMessage(std::string t) : text_(std::move(t)) {}
  std::string getContent() const override { return text_; }
};
```

```cpp

// EN: Base decorator — wraps an IMessage.
// TR: Temel dekoratör — bir IMessage sarar.
class MessageDecorator : public IMessage {
protected:
  std::unique_ptr<IMessage> wrapped_;

public:
  explicit MessageDecorator(std::unique_ptr<IMessage> m)
      : wrapped_(std::move(m)) {}
};

class EncryptedMessage : public MessageDecorator {
public:
  using MessageDecorator::MessageDecorator;
  std::string getContent() const override {
    return "[ENCRYPTED]{" + wrapped_->getContent() + "}";
  }
};

class CompressedMessage : public MessageDecorator {
public:
  using MessageDecorator::MessageDecorator;
  std::string getContent() const override {
    return "[COMPRESSED]{" + wrapped_->getContent() + "}";
  }
};

class LoggedMessage : public MessageDecorator {
public:
  using MessageDecorator::MessageDecorator;
  std::string getContent() const override {
    std::cout << "[LOG] Message accessed." << std::endl;
    return wrapped_->getContent();
  }
};

// ==========================================================================================
// [3. FACADE PATTERN — Simplified Interface]
// ==========================================================================================

// EN: Complex subsystem classes (Instrument Cluster subsystems).
// TR: Karmaşık alt sistem sınıfları (Gösterge paneli alt sistemleri).
class AudioSubsystem {
public:
  void playChime(const std::string &chime) {
    std::cout << "  [Audio] Playing chime: " << chime << std::endl;
  }
};

class SensorFusionEngine {
public:
  void calibrateSensors(int sensorCount) {
    std::cout << "  [SensorFusion] Calibrating " << sensorCount
              << " sensors" << std::endl;
  }
};

class DisplayRenderer {
public:
  void setResolution(int w, int h) {
    std::cout << "  [Display] Cluster resolution: " << w << "x" << h
              << std::endl;
  }
};

class CANGateway {
public:
  void connectBus(const std::string &busName) {
    std::cout << "  [CAN] Connected to " << busName << " bus" << std::endl;
  }
};
```

```cpp

// EN: Facade — ONE method to initialize the entire instrument cluster.
// TR: Facade — Tüm gösterge panelini başlatmak için TEK metod.
class InstrumentClusterFacade {
  AudioSubsystem audio_;
  SensorFusionEngine sensors_;
  DisplayRenderer display_;
  CANGateway can_;

public:
  void startCluster(const std::string &vehicleId) {
    std::cout << "[Facade] Initializing instrument cluster for "
              << vehicleId << "..." << std::endl;
    display_.setResolution(1920, 720);
    sensors_.calibrateSensors(12);
    audio_.playChime("startup_chime.wav");
    can_.connectBus("Vehicle CAN-HS");
    std::cout << "[Facade] Cluster ready!" << std::endl;
  }
};

// ==========================================================================================
// [4. COMPOSITE PATTERN — File System Tree]
// ==========================================================================================

class IFileComponent {
public:
  virtual ~IFileComponent() = default;
  virtual int getSize() const = 0;
  virtual void display(int depth = 0) const = 0;
  virtual std::string getName() const = 0;
};

class File : public IFileComponent {
  std::string name_;
  int size_;

public:
  File(std::string n, int s) : name_(std::move(n)), size_(s) {}
  int getSize() const override { return size_; }
  std::string getName() const override { return name_; }
  void display(int depth) const override {
    for (int i = 0; i < depth; i++) std::cout << "  ";
    std::cout << "□ " << name_ << " (" << size_ << " KB)" << std::endl;
  }
};

class Folder : public IFileComponent {
  std::string name_;
  std::vector<std::unique_ptr<IFileComponent>> children_;

public:
  explicit Folder(std::string n) : name_(std::move(n)) {}

  void add(std::unique_ptr<IFileComponent> child) {
    children_.push_back(std::move(child));
  }

  int getSize() const override {
    return std::accumulate(children_.begin(), children_.end(), 0,
                           [](int sum, const auto &c) { return sum + c->getSize(); });
  }

  std::string getName() const override { return name_; }

  void display(int depth) const override {
    for (int i = 0; i < depth; i++) std::cout << "  ";
    std::cout << "□ " << name_ << " (" << getSize() << " KB total)" << std::endl;
    for (const auto &child : children_) {
      child->display(depth + 1);
    }
  }
};
```

```
208
209 int main() {
210   std::cout << "=== MODULE 10: STRUCTURAL DESIGN PATTERNS ===\n" << std::endl;
211
212   // --- 1. Adapter ---
213   std::cout << "--- ADAPTER ---\n" << std::endl;
214   XMLParserAdapter xmlAdapter;
215   JSONParser jsonParser;
216   std::cout << xmlAdapter.parse("<user>Ahmet</user>") << std::endl;
217   std::cout << jsonParser.parse(R"({"name":"Ahmet"})") << "\n" << std::endl;
218
219   // --- 2. Decorator (Stacking) ---
220   std::cout << "--- DECORATOR (Stacking) ---\n" << std::endl;
221   // EN: Stack: Plain -> Compressed -> Encrypted -> Logged
222   // TR: Yığın: Düz -> Sıkıştırılmış -> Şifrelenmiş -> Loglanmış
223   auto msg = std::make_unique<PlainMessage>("Hello, World!");
224   auto compressed = std::make_unique<CompressedMessage>(std::move(msg));
225   auto encrypted = std::make_unique<EncryptedMessage>(std::move(compressed));
226   auto logged = std::make_unique<LoggedMessage>(std::move(encrypted));
227   std::cout << "Result: " << logged->getContent() << "\n" << std::endl;
228
229   // --- 3. Facade ---
230   std::cout << "--- FACADE ---\n" << std::endl;
231   InstrumentClusterFacade cluster;
232   cluster.startCluster("VIN_WBAPH5C55BA271234");
233   std::cout << std::endl;
234
235   // --- 4. Composite (File System) ---
236   std::cout << "--- COMPOSITE (File System Tree) ---\n" << std::endl;
237   auto root = std::make_unique<Folder>("project");
238   auto src = std::make_unique<Folder>("src");
239   src->add(std::make_unique<File>("main.cpp", 12));
240   src->add(std::make_unique<File>("utils.cpp", 8));
241   auto docs = std::make_unique<Folder>("docs");
242   docs->add(std::make_unique<File>("README.md", 3));
243   root->add(std::move(src));
244   root->add(std::move(docs));
245   root->add(std::make_unique<File>("Makefile", 1));
246   root->display(0);
247
248   std::cout << "\n=> All structural patterns demonstrated!" << std::endl;
249
250   return 0;
251 }
```

► 05. Testable Design

module_10_design_patterns/05_testable_design.cpp

Konu: Design Patterns: Testable Design — CRTP, Type Erasure, Policy-Based Design / Test

Teori / Theory

THEORY: CRTP — Curiously Recurring Template Pattern

CRTP is a technique where a class inherits from a template that takes ITSELF as the template parameter: `class Derived : public Base<Derived>`. This enables "static polymorphism" — the base class calls methods on Derived at COMPILE TIME without any virtual function overhead (no vtable, no indirection). Used for: mixin injection, compile-time interfaces, expression templates, and the "static interface" pattern.

TEORİ: CRTP — Merakla Tekrarlanan Şablon Kalıbı

CRTP, bir sınıfın KENDİSİNİ şablon parametresi olarak alan bir şablondan miras aldığı bir tekniktir: `class Derived : public Base<Derived>`. Bu "statik polimorfizm" sağlar — temel sınıf Derived üzerindeki metodları DERLEME ZAMANINDA hiçbir sanal fonksiyon maliyeti olmadan çağırır (vtable yok, dolaylı erişim yok). Kullanım: mixin enjeksiyonu, derleme zamanı arayüzleri, ifade şablonları ve "statik arayüz" kalıbı.

THEORY: Type Erasure — Polymorphism Without Inheritance

Type Erasure hides concrete types behind a uniform interface WITHOUT requiring inheritance from a common base class. It combines templates, a polymorphic wrapper class, and internal virtual dispatch. The standard library uses this in `std::function`, `std::any`, and `std::format`. The pattern: store any type-erased object, access it through a fixed interface. "Duck typing in C++".

TEORİ: Type Erasure — Kalıtımsız Polimorfizm

Type Erasure, ortak temel sınıftan kalıtım gerektirmeden somut tipleri tekdüze bir arayüz arkasına gizler. Şablonları, polimorfik bir sarmalayıcı sınıf ve dahili sanal dağıtımı birleştirir. Standart kütüphane bunu `std::function`, `std::any` ve `std::format`'ta kullanır. Kalıp: tip silinmiş herhangi bir nesneyi depola, sabit bir arayüz aracılığıyla eriş. "C++'ta Duck Typing".

THEORY: Policy-Based Design — Compile-Time Strategy Pattern

Policy-Based Design uses template parameters as "policies" that define behavior at COMPILE TIME. Instead of runtime virtual dispatch (Strategy pattern), the behavior is baked into the type itself. Zero runtime overhead! Popularized by Andrei Alexandrescu's "Modern C++ Design". You can mix and match policies freely: `Widget<ThreadSafe, JSONLogging, PoolAllocator>`.

TEORİ: Policy-Based Design — Derleme Zamanı Strateji Kalıbı

Policy-Based Design, davranışı DERLEME ZAMANINDA tanımlayan "politikalar" olarak şablon parametreleri kullanır. Çalışma zamanı sanal dağıtım (Strategy kalıbı) yerine davranış tipin kendisine gömülür. Sıfır çalışma zamanı maliyeti! Andrei Alexandrescu'nun "Modern C++ Design" ile popülerleşti. Politikaları serbestçe karıştırabilirsiniz: `Widget<ThreadSafe, JSONLogging, PoolAllocator>`.

THEORY: Compile-Time Polymorphism vs Runtime Polymorphism

Runtime polymorphism (virtual functions) selects behavior at RUNTIME via vtable lookup (~1 cache miss per call). Compile-time polymorphism (CRTP, templates, concepts) selects behavior at COMPILE TIME with zero overhead. Trade-off: compile-time = faster execution, longer compile times, larger binary; runtime = slower execution, faster compilation, smaller binary. Embedded & HPC prefer compile-time; plugin architectures need runtime.

TEORİ: Derleme Zamanı vs Çalışma Zamanı Polimorfizmi

Çalışma zamanı polimorfizmi (sanal fonksiyonlar) vtable araması ile ÇALIŞMA ZAMANINDA davranış seçer (~çağrı başına 1 önbellek kaçırma). Derleme zamanı polimorfizmi (CRTP, şablonlar, concepts) DERLEME ZAMANINDA sıfır maliyetle davranış seçer. Takas: derleme zamanı = daha hızlı çalışma, daha uzun derleme, daha büyük binary; çalışma zamanı = daha yavaş çalışma, daha hızlı derleme, daha küçük binary. Gömülü & HPC derleme zamanını tercih eder; eklenti mimarileri çalışma zamanına ihtiyaç duyar.

CPPREF Depth: static_cast<Derived*>(this) in CRTP — The Core Mechanism

In CRTP, the base class accesses derived methods via `static_cast<Derived*>(this)->method()`. This is safe because the base class KNOWS (via the template parameter) that `this` is actually a `Derived` object. The cast is resolved at compile time with zero cost. However, calling a derived method that doesn't exist causes a compile error — functioning as a static interface checker. cppreference.com/w/cpp/language/static_cast

CPPREF Derinlik: CRTP'de static_cast<Derived*>(this) — Temel Mekanizma

CRTP'de temel sınıf, türetilmiş metodlara `static_cast<Derived*>(this)->method()` ile erişir. Bu güvenlidir çünkü temel sınıf (şablon parametresi aracılığıyla) `this`'in aslında bir `Derived` nesnesi olduğunu BİLİR. Dönüşüm derleme zamanında sıfır maliyetle çözülür. Ancak var olmayan bir türetilmiş metodu çağırmak derleme hatası verir — statik arayüz denetleyicisi olarak işlev görür. cppreference.com/w/cpp/language/static_cast

CPPREF Depth: std::any, std::function — Standard Library Type Erasure

`std::any` (C++17) can hold ANY copyable type and retrieve it with `std::any_cast<T>`. It stores small objects inline (SBO - Small Buffer Optimization, typically 16-32 bytes) and allocates larger ones on the heap. `std::function` uses the same technique for callable objects. Both incur type-erasure overhead but provide maximum flexibility. cppreference.com/w/cpp/utility/any

CPPREF Derinlik: std::any, std::function — Standart Kütüphane Type Erasure

`std::any` (C++17) herhangi bir kopyalanabilir tipi tutabilir ve `std::any_cast<T>` ile alınır. Küçük nesneleri satır içi depolar (SBO - Küçük Tampon Optimizasyonu, tipik 16-32 bayt) ve büyükleri heap'te tahsis eder. `std::function` çağrılabilir nesneler için aynı tekniği kullanır. İkisi de tip-silme maliyetine sahiptir ama maksimum esneklik sağlar. cppreference.com/w/cpp/utility/any

Kaynak Kod / Source Code

```cpp
#include <cmath>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ==========================================================================================
// [1. CRTP — Curiously Recurring Template Pattern]
// ==========================================================================================

// EN: Base class uses static_cast to call Derived methods at compile time.
// TR: Temel sınıf, Derived metodlarını derleme zamanında çağırmak için static_cast kullanır.
template <typename Derived> class Shape {
public:
  double area() const {
    // EN: Compile-time dispatch — no vtable!
    // TR: Derleme zamanı dağıtım — vtable yok!
    return static_cast<const Derived *>(this)->areaImpl();
  }

  std::string name() const {
    return static_cast<const Derived *>(this)->nameImpl();
  }

  void print() const {
    std::cout << "[CRTP] " << name() << " area = " << area() << std::endl;
  }
};

class CRTPCircle : public Shape<CRTPCircle> {
  double radius_;

public:
  explicit CRTPCircle(double r) : radius_(r) {}
  double areaImpl() const { return 3.14159 * radius_ * radius_; }
  std::string nameImpl() const { return "Circle"; }
};

class CRTPSquare : public Shape<CRTPSquare> {
  double side_;

public:
  explicit CRTPSquare(double s) : side_(s) {}
  double areaImpl() const { return side_ * side_; }
  std::string nameImpl() const { return "Square"; }
};

// EN: Static polymorphism — works with any Shape<T> at compile time.
// TR: Statik polimorfizm — derleme zamanında herhangi bir Shape<T> ile çalışır.
template <typename T> void printShape(const Shape<T> &shape) {
  shape.print();
}

// ==========================================================================================
// [2. TYPE ERASURE — Polymorphism Without Inheritance]
// ==========================================================================================
// EN: Any object that has a `draw()` method can be stored in our Drawable container — WITHOUT
// inheriting from any base class! "Duck typing".
// TR: `draw()` metodu olan herhangi bir nesne Drawable kapsayıcımızda depolanabilir — herhangi
// bir temel sınıftan miras almadan! "Duck typing".

class Drawable {
  struct Concept {
    virtual ~Concept() = default;
    virtual void draw() const = 0;
    virtual std::unique_ptr<Concept> clone() const = 0;
  };
```

```cpp

  template <typename T> struct Model : Concept {
    T obj_;
    explicit Model(T o) : obj_(std::move(o)) {}
    void draw() const override { obj_.draw(); }
    std::unique_ptr<Concept> clone() const override {
      return std::make_unique<Model>(obj_);
    }
  };

  std::unique_ptr<Concept> pimpl_;

public:
  template <typename T>
  Drawable(T x) : pimpl_(std::make_unique<Model<T>>(std::move(x))) {}

  Drawable(const Drawable &other) : pimpl_(other.pimpl_->clone()) {}
  Drawable &operator=(const Drawable &other) {
    pimpl_ = other.pimpl_->clone();
    return *this;
  }
  Drawable(Drawable &&) = default;
  Drawable &operator=(Drawable &&) = default;

  void draw() const { pimpl_->draw(); }
};

// EN: These classes DON'T inherit from anything — but they have draw()!
// TR: Bu sınıflar hiçbir şeyden miras ALMAZ — ama draw() var!
struct CANMessage {
  void draw() const {
    std::cout << "[TypeErasure] Rendering CAN frame on bus monitor."
              << std::endl;
  }
};

struct LINFrame {
  std::string label;
  void draw() const {
    std::cout << "[TypeErasure] Rendering LIN frame: " << label << std::endl;
  }
};

struct DiagnosticReport {
  std::string text;
  void draw() const {
    std::cout << "[TypeErasure] Rendering DTC report: " << text << std::endl;
  }
};

// ==========================================================================================
// [3. POLICY-BASED DESIGN — Compile-Time Strategy]
// ==========================================================================================

// --- Logging Policies ---
struct ConsoleLog {
  static void log(const std::string &msg) {
    std::cout << "[ConsoleLog] " << msg << std::endl;
  }
};

struct SilentLog {
  static void log(const std::string &) {} // No output
};

// --- Validation Policies ---
struct StrictValidation {
  static bool validate(int value) { return value > 0 && value < 1000; }
  static std::string policyName() { return "Strict"; }
};
```

```

struct RelaxedValidation {
  static bool validate(int) { return true; } // Accept everything
  static std::string policyName() { return "Relaxed"; }
};

// EN: Widget class configured by policies at COMPILE TIME.
// TR: Widget sınıfı DERLEME ZAMANINDA politikalarla yapılandırılır.
template <typename LogPolicy, typename ValidationPolicy> class Widget {
  int value_ = 0;

public:
  void setValue(int v) {
    if (ValidationPolicy::validate(v)) {
      value_ = v;
      LogPolicy::log("Value set to " + std::to_string(v) +
                     " [" + ValidationPolicy::policyName() + "]");
    } else {
      LogPolicy::log("Validation FAILED for " + std::to_string(v) +
                     " [" + ValidationPolicy::policyName() + "]");
    }
  }
  int getValue() const { return value_; }
};

// ==========================================================================================
// [4. COMPILE-TIME INTERFACE CHECKING WITH CRTP]
// ==========================================================================================
// EN: CRTP can enforce that Derived implements required methods — compile error if missing. This
// acts as a "static interface" / concept.
// TR: CRTP, Derived'ın gerekli metodları uygulamasını zorunlu kılabilir — eksikse derleme
// hatası. "Statik arayüz" / concept olarak çalışır.

template <typename Derived> class Serializable {
public:
  std::string serialize() const {
    return static_cast<const Derived *>(this)->serializeImpl();
  }
};

class Config : public Serializable<Config> {
  std::string key_;
  std::string value_;

public:
  Config(std::string k, std::string v)
      : key_(std::move(k)), value_(std::move(v)) {}

  // EN: Required by Serializable CRTP — compile error if missing!
  // TR: Serializable CRTP tarafından gerekli — eksikse derleme hatası!
  std::string serializeImpl() const {
    return key_ + "=" + value_;
  }
};

int main() {
  std::cout << "=== MODULE 10: TESTABLE & ADVANCED DESIGN PATTERNS ===\n"
            << std::endl;

  // --- 1. CRTP (Static Polymorphism) ---
  std::cout << "--- CRTP (Static Polymorphism) ---\n" << std::endl;
  CRTPCircle circle(5.0);
  CRTPSquare square(4.0);
  printShape(circle);
  printShape(square);
  std::cout << std::endl;

  // --- 2. Type Erasure ---
  std::cout << "--- TYPE ERASURE (Duck Typing) ---\n" << std::endl;
```

```
  // EN: Store completely unrelated types in the same container!
  // TR: Tamamen ilişkisiz tipleri aynı kapsayıcıda depola!
  std::vector<Drawable> canvas;
  canvas.emplace_back(CANMessage{});
  canvas.emplace_back(LINFrame{"Door_Status"});
  canvas.emplace_back(DiagnosticReport{"P0300 — Random Misfire"});

  for (const auto &drawable : canvas) {
    drawable.draw();
  }
  std::cout << std::endl;

  // --- 3. Policy-Based Design ---
  std::cout << "--- POLICY-BASED DESIGN ---\n" << std::endl;

  // EN: Strict validation + Console logging
  Widget<ConsoleLog, StrictValidation> strictWidget;
  strictWidget.setValue(42);     // OK
  strictWidget.setValue(-1);     // FAILS validation
  strictWidget.setValue(99999);  // FAILS validation

  std::cout << std::endl;

  // EN: Relaxed validation + Silent logging (zero overhead!)
  Widget<SilentLog, RelaxedValidation> relaxedWidget;
  relaxedWidget.setValue(-999); // Accepted silently
  std::cout << "Relaxed widget value: " << relaxedWidget.getValue() << std::endl;

  std::cout << std::endl;

  // --- 4. CRTP as Static Interface (Serializable) ---
  std::cout << "--- CRTP STATIC INTERFACE ---\n" << std::endl;
  Config cfg("database_host", "localhost:5432");
  std::cout << "Serialized config: " << cfg.serialize() << std::endl;

  std::cout << "\n=> CRTP, Type Erasure, and Policy-Based Design demonstrated!"
            << std::endl;

  return 0;
}
```

► 06. Pattern Decision Guide

module_10_design_patterns/06_pattern_decision_guide.cpp

Konu: Pattern Decision Guide — When to Use, When NOT to Use, Comparisons, Missing Patterns /

Teori / Theory

THEORY: Why a Decision Guide?

Knowing HOW to implement a pattern is only half the battle. The harder skill is knowing WHEN to use it, WHEN NOT to use it, and which pattern to choose among similar alternatives. This file provides: (1) anti-pattern warnings, (2) head-to-head comparisons between confusing pairs, (3) before/after refactoring examples, and (4) two missing patterns: Proxy and Prototype.

TEORİ: Neden Bir Karar Rehberi?

Bir kalıbı NASIL uygulayacağınızı bilmek savaşın sadece yarısıdır. Daha zor beceri, NE ZAMAN kullanacağınızı, NE ZAMAN KULLANMAYACAĞINIZI ve benzer alternatifler arasında hangisini seçeceğinizi bilmektir. Bu dosya şunları sunar: (1) anti-pattern uyarıları, (2) kafa karıştıran çiftler arası karşılaştırmalar, (3) öncesi/sonrası refactoring örnekleri ve (4) eksik iki kalıp: Proxy ve Prototype.

THEORY: Singleton — Anti-Pattern Warnings

Singleton is the MOST overused and MOST criticized design pattern. Before using it, ask:
✓ USE when: there's a physical constraint (one GPU, one log file, one config file)
✗ DON'T USE when: you just want "global access" — that's a global variable in disguise
✗ DON'T USE when: you need testability — Singleton makes mocking nearly impossible
✗ DON'T USE when: constructor has side effects (DB connections, file I/O)
⚠ WARNING: Singleton hides dependencies. Functions that use Logger::getInstance() have an invisible dependency that doesn't appear in their parameter list, violating Dependency Inversion Principle (DIP). Prefer injecting the logger as a reference parameter.

TEORİ: Singleton — Anti-Pattern Uyarıları

Singleton en ÇOK aşırı kullanılan ve en ÇOK eleştirilen tasarım kalıbıdır. Kullanmadan önce sorun:
✓ KULLAN: fiziksel kısıt varsa (tek GPU, tek log dosyası, tek config dosyası)
✗ KULLANMA: sadece "global erişim" istiyorsan — bu kılık değiştirmiş global değişkendir
✗ KULLANMA: test edilebilirlik gerekiyorsa — Singleton mock'lamayı neredeyse imkansız kılar
✗ KULLANMA: constructor yan etkilere sahipse (DB bağlantısı, dosya I/O)
⚠ UYARI: Singleton bağımlılıkları gizler. Logger::getInstance() kullanan fonksiyonların parametre listesinde görünmeyen gizli bağımlılığı vardır — DIP ihlali.

THEORY: Observer — Memory Leak & Lifetime Risks

Observer pattern has a dangerous trap: DANGLING OBSERVERS. If an observer is destroyed without unsubscribing from the subject, the subject holds a dangling pointer. Next notify() call → undefined behavior (crash, corruption). Solutions:
• Use weak_ptr instead of raw pointers in the observer list
• RAII-based subscription: destructor automatically unsubscribes
• Use std::function + token-based unsubscribe (return an ID on subscribe)
⚠ WARNING: The simple EventBus in 03_behavioral_patterns.cpp uses raw pointers — this is intentionally minimal for teaching. Production code MUST handle observer lifetime.

TEORİ: Observer — Bellek Sızıntısı ve Ömür Riskleri

Observer kalıbının tehlikeli tuzağı: SARKAN GÖZLEMCİLER. Bir gözlemci konudan aboneliğini iptal etmeden yok edilirse, konu sarkan işaretçi tutar. Sonraki notify() çağrısı → tanımsız davranış (çökme, bozulma). Çözümler:
• Gözlemci listesinde ham işaretçi yerine weak_ptr kullan
• RAII tabanlı abonelik: yıkıcı otomatik aboneliği iptal eder
• std::function + token tabanlı abonelik iptali (subscribe'da ID döndür)

THEORY: Decorator — Complexity Cost

Each Decorator layer adds a virtual function call and a unique_ptr indirection. Stacking 5+ decorators creates a deep chain that's hard to debug (stack traces become unreadable).
✓ USE when: you need 2-4 combinable behaviors at runtime
✗ DON'T USE when: all combinations are known at compile time (use templates/CRTP instead)
✗ DON'T USE when: you need to inspect the inner object (decorators obscure the wrapped type) Alternative: Policy-Based Design (see 05_testable_design.cpp) provides compile-time decoration with ZERO runtime overhead.

TEORİ: Decorator — Karmaşıklık Maliyeti

Her Decorator katmanı sanal fonksiyon çağrısı ve unique_ptr yönlendirmesi ekler. 5+ dekoratör yığmak, debug'ı zor derin zincir oluşturur (stack trace'ler okunamaz hale gelir).
✓ KULLAN: çalışma zamanında 2-4 birleştirilebilir davranış gerektiğinde
✗ KULLANMA: tüm kombinasyonlar derleme zamanında biliniyorsa (CRTP/template kullan)
✗ KULLANMA: iç nesneyi incelemeniz gerekiyorsa (dekoratörler sarılı tipi gizler)

THEORY: Factory Method vs Abstract Factory — When Which?

These two are the most confused pair: FACTORY METHOD: Creates ONE product. Uses inheritance — subclass overrides the factory method. The factory IS the base class itself.
→ Use when: you have a single product hierarchy and creation logic varies by subclass. ABSTRACT FACTORY: Creates a FAMILY of related products. Uses composition — the factory is a separate object injected into the client.
→ Use when: you need multiple related objects that must be from the same "theme" (e.g., WindowsButton + WindowsCheckbox, never WindowsButton + MacCheckbox). RULE OF THUMB: "If creating ONE thing → Factory Method. If creating a SET → Abstract Factory."

TEORİ: Fabrika Metodu vs Soyut Fabrika — Hangisi Ne Zaman?

Bu ikisi en çok karıştırılan çifttir: FABRİKA METODU: TEK ürün oluşturur. Kalıtım kullanır — alt sınıf fabrika metodunu override eder. Fabrika temel sınıfın KENDİSİDİR.
→ Kullan: tek ürün hiyerarşiniz varsa ve oluşturma mantığı alt sınıfa göre değişiyorsa. SOYUT FABRİKA: İLİŞKİLİ ürünler AİLESİ oluşturur. Bileşim kullanır — fabrika istemciye enjekte edilen ayrı bir nesnedir.
→ Kullan: aynı "tema"dan olması gereken birden fazla ilişkili nesne gerektiğinde. KURAL: "TEK şey oluşturuyorsan → Fabrika Metodu. SET oluşturuyorsan → Soyut Fabrika."

THEORY: Strategy vs State — The Subtle Difference

Both patterns look identical in structure (context + interface + concrete classes). The difference is INTENT: STRATEGY: The CLIENT chooses which algorithm to use. The object doesn't change state — the behavior is externally selected. Example: sorting algorithm selection. STATE: The OBJECT ITSELF changes its behavior based on internal transitions. The client just calls `next()` — the state decides the next state. Example: traffic light cycling. KEY TEST: "Does the object decide its own transitions?" → State. "Does the caller pick the behavior?" → Strategy.

TEORİ: Strategy vs State — İnce Fark

Her iki kalıp yapısal olarak aynı görünür (bağlam + arayüz + somut sınıflar). Fark NİYETTEDİR: STRATEGY: İSTEMCİ hangi algoritmayı kullanacağını seçer. Nesne durum değiştirmez — davranış dışarıdan seçilir. Örnek: sıralama algoritması seçimi. STATE: NESNENİN KENDİSİ iç geçişlere göre davranışını değiştirir. İstemci sadece `next()` çağırır — durum bir sonraki durumu belirler. Örnek: trafik ışığı döngüsü. TEST: "Nesne kendi geçişlerine mi karar veriyor?" → State. "Çağıran mı davranışı seçiyor?" → Strategy.

THEORY: Adapter vs Facade — Wrapping for Different Reasons

Both "wrap" something, but the purpose differs: ADAPTER: Makes ONE incompatible interface match ANOTHER existing interface. The client already has an interface it expects — the adapter translates.
→ "I have a square peg and a round hole — I need an adapter." FACADE: Simplifies a COMPLEX SUBSYSTEM into one easy-to-use interface. There's no pre-existing interface to match — you're creating a new simplified API.
→ "I have 10 subsystems — I want ONE startCluster() call." ADAPTER = interface translation. FACADE = interface simplification.

TEORİ: Adapter vs Facade — Farklı Amaçlarla Sarma

İkisi de "sarar" ama amaç farklıdır: ADAPTER: TEK uyumsuz arayüzü mevcut BAŞKA bir arayüzle eşleştirir. İstemci zaten beklediği bir arayüze sahiptir — adaptör çeviri yapar. FACADE: KARMAŞIK ALT SİSTEMİ kullanımı kolay tek bir arayüze basitleştirir. Eşleştirilecek önceden var arayüz yoktur — yeni basitleştirilmiş API oluşturursunuz. ADAPTER = arayüz çevirisi. FACADE = arayüz basitleştirmesi.

CPPREF Depth: std::unique_ptr and Prototype clone()

The Prototype pattern uses a `clone()` method that returns a deep copy. In modern C++, the return type should be `std::unique_ptr<Base>` to enforce clear ownership. The base class declares `virtual std::unique_ptr<Base> clone() const = 0;` and each derived class returns `std::make_unique<Derived>(*this)`. This relies on the copy constructor — ensure Rule of Five compliance if the class manages resources. Covariant return types don't work with smart pointers, so the return type must be the BASE type. cppreference.com/w/cpp/memory/unique_ptr

CPPREF Derinlik: std::unique_ptr ve Prototype clone()

Prototype kalıbı derin kopya döndüren bir `clone()` metodu kullanır. Modern C++'ta dönüş tipi net sahiplik için `std::unique_ptr<Temel>` olmalıdır. Temel sınıf `virtual std::unique_ptr<Temel> clone() const = 0;` bildirir ve her türetilmiş sınıf `std::make_unique<Türetilmiş>(*this)` döndürür. Bu kopyalama yapıcısına dayanır — sınıf kaynak yönetiyorsa Beşin Kuralı uyumunu sağlayın. Kovaryant dönüş tipleri akıllı işaretçilerle çalışmaz, bu nedenle dönüş tipi TEMEL tip olmalıdır.

CPPREF Depth: Proxy vs Smart Pointer — Overlapping Responsibility

C++'s smart pointers (unique_ptr, shared_ptr) are themselves a form of Proxy pattern! They intercept access to the raw pointer and add behavior (reference counting, automatic deletion). When implementing a custom Proxy, consider whether a smart pointer with a custom deleter already solves the problem. Custom Proxy is needed when you want: access control, lazy initialization, logging, caching, or remote communication (RPC/gRPC stub). cppreference.com/w/cpp/memory/shared_ptr

CPPREF Derinlik: Proxy vs Akıllı İşaretçi — Örtüşen Sorumluluk

C++'ın akıllı işaretçileri (unique_ptr, shared_ptr) kendileri bir Proxy kalıbı biçimidir! Ham işaretçiye erişimi yakalar ve davranış ekler (referans sayımı, otomatik silme). Özel Proxy uygularken, özel silici ile akıllı işaretçinin sorunu zaten çözüp çözmediğini düşünün. Özel Proxy şunlar için gereklidir: erişim kontrolü, tembel başlatma, loglama, önbellekleme veya uzak iletişim (RPC/gRPC stub).

Kaynak Kod / Source Code

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ==========================================================================================
// SECTION 1: PROTOTYPE PATTERN — Deep Cloning
// Eksik kalıp: Prototype (clone ile derin kopyalama)
// ==========================================================================================

// EN: Abstract base — all prototypes must implement clone().
// TR: Soyut temel — tüm prototipler clone() uygulamalıdır.
class VehicleConfig {
public:
  virtual ~VehicleConfig() = default;
  virtual std::unique_ptr<VehicleConfig> clone() const = 0;
  virtual void display() const = 0;
  virtual std::string type() const = 0;
};

// EN: Concrete prototype — full deep copy via copy constructor.
// TR: Somut prototip — kopyalama yapıcısı ile tam derin kopya.
class ElectricVehicleConfig : public VehicleConfig {
  std::string model_;
  int batteryCapacity_; // kWh
  bool hasAutopilot_;

public:
  ElectricVehicleConfig(std::string model, int battery, bool autopilot)
      : model_(std::move(model)), batteryCapacity_(battery),
        hasAutopilot_(autopilot) {}

  // EN: clone() returns unique_ptr<Base> — covariant returns don't work with smart ptrs.
  // TR: clone() unique_ptr<Temel> döndürür — kovaryant dönüş akıllı işaretçilerle çalışmaz.
  std::unique_ptr<VehicleConfig> clone() const override {
    return std::make_unique<ElectricVehicleConfig>(*this); // Uses copy ctor
  }

  void display() const override {
    std::cout << "  EV Config: " << model_ << " | " << batteryCapacity_
              << " kWh | Autopilot: " << (hasAutopilot_ ? "Yes" : "No")
              << std::endl;
  }

  std::string type() const override { return "ElectricVehicle"; }
};

class HybridVehicleConfig : public VehicleConfig {
  std::string model_;
  int engineCC_;
  int batteryCapacity_;

public:
  HybridVehicleConfig(std::string model, int cc, int battery)
      : model_(std::move(model)), engineCC_(cc), batteryCapacity_(battery) {}

  std::unique_ptr<VehicleConfig> clone() const override {
    return std::make_unique<HybridVehicleConfig>(*this);
  }
```

```cpp

  void display() const override {
    std::cout << "  Hybrid Config: " << model_ << " | " << engineCC_
              << "cc + " << batteryCapacity_ << " kWh" << std::endl;
  }

  std::string type() const override { return "HybridVehicle"; }
};

// EN: Prototype Registry — store templates, clone on demand.
// TR: Prototip Kaydı — şablonları depola, talep üzerine klonla.
class VehicleConfigRegistry {
  std::vector<std::pair<std::string, std::unique_ptr<VehicleConfig>>>
      prototypes_;

public:
  void registerPrototype(const std::string &name,
                         std::unique_ptr<VehicleConfig> proto) {
    prototypes_.emplace_back(name, std::move(proto));
  }

  std::unique_ptr<VehicleConfig> create(const std::string &name) const {
    for (const auto &[key, proto] : prototypes_) {
      if (key == name) return proto->clone();
    }
    return nullptr;
  }
};

// ==========================================================================================
// SECTION 2: PROXY PATTERN — Lazy Loading & Access Control
// Eksik kalıp: Proxy (tembel yükleme ve erişim kontrolü)
// ==========================================================================================

// EN: Subject interface — both real object and proxy implement this.
// TR: Konu arayüzü — hem gerçek nesne hem proxy bunu uygular.
class IDatabase {
public:
  virtual ~IDatabase() = default;
  virtual std::string query(const std::string &sql) = 0;
  virtual bool isConnected() const = 0;
};

// EN: Real (heavy) object — expensive to create (simulates DB connection).
// TR: Gerçek (ağır) nesne — oluşturması pahalı (DB bağlantısını simüle eder).
class RealDatabase : public IDatabase {
  std::string connectionString_;
  bool connected_ = false;

public:
  explicit RealDatabase(std::string connStr)
      : connectionString_(std::move(connStr)) {
    // EN: Simulates expensive connection setup.
    // TR: Pahalı bağlantı kurulumunu simüle eder.
    std::cout << "  [RealDB] Connecting to " << connectionString_
              << " (heavy operation)..." << std::endl;
    connected_ = true;
  }

  std::string query(const std::string &sql) override {
    return "[RealDB] Result for: " + sql;
  }

  bool isConnected() const override { return connected_; }
};
```

```cpp

// EN: Proxy — Lazy initialization: creates RealDatabase only on first query().
//     Also adds access control: denies queries if user role is insufficient.
// TR: Proxy — Tembel başlatma: RealDatabase'i yalnızca ilk query()'de oluşturur.
//     Ayrıca erişim kontrolü ekler: kullanıcı rolü yetersizse sorguları reddeder.
class DatabaseProxy : public IDatabase {
  std::string connectionString_;
  std::string userRole_;
  mutable std::unique_ptr<RealDatabase> realDb_; // Lazy

public:
  DatabaseProxy(std::string connStr, std::string role)
      : connectionString_(std::move(connStr)), userRole_(std::move(role)) {
    std::cout << "  [Proxy] Created (no DB connection yet — lazy)." << std::endl;
  }

  std::string query(const std::string &sql) override {
    // EN: Access control — only "admin" and "analyst" can query.
    // TR: Erişim kontrolü — yalnızca "admin" ve "analyst" sorgulayabilir.
    if (userRole_ != "admin" && userRole_ != "analyst") {
      return "[Proxy] ACCESS DENIED for role: " + userRole_;
    }

    // EN: Lazy initialization — create real DB only when needed.
    // TR: Tembel başlatma — gerçek DB yalnızca gerektiğinde oluşturulur.
    if (!realDb_) {
      std::cout << "  [Proxy] First query — initializing real database..."
                << std::endl;
      realDb_ = std::make_unique<RealDatabase>(connectionString_);
    }

    // EN: Logging (Proxy can also add logging, caching, metrics...).
    // TR: Loglama (Proxy loglama, önbellekleme, metrik de ekleyebilir...).
    std::cout << "  [Proxy] Forwarding query: " << sql << std::endl;
    return realDb_->query(sql);
  }

  bool isConnected() const override {
    return realDb_ && realDb_->isConnected();
  }
};

// ==========================================================================================
// SECTION 3: BEFORE/AFTER — Strategy Pattern Refactoring
// Öncesi/Sonrası — Strategy kalıbı ile refactoring
// ==========================================================================================

// EN: BEFORE (Bad) — behavior selected by if/else chain.
//     Adding a new discount type requires modifying this function (violates OCP).
// TR: ÖNCESİ (Kötü) — davranış if/else zinciriyle seçilir.
//     Yeni indirim tipi eklemek bu fonksiyonu değiştirmeyi gerektirir (OCP ihlali).
namespace before {
double applyDiscount(double price, const std::string &type) {
  if (type == "student") return price * 0.80;
  if (type == "military") return price * 0.75;
  if (type == "employee") return price * 0.70;
  // EN: Every new discount type → modify this function. Fragile!
  // TR: Her yeni indirim tipi → bu fonksiyonu değiştir. Kırılgan!
  return price;
}
} // namespace before

// EN: AFTER (Good) — Strategy pattern, open for extension, closed for modification.
// TR: SONRASI (İyi) — Strategy kalıbı, genişlemeye açık, değişikliğe kapalı.
namespace after {
class IDiscountStrategy {
public:
  virtual ~IDiscountStrategy() = default;
  virtual double apply(double price) const = 0;
  virtual std::string name() const = 0;
};
```

```cpp

class StudentDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.80; }
  std::string name() const override { return "Student (20% off)"; }
};

class MilitaryDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.75; }
  std::string name() const override { return "Military (25% off)"; }
};

class EmployeeDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.70; }
  std::string name() const override { return "Employee (30% off)"; }
};

// EN: Adding VIPDiscount requires ZERO modification of existing code — just a new class.
// TR: VIPDiscount eklemek mevcut kodda SIFIR değişiklik gerektirir — sadece yeni sınıf.
class VIPDiscount : public IDiscountStrategy {
public:
  double apply(double price) const override { return price * 0.60; }
  std::string name() const override { return "VIP (40% off)"; }
};

double applyDiscount(double price, const IDiscountStrategy &strategy) {
  return strategy.apply(price);
}
} // namespace after

// ==========================================================================================
// SECTION 4: BEFORE/AFTER — Observer with Safe Lifetime (weak_ptr)
// Öncesi/Sonrası — Güvenli ömürlü Observer (weak_ptr)
// ==========================================================================================

// EN: Safe Observer using weak_ptr — expired observers are automatically skipped.
// TR: weak_ptr kullanan güvenli Observer — süresi dolan gözlemciler otomatik atlanır.
class SafeObserver : public std::enable_shared_from_this<SafeObserver> {
public:
  virtual ~SafeObserver() = default;
  virtual void onEvent(const std::string &event) = 0;
};

class SafeEventBus {
  std::vector<std::weak_ptr<SafeObserver>> observers_;

public:
  void subscribe(std::shared_ptr<SafeObserver> obs) {
    observers_.push_back(obs); // Stores weak_ptr (won't prevent destruction)
  }

  void notify(const std::string &event) {
    // EN: Iterate and lock — skip expired observers, no dangling pointers.
    // TR: İtera et ve kilitle — süresi dolan gözlemcileri atla, sarkan işaretçi yok.
    auto it = observers_.begin();
    while (it != observers_.end()) {
      if (auto locked = it->lock()) {
        locked->onEvent(event);
        ++it;
      } else {
        // EN: Observer was destroyed — remove from list.
        // TR: Gözlemci yok edildi — listeden kaldır.
        it = observers_.erase(it);
      }
    }
  }

  size_t subscriberCount() const { return observers_.size(); }
};
```

```cpp

class DashboardWidget : public SafeObserver {
  std::string name_;

public:
  explicit DashboardWidget(std::string n) : name_(std::move(n)) {}
  void onEvent(const std::string &event) override {
    std::cout << "  [" << name_ << "] Received: " << event << std::endl;
  }
  ~DashboardWidget() override {
    std::cout << "  [" << name_ << "] Destroyed (auto-unsubscribed)."
              << std::endl;
  }
};

// ==========================================================================================
// SECTION 5: QUICK REFERENCE — Pattern Selection Cheat Sheet
// Hızlı Referans — Kalıp Seçim Tablosu
// ==========================================================================================

//   ┌──────────────────────┬──────────────────────────────────────────────────────┐
//   │ Problem              │ Pattern to Use                                       │
//   ├──────────────────────┼──────────────────────────────────────────────────────┤
//   │ One global instance  │ Singleton (prefer DI / Dependency Injection first!) │
//   │ Hide concrete type   │ Factory Method                                       │
//   │ Family of objects    │ Abstract Factory                                     │
//   │ Complex constructor  │ Builder                                              │
//   │ Clone existing obj   │ Prototype                                            │
//   │ Wrap legacy API      │ Adapter                                              │
//   │ Simplify subsystem   │ Facade                                               │
//   │ Stack behaviors      │ Decorator                                            │
//   │ Tree structure       │ Composite                                            │
//   │ Lazy/access control  │ Proxy                                                │
//   │ Event notification   │ Observer                                             │
//   │ Swappable algorithm  │ Strategy                                             │
//   │ Undo/redo actions    │ Command                                              │
//   │ State machine        │ State                                                │
//   ├──────────────────────┼──────────────────────────────────────────────────────┤
//   │ Compile-time strat.  │ Policy-Based Design / CRTP (see 05_testable)         │
//   │ Type erasure         │ std::any / hand-rolled (see 05_testable)             │
//   └──────────────────────┴──────────────────────────────────────────────────────┘

// ==========================================================================================
// MAIN — Demonstrations
// ==========================================================================================

int main() {
  std::cout << "=== MODULE 10: PATTERN DECISION GUIDE ===\n" << std::endl;

  // ── 1. Prototype ──
  std::cout << "--- PROTOTYPE (Clone & Registry) ---\n" << std::endl;

  VehicleConfigRegistry registry;
  registry.registerPrototype(
      "ev_standard",
      std::make_unique<ElectricVehicleConfig>("Model S", 100, true));
  registry.registerPrototype(
      "hybrid_base",
      std::make_unique<HybridVehicleConfig>("Camry Hybrid", 2500, 18));
```

```cpp

    // EN: Clone from registry — no knowledge of concrete types needed.
    // TR: Kayıttan klonla — somut tip bilgisi gerekmez.
    auto car1 = registry.create("ev_standard");
    auto car2 = registry.create("ev_standard"); // Another clone of the same template
    auto car3 = registry.create("hybrid_base");

    std::cout << "Cloned configs:" << std::endl;
    if (car1) car1->display();
    if (car2) car2->display();
    if (car3) car3->display();
    std::cout << std::endl;

    // ── 2. Proxy ──
    std::cout << "--- PROXY (Lazy Init + Access Control) ---\n" << std::endl;

    // EN: Proxy created — no DB connection yet (lazy).
    // TR: Proxy oluşturuldu — henüz DB bağlantısı yok (tembel).
    DatabaseProxy adminDb("postgresql://prod:5432", "admin");
    DatabaseProxy guestDb("postgresql://prod:5432", "guest");

    std::cout << "\nAdmin queries:" << std::endl;
    std::cout << adminDb.query("SELECT * FROM sensors") << std::endl;
    std::cout << adminDb.query("SELECT * FROM diagnostics") << std::endl;

    std::cout << "\nGuest tries to query:" << std::endl;
    std::cout << guestDb.query("SELECT * FROM secrets") << std::endl;
    std::cout << std::endl;

    // ── 3. Before/After — Strategy ──
    std::cout << "--- BEFORE/AFTER: Strategy Refactoring ---\n" << std::endl;

    double price = 100.0;

    // BEFORE: if/else chain
    std::cout << "BEFORE (if/else): student=" << before::applyDiscount(price, "student")
              << ", military=" << before::applyDiscount(price, "military") << std::endl;

    // AFTER: Strategy pattern — new VIPDiscount added with ZERO modification
    after::StudentDiscount student;
    after::MilitaryDiscount military;
    after::VIPDiscount vip; // NEW type — no existing code changed

    std::cout << "AFTER  (Strategy): " << student.name() << "="
              << after::applyDiscount(price, student)
              << ", " << military.name() << "="
              << after::applyDiscount(price, military)
              << ", " << vip.name() << "="
              << after::applyDiscount(price, vip) << std::endl;
    std::cout << std::endl;

    // ── 4. Safe Observer (weak_ptr) ──
    std::cout << "--- SAFE OBSERVER (weak_ptr lifetime) ---\n" << std::endl;

    SafeEventBus safeBus;
    auto speedometer = std::make_shared<DashboardWidget>("Speedometer");
    auto tachometer = std::make_shared<DashboardWidget>("Tachometer");

    safeBus.subscribe(speedometer);
    safeBus.subscribe(tachometer);

    std::cout << "Subscribers: " << safeBus.subscriberCount() << std::endl;
    safeBus.notify("SPEED_UPDATE: 120 km/h");
```

```

    // EN: Destroy tachometer — it auto-unsubscribes (weak_ptr expires).
    // TR: Tachometer'ı yok et — otomatik abonelik iptali (weak_ptr süresi dolar).
    std::cout << "\nDestroying tachometer..." << std::endl;
    tachometer.reset();

    std::cout << "Notifying after destruction:" << std::endl;
    safeBus.notify("SPEED_UPDATE: 80 km/h");
    std::cout << "Subscribers after cleanup: " << safeBus.subscriberCount()
              << std::endl;

    // ── 5. Decision Summary ──
    std::cout << "\n--- PATTERN SELECTION SUMMARY ---\n" << std::endl;
    std::cout << "  Factory Method vs Abstract Factory: Single product vs Product family"
              << std::endl;
    std::cout << "  Strategy vs State: Caller picks vs Object transitions internally"
              << std::endl;
    std::cout << "  Adapter vs Facade: Interface translation vs Interface simplification"
              << std::endl;
    std::cout << "  Decorator vs CRTP: Runtime stacking vs Compile-time composition"
              << std::endl;
    std::cout << "  Singleton: Think twice — prefer Dependency Injection!" << std::endl;

    std::cout << "\n=> Pattern decision guide complete!" << std::endl;

    return 0;
}
```

Modül 11: Embedded & Real-Time — Gömülü Sistemler

Gömülü Sistemler

5 kaynak dosya · Standart: C++20

C++20

► 01. Volatile Hardware

module_11_embedded_systems/01_volatile_hardware.cpp

Konu: Embedded & Systems: volatile Keyword, Memory-Mapped I/O, ISR Concepts / volatile

Teori / Theory

THEORY: What is volatile?

The `volatile` qualifier tells the compiler: "Do NOT optimize reads or writes to this variable — its value can change at ANY time without the program's knowledge." This happens in three critical situations: 1) Memory-mapped hardware registers (GPIO pins, timers, ADC results) 2) Variables modified by Interrupt Service Routines (ISR) 3) Variables shared with DMA controllers or external hardware Without `volatile`, the compiler may cache the value in a CPU register and never re-read it from RAM — missing hardware changes entirely.

TEORİ: volatile Nedir?

`volatile` niteleyicisi derleyiciye şunu söyler: "Bu değişkene yapılan okumaları veya yazmaları OPTİMİZE ETME — değeri programın bilgisi dışında HERHANGİ bir anda değişebilir." Bu üç kritik durumda olur: 1) Bellek eşlemeli donanım kayıtları (GPIO pinleri, zamanlayıcılar, ADC) 2) Kesme Hizmet Rutinleri (ISR) tarafından değiştirilen değişkenler 3) DMA denetleyicileri veya harici donanım ile paylaşılan değişkenler `volatile` olmadan derleyici değeri CPU kaydında önbelleğe alabilir ve RAM'den asla tekrar okumayabilir — donanım değişikliklerini tamamen kaçırır.

THEORY: Memory-Mapped I/O — Hardware as Memory Addresses

In embedded systems, hardware peripherals are controlled by reading from and writing to specific MEMORY ADDRESSES. Writing `0x01` to address `0x40021000` might enable a GPIO clock. The CPU bus routes these addresses directly to hardware registers instead of RAM. These addresses are always accessed through `volatile` pointers to prevent compiler optimization.

TEORİ: Bellek Eşlemeli G/Ç — Bellek Adresi Olarak Donanım

Gömülü sistemlerde donanım çevre birimleri, belirli BELLEK ADRESLERİNE okuma ve yazma yaparak kontrol edilir. `0x40021000` adresine `0x01` yazmak bir GPIO saatini etkinleştirebilir. CPU veri yolu bu adresleri RAM yerine doğrudan donanım kayıtlarına yönlendirir. Bu adresler, derleyici optimizasyonunu önlemek için her zaman `volatile` işaretçiler aracılığıyla erişilir.

THEORY: Interrupt Service Routines (ISR)

An ISR is a function called by the hardware (CPU interrupt controller) when an external event occurs (button press, timer overflow, UART data received). ISRs preempt the main program at any instruction boundary. ISR rules: (1) Keep it SHORT, (2) No heap allocation, (3) No blocking calls, (4) Use `volatile` for shared variables, (5) Consider atomic types for multi-byte values.

TEORİ: Kesme Hizmet Rutinleri

ISR, harici bir olay gerçekleştiğinde (düğme basma, zamanlayıcı taşması, UART veri alma) donanım (CPU kesme denetleyicisi) tarafından çağrılan fonksiyondur. ISR'lar ana programı herhangi bir komut sınırında keser. ISR kuralları: (1) KISA tut, (2) Heap tahsisi yok, (3) Blokleyici çağrı yok, (4) Paylaşılan değişkenler için `volatile` kullan, (5) Çok baytlı değerler için atomik tipler düşün.

CPPREF Depth: volatile Does NOT Mean Thread-Safe!

A critical misconception: `volatile` in C++ does NOT provide atomicity or memory ordering guarantees. It prevents compiler optimization but says NOTHING about CPU cache coherency or instruction reordering. For multi-threaded code, use `std::atomic<T>` instead. `volatile` is ONLY for hardware I/O and ISR communication. In Java/C#, `volatile` has different (stronger) semantics — don't confuse them! cppreference.com/w/cpp/language/cv

CPPREF Derinlik: volatile Thread-Güvenli Demek DEĞİLDİR!

Kritik bir yanlış anlama: C++'ta `volatile` atomiklik veya bellek sıralama garantisi SAĞLAMAZ. Derleyici optimizasyonunu önler ama CPU önbellek tutarlılığı veya komut yeniden sıralama hakkında HİÇBİR ŞEY söylemez. Çoklu-thread kod için bunun yerine `std::atomic<T>` kullanın. `volatile` YALNIZCA donanım G/Ç ve ISR iletişimi içindir. Java/C#'ta `volatile` farklı (daha güçlü) anlamlara sahiptir — karıştırmayın! cppreference.com/w/cpp/language/cv

CPPREF Depth: volatile and const Can Coexist

`const volatile int* reg = (const volatile int*)0x40000000;` means: "I promise not to WRITE to this register, but its value can still CHANGE due to hardware." This is common for read-only status registers (e.g., ADC result, timer counter). The compiler won't optimize away reads, and the programmer cannot accidentally write to it. cppreference.com/w/cpp/language/cv

CPPREF Derinlik: volatile ve const Bir Arada Olabilir

`const volatile int* reg = (const volatile int*)0x40000000;` şu anlama gelir: "Bu kayda YAZMA sözü veriyorum, ama değeri donanım nedeniyle hâlâ DEĞİŞEBİLİR." Bu, salt okunur durum kayıtları (ADC sonucu, zamanlayıcı sayacı) için yaygındır. Derleyici okumaları optimize etmez ve programcı yanlışlıkla yazamaz. cppreference.com/w/cpp/language/cv

Kaynak Kod / Source Code

```cpp
#include <cstdint>
#include <iostream>

// ==========================================================================================
// [DEMO 1: volatile Prevents Compiler Optimization]
// ==========================================================================================

// EN: Without volatile, the compiler could optimize this to "while(true)" because it sees no
// code path that changes 'flag'.
// TR: volatile olmadan derleyici bunu "while(true)" olarak optimize edebilir çünkü 'flag'ı
// değiştiren hiçbir kod yolu görmez.
volatile bool hardwareFlag = false;

void simulateISR() {
  // EN: Simulated ISR — in real embedded, this would be triggered by hardware.
  // TR: Simüle ISR — gerçek gömülüde bu donanım tarafından tetiklenirdi.
  hardwareFlag = true;
  std::cout << "[ISR] Hardware flag SET to true." << std::endl;
}

// ==========================================================================================
// [DEMO 2: Memory-Mapped I/O Simulation]
// ==========================================================================================

// EN: Simulate memory-mapped hardware registers.
// TR: Bellek eşlemeli donanım kayıtlarını simüle et.
struct GPIO_Registers {
  volatile uint32_t MODER;   // Mode register
  volatile uint32_t ODR;     // Output data register
  volatile uint32_t IDR;     // Input data register (read-only in real HW)
  volatile uint32_t BSRR;   // Bit set/reset register
};

// EN: In real embedded, this would be a hardware address like 0x40020000.
// TR: Gerçek gömülüde bu 0x40020000 gibi bir donanım adresi olurdu.
GPIO_Registers simulated_gpio = {0, 0, 0, 0};

void configureGPIO() {
  // EN: Set pin 5 to output mode (bits 11:10 = 01 for output).
  // TR: Pin 5'i çıkış moduna ayarla (bitler 11:10 = 01 çıkış için).
  simulated_gpio.MODER |= (1U << 10);
  simulated_gpio.MODER &= ~(1U << 11);
  std::cout << "[GPIO] Pin 5 configured as OUTPUT. MODER=0x"
            << std::hex << simulated_gpio.MODER << std::dec << std::endl;
}

void setGPIOPin(uint32_t pin) {
  // EN: BSRR: writing to lower 16 bits SETS, upper 16 bits RESETS.
  // TR: BSRR: alt 16 bite yazma AYARLAR, üst 16 bit SIFIRLAR.
  simulated_gpio.BSRR = (1U << pin);
  simulated_gpio.ODR |= (1U << pin);
  std::cout << "[GPIO] Pin " << pin << " SET HIGH. ODR=0x"
            << std::hex << simulated_gpio.ODR << std::dec << std::endl;
}

void resetGPIOPin(uint32_t pin) {
  simulated_gpio.BSRR = (1U << (pin + 16U));
  simulated_gpio.ODR &= ~(1U << pin);
  std::cout << "[GPIO] Pin " << pin << " SET LOW. ODR=0x"
            << std::hex << simulated_gpio.ODR << std::dec << std::endl;
}
```

```cpp

// ==========================================================================================
// [DEMO 3: ISR-Safe Shared Variable Pattern]
// ==========================================================================================

// EN: volatile + flag pattern for ISR-main communication.
// TR: ISR-ana program iletişimi için volatile + bayrak kalıbı.
volatile uint32_t adcResult = 0;
volatile bool adcReady = false;

void simulateADC_ISR() {
  // EN: Simulates ADC conversion complete interrupt.
  // TR: ADC dönüşüm tamamlandı kesmesini simüle eder.
  adcResult = 3300; // 3.3V in millivolts
  adcReady = true;
  std::cout << "[ADC ISR] Conversion complete: " << adcResult << " mV"
            << std::endl;
}

// ==========================================================================================
// [DEMO 4: const volatile — Read-Only Hardware Register]
// ==========================================================================================

// EN: Simulating a read-only hardware status register.
// TR: Salt okunur donanım durum kaydını simüle etme.
volatile uint32_t hwStatusReg = 0x0000'00FF;

int main() {
  std::cout << "=== MODULE 11: VOLATILE & HARDWARE I/O ===\n" << std::endl;

  // --- Demo 1: volatile flag ---
  std::cout << "--- DEMO 1: volatile Flag ---\n" << std::endl;
  std::cout << "[Main] Flag before ISR: " << std::boolalpha << hardwareFlag
            << std::endl;
  simulateISR();
  if (hardwareFlag) {
    std::cout << "[Main] Flag detected as TRUE — ISR communication works!\n"
              << std::endl;
  }

  // --- Demo 2: Memory-Mapped GPIO ---
  std::cout << "--- DEMO 2: Memory-Mapped GPIO Simulation ---\n" << std::endl;
  configureGPIO();
  setGPIOPin(5);  // Turn on LED
  setGPIOPin(13); // Turn on buzzer
  resetGPIOPin(5); // Turn off LED
  std::cout << std::endl;

  // --- Demo 3: ISR-Safe ADC Pattern ---
  std::cout << "--- DEMO 3: ISR-Safe ADC Read ---\n" << std::endl;
  std::cout << "[Main] Waiting for ADC..." << std::endl;
  simulateADC_ISR(); // In real code, this happens asynchronously
  if (adcReady) {
    uint32_t voltage = adcResult;
    adcReady = false;
    std::cout << "[Main] Read ADC: " << voltage << " mV (" << (voltage / 1000.0)
              << " V)\n"
              << std::endl;
  }

  // --- Demo 4: const volatile ---
  std::cout << "--- DEMO 4: const volatile (Read-Only Register) ---\n"
            << std::endl;
  const volatile uint32_t *statusReg = &hwStatusReg;
  // *statusReg = 0; // COMPILE ERROR: cannot write to const volatile!
  std::cout << "[Main] Status register value: 0x" << std::hex << *statusReg
            << std::dec << std::endl;
  std::cout << "=> const volatile: compiler re-reads every time, but "
               "writing is forbidden at compile time."
            << std::endl;

  std::cout << "\n=> volatile, Memory-Mapped I/O, and ISR patterns "
               "demonstrated!" << std::endl;

  return 0;
}
```

► 02. Bitfields Protocols

module_11_embedded_systems/02_bitfields_protocols.cpp

Konu: Embedded & Systems: Bitfields, Protocol Parsing, Packed Structs / Bit Alanları,

Teori / Theory

THEORY: Bitfields — Packing Data into Minimal Space

C++ bitfields allow struct members to occupy a specific number of BITS instead of whole bytes. `uint8_t flag : 1;` uses exactly 1 bit. This is critical in embedded systems where every byte matters: network protocol headers, hardware register layouts, command packets over CAN/SPI/I2C buses. A 32-bit register might contain 8 different fields packed into one word.

TEORİ: Bit Alanları — Veriyi Minimum Alana Paketleme

C++ bit alanları, yapı üyelerinin tam baytlar yerine belirli sayıda BİT kaplamasına izin verir. `uint8_t flag : 1;` tam olarak 1 bit kullanır. Bu, her baytın önemli olduğu gömülü sistemlerde kritiktir: ağ protokolü başlıkları, donanım kayıt düzenleri, CAN/SPI/I2C veri yolları üzerinden komut paketleri. Bir 32-bit kayıt, tek bir kelimeye paketlenmiş 8 farklı alan içerebilir.

THEORY: Protocol Parsing with Structs

Network protocols (TCP/IP, CAN, USB) define exact byte layouts for packets. In C++, we can overlay a struct onto raw byte buffers to parse them directly — zero-copy parsing! The struct fields map directly to protocol fields. This is dramatically faster than parsing byte-by-byte. Requirements: `#pragma pack(1)` or `__attribute__((packed))` to disable compiler padding.

TEORİ: Yapılarla Protokol Ayrıştırma

Ağ protokolleri (TCP/IP, CAN, USB) paketler için kesin bayt düzenleri tanımlar. C++'ta ham bayt tamponlarına bir yapı bindirerek bunları doğrudan ayrıştırabiliriz — sıfır-kopya ayrıştırma! Yapı alanları doğrudan protokol alanlarına eşlenir. Bu, bayt bayt ayrıştırmadan dramatik şekilde hızlıdır. Gereksinimler: derleyici dolgusunu devre dışı bırakmak için `#pragma pack(1)` veya `__attribute__((packed))`.

THEORY: Endianness — Byte Order Matters

CPUs store multi-byte values in either "Little-Endian" (LSB first, x86, ARM) or "Big-Endian" (MSB first, network order). When parsing network protocols, you MUST convert between network byte order (big-endian) and host byte order using `htons()/ntohs()` (16-bit) and `htonl()/ntohl()` (32-bit). Ignoring endianness corrupts multi-byte fields across platforms.

TEORİ: Endianness — Bayt Sırası Önemlidir

CPU'lar çok baytlı değerleri "Little-Endian" (LSB önce, x86, ARM) veya "Big-Endian" (MSB önce, ağ sırası) olarak depolar. Ağ protokollerini ayrıştırırken, ağ bayt sırası (big-endian) ile ana makine bayt sırası arasında `htons()/ntohs()` (16-bit) ve `htonl()/ntohl()` (32-bit) kullanarak dönüştürmeniz ZORUNLUDUR. Endianness'ı görmezden gelmek platformlar arasında çok baytlı alanları bozar.

CPPREF Depth: Implementation-Defined Behavior of Bitfields

The C++ standard leaves several bitfield aspects implementation-defined: (1) Layout ordering (MSB-first or LSB-first within a byte), (2) Whether bitfields can span storage unit boundaries, (3) Alignment of bitfield storage units. This means bitfield layout is NOT portable across compilers! For cross-platform protocol parsing, prefer explicit bit manipulation (`& mask`, `>> shift`) over bitfields. cppreference.com/w/cpp/language/bit_field

CPPREF Derinlik: Bit Alanlarının Uygulamaya Bağlı Davranışı

C++ standardı birçok bit alanı yönünü uygulamaya bağlı bırakır: (1) Düzen sıralaması (bir bayt içinde MSB-önce veya LSB-önce), (2) Bit alanlarının depolama birimi sınırlarını geçip geçemeyeceği, (3) Bit alanı depolama birimlerinin hizalaması. Bu, bit alanı düzeninin derleyiciler arasında TAŞINABİLİR OLMADIĞI anlamına gelir! Platformlar arası protokol ayrıştırma için bit alanları yerine açık bit manipülasyonu (`& maske`, `>> kaydırma`) tercih edin. cppreference.com/w/cpp/language/bit_field

CPPREF Depth: std::byte and Type-Safe Byte Manipulation (C++17)

C++17 introduced `std::byte` as a distinct type for raw byte manipulation. Unlike `char` or `unsigned char`, `std::byte` only supports bitwise operations — no arithmetic! This prevents accidentally treating raw bytes as numbers. `std::byte b{0xFF}; auto v = std::to_integer<uint8_t>(b);` cppreference.com/w/cpp/types/byte

CPPREF Derinlik: std::byte ve Tip-Güvenli Bayt Manipülasyonu (C++17)

C++17, ham bayt manipülasyonu için ayrı bir tip olarak `std::byte`'ı tanıttı. `char` veya `unsigned char`'dan farklı olarak `std::byte` yalnızca bitsel işlemleri destekler — aritmetik yok! Bu, ham baytların yanlışlıkla sayı olarak işlenmesini önler. `std::byte b{0xFF}; auto v = std::to_integer<uint8_t>(b);` cppreference.com/w/cpp/types/byte

Kaynak Kod / Source Code

```cpp
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <iostream>

// ==========================================================================================
// [1. BITFIELD DEMO — Hardware Register Layout]
// ==========================================================================================

// EN: Simulates a UART control register with bitfields.
// TR: Bit alanlarıyla bir UART kontrol kaydını simüle eder.
struct UART_CR {
  uint32_t UE : 1;     // UART Enable
  uint32_t RE : 1;     // Receiver Enable
  uint32_t TE : 1;     // Transmitter Enable
  uint32_t IDLEIE : 1; // IDLE Interrupt Enable
  uint32_t RXNEIE : 1; // RX Not Empty Interrupt Enable
  uint32_t TCIE : 1;   // Transmission Complete Interrupt
  uint32_t TXEIE : 1;  // TX Empty Interrupt
  uint32_t PEIE : 1;   // Parity Error Interrupt
  uint32_t PS : 1;     // Parity Selection (0=Even, 1=Odd)
  uint32_t PCE : 1;    // Parity Control Enable
  uint32_t reserved : 22; // Remaining bits
};
```

```cpp

// ==========================================================================================
// [2. PACKED STRUCT — Protocol Header (CAN Bus Frame)]
// ==========================================================================================

// EN: #pragma pack(1) prevents compiler from inserting padding bytes.
// TR: #pragma pack(1) derleyicinin dolgu baytları eklemesini önler.
#pragma pack(push, 1)
struct CANFrame {
  uint32_t id : 11;     // CAN Standard ID (11 bits)
  uint32_t rtr : 1;     // Remote Transmission Request
  uint32_t ide : 1;     // Identifier Extension
  uint32_t dlc : 4;     // Data Length Code (0-8)
  uint32_t reserved : 15;
  uint8_t data[8];      // Payload (max 8 bytes)
};
#pragma pack(pop)

// ==========================================================================================
// [3. MANUAL BIT MANIPULATION — Portable Protocol Parsing]
// ==========================================================================================

// EN: Explicit bit manipulation is portable across all compilers.
// TR: Açık bit manipülasyonu tüm derleyicilerde taşınabilirdir.
struct SensorPacket {
  uint8_t raw[4]; // 4 bytes of raw data

  // EN: Extract sensor ID (bits 31:24 of first word)
  // TR: Sensör kimliği çıkar (ilk kelimenin bitleri 31:24)
  uint8_t getSensorId() const { return raw[0]; }

  // EN: Extract temperature (bits 23:12, 12-bit signed value)
  // TR: Sıcaklık çıkar (bitler 23:12, 12-bit işaretli değer)
  int16_t getTemperature() const {
    uint16_t val =
        static_cast<uint16_t>((static_cast<uint16_t>(raw[1]) << 4) |
                              (static_cast<uint16_t>(raw[2]) >> 4));
    // EN: Sign extend 12-bit to 16-bit
    // TR: 12-bit'i 16-bit'e işaret genişlet
    if (val & 0x800U) val |= 0xF000U;
    return static_cast<int16_t>(val);
  }

  // EN: Extract battery level (bits 11:4, 8-bit value, 0-255)
  // TR: Pil seviyesi çıkar (bitler 11:4, 8-bit değer, 0-255)
  uint8_t getBatteryLevel() const {
    return static_cast<uint8_t>(((raw[2] & 0x0FU) << 4) |
                                (raw[3] >> 4));
  }

  // EN: Extract alert flags (bits 3:0)
  // TR: Uyarı bayrakları çıkar (bitler 3:0)
  uint8_t getAlertFlags() const {
    return static_cast<uint8_t>(raw[3] & 0x0FU);
  }
};

// ==========================================================================================
// [4. ENDIANNESS DEMONSTRATION]
// ==========================================================================================

bool isLittleEndian() {
  uint16_t val = 0x0001;
  // EN: If first byte is 0x01, the LSB is stored first (little-endian).
  // TR: İlk bayt 0x01 ise, LSB önce depolanır (little-endian).
  uint8_t firstByte = 0;
  std::memcpy(&firstByte, &val, 1);
  return firstByte == 0x01;
}

uint16_t swapBytes16(uint16_t val) {
  return static_cast<uint16_t>((val >> 8) | (val << 8));
}
```

```cpp

uint32_t swapBytes32(uint32_t val) {
  return ((val >> 24) & 0xFFU) | ((val >> 8) & 0xFF00U) |
         ((val << 8) & 0xFF0000U) | ((val << 24) & 0xFF000000U);
}

int main() {
  std::cout << "=== MODULE 11: BITFIELDS & PROTOCOL PARSING ===\n" << std::endl;

  // --- Demo 1: Bitfield Register ---
  std::cout << "--- DEMO 1: UART Control Register (Bitfield) ---\n" << std::endl;
  UART_CR uart{};
  uart.UE = 1;    // Enable UART
  uart.RE = 1;    // Enable receiver
  uart.TE = 1;    // Enable transmitter
  uart.RXNEIE = 1; // Enable RX interrupt

  std::cout << "UART Enable:    " << uart.UE << std::endl;
  std::cout << "Receiver:       " << uart.RE << std::endl;
  std::cout << "Transmitter:    " << uart.TE << std::endl;
  std::cout << "RX Interrupt:   " << uart.RXNEIE << std::endl;
  std::cout << "sizeof(UART_CR)=" << sizeof(UART_CR) << " bytes\n" << std::endl;

  // --- Demo 2: CAN Frame ---
  std::cout << "--- DEMO 2: CAN Bus Frame (Packed Struct) ---\n" << std::endl;
  CANFrame frame{};
  frame.id = 0x123;    // Standard CAN ID
  frame.rtr = 0;       // Data frame (not remote)
  frame.ide = 0;       // Standard (not extended)
  frame.dlc = 3;       // 3 bytes of data
  frame.data[0] = 0xAA;
  frame.data[1] = 0xBB;
  frame.data[2] = 0xCC;

  std::cout << "CAN ID:   0x" << std::hex << frame.id << std::endl;
  std::cout << "DLC:      " << std::dec << frame.dlc << std::endl;
  std::cout << "Data:     ";
  for (int i = 0; i < frame.dlc; i++) {
    std::cout << "0x" << std::hex << std::setw(2) << std::setfill('0')
              << static_cast<int>(frame.data[i]) << " ";
  }
  std::cout << std::dec << "\nsizeof(CANFrame)=" << sizeof(CANFrame)
            << " bytes\n"
            << std::endl;

  // --- Demo 3: Manual Bit Parsing ---
  std::cout << "--- DEMO 3: Portable Bit Manipulation ---\n" << std::endl;
  SensorPacket packet{};
  // EN: Sensor ID=0x42, Temp=+25 (0x019), Battery=200, Alerts=0x05
  // TR: Sensör ID=0x42, Sıcaklık=+25 (0x019), Pil=200, Uyarılar=0x05
  packet.raw[0] = 0x42;
  packet.raw[1] = 0x01;
  packet.raw[2] = 0x9C;
  packet.raw[3] = 0x85;

  std::cout << "Sensor ID:     0x" << std::hex
            << static_cast<int>(packet.getSensorId()) << std::dec << std::endl;
  std::cout << "Temperature:   " << packet.getTemperature() << " C" << std::endl;
  std::cout << "Battery:       " << static_cast<int>(packet.getBatteryLevel())
            << "/255" << std::endl;
  std::cout << "Alert Flags:   0x" << std::hex
            << static_cast<int>(packet.getAlertFlags()) << std::dec << "\n"
            << std::endl;
```

```

    // --- Demo 4: Endianness ---
    std::cout << "--- DEMO 4: Endianness ---\n" << std::endl;
    std::cout << "System is: "
              << (isLittleEndian() ? "Little-Endian" : "Big-Endian") << std::endl;
    uint16_t val16 = 0x1234;
    uint32_t val32 = 0xDEADBEEF;
    std::cout << "0x" << std::hex << val16 << " swapped = 0x"
              << swapBytes16(val16) << std::endl;
    std::cout << "0x" << val32 << " swapped = 0x" << swapBytes32(val32)
              << std::dec << std::endl;

    std::cout << "\n=> Bitfields, packed structs, protocol parsing, and "
                 "endianness demonstrated!" << std::endl;

    return 0;
  }
```

► 03. Embedded Constraints

module_11_embedded_systems/03_embedded_constraints.cpp

Konu: Embedded & Systems: No-Heap Allocation, Static Buffers, constexpr, Placement New /

Teori / Theory

THEORY: No-Heap Programming — Why malloc/new Are Banned

In safety-critical embedded systems (automotive, medical, aerospace), dynamic memory allocation (malloc/new) is BANNED because: (1) Heap fragmentation can exhaust memory unpredictably. (2) Allocation time is non-deterministic — bad for real-time systems. (3) Heap metadata overhead wastes scarce RAM. (4) Memory leaks in long-running firmware are catastrophic. Instead, all memory is statically allocated at compile time using fixed-size arrays, stack variables, and memory pools (placement new).

TEORİ: Yığınsız Programlama — malloc/new Neden Yasaklıdır

Güvenlik-kritik gömülü sistemlerde (otomotiv, tıbbi, havacılık) dinamik bellek tahsisi (malloc/new) YASAKLIDIR çünkü: (1) Yığın parçalanması belleği öngörülemez şekilde tüketebilir. (2) Tahsis süresi belirleyici değildir — gerçek zamanlı sistemler icin kötü. (3) Yığın meta veri ek yükü kıt RAM'i israf eder. (4) Uzun süre çalışan donanım yazılımında bellek sızıntıları felakettir. Bunun yerine, tüm bellek derleme zamanında sabit boyutlu diziler, yığıt değişkenleri ve bellek havuzları (placement new) kullanılarak statik olarak tahsis edilir.

THEORY: constexpr — Compile-Time Computation

`constexpr` forces computation to happen at compile time, producing zero runtime overhead. In embedded, this means: - CRC/checksum lookup tables computed at compile time. - Baud rate divisor calculations resolved before flashing. - Trigonometric lookup tables baked into firmware ROM. C++17 allows `constexpr if` for compile-time branching, and C++20 allows `consteval` to GUARANTEE compile-time-only execution.

TEORİ: constexpr — Derleme Zamanı Hesaplama

`constexpr` hesaplamayı derleme zamanında gerçekleşmeye zorlar, sıfır çalışma zamanı ek yükü üretir. Gömülü sistemlerde bu şu anlama gelir: - CRC/sağlama toplamı arama tabloları derleme zamanında hesaplanır. - Baud hızı bölücü hesaplamaları flaşlamadan önce çözümlenir. - Trigonometrik arama tabloları donanım yazılımı ROM'una gömülür. C++17 derleme zamanı dallanma için `constexpr if`'e izin verir ve C++20 yalnızca derleme zamanı yürütmeyi GARANTİ etmek için `consteval`'e izin verir.

THEORY: Placement New — Object Construction in Pre-Allocated Memory

`placement new` constructs an object at a specific memory address without allocating. Syntax: `new (address) Type(args);` This is essential for: (1) Memory pools with fixed-size blocks. (2) DMA buffer management. (3) Constructing objects on stack-allocated aligned buffers. You must call the destructor manually: `obj->~Type();`

TEORİ: Placement New — Önceden Tahsis Edilmiş Bellekte Nesne Oluşturma

`placement new` tahsis yapmadan belirli bir bellek adresinde nesne oluşturur. Sözdizimi: `new (adres) Tip(argümanlar);` Bu, şunlar için gereklidir: (1) Sabit boyutlu bloklara sahip bellek havuzları. (2) DMA tampon yönetimi. (3) Yığıt-tahsisli hizalanmış tamponlarda nesne oluşturma. Yıkıcıyı manuel olarak çağırmanız gerekir: `obj->~Type();`

CPPREF Depth: constexpr Specifier Rules and Limits

A `constexpr` function must: (1) have a literal return type, (2) not contain goto, asm, or non-literal variables (pre-C++20). C++14 relaxed most restrictions (loops, multiple statements). C++20 allows virtual constexpr, try-catch, dynamic_cast. `consteval` (C++20) guarantees the function is ONLY called at compile time — if forced at runtime, it's a compilation error. `constinit` ensures static storage duration variables are initialized at compile time. cppreference.com/w/cpp/language/constexpr

CPPREF Derinlik: constexpr Belirleyici Kuralları ve Sınırları

Bir `constexpr` fonksiyon: (1) literal dönüş tipine sahip olmalı, (2) goto, asm veya literal olmayan değişkenler içermemeli (C++20 öncesi). C++14 çoğu kısıtlamayı gevşetti (döngüler, çoklu ifadeler). C++20 sanal constexpr, try-catch, dynamic_cast'e izin verir. `consteval` (C++20) fonksiyonun YALNIZCA derleme zamanında çağrıldığını garanti eder. `constinit` statik depolama süresine sahip değişkenlerin derleme zamanında başlatılmasını sağlar. cppreference.com/w/cpp/language/constexpr

CPPREF Depth: Placement New and Object Lifetime

Placement new does NOT allocate memory — it only calls the constructor at a given address. The programmer is responsible for: (1) ensuring proper alignment via `alignas` or `std::aligned_storage`, (2) manually calling the destructor before reusing memory, (3) NOT calling `delete` on placement-new objects. `std::launder` (C++17) is needed after placement new to access the object through old pointers when the type has const/reference members. cppreference.com/w/cpp/language/new#Placement_new

CPPREF Derinlik: Placement New ve Nesne Yaşam Süresi

Placement new bellek TAHSİS ETMEZ — yalnızca verilen adreste yapıcıyı çağırır. Programcı şunlardan sorumludur: (1) `alignas` veya `std::aligned_storage` ile uygun hizalamayı sağlama, (2) belleği yeniden kullanmadan önce yıkıcıyı manuel çağırma, (3) placement-new nesnelerinde `delete` ÇAĞIRMAMAK. `std::launder` (C++17) const/referans üyeleri olan türlerde eski işaretçiler aracılığıyla nesneye erişmek için placement new sonrası gereklidir. cppreference.com/w/cpp/language/new#Placement_new

Kaynak Kod / Source Code

```cpp
#include <cstdint>
#include <cstring>
#include <iostream>
#include <new>      // placement new
#include <array>
#include <type_traits>

// ==========================================================================================
// [1. STATIC RING BUFFER — No-Heap Circular Buffer]
// ==========================================================================================

// EN: A fixed-size ring buffer using no dynamic allocation.
// TR: Dinamik tahsis kullanmayan sabit boyutlu halka tampon.
template <typename T, std::size_t N>
class StaticRingBuffer {
  std::array<T, N> buffer_{};
  std::size_t head_ = 0;
  std::size_t tail_ = 0;
  std::size_t count_ = 0;

public:
  bool push(const T& item) {
    if (count_ == N) return false; // Full
    buffer_[tail_] = item;
    tail_ = (tail_ + 1) % N;
    ++count_;
    return true;
  }

  bool pop(T& item) {
    if (count_ == 0) return false; // Empty
    item = buffer_[head_];
    head_ = (head_ + 1) % N;
    --count_;
    return true;
  }

  [[nodiscard]] std::size_t size() const { return count_; }
  [[nodiscard]] bool empty() const { return count_ == 0; }
  [[nodiscard]] bool full() const { return count_ == N; }
};

// ==========================================================================================
// [2. CONSTEXPR — Compile-Time CRC-8 Table]
// ==========================================================================================

// EN: Generate a CRC-8 lookup table entirely at compile time.
// TR: CRC-8 arama tablosunu tamamen derleme zamanında oluştur.
constexpr uint8_t CRC8_POLY = 0x07;

constexpr std::array<uint8_t, 256> generateCRC8Table() {
  std::array<uint8_t, 256> table{};
  for (int i = 0; i < 256; ++i) {
    uint8_t crc = static_cast<uint8_t>(i);
    for (int bit = 0; bit < 8; ++bit) {
      if (crc & 0x80U) {
        crc = static_cast<uint8_t>((crc << 1) ^ CRC8_POLY);
      } else {
        crc = static_cast<uint8_t>(crc << 1);
      }
    }
    table[static_cast<std::size_t>(i)] = crc;
  }
  return table;
}
```

```cpp

// EN: Table is fully resolved at compile time — zero runtime cost.
// TR: Tablo tamamen derleme zamanında çözümlenir — sıfır çalışma zamanı maliyeti.
constexpr auto CRC8_TABLE = generateCRC8Table();

constexpr uint8_t computeCRC8(const uint8_t* data, std::size_t len) {
  uint8_t crc = 0x00;
  for (std::size_t i = 0; i < len; ++i) {
    crc = CRC8_TABLE[static_cast<std::size_t>(crc ^ data[i])];
  }
  return crc;
}

// ==========================================================================================
// [3. CONSTEXPR — Baud Rate Divisor]
// ==========================================================================================

// EN: Compute UART baud rate divisor at compile time.
// TR: UART baud hızı bölücüsünü derleme zamanında hesapla.
constexpr uint32_t computeBaudDivisor(uint32_t clockHz, uint32_t baudRate) {
  return (clockHz + baudRate / 2) / baudRate; // Rounded division
}

// EN: Resolved at compile time — no runtime division needed.
// TR: Derleme zamanında çözümlenir — çalışma zamanı bölmeye gerek yok.
constexpr uint32_t UART_DIVISOR_115200 = computeBaudDivisor(16000000, 115200);
constexpr uint32_t UART_DIVISOR_9600   = computeBaudDivisor(16000000, 9600);

// ==========================================================================================
// [4. PLACEMENT NEW — Memory Pool]
// ==========================================================================================

// EN: A simple fixed-size memory pool using placement new.
// TR: Placement new kullanan basit sabit boyutlu bellek havuzu.
struct Sensor {
  uint16_t id;
  float value;
  bool active;

  Sensor(uint16_t sensorId, float val)
      : id(sensorId), value(val), active(true) {}

  void print() const {
    std::cout << "  Sensor[" << id << "] value=" << value
              << " active=" << active << std::endl;
  }

  ~Sensor() {
    std::cout << "  ~Sensor[" << id << "] destroyed" << std::endl;
  }
};

// EN: Pool with compile-time capacity, no heap allocation.
// TR: Derleme zamanı kapasiteli havuz, yığın tahsisi yok.
template <typename T, std::size_t MaxObjects>
class StaticPool {
  // EN: Aligned raw storage for objects.
  // TR: Nesneler için hizalanmış ham depolama.
  alignas(T) uint8_t storage_[MaxObjects * sizeof(T)]{};
  bool occupied_[MaxObjects]{};

public:
  template <typename... Args>
```

```
  T* allocate(Args&&... args) {
    for (std::size_t i = 0; i < MaxObjects; ++i) {
      if (!occupied_[i]) {
        occupied_[i] = true;
        T* ptr = reinterpret_cast<T*>(&storage_[i * sizeof(T)]);
        new (ptr) T(std::forward<Args>(args)...); // placement new
        return ptr;
      }
    }
    return nullptr; // Pool exhausted
  }

  void deallocate(T* ptr) {
    auto addr = reinterpret_cast<uint8_t*>(ptr);
    auto base = &storage_[0];
    if (addr < base || addr >= base + MaxObjects * sizeof(T)) return;

    std::size_t idx = static_cast<std::size_t>(addr - base) / sizeof(T);
    if (occupied_[idx]) {
      ptr->~T(); // Manual destructor call
      occupied_[idx] = false;
    }
  }

  [[nodiscard]] std::size_t usedCount() const {
    std::size_t count = 0;
    for (std::size_t i = 0; i < MaxObjects; ++i) {
      if (occupied_[i]) ++count;
    }
    return count;
  }
};

// ==========================================================================================
// [5. STATIC LOOKUP TABLE — Compile-Time Sine Approximation]
// ==========================================================================================

constexpr double PI = 3.14159265358979323846;

constexpr double constexprSin(double x) {
  // EN: Taylor series sin(x) = x - x^3/3! + x^5/5! - x^7/7! + ...
  // TR: Taylor serisi sin(x) = x - x^3/3! + x^5/5! - x^7/7! + ...
  double result = 0.0;
  double term = x;
  for (int n = 1; n <= 15; n += 2) {
    result += term;
    term *= -x * x / static_cast<double>((n + 1) * (n + 2));
  }
  return result;
}

// EN: 16-entry sine table, built at compile time.
// TR: Derleme zamanında oluşturulan 16 girişli sinüs tablosu.
constexpr std::array<double, 16> generateSineTable() {
  std::array<double, 16> table{};
  for (int i = 0; i < 16; ++i) {
    table[static_cast<std::size_t>(i)] =
        constexprSin(2.0 * PI * static_cast<double>(i) / 16.0);
  }
  return table;
}

constexpr auto SINE_TABLE = generateSineTable();

int main() {
  std::cout << "=== MODULE 11: EMBEDDED CONSTRAINTS ===\n" << std::endl;
```

```cpp

    // --- Demo 1: Static Ring Buffer ---
    std::cout << "--- DEMO 1: Static Ring Buffer (No Heap) ---\n" << std::endl;
    StaticRingBuffer<int, 4> rb;
    rb.push(10);
    rb.push(20);
    rb.push(30);
    rb.push(40);
    std::cout << "Buffer full? " << (rb.full() ? "YES" : "NO") << std::endl;
    std::cout << "Push(50) success? " << (rb.push(50) ? "YES" : "NO") << std::endl;

    int val = 0;
    rb.pop(val);
    std::cout << "Popped: " << val << " | Size: " << rb.size() << "\n" << std::endl;

    // --- Demo 2: Compile-Time CRC-8 ---
    std::cout << "--- DEMO 2: Constexpr CRC-8 Table ---\n" << std::endl;
    const uint8_t testData[] = {0x01, 0x02, 0x03, 0x04, 0x05};
    uint8_t crc = computeCRC8(testData, 5);
    std::cout << "CRC-8 of {01,02,03,04,05} = 0x" << std::hex
              << static_cast<int>(crc) << std::dec << std::endl;
    std::cout << "CRC table[0x00]=" << static_cast<int>(CRC8_TABLE[0])
              << " table[0xFF]=" << static_cast<int>(CRC8_TABLE[255])
              << "\n" << std::endl;

    // --- Demo 3: Baud Rate Divisor ---
    std::cout << "--- DEMO 3: Compile-Time Baud Divisor ---\n" << std::endl;
    std::cout << "16MHz / 115200 baud => divisor=" << UART_DIVISOR_115200
              << std::endl;
    std::cout << "16MHz / 9600 baud   => divisor=" << UART_DIVISOR_9600
              << "\n" << std::endl;

    // --- Demo 4: Placement New Memory Pool ---
    std::cout << "--- DEMO 4: Static Memory Pool (Placement New) ---\n"
              << std::endl;
    StaticPool<Sensor, 4> pool;
    Sensor* s1 = pool.allocate(static_cast<uint16_t>(1), 23.5f);
    Sensor* s2 = pool.allocate(static_cast<uint16_t>(2), 45.1f);
    Sensor* s3 = pool.allocate(static_cast<uint16_t>(3), 67.8f);
    std::cout << "Pool used: " << pool.usedCount() << "/4" << std::endl;
    if (s1) s1->print();
    if (s2) s2->print();
    if (s3) s3->print();

    std::cout << "\nDeallocating sensor 2..." << std::endl;
    pool.deallocate(s2);
    std::cout << "Pool used: " << pool.usedCount() << "/4" << std::endl;

    Sensor* s4 = pool.allocate(static_cast<uint16_t>(4), 99.9f);
    if (s4) s4->print();
    std::cout << "Pool used: " << pool.usedCount() << "/4" << std::endl;

    // EN: Clean up remaining objects.
    // TR: Kalan nesneleri temizle.
    pool.deallocate(s1);
    pool.deallocate(s3);
    pool.deallocate(s4);

    // --- Demo 5: Compile-Time Sine Table ---
    std::cout << "\n--- DEMO 5: Constexpr Sine Lookup Table ---\n" << std::endl;
    std::cout << "16-point sine table (computed at compile time):" << std::endl;
    for (std::size_t i = 0; i < SINE_TABLE.size(); ++i) {
      std::cout << "  sin(" << i << "*22.5deg) = ";
      if (SINE_TABLE[i] >= 0) std::cout << " ";
      std::cout << SINE_TABLE[i] << std::endl;
    }

    std::cout << "\n=> No-heap, constexpr, memory pool, placement new "
                 "demonstrated!" << std::endl;

    return 0;
  }
```

► 04. Rtos Concurrency

module_11_embedded_systems/04_rtos_concurrency.cpp

Konu: Embedded & Systems: RTOS Patterns, Priority Inversion, Lock-Free Structures / RTOS

Teori / Theory

THEORY: RTOS Task Model — Cooperative vs Preemptive

Real-Time Operating Systems (FreeRTOS, AUTOSAR OS, QNX) manage tasks with fixed priorities. Preemptive scheduling: a higher-priority task instantly preempts a lower-priority one. Cooperative: tasks yield voluntarily. In automotive ECUs, critical tasks (brake ABS, airbag) run at highest priority. HMI/Infotainment tasks run at lower priority. Tasks communicate via message queues, semaphores, and shared memory.

TEORİ: RTOS Görev Modeli — İşbirlikli vs Öncelikli

Gerçek Zamanlı İşletim Sistemleri (FreeRTOS, AUTOSAR OS, QNX) görevleri sabit önceliklerle yönetir. Öncelikli zamanlama: yüksek öncelikli görev anında düşük öncelikli görevi keser. İşbirlikli: görevler gönüllü olarak CPU'yu bırakır. Otomotiv ECU'larda kritik görevler (fren ABS, hava yastığı) en yüksek öncelikte çalışır. HMI/Bilgi-eğlence görevleri daha düşük öncelikte çalışır. Görevler mesaj kuyrukları, semaforlar ve paylaşılan bellek aracılığıyla iletişim kurar.

THEORY: Priority Inversion — The Mars Pathfinder Bug

Priority Inversion occurs when a high-priority task waits for a low-priority task holding a shared mutex — while a medium-priority task preempts the low-priority one, BLOCKING the high-priority task indefinitely. This caused the Mars Pathfinder reboot loop in 1997. Solutions: (1) Priority Inheritance: temporarily boost the low-priority task's priority. (2) Priority Ceiling: mutex has a pre-assigned ceiling priority. (3) Lock-free structures: eliminate mutexes entirely.

TEORİ: Öncelik Tersine Çevrilmesi — Mars Pathfinder Hatası

Öncelik Tersine Çevrilmesi, yüksek öncelikli görev paylaşılan bir mutex'i tutan düşük öncelikli görevi beklediğinde oluşur — orta öncelikli görev düşük öncelikliyi keserken, yüksek öncelikli görevi SÜRESİZ BLOKE EDİYOR. Bu, 1997'de Mars Pathfinder yeniden başlatma döngüsüne neden oldu. Çözümler: (1) Öncelik Kalıtımı: düşük öncelikli görevin önceliğini geçici olarak yükselt. (2) Öncelik Tavanı: mutex'in önceden atanmış bir tavan önceliği vardır. (3) Kilitsiz yapılar: mutex'leri tamamen ortadan kaldır.

THEORY: Lock-Free Programming — CAS Operations

Lock-free data structures use atomic Compare-And-Swap (CAS) instead of mutexes. `std::atomic::compare_exchange_weak/strong` atomically: reads the current value, compares it to "expected," and if equal, swaps to "desired." This eliminates deadlocks, priority inversion, and lock contention. Used extensively in automotive middleware (AUTOSAR COM), sensor data pipelines, and real-time audio processing.

TEORİ: Kilitsiz Programlama — CAS İşlemleri

Kilitsiz veri yapıları mutex'ler yerine atomik Karşılaştır-Ve-Değiştir (CAS) kullanır. `std::atomic::compare_exchange_weak/strong` atomik olarak: mevcut değeri okur, "beklenen" ile karşılaştırır ve eşitse "istenen" ile değiştirir. Bu, kilitlenmeleri, öncelik tersine çevrilmesini ve kilit çekişmesini ortadan kaldırır. Otomotiv ara yazılımında (AUTOSAR COM), sensör veri hatlarında ve gerçek zamanlı ses işlemede yoğun olarak kullanılır.

CPPREF Depth: std::atomic::compare_exchange_weak vs strong

`compare_exchange_weak` may fail spuriously (return false even if current == expected) on some architectures (ARM LL/SC). It MUST be used in a loop. `compare_exchange_strong` never fails spuriously but may be slower on LL/SC platforms. Weak is preferred in CAS loops, strong for single-shot operations. Both take memory_order parameters for fine-grained control. cppreference.com/w/cpp/atomic/atomic/compare_exchange

CPPREF Derinlik: std::atomic::compare_exchange_weak vs strong

`compare_exchange_weak` bazı mimarilerde (ARM LL/SC) sahte başarısız olabilir (current == expected olsa bile false döner). Bir döngü içinde kullanılması ZORUNLUDUR. `compare_exchange_strong` asla sahte başarısız olmaz ancak LL/SC platformlarda daha yavaş olabilir. CAS döngülerinde weak, tek seferlik işlemlerde strong tercih edilir. cppreference.com/w/cpp/atomic/atomic/compare_exchange

CPPREF Depth: std::memory_order — Acquire/Release Semantics

Memory ordering controls how atomic operations are reordered by CPU/ compiler. `memory_order_relaxed`: no ordering guarantees (fastest, counters). `memory_order_acquire`: no reads/writes can move BEFORE this load. `memory_order_release`: no reads/writes can move AFTER this store. Acquire-Release pair synchronizes: if thread A releases and thread B acquires the same atomic, B sees all writes A made before the release. `memory_order_seq_cst` (default): full sequential consistency — slowest but safest. In embedded, relaxed+acquire/release is heavily used. cppreference.com/w/cpp/atomic/memory_order

CPPREF Derinlik: std::memory_order — Acquire/Release Semantiği

Bellek sıralaması, atomik işlemlerin CPU/derleyici tarafından nasıl yeniden sıralandığını kontrol eder. `memory_order_relaxed`: sıralama garantisi yok (en hızlı, sayaçlar). `memory_order_acquire`: bu yüklemeden ÖNCE hiçbir okuma/yazma taşınamaz. `memory_order_release`: bu depolamadan SONRA hiçbir okuma/yazma taşınamaz. Acquire-Release çifti senkronize eder. `memory_order_seq_cst` (varsayılan): tam sıralı tutarlılık — en yavaş ama en güvenli. Gömülü sistemlerde relaxed+acquire/release yoğun kullanılır. cppreference.com/w/cpp/atomic/memory_order

Kaynak Kod / Source Code

```cpp
#include <atomic>
#include <chrono>
#include <cstdint>
#include <functional>
#include <iostream>
#include <mutex>
#include <queue>
#include <string>
#include <thread>
#include <vector>

// ==========================================================================================
// [1. RTOS TASK SIMULATION — Priority-Based Scheduler]
// ==========================================================================================

// EN: Simulates RTOS-style tasks with priorities (like FreeRTOS).
// TR: Öncelikli RTOS tarzı görevleri simüle eder (FreeRTOS gibi).
struct RTOSTask {
  std::string name;
  int priority; // Higher = more important
  std::function<void()> taskFunc;
};
```

```cpp

class SimpleScheduler {
  std::vector<RTOSTask> tasks_;
  std::mutex mtx_;

public:
  void addTask(const std::string& name, int priority,
               std::function<void()> func) {
    std::lock_guard<std::mutex> lock(mtx_);
    tasks_.push_back({name, priority, std::move(func)});
  }

  // EN: Execute tasks in priority order (highest first).
  // TR: Görevleri öncelik sırasına göre çalıştır (en yüksek önce).
  void runAll() {
    std::lock_guard<std::mutex> lock(mtx_);
    // Sort by priority (descending)
    std::sort(tasks_.begin(), tasks_.end(),
              [](const RTOSTask& a, const RTOSTask& b) {
                return a.priority > b.priority;
              });

    for (const auto& task : tasks_) {
      std::cout << "  [SCHED] Running '" << task.name
                << "' (priority=" << task.priority << ")" << std::endl;
      task.taskFunc();
    }
    tasks_.clear();
  }
};

// ==========================================================================================
// [2. PRIORITY INVERSION DEMONSTRATION]
// ==========================================================================================

// EN: Shows how priority inversion can occur with shared mutexes.
// TR: Paylaşılan mutex'lerle öncelik tersine çevrilmesinin nasıl oluştuğunu gösterir.

std::mutex sharedResourceMtx;
std::atomic<bool> inversionDemo{true};

void lowPriorityTask() {
  std::cout << "  [LOW] Acquiring shared resource..." << std::endl;
  std::lock_guard<std::mutex> lock(sharedResourceMtx);
  std::cout << "  [LOW] Got resource. Working (simulating slow I/O)..."
            << std::endl;
  // EN: Simulate slow work while holding the lock.
  // TR: Kilidi tutarken yavaş çalışmayı simüle et.
  std::this_thread::sleep_for(std::chrono::milliseconds(80));
  std::cout << "  [LOW] Done. Releasing resource." << std::endl;
}

void highPriorityTask() {
  // EN: Small delay so low-priority grabs the lock first.
  // TR: Düşük öncelikli görev kilidi ilk alsın diye küçük gecikme.
  std::this_thread::sleep_for(std::chrono::milliseconds(10));
  std::cout << "  [HIGH] Trying to acquire shared resource..." << std::endl;
  auto start = std::chrono::steady_clock::now();
  std::lock_guard<std::mutex> lock(sharedResourceMtx);
  auto end = std::chrono::steady_clock::now();
  auto waitMs = std::chrono::duration_cast<std::chrono::milliseconds>(
                    end - start).count();
  std::cout << "  [HIGH] Got resource after waiting " << waitMs << "ms!"
            << std::endl;
  std::cout << "  [HIGH] => Priority inversion: HIGH waited for LOW!"
            << std::endl;
}
```

```cpp

// ==========================================================================================
// [3. LOCK-FREE SPSC QUEUE (Single Producer, Single Consumer)]
// ==========================================================================================

// EN: Lock-free SPSC queue using atomic indices — used in automotive sensor pipelines (ADC →
// DSP), CAN message buffers, and audio ring buffers.
// TR: Atomik indeksler kullanan kilitsiz SPSC kuyruğu — otomotiv sensör hatlarında (ADC → DSP),
// CAN mesaj tamponlarında kullanılır.
template <typename T, std::size_t Capacity>
class LockFreeSPSCQueue {
  std::array<T, Capacity> buffer_{};
  std::atomic<std::size_t> readIdx_{0};
  std::atomic<std::size_t> writeIdx_{0};

public:
  bool tryPush(const T& item) {
    std::size_t currWrite = writeIdx_.load(std::memory_order_relaxed);
    std::size_t nextWrite = (currWrite + 1) % Capacity;
    // EN: If next write would hit read, queue is full.
    // TR: Sonraki yazma okumaya çarparsa, kuyruk doludur.
    if (nextWrite == readIdx_.load(std::memory_order_acquire)) {
      return false; // Full
    }
    buffer_[currWrite] = item;
    writeIdx_.store(nextWrite, std::memory_order_release);
    return true;
  }

  bool tryPop(T& item) {
    std::size_t currRead = readIdx_.load(std::memory_order_relaxed);
    if (currRead == writeIdx_.load(std::memory_order_acquire)) {
      return false; // Empty
    }
    item = buffer_[currRead];
    readIdx_.store((currRead + 1) % Capacity,
                   std::memory_order_release);
    return true;
  }
};

// ==========================================================================================
// [4. ATOMIC CAS — Lock-Free Counter with Backoff]
// ==========================================================================================

// EN: Demonstrates compare_exchange_weak in a CAS loop with exponential backoff — common in
// lock-free embedded counters.
// TR: Üstel geri çekilme ile CAS döngüsünde compare_exchange_weak'i gösterir — kilitsiz gömülü
// sayaçlarda yaygındır.

std::atomic<int> lockFreeCounter{0};

void casIncrement(int iterations) {
  for (int i = 0; i < iterations; ++i) {
    int expected = lockFreeCounter.load(std::memory_order_relaxed);
    while (!lockFreeCounter.compare_exchange_weak(
        expected, expected + 1, std::memory_order_release,
        std::memory_order_relaxed)) {
      // EN: CAS failed — expected was updated, retry automatically.
      // TR: CAS başarısız — expected güncellendi, otomatik yeniden dene.
    }
  }
}

int main() {
  std::cout << "=== MODULE 11: RTOS CONCURRENCY PATTERNS ===\n" << std::endl;
```

```cpp

    // --- Demo 1: Priority-Based Scheduler ---
    std::cout << "--- DEMO 1: RTOS Priority Scheduler ---\n" << std::endl;
    SimpleScheduler scheduler;
    scheduler.addTask("HMI_Render", 2, []() {
      std::cout << "    -> Rendering instrument cluster gauges" << std::endl;
    });
    scheduler.addTask("ABS_BrakeControl", 10, []() {
      std::cout << "    -> ABS brake actuation — CRITICAL" << std::endl;
    });
    scheduler.addTask("Climate_FanSpeed", 3, []() {
      std::cout << "    -> Adjusting HVAC fan to target temperature" << std::endl;
    });
    scheduler.addTask("Airbag_Deploy", 15, []() {
      std::cout << "    -> Airbag deployment check — HIGHEST PRIORITY"
                << std::endl;
    });
    scheduler.addTask("CAN_Logger", 1, []() {
      std::cout << "    -> Logging CAN bus messages to flash" << std::endl;
    });
    scheduler.runAll();

    // --- Demo 2: Priority Inversion ---
    std::cout << "\n--- DEMO 2: Priority Inversion ---\n" << std::endl;
    std::cout << "Scenario: LOW holds mutex, HIGH must wait:" << std::endl;
    std::thread tLow(lowPriorityTask);
    std::thread tHigh(highPriorityTask);
    tLow.join();
    tHigh.join();

    // --- Demo 3: Lock-Free SPSC Queue ---
    std::cout << "\n--- DEMO 3: Lock-Free SPSC Queue ---\n" << std::endl;
    LockFreeSPSCQueue<int, 8> sensorQueue;

    // EN: Producer: ADC sampling interrupt pushes sensor readings.
    // TR: Üretici: ADC örnekleme kesmesi sensör okumalarını iter.
    std::thread producer([&sensorQueue]() {
      for (int i = 1; i <= 10; ++i) {
        int sensorVal = 1000 + i * 50; // Simulated EGT value in °C
        if (sensorQueue.tryPush(sensorVal)) {
          std::cout << "  [PRODUCER] Pushed EGT reading: " << sensorVal
                    << " C" << std::endl;
        } else {
          std::cout << "  [PRODUCER] Queue full! Dropped: " << sensorVal
                    << std::endl;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(5));
      }
    });

    // EN: Consumer: DSP task processes sensor readings.
    // TR: Tüketici: DSP görevi sensör okumalarını işler.
    std::thread consumer([&sensorQueue]() {
      int val = 0;
      int count = 0;
      while (count < 10) {
        if (sensorQueue.tryPop(val)) {
          std::cout << "  [CONSUMER] Processed EGT: " << val << " C"
                    << (val > 1300 ? " ** WARNING: OVERHEAT! **" : "")
                    << std::endl;
          ++count;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(8));
      }
    });

    producer.join();
    consumer.join();
```

```

  // --- Demo 4: Lock-Free CAS Counter ---
  std::cout << "\n--- DEMO 4: Lock-Free CAS Counter ---\n" << std::endl;
  lockFreeCounter.store(0);
  constexpr int THREADS = 4;
  constexpr int ITERS_PER_THREAD = 10000;
  std::vector<std::thread> workers;
  workers.reserve(THREADS);
  for (int t = 0; t < THREADS; ++t) {
    workers.emplace_back(casIncrement, ITERS_PER_THREAD);
  }
  for (auto& w : workers) w.join();

  std::cout << "  Expected: " << THREADS * ITERS_PER_THREAD << std::endl;
  std::cout << "  Actual:   " << lockFreeCounter.load() << std::endl;
  std::cout << "  Match:    "
            << (lockFreeCounter.load() == THREADS * ITERS_PER_THREAD
                    ? "YES (lock-free is correct!)"
                    : "NO (bug!)")
            << std::endl;

  std::cout << "\n=> RTOS scheduling, priority inversion, lock-free queue, "
               "CAS counter demonstrated!" << std::endl;

  return 0;
}
```

► 05. Advanced Parallelism

module_11_embedded_systems/05_advanced_parallelism.cpp

Konu: Embedded & Systems: Thread Pools, C++17 Parallel Algorithms, Work Stealing / İş

Teori / Theory

THEORY: Thread Pools — Why Not Spawn Per Task

Creating a new `std::thread` for each task is expensive: OS must allocate stack memory (~1-8MB), register the thread with the scheduler, and tear it down when done. In embedded/automotive systems with limited resources, this is unacceptable. A thread pool pre-creates N worker threads that continuously pull tasks from a shared queue. This gives: (1) bounded resource usage, (2) amortized creation cost, (3) predictable latency. AUTOSAR Adaptive Platform uses thread pools for service-oriented RPCs.

TEORİ: İş Parçacığı Havuzları — Neden Görev Başına Oluşturma Yapılmaz

Her görev için yeni bir `std::thread` oluşturmak pahalıdır: OS yığıt belleği (~1-8MB) ayırmalı, iş parçacığını zamanlayıcıya kaydetmeli ve bittiğinde yıkmalıdır. Sınırlı kaynaklara sahip gömülü/otomotiv sistemlerde bu kabul edilemez. Bir iş parçacığı havuzu, sürekli olarak paylaşılan bir kuyruktan görev çeken N işçi iş parçacığı önceden oluşturur. Bu şunları verir: (1) sınırlı kaynak kullanımı, (2) amortize oluşturma maliyeti, (3) öngörülebilir gecikme.

THEORY: C++17 Parallel Algorithms — std::execution Policies

C++17 added execution policies to STL algorithms: `std::execution::seq` (sequential), `std::execution::par` (parallel), `std::execution::par_unseq` (parallel + vectorized). This means existing `std::sort`, `std::transform`, `std::reduce` can run in parallel with just one extra argument! In automotive: parallel map data processing, multiple sensor fusion, batch diagnostics. Note: requires linking `-ltbb` on GCC (Intel TBB backend).

TEORİ: C++17 Paralel Algoritmalar — std::execution Politikaları

C++17, STL algoritmalarına yürütme politikaları ekledi: `std::execution::seq` (sıralı), `std::execution::par` (paralel), `std::execution::par_unseq` (paralel + vektörize). Bu, mevcut `std::sort`, `std::transform`, `std::reduce`'un tek bir ek argümanla paralel çalışabileceği anlamına gelir! Otomotivde: paralel harita veri işleme, çoklu sensör füzyonu, toplu tanılama. Not: GCC'de `-ltbb` bağlantısı gerektirir (Intel TBB arka ucu).

THEORY: Work Stealing — Load-Balanced Parallelism

In a basic thread pool, if one task is much longer than others, its worker stays busy while others idle. Work stealing solves this: each worker has its own deque. When a worker's deque is empty, it "steals" a task from another worker's deque (from the BACK, to minimize contention). Intel TBB, Java ForkJoinPool, and Tokio (Rust) all use work stealing. In automotive: useful for parallel sensor data pipelines with varying latencies.

TEORİ: İş Çalma — Yük-Dengeli Paralellik

Temel bir iş parçacığı havuzunda, bir görev diğerlerinden çok uzunsa, işçisi meşgul kalırken diğerleri boşta kalır. İş çalma bunu çözer: her işçinin kendi deque'ü vardır. Bir işçinin deque'ü boşaldığında, başka bir işçinin deque'ünden (çekişmeyi en aza indirmek için ARKADAN) bir görev "çalar". Intel TBB, Java ForkJoinPool ve Tokio (Rust) hepsi iş çalma kullanır.

CPPREF Depth: std::execution Policy Tag Types

`std::execution::seq` — no parallelism, same as before. Elements accessed in order on calling thread. `std::execution::par` — may execute in parallel on unspecified threads but elements are NOT interleaved within a thread. `std::execution::par_unseq` — may execute in parallel AND vectorized (SIMD). Element access functions must NOT use mutexes or blocking synchronization! `std::execution::unseq` (C++20) — vectorized on the calling thread only. cppreference.com/w/cpp/algorithm/execution_policy_tag

CPPREF Derinlik: std::execution Politika Etiket Tipleri

`std::execution::seq` — paralellik yok, öncekiyle aynı. `std::execution::par` — belirtilmemiş iş parçacıklarında paralel yürütülebilir ancak elemanlar bir iş parçacığı içinde serpiştirilmez. `std::execution::par_unseq` — paralel VE vektörize (SIMD) yürütülebilir. Eleman erişim fonksiyonları mutex veya blokleyici senkronizasyon KULLANMAMALIDIR! `std::execution::unseq` (C++20) — yalnızca çağıran iş parçacığında vektörize. cppreference.com/w/cpp/algorithm/execution_policy_tag

CPPREF Depth: std::reduce vs std::accumulate — Parallel Reduction

`std::accumulate` (C++98) is strictly left-fold: `((init op a) op b) op c`. Cannot be parallelized because order is fixed. `std::reduce` (C++17) is unordered: the operation can be applied in any order, enabling parallel execution. The operation MUST be associative and commutative (e.g., `+`, `*`, `max`). For floating-point, `reduce` may give slightly different results due to different summation order (affects rounding). `std::transform_reduce` is the parallel map-reduce primitive. cppreference.com/w/cpp/algorithm/reduce

CPPREF Derinlik: std::reduce vs std::accumulate — Paralel İndirgeme

`std::accumulate` (C++98) kesinlikle sol-katlama: sıra sabittir. Paralelleştirilemez. `std::reduce` (C++17) sırasızdır: işlem herhangi bir sırada uygulanabilir, paralel yürütmeyi mümkün kılar. İşlem ilişkisel ve değişmeli OLMALIDIR (ör. `+`, `*`, `max`). Kayan nokta için `reduce` farklı toplama sırası nedeniyle biraz farklı sonuçlar verebilir. `std::transform_reduce` paralel map-reduce ilkesidir. cppreference.com/w/cpp/algorithm/reduce

Kaynak Kod / Source Code

```cpp
#include <algorithm>
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdint>
#include <functional>
#include <iostream>
#include <mutex>
#include <numeric>
#include <queue>
#include <string>
#include <thread>
#include <vector>

// ==========================================================================================
// [1. THREAD POOL IMPLEMENTATION]
// ==========================================================================================

// EN: A production-quality thread pool for embedded task dispatching.
// TR: Gömülü görev dağıtımı için üretim kalitesinde iş parçacığı havuzu.
class ThreadPool {
  std::vector<std::thread> workers_;
  std::queue<std::function<void()>> taskQueue_;
  std::mutex mtx_;
  std::condition_variable cv_;
  std::atomic<bool> shutdown_{false};

public:
  explicit ThreadPool(std::size_t numThreads) {
    for (std::size_t i = 0; i < numThreads; ++i) {
      workers_.emplace_back([this]() { workerLoop(); });
    }
  }

  ~ThreadPool() {
    shutdown_.store(true, std::memory_order_release);
    cv_.notify_all();
    for (auto& w : workers_) {
      if (w.joinable()) w.join();
    }
  }

  // EN: Submit a task to the pool.
  // TR: Havuza bir görev gönder.
  void submit(std::function<void()> task) {
    {
      std::lock_guard<std::mutex> lock(mtx_);
      taskQueue_.push(std::move(task));
    }
    cv_.notify_one();
  }

  // EN: Wait until all queued tasks are completed.
  // TR: Kuyruktaki tüm görevler tamamlanana kadar bekle.
  void waitAll() {
    std::unique_lock<std::mutex> lock(mtx_);
    cv_.wait(lock, [this]() {
      return taskQueue_.empty();
    });
    // EN: Small grace period for in-flight tasks to complete.
    // TR: Devam eden görevlerin tamamlanması için küçük bekleme süresi.
    lock.unlock();
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
  }

private:
```

```cpp
  void workerLoop() {
    while (true) {
      std::function<void()> task;
      {
        std::unique_lock<std::mutex> lock(mtx_);
        cv_.wait(lock, [this]() {
          return shutdown_.load(std::memory_order_acquire) ||
                 !taskQueue_.empty();
        });
        if (shutdown_.load(std::memory_order_acquire) &&
            taskQueue_.empty()) {
          return;
        }
        task = std::move(taskQueue_.front());
        taskQueue_.pop();
      }
      task();
      cv_.notify_all(); // Wake waitAll()
    }
  }
};

// ==========================================================================================
// [2. SIMULATED SENSOR FUSION — Parallel Processing]
// ==========================================================================================

// EN: Simulate processing multiple automotive sensor streams in parallel.
// TR: Birden fazla otomotiv sensör akışını paralel olarak işlemeyi simüle et.
struct SensorReading {
  std::string sensorName;
  double value;
  std::string unit;
};

SensorReading processSensor(const std::string& name, double rawValue,
                            double calibrationOffset) {
  // EN: Simulate some computation (filtering, calibration).
  // TR: Bazı hesaplamaları simüle et (filtreleme, kalibrasyon).
  std::this_thread::sleep_for(std::chrono::milliseconds(20));
  double calibrated = rawValue + calibrationOffset;
  std::string unit = (name.find("EGT") != std::string::npos)   ? "C"
                     : (name.find("RPM") != std::string::npos)  ? "rpm"
                     : (name.find("MAP") != std::string::npos)  ? "kPa"
                     : (name.find("AFR") != std::string::npos)  ? "lambda"
                                                                : "raw";
  return {name, calibrated, unit};
}

// ==========================================================================================
// [3. MANUAL std::reduce EQUIVALENT — Portable Sum]
// ==========================================================================================

// EN: Since parallel std::execution requires -ltbb and may not be available, we demonstrate the
// concept with manual parallel reduction.
// TR: Paralel std::execution -ltbb gerektirdiğinden ve mevcut olmayabileceğinden kavramı manuel
// paralel indirgeme ile gösteriyoruz.

double parallelReduce(const std::vector<double>& data, std::size_t numThreads) {
  std::vector<double> partialSums(numThreads, 0.0);
  std::vector<std::thread> threads;
  std::size_t chunkSize = data.size() / numThreads;

  for (std::size_t t = 0; t < numThreads; ++t) {
    std::size_t start = t * chunkSize;
    std::size_t end = (t == numThreads - 1) ? data.size() : start + chunkSize;
    threads.emplace_back(
        [&data, &partialSums, t, start, end]() {
          double sum = 0.0;
```

```
135            for (std::size_t i = start; i < end; ++i) {
136              sum += data[i];
137            }
138            partialSums[t] = sum;
139          });
140    }
141    for (auto& th : threads) th.join();
142
143    // EN: Final sequential merge of partial sums.
144    // TR: Kısmi toplamların son sıralı birleştirmesi.
145    double total = 0.0;
146    for (double ps : partialSums) total += ps;
147    return total;
148  }
149
150  int main() {
151    std::cout << "=== MODULE 11: ADVANCED PARALLELISM ===\n" << std::endl;
152
153    // --- Demo 1: Thread Pool ---
154    std::cout << "--- DEMO 1: Thread Pool (4 workers) ---\n" << std::endl;
155    {
156      ThreadPool pool(4);
157      std::mutex printMtx;
158
159      // EN: Submit automotive ECU tasks to the pool.
160      // TR: Otomotiv ECU görevlerini havuza gönder.
161      std::vector<std::string> ecuTasks = {
162          "BCM: Check door lock status",
163          "Cluster: Update speedometer needle",
164          "HVAC: Read cabin temperature sensor",
165          "Powertrain: Calculate fuel injection timing",
166          "HMI: Render navigation overlay",
167          "Chassis: Process wheel speed ABS data",
168          "Telematics: Upload OBD-II snapshot",
169          "BCM: Toggle headlight relay"};
170
171      for (const auto& taskDesc : ecuTasks) {
172        pool.submit([&printMtx, taskDesc]() {
173          std::lock_guard<std::mutex> lock(printMtx);
174          std::cout << "  [Worker " << std::this_thread::get_id()
175                    << "] " << taskDesc << std::endl;
176        });
177      }
178      pool.waitAll();
179    }
180    std::cout << "  Thread pool destroyed — all workers joined.\n" << std::endl;
181
182    // --- Demo 2: Parallel Sensor Fusion ---
183    std::cout << "--- DEMO 2: Parallel Sensor Fusion ---\n" << std::endl;
184    {
185      struct SensorJob {
186        std::string name;
187        double rawVal;
188        double offset;
189      };
190
191      std::vector<SensorJob> jobs = {
192          {"EGT_Cyl1", 720.0, +2.5},  {"EGT_Cyl2", 735.0, -1.2},
193          {"EGT_Cyl3", 710.0, +0.8},  {"EGT_Cyl4", 745.0, -0.5},
194          {"RPM_Crank", 3200.0, 0.0},  {"MAP_Intake", 85.0, +1.1},
195          {"AFR_Wideband", 0.98, +0.02}};
196
197      std::vector<SensorReading> results(jobs.size());
198      std::vector<std::thread> threads;
199
200      auto startTime = std::chrono::steady_clock::now();
```

```
    for (std::size_t i = 0; i < jobs.size(); ++i) {
      threads.emplace_back([&jobs, &results, i]() {
        results[i] = processSensor(jobs[i].name, jobs[i].rawVal,
                                   jobs[i].offset);
      });
    }
    for (auto& t : threads) t.join();
    auto endTime = std::chrono::steady_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(
                       endTime - startTime).count();

    for (const auto& r : results) {
      std::cout << "  " << r.sensorName << " = " << r.value << " "
                << r.unit << std::endl;
    }
    std::cout << "  Parallel time: " << elapsed << "ms"
              << " (sequential would be ~" << jobs.size() * 20 << "ms)\n"
              << std::endl;
  }

  // --- Demo 3: Parallel Reduction ---
  std::cout << "--- DEMO 3: Parallel Reduction ---\n" << std::endl;
  {
    constexpr std::size_t DATA_SIZE = 1000000;
    std::vector<double> sensorData(DATA_SIZE);
    for (std::size_t i = 0; i < DATA_SIZE; ++i) {
      sensorData[i] = static_cast<double>(i % 100) * 0.01;
    }

    // EN: Sequential sum using std::accumulate.
    // TR: std::accumulate kullanarak sıralı toplam.
    auto t1 = std::chrono::steady_clock::now();
    double seqSum = std::accumulate(sensorData.begin(), sensorData.end(), 0.0);
    auto t2 = std::chrono::steady_clock::now();
    auto seqMs = std::chrono::duration_cast<std::chrono::microseconds>(
                     t2 - t1).count();

    // EN: Parallel sum using manual thread-based reduction.
    // TR: Manuel iş parçacığı tabanlı indirgeme kullanarak paralel toplam.
    auto t3 = std::chrono::steady_clock::now();
    double parSum = parallelReduce(sensorData, 4);
    auto t4 = std::chrono::steady_clock::now();
    auto parMs = std::chrono::duration_cast<std::chrono::microseconds>(
                     t4 - t3).count();

    std::cout << "  Sequential sum: " << seqSum << " (" << seqMs << " us)"
              << std::endl;
    std::cout << "  Parallel sum:   " << parSum << " (" << parMs << " us)"
              << std::endl;
    std::cout << "  Match: "
              << (std::abs(seqSum - parSum) < 0.001 ? "YES" : "DRIFT")
              << std::endl;
  }

  std::cout << "\n--- DEMO 4: std::execution Policy Reference ---\n"
            << std::endl;
  std::cout << "  // With -ltbb, these would work:" << std::endl;
  std::cout << "  // std::sort(std::execution::par, v.begin(), v.end());"
            << std::endl;
  std::cout << "  // std::reduce(std::execution::par, v.begin(), v.end());"
            << std::endl;
  std::cout << "  // std::transform_reduce(std::execution::par_unseq, ...);"
            << std::endl;
  std::cout << "  // See cppreference.com/w/cpp/algorithm/execution_policy_tag"
            << std::endl;

  std::cout << "\n=> Thread pool, sensor fusion, parallel reduction "
               "demonstrated!" << std::endl;

  return 0;
}
```

Ek: Derleme Bilgisi

Compilation Instructions

```
# Modül 1-8 (C++17):
g++ -std=c++17 -Wall -Wextra -Wpedantic -Wshadow \
    -Wconversion -Wsign-conversion -Wnull-dereference -Wformat=2 \
    <dosya_adi>.cpp -o output

# Modül 9-11 (C++20 gerektirir):
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wshadow \
    -Wconversion -Wsign-conversion -Wnull-dereference -Wformat=2 \
    <dosya_adi>.cpp -o output

# Concurrency dosyaları (-pthread gerekli):
# Modül 6/01, 6/05, 7/03, 8/04, 11/04, 11/05
# Yukarıdaki flag'lere ek olarak -pthread ekleyin.

# Modül 7/03 fork örneği (POSIX, sadece Linux/macOS):
g++ -std=c++17 -Wall -Wextra -Wpedantic -pthread \
    module_07_under_the_hood/03_fork_vs_thread_pool.cpp -o fork_demo
```

Not: Tüm dosyalar yukarıdaki sıkı flag'lerle **0 warning, 0 error** olarak derlenir.

Referanslar

- cppreference.com
- C++ Core Guidelines
- ISO C++
- Google Test